

System & Data Design Masterclass

The definitive guide to designing data systems
that scale

13

MODULES

60+

LESSONS

13

PROJECTS

5

CASE STUDIES

Course Modules

00 Foundations & Methodology

Welcome & Foundations · Interview Framework · Building Blocks · Slack Case Study

01 Data Modeling & Schema Design

Normalize vs Denormalize · Dimensional Modeling · Event-Driven Schema · Graph & Document Models

02 Storage Systems Deep Dive

OLTP vs OLAP · Columnar Storage & Table Formats · Object Storage Patterns

03 Batch Processing & ETL

Batch Processing · ETL vs ELT · Workflow Orchestration · Data Quality · Cost Optimization

04 Stream Processing & Real-Time Systems

Stream Fundamentals · Message Queues · Processing Frameworks · Real-Time Analytics · Event-Driven Architecture

05 Data Warehouse Architecture

Warehouse Architecture · Partitioning & Clustering · Materialized Views · Query Optimization · Multi-Tenant Analytics

06 Data Lake & Lakehouse Architecture

Data Lake Architecture · Lakehouse Strategies · Governance · Performance · Multi-Format Data

07 ML Platform & Feature Store Design

ML Pipelines · Feature Stores · Model Serving · ML Monitoring · Vector Databases

08 Event-Driven & Distributed Systems

Microservices Data · Event Sourcing & CQRS · CDC · API Design · Data Mesh

09 Monitoring, Observability & Data Quality

SLA Design · Data Quality Framework · Observability · Cost Monitoring · Security & Compliance

10 Cost Optimization & Capacity Planning

Cloud Cost Models · Compute Optimization · Storage Optimization · Query Optimization · Capacity Planning

11 Case Studies — Real-World Systems

Netflix · Uber Surge Pricing · Spotify Recommendations · Airbnb Search · Twitter Real-Time

12 Interview Preparation & Portfolio Projects

Interview Deep Dive · Common Questions · Portfolio Projects · Advanced Topics · Career Development

00

Foundations & Methodology

How to think about system design as a data engineer.
Master the interview framework that works for any
data system design question.

System & Data Design Masterclass

Module 0: Foundations & Methodology

What you'll learn

How to think about system design as a data engineer. You'll master the interview framework that works for any data system design question and understand what makes data systems fundamentally different from web applications.

0.1 Welcome & What Makes Data System Design Different

✂ TL;DR

Data system design is harder than web app design due to the 4 Vs: Volume, Variety, Velocity, Veracity - You're optimizing for read performance, not write performance - Failures are more expensive (lost data vs slow page load) - You deal with evolving schemas, late-arriving data, and complex dependencies

Most software engineers think they can design data systems. They've built web apps, they understand databases, they know about caching and load balancers. How hard could it be? Then they join a data team and realize their mental models are all wrong.

The Difference

Web applications are designed around the request-response cycle. A user clicks a button, you fetch some data, maybe write to a database, return a response. The focus is on low latency and high throughput for simple operations.

□ In Plain English

A web app is like a waiter — you ask for something (a request), the waiter brings it back (a response). A data system is more like a food factory — raw ingredients arrive on conveyor belts, get processed through multiple stations, and finished products come out the other end.

Data systems are designed around batch and stream processing. Data flows through pipelines, gets transformed, lands in warehouses, powers analytics. The focus is on correctness, scalability, and cost-effectiveness for complex operations on large volumes of data.

📖 In Plain English

Think of **batch processing** like doing laundry — you collect dirty clothes all week, then wash them all at once on Sunday. **Stream processing** is like a dishwasher that runs every time you put a plate in. Both get the job done, but they're suited for different situations.

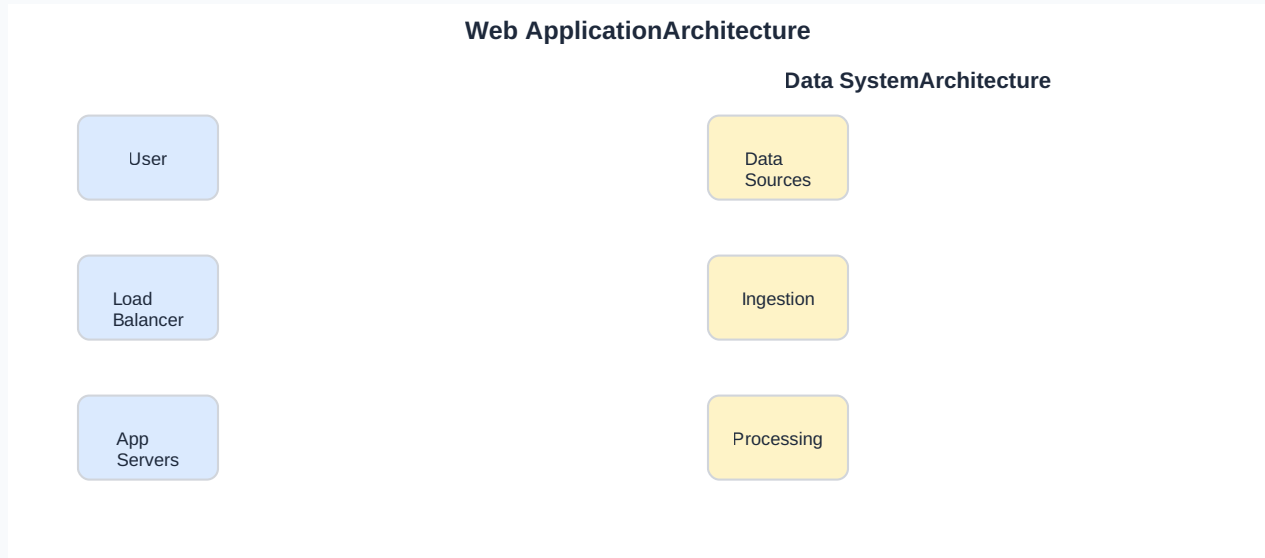


Figure 1: Two side-by-side architecture diagrams. Left side shows “Web Application Architecture”: User → Load Balancer → App Serve...

The Four Vs That Change Everything

1. Volume — Scale changes the rules

Web app database: 100GB, maybe 1TB Data warehouse: 10TB, 100TB, 1PB+

📖 In Plain English

A **data warehouse** is like a well-organized library — data is cleaned, structured, and cataloged before it goes in. It's optimized for answering business questions fast, but requires more upfront work to set up.

Web app queries: milliseconds, single records

Data warehouse queries: seconds/minutes, millions of records

When you're processing terabytes instead of gigabytes, techniques

that work perfectly fine for web apps become impossible. You can't `SELECT * FROM orders` when there are a billion orders. You can't `JOIN` without thinking about data skew. Normal databases can't handle it.

2. Variety — Data comes from everywhere, in every format

Web apps deal with structured data they control. Data engineers deal with:

- JSON from APIs that change schema without warning
- CSV files with inconsistent column orders
- Real-time event streams
- Third-party data feeds with different formats
- Legacy systems with ancient schemas

Key Concept

Concept In web apps, you control your schema. In data engineering, the schema controls you. Your job is to make sense of data you didn't design, that changes without notice, from systems you don't control.

3. Velocity — From real-time to archival, all at once

Web apps are mostly real-time. Data systems handle everything:

- Real-time: fraud detection (sub-second)
- Near real-time: dashboards (minutes)
- Batch: daily reports (hours)
- Archival: compliance data (years)

You need to design for multiple time horizons simultaneously.

4. Veracity — Data is dirty, and it's your problem

Web apps have validation. Users can't submit invalid forms. APIs reject bad data.

Data systems get whatever they get:

- 50% of customer records missing email addresses
- Product IDs that exist in the orders table but not the products table
- Timestamps in three different formats in the same file
- NULL vs empty string vs "NULL" vs "null" vs missing fields

Real-World Example: E-commerce Analytics

Let's compare how a web engineer vs data engineer would approach "show me today's revenue."

Web Engineer Approach:

```
SELECT SUM(total) FROM orders WHERE DATE(created_at) = CURRENT_DATE;
```

SQL

Data Engineer Questions:

- Which orders table? (There are probably three)
- Does total include tax? Shipping? Discounts?
- What timezone is created_at in?
- Are canceled orders included?
- What about refunds? Partial refunds?
- Are marketplace fees deducted?
- What about orders placed today but processed tomorrow?

The data engineer's version looks like this:

```
SELECT
    SUM(
        gross_order_value
        - COALESCE(discount_amount, 0)
        - COALESCE(refund_amount, 0)
        + COALESCE(tax_amount, 0)
        + COALESCE(shipping_amount, 0)
    ) as net_revenue
FROM fact_orders f
JOIN dim_date d ON f.date_key = d.date_key
WHERE d.calendar_date = CURRENT_DATE
    AND f.order_status NOT IN ('canceled', 'fraud')
    AND f.is_test_order = FALSE
    AND f.payment_processed_at IS NOT NULL;
```

That's the difference. Data engineering is about handling the real world's messiness at scale.

⚠ Common Mistake

⚠ Common Mistake New data engineers try to apply web app patterns to data problems. They build request-response APIs for terabyte datasets, or try to keep everything in real-time when batch processing would be 100x cheaper. Know when to break your old mental models.

□ Checkpoint

1. What are the 4 Vs and why does each one complicate data system design?
2. Why is schema evolution a bigger problem for data systems than web applications?
3. Give an example of a seemingly simple query that reveals data engineering complexity.

0.2 The Data System Design Interview Framework

✂ TL;DR

Follow the 7-step methodology: Clarify → Estimate → Design → Deep Dive → Scale → Monitor → Trade-offs - Spend 30% of your time on requirements, 70% on design - Always start with data flow, not compute architecture - Interview tip: State your assumptions out loud

Experienced interviewers report that the difference between strong candidates and others isn't technical knowledge — it's methodology. Strong candidates follow a systematic approach.

The 7-Step Data System Design Framework

Step 1: Clarify Requirements (5-10 minutes)

Functional requirements: What does the system need to do?

- Non-functional requirements: Scale, latency, consistency
- Business context: Is this internal analytics or customer-facing?

Step 2: Estimate Scale (5 minutes)

Data volume: How much data per day/hour/second?

- Query patterns: Read-heavy? Write-heavy? Mixed?
- User count: Internal team of 10 or external users in millions?

Step 3: High-Level Design (10 minutes) - Data flow diagram:

- Source → Processing → Storage → Consumption
- Major components: databases, queues, processing engines
- API boundaries between components

Step 4: Deep Dive (15-20 minutes) - Pick 1-2 components to detail -

Data models, schemas, processing logic - Technology choices and why

Step 5: Scale (5-10 minutes)

- Bottlenecks and how to address them
- Scaling strategies: horizontal, vertical, functional
- Performance optimizations

Step 6: Monitor (3-5 minutes) - Success metrics and SLAs -

Alerting and observability - Data quality monitoring

Step 7

- Trade-offs & Alternatives (Remaining time)
- What you optimized for and what you gave up
- Alternative architectures you considered
- When you'd choose differently

Step 1 Deep Dive: Clarifying Requirements

Most candidates rush this. They hear "design a data warehouse" and immediately start drawing boxes. Wrong.

- Functional Requirements
- Ask These Questions:
 - What types of queries will users run?
 - How fresh does the data need to be?
 - What data sources do we need to integrate?
 - Who are the users? (Analysts, executives, ML models, customer-facing apps?)

Non-Functional Requirements - The Numbers That Matter: -

- Volume: Data size now and in 2 years
- Velocity: Batch (daily/hourly) or real-time?
- Availability: Can we have downtime for maintenance?
- Consistency: Can data be eventually consistent or strictly consistent?

- Cost: Is this a cost-sensitive use case?

Example Clarification Dialogue:

Interviewer: "Design a data warehouse for our e-commerce company."

Bad Response: "Okay, so we'll use Snowflake and ingest data from..."

Good Response: "I'd like to understand the requirements first. What types of analyses will business users run on this warehouse? Are they looking for daily sales reports, real-time dashboards, or complex analytics like customer lifetime value modeling?"

Interviewer: "Mostly daily and weekly business reports, but the executive team wants to see metrics updated hourly."

Good Follow-up: "Got it. What's our current data volume and expected growth? How many data sources are we integrating? And what's our target query response time for these reports?"

The key is to ask questions that reveal the complexity. Don't just accept "design a data warehouse" — that could mean anything from a 100GB Postgres database to a multi-petabyte distributed system.

Step 2: Estimation - Getting the Numbers Right

Your scale estimates drive all your technology choices. Get them wrong and your entire design is wrong.

Data Volume Estimation Framework:

Records per day = Users × Events per user × Days
Data size = Records × Average record size × Compression factor

Example: E-commerce order analytics

- - Active users per day: 100k
- Orders per user per day: 0.1 (1 order per 10 users)
- Average order has 2 line items
- Records per day: $100k \times 0.1 \times 2 = 20k$ orders + line items
- Record size: ~500 bytes average

- Daily data: $20k \times 500 \text{ bytes} = 10\text{MB/day}$
- With growth (5x in 2 years): 50MB/day
- With history (3 years): $\sim 50\text{GB total}$

Conclusion: This easily fits in Postgres, no need for Snowflake.

Query Volume Estimation:

Active dashboard users: 50 Queries per user per hour: 10 Peak hour multiplier: 3x Peak queries per second: $50 \times 10 \times 3 / 3600 = 0.42 \text{ QPS}$

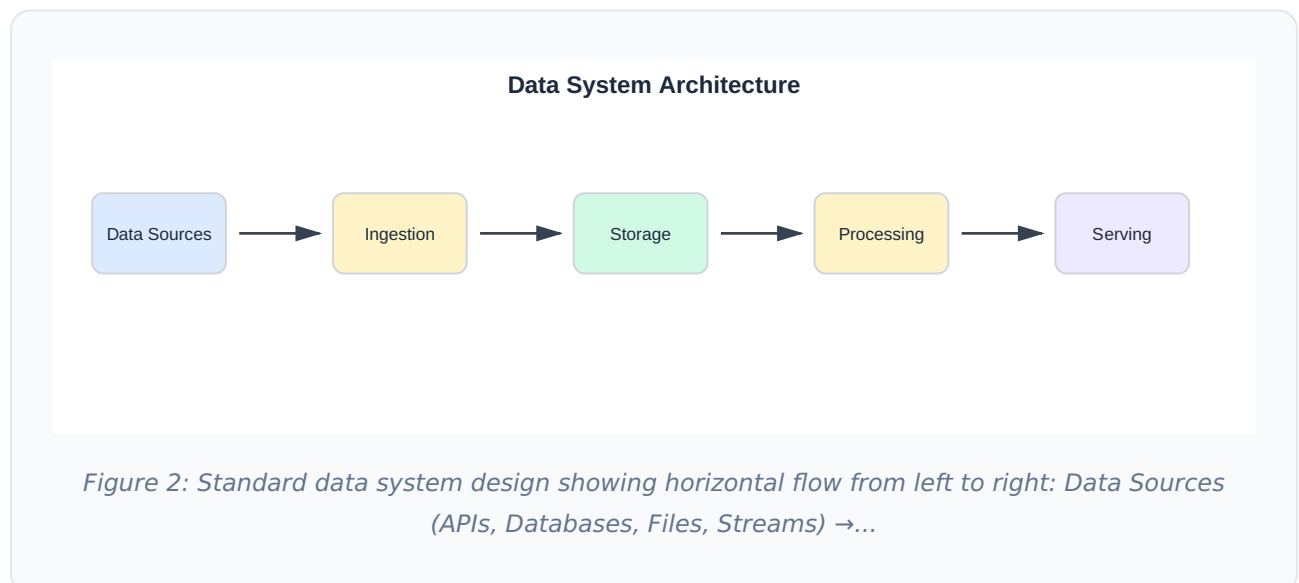
Conclusion: Query volume is not a concern, optimize for data freshness.

Pro Tip

State your assumptions out loud. "I'm assuming each user generates about 10 events per session, and we have 100k active users daily. If those numbers are off, it would change my design significantly."

Step 3

- High-Level Design - Start with Data Flow



Always start with data flow, not compute architecture. Web engineers start with load balancers and servers. Data engineers start with: Where does data come from? Where does it go? How does it get transformed along the way?

The Four-Layer Pattern: 1. Ingestion: How data enters the system 2. Storage: Where data lives at rest 3. Processing: How data gets transformed 4. Serving: How data gets consumed

This pattern works for 90% of data system designs. Start here, then add complexity only where needed.

Common Interview Mistakes

- ☐ Jumping to Technology Choices Too Early "We'll use Kafka for streaming and Spark for processing..." Why not Apache Pulsar? Why not Flink? Justify your choices.

☐ In Plain English

Apache Kafka is like a high-speed postal service for data. Producers send messages to topics (mailboxes), and consumers read from those topics. Messages are kept for a configurable time, so multiple consumers can read the same data independently.

- ☐ Ignoring Data Quality Designing a pipeline that assumes data. Real systems need validation, error handling, and monitoring at every step.
- ☐ Overengineering for Scale Building a distributed system for 10GB of data. Sometimes Postgres + cron is the right answer.
- ☐ Underengineering for Growth "We can always migrate to a bigger database later." Migration costs are enormous. Plan for 2-3 years of growth.

- ☐ Checkpoint

1. What are the 7 steps of the data system design framework?
2. Why is it important to clarify requirements before jumping into solutions?

3. What's the difference between starting with "data flow" vs "compute architecture"?

< TL;DR

Storage: OLTP (app data), OLAP (analytics), Object Store (files), Search (text queries) - Processing: Batch (high latency, high throughput), Stream (low latency, complex), Micro-batch (compromise) - Serving: APIs (programmatic), Dashboards (humans), Files (other systems) - Pick your battles: You can optimize for 2 out of 3: Fast, Cheap, Consistent

Before you can design systems, you need to know your building blocks. These are the patterns that every data system uses. Master these, and you can combine them to solve any problem.

Storage Layer Patterns

- OLTP (Online Transaction Processing)
- What: Systems optimized for small, fast transactions
- Examples: PostgreSQL, MySQL, MongoDB
- Pattern: Many small reads and writes, ACID transactions
- Use for: Application databases, transactional workloads
- Anti-pattern: Large analytical queries (kills performance)

□ In Plain English

OLTP (Online Transaction Processing) is your everyday database — it handles things like "add item to cart" or "update user email" quickly, one at a time. Think of it as a bank teller handling individual transactions.

```
-- Typical OLTP query - fast and targeted
UPDATE orders
SET status = 'shipped', shipped_at = NOW()
WHERE order_id = 123 AND status = 'pending';
```

SQL

- OLAP (Online Analytical Processing)

- What: Systems optimized for complex queries on large datasets
- Examples: Snowflake, BigQuery, ClickHouse, Redshift
- Pattern: Few complex queries scanning millions of rows
- Use for: Data warehouses, business intelligence, analytics
- Anti-pattern: High-frequency transactional updates

□ In Plain English

OLAP (Online Analytical Processing) is built for big questions like "what were our total sales by region last quarter?" It scans millions of rows efficiently. Think of it as a financial analyst crunching spreadsheets.

```
-- Typical OLAP query - complex aggregation across millions of rows
SELECT
    DATE_TRUNC('month', order_date) as month,
    product_category,
```

SQL

The corresponding SQL query:

```
SUM(revenue) as total_revenue,
    AVG(revenue) as avg_order_value,
    COUNT(DISTINCT customer_id) as unique_customers
FROM fact_orders
JOIN dim_product USING (product_key)
WHERE order_date >= '2023-01-01'
GROUP BY 1, 2
ORDER BY 1, 3 DESC;
```

SQL

- Object Store
- What: Highly scalable storage for files and unstructured data
- Examples: S3, Google Cloud Storage, Azure Blob
- Pattern: Write-once-read-many, unlimited scale
- Use for: Data lakes, backups, file storage, cheap archival
- Anti-pattern: Transactional updates, small file operations

In Plain English

A **data lake** is like a massive filing cabinet where you dump everything — spreadsheets, logs, images, JSON files — in their original format. You organize and process them later. Cheap to store, flexible to use.

- Search & Document Stores
- What: Systems optimized for text search and semi-structured data
- Examples: Elasticsearch, Solr, MongoDB
- Pattern: Full-text search, flexible schema, real-time indexing
- Use for: Search engines, logs analysis, flexible data models

Processing Layer Patterns

- Batch Processing
- Characteristics: High latency (minutes to hours), high throughput, fault tolerant
- Examples: Apache Spark, Hadoop MapReduce, dbt
- Use for: ETL pipelines, data warehousing, machine learning training
- Pattern: Process large volumes of data in scheduled jobs

```
# Batch processing example - process 1TB of data overnight
daily_orders = spark.read.parquet("s3://data-lake/orders/2024-03-
```

PYTHON

15/")

```
customer_summary = daily_orders.groupBy("customer_id").agg(
    sum("order_amount").alias("daily_spend"),
    count("*").alias("order_count")
)
customer_summary.write.mode("overwrite").parquet("s3://warehouse/customer_daily/2024-
```

PYTHON

03-15/")

Stream Processing

Characteristics: Low latency (sub-second to seconds), continuous processing

Examples: Kafka Streams, Apache Flink, Apache Pulsar
Use for: Real-time alerts, live dashboards, fraud detection
Pattern: Continuous processing of data as it arrives

PYTHON

```
# Stream processing example - real-time fraud detection
orders_stream.filter(
    lambda order: order['amount'] > 10000 or order['velocity_score']
    > 0.8
```

- Stream Processing
- Characteristics: Low latency (sub-second to seconds), continuous processing
- Examples: Kafka Streams, Apache Flink, Apache Pulsar
- Use for: Real-time alerts, live dashboards, fraud detection
- Pattern: Continuous processing of data as it arrives

```
# Stream processing example - real-time fraud detection
orders_stream.filter(
    lambda order: order['amount'] > 10000 or order['velocity_score']
```

PYTHON

> 0.8

```
).map(
    lambda order: send_fraud_alert(order)
)
```

PYTHON

- Micro-batch Processing
- Characteristics: Near real-time (seconds to minutes), compromise solution
- Examples: Spark Structured Streaming, Kafka with small batches
- Use for: Near real-time analytics, when true streaming is too complex
- Pattern: Small frequent batches that feel like streaming

Serving Layer Patterns

- APIs (Programmatic Access)
 - Use for: Other applications, microservices, real-time lookups
 - Pattern: Request-response, structured queries, authentication
 - Examples: REST APIs, GraphQL, gRPC
- Dashboards (Human Access)
 - Use for: Business users, analysts, executives
 - Pattern: Visual queries, interactive exploration, scheduled reports
 - Examples: Tableau, Looker, Metabase, Power BI
- Files/Data Products (System Access)
 - Use for: Other data systems, data sharing, bulk export
 - Pattern: Periodic dumps, standardized formats, self-describing
 - Examples: Parquet files, CSV exports, Data APIs

The Trade-off Triangle



Figure 3: Triangle with three points labeled "Fast" (top), "Cheap" (bottom-left), and "Consistent" (bottom-right). Inside the triangle...

You cannot have all three. Choose wisely:

Fast + Consistent = Expensive - In-memory databases, high-end

OLAP systems - Example: Redis for real-time recommendations

- Fast + Cheap = Eventually Consistent
- Caches, read replicas, best-effort processing

- Example: CDN for dashboards (might be 5 minutes stale)
- Cheap + Consistent = Slow
- Batch processing, disk-based systems
- Example: Daily ETL pipeline for financial reporting

Putting It Together: Common Architecture Patterns

Lambda Architecture (Batch + Speed Layer)

Real-time data → Speed layer (stream processing) → Real-time views

↘

Data lake ← Batch layer (batch processing) → Batch views

↗

Historical data ← Serving layer (queries) ← Combined views

Kappa Architecture (Streaming-first)

All data → Stream processing → Materialized views

📖 In Plain English

A **materialized view** is a pre-computed query result stored as a table. Instead of running an expensive query every time, you compute it once and refresh periodically. It trades storage for speed.

↘

Long-term storage ↗

Modern Data Stack (ELT Pattern)

Sources → Ingestion tools → Data warehouse → Transformation (dbt) → BI tools

(Looker)

Concept: Start Simple, Add Complexity Most problems can be solved with: PostgreSQL + Python + cron. Add complexity only when you hit real constraints. "We might have big data someday" is not a constraint.

- **Checkpoint**

1. What's the difference between OLTP and OLAP systems? Give examples of when you'd use each.
2. Explain the trade-off triangle. Why can't you have Fast + Cheap + Consistent?
3. What are the three main processing patterns and when would you use each?

Try It Yourself

You're designing a system that needs to:

- Process 1GB of order data daily
- Provide real-time fraud alerts (sub-second)
- Generate weekly business reports
- Support ad-hoc analytics queries

Map each requirement to storage and processing patterns. What trade-offs would you make?

0.4 Case Study Walkthrough: Design Slack's Analytics Platform

⚡ TL;DR

Use the 7-step framework on a real problem to see how it works - Requirements drive technology choices, not the other way around - Start with data flow, add components incrementally - Always consider both technical and business constraints

Let's walk through a complete system design using our framework. This is exactly how you'd approach it in an interview or on the job.

The Problem Statement

"Design an analytics platform for Slack that can answer questions like: How many messages are sent per day? Which channels are most active? How has usage grown over time? The system needs to support both internal analytics teams and external customer-facing analytics."

Step 1

Workspace-level metrics for enterprise customers - Real-time dashboards: Live metrics for ops teams - Ad-hoc queries: Data science and product teams exploring data

Non-Functional Requirements:

- Scale: 10 million daily active users, 100M+ messages per day
- Latency: Real-time dashboards (< 1 minute), batch reports (hourly)
- Availability: 99.9% uptime for customer-facing features
- Data retention: 7 years for compliance
- Security: Workspace data isolation, GDPR compliance

Questions to Ask

- What's more important: query speed or data freshness?
 - Do customers need real-time data or is daily/hourly sufficient?
 - What's the budget for this system?
 - Are there regulatory requirements for data handling?

Step 2: Estimate Scale (5 minutes)

Data Volume Estimation:

Messages per day: 100M Average message size: 200 bytes (text + metadata) Daily data volume: $100M \times 200 \text{ bytes} = 20GB/day$ Annual data volume: $20GB \times 365 = 7.3TB/year$ With 7-year retention: 50TB total

Events beyond messages:

- User sessions: $10M \text{ users} \times 5 \text{ sessions/day} = 50M \text{ events}$
- Channel activities: $1M \text{ channels} \times 10 \text{ events/day} = 10M \text{ events}$
- File uploads, reactions, etc.: $\sim 20M \text{ events}$ Total events per day: $\sim 180M \text{ events} = \sim 36GB/day$

Storage needs: $\sim 100TB$ with 7-year retention

Query Volume Estimation:

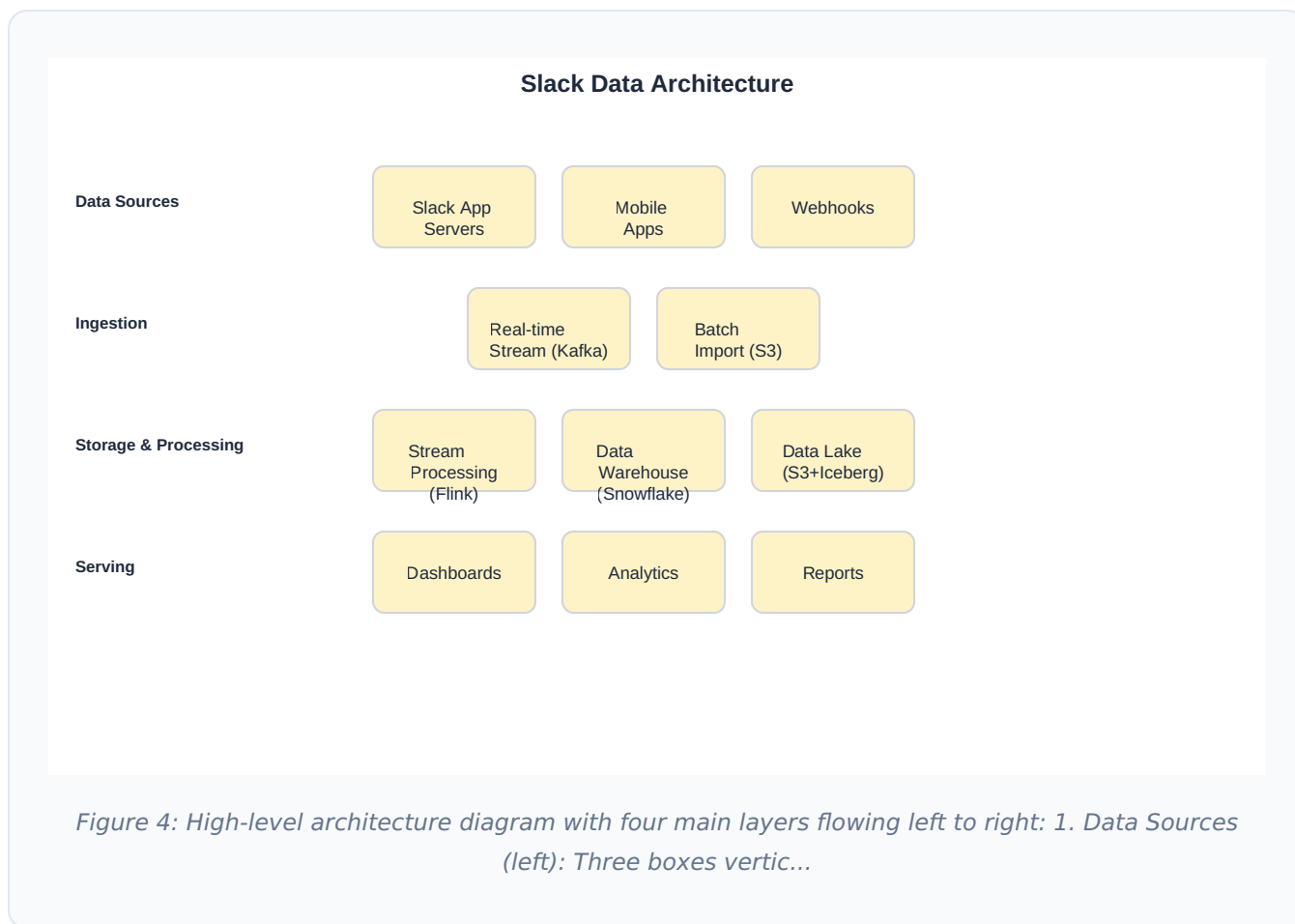
Internal users: 200 analysts/data scientists Customer dashboards: 50k enterprise workspaces Peak query load: $\sim 1000 \text{ QPS}$ (mostly customer dashboards)

Growth Projections:

Current: 100M messages/day 2-year projection: 300M messages/day (3x growth) Design for: 500M messages/day (5x, safety margin)

Step 3

High-Level Design (10 minutes)



Data Flow: 1. Sources: Slack application servers, mobile apps, webhooks 2. Ingestion: Kafka for real-time events, S3 for bulk exports 3. Processing: Flink for stream processing, Spark for batch ETL 4. Storage: Iceberg data lake for raw data, Snowflake for analytics 5. Serving: APIs for real-time, BI tools for dashboards

Key Design Decisions:

- Hybrid batch + stream: Real-time for ops, batch for analytics
- Data lake + warehouse: Lake for flexibility, warehouse for performance
- Multi-tenant: Workspace isolation for customer analytics

Step 4: Deep Dive - Data Model & Processing (20 minutes)

Event Schema Design:

```
{
  "event_id": "uuid",
  "timestamp": "2024-03-15T14:30:00Z",
  "event_type": "message_sent",
  "workspace_id": "W123456789",
  "user_id": "U987654321",
  "channel_id": "C555666777",
  "message": {
    "id": "msg_123",
    "text_length": 145,
    "has_attachments": true,
    "thread_id": null,
    "edited_at": null
  },
  "client": {
    "type": "desktop",
    "version": "4.29.149"
  },
  "metadata": {
    "ip_address": "192.168.1.1",
    "user_agent": "Slack/4.29.149"
  }
}
```

JSON

Stream Processing Pipeline:

```
# Flink job for real-time aggregations
events_stream = kafka_source("slack-events")

# Real-time metrics (1-minute windows)
message_counts = events_stream \
    .filter(lambda e: e['event_type'] == 'message_sent') \
    .key_by(lambda e: e['workspace_id']) \
    .window(TumblingProcessingTimeWindows.of(Time.minutes(1))) \
    .aggregate(CountAggregateFunction())

# Write to Redis for real-time dashboards
message_counts.add_sink(redis_sink())
```

PYTHON

Data Warehouse Schema (Dimensional Model):

SQL

```
-- Fact table: one row per message
CREATE TABLE fact_messages (
    message_key BIGINT PRIMARY KEY,
    workspace_key INTEGER,
    user_key INTEGER,
    channel_key INTEGER,
    date_key INTEGER,
    hour_of_day INTEGER,
    message_length INTEGER,
    has_attachments BOOLEAN,
    is_thread_reply BOOLEAN,
    client_type VARCHAR(20),
    created_at TIMESTAMP
);

-- Dimension: workspaces
CREATE TABLE dim_workspace (
    workspace_key INTEGER PRIMARY KEY,
    workspace_id VARCHAR(20),
    plan_type VARCHAR(20), -- free, pro, enterprise
    created_date DATE,
    is_current BOOLEAN
);

-- Pre-aggregated for performance
CREATE TABLE agg_daily_workspace_metrics (
    workspace_key INTEGER,
    date_key INTEGER,
    message_count INTEGER,
    active_users INTEGER,
    active_channels INTEGER,
    PRIMARY KEY (workspace_key, date_key)
);
```

ETL Pipeline Design:

```
# Daily batch ETL (Spark)
def process_daily_events(date):
    # Read from data lake
    raw_events = spark.read.parquet(f"s3://slack-events/{date}/")

    # Clean and validate
    clean_events = raw_events.filter(
        col("workspace_id").isNotNull() &
        col("timestamp").between(f"{date} 00:00:00", f"{date}
```

23:59:59")

```
)

# Join with dimensions to get surrogate keys
fact_data = clean_events.join(dim_workspace, "workspace_id") \
    .join(dim_user, "user_id") \
    .select("workspace_key", "user_key", ...)

# Write to warehouse
fact_data.write.mode("append").table("fact_messages")

# Generate aggregations
daily_metrics = fact_data.groupBy("workspace_key", "date_key") \
    .agg(count("*"),
```

```
countDistinct("user_key"))
```

```
daily_metrics.write.mode("overwrite") \
    .option("partition", f"date_key={date}") \
    .table("agg_daily_workspace_metrics")
```

Step 5: Scale (10 minutes)

Current Bottlenecks: 1. Kafka throughput: 180M events/day = 2k events/sec (easily handled) 2. Stream processing: Simple aggregations, Flink can handle 10x this volume 3. Warehouse queries: Customer dashboards could create hot spots

Scaling Strategies:

For 5x Growth (500M messages/day):

- Kafka: Add partitions (scale to 10k events/sec)
- Flink: Scale out to more task managers
- Snowflake: Use larger warehouses for peak query times
- Data lake: S3 scales infinitely

For 100x Growth (10B messages/day): - Real-time path: Consider

Apache Pulsar for geo-replication - Warehouse: Shard by workspace_id across multiple Snowflake accounts - Caching: Add

Redis cluster for frequently accessed metrics

Cost Optimization:

Current cost estimate (100M messages/day):

- Kafka: \$2k/month
- Snowflake: \$15k/month
- S3 storage: \$1k/month
- Processing: \$3k/month Total: ~\$21k/month

With 5x growth: ~\$80k/month (economies of scale)

Step 6: Monitor (5 minutes)

SLA Definitions:

- Real-time dashboards: < 1 minute lag, 99.9% availability
- Customer analytics: < 5 minute lag, 99.95% availability
- Batch reports: Complete within 2 hours, 99% reliability

Metrics:

```
# Data pipeline health
- Events per second (expected vs actual)
- Processing lag (Kafka consumer lag)
- Data quality score (% of valid events)
- Query response times (p95, p99)

# Business metrics
- Daily active workspaces
- Messages per workspace
- Feature adoption rates
```

Alerting Strategy:

- P0: Customer dashboards down, data pipeline stopped
- P1: Processing lag > 10 minutes, high error rates
- P2: Slow queries, cost anomalies

Step 7: Trade-offs & Alternatives (5 minutes)

Trade-offs Made:

- Complexity for flexibility: Hybrid batch/stream adds operational overhead but supports both real-time and batch use cases
- Cost for performance: Snowflake is expensive but provides query performance for customer analytics
- Storage duplication: Data lake + warehouse stores data twice but enables different access patterns

Alternative Architectures Considered:

Pure Streaming (Kappa):

- Pros: Simpler architecture, true real-time
- Cons: Complex stream processing for analytical queries, harder to replay historical data

Pure Batch (Lambda without speed layer):

- Pros: Simpler processing, cheaper
- Cons: No real-time dashboards, slower customer insights

Single Store (just Snowflake):

- Pros: One system to manage, simpler queries
- Cons: Higher costs, less flexibility for ML/advanced analytics

📌 Pro Tip

In interviews, always end with trade-offs. It shows you understand that every design decision has consequences. There are no architectures, only architectures that fit the requirements.

📌 Checkpoint

1. Walk through the 7 steps for the Slack analytics platform. What did we optimize for?
2. Why did we choose a hybrid batch + stream architecture instead of pure streaming?
3. What would you change if the requirement was “100x current scale” vs “real-time customer alerts”?

Try It Yourself

Apply the 7-step framework to this problem:

"Design an analytics platform for Netflix that tracks viewing patterns and powers their recommendation system. The system needs to handle 200M viewing events per day and provide real-time personalization."

Spend 20 minutes working through each step. Focus on requirements clarification and high-level design.

Module 0 Project: Master the Interview Framework

Objective

Practice the 7-step data system design framework on three progressively complex problems. Build confidence in requirements gathering, estimation, and systematic problem solving.

Prerequisites Understanding of basic data concepts (databases, APIs, batch vs streaming)

Familiarity with cloud storage and computing concepts

No specific tools required - this is design, not implementation

Expected Time

3-4 hours total

Problem Set

Problem 1: Beginner (45 minutes) Design a data pipeline for a startup's customer analytics dashboard. They have 10k users, want to track user behavior, and show daily/weekly metrics to the product team.

Problem 2: Intermediate (90 minutes) Design a real-time fraud detection system for a payment processor. They handle 1M transactions per day, need to flag suspicious transactions within 100ms, and minimize false positives.

Problem 3: Advanced (120 minutes) Design a data platform for a food delivery company that powers driver dispatch, demand forecasting, and business analytics. The system handles 1M orders per day across 100 cities.

Deliverables

For each problem, create a document following this template:

```
# Problem X: [Title]

## Step 1: Requirements Clarification
### Functional Requirements
- [Requirement 1]
- [Requirement 2]

### Non-Functional Requirements
- Scale: [numbers]
- Latency: [requirements]
- Availability: [requirements]

### Questions I Would Ask
- [Question 1]
- [Question 2]

## Step 2: Scale Estimation
### Data Volume
[calculations]

### Query Volume
[calculations]

### Growth Projections
[assumptions and projections]

## Step 3: High-Level Design
[ASCII diagram or description of major components]

### Data Flow
1. [Step 1]
2. [Step 2]

## Step 4: Deep Dive
### Component 1: [Name]
[Detailed design]

### Component 2: [Name]
[Detailed design]

## Step 5: Scaling
### Current Bottlenecks
- [Bottleneck 1]

### Scaling Strategies
- [Strategy 1]

## Step 6: Monitoring
```

```
### SLAs
- [SLA 1]

### Key Metrics
- [Metric 1]

## Step 7: Trade-offs
### Design Decisions
- [Decision 1]: [Trade-off]

### Alternatives Considered
- [Alternative 1]: [Why not chosen]
```

Evaluation Criteria

- Requirements Gathering (20 points)
 - ☐ Asked clarifying questions about functional requirements
 - ☐ Identified non-functional requirements (scale, latency, consistency)
 - ☐ Understood business context and user needs
- Scale Estimation (15 points)
 - ☐ Calculated data volume with reasonable assumptions
 - ☐ Estimated query patterns and load
 - ☐

Projected growth and capacity needs

- System Design (35 points)
 - ☐ Clear high-level architecture with data flow
 - ☐ Appropriate technology choices for requirements
 - ☐ Detailed design of 1-2 key components
 - ☐ Considered both batch and real-time processing where needed
- Scaling & Operations (20 points)
 - ☐ Identified realistic bottlenecks
 - ☐ Proposed concrete scaling strategies

- ☐ Defined monitoring and SLAs
- Trade-offs & Communication (10 points)
- ☐ Acknowledged design trade-offs
- ☐ Considered alternative approaches
- ☐ Clear, logical presentation

Starter Questions for Each Problem

- Problem 1 (Startup Analytics)
 - How many events per user per day?
 - What types of events (page views, clicks, purchases)?
 - How many analysts will use the system?
 - Is real-time data required or daily batch sufficient?
- Problem 2 (Fraud Detection)
 - What's the current fraud rate?
 - What data sources are available (transaction history, user profile, device info)?
 - What's the cost of false positives vs false negatives?
 - Can we block transactions in real-time or just flag them?
- Problem 3 (Food Delivery)
 - How many drivers, restaurants, customers in the system?
 - What's the geographic distribution (dense cities vs sprawling suburbs)?
 - How time-sensitive is driver dispatch (seconds? minutes?)?
 - What business analytics are most critical for operations?

Solution Approach Tips

1. Start simple: Don't over-engineer for problems that don't need it 2. State assumptions: "Assuming 10 events per user per day..." 3. Show your work: Walk through calculations step by step 4. Consider alternatives: "We could also use X, but Y is better because..." 5. Think operationally: How would you monitor this? What could go wrong?

After completing all three problems, reflect on:

- Which step was hardest for you?
- Where did you make technology choices too early?
- What questions would you ask differently next time?

This framework will serve you throughout your career - in interviews, architecture reviews, and designing production systems. # Module 1:

Data Modeling & Schema Design

What you'll learn: How to design schemas that scale, perform, and evolve. You'll master when to normalize vs denormalize, advanced dimensional modeling techniques, and modern patterns like event sourcing and schema evolution.

In Plain English

Event sourcing means storing every change as an event, rather than just the current state. Instead of updating your bank balance directly, you record every deposit and withdrawal. You can always replay events to reconstruct any past state.

In Plain English

Normalization means storing each piece of data exactly once. Like having a separate contacts list instead of writing someone's phone number in every email. It saves space and keeps things consistent, but you have to look things up more often.

01

Data Modeling & Schema Design

How to design schemas that scale, perform, and evolve. Master normalization, dimensional modeling, event sourcing, and modern data patterns.

1.1 When to Normalize vs Denormalize

✂ TL;DR

Normalize for OLTP systems (data integrity, storage efficiency) - Denormalize for OLAP systems (query performance, simplicity) - The spectrum runs: 3NF → Star Schema → OBT → Event Logs - Modern cloud warehouses change the normalization calculus - storage is cheap, compute is expensive

Every data engineer faces this question: "Should I normalize this data or not?" The answer isn't black and white. It depends on your workload, your platform, and your team's SQL skills. Get it wrong, and you'll either have impossible-to-maintain spaghetti schemas or queries that take 30 minutes to run.

The Traditional Rules (And When They Don't Apply)

Database normalization rules were created in the 1970s for a different world:

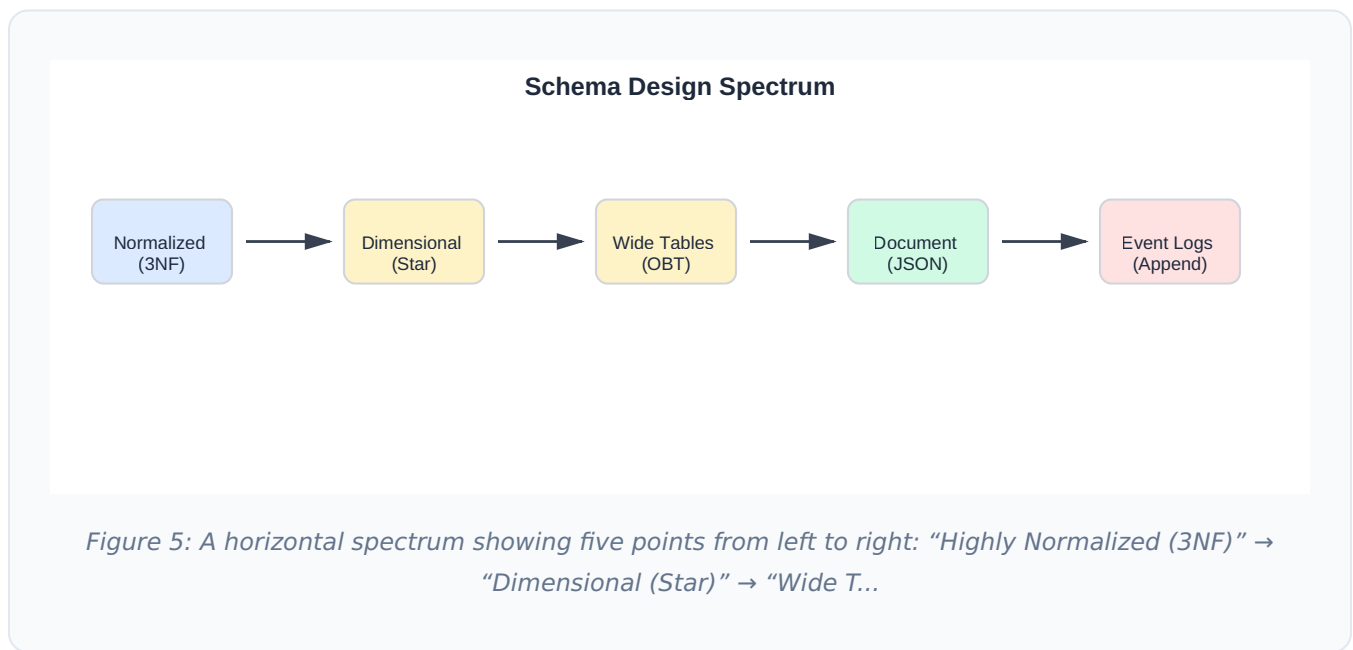
- Storage was expensive (\$10,000/GB in 1990)
- RAM was tiny (4MB was high-end)
- CPU was single-threaded
- Networks were slow

In 2026, the constraints are different:

- Storage is cheap (\$0.023/GB in S3)
- RAM is abundant (instances with 1TB+ RAM)
- CPU is massively parallel
- Networks are fast (10+ Gbps)

This means the old trade-offs don't always apply.

The Normalization Spectrum



1. Highly Normalized (3NF)

```
-- Traditional normalized approach
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);

CREATE TABLE addresses (
    address_id INT PRIMARY KEY,
    customer_id INT REFERENCES customers(customer_id),
    street VARCHAR(200),
    city VARCHAR(50),
    state VARCHAR(20)
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT REFERENCES customers(customer_id),
    order_date DATE,
    total_amount DECIMAL(10,2)
);

-- Simple insert, complex analytics
SELECT
    c.name,
    a.city,
    a.state,
    COUNT(o.order_id) as order_count,
    SUM(o.total_amount) as total_revenue
FROM customers c
JOIN addresses a ON c.customer_id = a.customer_id
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= '2024-01-01'
GROUP BY c.name, a.city, a.state;
```

2. Dimensional Modeling (Star Schema)

In Plain English

A **star schema** organizes data around a central events table (facts) surrounded by descriptive tables (dimensions). Imagine a sales receipt (fact) that references a customer lookup table, a product catalog, and a calendar — that's a star schema.

```
-- Denormalized for analytics
CREATE TABLE fact_orders (
    order_key BIGINT PRIMARY KEY,
    customer_key INT,
    date_key INT,
    product_key INT,
    quantity INT,
```

The corresponding SQL query:

```
    unit_price DECIMAL(10,2),
    total_amount DECIMAL(10,2)
);

CREATE TABLE dim_customer (
    customer_key INT PRIMARY KEY,
    customer_id INT,
    name VARCHAR(100),
    email VARCHAR(100),
    city VARCHAR(50),
    state VARCHAR(20),
    -- Denormalized: address info in customer dimension
    is_current BOOLEAN
);

-- Much simpler analytics query
SELECT
    c.name,
    c.city,
    c.state,
    COUNT(f.order_key) as order_count,
    SUM(f.total_amount) as total_revenue
FROM fact_orders f
JOIN dim_customer c ON f.customer_key = c.customer_key
JOIN dim_date d ON f.date_key = d.date_key
WHERE d.calendar_year = 2024
GROUP BY c.name, c.city, c.state;
```

3. One Big Table (OBT)

```
-- Fully denormalized
CREATE TABLE orders_obt (
  order_id INT,
  order_date DATE,
  customer_id INT,
  customer_name VARCHAR(100),
  customer_email VARCHAR(100),
  customer_city VARCHAR(50),
  customer_state VARCHAR(20),
  product_id INT,
  product_name VARCHAR(200),
  product_category VARCHAR(50),
  quantity INT,
  unit_price DECIMAL(10,2),
  total_amount DECIMAL(10,2)
);

-- Trivial analytics query
SELECT
  customer_name,
  customer_city,
  customer_state,
  COUNT(*) as order_count,
  SUM(total_amount) as total_revenue
FROM orders_obt
WHERE YEAR(order_date) = 2024
GROUP BY customer_name, customer_city, customer_state;
```

Decision Framework: When to Use Each

Pattern Use When Avoid When Examples

OLTP systems,		
data integrity	Analytics queries,	User profiles,

Normalized

critical,	read-heavy	financial
-----------	------------	-----------

(3NF)

frequent updates Balanced approach, mixed	workloads	transactions
		General

Star Simple analytics

workloads,	purpose
------------	---------

Schema only

team knows	warehouse
------------	-----------

SQL Analyst-heavy	Complex	
teams, simple	relationships,	BI dashboards,

OBT aggregations, executive

frequent schema columnar		reporting
	changes	
storage Audit trails,		
	Point-in-time	
ML features,		User behavior

Event Logs queries, storage

schema		tracking
	costs matter	
evolution		

Real-World Example: E-commerce Product Catalog

Let's see how the same data looks across different modeling approaches:

Normalized Approach:

```
-- 6 tables, 4 JOINS to get basic product info
CREATE TABLE brands (id, name);
CREATE TABLE categories (id, name, parent_id);
CREATE TABLE products (id, brand_id, category_id, name,
```

SQL

description);

```
CREATE TABLE product_variants (id, product_id, sku, price);
CREATE TABLE inventory (variant_id, quantity, location_id);
CREATE TABLE locations (id, name, type);

-- Query requires 5 JOINS
SELECT p.name, b.name, c.name, pv.price, i.quantity
FROM products p
JOIN brands b ON p.brand_id = b.id
JOIN categories c ON p.category_id = c.id
JOIN product_variants pv ON p.id = pv.product_id
JOIN inventory i ON pv.id = i.variant_id
WHERE i.location_id = 123;
```

SQL

Denormalized Approach:

```
-- 1 table, no JOINS
CREATE TABLE product_catalog (
    product_id INT,
    variant_id INT,
    sku VARCHAR(50),
    product_name VARCHAR(200),
    brand_name VARCHAR(100),
    category_name VARCHAR(100),
    category_path VARCHAR(500),    -- Electronics > Phones >
```

SQL

Smartphones

```
price DECIMAL(10,2),
    inventory_quantity INT,
    location_name VARCHAR(100),
    last_updated TIMESTAMP
);

-- Simple query
SELECT product_name, brand_name, category_name, price,
```

SQL

inventory_quantity

```
FROM product_catalog
WHERE location_name = 'Main Warehouse';
```

SQL

Trade-offs:

- Normalized: 85% less storage, consistency, complex queries
- Denormalized: 10x faster queries, easier to understand, data duplication

The Modern Cloud Warehouse Calculus

Why Denormalization Makes More Sense in 2026:

1. Columnar Storage + Compression — Repeated values compress extremely well

```
-- This table might look wasteful...
CREATE TABLE customer_orders (
    order_id INT,
    customer_name VARCHAR(100), -- Repeated for every order
    customer_city VARCHAR(50),  -- Repeated for every order
    ...
);

-- But with columnar compression:
-- "John Smith" repeated 50 times = ~30 bytes total (not 50 × 10
```

SQL

bytes)

```
-- Query scans only needed columns, skips the rest
```

2. Compute Costs > Storage Costs — JOINS require compute; storage is cheap

Storage cost: \$0.023/GB/month (S3) Compute cost: \$2-10/hour (Snowflake/BigQuery)

1TB of denormalized data = \$23/month storage Avoiding 1 hour of daily JOIN processing = \$60-300/month compute savings

3. Analyst Productivity — Simpler queries = faster insights

```
-- Normalized: Requires knowing the schema
SELECT c.segment, p.category, SUM(oi.quantity * oi.price)
FROM customers c
JOIN orders o ON c.id = o.customer_id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
WHERE o.order_date >= '2024-01-01'
GROUP BY c.segment, p.category;
```

```
-- Denormalized: Obvious from column names
SELECT customer_segment, product_category, SUM(revenue)
FROM orders_flat
WHERE order_date >= '2024-01-01'
GROUP BY customer_segment, product_category;
```

Pro Tip Start with star schema, then create OBT views on top for common access patterns. You get the benefits of both: maintainable base layer + easy queries for analysts.

SQL

When NOT to Denormalize

High Update Frequency:

```
-- BAD: Denormalized inventory that changes every second
CREATE TABLE product_inventory_flat (
  product_id INT,
  product_name VARCHAR(200),    -- Updated when product info
```

SQL

changes


```
current_inventory INT,          -- Updated every few seconds
  last_inventory_update TIMESTAMP
);
-- Every inventory update requires updating product_name across all
```

rows

Strict Consistency Requirements:

```
-- Financial systems: normalize to ensure consistency
CREATE TABLE accounts (account_id, balance, ...);
CREATE TABLE transactions (id, account_id, amount, type, ...);
-- Balance must always equal SUM(transactions.amount)
```

SQL

Complex Relationships:

```
-- Social networks: many-to-many relationships don't denormalize
```

well

```
CREATE TABLE users (id, name, ...);
CREATE TABLE friendships (user_id, friend_id, created_at);
-- Can't flatten this without massive data explosion
```

SQL

⚠ Common Mistake

⚠ Common Mistake Applying OLTP normalization rules to OLAP systems. Your data warehouse is not your application database. Different workloads need different models.

□ Checkpoint

1. Why do traditional normalization rules matter less in modern cloud warehouses?
2. Give an example where normalization is still the right choice.
3. What's the main benefit of the OBT approach for analyst teams?

Try It Yourself

You have order data that needs to support: 1. Real-time order processing (high-frequency updates) 2. Daily sales reporting (aggregations by region, product) 3. Customer analytics (lifetime value, segment analysis)

Design schemas for each use case. When would you normalize vs denormalize?

< TL;DR

Star schemas still dominate enterprise data warehouses for good reasons - Advanced SCD techniques (Types 4, 6, 7) solve complex change tracking problems - Modern platforms enable "wide" star schemas with 50+ dimension attributes - Break dimensional modeling rules only when you understand why they exist

Dimensional modeling isn't dead. Despite predictions that NoSQL and data lakes would kill it, star schemas power most production analytics systems. Why? Because they scale, they're understandable, and they work. But the techniques have evolved far beyond Ralph Kimball's original patterns.

Advanced SCD Techniques

You know SCD Types 1 and 2. Here are the advanced techniques for complex scenarios:

SCD Type 4: Add History Table

```
-- Current dimension (fast lookups)
CREATE TABLE dim_customer_current (
    customer_key INT PRIMARY KEY,
    customer_id INT,
    name VARCHAR(100),
    segment VARCHAR(20),
    effective_date DATE,
    current_flag BOOLEAN DEFAULT TRUE
);

-- History dimension (audit trail)
CREATE TABLE dim_customer_history (
    customer_key INT,
    customer_id INT,
    name VARCHAR(100),
    segment VARCHAR(20),
    effective_date DATE,
    expiry_date DATE,
    version_number INT
);

-- Benefits: Fast current lookups, complete history preserved
-- Use case: Dimensions with frequent changes but most queries use
```

current values

SCD Type 6: Hybrid (1 + 2 + 3)

SCD Type 6: Hybrid (1 + 2 + 3)

```
CREATE TABLE dim_customer_hybrid (  
    customer_key INT PRIMARY KEY,  
    customer_id INT,  
    -- Type 1: Always current  
    current_name VARCHAR(100),  
    current_segment VARCHAR(20),  
    -- Type 2: Historical tracking  
    historical_name VARCHAR(100),  
    historical_segment VARCHAR(20),  
    effective_date DATE,  
    expiry_date DATE,  
    is_current BOOLEAN,  
    -- Type 3: Previous value  
    previous_segment VARCHAR(20),  
    segment_changed_date DATE  
);  
  
-- Query current values across all history  
SELECT customer_id, current_segment, COUNT(*)  
FROM fact_orders f  
JOIN dim_customer_hybrid c ON f.customer_key = c.customer_key  
GROUP BY customer_id, current_segment;  
  
-- Query historical accuracy  
SELECT customer_id, historical_segment, COUNT(*)  
FROM fact_orders f  
JOIN dim_customer_hybrid c ON f.customer_key = c.customer_key  
GROUP BY customer_id, historical_segment;
```

SCD Type 7: Dual Key Structure

```
-- Dimension with both natural and durable keys
CREATE TABLE dim_customer_dual (
    customer_key INT PRIMARY KEY,          -- Unique for each version
    customer_durable_key INT,              -- Same across all versions
    customer_id INT,
    name VARCHAR(100),
    segment VARCHAR(20),
    effective_date DATE,
    expiry_date DATE,
    is_current BOOLEAN
);

-- Fact table with both keys
CREATE TABLE fact_orders_dual (
    order_key BIGINT,
    customer_key INT,                      -- Point-in-time accuracy
    customer_durable_key INT,              -- Current relationship
    date_key INT,
    revenue DECIMAL(12,2)
);

-- Current analysis (use durable key)
SELECT c.segment, SUM(f.revenue)
FROM fact_orders_dual f
JOIN dim_customer_dual c ON f.customer_durable_key =
```

c.customer_durable_key

```
WHERE c.is_current = TRUE
GROUP BY c.segment;

-- Historical accuracy (use versioned key)
SELECT c.segment, SUM(f.revenue)
FROM fact_orders_dual f
JOIN dim_customer_dual c ON f.customer_key = c.customer_key
GROUP BY c.segment;
```

Modern Star Schema Patterns

Wide Dimensions (50+ Attributes)

- Traditional advice: keep dimensions narrow
- Modern reality: cloud warehouses handle wide tables well

Creating the `dim_customer_wide` table:

```

CREATE TABLE dim_customer_wide (
    customer_key BIGINT PRIMARY KEY,

    -- Basic info
    customer_id VARCHAR(50),
    name VARCHAR(200),
    email VARCHAR(200),

    -- Demographics (would be separate table in old approach)
    age_range VARCHAR(20),
    income_level VARCHAR(20),
    education VARCHAR(30),
    occupation VARCHAR(50),
    marital_status VARCHAR(20),

    -- Preferences
    communication_preference VARCHAR(20),
    product_interests TEXT[], -- Array of interests
    preferred_language VARCHAR(10),

    -- Behavioral scores
    engagement_score DECIMAL(5,2),
    churn_probability DECIMAL(5,4),
    lifetime_value_score INT,

    -- Geographic
    address_line1 VARCHAR(200),
    city VARCHAR(100),
    state_province VARCHAR(50),
    postal_code VARCHAR(20),
    country VARCHAR(50),
    latitude DECIMAL(10,8),
    longitude DECIMAL(11,8),
    timezone VARCHAR(50),

    -- Metadata
    first_seen_date DATE,
    last_activity_date DATE,
    account_status VARCHAR(20),
    data_source VARCHAR(50),
    created_at TIMESTAMP,
    updated_at TIMESTAMP
);

```

Why wide dimensions work now: - Columnar storage: Only

- scanned columns are read from disk

- Compression: Similar values compress well (many customers in same city)
- Query optimization:

Warehouse optimizers are much better at projection pushdown

Array and JSON in Dimensions

```
-- Modern warehouses support semi-structured data in dimensions
CREATE TABLE dim_product_modern (
  product_key BIGINT PRIMARY KEY,
  product_id VARCHAR(50),
  name VARCHAR(200),
  category VARCHAR(100),

  -- Attributes as JSON (flexible schema)
  attributes JSONB, -- {"color": "red", "size": "large",
```

SQL

```
"material": "cotton"}

```

```
-- Tags as array
  tags VARCHAR(50)[],    -- ["organic", "fair-trade", "bestseller"]

  -- Variant information
  variants JSONB, -- [{"sku": "abc-123", "price": 29.99}, ...]

  effective_date DATE,
  is_current BOOLEAN
);

-- Query with JSON functions
```

The corresponding SQL query:

```
SELECT
    category,
    attributes->>'color' as color,
    AVG(CAST(variants->>0->>'price' AS DECIMAL)) as avg_price
FROM dim_product_modern
WHERE 'bestseller' = ANY(tags)
GROUP BY category, attributes->>'color';
```

Fact Table Design Patterns

Multiple Granularities in One Model

```
-- Transaction-level fact (lowest grain)
CREATE TABLE fact_sales_transaction (
    transaction_key BIGINT PRIMARY KEY,
    customer_key INT,
    product_key INT,
    store_key INT,
    date_key INT,
    time_key INT,
    quantity INT,
    unit_price DECIMAL(10,2),
    discount_amount DECIMAL(10,2),
    tax_amount DECIMAL(10,2),
    total_amount DECIMAL(10,2)
);

-- Daily aggregated fact (performance layer)
CREATE TABLE fact_sales_daily (
    customer_key INT,
    product_key INT,
    store_key INT,
    date_key INT,
    transaction_count INT,
    total_quantity INT,
    total_revenue DECIMAL(12,2),
    total_discount DECIMAL(12,2),
    unique_customers INT,
    PRIMARY KEY (customer_key, product_key, store_key, date_key)
);

-- Monthly aggregated fact (executive reporting)
CREATE TABLE fact_sales_monthly (
    product_key INT,
    store_key INT,
    month_key INT,
    revenue DECIMAL(15,2),
    units_sold INT,
    customers_count INT,
    avg_order_value DECIMAL(10,2),
    PRIMARY KEY (product_key, store_key, month_key)
);
```

Accumulating Snapshot Facts

SQL

```
-- Track process milestones in one row
CREATE TABLE fact_order_lifecycle (
    order_key BIGINT PRIMARY KEY,
    customer_key INT,

    -- Process dates (multiple dates in one fact)
    order_date_key INT,
    payment_date_key INT,
    ship_date_key INT,
    delivery_date_key INT,
    return_date_key INT,

    -- Duration measures (calculated from dates)
    payment_lag_days INT,      -- payment_date - order_date
    fulfillment_lag_days INT,  -- ship_date - payment_date
    delivery_lag_days INT,     -- delivery_date - ship_date
);
```

Status indicators:

SQL

```
-- Status indicators
    is_paid BOOLEAN,
    is_shipped BOOLEAN,
    is_delivered BOOLEAN,
    is_returned BOOLEAN,

    -- Financial measures
    order_amount DECIMAL(12,2),
    shipping_cost DECIMAL(8,2),
    return_amount DECIMAL(12,2)
);

-- Query: Average fulfillment time by quarter
SELECT
    d.quarter_name,
    AVG(fulfillment_lag_days) as avg_fulfillment_days,
    COUNT(CASE WHEN is_delivered THEN 1 END) as delivered_orders,
    COUNT(*) as total_orders
FROM fact_order_lifecycle f
JOIN dim_date d ON f.order_date_key = d.date_key
WHERE d.calendar_year = 2024
GROUP BY d.quarter_name;
```

When to Break Dimensional Modeling Rules

Rule: "All measures in fact tables" Break when: You have calculated measures that are expensive to compute

```
-- Store expensive calculations in dimensions
CREATE TABLE dim_customer_enriched (
  customer_key INT PRIMARY KEY,
  customer_id INT,
  name VARCHAR(100),

  -- Pre-calculated measures (breaks the rule, but saves compute)
  lifetime_order_count INT,
  lifetime_revenue DECIMAL(15,2),
  avg_days_between_orders DECIMAL(8,2),
  last_order_date DATE,
  churn_probability_score DECIMAL(5,4),

  -- Update these nightly via ETL
  calculated_date DATE
);
```

SQL

Rule: "Dimensions should be slowly changing"

Break when: You need real-time dimension updates for operational analytics

```
-- Real-time inventory dimension (updates every minute)
CREATE TABLE dim_product_inventory (
  product_key INT PRIMARY KEY,
  product_id VARCHAR(50),
  name VARCHAR(200),
  category VARCHAR(100),

  -- Fast-changing attributes
  current_inventory_qty INT,
  reserved_qty INT,
  available_qty INT,
  last_inventory_update TIMESTAMP,

  -- Slow-changing attributes
  base_price DECIMAL(10,2),
  effective_date DATE,
  is_current BOOLEAN
);
```

Rule: "One fact table per business process" Break when: You need unified cross-process analytics

```
-- Unified activity fact (multiple business processes)
```

Creating the fact_customer_activity table:

```
CREATE TABLE fact_customer_activity (
  activity_key BIGINT PRIMARY KEY,
  customer_key INT,
  date_key INT,
  activity_type VARCHAR(50), -- order, support_call, email_open,
```

etc.

```
-- Union of all possible measures (many will be NULL)
revenue DECIMAL(12,2),           -- Orders only
support_duration_minutes INT,    -- Support calls only
email_click_count INT,           -- Email activities only
website_session_duration INT,    -- Web activities only

-- Common measures
activity_count INT DEFAULT 1,
activity_value DECIMAL(12,2)     -- Normalized value for cross-
```

activity analysis

```
);
```

SQL

Pro Tip: Hybrid Approach Keep classical star schema as your foundation, but add modern patterns where they solve real problems. Don't break rules just to be different – break them when the benefits clearly outweigh the complexity.

Performance Optimization for Large Star Schemas

Partitioning Strategy:

□ In Plain English

Partitioning means splitting a large table into smaller chunks based on a column value (like date). It's like organizing a filing cabinet by year — when you need 2024 data, you only open the 2024 drawer instead of searching everything.


```
-- Partition fact tables by date for query performance
CREATE TABLE fact_orders (
    order_key BIGINT,
    customer_key INT,
    product_key INT,
    date_key INT,
    revenue DECIMAL(12,2)
) PARTITION BY RANGE (date_key);

-- Create monthly partitions
CREATE TABLE fact_orders_202401 PARTITION OF fact_orders
    FOR VALUES FROM (20240101) TO (20240201);
CREATE TABLE fact_orders_202402 PARTITION OF fact_orders
    FOR VALUES FROM (20240201) TO (20240301);
```

Indexing for Star Schema Queries:

```
-- Composite indexes on dimension keys
CREATE INDEX idx_fact_orders_dims
ON fact_orders (date_key, customer_key, product_key);

-- Covering indexes for common aggregations
CREATE INDEX idx_fact_orders_revenue_covering
ON fact_orders (customer_key, product_key)
INCLUDE (date_key, revenue, quantity);
```

□ Checkpoint

1. When would you use SCD Type 6 instead of Type 2?
2. Why do wide dimensions work better in modern cloud warehouses than they did 10 years ago?
3. Give an example of when you'd break the "measures in facts" rule.

Try It Yourself

Design a dimensional model for a subscription business that tracks: -

- Customer subscriptions (start, pause, resume, cancel)
- Usage events (logins, feature usage, support tickets)

- Billing events (charges, refunds, failed payments)

Consider: What's your fact table grain? How would you track subscription state changes?
What pre-calculated measures might you store in dimensions?

Event-Driven Schema Design

< TL;DR

Event sourcing captures every state change as an immutable event - Enables time travel, audit trails, and flexible downstream processing - Schema evolution is easier but querying point-in-time state is harder - Use for systems where "how did we get here" matters as much as "where are we now"

Traditional databases store current state. Event-driven systems store the sequence of changes that led to that state. Instead of updating a customer record, you append events: "customer_created", "address_changed", "subscription_started". The current state is derived by replaying events.

This sounds academic until you need to answer questions like "What was this customer's subscription status on March 15th?" or "How did this account balance become negative?" Traditional systems can't answer these questions. Event-driven systems can.

Event Sourcing Fundamentals

Traditional State-Based Approach:

```
-- Users table stores current state
CREATE TABLE users (
  user_id INT PRIMARY KEY,
  email VARCHAR(200),
  status VARCHAR(20),          -- active, suspended, deleted
  subscription_tier VARCHAR(20),
  last_login TIMESTAMP,
  created_at TIMESTAMP,
  updated_at TIMESTAMP
);

-- When user upgrades subscription
UPDATE users
SET subscription_tier = 'premium', updated_at = NOW()
WHERE user_id = 123;

-- Problem: We lost the history. When did they upgrade? What tier
```

were they before?

Event-Driven Approach:

```
-- Events table stores every change
CREATE TABLE user_events (
  event_id UUID PRIMARY KEY,
  user_id INT NOT NULL,
  event_type VARCHAR(50) NOT NULL,
  event_data JSONB NOT NULL,
  event_timestamp TIMESTAMP NOT NULL,
  event_version INT NOT NULL,
  metadata JSONB
);

-- When user upgrades subscription - append event, don't update
INSERT INTO user_events (event_id, user_id, event_type, event_data,
```

event_timestamp, event_version)

```
VALUES (
  gen_random_uuid(),
  123,
  'subscription_upgraded',
  '{"from_tier": "basic", "to_tier": "premium", "effective_date":
```

```
"2024-03-15"}',
```

```
    NOW(),
    1
);

-- Current state is derived by replaying events
WITH user_state_changes AS (
```

The corresponding SQL query:

```
SELECT
    user_id,
    event_type,
    event_data,
    event_timestamp,
    ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY
```

SQL

event_timestamp DESC) as rn

```
FROM user_events
    WHERE user_id = 123
)
SELECT
    user_id,
    event_data->>'subscription_tier' as current_tier,
    event_data->>'status' as current_status
FROM user_state_changes
WHERE rn = 1;
```

SQL

Schema Evolution Strategies

Backward Compatible Evolution:

```
-- Version 1 events
{
  "event_type": "order_placed",
  "event_data": {
    "order_id": "12345",
    "customer_id": "cust_789",
    "total_amount": 99.99,
    "items": [
      {"product_id": "prod_abc", "quantity": 2, "price":
```

```
49.99}
```

```
]
  },
  "event_version": 1
}
```

```
-- Version 2: Add shipping information (backward compatible)
{
  "event_type": "order_placed",
  "event_data": {
    "order_id": "12346",
    "customer_id": "cust_790",
    "total_amount": 89.99,
    "items": [
      {"product_id": "prod_def", "quantity": 1, "price":
```

```
89.99}
```

```

],
  "shipping": {
    "method": "express",
    "address": "123 Main St",
    "estimated_delivery": "2024-03-18"
  },
  "event_version": 2
}

-- Version 3: Restructure items (breaking change - new event type)
{
  "event_type": "order_placed_v3",
  "event_data": {
    "order_id": "12347",
    "customer_id": "cust_791",
    "financial": {
      "subtotal": 79.99,
      "tax": 8.00,
      "shipping_cost": 5.99,
      "total": 93.98
    },
    "line_items": [
      {
        "sku": "ABC-123-M",
        "product_name": "Blue T-Shirt Medium",

```

```

        "unit_price": 79.99,
        "quantity": 1,
        "line_total": 79.99
      }
    ]
  },
  "event_version": 3
}

```

Forward Compatible Schema Registry:

```
-- Schema registry approach (used with Kafka, Pulsar)
CREATE TABLE event_schemas (
  schema_id INT PRIMARY KEY,
  event_type VARCHAR(50),
  schema_version INT,
  avro_schema TEXT,
  created_at TIMESTAMP,
  is_active BOOLEAN
);

-- Events reference schema by ID
CREATE TABLE user_events_with_schema (
  event_id UUID PRIMARY KEY,
  user_id INT,
  event_type VARCHAR(50),
  schema_id INT REFERENCES event_schemas(schema_id),
  event_data BYTEA, -- Avro-encoded binary
  event_timestamp TIMESTAMP
);
```

Handling Late-Arriving and Out-of-Order Events

- Problem: Events don't always arrive in chronological order
- Network delays
- System restarts
- Batch processing of historical data
- Mobile apps coming back online

Solution 1: Event Time vs Processing Time

```
CREATE TABLE events_with_timing (
  event_id UUID PRIMARY KEY,
  user_id INT,
  event_type VARCHAR(50),
  event_data JSONB,

  -- Two timestamps: when it happened vs when we received it
  event_time TIMESTAMP NOT NULL,      -- When the event actually
```

occurred

SQL

```
processing_time TIMESTAMP DEFAULT NOW(),    -- When we ingested it

    -- Watermark for late data handling
    watermark_time TIMESTAMP,
    is_late_arrival BOOLEAN DEFAULT FALSE
);

-- Mark events that arrived more than 1 hour late
UPDATE events_with_timing
SET is_late_arrival = TRUE
WHERE processing_time > event_time + INTERVAL '1 hour';
```

Solution 2: Idempotent Event Processing

SQL

```
-- Events include deterministic IDs to handle duplicates
CREATE TABLE idempotent_events (
    event_id UUID PRIMARY KEY,           -- Globally unique
    idempotency_key VARCHAR(100) UNIQUE, -- Business-meaningful key
    user_id INT,
    event_type VARCHAR(50),
    event_data JSONB,
    event_time TIMESTAMP,

    -- Deduplication metadata
```

The corresponding SQL query:

SQL

```
first_seen_at TIMESTAMP DEFAULT NOW(),
    duplicate_count INT DEFAULT 0
);

-- Upsert pattern for handling duplicates
INSERT INTO idempotent_events (event_id, idempotency_key, user_id,
```

event_type, event_data, event_time)

```
VALUES (?, ?, ?, ?, ?, ?)
ON CONFLICT (idempotency_key)
DO UPDATE SET
    duplicate_count = idempotent_events.duplicate_count + 1;
```

Solution 3: Temporal Validity Windows

```
-- Events with validity windows for business logic
CREATE TABLE temporal_events (
  event_id UUID PRIMARY KEY,
  user_id INT,
  event_type VARCHAR(50),
  event_data JSONB,

  -- Temporal dimensions
  event_time TIMESTAMP NOT NULL,          -- When it happened
  effective_time TIMESTAMP NOT NULL,      -- When it takes effect
  discovered_time TIMESTAMP DEFAULT NOW(), -- When we learned
```

about it

```
-- Late arrival handling
  supersedes_event_id UUID, -- References event this one corrects
  is_correction BOOLEAN DEFAULT FALSE
);

-- Example: Backdated subscription cancellation
INSERT INTO temporal_events
VALUES (
  gen_random_uuid(),
  123,
  'subscription_cancelled',
  '{"reason": "billing_failure", "effective_date": "2024-03-10"}',
  NOW(), -- event_time: now
  '2024-03-10', -- effective_time: retroactive
  NOW(), -- discovered_time: now
  NULL,    -- supersedes_event_id
  FALSE    -- is_correction
);
```

Advanced Event Patterns

Saga Pattern for Distributed Transactions:

```
-- Multi-step process tracked as event sequence
CREATE TABLE order_saga_events (
    saga_id UUID,
    step_number INT,
    event_type VARCHAR(50),
    event_data JSONB,
    status VARCHAR(20), -- pending, completed, failed, compensated
    retry_count INT DEFAULT 0,
    created_at TIMESTAMP
);

-- Order processing saga
-- Step 1: Reserve inventory
INSERT INTO order_saga_events VALUES (
    'saga_12345', 1, 'inventory_reserve_requested',
    '{"order_id": "ord_123", "items": [...]}', 'pending', 0, NOW()
);

-- Step 2: Charge payment
INSERT INTO order_saga_events VALUES (
    'saga_12345', 2, 'payment_charge_requested',
    '{"order_id": "ord_123", "amount": 99.99}', 'pending', 0, NOW()
);
```

The corresponding SQL query:

```
);

-- If payment fails, compensate inventory reservation
INSERT INTO order_saga_events VALUES (
    'saga_12345', 1, 'inventory_reserve_compensated',
    '{"order_id": "ord_123", "reason": "payment_failed"}',
```

```
'completed', 0, NOW()
);
```

Event Compaction for Storage Efficiency:

```
-- Snapshot events to reduce replay time
CREATE TABLE user_snapshots (
    user_id INT PRIMARY KEY,
    snapshot_data JSONB,
    snapshot_version INT,
    last_event_id UUID,
    created_at TIMESTAMP
);

-- Replay optimization: start from latest snapshot
WITH latest_snapshot AS (
    SELECT * FROM user_snapshots WHERE user_id = 123
),
events_since_snapshot AS (
    SELECT * FROM user_events
    WHERE user_id = 123
    AND event_timestamp > (SELECT created_at FROM latest_snapshot)
    ORDER BY event_timestamp
)
-- Apply events to snapshot to get current state
SELECT apply_events_to_snapshot(
    (SELECT snapshot_data FROM latest_snapshot),
    array_agg(event_data ORDER BY event_timestamp)
) as current_state
FROM events_since_snapshot;
```

Querying Event-Sourced Data

Point-in-Time State Reconstruction:

```
-- What was this customer's status on March 15th?
WITH customer_events_up_to_date AS (
    SELECT *
    FROM customer_events
    WHERE customer_id = 'cust_123'
    AND event_time <= '2024-03-15 23:59:59'
    ORDER BY event_time
),
state_changes AS (
    SELECT
        event_type,
        event_data,
        event_time,
        CASE event_type
            WHEN 'customer_created' THEN event_data->>'status'
            WHEN 'status_changed' THEN event_data->>'new_status'
            ELSE NULL
        END as status_value,
        CASE event_type
            WHEN 'subscription_started' THEN event_data->>'tier'
            WHEN 'subscription_upgraded' THEN event_data->>'to_tier'
            WHEN 'subscription_downgraded' THEN event_data->>'to_tier'
```

```
>>'to_tier'
```

ELSE NULL

```
END as subscription_value
FROM customer_events_up_to_date
)
SELECT
    COALESCE(
        LAST_VALUE(status_value IGNORE NULLS) OVER (ORDER BY
```

```
event_time),
```

```
'unknown'
) as status_on_date,
COALESCE(
    LAST_VALUE(subscription_value IGNORE NULLS) OVER (ORDER BY
```

```
event_time),
```

SQL

```
'none'
    ) as subscription_on_date
FROM state_changes
ORDER BY event_time DESC
LIMIT 1;
```

Event Stream Analytics:

SQL

```
-- Pattern detection: Users who churn within 30 days of signup
WITH customer_lifecycle AS (
    SELECT
        customer_id,
        MIN(CASE WHEN event_type = 'customer_created' THEN
```

event_time END) as created_at,

```
MIN(CASE WHEN event_type = 'subscription_cancelled' THEN
```

event_time END) as churned_at

SQL

```
FROM customer_events
    GROUP BY customer_id
),
early_churners AS (
    SELECT
        customer_id,
        created_at,
        churned_at,
        churned_at - created_at as time_to_churn
    FROM customer_lifecycle
    WHERE churned_at IS NOT NULL
        AND churned_at - created_at <= INTERVAL '30 days'
)
SELECT
    DATE_TRUNC('month', created_at) as signup_month,
    COUNT(*) as early_churn_count,
    AVG(time_to_churn) as avg_time_to_churn
FROM early_churners
GROUP BY signup_month
ORDER BY signup_month;
```

When to Use Event Sourcing

Use Event Sourcing When: - Audit requirements: Financial

- systems, healthcare, compliance-heavy industries
- Complex workflows: Multi-step processes with compensation logic
- Time- based analysis: "How did we get here?" matters as much as current state
- Regulatory compliance: Need to prove what happened when
- Debugging: Production issues require understanding historical state changes

Avoid Event Sourcing When: - Simple CRUD: Basic create/read/update/delete doesn't benefit from event history -

Storage costs critical: Events require more storage than state

- snapshots
- Query complexity unacceptable: Team can't handle point-in-time reconstruction logic
- Performance requirements:

Sub-millisecond reads are required

⚠ Common Mistake

⚠ Common Mistake Using event sourcing everywhere because it's cool. It adds significant complexity. Use it only when the benefits (audit trail, time travel, workflow tracking) outweigh the costs (complex queries, more storage, learning curve).

□ Checkpoint

1. What's the difference between event time and processing time?
2. How would you handle an event that arrives 6 hours late?
3. When would you choose event sourcing over traditional state- based storage?

Try It Yourself

Design an event-sourced schema for an e-commerce order system that needs to track:

- Order placement, modification, and cancellation
- Payment processing (auth, capture, refund)
- Fulfillment (picking, packing, shipping, delivery)
- Returns and exchanges

Consider: What events would you capture? How would you handle out-of-order events? What queries would be difficult?

< TL;DR

Graph databases excel at relationship-heavy queries (social networks, fraud detection, recommendations) - Document databases handle semi-structured data and rapid schema evolution - Both complement relational systems rather than replacing them - Choose based on query patterns, not storage efficiency

Not everything fits neatly into rows and columns. Social networks have complex relationships. Product catalogs have nested attributes that change frequently. Fraud detection requires analyzing networks of connections. For these use cases, specialized data models often work better than forcing everything into relational tables.

When Relational Models Break Down

Problem 1: Deep Relationship Queries

```
-- Find friends of friends who like the same products (SQL
```

nightmare)

```
WITH friend_connections AS (  
    SELECT f1.user_id as user, f1.friend_id as friend  
    FROM friendships f1  
    UNION  
    SELECT f2.friend_id as user, f2.user_id as friend --
```

SQL

bidirectional

```

FROM friendships f2
),
friends_of_friends AS (
    SELECT DISTINCT
        fc1.user as original_user,
        fc2.friend as friend_of_friend
    FROM friend_connections fc1
    JOIN friend_connections fc2 ON fc1.friend = fc2.user
    WHERE fc1.user != fc2.friend -- not themselves
),
common_interests AS (
    SELECT
        fof.original_user,
        fof.friend_of_friend,
        COUNT(DISTINCT l1.product_id) as shared_likes
    FROM friends_of_friends fof
    JOIN likes l1 ON fof.original_user = l1.user_id
    JOIN likes l2 ON fof.friend_of_friend = l2.user_id
        AND l1.product_id = l2.product_id
    GROUP BY fof.original_user, fof.friend_of_friend
    HAVING COUNT(DISTINCT l1.product_id) >= 3
)
SELECT * FROM common_interests;

-- This query is: slow, hard to read, hard to modify, and doesn't

```

scale

Problem 2: Variable Schema Structure

```

-- Product catalog with different attributes per category
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    name VARCHAR(200),

```

The corresponding SQL query:

```

category VARCHAR(100),
    base_price DECIMAL(10,2),

    -- Electronics attributes
    brand VARCHAR(100),
    model VARCHAR(100),
    warranty_months INT,

    -- Clothing attributes
    size VARCHAR(10),
    color VARCHAR(50),
    material VARCHAR(100),

    -- Book attributes
    author VARCHAR(200),
    isbn VARCHAR(20),
    page_count INT,

    -- Home & Garden attributes
    dimensions VARCHAR(100),
    weight_kg DECIMAL(8,2),
    assembly_required BOOLEAN
);

-- Problems:
-- 1. Sparse table (most columns NULL for most products)
-- 2. Schema changes require ALTER TABLE
-- 3. Can't add category-specific attributes without code changes
-- 4. Complex queries to filter by category-specific attributes

```

Graph Database Design Patterns

Property Graph Model:

```

// Nodes with properties CREATE (u:User {id: 123, name: 'Alice', age: 28, city: 'NYC'})
CREATE (p:Product {id: 456, name: 'iPhone 15', category: 'Electronics'})

// Relationships with properties CREATE (u)-[:PURCHASED {date: '2024-03-15', amount:
999.99, rating: 5}]->(p) CREATE (u)-[:LIVES_IN]->(c:City {name: 'NYC', state: 'NY',
population: 8500000})

// Easy to extend relationships CREATE (u)-[:FOLLOWS {since: '2023-01-01',
notification_enabled: true}]->(friend:User)

```

Fraud Detection Pattern:

```
// Find suspicious networks: multiple users sharing payment methods and addresses
MATCH (u1:User)-[:USES_CARD]->(card:CreditCard)<-[:USES_CARD]- (u2:User) WHERE u1
<> u2 WITH u1, u2, card MATCH (u1)-[:LIVES_AT]->(addr:Address)<-[:LIVES_AT]- (u2) WITH
u1, u2, card, addr MATCH (u1)-[:PLACED_ORDER]->(o1:Order) MATCH (u2)-
[:PLACED_ORDER]->(o2:Order) WHERE o1.created_date > date('2024-01-01') AND o2.cre-
ated_date > date('2024-01-01') AND abs(duration.inDays(o1.created_date, o2.cre-
ated_date).days) <= 7 RETURN u1.id, u2.id, card.last_four, addr.zip_code, count(o1) +
count(o2) as total_recent_orders ORDER BY total_recent_orders DESC;
```

// This query would be incredibly complex in SQL but natural in graph query language

Recommendation Engine Pattern:

```
// Collaborative filtering: find products liked by similar users MATCH (target:User {id:
123})-[:PURCHASED]->(product:Product) WITH target, collect(product) as target_purchases
```

```
MATCH (similar:User)-[:PURCHASED]->(common:Product) WHERE common IN tar-
get_purchases AND similar <> target WITH target, similar, target_purchases,
count(common) as overlap WHERE overlap >= 3 -- users with at least 3 common pur-
chases
```

```
MATCH (similar)-[:PURCHASED]->(recommendation:Product) WHERE NOT recommendation
IN target_purchases WITH recommendation, count(similar) as recommender_count ORDER
BY recommender_count DESC LIMIT 10
```

```
RETURN recommendation.name, recommendation.category, recommender_count
```

Document Database Design Patterns

Flexible Product Catalog:

```
// Electronics product
{
  "_id": "prod_12345",
  "name": "iPhone 15 Pro",
  "category": "electronics",
  "base_price": 999.99,
  "brand": "Apple",
  "specifications": {
    "display": {
      "size_inches": 6.1,
      "resolution": "2556x1179",
      "technology": "Super Retina XDR"
    },
    "processor": {
      "chip": "A17 Pro",
      "cores": 6,
      "neural_engine": true
    },
    "camera": {
      "main_mp": 48,
      "ultra_wide_mp": 12,
      "telephoto_mp": 12,
      "features": ["Night mode", "Portrait mode", "4K video"]
    },
    "storage_options": [128, 256, 512, 1000],
    "colors": ["Natural Titanium", "Blue Titanium", "White
```

Titanium", "Black Titanium"]

```
},
  "compatibility": ["iOS 17", "MagSafe", "Qi wireless charging"],
  "tags": ["flagship", "5G", "wireless charging"],
  "created_at": "2024-03-15T10:00:00Z"
}

// Clothing product - completely different structure
{
  "_id": "prod_67890",
  "name": "Organic Cotton T-Shirt",
  "category": "clothing",
  "base_price": 29.99,
  "brand": "EcoWear",
  "clothing_details": {
    "material_composition": [
      {"material": "organic cotton", "percentage": 95},
```

The data structure:

```
{ "material": "elastane", "percentage": 5 }  
],  
"fit": "regular",  
"care_instructions": ["Machine wash cold", "Tumble dry low",
```

JSON

"Do not bleach"],

```
"size_chart": {  
  "S": { "chest_cm": 86, "length_cm": 71 },  
  "M": { "chest_cm": 91, "length_cm": 74 },  
  "L": { "chest_cm": 97, "length_cm": 77 }  
},  
"available_colors": [  
  { "name": "Forest Green", "hex": "#355E3B", "stock":
```

{ "S": 12, "M": 8, "L": 5 } },

```
{ "name": "Ocean Blue", "hex": "#0077BE", "stock": { "S":
```

JSON

15, "M": 10, "L": 3 } }

```
]  
},  
"certifications": ["GOTS Certified", "Fair Trade", "Carbon
```

Neutral"],

```
"tags": ["organic", "sustainable", "basic"],  
  "seasonal": {  
    "spring": true,  
    "summer": true,  
    "fall": false,  
    "winter": false  
  }  
}
```

User Profile with Embedded Preferences:

```
{
  "_id": "user_12345",
  "email": "alice@example.com",
  "profile": {
    "first_name": "Alice",
    "last_name": "Johnson",
    "birth_year": 1995,
    "location": {
      "city": "San Francisco",
      "state": "CA",
      "country": "US",
      "timezone": "America/Los_Angeles"
    }
  },
  "preferences": {
    "categories": [
      {"name": "electronics", "weight": 0.8},
      {"name": "books", "weight": 0.6},
      {"name": "home", "weight": 0.3}
    ],
    "brands": {
      "loved": ["Apple", "Nintendo", "Patagonia"],
      "avoided": ["FastFashion Co", "Cheap Electronics Inc"]
    },
    "price_sensitivity": {
      "max_electronics": 2000,
      "max_clothing": 100,
      "will_wait_for_sales": true
    },
    "communication": {
      "email_frequency": "weekly",
      "sms_enabled": false,
      "push_notifications": true,
      "preferred_language": "en-US"
    }
  },
  "behavior_scores": {
    "engagement": 0.85,
    "purchase_probability": 0.72,
    "churn_risk": 0.15,
    "lifetime_value_tier": "high",
    "last_calculated": "2024-03-15T08:30:00Z"
  }
}
```

```
},  
  "purchase_history_summary": {  
    "total_orders": 47,  
    "total_spent": 3821.50,  
    "avg_order_value": 81.31,  
    "first_purchase": "2022-06-15",  
    "last_purchase": "2024-03-10",  
    "favorite_categories": ["electronics", "books"],  
    "seasonal_patterns": {  
      "high_months": ["November", "December", "January"],  
      "low_months": ["February", "August"]  
    }  
  }  
}
```

Hybrid Patterns: Polyglot Persistence

Real-world architecture combines multiple data models:


```
-- Core transactional data in PostgreSQL
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    email VARCHAR(200),
    status VARCHAR(20),
    created_at TIMESTAMP
);

CREATE TABLE orders (
    order_id BIGINT PRIMARY KEY,
    user_id INT REFERENCES users(user_id),
    total_amount DECIMAL(12,2),
    order_date DATE
);

// Rich user profiles in MongoDB
db.user_profiles.insertOne({
    "user_id": 123,
    "preferences": {...},
    "behavior_data": {...},
    "recommendation_cache": [...]
});

// Product catalog in MongoDB
db.products.insertOne({
    "product_id": 456,
    "specifications": {...},
    "inventory": {...}
});
```

```
// Relationships in Neo4j
CREATE (u:User {id: 123})
CREATE (p:Product {id: 456})
CREATE (u)-[:VIEWED {timestamp: datetime()}]->(p)
CREATE (u)-[:PURCHASED {date: date(), amount: 99.99}]->(p)
```

```
-- Analytics in Snowflake (denormalized from all sources)
CREATE TABLE fact_user_interactions (
  interaction_id BIGINT PRIMARY KEY,
  user_id INT,
  product_id INT,
  interaction_type VARCHAR(50), -- viewed, purchased, reviewed
  interaction_date DATE,
  user_segment VARCHAR(50),      -- from MongoDB profile
  product_category VARCHAR(100), -- from MongoDB catalog
  relationship_strength FLOAT    -- from Neo4j analysis
);
```

Query Patterns Comparison

Graph Query (Neo4j Cypher):

// Find influencers: users whose purchases predict others' purchases

```
MATCH (influencer:User)-[:PURCHASED]->(product:Product)<-[:PURCHASED]-(follower:User)
WHERE influencer.follower_count > 1000 WITH influencer, product, collect(follower) as influenced_users
WHERE size(influenced_users) >= 10 MATCH (influencer)-[:PURCHASED]->(recent:Product)
WHERE recent.launch_date > date('2024-01-01') RETURN influencer.name, recent.name as trending_product,
size(influenced_users) as influence_score ORDER BY influence_score DESC;
```

Document Query (MongoDB):

```
// Find products matching complex preference criteria
db.products.find({
  $and: [
    {"category": "electronics"},
    {"base_price": {$lte: 1000}},
    {"specifications.display.size_inches": {$gte: 6}},
    {"tags": {$in: ["5G", "wireless charging"]}},
    {
      $expr: {
        $gt: [
          {$size: {$setIntersection: ["$compatibility",
["iOS 17", "MagSafe"]]}},
```

```

0
    ]
    }
  }
]
}).sort({"specifications.camera.main_mp": -1});

```

Relational Query (PostgreSQL):

```

-- Strong consistency for financial calculations
SELECT
  u.user_id,
  u.email,
  SUM(o.total_amount) as lifetime_value,
  COUNT(o.order_id) as order_count,
  AVG(o.total_amount) as avg_order_value
FROM users u
JOIN orders o ON u.user_id = o.user_id
WHERE o.order_date >= CURRENT_DATE - INTERVAL '12 months'
  AND u.status = 'active'
GROUP BY u.user_id, u.email
HAVING SUM(o.total_amount) > 1000;

```

SQL

When to Choose Each Model

Use Case Best Model Why

Complex

Social Networks Graph relationship

traversal
Pattern
detection

Fraud Detection Graph

across
connections
Variable
schema,

Product Catalogs Document

nested
attributes
Semi-
structured,

User Profiles Document

evolving
preferences
ACID

Financial compliance,

Relational

Transactions consistency

Aggregations,

Analytics/Reporting Relational (OLAP) historical

analysis
Relationships

Real-time

Graph + Document + flexible

content

Data Migration Strategies

From Relational to Graph:

```
# ETL script to migrate normalized data to graph
import neo4j

def migrate_social_network():
    # Extract from PostgreSQL
    pg_cursor.execute("""
        SELECT u.user_id, u.name, f.friend_id, f.created_date
        FROM users u
        JOIN friendships f ON u.user_id = f.user_id
        """)

    # Transform and load to Neo4j
    with neo4j_driver.session() as session:
        for row in pg_cursor.fetchall():
            session.run("""
                MERGE (u1:User {id: $user_id, name: $name})
                MERGE (u2:User {id: $friend_id})
                CREATE (u1)-[:FRIENDS_WITH {since: $created_date}]->
```

SQL

```
(u2)
```

```
""", user_id=row[0], name=row[1], friend_id=row[2],
```

```
created_date=row[3])
```

From Relational to Document:

```
# Aggregate normalized data into document structure
def migrate_user_profiles():
    pipeline = [
        # Aggregate user data from multiple tables
        {"$lookup": {
            "from": "orders",
            "localField": "user_id",
            "foreignField": "user_id",
            "as": "orders"
        }},
        {"$lookup": {
            "from": "preferences",
            "localField": "user_id",
            "foreignField": "user_id",
            "as": "preferences"
        }},
        # Transform into nested document
        {"$project": {
            "user_id": 1,
            "profile": {"name": "$name", "email": "$email"},
            "purchase_summary": {
                "total_orders": {"$size": "$orders"},
                "total_spent": {"$sum": "$orders.amount"},
                "avg_order_value": {"$avg": "$orders.amount"}
            },
            "preferences": {"$arrayElemAt": ["$preferences", 0]}
        }}
    ]
```

Pro Tip: Start with Relational, Add Specialized Models
Begin with PostgreSQL for your core data. Add specialized databases (graph, document) when you hit specific query patterns that relational databases handle poorly. Most applications need multiple data models.

□ Checkpoint

1. Give an example of a query that's much easier in a graph database than SQL.
2. When would you choose a document database over a relational database?
3. What's the main trade-off of using specialized data models like graph and document databases?

Try It Yourself

Design data models for a professional networking platform (like LinkedIn) that needs to support: 1. User profiles with variable skills and experience 2. Connection recommendations ("People you may know") 3. Job recommendations based on network and skills 4. Company analytics (employee movement, hiring patterns)

Consider: What data would you store in relational tables? What would benefit from graph modeling? What might work better as documents?

Module 1 Project: Design Multi-Model

Schema for Social Commerce Platform

Objective

Design and implement a comprehensive data architecture for a social commerce platform that combines social networking, e-commerce, and content creation. You'll use multiple data models (relational, graph, document) and justify your choices.

Prerequisites

Understanding of dimensional modeling, normalization/denormalization trade-offs Basic knowledge of SQL, and conceptual understanding of graph and document databases

Familiarity with JSON and nested data structures

Expected Time

4-5 hours

Problem Statement

"SocialShop" is a platform where:

- Creators post content (photos, videos, reviews) and earn commission from affiliate sales
- Users follow creators, like/share content, and purchase recommended products
- Brands sponsor creators and track campaign performance
- Analytics teams need to understand engagement patterns, sales attribution, and creator effectiveness

Business Requirements

1. Creator Analytics: Track follower growth, engagement rates, sales attribution 2. Social Features: Friend connections, content feeds, recommendation algorithms 3. E-commerce: Product catalog, orders, inventory, pricing 4. Campaign Management: Brand sponsorships, affiliate tracking, commission calculations 5. Real-time Features: Live engagement metrics, trending content 6. Compliance: Audit trails for financial transactions, content moderation history

Step 1: Architecture Decision

Design a polyglot persistence architecture. For each data domain, choose the appropriate data model:

Domain Recommended Model Reasoning

ACID

Core Users & compliance,

Relational (PostgreSQL)

Transactions financial

accuracy
Complex

Social Graph &

Graph (Neo4j) relationship

Recommendations

traversal
Semi-structured

Content &

Document (MongoDB) data, flexible

Product Catalogs

schema
Aggregations,

Analytics & Data Warehouse

historical

Reporting (Snowflake)

analysis
Time-series High-frequency

Real-time Metrics

(Redis/InfluxDB)

updates

Step 2: Relational Schema Design

Design PostgreSQL tables for core transactional data:

```
-- Design schemas for:
-- 1. User authentication and basic profile
-- 2. Order and payment processing
-- 3. Financial transactions (commissions, payouts)
-- 4. Audit logs for compliance

-- Consider:
-- - What needs strict consistency?
-- - What data changes frequently vs rarely?
-- - What queries need ACID guarantees?
```

Step 3: Graph Schema Design

Design Neo4j graph structure for social and recommendation features:

// Design node types and relationships for: // 1. User-to-user relationships (followers, friends) // 2. User-content interactions (likes, shares, comments) // 3. Content-product relationships (mentions, features) // 4. Brand-creator partnerships

- // Consider: //
- What traversal queries will you need? //
- How will you handle weighted relationships? //
- What properties belong on edges vs nodes?

Step 4: Document Schema Design

Design MongoDB collections for flexible content:

```
// Design document structures for:  
// 1. User profiles with dynamic preferences  
// 2. Product catalogs with category-specific attributes  
// 3. Content posts with metadata and engagement  
// 4. Campaign configurations with flexible rules  
  
// Consider:  
// - What data has variable schema?  
// - What needs fast reads vs complex queries?  
// - How will you handle schema evolution?
```

Step 5: Data Warehouse Schema

Design Snowflake dimensional model for analytics:

```
-- Design fact and dimension tables for:  
-- 1. Content engagement metrics (views, likes, shares)  
-- 2. Sales attribution (creator → content → purchase)  
-- 3. Creator performance (follower growth, commission earned)  
-- 4. Campaign effectiveness (reach, conversion, ROI)  
  
-- Consider:  
-- - What's your fact table grain?  
-- - How will you handle many-to-many relationships?  
-- - What pre-calculated metrics make sense?
```

Step 6: Integration and Data Flow

Design ETL/ELT processes that keep all systems synchronized:

```
# Design data pipeline for:  
# 1. Real-time event streaming (user actions → multiple systems)  
# 2. Batch synchronization (daily analytics refresh)  
# 3. Change data capture (transactional changes → analytics)  
# 4. Graph analytics (community detection, influence scoring)  
  
# Consider:  
# - How do you maintain consistency across systems?  
# - What's your strategy for handling conflicts?  
# - How do you handle system failures?
```

Deliverables

Create these artifacts:

- 1. Architecture Decision Document (architecture-decisions.md)
- Rationale for each data model choice
- Trade-offs and alternatives considered
- Integration strategy

2. Relational Schema (postgres-schema.sql)

```
-- Table definitions with:  
-- - Primary and foreign keys  
-- - Indexes for performance  
-- - Check constraints for data quality  
-- - Comments explaining business rules
```

3. Graph Schema (neo4j-schema.cypher)

- -- Node and relationship definitions with: --
- Property constraints --
- Index definitions --
- Example queries for key use cases

4. Document Schema (mongodb-schema.json)

```
// Collection schemas with:  
// - Example documents  
// - Index definitions  
// - Validation rules where applicable
```

SQL

5. Dimensional Model (snowflake-schema.sql)

```
-- Fact and dimension tables with:  
-- - SCD type definitions  
-- - Partitioning strategy  
-- - Aggregation tables for performance
```

- 6. Data Flow Diagram (data-flow.md)
- How data moves between systems

- Batch vs streaming decisions
- Error handling and recovery

Evaluation Criteria

- Architecture Decisions (25 points)
 - ☐ Clear rationale for each technology choice
 - ☐ Understanding of trade-offs (consistency, performance, complexity)
 - ☐ Realistic assessment of operational overhead
- Schema Design (40 points)
 - ☐ Proper use of each data model's strengths
 - ☐ Appropriate normalization/denormalization decisions
 - ☐ Performance considerations (indexes, partitioning)
 - ☐ Data quality constraints and validation
- Integration Design (20 points)
 - ☐ Clear data flow between systems
 - ☐ Handling of eventual consistency
 - ☐ Error handling and recovery strategies
- Business Alignment (15 points)
 - ☐ Schema supports all business requirements
 - ☐ Considers operational needs (analytics, reporting) -

☐ Checkpoint

1. ☐ Scalability for growth scenarios

Challenge Questions

As you design, consider these complexities:

1. Late-arriving Data: A user makes a purchase, but the attribution data (which creator/content influenced it) arrives 10 minutes later. How do you handle this?
2. Schema Evolution: Brands want to add new campaign types with different tracking requirements. How does your document model handle this?
3. Graph Performance: You need to find "influencers" (users whose content drives the most sales among their followers). How do you make this query fast on a graph with 10M users and 100M relationships?
4. Cross-system Analytics: Marketing wants to see "creator effectiveness" metrics that combine social engagement (graph), content performance (document), and sales results (relational). How do you efficiently combine data across systems?
5. Compliance: You need to implement "right to be forgotten" (GDPR). A user requests deletion of all their data. How do you handle this across your polyglot architecture?

Stretch Goals

1. Event Sourcing: Implement event sourcing for critical business events (follows, purchases, content posts)
2. Real-time Analytics: Design a real-time dashboard showing live engagement metrics
3. Machine Learning Integration: Design feature stores and pipelines for recommendation algorithms
4. Global Scale: Consider multi-region deployment and data sovereignty requirements

In Plain English

A **feature store** is a shared library of pre-computed data inputs for ML models. Instead of each data scientist computing "average order value" differently, everyone uses the same centrally managed calculation.

Solution Approach

1. Start with the transactions: Design PostgreSQL schema first - this is your source of truth
2. Add specialized models: Identify what queries are hard in relational, move those to graph/document
3. Design integration last: Once you know what data lives where, design

the pipelines 4. Validate with queries: Write example queries for each system to verify your design works

This project simulates real-world polyglot architecture decisions you'll face at companies like Instagram, TikTok, or any social commerce platform.# Module 2: Storage Systems Deep Dive

What you'll learn

How to choose the right storage technology for your data. You'll master the trade-offs between OLTP, OLAP, and hybrid systems, understand modern table formats, and design cost-effective storage architectures that scale.

02

Storage Systems Deep Dive Deep dive into OLTP, OLAP, columnar storage, table formats

< TL;DR

OLTP: Fast writes, simple queries, ACID compliance - use for operational systems - OLAP: Fast scans, complex analytics, denormalized data - use for warehouses and reporting - HTAP: Handles both workloads in one system - use when you need real-time analytics on operational data - Choose based on query patterns and consistency requirements, not marketing hype

The first decision in any storage architecture: Do you need to optimize for transactions (OLTP) or analytics (OLAP)? This choice affects everything downstream - your data model, your indexing strategy, your query patterns, even your team structure.

Most companies end up needing both. The question is: separate systems or hybrid?

OLTP Systems: Optimized for Operations

Core Characteristics: - Row-based storage: for

- reading/writing complete records
- ACID compliance: Strong consistency, reliable transactions
- High write throughput: Optimized for frequent inserts/updates
- Simple queries: Usually touching 1-1000 records
- Normalized schemas: Reduce data duplication, ensure consistency

Example Workload:

```
-- Typical OLTP queries - fast, targeted, transactional
BEGIN;
UPDATE account_balances
SET balance = balance - 1000
WHERE account_id = 12345 AND balance >= 1000;

INSERT INTO transactions (from_account, to_account, amount,
```

SQL

timestamp)

```
VALUES (12345, 67890, 1000, NOW());
```

SQL

```
UPDATE account_balances  
SET balance = balance + 1000  
WHERE account_id = 67890;  
COMMIT;
```

```
-- Query completes in milliseconds, touches 3 rows total
```

When to Choose OLTP: - Application backends: User

- management, order processing, inventory
- Financial systems: Payments, accounting, anything requiring ACID
- Real-time operations: Live chat, gaming, trading systems
- High consistency needs: Banking, healthcare, compliance-heavy industries

Technologies:

Traditional: PostgreSQL, MySQL, Oracle, SQL Server

NoSQL: MongoDB, CouchDB, Amazon DynamoDB

NewSQL: CockroachDB, TiDB, YugabyteDB (ACID + distributed)

OLAP Systems: Optimized for Analytics

Core Characteristics: - Columnar storage: Read only the columns

- you need
- Denormalized schemas: Pre-joined for query performance
- Massive parallel processing: Scan millions of rows in seconds
- Complex aggregations: GROUP BY, window functions, statistical functions
- Compression: Store more data in less space

Example Workload:

```
-- Typical OLAP queries - complex, scanning millions of rows
SELECT
    DATE_TRUNC('month', order_date) as month,
    customer_segment,
    product_category,
    COUNT(*) as order_count,
    SUM(order_amount) as revenue,
    AVG(order_amount) as avg_order_value,
    COUNT(DISTINCT customer_id) as unique_customers,
    SUM(order_amount) / COUNT(DISTINCT customer_id) as
```

SQL

revenue_per_customer

```
FROM fact_orders f
JOIN dim_customer c ON f.customer_key = c.customer_key
JOIN dim_product p ON f.product_key = p.product_key
JOIN dim_date d ON f.order_date_key = d.date_key
WHERE d.calendar_year IN (2023, 2024)
GROUP BY 1, 2, 3
ORDER BY month, revenue DESC;

-- Query scans 50M+ rows, completes in 5-30 seconds
```

SQL

When to Choose OLAP:

- Business intelligence: Dashboards, reports, executive metrics
- Data science: Feature engineering, model training, exploratory analysis
- Historical analysis: Trends, cohort analysis, time-series forecasting
- Regulatory reporting: Compliance, audit, financial reporting

Technologies:

Cloud Warehouses: Snowflake, BigQuery, Redshift, Azure Synapse

On-premise: Teradata, Vertica, Greenplum

Open Source: ClickHouse, Apache Druid, DuckDB

Lakehouse: Databricks, Apache Iceberg, Delta Lake

The Traditional Split: Why Two Systems?

Storage Format Differences:

OLTP (Row-based): Row 1: [id=1, name="Alice", email="alice@example.com", created="2024-01-15"] Row 2: [id=2, name="Bob", email="bob@example.com", created="2024-01-16"] Row 3: [id=3, name="Charlie", email="charlie@example.com", created="2024-01-17"]

OLAP (Column-based): ID Column: [1, 2, 3,...] Name Column: ["Alice", "Bob", "Charlie",...] Email Column: ["alice@example.com", "bob@example.com", "charlie@example.com",...] Created Column: ["2024-01-15", "2024-01-16", "2024-01-17",...]

Why This Matters: - OLTP query: "Get Alice's complete profile" →

Read one row, all columns - OLAP query: "Count users by month" → Scan entire Created column, ignore everything else

Index Strategy Differences:

```
-- OLTP indexes: Support fast lookups and transactions
CREATE INDEX idx_users_email ON users(email); -- Login queries
CREATE INDEX idx_orders_user_id ON orders(user_id); -- User's
```

orders

```
CREATE INDEX idx_orders_status ON orders(status) WHERE status =
```

SQL

'pending'; -- Operational queries

```
-- OLAP indexes: Support scan performance and aggregations
```

```
CREATE INDEX idx_fact_orders_date_customer ON
```

```
fact_orders(order_date, customer_key);
```

```
CREATE INDEX idx_fact_orders_product_month ON
```

```
fact_orders(product_key, DATE_TRUNC('month', order_date));
```

```
-- Often use bitmap indexes, bloom filters, zone maps instead of B-
```

trees

Modern Reality: HTAP (Hybrid Transaction/Analytical Processing)

The lines are blurring. Modern systems can handle both workloads:

TiDB Example (NewSQL HTAP):

```
-- Same database, different query patterns
-- Transactional workload (TiKV storage engine)
UPDATE user_balances SET balance = balance - 100 WHERE user_id =
```

SQL

123;

```
-- Analytical workload (TiFlash columnar engine)
SELECT
    DATE(created_at) as day,
    COUNT(*) as signups,
    AVG(initial_balance) as avg_balance
FROM users
WHERE created_at >= '2024-01-01'
GROUP BY DATE(created_at)
ORDER BY day;

-- Automatically routes queries to appropriate storage engine
```

ClickHouse Example (OLAP with fast writes):

```
-- Fast analytical queries
SELECT
    toYYYYMM(timestamp) as month,
    event_type,
    COUNT() as event_count
FROM events
WHERE timestamp >= '2024-01-01'
GROUP BY month, event_type
ORDER BY month, event_count DESC;

-- But also handles high-frequency inserts
INSERT INTO events (user_id, event_type, timestamp, properties)
VALUES
    (123, 'page_view', '2024-03-15 14:30:00', '{"page":
```

"/products"}'),

```
(456, 'purchase', '2024-03-15 14:30:01', '{"amount": 99.99}'),
    (789, 'signup', '2024-03-15 14:30:02', '{"source": "google"}');
-- Can handle 100k+ inserts/second while serving analytics
```

Decision Framework: Choosing Your Storage Strategy

Single System Approach (HTAP):

Use when:

❑ Checkpoint

1. ❑ Real-time analytics requirements (< 1 minute lag)

❑ Checkpoint

1. ❑ Small to medium data volumes (< 10TB)

❑ Checkpoint

1. ❑ Limited team resources (1-2 data engineers)

❑ Checkpoint

1. ❑ Simple data transformations

❑ Checkpoint

1. ❑ Cost sensitivity (one system to manage)

Technologies: TiDB, CockroachDB, SingleStore, ClickHouse

Dual System Approach (OLTP + OLAP):

Use when:

❑ Checkpoint

1. ❑ Large data volumes (10TB+)

□ Checkpoint

1. □ Complex transformations and data modeling

□ Checkpoint

1. □ Strict consistency requirements for transactions

□ Checkpoint

1. □ Mature team with specialized skills

□ Checkpoint

1. □ Different SLA requirements for operational vs analytical workloads

Technologies: PostgreSQL + Snowflake, MySQL + BigQuery, MongoDB + Redshift

Real-World Architecture Patterns

Pattern 1: E-commerce with Real-time Recommendations

OLTP: PostgreSQL

- - Users, products, orders, inventory
- Handles 10k+ transactions/second
- ACID compliance for payment processing

OLAP: ClickHouse

- - Event streaming from PostgreSQL via Kafka

- Real-time recommendation engine queries
- Sub-second analytics for ML models

Integration:

- Change data capture (Debezium) → Kafka → ClickHouse
- Real-time sync with 100ms latency

□ In Plain English

Change Data Capture (CDC) is like a security camera on your database — it watches for any changes and streams them to other systems in real-time. It's how you keep multiple databases in sync without expensive full copies.

Pattern 2: SaaS Platform with Customer Analytics

HTAP: TiDB

- - Customer data, usage events, billing
- Operational queries for app functionality
- Real-time dashboards for customer success team
- Multi-tenant with row-level security

Why HTAP works here:

- - Data volume: ~1TB per tenant
- Real-time analytics requirement
- Small engineering team
- Cost-effective for SaaS margins

Pattern 3: Financial Services with Compliance

- OLTP: PostgreSQL (primary)
- Account management, transactions
- Strict ACID compliance

- Regulatory audit requirements
- OLTP: PostgreSQL (read replica)
- Reporting queries that don't need real-time data
- Offload read traffic from primary

OLAP: Snowflake

- - Historical analysis, risk modeling
- Regulatory reporting (daily/monthly batches)
- Data retention for 7+ years

Why separate systems:

- - Regulatory isolation requirements
- Different backup/retention policies
- Performance isolation (analytics can't impact transactions)

Performance Characteristics Comparison

Metric OLTP OLAP HTAP

Write

Very High Low-Medium High

Throughput

Seconds- Milliseconds-

Query Latency Milliseconds

Minutes Seconds

Concurrent Users Thousands Hundreds Hundreds

Data Volume GB-TB TB-PB GB-TB

Query Simple-

Simple Very Complex

Complexity Medium

Consistency ACID Eventual ACID

Cost per Query Low Medium Low-Medium

Migration Strategies

From OLTP to OLAP:

```
# Daily ETL pattern - PostgreSQL to Snowflake
import psycopg2
import snowflake.connector

def daily_etl():
    # Extract from PostgreSQL
    pg_conn =
```

PYTHON

```
psycopg2.connect("postgresql://user:pass@host:5432/db")
```

```

pg_cursor = pg_conn.cursor()

# Get yesterday's data
pg_cursor.execute("""
    SELECT order_id, customer_id, product_id, amount, order_date
    FROM orders
    WHERE order_date >= CURRENT_DATE - INTERVAL '1 day'
    AND order_date < CURRENT_DATE
""")

orders = pg_cursor.fetchall()

# Load to Snowflake
sf_conn = snowflake.connector.connect(
    user='user', password='pass',
    account='account', warehouse='ETL_WH', database='ANALYTICS'
)

sf_cursor = sf_conn.cursor()

# Bulk insert with transformation
for order in orders:
    sf_cursor.execute("""
        INSERT INTO fact_orders (order_id, customer_key,

```

product_key,

```

date_key, amount, load_date)
    SELECT %(order_id)s,
           c.customer_key,
           p.product_key,
           d.date_key,
           %(amount)s,
           CURRENT_DATE
    FROM dim_customer c, dim_product p, dim_date d
    WHERE c.customer_id = %(customer_id)s
    AND p.product_id = %(product_id)s
    AND d.calendar_date = %(order_date)s
""", {
    'order_id': order[0],
    'customer_id': order[1],
    'product_id': order[2],
    'amount': order[3],
    'order_date': order[4]
})

```

Real-time CDC Pattern:

```
# Debezium configuration for real-time sync
version: '3'
services:
  postgres:
    image: postgres:14
    environment:
      POSTGRES_PASSWORD: password
    command: >
      postgres
      -c wal_level=logical
```

```
-c max_replication_slots=4

  debezium:
    image: debezium/connect:1.9
    environment:
      BOOTSTRAP_SERVERS: kafka:9092
      CONFIG_STORAGE_TOPIC: connect_configs

  kafka:
    image: confluentinc/cp-kafka:7.0.1

# Debezium connector config
{
  "name": "postgres-connector",
  "config": {
    "connector.class":
```

```
"io.debezium.connector.postgresql.PostgresConnector",
```

```
"database.hostname": "postgres",
"database.port": "5432",
"database.user": "user",
"database.password": "password",
"database.dbname": "ecommerce",
"database.server.name": "dbserver1",
"table.include.list":
```

```
"public.orders,public.customers,public.products"
```

```
}  
}
```

Pro Tip: Start Simple, Add Complexity Begin with a single

PostgreSQL instance. Add read replicas for reporting. Only move to separate OLAP when you hit clear limitations: query performance, storage costs, or team conflicts between operational and analytical workloads.

⚠ Common Mistake

⚠ Common Mistake: Premature Optimization "We'll have big data someday" is not a reason to start with a complex OLTP+OLAP architecture. Many companies successfully run analytics on PostgreSQL with proper indexing and partitioning. Add complexity only when the benefits clearly outweigh the operational overhead.

□ Checkpoint

1. What are the main differences between OLTP and OLAP workloads in terms of query patterns?
2. When would you choose an HTAP system over separate OLTP and OLAP systems?
3. What's the trade-off between real-time analytics and system complexity?

Try It Yourself

Design storage architecture for these scenarios:

- Scenario A: Social media platform
- 1M active users posting 10M events/day
- Need real-time engagement metrics
- Historical analysis for content recommendation
- Limited engineering team (3 people)

Scenario B: Financial trading platform

- - 100k trades/second during peak hours
- Regulatory requirement: all trades auditable for 7 years
- Risk analytics on historical data
- Sub- millisecond latency for trade execution

Which would you choose: single HTAP system, dual OLTP/OLAP, or something else? Why?

2.2 Columnar Storage & Table Formats

⚡ TL;DR

Columnar formats (Parquet, ORC) provide 10x better compression and query performance for analytics - Modern table formats (Delta, Iceberg, Hudi) add ACID transactions and time travel to data lakes - Choose Parquet for simple analytics, Delta/Iceberg for production data platforms - Partitioning strategy matters more than file format for query performance

Row-based storage made sense when databases lived on spinning disks and queries touched complete records. In the cloud era, with NVMe SSDs and analytics workloads, columnar storage isn't just better—it's for any serious data platform.

Why Columnar Storage Dominates Analytics

Data Layout Comparison:

Row-based (CSV, JSON, traditional databases):

Record 1: [user_id=123, name="Alice", age=25, city="NYC", signup_date="2024-01-15"]

Record 2: [user_id=124, name="Bob", age=30, city="LA", signup_date="2024-01-16"]

Record 3: [user_id=125, name="Charlie", age=35, city="NYC", signup_date="2024-01-17"]

Columnar (Parquet, ORC): user_id column: [123, 124, 125,...] name column: ["Alice", "Bob", "Charlie",...] age column: [25, 30, 35,...] city column: ["NYC", "LA", "NYC",...] signup_date column: ["2024-01-15", "2024-01-16", "2024-01-17",...]

- Query: "Count users by city"
- Row-based: Read all columns of all records = 100% of data
- Columnar: Read only city column = ~5% of data

Real-world example:


```
-- Query: Monthly user signups by city
SELECT
    DATE_TRUNC('month', signup_date) as month,
    city,
    COUNT(*) as signup_count
FROM users
WHERE signup_date >= '2023-01-01'
GROUP BY month, city;

-- Performance on 100M user records:
-- CSV file: 45 seconds (reads all 20 columns)
-- Parquet file: 3 seconds (reads only signup_date and city columns)
```

Compression Benefits

Why Columnar Compresses Better:

Row-based data (random patterns): Row 1: ["Alice", 25, "NYC", "2024-01-15", "Premium",...] Row 2: ["Bob", 30, "LA", "2024-01-16", "Free",...] Row 3: ["Charlie", 35, "NYC", "2024-01-17", "Premium",...]

Columnar data (similar values together): city column: ["NYC", "NYC", "NYC", "LA", "LA", "Chicago", "Chicago",...] plan column: ["Premium", "Premium", "Premium", "Free", "Free", "Premium",...]

Compression Techniques:

- Dictionary encoding: "NYC" → 1, "LA" → 2, "Chicago" → 3
- Run-length encoding: [NYC × 1000, LA × 500, Chicago × 200]
- Delta encoding: [2024-01-15, +1 day, +1 day, +1 day, ...]
- Bit packing: Store small integers in fewer bits

Real compression ratios:

Raw data: 1TB CSV (row-based): 800GB (20% compression from gzip) Parquet (columnar): 150GB (85% compression from columnar + advanced encoding)

Parquet Deep Dive

File Structure:

Parquet File

└─ File Metadata ─┬─ Row Group 1 (64-128MB) │ └─ Column Chunk 1 (user_id) │ │ └─
Page 1 (1MB) │ │ └─ Page 2 (1MB) │ │ └─... │ └─ Column Chunk 2 (name) │ └─... └─
Row Group 2 └─...

Predicate Pushdown:

```
import pandas as pd

# Query with filter
df = pd.read_parquet('users.parquet',
                    columns=['name', 'city'], # Column pruning
                    filters=[('signup_date', '>=', '2024-01-01')])
```

PYTHON

Predicate pushdown

```
# Parquet statistics enable skipping entire row groups:
# Row Group 1: signup_date min=2023-01-01, max=2023-12-31 → SKIP
# Row Group 2: signup_date min=2024-01-01, max=2024-06-30 → READ
# Row Group 3: signup_date min=2024-07-01, max=2024-12-31 → READ
```

Schema Evolution:

```
# Original schema
original_schema = pa.schema([
    ('user_id', pa.int64()),
    ('name', pa.string()),
    ('email', pa.string())
])

# Add new column (backward compatible)
new_schema = pa.schema([
    ('user_id', pa.int64()),
    ('name', pa.string()),
    ('email', pa.string()),
    ('signup_date', pa.timestamp('s')) # New column
])

# Old readers can still read the file (ignoring new columns)
# New readers get NULL values for signup_date in old files
```

Modern Table Formats: Delta Lake, Iceberg, Hudi

The Problem with Plain Parquet:

```
# Data lake with plain Parquet files
s3://data-lake/users/
├─ year=2023/month=01/part-001.parquet
├─ year=2023/month=01/part-002.parquet
├─ year=2023/month=02/part-001.parquet
└─ ...

# Problems:
# 1. No ACID transactions - readers see partial writes
# 2. No schema evolution - adding columns breaks old readers
```

```
# 3. No time travel - can't query historical versions
# 4. Small file problem - streaming creates thousands of tiny files
# 5. No upserts/deletes - Parquet is append-only
```

Delta Lake Solution:

```
from delta import DeltaTable
import pyspark.sql.functions as F

# Create Delta table
df.write.format("delta").save("s3://data-lake/users_delta/")

# ACID transactions
delta_table = DeltaTable.forPath(spark, "s3://data-
```

PYTHON

lake/users_delta/")

```
# UPDATE operation (impossible with plain Parquet)
delta_table.update(
    condition=F.col("city") == "New York",
    set={"city": "'NYC'"} # Standardize city names
)

# DELETE operation
delta_table.delete(condition=F.col("signup_date") < "2022-01-01")

# MERGE operation (upsert)
new_users = spark.createDataFrame([
    (126, "Diana", "diana@example.com", "Chicago", "2024-03-15"),
    (123, "Alice Updated", "alice@newemail.com", "NYC", "2024-01-
```

15") # Update existing

```
], ["user_id", "name", "email", "city", "signup_date"])

delta_table.alias("target").merge(
    new_users.alias("source"),
    "target.user_id = source.user_id"
).whenMatchedUpdateAll() \
  .whenNotMatchedInsertAll() \
  .execute()
```

Time Travel:

```
# Query historical versions
df_v1 = spark.read.format("delta") \
    .option("versionAsOf", 1) \
    .load("s3://data-lake/users_delta/")

df_yesterday = spark.read.format("delta") \
    .option("timestampAsOf", "2024-03-14") \
    .load("s3://data-lake/users_delta/")

# View change history
delta_table.history().show()
```

Schema Evolution:

```
# Add new column safely
new_data = spark.createDataFrame([
    (127, "Eve", "eve@example.com", "Boston", "2024-03-16",
```

```
"Premium")
```

```
], ["user_id", "name", "email", "city", "signup_date", "plan_type"])

new_data.write.format("delta") \
    .mode("append") \
    .option("mergeSchema", "true") \
    .save("s3://data-lake/users_delta/")
```

Apache Iceberg: Engine-Agnostic Table Format

Multi-Engine Support:

```
-- Same table, different query engines
-- Spark SQL
SELECT city, COUNT(*)
```

The corresponding SQL query:

```

FROM iceberg.db.users
GROUP BY city;

-- Presto/Trino
SELECT city, COUNT(*)
FROM iceberg.db.users
GROUP BY city;

-- Flink (streaming)
CREATE TABLE users_iceberg (
    user_id BIGINT,
    name STRING,
    city STRING
) WITH (
    'connector' = 'iceberg',
    'catalog-name' = 'hadoop',
    'catalog-type' = 'hadoop',
    'warehouse' = 's3://data-lake/'
);

```

Advanced Partitioning:

```

# Iceberg hidden partitioning
iceberg_table = spark.catalog.createTable(
    "users_iceberg",
    schema=schema,
    # Partition by month, but users don't specify it in queries
    partitions=["month(signup_date)"]
)

# Queries automatically use partition pruning
spark.sql("SELECT * FROM users_iceberg WHERE signup_date = '2024-03-

```

15")

```

# Automatically scans only March 2024 partition

```

Evolution Operations:

```
-- Iceberg supports schema evolution via SQL
ALTER TABLE users_iceberg ADD COLUMN last_login_date DATE;
ALTER TABLE users_iceberg DROP COLUMN old_column;
ALTER TABLE users_iceberg RENAME COLUMN email TO email_address;

-- Partition evolution (change partitioning without rewriting data)
ALTER TABLE users_iceberg SET PARTITION SPEC (month(signup_date),
```

city);

Apache Hudi: Real-time Data Lakes

Incremental Processing:

```
# Hudi Copy-on-Write (optimized for read performance)
hudi_options = {
    'hoodie.table.name': 'users_hudi',
    'hoodie.datasource.write.recordkey.field': 'user_id',
    'hoodie.datasource.write.precombine.field': 'signup_date',
    'hoodie.datasource.write.table.type': 'COPY_ON_WRITE'
}

df.write.format("hudi") \
    .options(**hudi_options) \
    .mode("overwrite") \
    .save("s3://data-lake/users_hudi/")

# Incremental reads (only changed data since last query)
incremental_df = spark.read.format("hudi") \
    .option("hoodie.datasource.query.type", "incremental") \
    .option("hoodie.datasource.read.begin.instanttime",
```

"20240315000000") \

```
.load("s3://data-lake/users_hudi/")
```

Merge-on-Read for Streaming:

```
# Hudi Merge-on-Read (optimized for write performance)
```

```

mor_options = {
    'hoodie.table.name': 'events_hudi',
    'hoodie.datasource.write.table.type': 'MERGE_ON_READ',
    'hoodie.compact.inline': 'false', # Compact asynchronously
    'hoodie.deltastreamer.source.kafka.topic': 'user-events'
}

# Streaming writes
events_stream.writeStream \
    .format("hudi") \
    .options(**mor_options) \
    .outputMode("append") \
    .option("checkpointLocation", "s3://checkpoints/events/") \
    .start("s3://data-lake/events_hudi/")

```

Format Comparison Matrix

Delta

Feature Parquet Iceberg Hudi

Lake

ACID

□ □ □ □

Transactions Time Travel □ □ □ □ Schema

Limited □ □ □

Evolution Upserts/Deletes □ □ □ □

Spark- Spark-

Multi-Engine □ □

focused

focused

Streaming

▣

Good

Good

Excellent

Ingestion Query Very

Very Good Good

Performance Good

Very

Maturity High Medium Medium

High

Ecosystem Universal Databricks Apache Apache

Partitioning Strategies for Performance

Date-based Partitioning:

```
# Physical partitioning (directory structure)
df.write.format("delta") \
    .partitionBy("year", "month", "day") \
    .save("s3://data-lake/events/")

# Directory structure:
# s3://data-lake/events/
# └─ year=2024/month=03/day=15/part-001.parquet
# └─ year=2024/month=03/day=16/part-001.parquet
# └─ ...

# Query with partition pruning
spark.sql("""
    SELECT event_type, COUNT(*)
    FROM events
    WHERE year = 2024 AND month = 3 AND day = 15
    GROUP BY event_type
""")
# Scans only 1 day's data instead of entire table
```

Hash Partitioning for Even Distribution:

```
# Avoid hotspots with hash partitioning
df.withColumn("partition_key", F.hash("user_id") % 100) \
    .write.format("delta") \
    .partitionBy("partition_key", "date") \
    .save("s3://data-lake/user_events/")
```

Multi-dimensional Partitioning:

```
-- Iceberg supports multiple partition transforms
CREATE TABLE events (
    user_id BIGINT,
    event_type STRING,
    timestamp TIMESTAMP,
    properties STRING
) USING ICEBERG
PARTITIONED BY (
    days(timestamp),
    bucket(16, user_id), -- Hash partitioning
    event_type
);
```

File Size Optimization

The Goldilocks Problem:

Too small files (< 10MB):

- High metadata overhead
- Slow query planning
- Inefficient compression

Too large files (> 1GB):

- Slow parallel processing
- Memory pressure
- All-or-nothing reads

Just right (64-256MB):

- Optimal compression
- Good parallelism
- Efficient metadata

Auto-optimization:

```
# Delta Lake auto-optimization
spark.sql("""
    OPTIMIZE users_delta
    ZORDER BY (city, signup_date)
    """)

# Vacuum old files (data retention)
spark.sql("VACUUM users_delta RETAIN 168 HOURS")    # Keep 7 days
```

SQL

Compaction for Streaming Workloads:

```
# Hudi async compaction
compaction_options = {
    'hoodie.compact.inline': 'false',
    'hoodie.compact.inline.max.delta.commits': '5',
    'hoodie.compaction.strategy':
```

```
'org.apache.hudi.table.action.compact.strategy.LogFileSizeBasedCompactionStrategy'
```

```
}
```

Cost Optimization Strategies

Storage Tier Strategy:

```
# Lifecycle policies for cost optimization
lifecycle_config = {
    'Rules': [
        {
            'Status': 'Enabled',
            'Transitions': [
                {
                    'Days': 30,
                    'StorageClass': 'STANDARD_IA'    # Infrequent
```

Access

```
},
{
```

```
    'Days': 90,
    'StorageClass': 'GLACIER'        # Archive
},
{
    'Days': 365,
    'StorageClass': 'DEEP_ARCHIVE'   # Long-term
```

archive

```
}  
    ]  
  }  
]  
}
```

Query Cost Optimization:

```
-- Avoid scanning unnecessary data  
SELECT event_type, COUNT(*)  
FROM events  
WHERE date_partition = '2024-03-15' -- Use partition pruning  
      AND event_type IN ('purchase', 'signup') -- Selective filters  
GROUP BY event_type;  
  
-- vs inefficient query  
SELECT event_type, COUNT(*)  
FROM events  
WHERE timestamp::DATE = '2024-03-15' -- Forces full scan  
GROUP BY event_type;
```

SQL

Pro Tip: Compression vs Query Speed More compression =

lower storage costs but higher CPU costs. For frequently queried data, optimize for query speed. For archival data, optimize for compression. Many systems let you choose compression algorithms per table.

⚠ Common Mistake

⚠ Common Mistake: Over-partitioning Don't create partitions with < 1GB of data. Small partitions create more overhead than benefit. A table with 1000 daily partitions of 10MB each performs worse than 10 partitions of 1GB each.

□ Checkpoint

1. Why does columnar storage provide better query performance for analytics workloads?
2. What problems do modern table formats (Delta/Iceberg/Hudi) solve that plain Parquet doesn't?
3. When would you choose Delta Lake vs Apache Iceberg?

Try It Yourself

Design storage strategy for these scenarios:

- Scenario A: IoT sensor data
- 1 billion events per day
- Mix of batch analytics and real-time alerts
- 7-year retention requirement
- Need to update sensor metadata regularly
- Scenario B: Financial trading data
- Immutable trade records (no updates/deletes needed)
- Extremely high query performance requirements
- Complex time-series analysis
- Regulatory compliance (audit trails)

What table format would you choose for each? How would you partition the data?

⚡ TL;DR

Object storage (S3, GCS, Azure Blob) is the foundation of modern data lakes - Design your bucket/prefix structure for query performance, not just organization - Lifecycle policies save 80%+ on storage costs for inactive data - Multi-region and cross- cloud strategies require careful planning for data sovereignty

Object storage revolutionized data architecture. Before S3, storing petabytes required expensive SAN arrays and full-time storage administrators. Now you upload files to a bucket and pay per-byte. But the simplicity is deceiving - object storage design patterns make or break data platform performance and costs.

Object Storage as Data Lake Foundation

Why Object Storage Won:

Traditional NAS/SAN:

- Expensive: \$10,000+ per TB
- Limited: Scale to hundreds of TB
- Complex: Storage administrators, hardware refresh cycles
- Single point of failure

Object Storage:

- Cheap: \$0.023/GB/month (S3 Standard)
- Unlimited: Exabytes and beyond
- Simple: API-driven, no hardware management
- 99.999999999% durability (11 9's)

Data Lake Architecture Pattern: s3://company-data-lake/ | — raw/ # Landing zone - raw data as-is | | — source=salesforce/ | | — source=api_logs/ | | — source=kafka_events/ | — processed/ # Cleaned and standardized | | — events/ | | — customers/ | | —

```
products/ |— curated/ # Business-ready datasets | |— fact_sales/ | |— dim_customer/ |
|— daily_aggregates/ |— archive/ # Long-term storage

|— year=2020/
|— year=2021/
|— ...
```

Path Design for Query Performance

The Insight: Paths Become Filters

Most analytics queries include date filters. Design your path structure to enable partition pruning:

Good Path Structure:

```
s3://data-lake/events/
|— year=2024/month=03/day=15/hour=14/
|   |— part-00000.parquet
|   |— part-00001.parquet
|   |— part-00002.parquet
|— year=2024/month=03/day=15/hour=15/
|   |— part-00000.parquet
|   |— part-00001.parquet

# Query: Events from March 15th, 2024
# Scans only: year=2024/month=03/day=15/ paths
# Skips: All other dates (99.9% of data)
```

Bad Path Structure:

```
s3://data-lake/events/
|— kafka_partition_0/
|   |— 2024-03-15-14-00-events.parquet
|   |— 2024-03-15-15-00-events.parquet
|   |— 2024-03-16-14-00-events.parquet
|— kafka_partition_1/
```



```
|— 2024-03-15-14-00-events.parquet
  |— 2024-03-16-14-00-events.parquet
```

```
# Query: Events from March 15th, 2024
# Must scan: ALL paths (can't determine dates from path structure)
# Reads: 100% of data to find 1 day's worth
```

Multi-dimensional Partitioning:

```
# When you have multiple common filter patterns
s3://data-lake/sales/
|— region=us-west/year=2024/month=03/
|— region=us-east/year=2024/month=03/
|— region=europe/year=2024/month=03/
|— region=asia/year=2024/month=03/

# Supports efficient queries by:
# - Date range: WHERE year = 2024 AND month = 3
# - Region: WHERE region = 'us-west'
# - Both: WHERE region = 'us-west' AND year = 2024
```

Lifecycle Management & Cost Optimization

Storage Class Strategy:

```
# S3 storage classes and use cases
storage_strategy = {
  "hot_data": {
    "class": "S3 Standard",
    "cost_per_gb": "$0.023",
    "use_case": "Frequently accessed (daily/weekly queries)",
    "retrieval_time": "milliseconds"
  },
  "warm_data": {
    "class": "S3 Standard-IA",
    "cost_per_gb": "$0.0125",
    "use_case": "Monthly reports, ad-hoc analysis",
    "retrieval_time": "milliseconds",
    "minimum_storage": "30 days"
  },
  "cold_data": {
    "class": "S3 Glacier Flexible",
    "cost_per_gb": "$0.0036",
    "use_case": "Compliance, audit, backup",
    "retrieval_time": "1-5 minutes",
    "minimum_storage": "90 days"
  },
  "archive_data": {
    "class": "S3 Glacier Deep Archive",
    "cost_per_gb": "$0.00099",
    "use_case": "Long-term legal retention",
    "retrieval_time": "12+ hours",
    "minimum_storage": "180 days"
  }
}
```

Automated Lifecycle Policy:

```

{
  "Rules": [
    {
      "ID": "data-lake-lifecycle",
      "Status": "Enabled",
      "Filter": {
        "Prefix": "raw/"
      },
      "Transitions": [
        {
          "Days": 30,
          "StorageClass": "STANDARD_IA"
        },
        {
          "Days": 90,
          "StorageClass": "GLACIER"
        },
        {
          "Days": 365,
          "StorageClass": "DEEP_ARCHIVE"
        }
      ]
    },
    {
      "ID": "processed-data-lifecycle",
      "Filter": {
        "Prefix": "processed/"
      },
      "Transitions": [
        {
          "Days": 60,
          "StorageClass": "STANDARD_IA"
        },
        {
          "Days": 180,
          "StorageClass": "GLACIER"
        }
      ]
    }
  ]
}

```

Cost Impact Example:

Dataset: 100TB over 3 years Query pattern: Last 30 days frequently, older data rarely

Without lifecycle management: $100\text{TB} \times \$23/\text{TB}/\text{month} \times 36 \text{ months} = \$82,800$

With lifecycle management:

- Month 1-30: $100\text{TB} \times \$23/\text{TB} = \$2,300$
- Month 31-90: $100\text{TB} \times \$12.5/\text{TB} = \$1,250$
- Month 91+: $100\text{TB} \times \$3.6/\text{TB} = \360 Total 3-year cost: \$12,600 (85% savings)

Data Organization Best Practices

Hive-style Partitioning:

```
# Standard format understood by most analytics engines
s3://bucket/table_name/
├─ partition_col1=value1/partition_col2=value2/
├─ partition_col1=value1/partition_col2=value3/
└─ partition_col1=value2/partition_col2=value1/

# Example: sales data
s3://company-data/sales/
├─ year=2024/month=03/region=us-west/
├─ year=2024/month=03/region=us-east/
├─ year=2024/month=03/region=europe/
└─ year=2024/month=04/region=us-west/
```

File Naming Conventions:

```
# Include metadata in filenames for debugging
s3://data-lake/processed/events/year=2024/month=03/day=15/
├─ spark-job-20240315-143022-part-00000.parquet
├─ spark-job-20240315-143022-part-00001.parquet
└─ spark-job-20240315-143022-part-00002.parquet

# Filename components:
# - spark-job: Source application
# - 20240315-143022: Timestamp (YYYYMMDD-HHMMSS)
# - part-00000: Part number for deduplication
```

Environment Separation:

```
# Separate buckets or clear prefixes for environments
s3://company-data-dev/
s3://company-data-staging/
s3://company-data-prod/

# Or within single bucket:
s3://company-data/
├─ dev/
├─ staging/
└─ prod/
```

Multi-Region Strategies

Cross-Region Replication (CRR):

```
{
  "Role": "arn:aws:iam::account:role/replication-role",
  "Rules": [
    {
      "ID": "disaster-recovery",
      "Status": "Enabled",
      "Filter": {
        "Prefix": "critical-data/"
      },
      "Destination": {
        "Bucket": "arn:aws:s3::company-data-backup-us-west-
```

JSON

2",

```
    "StorageClass": "STANDARD_IA"
  }
}
]
```

Multi-Region Architecture:

Primary Region (us-east-1): ── Production data lake ── Real-time analytics ──
Customer-facing APIs

Secondary Region (us-west-2): |— Disaster recovery backup |— Batch processing (lower costs) |— Development/staging environments

International Region (eu-west-1): |— GDPR-compliant data for EU users |— Regional analytics |— Localized reporting

Security & Access Patterns

Bucket Policies for Data Governance:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AnalystsReadOnlyCurated",
      "Effect": "Allow",
      "Principal": {"AWS":
```

JSON

"arn:aws:iam::account:role/analysts"}],

```
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": [
      "arn:aws:s3:::data-lake/curated/*",
      "arn:aws:s3:::data-lake"
    ]
  },
  {
    "Sid": "DataEngineersFullRaw",
```

```
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::account:role/data-
```

engineers"}],

```

"Action": "s3:*",
    "Resource": [
        "arn:aws:s3:::data-lake/raw/*",
        "arn:aws:s3:::data-lake/processed/*"
    ]
},
{
    "Sid": "DenyDirectDeleteCurated",
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:DeleteObject",
    "Resource": "arn:aws:s3:::data-lake/curated/*"
}
]
}

```

VPC Endpoints for Private Access:

```

# Access S3 privately without internet gateway
vpc_endpoint_config = {
    "VpcEndpointType": "Gateway",
    "VpcId": "vpc-12345678",
    "ServiceName": "com.amazonaws.us-east-1.s3",
    "RouteTableIds": ["rtb-12345678"]
}

# Benefits:
# - No data transfer charges for same-region access
# - Traffic stays within AWS network
# - Enhanced security (no internet exposure)

```

When Object Storage Isn't the Answer

Anti-patterns:

- ☐ High-frequency small updates
- Problem: Each update creates new object
- Solution: Use database for transactional data
- ☐ Strong consistency requirements
- Problem: Eventually consistent (though S3 is now strongly consistent)

- Solution: Use database for critical consistency needs
- ❌ Complex queries with joins
- Problem: Object storage doesn't understand relationships
- Solution: Use data warehouse for complex analytics
- ❌ Low-latency random access
- Problem: Network latency + object overhead
- Solution: Use in-memory cache or database

Hybrid Architectures:

Real-time Path: API → Database → Cache → Application (PostgreSQL + Redis for sub-second queries)

Batch Path: API → Message Queue → Object Storage → Warehouse (Kafka + S3 + Snowflake for analytical queries)

Performance Optimization

Request Rate Optimization:

```
# Distribute load across prefixes to avoid hotspots
import random
import hashlib
```

PYTHON

The generate_balanced_prefix implementation:


```
def generate_balanced_prefix(key, prefix_count=100):
    """Generate prefix that distributes load evenly."""
    hash_value = hashlib.md5(key.encode()).hexdigest()[:2]
    return f"{hash_value}/{key}"

# Instead of:
# s3://bucket/2024/03/15/file1.parquet
# s3://bucket/2024/03/15/file2.parquet    # Hot partition

# Use:
# s3://bucket/a1/2024/03/15/file1.parquet
# s3://bucket/b2/2024/03/15/file2.parquet    # Distributed load
```

Multipart Upload for Large Files:

```
import boto3

def upload_large_file(file_path, bucket, key):
    """Upload large files efficiently with multipart."""
    s3_client = boto3.client('s3')

    # Configure multipart upload
    config = boto3.s3.transfer.TransferConfig(
        multipart_threshold=1024 * 25, # 25MB
        max_concurrency=10,
        multipart_chunksize=1024 * 25,
        use_threads=True
    )

    s3_client.upload_file(
        file_path, bucket, key,
        Config=config
    )
```

Intelligent Tiering:

```
{
  "Id": "intelligent-tiering",
  "Status": "Enabled",
  "Filter": {
    "Prefix": "analytics/"
  },
  "Configurations": [
    {
      "Id": "archive-config",
      "Status": "Enabled",
      "Filter": {
        "Prefix": "analytics/historical/"
      },
      "Tiering": {
        "Days": 90,
        "AccessTier": "ARCHIVE_ACCESS"
      }
    }
  ]
}
```

Cross-Cloud Object Storage

Multi-cloud Strategy:

```
# Abstract storage interface for cloud portability
class CloudStorage:
    def __init__(self, provider, config):
        self.provider = provider
        if provider == "aws":
            self.client = boto3.client('s3')
        elif provider == "gcp":
            self.client = storage.Client()
        elif provider == "azure":
            self.client = BlobServiceClient()
```

The upload implementation:

```
def upload(self, local_path, remote_path):
    if self.provider == "aws":
        return self._upload_s3(local_path, remote_path)
    elif self.provider == "gcp":
        return self._upload_gcs(local_path, remote_path)
    # ... etc
```

Data Transfer Strategies:

```
# AWS DataSync for large migrations
aws datasync create-task \
    --source-location-arn
```

"arn:aws:datasync:region:account:location/loc-source" \

```
--destination-location-arn
```

"arn:aws:datasync:region:account:location/loc-dest"

```
# Cross-cloud replication
gsutil -m rsync -r -d gs://source-bucket s3://destination-bucket

# Bandwidth optimization
aws configure set max_bandwidth 1GB/s
aws configure set max_concurrent_requests 10
```

Pro Tip: Design for Query Patterns Don't just dump files in

object storage. Design your prefix structure based on how the data will be queried. The most common filter in your queries should be the first level of partitioning.

⚠ Common Mistake

⚠ Common Mistake: Over-partitioning Creating too many small partitions (< 100MB each) hurts performance more than it helps. Better to have fewer, larger partitions that still enable meaningful filtering.

□ Checkpoint

1. Why is path structure crucial for query performance in object storage?
2. How can lifecycle policies reduce storage costs by 80%+?
3. When would you use cross-region replication vs intelligent tiering?

Try It Yourself

Design object storage strategy for:

- Scenario: Media streaming platform
- Video files (GB each) uploaded daily
- Metadata (JSON, KB each) queried frequently
- Popular content accessed often, older content rarely
- Global audience requires low- latency access
- 7-year retention for legal compliance

Design: 1. Bucket/prefix structure 2. Lifecycle policies 3. Multi-region strategy 4. Security model

Module 2 Project: Design Storage

Architecture for Multi-Tenant SaaS

Platform

Objective

Design a comprehensive storage architecture for a multi-tenant SaaS platform that handles both operational and analytical workloads. You'll make technology choices, design data models, and plan for scale while balancing performance, cost, and operational complexity.

Prerequisites Understanding of OLTP vs OLAP trade-offs

Knowledge of table formats and object storage patterns

Basic familiarity with cloud storage services

Understanding of data partitioning concepts

Expected Time

4-6 hours

Problem Statement

"DataCorp Analytics" is a B2B SaaS platform that helps companies analyze their customer data. The platform serves 1,000+ tenants (companies), each with their own isolated datasets and analytics dashboards.

Business Requirements

Operational Workloads: - User management and authentication

- (1M+ users across all tenants)
- Real-time dashboard queries (sub- second response times)
- Data ingestion via APIs and file uploads (10GB-1TB per tenant per month)
- Tenant configuration and billing management

Analytical Workloads: - Complex customer segmentation queries

- (minutes acceptable)
- Cross-tenant analytics for platform metrics (internal use only)
- Historical trend analysis (up to 3 years of data)
- Export capabilities (CSV, API) for large datasets

Scale Requirements:

- Current: 1,000 tenants, 10TB total data
- 2- year projection: 5,000 tenants, 100TB total data
- Largest tenants: 1TB of data each
- Smallest tenants: 100MB of data each
- Query load: 10,000 queries/day across all tenants

Compliance & Security:

- Strict tenant data isolation (one tenant cannot see another's data)
- GDPR compliance (right to be forgotten, data export)
- SOC 2 requirements (audit logs, encryption, access controls)
- Data retention: 3 years for analytics, 7 years for billing

Step 1

Storage Technology Selection

For each workload, choose appropriate storage technology and justify your decision:

Operational Data Storage:

Options to consider:

- Single PostgreSQL instance with row-level security
- PostgreSQL instance per tenant
- Multi-tenant NoSQL (MongoDB, DynamoDB)

- Hybrid approach (PostgreSQL + Redis cache)

Consider:

- Tenant isolation requirements
- Operational complexity
- Query performance
- Cost at scale

Analytical Data Storage:

Options to consider:

- Shared data warehouse (Snowflake, BigQuery)
- Per-tenant data marts
- Data lake with query engine (S3 + Athena/Presto)
- Hybrid lakehouse (Delta Lake, Iceberg)

Consider:

- Cross-tenant analytics needs
- Query performance requirements
- Storage costs
- Schema flexibility

Step 2: Data Architecture Design Design schemas for both operational and analytical workloads:

Operational Schema Design:

- Design tables for:
 - 1. Multi-tenant user and organization management
 - 2. Data ingestion tracking and status
 - 3. Query execution logs and performance metrics
 - 4. Billing and usage tracking
- Consider:
 - - How do you enforce tenant isolation?
 - - What indexes optimize for multi-tenant queries?
 - - How do you handle tenant-specific configuration?

Analytical Schema Design:

- Design dimensional model for:
 - 1. Customer data (tenant's customers, not our tenants)
 - 2. Platform usage analytics (our internal metrics)
 - 3. Cross-tenant benchmarking (aggregated/anonymized)
 - 4. Historical trend analysis
- Consider:
 - - Grain of fact tables in multi-tenant environment
 - - How to handle different tenant schemas
 - - Performance optimization for largest tenants

Step 3: Multi-Tenant Isolation Strategy

Choose and implement a tenant isolation model:

Option A: Shared Schema with Tenant ID

```
CREATE TABLE customer_data (  
    tenant_id UUID NOT NULL,  
    customer_id BIGINT NOT NULL,  
    customer_data JSONB,  
    created_at TIMESTAMP,  
    PRIMARY KEY (tenant_id, customer_id)  
);  
  
-- Row-level security  
CREATE POLICY tenant_isolation ON customer_data  
    FOR ALL TO application_role  
    USING (tenant_id = get_current_tenant_id());
```

SQL

Option B: Schema per Tenant

```
-- Tenant ABC schema  
CREATE SCHEMA tenant_abc;  
CREATE TABLE tenant_abc.customer_data (...);  
  
-- Tenant XYZ schema  
CREATE SCHEMA tenant_xyz;  
CREATE TABLE tenant_xyz.customer_data (...);
```

SQL

Option C: Database per Tenant

```
-- Separate database for each tenant  
-- Database: tenant_abc_prod  
-- Database: tenant_xyz_prod
```

Step 4: Object Storage Design

Design object storage strategy for raw data and analytics:

Bucket Structure:

```
# Design prefix strategy for:
```

```
s3://datacorp-platform/  
├─ raw-data/  
│   └─ tenant_id=abc-123/year=2024/month=03/day=15/  
├─ processed-data/  
│   └─ tenant_id=abc-123/table=customers/year=2024/month=03/  
├─ analytics-exports/  
├─ platform-analytics/  
└─ backups/
```

```
# Consider:
```

- # - Query performance optimization
- # - Cost optimization with lifecycle policies
- # - Security and access controls
- # - Cross-tenant analytics requirements

Table Format Selection:

```
# Choose table format and justify:
```

- # - Plain Parquet
- # - Delta Lake
- # - Apache Iceberg
- # - Apache Hudi

```
# Consider tenant-specific requirements:
```

- # - Schema evolution (tenants define custom fields)
- # - GDPR deletion requirements
- # - Time travel for auditing
- # - Streaming vs batch ingestion patterns

Step 5: Data Pipeline Architecture

Design ETL/ELT processes for the multi-tenant environment:

```
# Design pipeline for:
# 1. Tenant data ingestion (API uploads, file imports)
# 2. Real-time operational data sync
# 3. Batch analytical data processing
# 4. Cross-tenant aggregations
# 5. Data quality and validation

# Consider:
# - How do you process data from 5,000 tenants efficiently?
# - How do you handle schema differences between tenants?
# - How do you ensure tenant isolation in processing?
# - How do you handle failures and retries?
```

Step 6: Performance & Cost Optimization

Design optimization strategies:

Query Performance:

```
-- Design caching strategy for:
-- - Frequently accessed dashboards
-- - Common analytical queries
-- - Cross-tenant benchmarks

-- Partitioning strategy for:
-- - Large tenant data (>100GB)
-- - Time-series data
-- - Multi-dimensional access patterns
```

Cost Management:

```
# Design cost optimization for:
# - Storage lifecycle management
# - Compute resource auto-scaling
# - Query result caching
# - Cold data archival

# Cost allocation model:
# - How do you track costs per tenant?
```

```
# - How do you handle shared infrastructure costs?
# - What's your pricing model for storage vs compute?
```

Deliverables

1. Technology Selection Document (technology-choices.md)

1. Technology Selection Document (technology-choices.md)

```
## Operational Storage: [Technology]
**Decision:** [PostgreSQL/MongoDB/etc.]
**Reasoning:** [Performance, cost, complexity trade-offs]
**Alternatives Considered:** [Other options and why rejected]

## Analytical Storage: [Technology]
**Decision:** [Snowflake/BigQuery/Data Lake/etc.]
**Reasoning:** [Query performance, cost, flexibility]
**Alternatives Considered:** [Other options and why rejected]
```

2. Schema Definitions (schemas.sql)

```
-- Operational schema with tenant isolation
-- Analytical dimensional model
-- Index definitions and performance optimizations
-- Security policies and access controls
```

3. Object Storage Design (storage-design.md)

```
# Bucket/prefix structure
# Table format selection and configuration
# Lifecycle policies
# Security and access patterns
```

SQL

4. Multi-Tenant Architecture (multi-tenant-design.md)

```
## Isolation Strategy: [Shared/Schema/Database per tenant]
**Trade-offs:** [Security vs operational complexity]
**Scaling considerations:** [How does this work with 5,000 tenants?]
**GDPR compliance:** [Data deletion and export strategies]
```

BASH

5. Performance Plan (performance-optimization.md)

```
## Query Performance
- Partitioning strategies
- Caching approaches
- Index optimization

## Cost Optimization
- Storage lifecycle policies
- Compute scaling strategies
- Cost allocation model
```

Evaluation Criteria

- Technology Selection (25 points)
 - ☐ Clear justification for each technology choice
 - ☐ Understanding of trade-offs (performance vs complexity vs cost)
 - ☐ Consideration of multi-tenant specific requirements
- Schema Design (25 points)
 - ☐ Proper multi-tenant isolation implementation
 - ☐ Performance optimization (indexing, partitioning) -

☐ Checkpoint

1. ☐ Support for both operational and analytical workloads - ☐

Compliance requirements addressed

- Scalability Design (25 points)
 - ☐ Architecture supports 5x growth (1K to 5K tenants)
 - ☐ Query performance maintained as data grows -

□ Checkpoint

1. □ Cost scaling is predictable and manageable

- Operational Considerations (25 points)
- □ Backup and disaster recovery strategy
- □ Monitoring and observability plan
- □ Security and compliance implementation
- □ Realistic assessment of operational complexity

Challenge Questions

Consider these edge cases in your design:

1. Tenant Migration: A large tenant (500GB) wants to upgrade to a dedicated instance. How do you migrate their data with minimal downtime?
2. GDPR Deletion: A tenant requests deletion of all data for a specific customer. How do you ensure complete deletion across operational and analytical systems?
3. Schema Evolution: Tenant A wants to add custom fields to their customer data. How does this work with your analytical pipeline?
4. Query Performance: Your largest tenant's dashboard queries are timing out. How do you optimize without affecting other tenants?
5. Cost Attribution: How do you accurately allocate shared infrastructure costs (networking, compute, storage) to individual tenants for billing purposes?

Stretch Goals

1. Real-time Analytics: Design streaming pipeline for real-time dashboard updates
2. Global Deployment: Plan multi-region deployment for international tenants
3. Disaster Recovery: Design cross-region backup and failover strategy
4. Auto-scaling: Design automatic resource scaling based on tenant usage patterns

Solution Validation

Test your design with these scenarios:

Load Test:

- 1,000 concurrent dashboard queries across 500 tenants
- 50 simultaneous data uploads
- Complex analytical query on largest tenant (1TB data)

Failure Test: - Primary database fails during peak usage - Object storage temporarily unavailable - ETL pipeline fails for one tenant

Growth Test:

- Add 100 new tenants in one month
- Largest tenant grows from 100GB to 1TB
- Query volume doubles year-over-year

Your architecture should handle these scenarios gracefully while maintaining performance and cost efficiency.# Module 3: Batch Processing Architecture

What you'll learn

Master the design patterns for building reliable, scalable batch data pipelines. You'll understand when to use batch vs stream processing, how to orchestrate complex workflows, and how to build systems that handle petabytes of data reliably and cost-effectively.

03

Batch Processing & ETL

Master batch processing with Spark, ETL/ELT design patterns, workflow orchestration with Airflow, data quality, and cost optimization.

< TL;DR

Batch processing trades latency for throughput - for large-scale analytics and ETL - Lambda architecture combines batch + stream, but adds complexity - Kappa architecture is stream-only, but requires more sophisticated streaming infrastructure - Choose based on your latency requirements, not industry hype - Most problems can be solved with well-designed batch processing

Why Batch Processing Still Matters

In 2026, everyone talks about real-time everything. Stream processing. Event-driven architectures. Real-time analytics. But here's the truth: most data problems don't need real-time solutions. They need reliable, cost-effective, scalable solutions. And for those, batch processing often wins.

Batch processing excels when:

- You need to process large volumes of data efficiently
- Latency requirements are measured in hours, not seconds
- Cost optimization is important (batch is 5-10x cheaper than streaming)
- You need strong consistency guarantees
- Complex analytics require full dataset context

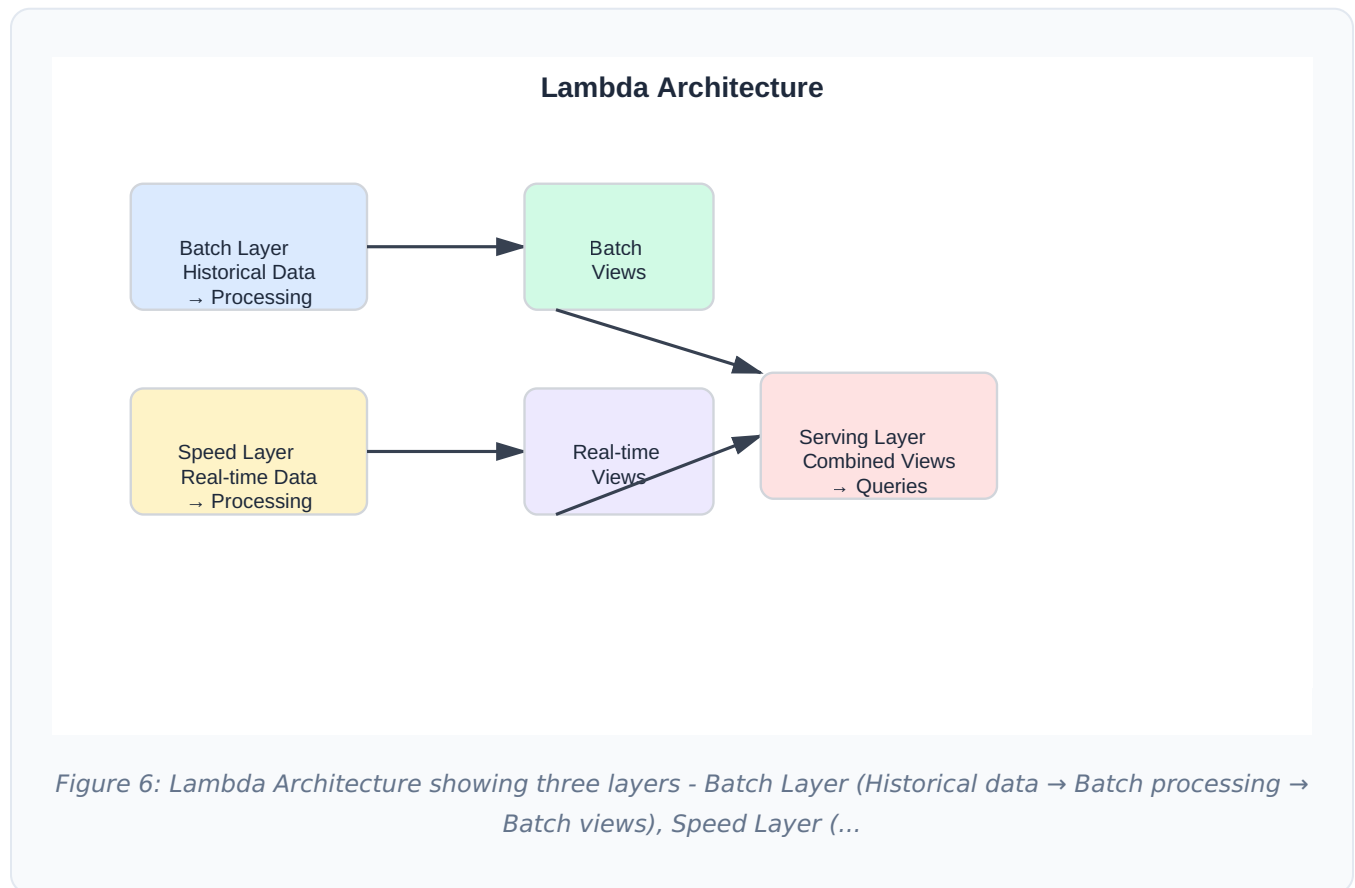
Stream processing excels when:

- Latency requirements are sub- second to minutes
- You need to react to events as they happen
- Data volumes are manageable in real-time
- Approximate results are acceptable

Let's dive into the patterns.

Pattern 1: Lambda Architecture

The Lambda architecture, coined by Nathan Marz, handles both batch and stream processing by maintaining separate "hot" and "cold" paths.



Components: - Batch Layer: Processes historical data, provides

- accurate results with hours/days latency
- Speed Layer: Processes recent data, provides approximate results with seconds/minutes latency
- Serving Layer: Combines batch and speed layer results for queries

Real-World Example

- Netflix Recommendations

PYTHON

```
# Batch Layer - Daily collaborative filtering
def batch_recommendations(date):
    """Generate accurate recommendations using full user history"""
    user_history =
```

```
spark.read.parquet(f"s3://data/user_views/{date}/")
```

```
content_features =
```

```
spark.read.parquet(f"s3://data/content_features/{date}/")
```

```
# Complex matrix factorization on full dataset
model = train_collaborative_filtering(user_history,
```

```
content_features)
```

```
recommendations = model.predict_all_users()
```

```
# Write to serving layer
recommendations.write.mode("overwrite") \
```

```
.parquet(f"s3://serving/batch_recommendations/{date}/")
```

PYTHON

```
# Speed Layer - Real-time updates based on current session
def stream_recommendations(user_event):
    """Update recommendations based on real-time behavior"""
    user_id = user_event['user_id']
    content_id = user_event['content_id']

    # Simple rule-based adjustments
    if user_event['action'] == 'thumbs_up':
        boost_similar_content(user_id, content_id)
    elif user_event['action'] == 'skip_after_30_seconds':
        downrank_similar_content(user_id, content_id)
```

When to Use Lambda

- You need both accurate historical analysis

AND fast approximate results



- ☐ Checkpoint

- 1.
2. ☐ Different SLAs for different use cases (batch for reporting, stream for user experience)

- ☐ **Checkpoint**

- 1.
2. ☐ Have the team bandwidth to maintain two processing paradigms

When to Avoid Lambda

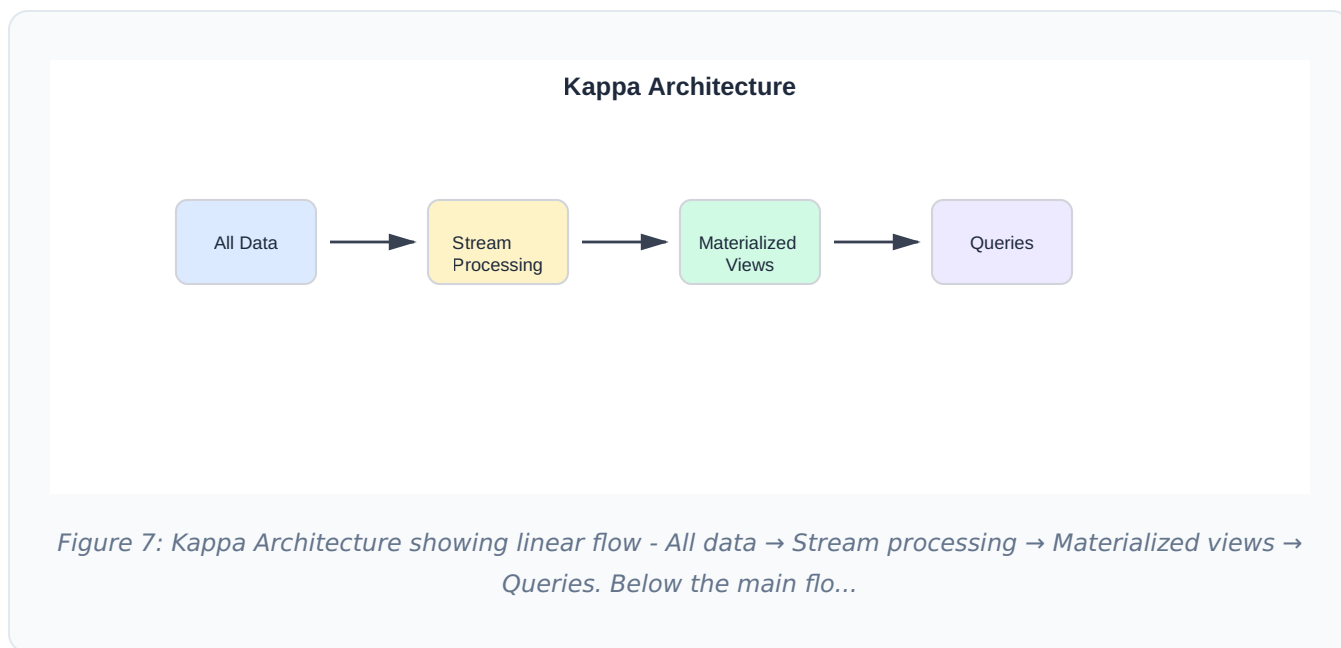
- ☐ Single processing paradigm can meet all requirements
- ☐ Team lacks streaming expertise
- ☐ Operational complexity outweighs benefits

☐ **Pro Tip**

Lambda architecture is often overengineered. Start with just batch processing. Add the speed layer only when you have proven latency requirements that batch can't meet. Many "real-time" requirements are actually "fast batch" requirements (15-minute updates vs 24-hour updates).

Pattern 2: Kappa Architecture

Kappa architecture, proposed by Jay Kreps, eliminates the batch layer entirely. Everything is stream processing, with historical data reprocessed as a stream.



Core Principle: Treat everything as a stream, including historical data. When you need to reprocess historical data, replay the stream.

Real-World Example: LinkedIn Activity Feeds

SQL

```
# Single stream processing job handles both real-time and batch
def process_activity_stream():
    """Process all user activities as a stream"""
    activity_stream = kafka_source("user-activities")

    # Window-based aggregations for feeds
    user_feeds = activity_stream \
        .keyBy("user_id") \
        .window(TumblingProcessingTimeWindows.of(Time.minutes(5))) \
        .aggregate(ActivityAggregator()) \
        .sink(redis_sink("user-feeds"))

    # Also materialize to data lake for analytics
    activity_stream \
        .sink(s3_sink("s3://data-lake/activities/"))

# Historical reprocessing is just replaying the stream
def reprocess_historical_data(start_date, end_date):
    """Reprocess historical data by replaying events"""
    historical_source = kafka_source(
        "user-activities",
        from_timestamp=start_date,
        to_timestamp=end_date
    )

    # Same processing logic as real-time stream
    process_activity_stream()
```

When to Use Kappa

□ Strong streaming infrastructure and expertise

□ Checkpoint

1. □ Uniform processing requirements across all data

□ Checkpoint

1. □ Need to frequently reprocess historical data

□ Checkpoint

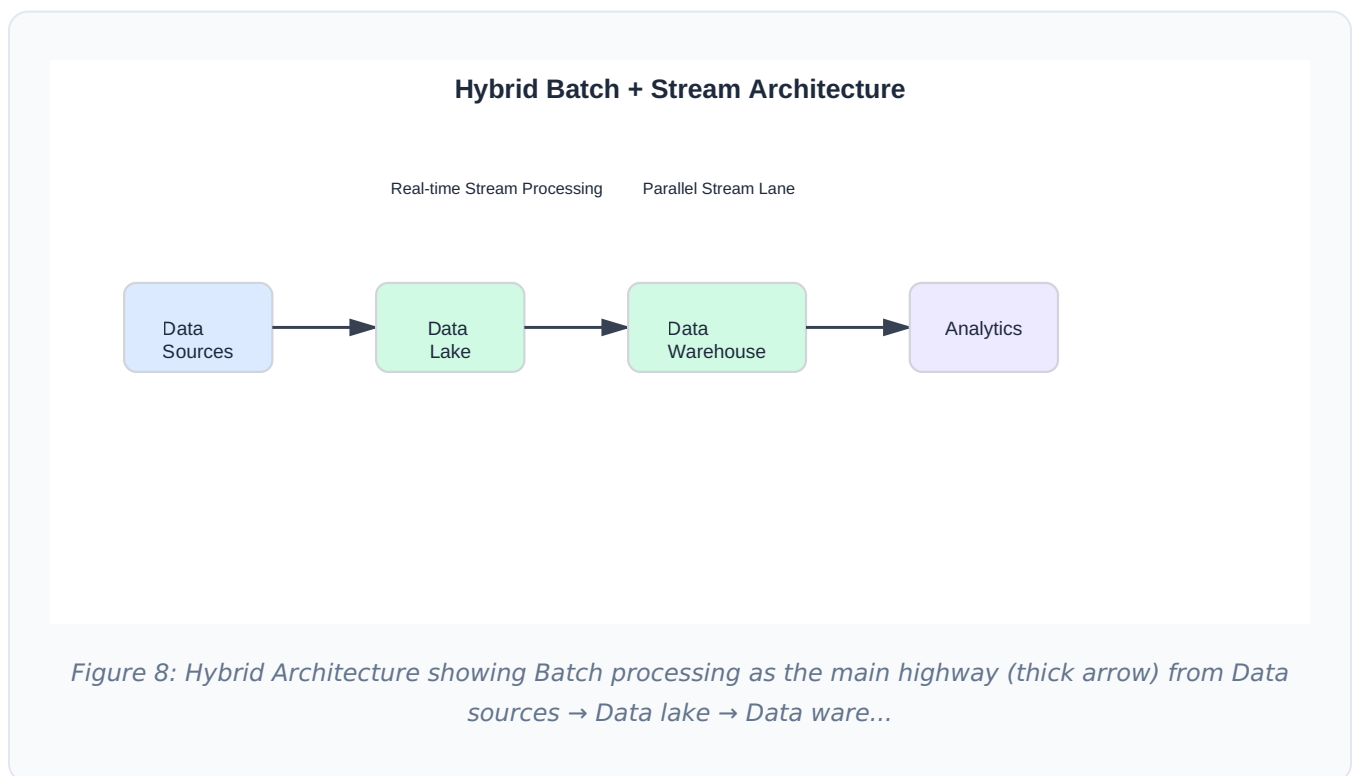
1. □ Can model all data as events

When to Avoid Kappa

- □ Complex analytical queries that need full dataset context
- □ Limited streaming infrastructure
- □ Batch-friendly processing requirements (large ML model training)

Pattern 3: Hybrid Batch-First Architecture

Most successful data platforms use a hybrid approach: batch processing as the foundation, with targeted stream processing for specific use cases.



Real-World Example

- Airbnb's Data Platform

SQL

```
# Primary batch pipeline processes all data overnight
def daily_etl_pipeline():
    """Main batch pipeline processing all Airbnb data"""
    # Extract from production databases
    bookings = extract_bookings(yesterday)
    listings = extract_listings(yesterday)
    searches = extract_searches(yesterday)

    # Transform and load into warehouse
    fact_bookings = transform_bookings(bookings, listings)
    dim_listings = transform_listings(listings)

    warehouse.load(fact_bookings, "fact_bookings",
```

```
partition_date=yesterday)
```

```
warehouse.load(dim_listings, "dim_listings",
```

```
partition_date=yesterday)
```

PYTHON

```
# Generate business metrics
generate_daily_metrics()
refresh_dashboard_cache()

# Targeted stream processing for specific real-time needs
def real_time_fraud_detection():
    """Stream processing only for fraud detection"""
    booking_stream = kafka_source("booking-events")

    suspicious_bookings = booking_stream \
        .filter(lambda b: b['amount'] > 10000 or b['velocity_score']
```

```
> 0.8) \
```

PYTHON

```
.map(lambda b: enriched_booking(b)) \
    .filter(lambda b: ml_fraud_score(b) > 0.9)

suspicious_bookings.sink(fraud_alert_sink())

# 99% of use cases use batch results, 1% use stream results
```


Benefits of Batch-First:

- Simpler architecture and operations
- Lower cost (batch compute is 5-10x cheaper)
- Better for complex analytics
- Easier debugging and reprocessing
- Most business requirements are actually batch-friendly

- ⚠ Common Mistake
- ⚠ Common Mistake Don't default to Lambda or Kappa because they sound more sophisticated. Start with batch processing and add complexity only when you have concrete requirements that batch can't meet. The best architecture is the simplest one that meets your requirements.

Orchestration vs Choreography

When building batch systems, you need to coordinate multiple jobs.

Two patterns emerge:

Orchestration (Centralized Control)

```
# Airflow DAG - centralized orchestration
from airflow import DAG
from airflow.operators.python import PythonOperator

dag = DAG('daily_etl')

extract_bookings = PythonOperator(
    task_id='extract_bookings',
    python_callable=extract_bookings_data
)
```

SQL

The corresponding SQL query:

```

extract_listings = PythonOperator(
    task_id='extract_listings',
    python_callable=extract_listings_data
)

transform_data = PythonOperator(
    task_id='transform_data',
    python_callable=transform_and_join
)

# Explicit dependencies
extract_bookings >> transform_data
extract_listings >> transform_data

```

Choreography (Event-Driven)

```

# Event-driven choreography
def on_bookings_extracted(event):
    """Triggered when bookings extraction completes"""
    publish_event("bookings.extracted", event.data)

def on_listings_extracted(event):
    """Triggered when listings extraction completes"""
    publish_event("listings.extracted", event.data)

def on_both_extracted(bookings_event, listings_event):
    """Triggered when both extractions complete"""
    transform_and_join(bookings_event.data, listings_event.data)
    publish_event("transform.completed", result)

```

Orchestration Benefits:

- Clear dependency visualization
- Centralized monitoring and debugging
- Better for complex workflows
- Easier testing and development

Choreography Benefits:

- More resilient to failures
- Better for loosely coupled systems
- Easier to add new consumers

- More scalable for many participants

Recommendation: Use orchestration for ETL pipelines, choreography for event-driven systems.

Idempotency and Retry Strategies

Batch processing jobs must be idempotent - running them multiple times produces the same result.

Idempotency Patterns:

```
# Pattern 1: Partition-based idempotency
def process_daily_data(date):
    """Idempotent by overwriting the entire partition"""
    output_path = f"s3://warehouse/fact_orders/date={date}/"

    # Always overwrite the entire partition
    daily_orders.write.mode("overwrite").parquet(output_path)

# Pattern 2: Upsert-based idempotency
def update_customer_metrics(date):
    """Idempotent by upsert on primary key"""
    new_metrics = calculate_customer_metrics(date)

    # Upsert - INSERT if not exists, UPDATE if exists
    warehouse.upsert(
        new_metrics,
        target_table="customer_metrics",
        merge_keys=["customer_id", "date"]
    )

# Pattern 3: Checksum-based idempotency
def process_if_changed(source_path, output_path):
    """Only process if source data has changed"""
    source_checksum = calculate_checksum(source_path)
    last_checksum = get_last_checksum(output_path)

    if source_checksum != last_checksum:
        process_data(source_path, output_path)
        save_checksum(output_path, source_checksum)
```

PYTHON

Retry Strategies:

PYTHON

```
class BatchJobRetry:
    def __init__(self, max_retries=3, backoff_factor=2):
        self.max_retries = max_retries
        self.backoff_factor = backoff_factor

    def run_with_retry(self, job_function, *args, **kwargs):
        """Run job with exponential backoff retry"""
        for attempt in range(self.max_retries + 1):
            try:
                return job_function(*args, **kwargs)
            except RetryableException as e:
                if attempt == self.max_retries:
                    raise FinalFailureException(f"Failed after
```

```
{self.max_retries} retries: {e}")
```

PYTHON

```
# Exponential backoff
        sleep_time = (self.backoff_factor ** attempt) * 60
        time.sleep(sleep_time)

    except NonRetryableException as e:
        # Don't retry for non-retryable errors
        raise e

# Usage
@retry_on_failure(max_retries=3)
def daily_etl_job(date):
    try:
        extract_data(date)
        transform_data(date)
        load_data(date)
    except ConnectionError:
        # Retryable - temporary network issue
        raise RetryableException("Database connection failed")
    except DataValidationError:
        # Non-retryable - data quality issue needs investigation
        raise NonRetryableException("Data validation failed")
```

- ☐ Checkpoint

1. When would you choose Lambda architecture over pure batch processing? a. When you need real-time dashboards and daily reports with

different accuracy requirements

b. When you want to use the latest technology c. When you have a small dataset d. When you need to save money 2. What makes a batch processing job idempotent? a. It runs fast b. It produces the same result when run multiple times c. It never fails d. It processes data in order 3. Which retry strategy is appropriate for a temporary database connection failure? a. Immediate retry b. No retry - fail fast c. Exponential backoff retry d. Linear retry every second

Answers: 1-a, 2-b, 3-c

ETL vs ELT Design Decisions

⚡ TL;DR

ETL: Extract, Transform, Load - transform data before loading to warehouse - ELT: Extract, Load, Transform - load raw data first, transform in warehouse - ELT wins for cloud warehouses with cheap storage and fast compute - ETL still needed for complex transformations, real-time processing, and data quality - Modern data teams use hybrid approach: ELT for analytics, ETL for operational systems

The choice between ETL and ELT fundamentally shapes your data architecture. This isn't just about tool selection - it's about where computation happens, how you handle data quality, and what skills your team needs.

The Great ELT Revolution

For decades, data warehousing meant ETL. Extract data from source systems, transform it extensively, then load clean, structured data into expensive warehouse storage.

Then cloud warehouses changed everything:

Traditional ETL (2000-2015):

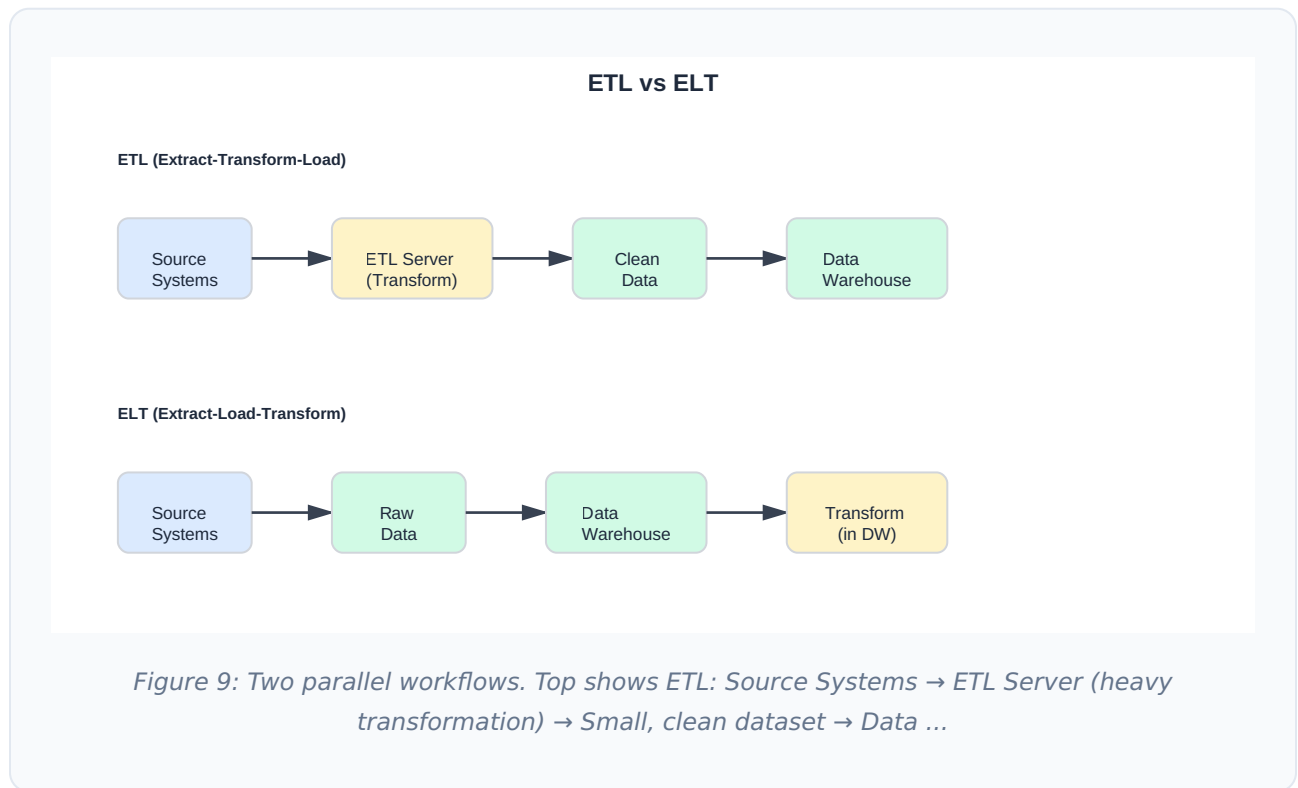
- Expensive warehouse storage (\$1000s per TB)
- Slow, fixed-size compute clusters
- Transform data before loading to minimize storage costs
- Complex

ETL tools and specialized jobs

Modern ELT (2015-present):

- Cheap warehouse storage (~\$20 per TB)
- Elastic compute that scales in seconds

- Load raw data first, transform in warehouse
- SQL-based transformations using warehouse compute



When to Choose ELT

ELT Excels When:

```
-- Example: Complex revenue calculations in Snowflake
WITH order_metrics AS (
  SELECT
    customer_id,
    DATE_TRUNC('month', order_date) as month,
    SUM(gross_amount) as gross_revenue,
    SUM(gross_amount - discount_amount - refund_amount) as
```

SQL

net_revenue,

```

COUNT(*) as order_count,
    AVG(gross_amount) as avg_order_value
FROM raw_orders
WHERE order_status NOT IN ('cancelled', 'fraud')
    AND order_date >= '2023-01-01'
GROUP BY 1, 2
),

customer_cohorts AS (
    SELECT
        customer_id,
        MIN(month) as first_order_month,
        MAX(month) as last_order_month,
        SUM(net_revenue) as lifetime_value
    FROM order_metrics
    GROUP BY 1
)

SELECT
    om.month,
    cc.first_order_month,
    COUNT(DISTINCT om.customer_id) as active_customers,
    SUM(om.net_revenue) as total_revenue,
    AVG(om.avg_order_value) as cohort_avg_order_value

```

The corresponding SQL query:

```

FROM order_metrics om
JOIN customer_cohorts cc ON om.customer_id = cc.customer_id
GROUP BY 1, 2;

```

Benefits of this approach:

- Warehouse speed: Modern warehouses execute this query in seconds on terabytes
- Flexibility:

Easy to change business logic without rebuilding ETL pipelines -

Collaboration: Business analysts can modify transformations directly

- Version control: SQL transformations in git like any other code

ELT Works Best For:

- ☐ Analytical workloads with complex business logic

- ☐ Checkpoint

1.

2. ☐ Teams with strong SQL skills

- ☐ Checkpoint

1.

2. ☐ Requirements that change frequently

- ☐ Checkpoint

1.

2. ☐ Cloud warehouse environments (Snowflake, BigQuery, Redshift)

- ☐ Checkpoint

1.

2. ☐ Use cases where storage is cheaper than compute time

When ETL Still Wins

ETL Excels When:

```
# Example: Real-time fraud detection ETL
def detect_fraud_realtime(transaction_event):
    """Complex ETL processing before loading to operational
```

```
systems"""
```

```
# Step 1: Extract and enrich transaction
transaction = extract_transaction(transaction_event)
user_history = get_user_history(transaction.user_id, days=30)
device_fingerprint = extract_device_info(transaction.device_id)

# Step 2: Complex transformations using multiple data sources
features = {
    'amount': transaction.amount,
    'velocity_score': calculate_velocity(user_history),
    'device_risk': score_device_risk(device_fingerprint),
    'merchant_risk':
```

```
get_merchant_risk_score(transaction.merchant_id),
```

```
'time_since_last':
```

```
time_since_last_transaction(user_history),
```

```
'amount_zscore': calculate_amount_zscore(transaction.amount,
```

```
user_history)
```

```
}

# Step 3: ML model inference
fraud_probability = fraud_model.predict(features)

# Step 4: Business rules
if fraud_probability > 0.95 or features['amount'] > 50000:
    action = 'block'
elif fraud_probability > 0.7:
    action = 'review'
else:
    action = 'approve'

# Step 5: Load enriched, transformed result
result = {
    'transaction_id': transaction.id,
    'fraud_score': fraud_probability,
    'action': action,
    'features': features,
    'processed_at': datetime.now()
}

# Load to operational system for real-time decision
fraud_detection_db.insert(result)

# Also send to analytics warehouse (raw + enriched)
analytics_stream.send(transaction_event)
analytics_stream.send(result)
```

ETL Works Best For:

- ☐ Real-time/low-latency requirements

- ☐ Checkpoint

- 1.
2. ☐ Complex data quality validation and cleansing

- ☐ Checkpoint

- 1.
2. ☐ Integration with multiple external APIs

- ☐ Checkpoint

- 1.
2. ☐ Machine learning feature engineering

- ☐ Checkpoint

- 1.
2. ☐ Operational systems that need clean, validated data

- ☐ Checkpoint

- 1.
2. ☐ Cases where raw data is sensitive and shouldn't be stored

The Modern Hybrid Approach

Smart data teams don't choose ETL vs ELT - they use both strategically.

Real-World Example: Stripe's Data Architecture

```
# ETL for operational systems (fraud, risk, payments)
class PaymentETLPipeline:
    def process_payment_event(self, event):
        """ETL for operational payment processing"""
        # Extract
        payment = self.extract_payment(event)
        user = self.extract_user(payment.user_id)
        merchant = self.extract_merchant(payment.merchant_id)

        # Transform - real-time risk scoring
        risk_features = self.calculate_risk_features(payment, user,
```

PYTHON

merchant)

```
        risk_score = self.risk_model.predict(risk_features)

        # Load - operational database for immediate decisions
        enriched_payment = {
            **payment.dict(),
            'risk_score': risk_score,
            'risk_features': risk_features,
            'processed_at': datetime.utcnow()
        }
        self.operational_db.insert('payments', enriched_payment)

        # Also send raw event to analytics pipeline
        self.analytics_stream.send(event)

# ELT for analytics (business intelligence, reporting)
# Raw payment events land in data warehouse, then transformed with
```

PYTHON

dbt

```
-- dbt model for analytics (ELT approach)
-- models/marts/finance/daily_payment_metrics.sql
SELECT
    payment_date,
    merchant_category,
    payment_method,
    COUNT(*) as transaction_count,
    SUM(payment_amount) as gross_volume,
    SUM(CASE WHEN payment_status = 'succeeded' THEN payment_amount
```

ELSE 0 END) as net_volume,

```
AVG(risk_score) as avg_risk_score,
    COUNT(DISTINCT user_id) as unique_payers
FROM {{ ref('raw_payments') }}
WHERE payment_date >= current_date - 90
GROUP BY 1, 2, 3
```

Hybrid Architecture Benefits:

- Operational systems get clean, fast ETL processing
- Analytics teams get flexible ELT with raw data access
- Different teams can optimize for their requirements
- Raw data preservation for future analytics

📌 Pro Tip

Use ETL for systems that serve applications (APIs, dashboards, operational databases). Use ELT for systems that serve humans (BI tools, ad-hoc analysis, data science). This gives you both speed and flexibility.

Data Quality: ETL vs ELT Approaches

ETL Quality Approach:

```
def etl_with_quality_checks(source_data):
    """ETL with strict quality gates"""

    # Quality check 1: Schema validation
    if not validate_schema(source_data):
        raise DataQualityException("Schema validation failed")

    # Quality check 2: Business rules
    invalid_records = source_data.filter(
        (col("amount") <= 0) |
        (col("user_id").isNull()) |
        (col("timestamp") < "2020-01-01")
    )

    if invalid_records.count() > 0:
        # Option 1: Fail the entire batch
        raise DataQualityException(f"Found {invalid_records.count()}
invalid records")
```

```
# Option 2: Quarantine bad records, process good ones
# quarantine_bad_records(invalid_records)
# source_data = source_data.subtract(invalid_records)

# Transform only clean data
transformed_data = transform_clean_data(source_data)

# Load to warehouse
load_to_warehouse(transformed_data)
```

ELT Quality Approach:

```
-- Load all data first, then identify quality issues
CREATE OR REPLACE VIEW data_quality_issues AS
SELECT
    'negative_amount' as issue_type,
    COUNT(*) as issue_count,
    current_timestamp as checked_at
FROM raw_transactions
WHERE amount <= 0

UNION ALL

SELECT
    'missing_user_id' as issue_type,
    COUNT(*) as issue_count,
    current_timestamp as checked_at
FROM raw_transactions
WHERE user_id IS NULL

UNION ALL

SELECT
    'future_timestamp' as issue_type,
    COUNT(*) as issue_count,
    current_timestamp as checked_at
FROM raw_transactions
WHERE transaction_date > current_date;

-- Create clean view excluding bad records
CREATE OR REPLACE VIEW clean_transactions AS
SELECT * FROM raw_transactions
WHERE amount > 0
    AND user_id IS NOT NULL
    AND transaction_date <= current_date;
```

Quality Trade-offs:

Aspect ETL Approach ELT Approach

Bad data Fail fast or quarantine Load everything, clean handling in views Quality Hidden (processed Transparent (quality visibility before warehouse) issues visible)

Reprocess entire ETL Adjust SQL

Recovery

pipeline	transformation
Faster queries (pre-	Slower queries (filter on

Performance

cleaned)	read)
Hard to change quality	Easy to change quality

Flexibility

rules	rules
-------	-------

Cost Considerations

ETL Cost Model:

ETL Infrastructure:

- Dedicated ETL servers: \$5,000/month
- ETL tool licenses: \$3,000/month
- Data engineering team: 2 FTE = \$30,000/month
- Warehouse storage (clean data only): 10TB = \$200/month Total monthly cost: ~\$38,200

Benefits: Fast warehouse queries, clean data

ELT Cost Model:

ELT Infrastructure:

- Warehouse storage (raw + clean): 50TB = \$1,000/month
- Warehouse compute: 100 credit-hours/month = \$4,000/month

- dbt Cloud: \$500/month
- Data engineering team: 1.5 FTE = \$22,500/month Total monthly cost: ~\$28,000

Benefits: Flexibility, faster development, analyst independence

Cost Analysis:

- ELT typically 20-30% cheaper due to reduced infrastructure and team costs
- ETL can be cheaper for high-query- volume scenarios (pre-computed results)
- ELT scales better with team growth (analysts can self-serve)

Tool Selection Guide

ETL Tools:

```
# Apache Airflow - orchestration + custom ETL
from airflow import DAG
from airflow.operators.python import PythonOperator

def extract_and_transform():
    # Custom Python ETL logic
    raw_data = extract_from_api()
    clean_data = validate_and_transform(raw_data)
    load_to_warehouse(clean_data)

# AWS Glue - managed ETL service
import sys
from awsglue.transforms import *
from awsglue.context import GlueContext

glueContext = GlueContext(SparkContext.getOrCreate())

# Read from S3, transform, write to Redshift
datasource = glueContext.create_dynamic_frame.from_catalog(
    database="source_db", table_name="raw_transactions")

transformed = ApplyMapping.apply(frame=datasource, mappings=[
    ("transaction_id", "string", "id", "string"),
    ("amount", "double", "amount_cents", "long"),
    ("user_id", "string", "customer_id", "string")
])
```

The corresponding SQL query:

```
glueContext.write_dynamic_frame.from_catalog(
    frame=transformed, database="warehouse_db",
```

```
table_name="clean_transactions")
```

ELT Tools:

```
-- dbt - SQL-based transformations
-- models/staging/stg_transactions.sql
SELECT
    transaction_id as id,
    CAST(amount * 100 AS INTEGER) as amount_cents,
    user_id as customer_id,
    CAST(transaction_timestamp AS TIMESTAMP) as transaction_at
FROM {{ source('raw', 'transactions') }}
WHERE transaction_timestamp IS NOT NULL

-- models/marts/daily_revenue.sql
SELECT
    DATE(transaction_at) as revenue_date,
    SUM(amount_cents) / 100.0 as total_revenue,
    COUNT(*) as transaction_count
FROM {{ ref('stg_transactions') }}
GROUP BY 1
```

Selection Criteria:

Use ETL When Use ELT When

Real-time requirements **Batch analytics requirements**

Complex multi-system

Single warehouse ecosystem integration Strong Python/Java team Strong SQL/analytics team

Operational data products **Business intelligence focus**

Sensitive data that can't be Historical data preservation stored raw important

- □ Checkpoint
1. What's the main advantage of ELT over ETL in modern cloud warehouses? a. ELT is always faster b. ELT leverages cheap storage and elastic compute c. ELT requires less engineering time d. ELT handles real-time data better
 2. When should you choose ETL over ELT? a. When you need to save money b. When you have real-time latency requirements c. When you use Snowflake d. When your data is small
 3. What's a hybrid ETL/ELT approach? a. Using both tools at the same time b. ETL for operational systems, ELT for analytics c. Running ETL first, then ELT d. Using ELT tools to do ETL

Answers: 1-b, 2-b, 3-b

3.3 Workflow Orchestration at Scale

⚡ TL;DR

Orchestration is the backbone of reliable data pipelines - it handles scheduling, dependencies, retries, and monitoring - Choose tools based on your complexity: cron → Airflow → Dagster → cloud-native solutions - Design for failure: every task can fail, network can partition, dependencies can be delayed - Backfilling is inevitable - design your workflows to handle historical data reprocessing - Monitoring and alerting are more important than the orchestration tool choice

When your data platform grows beyond a few daily jobs, you need orchestration. Not just scheduling - true orchestration that handles dependencies, failures, monitoring, and the inevitable need to reprocess historical data.

The Evolution of Data Orchestration

Stage 1: Cron Jobs (0-10 pipelines)

```
# Simple cron-based scheduling
# /etc/crontab

# Daily ETL at 2 AM
0 2 * * * /opt/data/scripts/extract_orders.py
30 2 * * * /opt/data/scripts/transform_orders.py
0 3 * * * /opt/data/scripts/load_orders.py

# Weekly aggregations every Sunday at 4 AM
0 4 * * 0 /opt/data/scripts/weekly_metrics.py
```

Problems with cron:

- No dependency management (what if extract_orders.py fails?)
- No retry logic
- No monitoring or alerting -

No visibility into job status - Nightmare to debug failures

Stage 2: Orchestration Framework (10+ pipelines)

```

# Apache Airflow DAG
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'data-team',
    'depends_on_past': False,
    'email_on_failure': True,
    'email_on_retry': False,
    'retries': 3,
    'retry_delay': timedelta(minutes=5)
}

dag = DAG(
    'daily_orders_etl',
    default_args=default_args,
    description='Daily orders ETL pipeline',
    schedule_interval='0 2 * * *', # 2 AM daily
    start_date=datetime(2024, 1, 1),
    catchup=True # Enable backfilling
)

# Tasks with dependencies
extract_orders = PythonOperator(
    task_id='extract_orders',
    python_callable=extract_orders_data,
    dag=dag
)

validate_data = PythonOperator(
    task_id='validate_data',
    python_callable=validate_orders_data,
    dag=dag
)

transform_orders = PythonOperator(
    task_id='transform_orders',
    python_callable=transform_orders_data,
    dag=dag
)

load_warehouse = PythonOperator(
    task_id='load_warehouse',
    python_callable=load_to_warehouse,

```



```
dag=dag
)

update_metrics = PythonOperator(
    task_id='update_metrics',
    python_callable=refresh_business_metrics,
    dag=dag
)

# Define dependencies
extract_orders >> validate_data >> transform_orders >>
```

load_warehouse >> update_metrics

Benefits of orchestration:

- Dependency management: Tasks run in correct order
- Retry logic: Automatic retries with exponential backoff
- Monitoring: Web UI shows job status and logs
- Alerting: Email/Slack notifications on failures
- Backfilling: Reprocess historical data automatically

Choosing the Right Orchestration Tool

Apache Airflow: The Industry Standard

```
# Pros: Mature ecosystem, Python-native, great for complex workflows
# Cons: Steep learning curve, resource-heavy, operational complexity
```

```
# Example: Complex multi-system coordination
def check_upstream_data_availability(**context):
    """Check if upstream systems have fresh data"""
    execution_date = context['execution_date']

    # Check multiple upstream systems
    systems_ready = []
    for system in ['salesforce', 'mysql', 'kafka']:
        last_update = get_last_update_time(system)
        if last_update > execution_date - timedelta(hours=1):
            systems_ready.append(system)

    if len(systems_ready) < 3:
        raise AirflowException(f"Not all systems ready:
{systems_ready}")
```

```
return systems_ready

# Sensor to wait for upstream data
data_sensor = PythonSensor(
    task_id='wait_for_upstream_data',
    python_callable=check_upstream_data_availability,
    timeout=3600, # Wait up to 1 hour
    poke_interval=300, # Check every 5 minutes
    dag=dag
)
```

When to choose Airflow

- ☐ Complex workflows with conditional logic

☐ Checkpoint

1. ☐ Strong Python team

❑ Checkpoint

1. ❑ Need advanced features (sensors, branching, external triggers)

❑ Checkpoint

1. ❑ Want battle-tested solution with large community

Dagster: The Modern Alternative

```
# Pros: Data-aware, better testing, type safety
# Cons: Newer, smaller community, learning curve

from dagster import asset, get_dagster_logger

@asset(group_name="raw_data")
def raw_orders():
    """Extract raw orders from production database"""
    logger = get_dagster_logger()

    orders = extract_orders_from_db()
```

PYTHON

The PYTHON code:

```
logger.info(f"Extracted {len(orders)} orders")

    return orders

@asset(group_name="processed_data")
def clean_orders(raw_orders):
    """Clean and validate orders data"""
    # Input is automatically provided by Dagster
    clean_data = validate_and_clean(raw_orders)

    # Data quality checks
    assert len(clean_data) > 0, "No clean orders found"
    assert clean_data['amount'].min() > 0, "Found negative order"
```

PYTHON

amounts"

```
return clean_data
```

PYTHON

```
@asset(group_name="warehouse")
def orders_fact_table(clean_orders):
    """Load orders to warehouse fact table"""
    warehouse.load(clean_orders, "fact_orders", mode="append")
    return {"loaded_rows": len(clean_orders)}
```

When to choose Dagster

□ Data-focused team that values testing and data quality

□ Checkpoint

1. □ Want software engineering best practices

□ Checkpoint

1. □ Need better development experience

□ Checkpoint

1. □ Building modern data platform from scratch

Cloud-Native Solutions

```
# Google Cloud Composer (managed Airflow)
# AWS Step Functions for simpler workflows
# Azure Data Factory for Microsoft ecosystem

# Example: AWS Step Functions state machine
{
  "Comment": "Daily ETL Pipeline",
  "StartAt": "ExtractData",
  "States": {
    "ExtractData": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-
```

1:123456789012:function:extract-orders",

```
"Retry": [
  {
    "ErrorEquals": ["States.TaskFailed"],
    "IntervalSeconds": 300,
    "MaxAttempts": 3,
    "BackoffRate": 2.0
  }
],
"Next": "TransformData"
},
"TransformData": {
  "Type": "Task",
  "Resource": "arn:aws:lambda:us-east-
```

1:123456789012:function:transform-orders",

```
"Next": "LoadData"
},
"LoadData": {
  "Type": "Task",
  "Resource": "arn:aws:lambda:us-east-
```

1:123456789012:function:load-warehouse",

```
"End": true
  }
}
```

When to choose cloud-native

- ☐ Simplicity over flexibility

- ☐ Checkpoint

- 1.
2. ☐ Serverless/managed operations preferred

- ☐ Checkpoint

- 1.
2. ☐ Team lacks orchestration expertise

- ☐ Checkpoint

- 1.
2. ☐ Workflows are relatively simple

☐ Pro Tip

Don't get caught up in tool wars. Airflow, Dagster, and cloud solutions can all build reliable data pipelines. Choose based on your team's skills and operational preferences, not feature comparisons.

Design Patterns for Reliable Workflows

Pattern 1: Idempotent Tasks

```
# BAD: Non-idempotent task
def bad_daily_aggregation(date):
    """This will double-count if run twice"""
    daily_orders = get_orders_for_date(date)

    for order in daily_orders:
        # This adds to existing values - running twice doubles the
```

PYTHON

metrics!

```
update_customer_metrics(order.customer_id, order.amount)

# GOOD: Idempotent task
def good_daily_aggregation(date):
    """Safe to run multiple times"""
    daily_orders = get_orders_for_date(date)

    # Calculate complete metrics for the date
    customer_metrics = {}
    for order in daily_orders:
        if order.customer_id not in customer_metrics:
            customer_metrics[order.customer_id] = {'orders': 0,
```

PYTHON

'total': 0}

```
customer_metrics[order.customer_id]['orders'] += 1
customer_metrics[order.customer_id]['total'] += order.amount

# Replace all metrics for this date (idempotent)
replace_customer_metrics_for_date(date, customer_metrics)
```

Pattern 2: Atomic Batch Processing

```
def atomic_data_processing(date):
    """Process data atomically to avoid partial state"""

    # Process to temporary location first
    temp_output_path = f"s3://temp-bucket/daily-
```

PYTHON

```
metrics/{date}/{uuid.uuid4()}/"
```

```
try:
    # Do all processing
    orders = extract_orders(date)
    metrics = calculate_metrics(orders)

    # Write to temporary location
    write_parquet(metrics, temp_output_path)

    # Validate output
    validate_metrics(temp_output_path)

    # Atomic move to final location (S3 copy + delete is atomic)
    final_path = f"s3://warehouse/daily-metrics/date={date}/"
    s3.copy(temp_output_path, final_path)
    s3.delete(temp_output_path)

    # Update metadata last
    update_table_metadata("daily_metrics", date)

except Exception as e:
    # Clean up temp files on failure
    s3.delete(temp_output_path)
    raise e
```

Pattern 3: Graceful Degradation

```
def resilient_enrichment_pipeline(date):
```

PYTHON

```
    """Pipeline that continues even if enrichment fails"""

    # Core data is required - fail if this doesn't work
    orders = extract_orders(date)
    if not orders:
        raise Exception("No orders found - this should never
```

```
happen")
```



```

# Enrichment data is optional - continue without it
enriched_orders = []

for order in orders:
    try:
        # Try to enrich with customer data
        customer_data = get_customer_data(order.customer_id)
        order_with_customer = {**order.dict(), **customer_data}

        # Try to enrich with product data
        product_data = get_product_data(order.product_id)
        fully_enriched = {**order_with_customer, **product_data}

        enriched_orders.append(fully_enriched)

    except CustomerServiceDown:
        # Customer service is down, use basic customer info
        basic_order = {**order.dict(), 'customer_tier':

```

'unknown']}

```

        enriched_orders.append(basic_order)

    except ProductServiceDown:
        # Product service is down, continue without product info
        enriched_orders.append(order.dict())

# Load whatever we have
load_to_warehouse(enriched_orders)

# Send alert about degraded data quality
if any('customer_tier' in order and order['customer_tier'] ==

```

'unknown'

```

for order in enriched_orders):
    send_alert("Pipeline running in degraded mode - customer

```

enrichment failed")

Backfilling Strategies

- Backfilling

- reprocessing historical data
- is inevitable in data engineering. Design for it from day one.

Backfill Pattern 1: Date-Parameterized Tasks

```
# Design tasks to accept arbitrary dates
def process_orders_for_date(date_str):
    """Process orders for any date - enables easy backfilling"""
    date = datetime.strptime(date_str, '%Y-%m-%d')

    # Extract data for specific date
    orders = extract_orders_for_date(date)

    # Process and load with date partition
    processed = transform_orders(orders)
    load_with_partition(processed, partition_date=date)

# Airflow makes backfilling automatic
dag = DAG(
    'daily_orders',
    start_date=datetime(2024, 1, 1),
    schedule_interval='@daily',
    catchup=True # This enables backfilling
)

# To backfill missing dates:
# airflow dags backfill -s 2024-02-01 -e 2024-02-10 daily_orders
```

PYTHON

Backfill Pattern 2: Incremental vs Full Refresh

```
def smart_backfill_strategy(table_name, start_date, end_date,
```

PYTHON

mode='incremental'):

```

"""Smart backfilling that chooses optimal strategy"""

date_range = pd.date_range(start_date, end_date)
days_to_backfill = len(date_range)

if mode == 'incremental' and days_to_backfill <= 7:
    # Small backfill - process each day individually
    for date in date_range:
        process_daily_partition(table_name, date)

elif mode == 'full_refresh' or days_to_backfill > 30:
    # Large backfill - full refresh is more efficient
    full_refresh_table(table_name, start_date, end_date)

else:
    # Medium backfill - batch by weeks
    weekly_ranges = group_dates_by_week(date_range)
    for week_start, week_end in weekly_ranges:
        process_date_range(table_name, week_start, week_end)

```

Backfill Pattern 3: Dependency-Aware Backfilling

```

class BackfillManager:
    """Manages complex backfills across multiple dependent tables"""

    def __init__(self):
        self.dependency_graph = {
            'raw_orders': [], # No dependencies
            'clean_orders': ['raw_orders'],
            'customer_metrics': ['clean_orders'],
            'daily_revenue': ['clean_orders'],
            'executive_dashboard': ['customer_metrics'],

```

'daily_revenue']

```

}

def backfill_with_dependencies(self, table_name, start_date,

```

end_date):

```
"""Backfill table and all its dependencies in correct
```

```
order"""
```

```
# Find all tables that need backfilling
tables_to_backfill = self._find_dependencies(table_name)

# Topological sort to determine processing order
processing_order =
```

PYTHON

```
self._topological_sort(tables_to_backfill)
```

```
for table in processing_order:
    print(f"Backfilling {table} from {start_date} to
```

SQL

```
{end_date}")
```

```
self._backfill_table(table, start_date, end_date)
    print(f"Completed {table}")

def _find_dependencies(self, table_name):
    """Recursively find all dependency tables"""
    dependencies = set()

    def collect_deps(table):
        dependencies.add(table)
        for dep in self.dependency_graph.get(table, []):
            collect_deps(dep)

    collect_deps(table_name)
    return dependencies
```

PYTHON

Monitoring and Alerting

Monitoring Strategy:

PYTHON

```
class PipelineMonitor:
    """Comprehensive monitoring for data pipelines"""

    def __init__(self, slack_webhook, pagerduty_key):
        self.slack_webhook = slack_webhook
        self.pagerduty_key = pagerduty_key

    def check_pipeline_health(self, dag_name, hours_back=24):
        """Check pipeline health and send appropriate alerts"""

        # Check for failed tasks
        failed_tasks = self.get_failed_tasks(dag_name, hours_back)
        if failed_tasks:
            self.send_critical_alert(f"Pipeline {dag_name} has
```

failed tasks: {failed_tasks}")

PYTHON

```
# Check for missing runs (data freshness)
expected_runs = self.calculate_expected_runs(dag_name,
```

hours_back)

PYTHON

```
actual_runs = self.get_actual_runs(dag_name, hours_back)

if len(actual_runs) < expected_runs:
    self.send_warning_alert(f"Pipeline {dag_name} missing
```

runs. Expected: {expected_runs}, Actual: {len(actual_runs)}")

PYTHON

```
# Check for slow-running tasks
slow_tasks = self.get_slow_tasks(dag_name, hours_back)
if slow_tasks:
    self.send_info_alert(f"Pipeline {dag_name} has slow
```

tasks: {slow_tasks}")

PYTHON

```
# Check data quality metrics
quality_issues = self.check_data_quality(dag_name)
if quality_issues:
    self.send_warning_alert(f"Data quality issues in
```

```
{dag_name}: {quality_issues}")
```

PYTHON

```
def send_critical_alert(self, message):
    """Send to both Slack and PagerDuty for immediate
```

```
response"""
```

PYTHON

```
self.send_slack_alert(message, color='danger')
self.send PagerDuty alert(message, severity='critical')

def send_warning_alert(self, message):
    """Send to Slack for team awareness"""
    self.send_slack_alert(message, color='warning')

def send_info_alert(self, message):
    """Send to Slack for information only"""
    self.send_slack_alert(message, color='good')
```

SLA Monitoring:

```
# Define SLAs in your DAG
dag = DAG(
    'critical_customer_pipeline',
    sla_miss_callback=sla_miss_callback,    # Function called on SLA
```

```
miss
```

```
default_args={
    'sla': timedelta(hours=4),    # Pipeline must complete in 4
```

```
hours
```

```
'email_on_failure': True,
    'email': ['data-team@company.com', 'on-call@company.com']
}
)

def sla_miss_callback(dag, task_list, blocking_task_list, slas,
```

blocking_tis):

```
"""Custom SLA miss handling"""
message = f"""
SLA MISS ALERT

    DAG: {dag.dag_id}
```

```
Missed Tasks: {[task.task_id for task in task_list]}
Blocking Tasks: {[task.task_id for task in blocking_task_list]}

This pipeline feeds customer-facing dashboards. Please
```

investigate immediately.

```
"""

send Pagerduty alert(message, severity='critical')
send_slack_alert(message, channel='#data-incidents')
```

- ☐ Checkpoint
- 1. What's the main benefit of using an orchestration tool over cron jobs? a. Orchestration tools are faster b. Dependency management and retry logic c. Orchestration tools cost less d. Better for small teams
- 2. What makes a workflow task idempotent? a. It runs quickly b. It produces the same result when run multiple times c. It never fails d. It processes data in order

3. When should you choose full refresh over incremental backfilling? a. When you need to save time b. When backfilling many days and full refresh is more efficient c. When the data is small d. Never - incremental is always better

Answers: 1-b, 2-b, 3-b

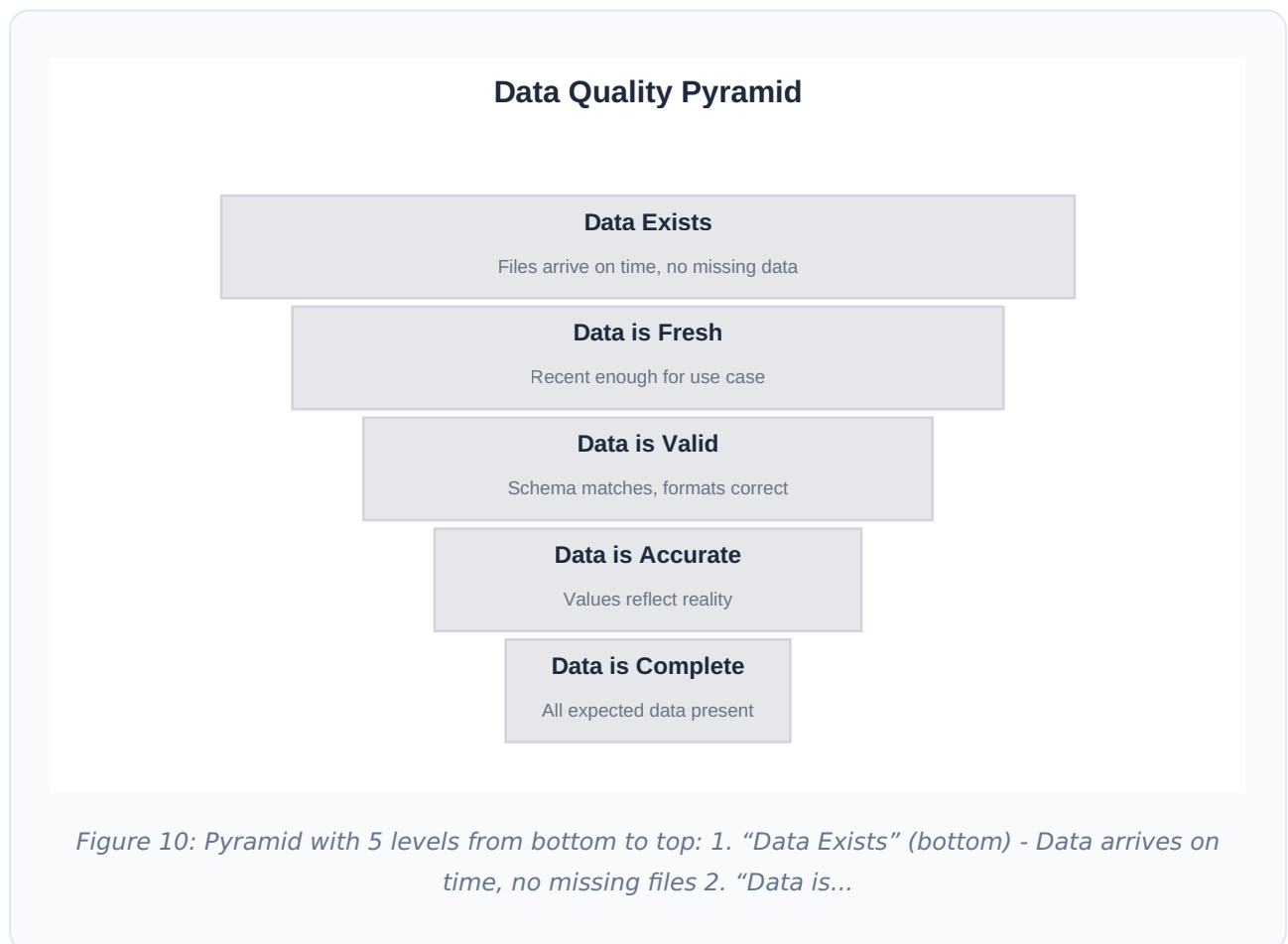
Data Quality in Batch Systems

⚡ TL;DR

Data quality issues are inevitable - plan for them, don't hope they won't happen - Implement quality checks at every stage: ingestion, processing, serving - Fail fast for critical issues, degrade gracefully for minor ones - Monitoring data quality is as important as monitoring infrastructure - Great expectations beats manual testing - automate everything

Data quality is the difference between a data platform that teams trust and one they work around. Poor data quality destroys confidence, leads to wrong decisions, and creates enormous debugging overhead.

The Data Quality Hierarchy of Needs



Most teams focus on the top of the pyramid (accuracy and completeness) while ignoring the foundation. This leads to fragile systems where small changes break everything.

Data Quality Framework

Level 1: Schema and Format Validation

```
import great_expectations as ge
from pydantic import BaseModel, validator
from typing import Optional
import pandas as pd

# Define data contracts with Pydantic
class OrderRecord(BaseModel):
```

The PYTHON code:

```
order_id: str
customer_id: str
amount: float
currency: str
order_date: str
status: str

@validator('amount')
def amount_must_be_positive(cls, v):
    if v <= 0:
        raise ValueError('Amount must be positive')
    return v

@validator('currency')
def currency_must_be_valid(cls, v):
    valid_currencies = ['USD', 'EUR', 'GBP', 'CAD']
    if v not in valid_currencies:
        raise ValueError(f'Currency must be one of
```

{valid_currencies}')

```
return v

@validator('status')
def status_must_be_valid(cls, v):
    valid_statuses = ['pending', 'confirmed', 'shipped',
```

'delivered', 'cancelled']

```
if v not in valid_statuses:
    raise ValueError(f'Status must be one of
```

```
{valid_statuses}')
```

```
return v

# Validate incoming data
def validate_order_batch(df):
    """Validate entire batch of orders"""
    validation_results = {
        'valid_records': [],
        'invalid_records': [],
        'validation_errors': []
    }

    for index, row in df.iterrows():
        try:
            # Validate each record
            order = OrderRecord(**row.to_dict())
            validation_results['valid_records'].append(order.dict())
        except Exception as e:
            validation_results['invalid_records'].append({
                'row_index': index,
                'data': row.to_dict(),
                'error': str(e)
            })
            validation_results['validation_errors'].append(str(e))

    return validation_results
```

PYTHON

Level 2: Business Rule Validation

```
def validate_business_rules(df):
    """Validate complex business rules that span multiple fields"""

    issues = []

    # Rule 1: Order amounts should be reasonable for the customer
```

PYTHON

segment

```
high_value_orders = df[df['amount'] > 50000]
for _, order in high_value_orders.iterrows():
    customer_history =
```

```
get_customer_order_history(order['customer_id'])
```

```
avg_order_value = customer_history['amount'].mean()
```

```
if order['amount'] > avg_order_value * 10:
    issues.append({
        'rule': 'unusual_order_amount',
        'order_id': order['order_id'],
```

```
        'amount': order['amount'],
        'customer_avg': avg_order_value,
        'severity': 'warning'
    })
```

```
# Rule 2: Orders shouldn't be backdated more than 7 days
df['order_date_parsed'] = pd.to_datetime(df['order_date'])
today = datetime.now()
old_orders = df[df['order_date_parsed'] < today -
```

```
timedelta(days=7)]
```

```
for _, order in old_orders.iterrows():
    issues.append({
        'rule': 'backdated_order',
        'order_id': order['order_id'],
        'order_date': order['order_date'],
        'days_old': (today - order['order_date_parsed']).days,
        'severity': 'error'
    })
```

```
# Rule 3: Customer should exist in customer database
unique_customers = df['customer_id'].unique()
existing_customers = set(get_existing_customer_ids())
missing_customers = set(unique_customers) - existing_customers
```

```
if missing_customers:
    for customer_id in missing_customers:
        issues.append({
            'rule': 'unknown_customer',
            'customer_id': customer_id,
            'affected_orders': df[df['customer_id'] ==
```

```
customer_id]['order_id'].tolist(),
```

```
    'severity': 'error'
    })
```

```
    return issues
```

Level 3: Statistical Anomaly Detection

PYTHON

```
import numpy as np
from scipy import stats

class AnomalyDetector:
    """Statistical anomaly detection for data quality"""

    def __init__(self, historical_window_days=30):
        self.historical_window_days = historical_window_days

    def detect_volume_anomalies(self, current_date, current_count):
        """Detect if today's data volume is anomalous"""

        # Get historical data for same day of week
        historical_counts = self.get_historical_counts_for_dow(
            current_date, self.historical_window_days
        )

        if len(historical_counts) < 5: # Not enough history
            return {'anomaly': False, 'reason':
```

```
'insufficient_history'}
```

```
        # Z-score based detection
        mean_count = np.mean(historical_counts)
        std_count = np.std(historical_counts)
        z_score = abs((current_count - mean_count) / std_count) if
```

```
std_count > 0 else 0
```

```
# Configurable thresholds
if z_score > 3: # 3 standard deviations
    return {
        'anomaly': True,
        'severity': 'critical',
        'z_score': z_score,
        'current_count': current_count,
```

```
'expected_range': (mean_count - 2*std_count,
```

```
mean_count + 2*std_count)
```

```
}
elif z_score > 2: # 2 standard deviations
    return {
        'anomaly': True,
        'severity': 'warning',
        'z_score': z_score,
        'current_count': current_count,
        'expected_range': (mean_count - 2*std_count,
```

PYTHON

```
mean_count + 2*std_count)
```

```
}

    return {'anomaly': False}

def detect_distribution_shift(self, current_data,
```

PYTHON

```
feature_column):
```

```
"""Detect if the distribution of a feature has shifted"""

    historical_data = self.get_historical_feature_data(
        feature_column, self.historical_window_days
    )

    if len(historical_data) < 1000: # Need enough data for
```

PYTHON

```
statistical tests
```

```

return {'shift': False, 'reason': 'insufficient_data'}

# Kolmogorov-Smirnov test for distribution change
ks_statistic, p_value = stats.ks_2samp(historical_data,

```

current_data)

```

if p_value < 0.01: # Significant shift detected
    return {
        'shift': True,
        'test': 'ks_test',
        'ks_statistic': ks_statistic,
        'p_value': p_value,
        'interpretation': 'Significant distribution change

```

detected'

```

}

return {'shift': False}

```

Quality Gates and Circuit Breakers

```

class DataQualityGate:
    """Quality gate that stops processing if data quality is too

```

PYTHON

poor"""

```

def __init__(self, config):
    self.config = config
    self.quality_history = []

def evaluate_quality(self, data, validation_results):
    """Evaluate overall data quality and decide whether to

```

PYTHON

proceed"""


```

total_records = len(data)
valid_records = len(validation_results['valid_records'])
invalid_records = len(validation_results['invalid_records'])

quality_score = valid_records / total_records if

```

total_records > 0 else 0

```

# Different thresholds for different pipeline criticality
min_quality_score = self.config.get('min_quality_score',

```

PYTHON

0.95)

```

max_invalid_records = self.config.get('max_invalid_records',

```

PYTHON

100)

```

quality_check = {
    'timestamp': datetime.now(),

```

```

    'total_records': total_records,
    'valid_records': valid_records,
    'invalid_records': invalid_records,
    'quality_score': quality_score,
    'passed': False,
    'issues': []
}

# Check quality thresholds
if quality_score < min_quality_score:
    quality_check['issues'].append({
        'type': 'low_quality_score',
        'message': f'Quality score {quality_score:.2%} below

```

threshold {min_quality_score:.2%}'

```

    })

    if invalid_records > max_invalid_records:
        quality_check['issues'].append({
            'type': 'too_many_invalid_records',
            'message': f'Invalid records {invalid_records} above

```

threshold {max_invalid_records}'

```

    })

    # Trend-based quality checks
    if len(self.quality_history) >= 3:
        recent_scores = [q['quality_score'] for q in

```

PYTHON

```

self.quality_history[-3:]]

```

```

    if all(score < quality_score * 0.9 for score in

```

recent_scores):

```

        quality_check['issues'].append({
            'type': 'quality_trend_decline',
            'message': 'Data quality has been declining over

```

last 3 runs'

```

    })

    # Overall decision
    quality_check['passed'] = len(quality_check['issues']) == 0

    # Store for trend analysis
    self.quality_history.append(quality_check)
    if len(self.quality_history) > 30: # Keep only last 30 runs
        self.quality_history = self.quality_history[-30:]

    return quality_check

def handle_quality_failure(self, quality_check,

```

pipeline_config):

```

    """Handle quality gate failure based on pipeline

```

criticality"""

```

    criticality = pipeline_config.get('criticality', 'medium')

    if criticality == 'critical':
        # Stop pipeline completely for critical systems
        raise DataQualityException(f"Critical pipeline failed

```

quality gate: {quality_check['issues']})

```

    elif criticality == 'high':
        # Process valid records only, alert about issues
        self.send_quality_alert(quality_check, severity='high')
        return 'process_valid_only'

    else:
        # Log issues but continue processing for low criticality
        self.send_quality_alert(quality_check, severity='low')
        return 'continue_with_warnings'

```

Data Quality Monitoring

Real-time Quality Dashboards:

```
class DataQualityMetrics:
```

PYTHON

Collect and expose data quality metrics:

```
"""Collect and expose data quality metrics"""

def __init__(self, metrics_backend):
    self.metrics = metrics_backend

def record_pipeline_quality(self, pipeline_name,
```

PYTHON

```
quality_results):
```

```

"""Record quality metrics for monitoring dashboards"""

# Record basic quality metrics
self.metrics.gauge(
    'data_quality.quality_score',
    quality_results['quality_score'],
    tags={'pipeline': pipeline_name}
)

self.metrics.gauge(
    'data_quality.record_count',
    quality_results['total_records'],
    tags={'pipeline': pipeline_name}
)

self.metrics.gauge(
    'data_quality.invalid_record_count',
    quality_results['invalid_records'],
    tags={'pipeline': pipeline_name}
)

# Record specific validation failures
for issue in quality_results.get('issues', []):
    self.metrics.increment(
        'data_quality.validation_failures',
        tags={
            'pipeline': pipeline_name,
            'issue_type': issue['type']
        }
    )

# Record quality gate pass/fail
self.metrics.increment(
    'data_quality.gate_result',
    tags={
        'pipeline': pipeline_name,
        'result': 'pass' if quality_results['passed'] else

```

'fail'

```

    }
    )

    def record_anomaly_detection(self, pipeline_name,

```

anomaly_results):

```
"""Record anomaly detection results"""

for anomaly_type, result in anomaly_results.items():
    if result.get('anomaly') or result.get('shift'):
        self.metrics.increment(
            'data_quality.anomaly_detected',
            tags={
                'pipeline': pipeline_name,
                'anomaly_type': anomaly_type,
                'severity': result.get('severity',
```

PYTHON

'unknown')

```
    }
)
```

Data Lineage and Impact Analysis

```
class DataLineageTracker:
    """Track data lineage to understand quality issue impact"""

    def __init__(self):
        self.lineage_graph = {}
        self.quality_events = []
```

PYTHON

The register_data_dependency implementation:

```
def register_data_dependency(self, source_table, target_table,
```

PYTHON

transformation):

```

"""Register a data dependency relationship"""
    if source_table not in self.lineage_graph:
        self.lineage_graph[source_table] = []

    self.lineage_graph[source_table].append({
        'target': target_table,
        'transformation': transformation,
        'registered_at': datetime.now()
    })

def propagate_quality_issue(self, source_table, issue_details):
    """Find all downstream tables affected by a quality issue"""

    affected_tables = set()

    def find_downstream(table, path=[]):
        if table in path: # Prevent cycles
            return

        path = path + [table]
        affected_tables.add(table)

        for dependency in self.lineage_graph.get(table, []):
            find_downstream(dependency['target'], path)

    find_downstream(source_table)

    # Record the quality event
    quality_event = {
        'timestamp': datetime.now(),
        'source_table': source_table,
        'issue': issue_details,
        'affected_tables': list(affected_tables),
        'impact_scope': len(affected_tables)
    }

    self.quality_events.append(quality_event)

    # Send alerts about downstream impact
    if len(affected_tables) > 5: # High impact
        self.send_impact_alert(quality_event, severity='high')
    elif len(affected_tables) > 1: # Medium impact
        self.send_impact_alert(quality_event, severity='medium')

    return quality_event

```

Great Expectations Integration

PYTHON

```
# Great Expectations configuration for automated data quality
import great_expectations as ge

# Define expectations for orders table
def create_orders_expectations():
    """Define data quality expectations for orders data"""

    # Load data into Great Expectations DataFrame
    orders_df = ge.dataset.PandasDataset(df)

    # Basic expectations
    orders_df.expect_table_row_count_to_be_between(min_value=1000,
```

```
max_value=100000)
```

```
orders_df.expect_column_values_to_not_be_null('order_id')
orders_df.expect_column_values_to_be_unique('order_id')
orders_df.expect_column_values_to_not_be_null('customer_id')
orders_df.expect_column_values_to_not_be_null('amount')
orders_df.expect_column_values_to_be_of_type('amount', 'float')
orders_df.expect_column_values_to_be_between('amount',
```

```
min_value=0.01, max_value=1000000)
```

```
# Advanced expectations
orders_df.expect_column_values_to_be_in_set('currency', ['USD',
```

```
'EUR', 'GBP', 'CAD'])
```

```
orders_df.expect_column_values_to_be_in_set('status',
    ['pending', 'confirmed', 'shipped', 'delivered',
```

```
'cancelled'])
```



```
# Custom expectation for date format
orders_df.expect_column_values_to_match_regex('order_date',
```

```
r'^\d{4}-\d{2}-\d{2}$')
```

```
# Business rule expectations
```

```
orders_df.expect_column_pair_values_A_to_be_greater_than_B('order_date',
'2020-01-01')
```

```
return orders_df.get_expectation_suite()

# Run expectations in data pipeline
def validate_with_great_expectations(df, expectation_suite):
    """Validate data using Great Expectations"""

    # Convert to GE DataFrame
    ge_df = ge.dataset.PandasDataset(df)

    # Run validation
    validation_results =
```

PYTHON

```
ge_df.validate(expectation_suite=expectation_suite)
```

```
# Process results
success_rate = validation_results.meta['success_percent']
failed_expectations = [
    result for result in validation_results.results
    if not result['success']]

return {
    'success': validation_results.success,
    'success_rate': success_rate,
    'failed_expectations': failed_expectations,
    'detailed_results': validation_results
}
```

Pro Tip Start with simple quality checks (schema validation,

null checks) and gradually add more sophisticated validation.

It's better to have basic quality monitoring in place than to spend months building a quality framework that never gets deployed.



- ☐ Checkpoint
1. What should you do when data quality issues are detected? a. Always stop the pipeline completely b. Always continue processing c. Decide based on pipeline criticality and issue severity d. Ignore minor issues
 2. What's the purpose of a data quality gate? a. To make pipelines run faster b. To decide whether data is good enough to proceed with

processing

c. To fix data quality issues automatically d. To send email alerts 3. Why is data lineage important for data quality? a. It makes queries run faster b. It helps understand which downstream systems are affected by

quality issues

c. It reduces storage costs d. It improves data security

Answers: 1-c, 2-b, 3-b

3.5 Performance and Cost Optimization

⚡ TL;DR

Performance optimization starts with understanding your bottlenecks - measure before optimizing - Most performance issues are caused by data skew, inefficient joins, or poor partitioning - Cost optimization is about right-sizing resources and avoiding unnecessary compute - Spot instances can reduce costs by 50-90% for fault-tolerant batch workloads - The fastest code is code that doesn't run - eliminate unnecessary work first

Performance and cost are the yin and yang of batch processing. Push too hard on performance and costs explode. Optimize too aggressively for cost and performance suffers. The art is finding the sweet spot for your specific workloads.

Understanding Performance Bottlenecks

The Performance Debugging Framework:

```

import time
import psutil
from functools import wraps

class PerformanceProfiler:
    """Profile batch job performance to identify bottlenecks"""

    def __init__(self):
        self.metrics = {}

    def profile_function(self, func_name):
        """Decorator to profile function execution"""
        def decorator(func):
            @wraps(func)
            def wrapper(*args, **kwargs):
                start_time = time.time()
                start_memory = psutil.virtual_memory().used

                try:
                    result = func(*args, **kwargs)
                    success = True
                except Exception as e:
                    result = e
                    success = False

                end_time = time.time()
                end_memory = psutil.virtual_memory().used

                # Record metrics
                self.metrics[func_name] = {
                    'execution_time': end_time - start_time,
                    'memory_delta': end_memory - start_memory,
                    'success': success,
                    'timestamp': datetime.now()
                }

                if not success:
                    raise result

                return result
            return wrapper
        return decorator

    def get_performance_summary(self):
        """Get summary of performance metrics"""
        return {
            'total_execution_time': sum(m['execution_time'] for m in

```

```
self.metrics.values()),
```

```
'peak_memory_usage': max(m['memory_delta'] for m in
```

```
self.metrics.values()),
```

```
'slowest_functions': sorted(
```

The corresponding SQL query:

```
self.metrics.items(),
    key=lambda x: x[1]['execution_time'],
    reverse=True
)[:5],
'memory_intensive_functions': sorted(
    self.metrics.items(),
    key=lambda x: x[1]['memory_delta'],
    reverse=True
)[:5]
}

# Usage in batch pipeline
profiler = PerformanceProfiler()

@profiler.profile_function('extract_orders')
def extract_orders(date):
    """Extract orders with performance profiling"""
    return db.query(f"SELECT * FROM orders WHERE date = '{date}'")

@profiler.profile_function('transform_orders')
def transform_orders(orders_df):
    """Transform orders with performance profiling"""
    # Transformation logic here
    return transformed_df
```

SQL

Common Performance Patterns:

Pattern 1: Data Skew Detection

```
def detect_data_skew(df, partition_column):
    """Detect if data is skewed across partitions"""

    partition_counts =
```

```
df.groupBy(partition_column).count().collect()
```

```
counts = [row['count'] for row in partition_counts]

if not counts:
    return {'skewed': False}

max_count = max(counts)
min_count = min(counts)
avg_count = sum(counts) / len(counts)

# Consider skewed if max partition has 5x more data than average
skew_ratio = max_count / avg_count if avg_count > 0 else 0

return {
    'skewed': skew_ratio > 5.0,
    'skew_ratio': skew_ratio,
    'max_partition_size': max_count,
    'min_partition_size': min_count,
    'avg_partition_size': avg_count,
    'recommendation': 'Consider salting or different partition
```

```
key' if skew_ratio > 5.0 else 'Partitioning looks good'
```

```
}
```

Pattern 2: Join Optimization Analysis

```
def analyze_join_performance(left_df, right_df, join_keys):
    """Analyze join characteristics to optimize performance"""

    analysis = {}

    # Check join key cardinality
    left_cardinality = left_df.select(join_keys).distinct().count()
    right_cardinality =
```

```
right_df.select(join_keys).distinct().count()
```

```
left_total = left_df.count()
right_total = right_df.count()

analysis['join_selectivity'] = {
    'left_cardinality': left_cardinality,
    'right_cardinality': right_cardinality,
    'left_total_rows': left_total,
    'right_total_rows': right_total,
```

SQL

The corresponding SQL query:

```
'left_key_selectivity': left_cardinality / left_total if
```

SQL

```
left_total > 0 else 0,
```

```
'right_key_selectivity': right_cardinality / right_total if
```

SQL

```
right_total > 0 else 0
```

```
}

# Recommend join strategy
if right_total < 10000: # Small table
    analysis['recommended_strategy'] = 'broadcast_join'
    analysis['reason'] = 'Right table is small enough for
```

SQL

```
broadcast'
```

```
elif analysis['join_selectivity']['left_key_selectivity'] < 0.1:
    analysis['recommended_strategy'] = 'filter_pushdown'
    analysis['reason'] = 'Low selectivity - filter before join'
else:
    analysis['recommended_strategy'] = 'sort_merge_join'
    analysis['reason'] = 'Standard sort-merge join appropriate'

return analysis
```

SQL

Spark Optimization Strategies

```
# Spark configuration for different workload types
class SparkOptimizer:
    """Optimize Spark configuration for different batch workloads"""

    @staticmethod
    def get_config_for_workload(workload_type, data_size_gb,
```

PYTHON

cluster_size):

```
    """Get optimized Spark config for specific workload"""

    base_config = {
        "spark.sql.adaptive.enabled": "true",
        "spark.sql.adaptive.coalescePartitions.enabled": "true",
        "spark.serializer":
```

PYTHON

```
"org.apache.spark.serializer.KryoSerializer",
```

```
"spark.sql.adaptive.skewJoin.enabled": "true"
    }

    if workload_type == 'etl_heavy':
        # ETL with lots of transformations
        config = {
            **base_config,
            "spark.sql.adaptive.advisoryPartitionSizeInBytes":
```

SQL

"128MB",

```
"spark.sql.shuffle.partitions": str(cluster_size *
```

PYTHON

4),

```
"spark.sql.broadcastTimeout": "600s",
"spark.sql.adaptive.localShuffleReader.enabled":
```

PYTHON

"true"

```
}

elif workload_type == 'join_heavy':
    # Workload with many joins
    config = {
        **base_config,
        "spark.sql.autoBroadcastJoinThreshold": "100MB",
        "spark.sql.join.preferSortMergeJoin": "true",
```

PYTHON

"spark.sql.adaptive.optimizeSkewsInRebalancePartitions.enabled": "true",

"spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes": "256MB"

```
}

elif workload_type == 'aggregation_heavy':
    # Workload with lots of GROUP BY operations
    config = {
        **base_config,
        "spark.sql.adaptive.advisoryPartitionSizeInBytes":
```

PYTHON

"256MB",

"spark.sql.adaptive.coalescePartitions.minPartitionNum": "1",

"spark.sql.adaptive.coalescePartitions.parallelismFirst": "false"

```
}

else: # general purpose
    config = base_config

# Adjust for data size
if data_size_gb > 1000:    # Large datasets
```

```
config["spark.sql.adaptive.advisoryPartitionSizeInBytes"] = "512MB"
```

```
config["spark.sql.shuffle.partitions"] =
```

PYTHON

```
str(cluster_size * 8)
```

PYTHON

```
return config

# Example usage
def run_optimized_spark_job(workload_type, input_path, output_path):
    """Run Spark job with workload-specific optimizations"""

    # Estimate data size
    data_size_gb = estimate_data_size(input_path)
    cluster_size = get_cluster_size()

    # Get optimized config
    spark_config = SparkOptimizer.get_config_for_workload(
        workload_type, data_size_gb, cluster_size
    )

    # Create Spark session with optimizations
    spark = SparkSession.builder \
        .appName(f"OptimizedJob-{workload_type}") \
        .config(spark_config) \
        .getOrCreate()

    # Set optimal number of partitions based on data size
    optimal_partitions = calculate_optimal_partitions(data_size_gb,
```

cluster_size)

PYTHON

```
# Read and process data
df = spark.read.parquet(input_path)

if df.rdd.getNumPartitions() != optimal_partitions:
    df = df.repartition(optimal_partitions)

# Your transformation logic here
result_df = process_data(df)

# Write with optimal partitioning for downstream consumption
result_df.write \
    .mode("overwrite") \
    .option("maxRecordsPerFile", 1000000) \
    .parquet(output_path)
```

Cost Optimization Strategies

Spot Instance Strategy for Batch Processing:

```
import boto3
from datetime import datetime, timedelta

class SpotInstanceManager:
    """Manage Spot instances for cost-effective batch processing"""

    def __init__(self):
        self.ec2 = boto3.client('ec2')
        self.emr = boto3.client('emr')

    def get_spot_price_history(self, instance_type,
```

PYTHON

availability_zone, days=7):

```
    """Get recent spot price history to make bidding
```

decisions"""

```
        end_time = datetime.utcnow()
        start_time = end_time - timedelta(days=days)

        response = self.ec2.describe_spot_price_history(
            InstanceTypes=[instance_type],
            ProductDescriptions=['Linux/UNIX'],
            AvailabilityZone=availability_zone,
            StartTime=start_time,
            EndTime=end_time
        )

        prices = [float(item['SpotPrice']) for item in
```

PYTHON

```
        response['SpotPriceHistory']]
```

PYTHON

```

return {
    'min_price': min(prices),
    'max_price': max(prices),
    'avg_price': sum(prices) / len(prices),
    'current_price': prices[0] if prices else None,
    'price_volatility': max(prices) - min(prices)
}

def calculate_optimal_bid(self, instance_type,

```

availability_zone, target_reliability=0.95):

PYTHON

```

"""Calculate optimal bid price for target reliability"""

price_history = self.get_spot_price_history(instance_type,

```

availability_zone)

```

# Bid at 95th percentile of recent prices for high

```

reliability

PYTHON

```

if target_reliability >= 0.95:
    bid_price = price_history['max_price'] * 0.95
elif target_reliability >= 0.90:
    bid_price = price_history['avg_price'] * 1.5
else:
    bid_price = price_history['avg_price'] * 1.2

# Get on-demand price for comparison
on_demand_price = self.get_on_demand_price(instance_type)
potential_savings = (on_demand_price - bid_price) /

```

on_demand_price

```

return {
    'recommended_bid': bid_price,
    'on_demand_price': on_demand_price,
    'potential_savings': potential_savings,
    'price_history': price_history
}

def create_fault_tolerant_emr_cluster(self, job_config):
    """Create EMR cluster with mix of on-demand and spot

```

instances"""

```

# Master node always on-demand for stability
master_instance = {
    'Name': 'Master',
    'InstanceRole': 'MASTER',
    'InstanceType': 'm5.xlarge',
    'InstanceCount': 1,
    'Market': 'ON_DEMAND'
}

# Core nodes on-demand for HDFS reliability
core_instances = {
    'Name': 'Core',
    'InstanceRole': 'CORE',
    'InstanceType': 'm5.2xlarge',
    'InstanceCount': 2,

```

The PYTHON code:

```

'Market': 'ON_DEMAND'
}

# Task nodes on spot for cost savings
task_instances = {
    'Name': 'Task',
    'InstanceRole': 'TASK',
    'InstanceType': 'm5.4xlarge',
    'InstanceCount': 8,
    'Market': 'SPOT',
    'BidPrice': str(self.calculate_optimal_bid('m5.4xlarge',

```

'us-east-1a')['recommended_bid'])

```
}

cluster_config = {
    'Name': f"BatchProcessing-
```

```
{datetime.now().strftime('%Y%m%d-%H%M')}"
```

```
'InstanceFleets': [master_instance, core_instances,
```

```
task_instances],
```

```
'LogUri': 's3://my-emr-logs/',
    'ReleaseLabel': 'emr-6.9.0',
    'Applications': [{'Name': 'Spark'}, {'Name': 'Hadoop'}],
    'Configurations': self.get_cost_optimized_config()
}

return self.emr.run_job_flow(**cluster_config)
```

PYTHON

Auto-scaling for Cost Optimization:

```

class CostOptimizedAutoScaler:
    """Auto-scaler that optimizes for cost while meeting SLAs"""

    def __init__(self, min_nodes=2, max_nodes=50, target_cpu=70):
        self.min_nodes = min_nodes
        self.max_nodes = max_nodes
        self.target_cpu = target_cpu
        self.scaling_history = []

    def calculate_scaling_decision(self, current_metrics):
        """Decide whether to scale up, down, or stay the same"""

        current_nodes = current_metrics['node_count']
        avg_cpu = current_metrics['avg_cpu_utilization']
        queue_length = current_metrics.get('pending_tasks', 0)
        cost_per_hour = current_metrics['current_cost_per_hour']

        decision = {
            'action': 'no_change',
            'target_nodes': current_nodes,
            'reason': 'Metrics within acceptable range'
        }

        # Scale up conditions
        if avg_cpu > 85 and current_nodes < self.max_nodes:
            # High CPU utilization
            target_nodes = min(current_nodes + 2, self.max_nodes)
            decision = {
                'action': 'scale_up',
                'target_nodes': target_nodes,
                'reason': f'High CPU utilization: {avg_cpu}%'
            }

        elif queue_length > 100 and current_nodes < self.max_nodes:
            # Long queue suggests need for more compute
            additional_nodes = min(queue_length // 50, 5) # Roughly

```

50 tasks per node

```
target_nodes = min(current_nodes + additional_nodes,
```

```
self.max_nodes)
```

```
decision = {
    'action': 'scale_up',
    'target_nodes': target_nodes,
    'reason': f'Long task queue: {queue_length} pending'
```

The PYTHON code:

```
}

# Scale down conditions
elif avg_cpu < 30 and current_nodes > self.min_nodes and
```

PYTHON

```
queue_length == 0:
```

```
# Low utilization and no pending work
target_nodes = max(current_nodes - 1, self.min_nodes)
potential_savings = (current_nodes - target_nodes) *
```

PYTHON

cost_per_hour

```
decision = {
    'action': 'scale_down',
    'target_nodes': target_nodes,
    'reason': f'Low utilization: {avg_cpu}%, potential
```

```
savings: ${potential_savings:.2f}/hour'
```



```

    }

    # Record decision for analysis
    self.scaling_history.append({
        'timestamp': datetime.now(),
        'decision': decision,
        'metrics': current_metrics
    })

    return decision

def implement_scaling_decision(self, decision, cluster_id):
    """Actually implement the scaling decision"""

    if decision['action'] == 'no_change':
        return

    try:
        if decision['action'] == 'scale_up':
            self.add_nodes(cluster_id, decision['target_nodes'])

```

- decision['current_nodes'])

```

elif decision['action'] == 'scale_down':
    self.remove_nodes(cluster_id,

```

```

decision['current_nodes'] - decision['target_nodes'])

```

```

# Log the scaling action
self.log_scaling_action(decision)

except Exception as e:
    self.send_scaling_alert(f"Failed to implement scaling

```

decision: {e}")

Resource Right-Sizing

```
class ResourceOptimizer:
    """Analyze historical usage to recommend optimal resource
```

PYTHON

```
sizing"""
```

```
def __init__(self):
    self.usage_history = []

def collect_usage_metrics(self, job_id, metrics):
    """Collect resource usage metrics for analysis"""

    self.usage_history.append({
        'job_id': job_id,
        'timestamp': datetime.now(),
        'metrics': {
            'cpu_utilization': metrics.get('cpu_utilization',
```

PYTHON

```
[]),
```

```
    'memory_utilization':
```

```
        metrics.get('memory_utilization', []),
```

```
        'disk_utilization': metrics.get('disk_utilization',
```

```
[]),
```

```
        'network_utilization':
```

```
        metrics.get('network_utilization', []),
```

```
        'job_duration': metrics.get('job_duration', 0),
```

```
'data_processed_gb':
```

```
metrics.get('data_processed_gb', 0),
```

```
'cost': metrics.get('cost', 0)
    }
})
```

PYTHON

```
def analyze_resource_efficiency(self, job_pattern,
```

days_back=30):

```
"""Analyze resource efficiency for specific job pattern"""
```

PYTHON

```
# Filter relevant jobs
cutoff_date = datetime.now() - timedelta(days=days_back)
relevant_jobs = [
    job for job in self.usage_history
    if job['timestamp'] > cutoff_date and job_pattern in
```

```
job['job_id']
```

```
]

if len(relevant_jobs) < 5:
    return {'error': 'Not enough historical data'}
```

```
# Calculate efficiency metrics
cpu_utilizations = []
memory_utilizations = []
costs_per_gb = []
```

```
for job in relevant_jobs:
    metrics = job['metrics']

    # Average utilization during job
    if metrics['cpu_utilization']:
```

```
cpu_utilizations.append(np.mean(metrics['cpu_utilization']))
```

```
if metrics['memory_utilization']:
```

```
memory_utilizations.append(np.mean(metrics['memory_utilization']))
```

```
# Cost efficiency
if metrics['data_processed_gb'] > 0:
    costs_per_gb.append(metrics['cost'] /
```

```
metrics['data_processed_gb'])
```

```
analysis = {
    'job_count': len(relevant_jobs),
    'avg_cpu_utilization': np.mean(cpu_utilizations) if
```

```
cpu_utilizations else 0,
```

```
'avg_memory_utilization': np.mean(memory_utilizations)
```

```
if memory_utilizations else 0,
```

```
'avg_cost_per_gb': np.mean(costs_per_gb) if costs_per_gb
```

```
else 0,
```

```
'cpu_waste': max(0, 100 - np.mean(cpu_utilizations)) if
```

```
cpu_utilizations else 0,
```

```
'memory_waste': max(0, 100 -
```

```
np.mean(memory_utilizations)) if memory_utilizations else 0
```

```
}

# Generate recommendations
recommendations = []

if analysis['avg_cpu_utilization'] < 50:
    recommendations.append({
        'type': 'downsize_cpu',
        'current_utilization':
```

```
analysis['avg_cpu_utilization'],
```

```
'recommendation': 'Consider using smaller instance
```

types or fewer cores',

```
'potential_savings': f"{analysis['cpu_waste']:.1f}%
```

CPU cost reduction"

```
})

if analysis['avg_memory_utilization'] < 60:
    recommendations.append({
```

```
'type': 'downsize_memory',
'current_utilization':
```

```
analysis['avg_memory_utilization'],
```

```
'recommendation': 'Consider memory-optimized
```

instances with less RAM',

```
'potential_savings': f"
```

```
{analysis['memory_waste']:.1f}% memory cost reduction"
```

```
})
```

```
if analysis['avg_cpu_utilization'] > 85:  
    recommendations.append({  
        'type': 'upscale_cpu',  
        'current_utilization':
```

```
analysis['avg_cpu_utilization'],
```

```
'recommendation': 'Consider larger instance types
```

```
for better performance',
```

```
'benefit': 'Faster job completion, reduced queue
```

```
time'
```

```
})
```

```
analysis['recommendations'] = recommendations  
return analysis
```

Performance Monitoring and Alerting

PYTHON

```
class PerformanceMonitor:
    """Monitor batch job performance and detect regressions"""

    def __init__(self, alerting_backend):
        self.alerting = alerting_backend
        self.performance_baselines = {}

    def establish_baseline(self, job_name, historical_runs):
        """Establish performance baseline from historical data"""

        if len(historical_runs) < 10:
            return {'error': 'Need at least 10 runs to establish
```

```
baseline'}
```

```
        durations = [run['duration'] for run in historical_runs]
        costs = [run['cost'] for run in historical_runs]
        throughputs = [run['data_processed_gb'] / run['duration']
                       for run in historical_runs if run['duration']
```

```
> 0]
```

```
        baseline = {
            'median_duration': np.median(durations),
            'p95_duration': np.percentile(durations, 95),
            'median_cost': np.median(costs),
            'median_throughput': np.median(throughputs) if
```

```
throughputs else 0,
```

```

    'established_at': datetime.now(),
    'sample_size': len(historical_runs)
}

self.performance_baselines[job_name] = baseline
return baseline

def check_performance_regression(self, job_name, current_run):
    """Check if current run shows performance regression"""

    if job_name not in self.performance_baselines:
        return {'status': 'no_baseline'}

    baseline = self.performance_baselines[job_name]
    alerts = []

    # Duration regression check
    if current_run['duration'] > baseline['p95_duration']:
        alerts.append({
            'type': 'duration_regression',
            'severity': 'warning',

```

```

    'current': current_run['duration'],
    'baseline_p95': baseline['p95_duration'],
    'regression_factor': current_run['duration'] /

```

```

baseline['median_duration']

```

```

}))

# Cost regression check
cost_threshold = baseline['median_cost'] * 1.5    # 50% cost

```

increase


```
if current_run['cost'] > cost_threshold:
    alerts.append({
        'type': 'cost_regression',
        'severity': 'warning',
        'current_cost': current_run['cost'],
        'baseline_median': baseline['median_cost'],
        'cost_increase_factor': current_run['cost'] /
```

```
baseline['median_cost']
```

```
    })

    # Throughput regression check
    if baseline['median_throughput'] > 0:
        current_throughput = current_run['data_processed_gb'] /
```

```
current_run['duration']
```

```
throughput_threshold = baseline['median_throughput'] *
```

```
0.7 # 30% decrease
```

```
if current_throughput < throughput_threshold:
    alerts.append({
        'type': 'throughput_regression',
        'severity': 'warning',
        'current_throughput': current_throughput,
        'baseline_median':
```

```
baseline['median_throughput'],
```

```
'throughput_decrease_factor':
```

```
baseline['median_throughput'] / current_throughput
```

```

}))

# Send alerts if any regressions detected
for alert in alerts:
    self.send_regression_alert(job_name, alert)

return {
    'status': 'checked',
    'regressions_detected': len(alerts),
    'alerts': alerts
}

```

- ⚠ Common Mistake
- ⚠ Common Mistake Don't optimize prematurely. Profile your actual workloads first. The most expensive optimization is optimizing the wrong thing. Measure twice, optimize once.

- ❑ Checkpoint
1. What's the first step in performance optimization? a. Switch to faster hardware b. Measure and identify bottlenecks c. Rewrite code in a faster language d. Add more memory
 2. When should you use Spot instances for batch processing? a. For all workloads to save money b. For fault-tolerant workloads that can handle interruptions c. Only for development environments d. Never - they're too unreliable
 3. What indicates data skew in a distributed processing job? a. All partitions have the same amount of data b. Some partitions have significantly more data than others c. The job runs quickly d. Memory usage is low

Answers: 1-b, 2-b, 3-b

Try It Yourself: Exercises

Exercise 1: Batch Processing Pattern Selection (30 minutes)

Scenario: You're designing a data platform for a financial services company. They have:

- Real-time fraud detection requirements (sub- second)
- Daily regulatory reports (can be hours old)
- Customer analytics dashboards (can be 15 minutes old)
- Risk models that need to be retrained weekly

Your Task: 1. Choose between Lambda, Kappa, or Hybrid architecture 2. Justify your choice with specific requirements mapping 3. Design the high-level data flow 4. Identify which components would be batch vs stream processing

Deliverable: A 1-page architecture decision document

Exercise 2: ETL vs ELT Decision Matrix (45 minutes)

Scenario: You need to process customer data for three different use cases: 1. Real-time personalization: Website recommendations updated as users browse 2. Business intelligence: Daily reports for executives on customer behavior 3. Data science: Monthly customer segmentation models using full historical data

Your Task: 1. For each use case, decide between ETL or ELT approach 2. Design the data transformations required 3. Estimate processing latency and cost implications 4. Write pseudo-code for the critical transformations

Deliverable: Comparison table with reasoning for each approach

Exercise 3: Data Quality Framework Design (60 minutes)

Scenario: Your e-commerce data pipeline processes 1M orders daily. Recent issues include:

- Orders with negative amounts (data entry errors)
- Duplicate order IDs (system bugs)

- Orders for non-existent customers (integration issues)
- Unusual spikes in order volume on weekends

Your Task: 1. Design a multi-level data quality framework 2. Write Python code for key validation rules 3. Define quality gates and escalation procedures 4. Design monitoring dashboards for data quality metrics

Deliverable: Complete data quality framework with code samples

Module 3 Project: Financial Reconciliation System

Objective

Design and architect a comprehensive batch processing system for financial reconciliation that handles daily transaction processing, multi-system data integration, and regulatory reporting.

Business Context

You're building a financial reconciliation system for a payment processor that:

- Processes 10M transactions daily across multiple payment methods
- Must reconcile data from 15+ banking partners with different formats and timing
- Generates regulatory reports with zero-tolerance for errors
- Provides daily financial statements to executives
- Must handle month-end processing with 50x normal data volumes

Requirements

Functional Requirements: - Daily reconciliation of transactions

- across all banking partners
- Real-time alerting for reconciliation discrepancies > \$1000
- Monthly regulatory reports (SOX compliance)
- Executive dashboards with daily P&L statements

- Historical data reprocessing capabilities for audits

Non-Functional Requirements:

- Process 10M transactions daily, scale to 500M during peak periods
- 99.9% accuracy for financial calculations
- Daily reconciliation must complete by 8 AM ET
- Month- end processing must complete within 48 hours
- Full audit trail for 7 years of transaction data

Deliverables

- 1. Architecture Design Document (4-6 pages)
 - High-level system architecture
 - Batch processing workflow design
 - Technology stack selection and justification
 - Data flow diagrams
- 2. Implementation Specifications (6-8 pages)
 - Detailed job orchestration plan (DAG design)
 - Data quality framework specific to financial data
 - Error handling and retry strategies
 - Performance optimization plan
- 3. Operational Plan (3-4 pages)
 - Monitoring and alerting strategy
 - SLA definitions and measurement
 - Disaster recovery procedures -

Cost analysis and optimization plan

Evaluation Criteria

Architecture Quality (40 points) -

- ☐ Appropriate batch processing pattern selection -
- ☐ Scalable and fault-tolerant design -
- ☐ Clear separation of concerns -
- ☐ Technology choices justified by requirements

Data Quality & Reliability (30 points) -

- ☐ Comprehensive data validation framework -
- ☐ Financial calculation accuracy measures -
- ☐ Audit trail and compliance considerations -
- ☐ Error handling and recovery procedures

Performance & Cost (20 points) -

- ☐ Scalability analysis for 50x peak volumes -
- ☐ Performance optimization strategies -
- ☐ Cost estimation and optimization plan -
- ☐ Resource sizing recommendations

Operational Excellence (10 points) -

- ☐ Monitoring and alerting design -
- ☐ SLA definitions and tracking -
- ☐ Documentation quality and completeness

Bonus Challenges (Optional)

1. Real-time Enhancement: How would you add real-time fraud alerts while maintaining the batch reconciliation foundation?
2. Multi-Region Design: Extend the architecture to support processing in multiple geographic regions for compliance reasons.
3. ML Integration: Design how you'd integrate machine learning models for anomaly detection in the reconciliation process.

Getting Started

1. Week 1: Architecture design and technology selection
2. Week 2: Detailed implementation planning and data quality framework
3. Week 3: Performance analysis and operational planning
4. Week 4: Documentation completion and presentation preparation

Resources

Sample financial data formats and banking partner APIs

Regulatory reporting templates

Performance benchmarking guidelines

Cost modeling spreadsheet

This project will demonstrate your mastery of batch processing architecture principles while tackling a real-world system with complex requirements, multiple constraints, and high reliability needs.

Further Reading

Resources:

1. "Designing Data-Intensive Applications" by Martin Kleppmann

Chapters 10-12 on batch processing and workflow coordination
Best comprehensive resource on batch processing fundamentals

2. "The Data Warehouse Toolkit" by Ralph Kimball

ETL design patterns and best practices

Dimensional modeling for batch systems

3. Apache Airflow Documentation

Best Practices Guide

Real-world DAG patterns and optimization techniques

Advanced Topics:

4. "Building Analytics Applications with AWS" by Jeff Butte

Cloud-native batch processing architectures

Cost optimization strategies for large-scale batch processing

5. Great Expectations Documentation

Data Quality Patterns

Automated testing for data pipelines

Industry Case Studies:

6. Netflix Technology Blog

Data Infrastructure at Netflix
Real-world batch processing at petabyte scale

7. Uber Engineering Blog

Building Uber's Data Platform
Batch and stream processing architecture evolution

Tools and Technologies:

8. dbt Documentation

Best Practices Guide

Modern ELT patterns and SQL transformations

9. Apache Spark Performance Tuning Guide

Official Tuning Guide
Advanced optimization techniques for large-scale batch processing

Research Papers:

10. "MapReduce: Simplified Data Processing on Large Clusters"

(Dean & Ghemawat, 2004)

Foundational paper on distributed batch processing

Understanding the principles behind modern batch frameworks

Remember: The best way to master batch processing is to build systems. Start with simple daily ETL jobs and gradually add complexity as you encounter real-world requirements and constraints. # Module 4: Stream Processing & Real-Time Systems

What you'll learn

04

Stream Processing & Real-Time Systems

4.1 Stream Processing Fundamentals

⚡ TL;DR

Stream processing handles unbounded data flows in real-time vs bounded batch data - challenges: ordering, windowing, late data, and exactly-once processing - Choose between at-least-once (easier) vs exactly-once (harder but sometimes required) - Watermarks help handle late-arriving data gracefully - Most "real-time" requirements can be met with fast batch processing - question the need first

Understanding Streams vs Batches

The difference between stream and batch processing

isn't just about speed - it's about how you think about data.

Batch Processing Mindset:

Data = Finite dataset with clear boundaries Processing = Function that transforms input to output Time = When processing starts and ends

Stream Processing Mindset:

Data = Infinite flow of events over time Processing = Continuous transformations on flowing data Time = When events occurred (event time) vs when processed (processing time)

Batch vs Stream Processing

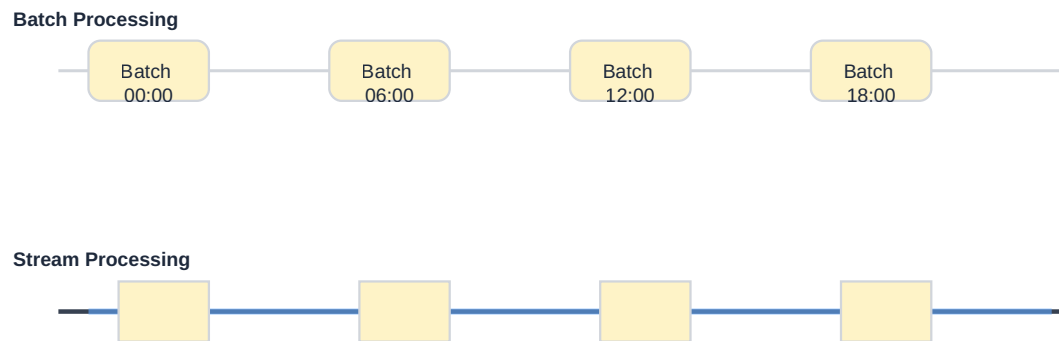


Figure 11: Two timelines side by side. Top shows “Batch Processing”: discrete boxes representing batches at regular intervals (00:00...

The Streaming Data Model

PYTHON

```
# Stream processing operates on infinite sequences of events
from dataclasses import dataclass
from typing import Iterator, Optional
import time

@dataclass
class StreamEvent:
    """Individual event in a data stream"""
    event_id: str
    event_time: int # Unix timestamp when event occurred
    processing_time: int # Unix timestamp when event was processed
    user_id: str
    event_type: str
    payload: dict

    @property
    def latency(self) -> int:
        """How long between event occurrence and processing"""
        return self.processing_time - self.event_time

# Stream is an infinite iterator of events
class EventStream:
    """Represents an unbounded stream of events"""

    def __init__(self, source_name: str):
        self.source_name = source_name
        self.offset = 0
```

The `__iter__` implementation:

SQL

```
def __iter__(self) -> Iterator[StreamEvent]:
    """Infinite iterator over events"""
    while True:
        # In real implementation, this would read from Kafka,
```

Kinesis, etc.

```

event = self.get_next_event()
if event:
    yield event
else:
    time.sleep(0.1) # Brief pause if no events

```

available

```

def get_next_event(self) -> Optional[StreamEvent]:
    """Get next event from the stream"""
    # Implementation would read from actual stream source
    pass

# Stream transformations are applied continuously
def process_user_events(event_stream: EventStream):
    """Example stream processing - runs indefinitely"""

    for event in event_stream:
        try:
            # Transform each event as it arrives
            if event.event_type == 'page_view':
                enriched_event = enrich_page_view(event)
                send_to_analytics(enriched_event)

            elif event.event_type == 'purchase':
                enriched_event = enrich_purchase(event)
                update_real_time_metrics(enriched_event)
                check_fraud_rules(enriched_event)

        except Exception as e:
            # Error handling in streams is critical
            handle_processing_error(event, e)

```

Time Windows in Stream Processing

Fixed/Tumbling Windows:

```

from datetime import datetime, timedelta
from collections import defaultdict

class TumblingWindow:
    """Fixed-size non-overlapping time windows"""

    def __init__(self, window_size_minutes: int = 5):
        self.window_size = timedelta(minutes=window_size_minutes)
        self.windows = defaultdict(list) # window_start -> events

    def add_event(self, event: StreamEvent):
        """Add event to appropriate window"""
        event_time = datetime.fromtimestamp(event.event_time)

        # Calculate which window this event belongs to
        window_start = self.get_window_start(event_time)
        self.windows[window_start].append(event)

    def get_window_start(self, event_time: datetime) -> datetime:
        """Calculate window start for given event time"""
        minutes_since_epoch = int(event_time.timestamp() // 60)
        window_minutes = self.window_size.total_seconds() // 60
        window_start_minutes = (minutes_since_epoch //

```

window_minutes) * window_minutes

```

return datetime.fromtimestamp(window_start_minutes * 60)

def process_completed_windows(self, current_time: datetime):
    """Process windows that are complete (past the window end

```

time)"""

```

completed_windows = []

```

The PYTHON code:


```

for window_start, events in list(self.windows.items()):
    window_end = window_start + self.window_size

    if current_time >= window_end:
        # Window is complete, process it
        aggregated_result =

```

PYTHON

```

self.aggregate_window(window_start, events)

```

```

completed_windows.append(aggregated_result)

        # Remove processed window to free memory
        del self.windows[window_start]

    return completed_windows

def aggregate_window(self, window_start: datetime, events: list)

```

PYTHON

-> dict:

```

"""Aggregate events in a completed window"""
return {
    'window_start': window_start,
    'window_end': window_start + self.window_size,
    'event_count': len(events),
    'unique_users': len(set(event.user_id for event in

```

PYTHON

events)),

```

    'event_types': defaultdict(int)
}

```

Sliding Windows:

```

class SlidingWindow:
    """Overlapping time windows that slide continuously"""

    def __init__(self, window_size_minutes: int = 15,

```

PYTHON

slide_interval_minutes: int = 5):

```
self.window_size = timedelta(minutes=window_size_minutes)
self.slide_interval =
```

PYTHON

```
timedelta(minutes=slide_interval_minutes)
```

```
self.events = [] # All events within current window range

def add_event(self, event: StreamEvent):
    """Add event and maintain sliding window"""
    event_time = datetime.fromtimestamp(event.event_time)
    self.events.append((event_time, event))

    # Remove events that are outside the window
    cutoff_time = event_time - self.window_size
    self.events = [(t, e) for t, e in self.events if t >=
```

PYTHON

cutoff_time]

```
def get_current_window_metrics(self) -> dict:
    """Get metrics for current sliding window"""
    if not self.events:
        return {'event_count': 0}

    latest_time = max(t for t, e in self.events)
    window_start = latest_time - self.window_size

    return {
        'window_start': window_start,
        'window_end': latest_time,
        'event_count': len(self.events),
        'events_per_minute': len(self.events) /
```

PYTHON

```
self.window_size.total_seconds() * 60,
```

```
'unique_users': len(set(event.user_id for t, event in
```

```
self.events))
```

```
}
```

Handling Late-Arriving Data

One of the biggest challenges in stream processing is handling events that arrive out of order or late.

```
class WatermarkManager:
    """Manages watermarks to handle late-arriving data"""

    def __init__(self, max_lateness_seconds: int = 300):    # 5 minute
```

PYTHON

tolerance

```
self.max_lateness = timedelta(seconds=max_lateness_seconds)
self.watermark = None
self.late_events_count = 0

def update_watermark(self, event_time: datetime,
```

PYTHON

processing_time: datetime):

```
    """Update watermark based on observed event times"""
    # Conservative watermark: current time minus max expected
```

lateness

```
new_watermark = processing_time - self.max_lateness

    if self.watermark is None or new_watermark > self.watermark:
        self.watermark = new_watermark

def is_event_late(self, event: StreamEvent) -> bool:
    """Check if event arrives too late to be processed"""
    if self.watermark is None:
        return False

    event_time = datetime.fromtimestamp(event.event_time)
    return event_time < self.watermark

def handle_late_event(self, event: StreamEvent) -> str:
    """Decide how to handle a late-arriving event"""
    self.late_events_count += 1

    lateness = datetime.now() -
```

```
datetime.fromtimestamp(event.event_time)
```

```

if lateness > timedelta(hours=1):
    # Very late - probably historical data, handle specially
    return 'archive_for_batch_reprocessing'
elif lateness > self.max_lateness:
    # Late but not too late - log and drop
    return 'log_and_drop'
else:
    # Acceptable lateness - process normally
    return 'process_normally'

# Usage in stream processor
def process_with_watermarks(event_stream: EventStream):
    """Process stream with watermark-based late data handling"""
    watermark_manager = WatermarkManager()
    window_processor = TumblingWindow()

    for event in event_stream:
        current_time = datetime.now()
        event_time = datetime.fromtimestamp(event.event_time)

        # Update watermark
        watermark_manager.update_watermark(event_time, current_time)

        # Check if event is too late
        if watermark_manager.is_event_late(event):
            action = watermark_manager.handle_late_event(event)

            if action == 'log_and_drop':
                log_late_event(event, watermark_manager.watermark)
                continue

        # Process event normally
        window_processor.add_event(event)

        # Process any completed windows
        completed_windows =

```

```

window_processor.process_completed_windows(current_time)

```

```

for window_result in completed_windows:

```

```

    emit_window_result(window_result)

```

Processing Guarantees: At-Least-Once vs Exactly-Once

At-Least-Once Processing:

```
class AtLeastOnceProcessor:
    """Simpler processing with potential duplicates"""

    def __init__(self, kafka_consumer, output_database):
        self.consumer = kafka_consumer
        self.database = output_database

    def process_events(self):
        """Process events with at-least-once guarantee"""

        while True:
            # Read batch of messages
            messages = self.consumer.poll(timeout=1000)

            for message in messages:
                try:
                    # Process message
                    result = self.transform_event(message.value)

                    # Write to output (might happen multiple times
```

PYTHON

if we fail after this)

```
self.database.insert(result)

except Exception as e:
    # Log error but don't commit offset - will
```

PYTHON

reprocess

```
logger.error(f"Failed to process message: {e}")
continue

# Commit offset only after all messages processed
# If we crash before this, messages will be reprocessed
self.consumer.commit()
```

PYTHON

Exactly-Once Processing (More Complex):

```
class ExactlyOnceProcessor:
    """More complex processing that avoids duplicates"""

    def __init__(self, kafka_consumer, output_database):
        self.consumer = kafka_consumer
        self.database = output_database
        self.processed_offsets = set() # Track what we've processed

    def process_events(self):
        """Process events with exactly-once guarantee"""

        while True:
            messages = self.consumer.poll(timeout=1000)

            # Start database transaction
            with self.database.transaction() as tx:
                processed_in_batch = []

                for message in messages:
                    message_key = f"{message.topic}-
```

```
{message.partition}-{message.offset}"
```

```
# Skip if already processed (idempotency)
if
```

`self.database.is_already_processed(message_key):`

```
    continue

    try:
        # Process message
        result = self.transform_event(message.value)
```

```
# Write result and mark as processed in same
```

`transaction`

PYTHON

PYTHON

```
tx.insert_result(result)
tx.mark_processed(message_key,
```

```
message.offset)
```

```
processed_in_batch.append(message.offset)

except Exception as e:
    # Rollback transaction and don't commit
```

offsets

```
logger.error(f"Failed to process batch:
```

```
{e}")
```

```
tx.rollback()
    break
else:
    # All messages processed successfully
    tx.commit()

    # Only commit offsets after database transaction
```

commits

```
self.consumer.commit_sync()
```

PYTHON

Pro Tip Start with at-least-once processing - it's much simpler and often sufficient. Only move to exactly-once when you have concrete business requirements that can't tolerate duplicates. The complexity increase is significant.

Stream Processing Use Cases

Good Use Cases for Stream Processing:


```
# 1. Real-time fraud detection
def fraud_detection_stream(payment_events):
    """Real-time fraud detection - sub-second response needed"""

    user_velocity_tracker = SlidingWindow(window_size_minutes=10)

    for event in payment_events:
        if event.event_type == 'payment_attempted':
            # Add to velocity tracking
            user_velocity_tracker.add_event(event)

            # Get recent activity for this user
            recent_activity =
```

```
user_velocity_tracker.get_user_activity(event.user_id)
```

```
# Real-time fraud scoring
fraud_score = calculate_fraud_score(event,
```

```
recent_activity)
```

```
if fraud_score > 0.9:
    # Block payment immediately
    block_payment(event.payment_id)
    alert_fraud_team(event, fraud_score)
elif fraud_score > 0.7:
    # Flag for manual review
    flag_for_review(event.payment_id, fraud_score)

# 2. Live dashboards and metrics
def real_time_metrics_stream(user_events):
    """Update live dashboard metrics"""

    metrics_window = TumblingWindow(window_size_minutes=1)

    for event in user_events:
        metrics_window.add_event(event)

        # Process completed windows
        completed =
```

```
metrics_window.process_completed_windows(datetime.now())
```

```
for window_result in completed:
    # Update live dashboard
    update_dashboard_metrics({
        'active_users_last_minute':
```

```
window_result['unique_users'],
```

```
'events_per_second': window_result['event_count'] /
```

60,

```
'top_event_types': window_result['event_types']
    })
```

Poor Use Cases for Stream Processing:

```
# DON'T USE STREAMING FOR THESE
```

SQL

```
# 1. Complex analytics requiring full dataset
def monthly_cohort_analysis():
    """This needs full historical data - use batch processing"""
    # Requires joining months of historical data
    # Complex aggregations across time periods
    # Not time-sensitive (monthly report)
    pass

# 2. Heavy ML model training
def train_recommendation_model():
    """Model training needs stable datasets - use batch"""
    # Requires consistent snapshots of data
    # Long-running process (hours)
    # Not real-time requirement
    pass

# 3. Regulatory reporting
def generate_compliance_report():
    """Compliance needs accuracy over speed - use batch"""
    # Must be 100% accurate
    # Complex business rules
    # Can tolerate daily/weekly latency
    pass
```

□ Checkpoint

1. What's the main difference between batch and stream processing? a. Stream processing is always faster b. Batch handles finite datasets, streams handle infinite data flows c. Stream processing is cheaper d. Batch processing can't handle real-time data
2. When should you choose exactly-once over at-least-once processing? a. Always - it's better b. When you can't tolerate any duplicate processing c. When you need better performance d. When using Kafka
3. What is a watermark in stream processing? a. A performance benchmark b. A marker to handle late-arriving data c. A type of window d. A security feature

Answers: 1-b, 2-b, 3-b

4.2 Message Queue & Event Streaming Design

⚡ TL;DR

Message queues (SQS, RabbitMQ) for point-to-point reliable delivery - Event streams (Kafka, Pulsar) for high- throughput pub/sub with persistence - Choose based on your

pattern: work queues vs event logs vs pub/sub - Partition strategy determines scalability and ordering guarantees -

Schema evolution is critical for long-term stream system success

Message Patterns: Queue vs Log vs Pub/Sub

Understanding the messaging patterns helps you choose the right tool and design the right architecture.

Message Queue Pattern:

```
# Point-to-point delivery, each message consumed once
# Good for: Work distribution, task processing, load balancing
```

```
import boto3
from typing import List, Dict

class WorkQueue:
    """Traditional message queue for work distribution"""

    def __init__(self, queue_url: str):
        self.sqs = boto3.client('sqs')
        self.queue_url = queue_url

    def send_work_item(self, task_data: Dict):
        """Send work item to queue"""
        message = {
            'task_id': task_data['task_id'],
            'task_type': task_data['task_type'],
            'payload': task_data['payload'],
            'priority': task_data.get('priority', 'normal'),
            'created_at': datetime.now().isoformat()
        }

        self.sqs.send_message(
            QueueUrl=self.queue_url,
            MessageBody=json.dumps(message),
            MessageAttributes={
                'task_type': {
                    'StringValue': task_data['task_type'],
                    'DataType': 'String'
                }
            }
        )

    def process_work_items(self, worker_function):
        """Process work items from queue"""
        while True:
            # Poll for messages
            response = self.sqs.receive_message(
                QueueUrl=self.queue_url,
                MaxNumberOfMessages=10,
                WaitTimeSeconds=20 # Long polling
            )

            messages = response.get('Messages', [])

            for message in messages:
                try:
```

```
# Process work item
task_data = json.loads(message['Body'])
result = worker_function(task_data)

# Delete message only after successful
```

processing

```
self.sqs.delete_message(
    QueueUrl=self.queue_url,
    ReceiptHandle=message['ReceiptHandle']
)

except Exception as e:
    # Message will become visible again for retry
```

PYTHON

The *PYTHON* code:

```
logger.error(f"Failed to process task: {e}")

# Example usage
def process_image_resize_task(task_data):
    """Example work item processor"""
    image_url = task_data['payload']['image_url']
    target_size = task_data['payload']['target_size']

    # Resize image
    resized_image = resize_image(image_url, target_size)

    # Upload result
    result_url = upload_image(resized_image)

    return {'result_url': result_url}
```

PYTHON

Event Log Pattern:

PYTHON

```
# Durable log of events, multiple consumers can read
# Good for: Event sourcing, audit trails, replay capability

from kafka import KafkaProducer, KafkaConsumer
import json

class EventLog:
    """Event log pattern with Kafka"""

    def __init__(self, bootstrap_servers: List[str]):
        self.producer = KafkaProducer(
            bootstrap_servers=bootstrap_servers,
            value_serializer=lambda v: json.dumps(v).encode('utf-
```

8'),

PYTHON

```
key_serializer=lambda k: k.encode('utf-8') if k else
```

None,

PYTHON

```
# Durability settings
    acks='all', # Wait for all replicas
    retries=10,
    retry_backoff_ms=100
)

def append_event(self, topic: str, event_data: Dict,
```

partition_key: str = None):

PYTHON

```

"""Append event to log"""
event = {
    'event_id': str(uuid.uuid4()),
    'event_type': event_data['event_type'],
    'timestamp': datetime.now().isoformat(),
    'data': event_data['data']
}

# Send event to log
future = self.producer.send(
    topic=topic,
    key=partition_key,
    value=event
)

# Wait for confirmation (synchronous for reliability)
record_metadata = future.get(timeout=10)

return {
    'offset': record_metadata.offset,
    'partition': record_metadata.partition,
    'topic': record_metadata.topic
}

def read_events_from_position(self, topic: str, start_offset:

```

```
int = 0,
```

SQL

```

consumer_group: str = None):
"""Read events from specific position in log"""
consumer = KafkaConsumer(
    topic,
    bootstrap_servers=self.bootstrap_servers,

```

```

group_id=consumer_group,
auto_offset_reset='earliest' if start_offset == 0 else

```

```
'none',
```

PYTHON

```
value_deserializer=lambda m: json.loads(m.decode('utf-
```

```
8'))
```



```

)

# Seek to specific offset if needed
if start_offset > 0 and not consumer_group:
    partitions = consumer.partitions_for_topic(topic)
    for partition in partitions:
        tp = TopicPartition(topic, partition)
        consumer.assign([tp])
        consumer.seek(tp, start_offset)

for message in consumer:
    yield {
        'offset': message.offset,
        'partition': message.partition,
        'timestamp': message.timestamp,
        'event': message.value
    }

# Event sourcing example
class OrderEventLog:
    """Order management using event sourcing"""

    def __init__(self, event_log: EventLog):
        self.event_log = event_log
        self.topic = 'order-events'

    def create_order(self, order_data: Dict):
        """Create new order by appending event"""
        event = {
            'event_type': 'order_created',
            'data': {
                'order_id': order_data['order_id'],
                'customer_id': order_data['customer_id'],
                'items': order_data['items'],
                'total_amount': order_data['total_amount']
            }
        }

        return self.event_log.append_event(
            topic=self.topic,
            event_data=event,
            partition_key=order_data['order_id']    # Partition by

```

order ID

```

)

def update_order_status(self, order_id: str, new_status: str):
    """Update order status by appending event"""
    event = {
        'event_type': 'order_status_changed',
        'data': {
            'order_id': order_id,
            'new_status': new_status,
            'previous_status': self.get_current_status(order_id)
        }
    }

    return self.event_log.append_event(
        topic=self.topic,
        event_data=event,
        partition_key=order_id
    )

def rebuild_order_state(self, order_id: str) -> Dict:
    """Rebuild current order state from event log"""
    order_state = {}

    for event_record in

```

`self.event_log.read_events_from_position(self.topic):`

```

event = event_record['event']

# Only process events for this order
if event['data'].get('order_id') == order_id:
    if event['event_type'] == 'order_created':
        order_state = {
            'order_id': order_id,
            'customer_id': event['data']['customer_id'],
            'items': event['data']['items'],
            'total_amount': event['data']

```

`['total_amount'],`

```

        'status': 'created',
        'created_at': event_record['timestamp']
    }
    elif event['event_type'] == 'order_status_changed':
        order_state['status'] = event['data']

```

PYTHON

['new_status']

```

order_state['last_updated'] =

```

```

event_record['timestamp']

```

```

return order_state

```

Pub/Sub Pattern:

```

# Multiple subscribers receive all messages
# Good for: Event notifications, fan-out processing, real-time

```

updates

```

import redis
from typing import Callable

class PubSubSystem:
    """Pub/Sub pattern with Redis"""

    def __init__(self, redis_host: str = 'localhost', redis_port:

```

PYTHON

```

int = 6379):

```

```

self.redis_client = redis.Redis(host=redis_host,

```

PYTHON

```

port=redis_port)

```

```

self.psubsub = self.redis_client.psubsub()

def publish_event(self, channel: str, event_data: Dict):
    """Publish event to channel"""
    message = {
        'event_id': str(uuid.uuid4()),
        'timestamp': datetime.now().isoformat(),
        'data': event_data
    }

    # Publish to channel - all subscribers receive it
    subscriber_count = self.redis_client.publish(
        channel, json.dumps(message)
    )

    return {
        'subscribers_notified': subscriber_count,
        'event_id': message['event_id']
    }

def subscribe_to_channel(self, channel: str, handler:

```

```
Callable[[Dict], None]):
```

```

"""Subscribe to channel and process messages"""
self.psubsub.subscribe(channel)

try:
    for message in self.psubsub.listen():
        if message['type'] == 'message':
            try:
                event_data = json.loads(message['data'])
                handler(event_data)
            except Exception as e:
                logger.error(f"Error processing message:
{e}")

```

```

finally:
    self.pubsub.unsubscribe(channel)

# Example: Real-time notification system
class NotificationSystem:
    """Multi-channel notification system using pub/sub"""

    def __init__(self, pubsub: PubSubSystem):
        self.pubsub = pubsub

    def send_user_notification(self, user_id: str,

```

notification_data: Dict):

```

    """Send notification to all user's connected clients"""
    channel = f"user:{user_id}:notifications"

    event = {
        'notification_type': notification_data['type'],
        'title': notification_data['title'],
        'message': notification_data['message'],
        'action_url': notification_data.get('action_url')
    }

    return self.pubsub.publish_event(channel, event)

    def subscribe_user_notifications(self, user_id: str,

```

client_handler):

```

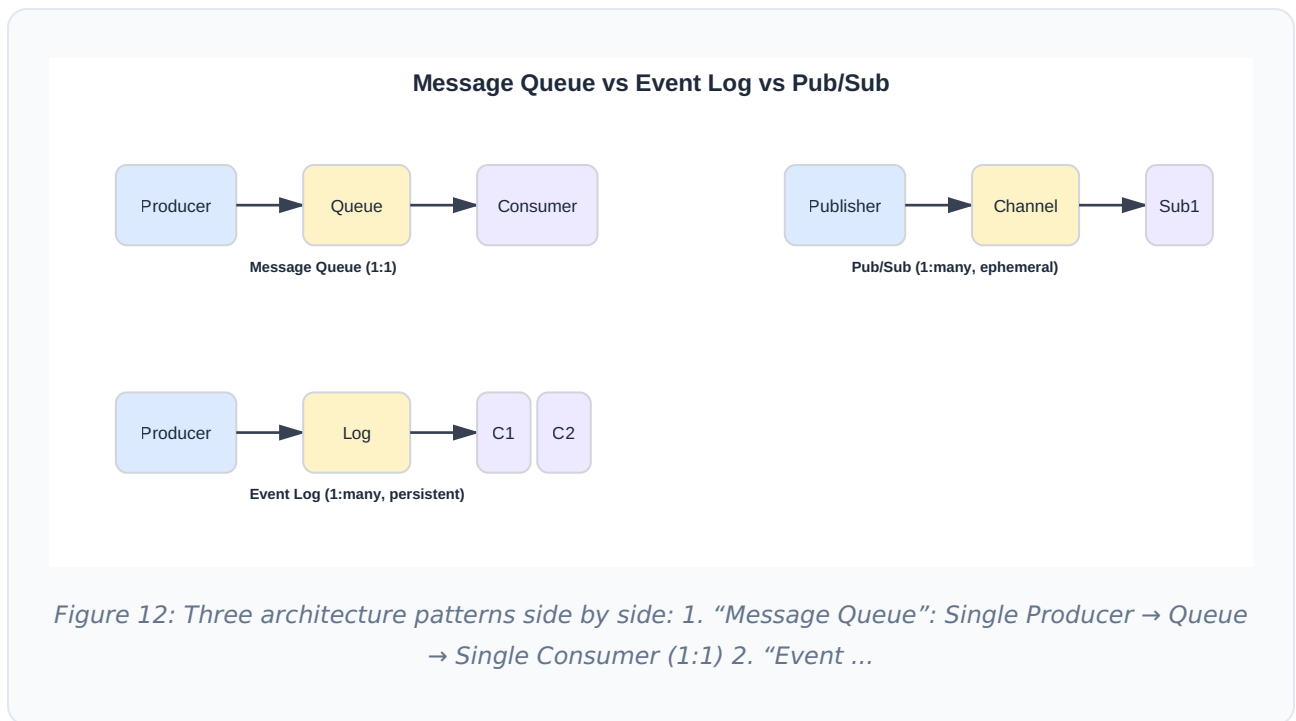
    """Subscribe client to user's notification channel"""
    channel = f"user:{user_id}:notifications"

    def notification_handler(event):
        """Handle incoming notification"""
        # Send to client via WebSocket, push notification, etc.
        client_handler.send_notification(event['data'])

    self.pubsub.subscribe_to_channel(channel,

```

notification_handler)



Kafka Architecture and Design Principles

```
# Kafka partitioning strategy for scale and ordering
class KafkaTopicDesign:
    """Design Kafka topics for specific use cases"""

    @staticmethod
    def design_user_events_topic(expected_users: int,
```

PYTHON

events_per_user_per_day: int):

```
"""Design topic for user event tracking"""

daily_events = expected_users * events_per_user_per_day
events_per_second = daily_events / (24 * 60 * 60)

# Rule of thumb: ~10MB per partition per second max
```

throughput

```
# Each event ~1KB average
partition_throughput_events_per_sec = (10 * 1024 * 1024) /
```

1024 # ~10k events/sec

```
min_partitions = max(1, int(events_per_second /
```

partition_throughput_events_per_sec))

```
# Round up to power of 2 for better distribution
recommended_partitions = 1
while recommended_partitions < min_partitions:
    recommended_partitions *= 2
```

```
return {
    'topic_name': 'user-events',
    'partitions': recommended_partitions,
    'replication_factor': 3, # For production
    'retention_ms': 7 * 24 * 60 * 60 * 1000, # 7 days
    'partition_key_strategy': 'user_id', # Ensures user
```

event ordering

```
'estimated_throughput_per_partition': f"
```

{partition_throughput_events_per_sec} events/sec",

```
'estimated_storage_per_day': f"{daily_events * 1024 /
```

```
(1024**3):.2f} GB"
```

```

}

@staticmethod
def design_order_events_topic():
    """Design topic for order processing with strict ordering"""
    return {
        'topic_name': 'order-events',
        'partitions': 32, # Moderate partitioning for order

```

throughput

```

'replication_factor': 3,
'retention_ms': 365 * 24 * 60 * 60 * 1000, # 1 year for

```

audit

```

'partition_key_strategy': 'order_id', # Strict order

```

event ordering

```

'cleanup_policy': 'compact', # Keep latest state for

```

each order

```

'min_insync_replicas': 2, # Durability requirement
'config': {
    'acks': 'all', # Wait for all replicas
    'enable_idempotence': True, # Exactly-once

```

semantics

```

'max_in_flight_requests_per_connection': 1 #

```


Preserve ordering

```
}  
    }  
  
# Advanced partitioning strategies  
class PartitioningStrategy:  
    """Different partitioning strategies for various use cases"""  
  
    @staticmethod  
    def user_based_partitioning(user_id: str, total_partitions: int)
```

PYTHON

-> int:

```
    """Partition by user ID - maintains user event ordering"""  
    return hash(user_id) % total_partitions  
  
    @staticmethod  
    def time_based_partitioning(timestamp: datetime,
```

PYTHON

total_partitions: int) -> int:

```
    """Partition by time - useful for time-series data"""  
    # Partition by hour of day for even distribution  
    hour = timestamp.hour  
    return hour % total_partitions  
  
    @staticmethod  
    def round_robin_partitioning(total_partitions: int) -> int:  
        """Round-robin partitioning - maximum throughput, no
```

PYTHON

ordering"""

```
    # In practice, Kafka producer handles this automatically  
    return None # Let producer choose  
  
    @staticmethod  
    def geography_based_partitioning(user_location: str,
```

PYTHON

total_partitions: int) -> int:

```

"""Partition by geography for locality"""
region_map = {
    'us-east': 0, 'us-west': 1, 'eu-west': 2, 'eu-east': 3,
    'asia-pacific': 4, 'latin-america': 5
}

region_partitions = total_partitions // len(region_map)

```

```

region_base = region_map.get(user_location, 0) *

```

region_partitions

```

# Sub-partition within region
    return region_base + hash(user_location) % region_partitions

# Consumer group management
class ConsumerGroupManager:
    """Manage Kafka consumer groups for different processing

```

PYTHON

patterns"""

```

def __init__(self, bootstrap_servers: List[str]):
    self.bootstrap_servers = bootstrap_servers

def create_real_time_processor(self, topic: str, group_id: str,
                               processor_function: Callable):
    """Create consumer for real-time processing"""
    consumer = KafkaConsumer(
        topic,
        bootstrap_servers=self.bootstrap_servers,
        group_id=group_id,
        auto_offset_reset='latest', # Only new messages
        enable_auto_commit=True,
        auto_commit_interval_ms=1000,
        value_deserializer=lambda m: json.loads(m.decode('utf-

```

PYTHON

8'))

```

)

for message in consumer:
    try:
        processor_function(message.value)
    except Exception as e:
        # Don't commit offset if processing fails
        logger.error(f"Processing failed: {e}")
        continue

def create_batch_processor(self, topic: str, group_id: str,
                           batch_processor_function: Callable):
    """Create consumer for batch processing"""
    consumer = KafkaConsumer(
        topic,
        bootstrap_servers=self.bootstrap_servers,
        group_id=group_id,
        auto_offset_reset='earliest', # Process from beginning
        enable_auto_commit=False, # Manual commit for batch

```

safety

```

max_poll_records=1000, # Large batches
value_deserializer=lambda m: json.loads(m.decode('utf-

```

8'))

```

)

batch = []
for message in consumer:
    batch.append(message.value)

    if len(batch) >= 1000: # Process in batches of 1000
        try:
            batch_processor_function(batch)
            consumer.commit() # Commit after successful

```

batch

```
batch = []
except Exception as e:
    logger.error(f"Batch processing failed: {e}")
    # Don't commit - will reprocess batch
    batch = []
```

Schema Evolution and Versioning

```
# Schema evolution strategy for long-lived streams
from avro import schema, io
import avro.schema
from typing import Dict, Any
```

SQL

The SchemaRegistry implementation:

```
class SchemaRegistry:
    """Manage schema evolution for streaming data"""

    def __init__(self):
        self.schemas = {} # schema_name -> {version -> schema}
        self.compatibility_rules = {}

    def register_schema(self, schema_name: str, schema_definition:
```

PYTHON

str,

```

version: int = 1):
    """Register new schema version"""
    parsed_schema = avro.schema.parse(schema_definition)

    if schema_name not in self.schemas:
        self.schemas[schema_name] = {}

    # Check compatibility with previous version
    if version > 1:
        previous_version = version - 1
        if previous_version in self.schemas[schema_name]:
            compatibility_check = self.check_compatibility(
                schema_name, previous_version, parsed_schema
            )
            if not compatibility_check['compatible']:
                raise SchemaCompatibilityError(
                    f"Schema version {version} incompatible with

```

{previous_version}: "

```

        f"{compatibility_check['issues']}"
        )

        self.schemas[schema_name][version] = parsed_schema
        return {'schema_name': schema_name, 'version': version}

    def check_compatibility(self, schema_name: str, old_version:

```

int,

```

new_schema: avro.schema.Schema) -> Dict:
    """Check if new schema is compatible with old version"""
    old_schema = self.schemas[schema_name][old_version]

    issues = []

    # Check field compatibility
    old_fields = {f.name: f for f in old_schema.fields}
    new_fields = {f.name: f for f in new_schema.fields}

    # Removed fields should have default values in old schema
    for field_name in old_fields:
        if field_name not in new_fields:
            if not old_fields[field_name].has_default:
                issues.append(f"Removed field '{field_name}' had

```

no default value")

```

    # Added fields should have default values
    for field_name in new_fields:
        if field_name not in old_fields:
            if not new_fields[field_name].has_default:
                issues.append(f"New field '{field_name}' has no

```

default value")

```

    # Type changes are generally incompatible
    for field_name in old_fields:
        if field_name in new_fields:
            old_type = old_fields[field_name].type
            new_type = new_fields[field_name].type
            if old_type != new_type:
                issues.append(f"Field '{field_name}' type

```

changed from {old_type} to {new_type}")

```

    return {
        'compatible': len(issues) == 0,
        'issues': issues
    }

```

```

}

# Example schema evolution
user_event_schema_v1 = """
{
  "type": "record",
  "name": "UserEvent",
  "fields": [
    {"name": "user_id", "type": "string"},
    {"name": "event_type", "type": "string"},
    {"name": "timestamp", "type": "long"},
    {"name": "properties", "type": {"type": "map", "values":

```

"string"}}}

```

]
}
"""

user_event_schema_v2 = """
{
  "type": "record",
  "name": "UserEvent",
  "fields": [
    {"name": "user_id", "type": "string"},
    {"name": "event_type", "type": "string"},
    {"name": "timestamp", "type": "long"},
    {"name": "properties", "type": {"type": "map", "values":

```

"string"}}},

```

{"name": "session_id", "type": ["null", "string"], "default":

```

JSON

null},

JSON

```
{
  "name": "device_info", "type": ["null", {
    "type": "record",
    "name": "DeviceInfo",
    "fields": [
      {"name": "platform", "type": "string"},
      {"name": "version", "type": "string"}
    ]
  }], "default": null}
]
```

```
# Version-aware event processing
class VersionedEventProcessor:
    """Process events with schema version awareness"""

    def __init__(self, schema_registry: SchemaRegistry):
        self.schema_registry = schema_registry

    def process_event(self, event_data: bytes, schema_name: str,
```

schema_version: int):

PYTHON

```
"""Process event with version-specific logic"""

# Get appropriate schema
schema = self.schema_registry.schemas[schema_name]
```

[schema_version]

PYTHON

```
# Deserialize with schema
reader = io.DatumReader(schema)
decoder = io.BinaryDecoder(io.BytesIO(event_data))
event = reader.read(decoder)

# Process based on schema version
if schema_version == 1:
    return self.process_v1_event(event)
elif schema_version == 2:
    return self.process_v2_event(event)
else:
    raise ValueError(f"Unsupported schema version:
```

{schema_version}")


```
def process_v1_event(self, event: Dict) -> Dict:
    """Process version 1 events"""
```

The corresponding SQL query:

```
return {
    'user_id': event['user_id'],
    'event_type': event['event_type'],
    'timestamp': event['timestamp'],
    'properties': event['properties'],
    # Fill in missing fields from v2 with defaults
    'session_id': None,
    'device_info': None
}

def process_v2_event(self, event: Dict) -> Dict:
    """Process version 2 events with additional fields"""
    return {
        'user_id': event['user_id'],
        'event_type': event['event_type'],
        'timestamp': event['timestamp'],
        'properties': event['properties'],
        'session_id': event.get('session_id'),
        'device_info': event.get('device_info')
    }
```

Message Delivery Patterns and Error Handling

```
class RobustMessageHandler:
    """Handle various message delivery failure scenarios"""

    def __init__(self, primary_queue, dead_letter_queue,
```

```
retry_queue):
```

PYTHON

```

self.primary_queue = primary_queue
self.dead_letter_queue = dead_letter_queue
self.retry_queue = retry_queue
self.max_retries = 3
self.retry_delays = [60, 300, 900] # 1min, 5min, 15min

def process_with_retries(self, message_handler: Callable):
    """Process messages with exponential backoff retries"""

    while True:
        try:
            # Get message from primary queue
            messages =

```

```

self.primary_queue.receive_messages(max_messages=1)

```

PYTHON

```

if not messages:
    time.sleep(1)
    continue

message = messages[0]
message_data = json.loads(message.body)

# Check if this is a retry message
retry_count = message_data.get('retry_count', 0)

try:
    # Process message
    result = message_handler(message_data)

    # Success - delete message
    message.delete()

except RetryableError as e:
    # Handle retryable error
    if retry_count < self.max_retries:
        self.schedule_retry(message_data,

```

```

retry_count + 1, str(e))

```

SQL

```

message.delete() # Remove from primary

```

queue

PYTHON

```

else:
    # Max retries exceeded - send to DLQ
    self.send_to_dead_letter_queue(message_data,

```

```

str(e))

```

```

message.delete()

```

The PYTHON code:

PYTHON

```

except NonRetryableError as e:
    # Permanent error - send directly to DLQ
    self.send_to_dead_letter_queue(message_data,

```

```

str(e))

```

```

message.delete()

```

```

except Exception as e:
    logger.error(f"Unexpected error in message

```

processing: {e}")

PYTHON

```

time.sleep(5)

```

```

def schedule_retry(self, message_data: Dict, retry_count: int,

```

error_message: str):

```

"""Schedule message for retry with delay"""
retry_message = {
    *message_data,
    'retry_count': retry_count,
    'last_error': error_message,
    'retry_at': datetime.now() +

```

```

timedelta(seconds=self.retry_delays[retry_count - 1])

```

```

}

# Send to retry queue with delay
self.retry_queue.send_message(
    MessageBody=json.dumps(retry_message),
    DelaySeconds=self.retry_delays[retry_count - 1]
)

def send_to_dead_letter_queue(self, message_data: Dict,

```

error_message: str):

```

"""Send failed message to dead letter queue for

```

investigation"""

```

dlq_message = {
    **message_data,
    'failed_at': datetime.now().isoformat(),
    'final_error': error_message,
    'retry_count': message_data.get('retry_count', 0)
}

self.dead_letter_queue.send_message(
    MessageBody=json.dumps(dlq_message)
)

```

Pro Tip Design for message failures from day one. Every

message can fail to process, and you need a strategy for retries, dead letter queues, and manual intervention. The happy path is easy - the edge cases are what separate robust systems from fragile ones.

□ Checkpoint

1. When would you choose a message queue over an event stream? a. When you need high throughput b. When you need work distribution with each task processed

once

c. When you need multiple consumers d. When you need persistence 2. What's the purpose of partitioning in Kafka? a. To improve security b. To enable horizontal scaling and maintain ordering within

partitions

c. To reduce storage costs d. To improve message compression 3. Why is schema evolution important for streaming systems? a. It makes systems run faster b. It enables long-term compatibility as data formats change c. It reduces storage costs d. It improves security

Answers: 1-b, 2-b, 3-b

4.3

Stream Processing Frameworks

≤ TL;DR

Apache Flink for complex event processing and low- latency requirements - Kafka Streams for simpler processing when you're already using Kafka - Spark Structured Streaming for batch-stream unification and team familiarity - Cloud-native options (Kinesis Analytics, Dataflow) for managed simplicity - Choose based on complexity, latency requirements, and team expertise

Framework Comparison Matrix

Stream Processing Frameworks

Flink

Kafka Streams

Spark Streaming

Cloud Services

Complexity

Latency

Throughput

Learning Curve

Figure 13: Comparison matrix showing four frameworks (Flink, Kafka Streams, Spark Streaming, Cloud Services) across dimensions: Com...

Let's dive deep into each framework with real-world examples and use cases.
Apache Flink: Complex Event Processing Powerhouse

PYTHON

```
# Flink excels at complex event processing with low latency
from pyflink.datastream import StreamExecutionEnvironment
from pyflink.table import StreamTableEnvironment
from pyflink.table.expressions import lit, col
import json

class FlinkRealTimeFraudDetection:
    """Advanced fraud detection using Flink's CEP capabilities"""
```

```
def __init__(self):  
    # Set up Flink environment  
    self.env =  
  
    StreamExecutionEnvironment.get_execution_environment()
```

Answers: 1-b, 2-b, 3-b

TL;DR

Apache Flink for complex event processing and low- latency requirements - Kafka Streams for simpler processing when you're already using Kafka - Spark Structured Streaming for batch-stream unification and team familiarity - Cloud-native options (Kinesis Analytics, Dataflow) for managed simplicity - Choose based on complexity, latency requirements, and team expertise

Framework Comparison Matrix

Stream Processing Frameworks				
	Flink	Kafka Streams	Spark Streaming	Cloud Services
Comp				
Laten				
Throu				
Learn				

Figure 13: Comparison matrix showing four frameworks (Flink, Kafka Streams, Spark Streaming, Cloud Services) across dimensions: Com...

Let's dive deep into each framework with real-world examples and use cases.

Apache Flink: Complex Event Processing Powerhouse

PYTHON

```
# Flink excels at complex event processing with low latency
from pyflink.datastream import StreamExecutionEnvironment
from pyflink.table import StreamTableEnvironment
from pyflink.table.expressions import lit, col
import json

class FlinkRealTimeFraudDetection:
    """Advanced fraud detection using Flink's CEP capabilities"""

    def __init__(self):
        # Set up Flink environment
        self.env =
```

StreamExecutionEnvironment.get_execution_environment()

PYTHON

```
self.env.set_parallelism(4)
self.t_env = StreamTableEnvironment.create(self.env)

def setup_fraud_detection_pipeline(self):
    """Set up complex fraud detection using CEP patterns"""

    # Define input schema
    self.t_env.execute_sql("""
        CREATE TABLE transactions (
            user_id STRING,
            transaction_id STRING,
            amount DECIMAL(10,2),
            merchant_id STRING,
            location STRING,
            transaction_time TIMESTAMP(3),
            WATERMARK FOR transaction_time AS transaction_time -
```

INTERVAL '5' SECOND

```

) WITH (
    'connector' = 'kafka',
    'topic' = 'transactions',
    'properties.bootstrap.servers' = 'localhost:9092',
    'format' = 'json'
)
)
)

# Pattern 1: Velocity check - multiple high-value

```

transactions

```

velocity_alerts = self.t_env.sql_query("""
    SELECT
        user_id,
        COUNT(*) as transaction_count,
        SUM(amount) as total_amount,
        TUMBLE_START(transaction_time, INTERVAL '5' MINUTE)

```

SQL

as window_start

FROM transactions

```

WHERE amount > 1000
    GROUP BY
        user_id,
        TUMBLE(transaction_time, INTERVAL '5' MINUTE)
    HAVING COUNT(*) >= 3
)
)

# Pattern 2: Geographic anomaly - transactions from

```

SQL

different locations

```
geo_alerts = self.t_env.sql_query("""
    SELECT
        user_id,
        COUNT(DISTINCT location) as location_count,
        COLLECT(DISTINCT location) as locations,
        TUMBLE_START(transaction_time, INTERVAL '10' MINUTE)
```

SQL

as window_start

FROM transactions

GROUP BY

```
user_id,
    TUMBLE(transaction_time, INTERVAL '10' MINUTE)
    HAVING COUNT(DISTINCT location) > 3
""")

return velocity_alerts, geo_alerts

def setup_complex_cep_pattern(self):
    """Complex Event Processing for sophisticated fraud
```

SQL

patterns"""

```
# This would use Flink's CEP library in Java/Scala
# Python API for CEP is limited, showing conceptual
```

structure

```
fraud_pattern = """
Pattern:
1. Large withdrawal (amount > 5000)
2. Followed by multiple small transactions (amount < 100)
```

within 1 hour

SQL

3. From same user but different merchants
4. At least 5 such small transactions

```
This pattern suggests account takeover + money laundering
"""
```

```
return fraud_pattern
```

```
# Real Flink application structure (would be in Java/Scala for
```

production)

PYTHON

```
flink_fraud_detector = """
public class AdvancedFraudDetection {
    public static void main(String[] args) throws Exception {
        StreamExecutionEnvironment env =
```

```
StreamExecutionEnvironment.getExecutionEnvironment();
```

```
// Configure checkpointing for fault tolerance
env.enableCheckpointing(5000); // checkpoint every 5 seconds
```

```
env.getCheckpointConfig().setCheckpointingMode(CheckpointingMode.EXACTLY_ONCE);
```

SQL

```
// Read from Kafka
DataStream<Transaction> transactions = env
    .addSource(new FlinkKafkaConsumer<>("transactions", new
```

```
TransactionDeserializer(), props))
```

```
.assignTimestampsAndWatermarks(WatermarkStrategy
```

```
.
```

```
forBoundedOutOfOrderness(Duration.ofSeconds(20))
```

```
.withTimestampAssigner((event, timestamp) ->
```

```
event.getTimestamp()));
```

```
// CEP Pattern for account takeover
```

```
Pattern<Transaction, ?> accountTakeoverPattern = Pattern.
```

```
begin("largeWithdrawal")
```

```
.where(SimpleCondition.of(transaction ->
```

SQL

```
transaction.getAmount() > 5000))
```

```
.next("smallTransactions")  
.where(SimpleCondition.of(transaction ->
```

SQL

```
transaction.getAmount() < 100))
```

```
.timesOrMore(5)  
    .within(Time.hours(1));  
  
// Apply pattern  
PatternStream<Transaction> patternStream = CEP.pattern(  
    transactions.keyBy(Transaction::getUserId),  
    accountTakeoverPattern  
);  
  
// Extract matching patterns  
DataStream<Alert> alerts = patternStream.select(  
    (PatternSelectFunction<Transaction, Alert>) pattern -> {  
        Transaction largeWithdrawal =
```

SQL

```
pattern.get("largeWithdrawal").get(0);
```

```
List<Transaction> smallTransactions =
```

```
pattern.get("smallTransactions");
```

```
return new Alert(  
    largeWithdrawal.getUserId(),  
    "ACCOUNT_TAKEOVER",  
    largeWithdrawal.getAmount(),  
    smallTransactions.size()  
);  
}  
);  
  
// Send alerts to output sink  
alerts.addSink(new AlertSink());  
  
env.execute("Advanced Fraud Detection");  
}  
}
```

When to Choose Flink

□ Complex event processing (CEP) requirements

□ Checkpoint

1. □ Sub-second latency requirements

□ Checkpoint

1. □ Sophisticated windowing and state management

□ Checkpoint

1. □ Exactly-once processing guarantees critical

□ Checkpoint

1. □ Team has JVM expertise

Kafka Streams: Lightweight and Kafka-Native

```
# Kafka Streams is perfect for simpler processing tightly integrated
```

with Kafka

```
# Python example showing the conceptual approach (actual
```

implementation in Java)

```
class KafkaStreamsUserSessionization:
    """User sessionization using Kafka Streams"""

    def __init__(self):
        self.properties = {
            'application.id': 'user-sessionization',
            'bootstrap.servers': 'localhost:9092',
            'default.key.serde': 'Serdes.String()',
            'default.value.serde': 'Serdes.String()'
        }

    def build_sessionization_topology(self):
        """Build Kafka Streams topology for user sessionization"""

        topology_description = """
        Kafka Streams Topology:
```

PYTHON

The corresponding SQL query:

1. Source: user-events topic
2. Transform: Parse and enrich events
3. Group by: user_id
4. Session Windows: 30-minute inactivity gap
5. Aggregate: Count events, calculate session duration
6. Sink: user-sessions topic

```
"""
```

```
return topology_description
```

```
# Actual Kafka Streams code (Java)
```

```
kafka_streams_java = """
```

```
public class UserSessionization {
```

```
    public static void main(String[] args) {
```

```
        Properties props = new Properties();
```

```
        props.put(StreamsConfig.APPLICATION_ID_CONFIG, "user-
```

```
sessionization");
```

```
        props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG,
```

```
"localhost:9092");
```

```
        StreamsBuilder builder = new StreamsBuilder();
```

```
        // Input stream of user events
```

```
KStream<String, UserEvent> userEvents =
```

```
        builder.stream("user-events");
```

```
        // Session-based aggregation
```

```
KTable<Windowed<String>, SessionAggregate> userSessions =
```

```
userEvents
```



```

.groupByKey()
.windowedBy(SessionWindows.with(Duration.ofMinutes(30)))
.aggregate(
    () -> new SessionAggregate(), // Initializer
    (key, value, aggregate) -> { // Aggregator
        aggregate.addEvent(value);
        return aggregate;
    },
    (aggValue1, aggValue2) -> { // Merger for session

```

window merging

```

return aggValue1.merge(aggValue2);
    }
    );

// Transform to output format and send to output topic
userSessions
    .toStream()
    .map((windowedUserId, sessionAggregate) ->
        KeyValue.pair(
            windowedUserId.key(),
            sessionAggregate.toJson()
        )
    )
    .to("user-sessions");

// Start the streams application
KafkaStreams streams = new KafkaStreams(builder.build(),

```

PYTHON

props);

```

streams.start();

// Graceful shutdown
Runtime.getRuntime().addShutdownHook(new

```

```

Thread(streams::close));

```

PYTHON

```
}
}
"""
```

```
class KafkaStreamsRealTimeMetrics:
    """Real-time metrics calculation with Kafka Streams"""

    def design_metrics_topology(self):
        """Design for real-time business metrics"""
```

```
return {
    'input_topics': ['page_views', 'purchases',
```

```
'user_signups'],
```

```
'processing_steps': [
    {
        'step': 'parse_and_filter',
        'description': 'Parse JSON events and filter
```

```
invalid data'
```

SQL

```
},
{
    'step': 'enrich_events',
    'description': 'Join with user profile data from
```

```
global table'
```

```
},
{
    'step': 'windowed_aggregation',
    'description': 'Tumbling 1-minute windows for
```

```
real-time metrics'
```

```

    },
    {
      'step': 'format_output',
      'description': 'Convert to dashboard-friendly

```

format'

```

    }
  ],
  'output_topics': ['real_time_metrics',

```

'user_behavior_metrics'],

```

  'state_stores': ['user_profiles_store'],
  'scaling_strategy': {
    'partitions': 12,
    'instances': 3, # 3 instances × 4 threads = 12

```

total threads

```

    'threads_per_instance': 4
  }
}

```

When to Choose Kafka Streams

□ Already using Kafka ecosystem heavily

□ Checkpoint

1. □ Simpler processing logic (aggregations, joins, filtering)

□ Checkpoint

1. □ Want to avoid separate cluster management

□ Checkpoint

1. □ Java/Scala development team

□ Checkpoint

1. □ Need tight integration with Kafka Connect

Spark Structured Streaming: Batch-Stream

Unification

```
# Spark Structured Streaming for teams familiar with Spark batch
```

processing

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

class SparkStreamingPipeline:
    """Unified batch and stream processing with Spark"""

    def __init__(self):
        self.spark = SparkSession.builder \
            .appName("StreamingAnalytics") \
            .config("spark.sql.streaming.schemaInference", "true") \
            .getOrCreate()

    def setup_streaming_etl(self):
        """Set up streaming ETL pipeline"""

        # Define schema for incoming events
        event_schema = StructType([
            StructField("event_id", StringType(), True),
            StructField("user_id", StringType(), True),
            StructField("event_type", StringType(), True),
            StructField("timestamp", TimestampType(), True),
            StructField("properties", MapType(StringType(),

```

```
StringType()), True)
```

```

])

# Read from Kafka stream
raw_events = self.spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "user-events") \
    .load()

# Parse JSON and apply schema
parsed_events = raw_events.select(
    from_json(col("value").cast("string"),

```

```
event_schema).alias("data")
```

```

).select("data.*")

# Add watermark for handling late data
events_with_watermark = parsed_events \
    .withWatermark("timestamp", "10 minutes")

return events_with_watermark

def real_time_aggregations(self, events_df):
    """Create real-time aggregations"""

    # 1. Real-time event counts by type
    event_counts = events_df \
        .groupBy(
            window(col("timestamp"), "1 minute"),
            col("event_type")
        ) \
        .count() \
        .select(
            col("window.start").alias("window_start"),
            col("window.end").alias("window_end"),
            col("event_type"),
            col("count").alias("event_count")
        )

    # 2. Active user counts in 5-minute windows
    active_users = events_df \
        .groupBy(
            window(col("timestamp"), "5 minutes"),
            col("user_id")
        ) \
        .count() \
        .groupBy("window") \
        .count() \
        .select(
            col("window.start").alias("window_start"),
            col("window.end").alias("window_end"),
            col("count").alias("active_users")
        )

    return event_counts, active_users

def setup_output_sinks(self, event_counts_df, active_users_df):
    """Set up output sinks for processed data"""

    # Write event counts to console (for debugging)
    event_counts_query = event_counts_df.writeStream \
        .outputMode("append") \

```

```

        .format("console") \
        .trigger(processingTime="30 seconds") \
        .start()

# Write active users to Delta table for analytics
active_users_query = active_users_df.writeStream \
    .outputMode("append") \
    .format("delta") \
    .option("path", "/delta/active_users") \
    .option("checkpointLocation",

```

"/checkpoints/active_users") \

```

        .trigger(processingTime="1 minute") \
        .start()

# Write to Kafka for downstream consumers
kafka_output_query = event_counts_df \
    .select(to_json(struct("*")).alias("value")) \
    .writeStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("topic", "real-time-metrics") \
    .option("checkpointLocation",

```

SQL

"/checkpoints/kafka_output") \

```

        .start()

return [event_counts_query, active_users_query,

```

kafka_output_query]

```

def run_streaming_pipeline(self):
    """Run the complete streaming pipeline"""

    # Set up streaming source
    events_df = self.setup_streaming_etl()

    # Create aggregations
    event_counts_df, active_users_df =

```

PYTHON

```
self.real_time_aggregations(events_df)
```

```
# Set up output sinks  
queries = self.setup_output_sinks(event_counts_df,
```

PYTHON

```
active_users_df)
```



```

# Wait for all queries to terminate
for query in queries:
    query.awaitTermination()

# Advanced Spark Streaming patterns
class AdvancedSparkStreamingPatterns:
    """Advanced patterns for production Spark Streaming"""

    def stream_to_batch_join(self, spark):
        """Join streaming data with batch dimension tables"""

        # Read streaming events
        streaming_orders = spark.readStream \
            .format("kafka") \
            .option("kafka.bootstrap.servers", "localhost:9092") \
            .option("subscribe", "orders") \
            .load()

        # Read batch dimension table (slowly changing)
        customer_dim = spark.read \
            .format("delta") \
            .load("/delta/customer_dimension")

        # Broadcast join for efficiency
        enriched_orders = streaming_orders \
            .join(broadcast(customer_dim), "customer_id") \
            .select(
                col("order_id"),
                col("customer_id"),
                col("customer_tier"),
                col("order_amount"),
                col("order_timestamp")
            )

        return enriched_orders

    def deduplication_pattern(self, spark):
        """Deduplicate events in streaming data"""

        events = spark.readStream.format("kafka").load()

```

Deduplicate based on event_id within 1-hour window:

```
# Deduplicate based on event_id within 1-hour window
deduplicated = events \
    .withWatermark("timestamp", "1 hour") \
    .dropDuplicates(["event_id", "timestamp"])

return deduplicated

def exactly_once_processing(self, spark):
    """Implement exactly-once processing with idempotent
```

sinks"""

```
processed_events = spark.readStream.format("kafka").load()

# Write to idempotent sink with unique keys
query = processed_events.writeStream \
    .format("delta") \
    .option("path", "/delta/processed_events") \
    .option("checkpointLocation",
```

"/checkpoints/exactly_once") \

```
.outputMode("append") \
    .trigger(processingTime="30 seconds") \
    .start()

return query
```

When to Choose Spark Structured Streaming

□ Team already familiar with Spark batch processing

□ Checkpoint

1. □ Need to unify batch and stream processing code

❑ Checkpoint

1. ❑ Complex SQL-based transformations

❑ Checkpoint

1. ❑ Integration with Delta Lake or other Spark ecosystem

❑ Checkpoint

1. ❑ Micro-batch processing acceptable (not true streaming)

Cloud-Native Streaming Solutions

```
# AWS Kinesis Analytics example
class KinesisAnalyticsPipeline:
    """Serverless stream processing with Kinesis Analytics"""

    def setup_kinesis_sql_application(self):
        """Set up SQL-based stream processing"""

        kinesis_sql = """
        CREATE OR REPLACE STREAM "DESTINATION_SQL_STREAM" (
            window_start TIMESTAMP,
            event_type VARCHAR(50),
            event_count INTEGER,
            unique_users INTEGER
        );

        CREATE OR REPLACE PUMP "STREAM_PUMP" AS INSERT INTO
```

PYTHON

"DESTINATION_SQL_STREAM"

SELECT STREAM

```
ROWTIME_TO_TIMESTAMP(  
  RANGE_START(  
    CHAR_TO_TIMESTAMP('yyyy-MM-dd HH:mm:ss',
```

'event_timestamp')

```
)  
  ) as window_start,  
  event_type,  
  COUNT(*) as event_count,  
  COUNT(DISTINCT user_id) as unique_users  
FROM "SOURCE_SQL_STREAM_001"  
WHERE event_timestamp IS NOT NULL  
GROUP BY  
  event_type,  
  RANGE(  
    CHAR_TO_TIMESTAMP('yyyy-MM-dd HH:mm:ss',
```

SQL

event_timestamp)

```
RANGE INTERVAL '1' MINUTE  
  );  
""
```

```
return {  
  'application_name': 'RealTimeAnalytics',  
  'sql_queries': kinesis_sql,  
  'input_streams': ['user-events-kinesis-stream'],  
  'output_destinations': ['processed-events-kinesis-
```

stream'],

PYTHON

```
'scaling': {
    'parallelism': 4,
    'parallelism_per_kpu': 1
}

# Google Cloud Dataflow example
class DataflowStreamingPipeline:
    """Apache Beam pipeline for Google Cloud Dataflow"""

    def build_beam_pipeline(self):
        """Build Apache Beam pipeline for Dataflow"""

        beam_pipeline = """
        import apache_beam as beam
        from apache_beam.options.pipeline_options import
```

PipelineOptions

PYTHON

```
def run_pipeline():
    pipeline_options = PipelineOptions([
        '--project=my-project',
        '--region=us-central1',
        '--runner=DataflowRunner',
        '--streaming',
        '--temp_location=gs://my-bucket/temp'
    ])

    with beam.Pipeline(options=pipeline_options) as
```

pipeline:

SQL

```
# Read from Pub/Sub
events = (
    pipeline
    | 'ReadFromPubSub' >> beam.io.ReadFromPubSub(
        subscription='projects/my-
```

project/subscriptions/events-sub'

```

    )
    | 'ParseJSON' >> beam.Map(json.loads)
    )

# Windowed aggregation
windowed_counts = (
    events
    | 'AddTimestamps' >> beam.Map(lambda x:

```

```
beam.window.TimestampedValue(
```

```

x, x['timestamp']
))
| 'WindowIntoFixed' >> beam.WindowInto(
    beam.window.FixedWindows(60) # 1-minute

```

windows

```

    )
    | 'GroupByEventType' >> beam.Map(lambda x:

```

```
(x['event_type'], 1))
```

```

    | 'CountByWindow' >> beam.CombinePerKey(sum)
    )

```

```

# Write to BigQuery
windowed_counts | 'WriteToBigQuery' >>

```

```
beam.io.WriteToBigQuery(
```

```
'my-project:analytics.event_counts',
```

```

schema='event_type:STRING,count:INTEGER>window_start:TIMESTAMP',
write_disposition=beam.io.BigQueryDisposition.WRITE_APPEND

```

```
)  
  
run_pipeline()
```

The corresponding SQL query:

```
"""  
  
    return beam_pipeline  
  
# Azure Stream Analytics  
class AzureStreamAnalyticsPipeline:  
    """Stream processing with Azure Stream Analytics"""  
  
    def setup_asa_query(self):  
        """Set up Stream Analytics SQL query"""  
  
        asa_sql = """  
        WITH EventCounts AS (  
            SELECT  
                event_type,  
                COUNT(*) as event_count,  
                COUNT(DISTINCT user_id) as unique_users,  
                System.Timestamp() AS window_end  
            FROM EventHub  
            GROUP BY  
                event_type,  
                TumblingWindow(minute, 1)  
        )  
  
        SELECT  
            event_type,  
            event_count,  
            unique_users,  
            window_end  
        INTO OutputServiceBus  
        FROM EventCounts;  
  
        -- Anomaly detection query  
        WITH AnomalyDetection AS (  
            SELECT  
                event_type,  
                COUNT(*) as current_count,  
                AnomalyDetection(COUNT(*)) OVER (LIMIT
```

SQL

```
DURATION(hour, 1)) as anomaly_score,
```

SQL

```
System.Timestamp() as event_time
FROM EventHub
GROUP BY
    event_type,
    TumblingWindow(minute, 5)
)

SELECT *
INTO AlertOutput
FROM AnomalyDetection
WHERE anomaly_score > 0.8;
"""

return {
    'query': asa_sql,
    'inputs': ['EventHub'],
    'outputs': ['OutputServiceBus', 'AlertOutput'],
    'streaming_units': 6 # Scaling units
}
```

When to Choose Cloud-Native Solutions

□ Want fully managed services with minimal operations

□ **Checkpoint**

1. □ Simpler processing requirements (aggregations, filtering)

□ **Checkpoint**

1. □ Integration with cloud ecosystem (BigQuery, S3, etc.)

□ **Checkpoint**

1. □ Variable/unpredictable workloads

❏ Checkpoint

- ❏ Small team without streaming expertise

Framework Selection Decision Tree

```
def choose_streaming_framework(requirements):  
    """Decision tree for choosing streaming framework"""  
  
    # Primary decision factors
```

PYTHON

```
complexity = requirements.get('processing_complexity') #
```

simple, medium, complex

```
latency_requirement = requirements.get('latency_ms')    #
```

milliseconds

```
team_expertise = requirements.get('team_skills')        # java,
```

python, sql, devops

```
operational_preference = requirements.get('operations') #
```

managed, self-hosted

```
decision_tree = {
    'complex_processing': {
        'condition': complexity == 'complex',
        'recommendation': 'Apache Flink',
        'reasons': [
            'Complex Event Processing (CEP) capabilities',
            'Advanced windowing and state management',
            'Exactly-once processing guarantees',
            'Sub-second latency possible'
        ]
    },
    'kafka_ecosystem': {
        'condition': (
            complexity in ['simple', 'medium'] and
            'kafka' in
```

```
requirements.get('existing_infrastructure', [])
```

```

    ),
    'recommendation': 'Kafka Streams',
    'reasons': [
        'Native Kafka integration',
        'Lightweight deployment',
        'Simple scaling model',
        'Good for Kafka-centric architectures'
    ]
},
'spark_familiarity': {
    'condition': (
        'spark' in team_expertise and
        requirements.get('batch_stream_unification', False)
    ),
    'recommendation': 'Spark Structured Streaming',
    'reasons': [
        'Unified batch and stream processing',
        'Familiar Spark APIs',
        'Good ecosystem integration',
        'Micro-batch acceptable'
    ]
},
'managed_simplicity': {
    'condition': (
        operational_preference == 'managed' and
        complexity == 'simple'
    ),
    'recommendation': 'Cloud Native (Kinesis/Dataflow/ASA)',
    'reasons': [
        'Fully managed operations',
        'Pay-per-use scaling',
        'Cloud ecosystem integration',
        'Minimal operational overhead'
    ]
}
}

# Evaluate decision tree
for criteria, config in decision_tree.items():
    if config['condition']:
        return {
            'recommended_framework': config['recommendation'],
            'reasons': config['reasons'],
            'criteria_matched': criteria
        }

return {

```

The PYTHON code:

```
'recommended_framework': 'Start with cloud-native solution',
    'reasons': ['Safe default for most use cases'],
    'criteria_matched': 'default'
}

# Example usage
requirements = {
    'processing_complexity': 'medium',
    'latency_ms': 1000,
    'team_skills': ['python', 'sql'],
    'operations': 'managed',
    'existing_infrastructure': ['kafka', 'spark'],
    'batch_stream_unification': True
}

recommendation = choose_streaming_framework(requirements)
print(f"Recommended: {recommendation['recommended_framework']}")
```

PYTHON

⚠ Common Mistake

⚠ Common Mistake Don't choose a framework based on hype or resume-driven development. Start with the simplest solution that meets your requirements. You can always migrate to more sophisticated frameworks as your needs grow.

□ Checkpoint

1. When should you choose Apache Flink over Kafka Streams? a. When you need managed services b. When you need complex event processing and sub-second

latency

- c. When your team knows Python better than Java d. When you want simpler operations
2. What's the main advantage of Spark Structured Streaming? a. It's the fastest framework b. It unifies batch and stream processing with familiar APIs c. It's cheaper than alternatives d. It requires no configuration
3. When are cloud-native streaming solutions the best choice? a. For complex event processing b. For high-throughput re-

quirements c. For simple processing with managed operations d. For exactly-once processing guarantees

Answers: 1-b, 2-b, 3-c

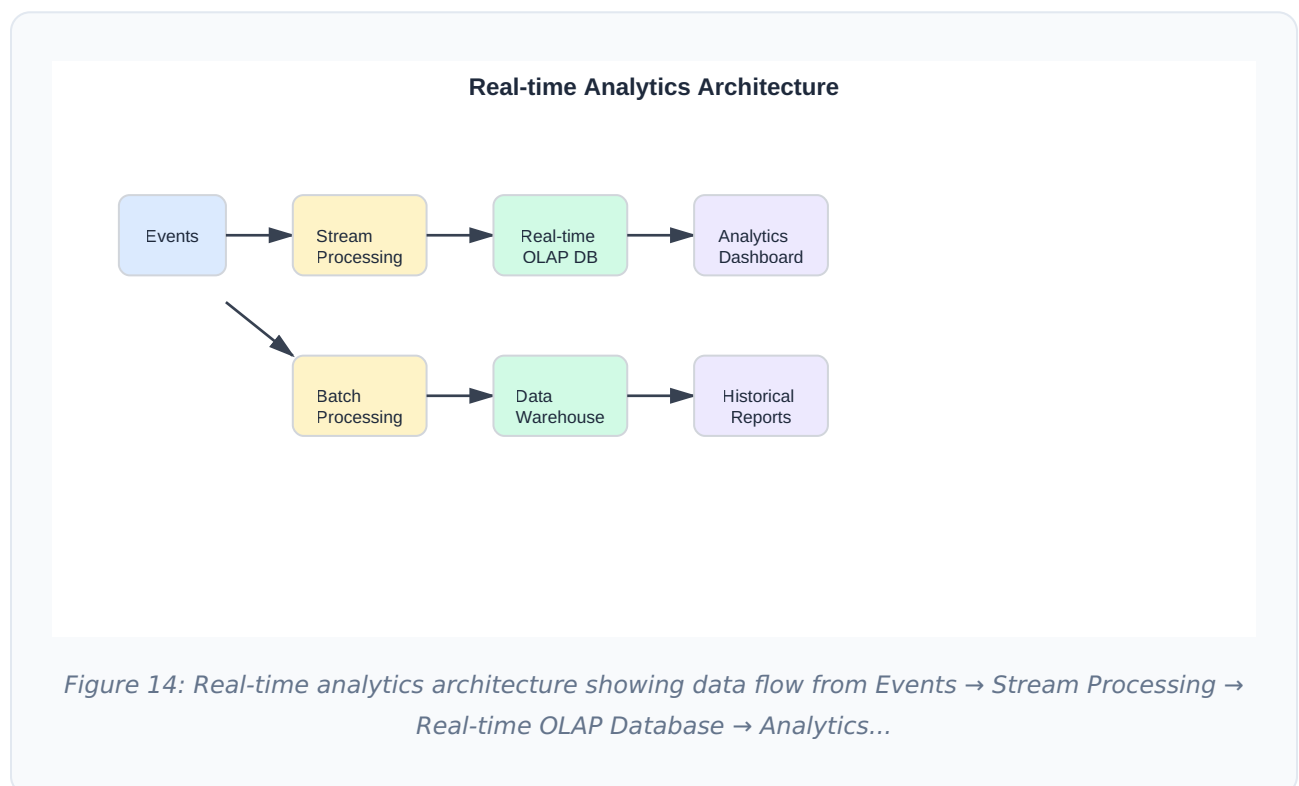
4.4 Real-Time Analytics Architectures

TL;DR

OLAP databases (ClickHouse, Druid, Pinot) power real-time analytics dashboards - Lambda architecture separates batch accuracy from stream speed - Kappa architecture uses streaming for everything but requires more sophistication - Pre-aggregation and materialized views for sub-second query performance - Most "real-time" analytics can tolerate 1-5 minute latency - question the requirements

The Real-Time Analytics Stack

Real-time analytics requires different architecture patterns than traditional batch analytics. Users expect sub-second query responses on data that's minutes or seconds old.



```
class RealTimeAnalyticsArchitecture:
    """Architecture for real-time analytics system"""
```

PYTHON

The `__init__` implementation:

```
def __init__(self):
    self.speed_layer = StreamProcessingEngine()    # Real-time
```

PYTHON

processing

```
self.batch_layer = BatchProcessingEngine()    # Historical
```

PYTHON

accuracy

```
self.serving_layer = RealTimeOLAP()    # Query
```

PYTHON

serving

```
self.dashboard = AnalyticsDashboard()    # User
```

PYTHON

interface

```
def design_speed_layer(self):
    """Design real-time processing for fresh data"""

    return {
        'input_streams': ['user_events', 'transactions',
```

PYTHON

'system_metrics'],

```

'processing_windows': {
    'real_time_metrics': '1 minute tumbling windows',
    'trending_analysis': '5 minute sliding windows',
    'anomaly_detection': '30 second tumbling windows'
},
'aggregations': [
    'COUNT events by type',
    'SUM revenue by merchant',
    'AVG response time by service',
    'DISTINCT active users',
    'TOP 10 trending content'
],
'output_destinations': ['real_time_olap_db',

```

'alert_system'],

```

'latency_target': '< 30 seconds end-to-end'
}

def design_serving_layer(self):
    """Design serving layer for fast analytical queries"""

    return {
        'database_choice': 'ClickHouse', # or Druid, Pinot
        'data_model': {
            'fact_tables': [
                'real_time_events',      # Raw events with 1-day

```

PYTHON

retention

```

'minute_aggregates',      # Pre-computed minute-

```

level metrics

```

'hourly_aggregates',      # Pre-computed hour-

```

level metrics


```
],
'dimensions': [
    'users', 'content', 'geography',
```

'time_dimensions'

```
]
},
'indexing_strategy': {
    'primary_keys': ['timestamp', 'user_id',
```

'event_type'],

```
'secondary_indexes': ['geography', 'content_id'],
    'bloom_filters': True, # For fast existence checks
},
'query_optimization': {
    'materialized_views': True,
    'pre_aggregation': True,
    'result_caching': '5 minutes TTL'
}
}
```

ClickHouse for Real-Time Analytics

```
# ClickHouse excels at real-time analytical queries
import clickhouse_connect
from datetime import datetime, timedelta

class ClickHouseRealTimeAnalytics:
```

PYTHON

Real-time analytics with ClickHouse:

```
"""Real-time analytics with ClickHouse"""

def __init__(self, host='localhost', port=8123):
    self.client = clickhouse_connect.get_client(host=host,
```

PYTHON

```
port=port)
```

```

def setup_real_time_tables(self):
    """Set up optimized tables for real-time analytics"""

    # Raw events table with automatic partitioning
    events_table_ddl = """
CREATE TABLE IF NOT EXISTS events (
    timestamp DateTime64(3),
    event_id String,
    user_id UInt64,
    event_type LowCardinality(String),
    content_id Nullable(UInt64),
    session_id String,
    properties Map(String, String),
    revenue Nullable(Decimal(10,2))
)
ENGINE = MergeTree()
PARTITION BY toYYYYMMDD(timestamp)
ORDER BY (timestamp, user_id, event_type)
TTL timestamp + INTERVAL 7 DAY -- Auto-delete old data
SETTINGS index_granularity = 8192;
"""

    # Pre-aggregated table for fast dashboard queries
    metrics_table_ddl = """
CREATE TABLE IF NOT EXISTS minute_metrics (
    minute_start DateTime,
    event_type LowCardinality(String),
    event_count UInt64,
    unique_users UInt64,
    total_revenue Decimal(12,2),
    avg_session_duration Float32
)
ENGINE = SummingMergeTree()
PARTITION BY toYYYYMM(minute_start)
ORDER BY (minute_start, event_type)
TTL minute_start + INTERVAL 30 DAY;
"""

    # Materialized view to automatically populate metrics
    materialized_view_ddl = """
CREATE MATERIALIZED VIEW IF NOT EXISTS minute_metrics_mv
TO minute_metrics
AS SELECT
    toStartOfMinute(timestamp) as minute_start,
    event_type,
    count() as event_count,
    uniq(user_id) as unique_users,
    sum(revenue) as total_revenue,

```

```

        avg(session_duration) as avg_session_duration
    FROM events
    GROUP BY minute_start, event_type;
    """

    self.client.command(events_table_ddl)
    self.client.command(metrics_table_ddl)
    self.client.command(materialized_view_ddl)

    def insert_real_time_events(self, events_batch):
        """Insert batch of events efficiently"""

        # ClickHouse handles large batches very efficiently
        self.client.insert('events', events_batch,
                           column_names=['timestamp', 'event_id',

```

'user_id',

'event_type', 'content_id',

'session_id',

```

'properties', 'revenue']]

def get_real_time_dashboard_metrics(self, minutes_back=60):
    """Get metrics for real-time dashboard"""

    # Fast query using pre-aggregated data
    query = f"""
    SELECT
        minute_start,
        event_type,
        event_count,
        unique_users,
        total_revenue
    FROM minute_metrics
    WHERE minute_start >= now() - INTERVAL {minutes_back} MINUTE
    ORDER BY minute_start DESC, event_count DESC
    """

    return self.client.query(query).result_rows

def get_trending_content(self, hours_back=2):
    """Get trending content in real-time"""

    query = f"""
    WITH content_stats AS (
        SELECT
            content_id,
            count() as total_views,
            uniq(user_id) as unique_viewers,
            count() / uniq(user_id) as virality_score
        FROM events
        WHERE timestamp >= now() - INTERVAL {hours_back} HOUR
            AND event_type = 'content_view'
            AND content_id IS NOT NULL
        GROUP BY content_id
        HAVING total_views >= 100 -- Minimum threshold
    )
    SELECT
        content_id,
        total_views,
        unique_viewers,
        virality_score
    FROM content_stats
    ORDER BY virality_score DESC
    LIMIT 20
    """

    return self.client.query(query).result_rows

```

```
def get_user_cohort_analysis(self, cohort_date):
    """Real-time cohort analysis"""

    query = f"""
    WITH user_cohorts AS (
        SELECT
            user_id,
            toDate(min(timestamp)) as cohort_date,
            toDate(timestamp) as activity_date,
            dateDiff('day', cohort_date, activity_date) as
```

day_number

FROM events

```
WHERE timestamp >= '{cohort_date}'
    GROUP BY user_id, toDate(timestamp)
),
cohort_data AS (
    SELECT
        cohort_date,
        day_number,
        count(DISTINCT user_id) as active_users
    FROM user_cohorts
    GROUP BY cohort_date, day_number
),
```

SQL

The corresponding SQL query:

```
cohort_sizes AS (  
    SELECT  
        cohort_date,  
        active_users as cohort_size  
    FROM cohort_data  
    WHERE day_number = 0  
)  
SELECT  
    cd.cohort_date,  
    cd.day_number,  
    cd.active_users,  
    cs.cohort_size,  
    cd.active_users / cs.cohort_size as retention_rate  
FROM cohort_data cd  
JOIN cohort_sizes cs ON cd.cohort_date = cs.cohort_date  
ORDER BY cd.cohort_date, cd.day_number  
"""  
  
return self.client.query(query).result_rows
```

Apache Druid for Time-Series Analytics

PYTHON

```
# Apache Druid for real-time OLAP with excellent time-series support
class DruidRealTimeAnalytics:
    """Real-time analytics with Apache Druid"""

    def __init__(self, druid_host='localhost', druid_port=8082):
        self.druid_host = druid_host
        self.druid_port = druid_port
        self.base_url = f"http://{druid_host}:{druid_port}"

    def setup_druid_datasource(self):
        """Set up Druid datasource with real-time ingestion"""

        # Druid ingestion spec for Kafka streaming
        ingestion_spec = {
            "type": "kafka",
            "spec": {
                "dataSchema": {
                    "dataSource": "user_events",
                    "timestampSpec": {
                        "column": "timestamp",
                        "format": "iso",
                        "missingValue": None
                    },
                    "dimensionsSpec": {
                        "dimensions": [
                            "user_id",
                            "event_type",
                            "content_id",
                            "country",
                            "device_type"
                        ]
                    },
                    "metricsSpec": [
                        {"type": "count", "name": "event_count"},
                        {"type": "longSum", "name": "revenue",
```

```
"fieldName": "revenue_cents"}},
```

JSON

```
{"type": "hyperUnique", "name":
```

```
"unique_users", "fieldName": "user_id"}
```



```

],
  "granularitySpec": {
    "type": "uniform",
    "segmentGranularity": "HOUR",
    "queryGranularity": "MINUTE",
    "rollup": True
  }
},
"ioConfig": {
  "topic": "user_events",
  "consumerProperties": {
    "bootstrap.servers": "localhost:9092"
  }
}

```

The corresponding SQL query:

```

},
    "taskCount": 3,
    "replicas": 2,
    "taskDuration": "PT1H" # 1 hour task duration
  },
  "tuningConfig": {
    "type": "kafka",
    "maxRowsInMemory": 100000,
    "maxBytesInMemory": 50000000,
    "intermediatePersistPeriod": "PT10M",
    "maxPendingPersists": 0,
    "reportParseExceptions": True
  }
}

```

return ingestion_spec

```

def query_real_time_metrics(self, start_time, end_time):
    """Query real-time metrics from Druid"""

```

Druid native query for fast aggregation

```

native_query = {
    "queryType": "timeseries",
    "dataSource": "user_events",
    "granularity": "minute",
    "intervals": [f"{start_time}/{end_time}"],
    "aggregations": [
        {"type": "longSum", "name": "total_events",

```

"fieldName": "event_count"}},

SQL

```
{"type": "longSum", "name": "total_revenue",
```

JSON

```
"fieldName": "revenue"},
```

```
{"type": "cardinality", "name": "unique_users",
```

JSON

```
"fields": ["user_id"]}
```

```
],  
"postAggregations": [  
  {  
    "type": "arithmetic",  
    "name": "revenue_per_user",  
    "fn": "/",  
    "fields": [  
      {"type": "fieldAccess", "fieldName":
```

```
"total_revenue"},
```

```
{"type": "fieldAccess", "fieldName":
```

JSON

```
"unique_users"}]
```

```

    ]
        }
    ]
}

return native_query

def query_top_content_by_geography(self):
    """Get top content by geography in real-time"""

    # SQL query to Druid (also supports native JSON queries)
    sql_query = """
    SELECT
        country,
        content_id,
        SUM(event_count) as total_views,
        COUNT(DISTINCT user_id) as unique_viewers
    FROM user_events
    WHERE __time >= CURRENT_TIMESTAMP - INTERVAL '1' HOUR
        AND event_type = 'content_view'
    GROUP BY country, content_id
    ORDER BY total_views DESC
    LIMIT 50
    """

    return sql_query

```

Materialized Views and Pre-Aggregation

```

class MaterializedViewStrategy:
    """Strategy for materialized views in real-time analytics"""

    def __init__(self, olap_database):
        self.database = olap_database

    def design_mv_hierarchy(self):
        """Design hierarchy of materialized views for different
latencies"""

```

```
mv_strategy = {
  'raw_events': {
    'retention': '24 hours',
    'granularity': 'second',
    'use_case': 'Real-time debugging, detailed analysis'
  },
  'minute_aggregates': {
    'retention': '7 days',
    'granularity': 'minute',
    'dimensions': ['event_type', 'country',
```

'device_type'],

```
'metrics': ['event_count', 'unique_users',
```

'revenue'],

```
'use_case': 'Real-time dashboards'
},
'hour_aggregates': {
  'retention': '90 days',
  'granularity': 'hour',
  'dimensions': ['event_type', 'country',
```

'device_type', 'content_category'],

```
'metrics': ['event_count', 'unique_users',
```

'revenue', 'avg_session_duration'],

```
'use_case': 'Business analytics, trending analysis'
},
'daily_aggregates': {
  'retention': '2 years',
  'granularity': 'day',
  'dimensions': ['event_type', 'country',
```

'cohort_week', 'content_category'],

```
'metrics': ['event_count', 'unique_users',
```

```
'revenue', 'retention_rate'],
```

```
'use_case': 'Historical analysis, cohort analysis'
    }
}

return mv_strategy

def create_incremental_mv_update_job(self):
    """Create job to incrementally update materialized views"""

    update_logic = """
    -- Incremental update strategy for minute aggregates
    INSERT INTO minute_aggregates
    SELECT
        toStartOfMinute(timestamp) as minute_start,
        event_type,
        country,
        device_type,
        count() as event_count,
        uniq(user_id) as unique_users,
        sum(revenue_cents) / 100.0 as revenue
    FROM raw_events
    WHERE timestamp >= (
        SELECT max(minute_start) FROM minute_aggregates
    ) - INTERVAL 5 MINUTE -- 5-minute overlap for late data
    GROUP BY minute_start, event_type, country, device_type;

    -- Clean up old raw events to manage storage
    ALTER TABLE raw_events DELETE
    WHERE timestamp < now() - INTERVAL 25 HOUR;
```

SQL

```
"""

return {
    'schedule': 'every 1 minute',
    'logic': update_logic,
    'monitoring': {
        'lag_alert_threshold': '2 minutes',
        'failure_alert': 'immediate',
        'data_quality_checks': ['row_count_validation',
```

'revenue_sum_validation']

```
}  
    }  
  
# Advanced aggregation patterns  
class RealTimeAggregationPatterns:  
    """Advanced patterns for real-time aggregations"""  
  
    def sliding_window_aggregation(self):  
        """Sliding window aggregation for trending analysis"""  
  
        # Maintain sliding window state for trending calculations  
        sliding_window_sql = """  
        WITH sliding_window AS (  
            SELECT  
                content_id,  
                timestamp,  
                count() OVER (  
                    PARTITION BY content_id  
                    ORDER BY timestamp  
                    RANGE BETWEEN INTERVAL 1 HOUR PRECEDING AND
```

SQL

CURRENT ROW

```
) as views_last_hour,  
count() OVER (  
    PARTITION BY content_id  
    ORDER BY timestamp  
    RANGE BETWEEN INTERVAL 24 HOUR PRECEDING AND
```

SQL

CURRENT ROW

```
) as views_last_day
    FROM events
    WHERE event_type = 'content_view'
        AND timestamp >= now() - INTERVAL 25 HOUR
)
SELECT
    content_id,
    views_last_hour,
    views_last_day,
    views_last_hour::FLOAT / NULLIF(views_last_day, 0) as
```

SQL

trending_score

```
FROM sliding_window
    WHERE timestamp = (
        SELECT max(timestamp) FROM sliding_window sw2
        WHERE sw2.content_id = sliding_window.content_id
    )
ORDER BY trending_score DESC
LIMIT 100;
"""

return sliding_window_sql

def approximate_algorithms_for_scale(self):
    """Use approximate algorithms for massive scale"""

    approximate_queries = {
        'hyperloglog_unique_users': """
            SELECT
                date_trunc('hour', timestamp) as hour,
                event_type,
                uniq(user_id) as unique_users_approximate
            FROM events
            GROUP BY hour, event_type
            -- HyperLogLog provides ~1% error for unique counts
        """,
```

SQL

The corresponding SQL query:

```
'tdigest_percentiles': ""
SELECT
    date_trunc('minute', timestamp) as minute,
    quantile(0.5)(response_time_ms) as
```

SQL

p50_response_time,

```
quantile(0.95)(response_time_ms) as
```

p95_response_time,

```
quantile(0.99)(response_time_ms) as
```

p99_response_time

```
FROM performance_events
GROUP BY minute
-- T-Digest for approximate percentile calculations
"",
'count_min_sketch_frequency': ""
-- Count-Min Sketch for approximate frequency
```

SQL

counting

```
-- (Implementation depends on specific database)
SELECT
    content_id,
    approx_count_distinct(session_id) as
```

SQL

estimated_sessions

FROM events

```
WHERE event_type = 'content_view'
      GROUP BY content_id
      """
}

return approximate_queries
```

SQL

Caching Strategies for Real-Time Analytics

```
import redis
import json
from typing import Dict, List, Optional

class RealTimeAnalyticsCache:
    """Multi-layer caching for real-time analytics"""

    def __init__(self, redis_host='localhost', redis_port=6379):
        self.redis_client = redis.Redis(host=redis_host,
```

PYTHON

```
port=redis_port)
```

```

self.cache_ttls = {
    'real_time_metrics': 60,      # 1 minute
    'trending_content': 300,     # 5 minutes
    'user_segments': 600,       # 10 minutes
    'dashboard_summary': 30      # 30 seconds
}

def cache_dashboard_metrics(self, metrics_data: Dict):
    """Cache dashboard metrics with appropriate TTL"""

    cache_key = "dashboard:real_time_metrics"

    # Store with expiration
    self.redis_client.setex(
        cache_key,
        self.cache_ttls['real_time_metrics'],
        json.dumps(metrics_data)
    )

    # Also store individual metric components for partial

```

updates

```

for metric_name, metric_value in metrics_data.items():
    component_key = f"metric:{metric_name}"
    self.redis_client.setex(
        component_key,
        self.cache_ttls['real_time_metrics'],
        json.dumps(metric_value)
    )

def get_cached_metrics_with_fallback(self, fallback_query_func):
    """Get metrics from cache with database fallback"""

```

The PYTHON code:

```

cache_key = "dashboard:real_time_metrics"
cached_data = self.redis_client.get(cache_key)

if cached_data:
    return json.loads(cached_data)

# Cache miss - query database and cache result
fresh_data = fallback_query_func()
self.cache_dashboard_metrics(fresh_data)

return fresh_data

def invalidate_cache_on_schema_change(self, affected_metrics:

```

List[str]):

```

"""Invalidate cache when underlying data schema changes"""

keys_to_delete = []

for metric in affected_metrics:
    # Find all cache keys related to this metric
    pattern = f"*{metric}*"
    matching_keys = self.redis_client.keys(pattern)
    keys_to_delete.extend(matching_keys)

if keys_to_delete:
    self.redis_client.delete(*keys_to_delete)

# Query result caching with intelligent invalidation
class IntelligentQueryCache:
    """Smart caching that understands data freshness requirements"""

    def __init__(self, redis_client):
        self.redis = redis_client
        self.query_cache_configs = {}

    def register_query_cache_config(self, query_id: str, config:

```

Dict):

```

"""Register caching configuration for specific query

```

types"""

```
self.query_cache_configs[query_id] = {  
    'ttl_seconds': config.get('ttl_seconds', 300),  
    'refresh_threshold': config.get('refresh_threshold',
```

PYTHON

```
0.8), # Refresh at 80% of TTL
```

```
'data_dependencies': config.get('data_dependencies',
```

[[]), # Tables this query depends on

```
'invalidation_strategy':
```

```
config.get('invalidation_strategy', 'ttl_only')
```

```
}
```

```
def execute_cached_query(self, query_id: str, query_params:
```

PYTHON

Dict,

```
query_executor_func):  
    """Execute query with intelligent caching"""  
  
    # Create cache key from query ID and parameters  
    cache_key = f"query:{query_id}:"
```

SQL

```
{hash(str(sorted(query_params.items()))))}"
```

```
# Check cache  
cached_result = self.redis.hgetall(cache_key)  
  
if cached_result and self._is_cache_valid(query_id,
```

PYTHON

cached_result):

```

return json.loads(cached_result[b'data'])

# Cache miss or expired - execute query
fresh_result = query_executor_func(query_params)

# Cache the result
self._cache_query_result(query_id, cache_key, fresh_result)

```

PYTHON

The PYTHON code:

```

return fresh_result

def _is_cache_valid(self, query_id: str, cached_result: Dict) ->

```

PYTHON

bool:

```

"""Check if cached result is still valid"""

if not cached_result:
    return False

config = self.query_cache_configs.get(query_id, {})
cached_timestamp = float(cached_result.get(b'timestamp', 0))
current_time = time.time()

# Check TTL
ttl = config.get('ttl_seconds', 300)
if current_time - cached_timestamp > ttl:
    return False

# Check if we're past refresh threshold
refresh_threshold = config.get('refresh_threshold', 0.8)
if current_time - cached_timestamp > (ttl *

```

PYTHON

refresh_threshold):

```

# Asynchronously refresh cache in background
self._schedule_background_refresh(query_id,

```

PYTHON

cached_result)

```
return True
```

```
def _cache_query_result(self, query_id: str, cache_key: str,
```

```
result_data):
```

```
"""Cache query result with metadata"""
```

```
    config = self.query_cache_configs.get(query_id, {})
    ttl = config.get('ttl_seconds', 300)
```

```
    cache_data = {
        'data': json.dumps(result_data),
        'timestamp': str(time.time()),
        'query_id': query_id
    }
```

```
    # Store with TTL
    self.redis.hmset(cache_key, cache_data)
    self.redis.expire(cache_key, ttl)
```

Pro Tip Real-time analytics is about perceived performance, not absolute speed. A dashboard that shows 2-minute-old data but responds in 100ms feels more “real-time” than one that shows 10-second-old data but takes 5 seconds to load. Focus on query performance first, data freshness second.

□ Checkpoint

1. What’s the primary benefit of pre-aggregated materialized views? a. They save storage space b. They provide faster query response times for common

aggregations

- c. They improve data quality d. They reduce network bandwidth
2. When should you choose ClickHouse over traditional OLTP databases for analytics? a. When you need ACID transactions b. When you need complex joins c. When you need fast analytical queries on large datasets d. When you need to update individual records frequently
 3. What's the main trade-off in real-time analytics architectures? a. Cost vs performance

b. Data freshness vs query performance c. Storage vs compute d. Security vs accessibility

Answers: 1-b, 2-c, 3-b

4.5 Event-Driven Architecture Patterns

⚡ TL;DR

Event-driven architectures enable loose coupling and independent scaling - Event sourcing provides complete audit trails but adds complexity - CQRS separates read and write models for better performance - Saga patterns handle distributed transactions across microservices - Choose choreography for simple flows, orchestration for complex business logic

Event-Driven Architecture Fundamentals

Event-driven architecture (EDA) changes how we think about system interactions. Instead of direct API calls between services, systems communicate through events, creating more resilient and scalable architectures.

```
# Traditional synchronous approach
class TraditionalOrderService:
    """Traditional approach with tight coupling"""

    def __init__(self, inventory_service, payment_service,
```

PYTHON

```
shipping_service, email_service):
```


PYTHON

```

self.inventory_service = inventory_service
self.payment_service = payment_service
self.shipping_service = shipping_service
self.email_service = email_service

def process_order(self, order_data):
    """Synchronous order processing - tightly coupled"""

    try:
        # Step 1: Check inventory
        inventory_result =

```

```

self.inventory_service.reserve_items(order_data['items'])

```

PYTHON

```

if not inventory_result.success:
    raise Exception("Insufficient inventory")

# Step 2: Process payment
payment_result = self.payment_service.charge_customer(
    order_data['customer_id'],
    order_data['total_amount']
)
if not payment_result.success:
    # Rollback inventory

```

```

self.inventory_service.release_items(order_data['items'])

```

```

raise Exception("Payment failed")

# Step 3: Create shipment
shipping_result =

```

```

self.shipping_service.create_shipment(order_data)

```

```

if not shipping_result.success:
    # Rollback payment and inventory

```

```

self.payment_service.refund(payment_result.transaction_id)
self.inventory_service.release_items(order_data['items'])

```

self.invent-

```
raise Exception("Shipping failed")

# Step 4: Send confirmation email
self.email_service.send_order_confirmation(order_data)

return {"status": "success", "order_id":
```

```
order_data['order_id']}]}
```

```
except Exception as e:
    # Complex error handling and rollback logic
    return {"status": "failed", "error": str(e)}

# Event-driven approach
class EventDrivenOrderService:
    """Event-driven approach with loose coupling"""

    def __init__(self, event_publisher):
        self.event_publisher = event_publisher

    def process_order(self, order_data):
        """Asynchronous order processing via events"""

        # Simply publish order created event
        order_created_event = {
            'event_type': 'order_created',
            'order_id': order_data['order_id'],
            'customer_id': order_data['customer_id'],
            'items': order_data['items'],
            'total_amount': order_data['total_amount'],
            'timestamp': datetime.now().isoformat()
        }

        self.event_publisher.publish('order_events',
```

```
order_created_event)
```

```

return {
    "status": "processing",
    "order_id": order_data['order_id'],
    "message": "Order submitted for processing"
}

```

Separate services handle events independently

```

class InventoryEventHandler:
    """Handles inventory-related events"""

    def __init__(self, event_publisher):
        self.event_publisher = event_publisher

    def handle_order_created(self, event):
        """Handle order created event"""

        try:
            # Check and reserve inventory
            reservation_result =

```

```

self.reserve_inventory(event['items'])

```

```

if reservation_result.success:
    # Publish inventory reserved event
    inventory_event = {
        'event_type': 'inventory_reserved',
        'order_id': event['order_id'],
        'reservation_id':

```

```

reservation_result.reservation_id,

```

```

'items': event['items']
}
self.event_publisher.publish('inventory_events',

```

```

inventory_event)

```

PYTHON

```

else:
    # Publish inventory failed event
    failure_event = {
        'event_type': 'inventory_reservation_failed',
        'order_id': event['order_id'],
        'reason': reservation_result.failure_reason
    }
    self.event_publisher.publish('order_events',

```

failure_event)

```

except Exception as e:
    # Publish error event

```

The *PYTHON* code:

PYTHON

```

error_event = {
    'event_type': 'inventory_service_error',
    'order_id': event['order_id'],
    'error': str(e)
}
self.event_publisher.publish('error_events',

```

error_event)

Event Sourcing Pattern

Event sourcing stores the state of a business entity as a sequence of state-changing events. Instead of storing current state, you store all the events that led to the current state.

```

from typing import List, Dict, Any
from dataclasses import dataclass, asdict
from datetime import datetime
import json

@dataclass
class DomainEvent:
    """Base class for domain events"""
    event_type: str
    aggregate_id: str
    event_id: str
    timestamp: datetime
    event_data: Dict[str, Any]
    version: int

class EventStore:
    """Event store for persisting and retrieving events"""

    def __init__(self, storage_backend):
        self.storage = storage_backend

    def append_events(self, aggregate_id: str, events:

```

```
List[DomainEvent],
```

```

    expected_version: int = None):
        """Append events to aggregate stream with optimistic

```

```
concurrency"""
```

```

# Check current version for optimistic concurrency control
    current_version = self.get_current_version(aggregate_id)

    if expected_version is not None and current_version !=

```

```
expected_version:
```

```

    raise ConcurrencyException(
        f"Expected version {expected_version}, but current

```

```
is {current_version}"
```

```

)

# Assign version numbers to new events
for i, event in enumerate(events):
    event.version = current_version + i + 1

# Persist events atomically
self.storage.append_events(aggregate_id, events)

# Publish events to event bus for projections and other

```

consumers

```

self.publish_events(events)

def get_events(self, aggregate_id: str, from_version: int = 0) -

```

> List[DomainEvent]:

```

"""Get events for aggregate from specific version"""
    return self.storage.get_events(aggregate_id, from_version)

    def get_current_version(self, aggregate_id: str) -> int:
        """Get current version of aggregate"""
        return self.storage.get_current_version(aggregate_id)

# Example: Order aggregate with event sourcing

```

The OrderAggregate implementation:

```
class OrderAggregate:
    """Order aggregate using event sourcing"""

    def __init__(self, order_id: str):
        self.order_id = order_id
        self.version = 0
        self.uncommitted_events = []

        # Current state derived from events
        self.status = None
        self.customer_id = None
        self.items = []
        self.total_amount = 0
        self.payment_status = None
        self.shipping_status = None

    def create_order(self, customer_id: str, items: List[Dict],
```

```
total_amount: float):
```

```

"""Create new order - generates event"""

if self.status is not None:
    raise Exception("Order already exists")

event = DomainEvent(
    event_type='OrderCreated',
    aggregate_id=self.order_id,
    event_id=str(uuid.uuid4()),
    timestamp=datetime.now(),
    event_data={
        'customer_id': customer_id,
        'items': items,
        'total_amount': total_amount
    },
    version=0 # Will be set by event store
)

self.apply_event(event)
self.uncommitted_events.append(event)

def confirm_payment(self, payment_id: str, amount: float):
    """Confirm payment - generates event"""

    if self.status != 'created':
        raise Exception("Cannot confirm payment for order in

status: " + self.status)

```



```

event = DomainEvent(
    event_type='PaymentConfirmed',
    aggregate_id=self.order_id,
    event_id=str(uuid.uuid4()),
    timestamp=datetime.now(),
    event_data={
        'payment_id': payment_id,
        'amount': amount
    },
    version=0
)

self.apply_event(event)
self.uncommitted_events.append(event)

def ship_order(self, tracking_number: str, carrier: str):
    """Ship order - generates event"""

    if self.payment_status != 'confirmed':
        raise Exception("Cannot ship order without confirmed

```

payment")

```

event = DomainEvent(
    event_type='OrderShipped',
    aggregate_id=self.order_id,
    event_id=str(uuid.uuid4()),

```

The PYTHON code:

```

timestamp=datetime.now(),
    event_data={
        'tracking_number': tracking_number,
        'carrier': carrier
    },
    version=0
)

self.apply_event(event)
self.uncommitted_events.append(event)

def apply_event(self, event: DomainEvent):
    """Apply event to update aggregate state"""

    if event.event_type == 'OrderCreated':
        self.status = 'created'
        self.customer_id = event.event_data['customer_id']
        self.items = event.event_data['items']
        self.total_amount = event.event_data['total_amount']

    elif event.event_type == 'PaymentConfirmed':
        self.payment_status = 'confirmed'

    elif event.event_type == 'OrderShipped':
        self.status = 'shipped'
        self.shipping_status = 'shipped'

    elif event.event_type == 'OrderCancelled':
        self.status = 'cancelled'

    self.version = event.version

def load_from_history(self, events: List[DomainEvent]):
    """Rebuild aggregate state from event history"""

    for event in events:
        self.apply_event(event)

    self.uncommitted_events = [] # Clear since these are

```

historical

```
# Repository for event-sourced aggregates
class OrderRepository:
    """Repository for order aggregates using event sourcing"""

    def __init__(self, event_store: EventStore):
        self.event_store = event_store

    def get_by_id(self, order_id: str) -> OrderAggregate:
        """Reconstruct order from events"""

        events = self.event_store.get_events(order_id)

        if not events:
            return None

        order = OrderAggregate(order_id)
        order.load_from_history(events)

        return order

    def save(self, order: OrderAggregate):
        """Save order by persisting uncommitted events"""

        if not order.uncommitted_events:
            return

        self.event_store.append_events(
            order.order_id,
            order.uncommitted_events,
            expected_version=order.version -
```

```
len(order.uncommitted_events)
```

```

)

    order.uncommitted_events = []

# Projections for read models
class OrderProjectionHandler:
    """Creates read models from events"""

    def __init__(self, read_database):
        self.read_db = read_database

    def handle_order_created(self, event: DomainEvent):
        """Create read model when order is created"""

        order_view = {
            'order_id': event.aggregate_id,
            'customer_id': event.event_data['customer_id'],
            'status': 'created',
            'total_amount': event.event_data['total_amount'],
            'items': event.event_data['items'],
            'created_at': event.timestamp,
            'last_updated': event.timestamp
        }

        self.read_db.insert('order_views', order_view)

    def handle_payment_confirmed(self, event: DomainEvent):
        """Update read model when payment confirmed"""

        self.read_db.update(
            'order_views',
            {'order_id': event.aggregate_id},
            {
                'payment_status': 'confirmed',
                'last_updated': event.timestamp
            }
        )

    def handle_order_shipped(self, event: DomainEvent):
        """Update read model when order shipped"""

        self.read_db.update(
            'order_views',
            {'order_id': event.aggregate_id},
            {
                'status': 'shipped',
                'tracking_number':

```

```
event.event_data['tracking_number'],
```

```
    'carrier': event.event_data['carrier'],  
    'shipped_at': event.timestamp,  
    'last_updated': event.timestamp  
    }  
)
```

Benefits of Event Sourcing: □ Complete audit trail of all changes

□ Checkpoint

1. □ Ability to rebuild any historical state

□ Checkpoint

1. □ Natural fit for event-driven architectures

□ Checkpoint

1. □ Temporal queries (state at any point in time)

Drawbacks of Event Sourcing: □ Increased complexity □ Event versioning challenges □
Query complexity for current state □ Storage overhead

Command Query Responsibility Segregation (CQRS)

CQRS separates read and write operations into different models, often used with event sourcing.

```
# Write side (Commands)  
class OrderCommandHandler:  
    """Handles commands that modify order state"""
```

PYTHON

The `__init__` implementation:

```
def __init__(self, order_repository: OrderRepository):
    self.repository = order_repository

def handle_create_order(self, command: CreateOrderCommand):
    """Handle order creation command"""

    # Business validation
    if command.total_amount <= 0:
        raise ValueError("Order amount must be positive")

    if not command.items:
        raise ValueError("Order must have items")

    # Create aggregate and apply business logic
    order = OrderAggregate(command.order_id)
    order.create_order(
        command.customer_id,
        command.items,
        command.total_amount
    )

    # Persist changes
    self.repository.save(order)

    return {"order_id": command.order_id, "status": "created"}

def handle_confirm_payment(self, command:
```

ConfirmPaymentCommand):

```

"""Handle payment confirmation command"""

# Load aggregate
order = self.repository.get_by_id(command.order_id)
if not order:
    raise Exception(f"Order {command.order_id} not found")

# Apply business logic
order.confirm_payment(command.payment_id, command.amount)

# Persist changes
self.repository.save(order)

# Read side (Queries)
class OrderQueryHandler:
    """Handles queries for order data"""

    def __init__(self, read_database):
        self.read_db = read_database

    def get_order_summary(self, order_id: str) -> Dict:
        """Get order summary for display"""

        query = """
        SELECT
            order_id,
            customer_id,
            status,
            total_amount,
            payment_status,
            tracking_number,
            created_at,
            last_updated
        FROM order_views
        WHERE order_id = %s
        """

        result = self.read_db.query_one(query, [order_id])
        return result

    def get_customer_orders(self, customer_id: str, limit: int = 20)

```

-> List[Dict]:

```

"""Get orders for customer"""

```

The corresponding SQL query:

```
query = """
SELECT
    order_id,
    status,
    total_amount,
    created_at
FROM order_views
WHERE customer_id = %s
ORDER BY created_at DESC
LIMIT %s
"""

results = self.read_db.query_many(query, [customer_id,
```

SQL

```
limit])
```



```

return results

def get_orders_by_status(self, status: str) -> List[Dict]:
    """Get orders filtered by status"""

    query = """
    SELECT
        order_id,
        customer_id,
        total_amount,
        created_at
    FROM order_views
    WHERE status = %s
    ORDER BY created_at DESC
    """

    results = self.read_db.query_many(query, [status])
    return results

# CQRS with different databases for read and write
class CQRSOrderService:
    """Order service implementing CQRS pattern"""

    def __init__(self, command_handler: OrderCommandHandler,
                  query_handler: OrderQueryHandler):
        self.command_handler = command_handler
        self.query_handler = query_handler

    # Command operations (writes)
    def create_order(self, customer_id: str, items: List[Dict],

```

total_amount: float):

```

"""Create order command"""
    command = CreateOrderCommand(
        order_id=str(uuid.uuid4()),
        customer_id=customer_id,
        items=items,
        total_amount=total_amount
    )
    return self.command_handler.handle_create_order(command)

def confirm_payment(self, order_id: str, payment_id: str,

```

amount: float):

```

"""Confirm payment command"""
command = ConfirmPaymentCommand(
    order_id=order_id,
    payment_id=payment_id,
    amount=amount
)
return self.command_handler.handle_confirm_payment(command)

# Query operations (reads)
def get_order(self, order_id: str):
    """Get order query"""
    return self.query_handler.get_order_summary(order_id)

def get_customer_orders(self, customer_id: str, limit: int =

```

20):

```

"""Get customer orders query"""
return self.query_handler.get_customer_orders(customer_id,

```

limit)

Saga Pattern for Distributed Transactions

Sagas manage long-running business processes across multiple services without distributed transactions.

```

# Choreography-based saga (event-driven)
class OrderProcessingSaga:
    """Choreography-based saga for order processing"""

    def __init__(self, event_publisher, saga_state_store):
        self.event_publisher = event_publisher
        self.state_store = saga_state_store

    def handle_order_created(self, event):
        """Handle order created event - start saga"""

        saga_id = f"order_saga_{event['order_id']}"

        # Initialize saga state
        saga_state = {
            'saga_id': saga_id,
            'order_id': event['order_id'],
            'status': 'started',
            'steps_completed': [],
            'steps_failed': [],
            'created_at': datetime.now().isoformat()
        }

        self.state_store.save_saga_state(saga_id, saga_state)

        # Start first step: inventory reservation
        inventory_command = {
            'command_type': 'reserve_inventory',
            'saga_id': saga_id,
            'order_id': event['order_id'],
            'items': event['items']
        }

        self.event_publisher.publish('inventory_commands',

```

inventory_command)

```
def handle_inventory_reserved(self, event):
    """Handle inventory reserved - continue saga"""

    saga_id = event['saga_id']
    saga_state = self.state_store.get_saga_state(saga_id)

    if not saga_state:
        return # Saga doesn't exist or already completed

    # Update saga state
    saga_state['steps_completed'].append('inventory_reserved')
    self.state_store.save_saga_state(saga_id, saga_state)

    # Next step: process payment
    payment_command = {
        'command_type': 'process_payment',
        'saga_id': saga_id,
        'order_id': saga_state['order_id'],
        'amount': event['total_amount']
    }

    self.event_publisher.publish('payment_commands',
```

payment_command)

```
def handle_payment_processed(self, event):
    """Handle payment processed - continue saga"""
```

The PYTHON code:

```

saga_id = event['saga_id']
saga_state = self.state_store.get_saga_state(saga_id)

saga_state['steps_completed'].append('payment_processed')
self.state_store.save_saga_state(saga_id, saga_state)

# Final step: create shipment
shipping_command = {
    'command_type': 'create_shipment',
    'saga_id': saga_id,
    'order_id': saga_state['order_id']
}

self.event_publisher.publish('shipping_commands',

```

shipping_command)

```

def handle_shipment_created(self, event):
    """Handle shipment created - complete saga"""

    saga_id = event['saga_id']
    saga_state = self.state_store.get_saga_state(saga_id)

    saga_state['steps_completed'].append('shipment_created')
    saga_state['status'] = 'completed'
    self.state_store.save_saga_state(saga_id, saga_state)

    # Publish saga completed event
    completion_event = {
        'event_type': 'order_processing_completed',
        'saga_id': saga_id,
        'order_id': saga_state['order_id'],
        'completed_at': datetime.now().isoformat()
    }

    self.event_publisher.publish('order_events',

```

completion_event)

```
def handle_step_failed(self, event):
    """Handle step failure - trigger compensations"""

    saga_id = event['saga_id']
    saga_state = self.state_store.get_saga_state(saga_id)

    if not saga_state:
        return

    failed_step = event['failed_step']
    saga_state['steps_failed'].append(failed_step)
    saga_state['status'] = 'compensating'

    # Trigger compensating actions for completed steps
    completed_steps = saga_state['steps_completed']

    for step in reversed(completed_steps): # Compensate in
```

reverse order

```
compensation_command =
```

```
self._create_compensation_command(step, saga_state)
```

```
if compensation_command:
    self.event_publisher.publish(
        compensation_command['topic'],
        compensation_command['command']
    )

# Orchestration-based saga
class OrderSagaOrchestrator:
    """Orchestration-based saga with central coordinator"""

    def __init__(self, command_dispatcher, saga_state_store):
        self.command_dispatcher = command_dispatcher
        self.state_store = saga_state_store

    def start_order_processing_saga(self, order_data):
```

```
"""Start order processing saga with orchestrator""" saga_id = f"or-
der_saga_{order_data['order_id']}"
```

```
# Define saga workflow saga_definition = {
```

```
    'saga_id': saga_id,
    'steps': [
        {
            'step_name': 'reserve_inventory',
            'command': 'ReserveInventoryCommand',
            'service': 'inventory_service',
            'compensation': 'ReleaseInventoryCommand'
        },
        {
            'step_name': 'process_payment',
            'command': 'ProcessPaymentCommand',
            'service': 'payment_service',
            'compensation': 'RefundPaymentCommand'
        },
        {
            'step_name': 'create_shipment',
            'command': 'CreateShipmentCommand',
            'service': 'shipping_service',
            'compensation': 'CancelShipmentCommand'
        }
    ],
    'current_step': 0,
    'status': 'running',
    'order_data': order_data
```

```
} self.state_store.save_saga_state(saga_id, saga_definition)
```

```
# Execute first step self._execute_next_step(saga_id) def handle_step_completed(self,
saga_id: str, step_result):
```

```
    """Handle step completion""" saga_state = self.state_store.get_saga_state(saga_id)
```

```
# Move to next step saga_state['current_step'] += 1 if saga_state['current_step'] >=
len(saga_state['steps']):
```

```
    # Saga completed
    saga_state['status'] = 'completed'
    self.state_store.save_saga_state(saga_id, saga_state)
```

PYTHON

```
else:
```

```
# Execute next step
self.state_store.save_saga_state(saga_id, saga_state)
self._execute_next_step(saga_id)
```

```
def handle_step_failed(self, saga_id: str, error):
```

```
"""Handle step failure - trigger compensation""" saga_state =
self.state_store.get_saga_state(saga_id) saga_state['status'] = 'compensating'

# Execute compensations for completed steps current_step =
saga_state['current_step'] for i in range(current_step - 1, -1, -1): # Reverse order
```

```
step_def = saga_state['steps'][i]
compensation_command = step_def['compensation']

self.command_dispatcher.send_command(
    step_def['service'],
    compensation_command,
    saga_state['order_data']
)
```

The `_execute_next_step` implementation:

```
def _execute_next_step(self, saga_id: str):
    """Execute the next step in the saga"""

    saga_state = self.state_store.get_saga_state(saga_id)
    current_step_index = saga_state['current_step']
    step_def = saga_state['steps'][current_step_index]

    # Send command to appropriate service
    self.command_dispatcher.send_command(
        step_def['service'],
        step_def['command'],
        saga_state['order_data']
    )
```

Choreography vs Orchestration Trade-offs:

Aspect Choreography Orchestration

Complexity Distributed complexity Centralized complexity

Coupling Loose coupling Tighter coupling

Observability Harder to trace Easier to trace

Failure Distributed error

Centralized error handling

Handling handling

Performance Better performance Additional network hop

Evolution Harder to change Easier to change

Pro Tip Start with choreography for simple business processes where services naturally know what to do next. Move to orchestration when you need complex business logic, conditional branching, or better observability into the process flow.

□ Checkpoint

1. What's the main benefit of event-driven architecture? a. Better performance
b. Loose coupling between services c. Reduced storage costs d. Simpler code
2. When should you use event sourcing? a. For all applications b. When you need a complete audit trail and temporal queries c. When you want better performance d. When you need simple CRUD operations
3. What's the difference between choreography and orchestration in sagas? a. Choreography is faster b. Choreography distributes coordination logic, orchestration centralizes it

centralizes it

c. Orchestration is always better d. There's no significant difference

Answers: 1-b, 2-b, 3-b

Try It Yourself: Exercises

Exercise 1: Stream Processing Framework Selection (45 minutes)

Scenario: You're building a real-time personalization system for an e-commerce platform with these requirements:

- Process 100k events/second during peak traffic
- Update user recommendations in real-time (< 500ms latency)
- Join streaming events with user profiles and product catalogs
- Handle late-arriving data gracefully
- Team has strong Java skills but limited streaming experience

Your Task: 1. Choose between Flink, Kafka Streams, and Spark Structured Streaming 2. Design the stream processing topology 3. Define windowing strategies for recommendation updates 4. Plan for scaling to 1M events/second

Deliverable: Architecture decision document with framework choice justification

Exercise 2: Event Sourcing vs CRUD Design (60 minutes)

Scenario: Design a order management system for a marketplace that needs to:

- Track complete order lifecycle (created → paid → fulfilled → delivered)
- Support order cancellations and refunds at any stage
- Provide audit trail for financial compliance
- Handle concurrent updates from multiple services
- Support complex business rules that change frequently

Your Task: 1. Design the system using traditional CRUD approach 2. Design the same system using event sourcing + CQRS 3. Compare the trade-offs of each approach 4. Recommend which approach for this use case

Deliverable: Two architecture designs with detailed comparison matrix

Exercise 3: Real-Time Analytics Dashboard (90 minutes)

Scenario: Build a real-time analytics dashboard for a gaming platform showing:

- Active players by game and region (updated every 30 seconds)
- Game session metrics (duration, actions per minute)
- Revenue metrics (in-app purchases, subscription renewals)
- Trending games based on recent activity
- Anomaly detection for unusual traffic patterns

Your Task: 1. Design the stream processing pipeline 2. Choose the OLAP database and design schema 3. Design caching strategy for dashboard performance 4. Plan for handling 1B gaming events per day

Deliverable: Complete real-time analytics architecture with performance analysis

Module 4 Project: Real-Time Fraud Detection System

Objective

Design and architect a comprehensive real-time fraud detection system for a payment processor that handles millions of transactions per hour with sub-second decision latency.

Business Context

You're building a fraud detection system for a global payment processor that:

- Processes 10M transactions per day across 50 countries
- Must make fraud decisions in under 200ms
- Handles multiple payment methods (credit cards, digital wallets, bank transfers)
- Must adapt to new fraud patterns in real-time
- Requires 99.99% uptime (fraud decisions can't fail)
- Must minimize false positives (blocking legitimate transactions)

Requirements

Functional Requirements: - Real-time fraud scoring for all

- transactions (< 200ms)
- Machine learning model updates without system downtime
- Rule-based fraud detection for known patterns -
- Historical fraud pattern analysis for model training
- Real-time alerts for high-risk transactions
- Integration with external fraud detection services

Non-Functional Requirements:

- Process 10M transactions/day, scale to 100M/day
- 99.99% uptime for fraud decisions
- < 200ms end- to-end fraud decision latency
- < 0.1% false positive rate
- Store 5 years of transaction data for analysis
- Comply with PCI DSS and financial regulations

Technical Constraints

Must integrate with existing Kafka-based transaction stream

Use cloud infrastructure (AWS, GCP, or Azure) Support A/B testing of fraud models

Provide real-time monitoring and alerting

Handle geographic distribution (US, EU, Asia)

Deliverables

- 1. System Architecture (6-8 pages)
- High-level system design with data flow
- Stream processing architecture
- Real-time model serving design
- Data storage strategy (streaming, batch, serving) -

Geographic distribution and latency optimization

- 2. Implementation Specifications (8-10 pages)
- Stream processing framework selection and justification
- Event-driven architecture design
- Machine learning pipeline integration -

Monitoring and alerting strategy - Security and compliance considerations

- 3. Performance & Scaling Plan (4-6 pages)
- Latency optimization strategies
- Scaling plan for 10x transaction growth
- Fault tolerance and disaster recovery
- Cost analysis and optimization

Warehouse & Analytics Design

What you'll learn

Build high-performance analytical data warehouses that scale to petabytes and serve thousands of concurrent users. Master modern warehouse architectures, advanced optimization techniques, and multi-tenant design patterns that power today's largest analytics platforms.

05

Data Warehouse Architecture

Design and optimize data warehouses. Master partitioning, materialized views, query optimization, and multi-tenant analytics.

5.1 Modern Data Warehouse Architecture

⚡ TL;DR

Modern warehouses separate storage and compute for elastic scaling and cost optimization - Cloud warehouses (Snowflake, BigQuery, Redshift) changed the game with automatic scaling and management - Data lakehouse architecture combines warehouse performance with lake flexibility - Choose architecture based on workload patterns: OLAP for analytics, hybrid for mixed workloads - ELT has largely replaced ETL for cloud warehouse environments

Evolution from Traditional to Modern Warehouses

The data warehouse landscape has been revolutionized in the past decade. Understanding this evolution helps you make better architectural decisions.

Traditional Data Warehouse (2000-2015):

Characteristics:

- Tightly coupled storage and compute
- Fixed-size clusters with pre-provisioned capacity
- - Expensive proprietary hardware (Teradata, Oracle Exadata)
- Complex ETL processes before loading
- DBA-heavy operations and tuning

Modern Cloud Data Warehouse (2015-present):

Characteristics:

- Separate storage and compute layers
- Elastic scaling based on workload demand
- Commodity cloud infrastructure

- ELT approach with transformation in warehouse
- Automatic optimization and management



Cloud Warehouse Architecture Patterns

PYTHON

```
class ModernDataWarehouseArchitecture:
    """Design patterns for modern cloud data warehouses"""

    def __init__(self, warehouse_platform='snowflake'):
        self.platform = warehouse_platform
        self.compute_clusters = {}
        self.storage_layers = {}

    def design_multi_cluster_architecture(self):
        """Design multi-cluster setup for workload isolation"""

        cluster_design = {
            'ingestion_cluster': {
                'purpose': 'Data loading and transformation',
                'size': 'Medium',
                'auto_scaling': True,
                'schedule': '24/7 for continuous ingestion',
                'workload_type': 'ETL/ELT processes'
            },
            'analytics_cluster': {
                'purpose': 'Business intelligence and reporting',
                'size': 'Large',
                'auto_scaling': True,
                'schedule': 'Business hours (8 AM - 8 PM)',
                'workload_type': 'Dashboard queries, ad-hoc
```

analysis'

```
    },
    'data_science_cluster': {
        'purpose': 'ML model training and experimentation',
        'size': 'X-Large',
        'auto_scaling': False, # Predictable workloads
        'schedule': 'On-demand',
        'workload_type': 'Complex analytical queries, ML
```

training'

```

},
'customer_facing_cluster': {
    'purpose': 'Customer-facing analytics applications',
    'size': 'Large',
    'auto_scaling': True,
    'schedule': '24/7',
    'workload_type': 'Low-latency queries, high

```

concurrency'

```

}
}

return cluster_design

def design_storage_architecture(self):
    """Design storage layer for different data types and access

```

PYTHON

patterns"""

```

storage_design = {

```

```

'raw_data_layer': {
    'location': 'Cloud object storage (S3/GCS/ADLS)',
    'format': 'Parquet/ORC with compression',
    'partitioning': 'By date and source system',
    'retention': '7 years',
    'access_pattern': 'Infrequent access, batch

```

processing'

```

},
'processed_data_layer': {
  'location': 'Warehouse native storage',
  'format': 'Columnar with automatic optimization',
  'partitioning': 'By business dimensions',
  'retention': '2 years active, then archived',
  'access_pattern': 'Frequent analytical queries'
},
'aggregated_data_layer': {
  'location': 'Warehouse with SSD storage',
  'format': 'Pre-aggregated tables and materialized

```

views',

```

'partitioning': 'By time and key dimensions',
'retention': '1 year',
'access_pattern': 'Very frequent, low-latency

```

queries'

```

},
'metadata_layer': {
  'location': 'Warehouse system tables + external

```

catalog',

```

  'format': 'Structured metadata tables',
    'partitioning': 'By object type and schema',
    'retention': 'Indefinite',
    'access_pattern': 'Frequent metadata queries'
  }
}

return storage_design

```

Snowflake Architecture Deep Dive

```

-- Snowflake-specific optimization patterns
-- Warehouse sizing and auto-scaling configuration

-- Create dedicated warehouses for different workloads
CREATE OR REPLACE WAREHOUSE LOADING_WH WITH
    WAREHOUSE_SIZE = 'LARGE'
    AUTO_SUSPEND = 60 -- Auto-suspend after 1 minute of inactivity
    AUTO_RESUME = TRUE
    INITIALLY_SUSPENDED = FALSE
    COMMENT = 'Warehouse for data loading and ETL processes';

CREATE OR REPLACE WAREHOUSE ANALYTICS_WH WITH
    WAREHOUSE_SIZE = 'X-LARGE'
    AUTO_SUSPEND = 300 -- Auto-suspend after 5 minutes
    AUTO_RESUME = TRUE
    MAX_CLUSTER_COUNT = 10
    MIN_CLUSTER_COUNT = 1
    SCALING_POLICY = 'STANDARD' -- Balance cost and performance
    COMMENT = 'Auto-scaling warehouse for analytics workloads';

CREATE OR REPLACE WAREHOUSE REPORTING_WH WITH
    WAREHOUSE_SIZE = 'MEDIUM'
    AUTO_SUSPEND = 60
    AUTO_RESUME = TRUE
    MAX_CLUSTER_COUNT = 3
    MIN_CLUSTER_COUNT = 1
    SCALING_POLICY = 'ECONOMY' -- Prioritize cost savings
    COMMENT = 'Cost-optimized warehouse for standard reporting';

-- Advanced clustering and partitioning
CREATE OR REPLACE TABLE fact_sales (
    sale_date DATE,
    customer_id NUMBER,
    product_id NUMBER,

```

The corresponding SQL query:

```

store_id NUMBER,
    quantity NUMBER,
    unit_price DECIMAL(10,2),
    total_amount DECIMAL(12,2),
    created_at TIMESTAMP_NTZ
)
CLUSTER BY (sale_date, customer_id) -- Automatic clustering
COMMENT = 'Sales fact table with automatic clustering';

-- Time Travel and Fail-safe configuration
CREATE OR REPLACE TABLE critical_business_data (
    id NUMBER,
    business_critical_field STRING,
    amount DECIMAL(15,2),
    updated_at TIMESTAMP_NTZ
)
DATA_RETENTION_TIME_IN_DAYS = 90 -- 90 days of time travel
COMMENT = 'Critical data with extended time travel retention';

-- Result caching optimization
-- Enable result caching at session level
ALTER SESSION SET USE_CACHED_RESULT = TRUE;

-- Query optimization with search optimization service
CREATE OR REPLACE TABLE user_events (
    event_id STRING,
    user_id STRING,
    event_type STRING,
    event_timestamp TIMESTAMP_NTZ,
    properties VARIANT
);

-- Enable search optimization for frequently filtered columns
ALTER TABLE user_events ADD SEARCH OPTIMIZATION ON

```

```
SUBSTRING(user_id);
```

```
ALTER TABLE user_events ADD SEARCH OPTIMIZATION ON
```

```
EQUALITY(event_type);
```

BigQuery Architecture Patterns

SQL

```
-- BigQuery-specific optimization patterns
-- Dataset organization and table design

-- Create datasets for different data layers
CREATE SCHEMA `company-data-warehouse.raw_data`
OPTIONS(
  description="Raw data from source systems",
  location="US"
);

CREATE SCHEMA `company-data-warehouse.processed_data`
OPTIONS(
  description="Cleaned and transformed data",
  location="US"
);

CREATE SCHEMA `company-data-warehouse.analytics_data`
OPTIONS(
  description="Business-ready analytics tables",
  location="US"
);

-- Partitioned and clustered table design
CREATE OR REPLACE TABLE `company-data-
```

warehouse.analytics_data.sales_fact` (

```
  sale_date DATE,
  customer_id INT64,
  product_id INT64,
  store_id INT64,
  quantity INT64,
  unit_price NUMERIC,
  total_amount NUMERIC,
```

```
created_timestamp TIMESTAMP
)
PARTITION BY sale_date
CLUSTER BY customer_id, product_id
OPTIONS(
  description="Sales fact table partitioned by date and clustered by
```

customer and product",

```
partition_expiration_days=2555 -- 7 years retention
);

-- Materialized views for query acceleration
CREATE MATERIALIZED VIEW `company-data-
```

```
warehouse.analytics_data.daily_sales_summary`
```



```

PARTITION BY sale_date
CLUSTER BY store_id
AS
SELECT
    sale_date,
    store_id,
    COUNT(*) as transaction_count,
    SUM(quantity) as total_quantity,
    SUM(total_amount) as total_revenue,
    AVG(total_amount) as avg_transaction_value,
    COUNT(DISTINCT customer_id) as unique_customers
FROM `company-data-warehouse.analytics_data.sales_fact`
GROUP BY sale_date, store_id;

-- Scheduled queries for data pipeline automation
CREATE OR REPLACE SCHEDULE `daily_sales_etl`
OPTIONS(
    description='Daily ETL for sales data',
    schedule='0 2 * * *', -- Run at 2 AM daily
    timezone='America/New_York'
)
AS
INSERT `company-data-warehouse.analytics_data.sales_fact`
SELECT
    DATE(order_timestamp) as sale_date,
    customer_id,
    product_id,
    store_id,
    quantity,
    unit_price,
    quantity * unit_price as total_amount,
    CURRENT_TIMESTAMP() as created_timestamp
FROM `company-data-warehouse.raw_data.orders`
WHERE DATE(order_timestamp) = DATE_SUB(CURRENT_DATE(), INTERVAL 1

```

DAY)

```

AND processed_flag = FALSE;

```

Data Lakehouse Architecture

```
class DataLakehouseArchitecture:
    """Implement lakehouse pattern combining lake flexibility with
```

PYTHON

```
warehouse performance"""
```

```
def __init__(self):
    self.storage_layer = "Delta Lake" # or Iceberg, Hudi
    self.compute_engines = ["Spark", "Presto", "Trino"]
    self.catalog_service = "AWS Glue" # or Apache Hive, Unity
```

PYTHON

Catalog

```
def design_lakehouse_layers(self):
    """Design multi-layered lakehouse architecture"""

    architecture = {
        'bronze_layer': {
            'description': 'Raw data ingestion layer',
            'data_format': 'Delta Lake tables',
            'schema_enforcement': 'Minimal - preserve source
```

PYTHON

```
format',
```

```
    'data_quality': 'Basic validation only',
```

```
    'retention': 'Indefinite (compressed after 90
```

```
days)',
```

```

    'access_pattern': 'Append-only, time-series access',
    'example_tables': [
        'raw_application_logs',
        'raw_database_changes',
        'raw_api_events'
    ]
},
'silver_layer': {
    'description': 'Cleaned and transformed data',
    'data_format': 'Delta Lake with enforced schema',
    'schema_enforcement': 'Strict schema with

```

evolution',

```

    'data_quality': 'Comprehensive validation and

```

cleaning',

```

    'retention': '7 years',
    'access_pattern': 'Read-heavy analytical queries',
    'example_tables': [
        'cleaned_user_events',
        'normalized_transactions',
        'enriched_customer_data'
    ]
},
'gold_layer': {
    'description': 'Business-ready aggregated data',
    'data_format': 'Optimized Delta Lake with Z-

```

ordering',

```

    'schema_enforcement': 'Business-defined canonical

```

schema',

```

    'data_quality': 'Business rule validation',
    'retention': '2 years active + archival',
    'access_pattern': 'High-frequency dashboard

```

queries',

```
'example_tables': [  
    'daily_customer_metrics',  
    'product_performance_summary',  
    'financial_reporting_tables'  
]  
,  
'feature_store_layer': {  
    'description': 'ML feature engineering and serving',  
    'data_format': 'Delta Lake with versioning',  
    'schema_enforcement': 'ML schema with feature
```

validation',

```
'data_quality': 'Statistical validation and drift
```

detection',

```
'retention': '1 year + model lineage tracking',  
'access_pattern': 'Point-in-time lookups and batch
```

training',

```
'example_tables': [  
    'user_behavior_features',  
    'product_recommendation_features',  
    'risk_scoring_features'  
]  
}  
  
return architecture  
  
def implement_delta_lake_optimization(self):  
    """Delta Lake specific optimization strategies"""  
  
    delta_optimizations = """  
    -- Optimize table layout with Z-ordering  
    OPTIMIZE sales_fact  
    ZORDER BY (customer_id, product_id, sale_date)  
  
    -- Vacuum old files to reclaim storage  
    VACUUM sales_fact RETAIN 168 HOURS -- Keep 7 days of
```

history

```

-- Auto-compaction for streaming writes
ALTER TABLE user_events SET TBLPROPERTIES (
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact' = 'true'
)

-- Partitioning strategy
CREATE TABLE partitioned_events
USING DELTA
PARTITIONED BY (year, month, day)
LOCATION 's3://data-lake/events/'
AS SELECT
    *,
    YEAR(event_timestamp) as year,
    MONTH(event_timestamp) as month,
    DAY(event_timestamp) as day
FROM raw_events

-- Change Data Feed for incremental processing
ALTER TABLE customer_data SET TBLPROPERTIES (
    'delta.enableChangeDataFeed' = 'true'
)

-- Time travel queries
SELECT * FROM sales_fact VERSION AS OF 'yesterday'
SELECT * FROM sales_fact TIMESTAMP AS OF '2024-01-01'

```

00:00:00'

"""

return delta_optimizations

Cost Optimization Strategies

PYTHON

```
class WarehouseCostOptimization:
    """Strategies for optimizing data warehouse costs"""

    def __init__(self, platform, monthly_budget):
        self.platform = platform
        self.budget = monthly_budget
        self.cost_metrics = {}

    def design_cost_optimization_strategy(self):
        """Comprehensive cost optimization approach"""

        strategies = {
            'compute_optimization': {
                'auto_scaling': {
                    'strategy': 'Use auto-scaling warehouses',
                    'potential_savings': '30-70%',
                    'implementation': [
                        'Set appropriate auto-suspend timeouts',
                        'Use multi-cluster warehouses for concurrent
```

workloads',

```
        'Right-size warehouse based on query
```

patterns'

```
    ],
    },
    'workload_scheduling': {
        'strategy': 'Schedule non-urgent workloads
```

during off-peak hours',

```
        'potential_savings': '20-40%',
        'implementation': [
            'Move ETL jobs to off-peak hours',
            'Use smaller warehouses for
```

development/testing',

```
'Implement query queuing for burst
```

workloads'

```
]
  }
},
'storage_optimization': {
  'data_lifecycle': {
```

```
'strategy': 'Implement intelligent data
```

lifecycle management',

```
'potential_savings': '40-60%',
  'implementation': [
    'Archive old data to cheaper storage tiers',
    'Compress historical data',
    'Delete or sample non-critical data'
  ]
},
'table_optimization': {
  'strategy': 'Optimize table design and
```

maintenance',

```
'potential_savings': '15-30%',
'implementation': [
  'Use appropriate data types',
  'Implement proper clustering and
```

partitioning',

```
'Regular table maintenance (vacuum,
```

optimize)'


```

]
  }
},
'query_optimization': {
  'result_caching': {
    'strategy': 'Maximize use of result caching',
    'potential_savings': '20-50%',
    'implementation': [
      'Enable automatic result caching',
      'Design queries to leverage cache',
      'Use materialized views for common

```

aggregations'

```

]
},
'query_efficiency': {
  'strategy': 'Optimize query patterns and

```

design',

```

'potential_savings': '30-70%',
    'implementation': [
        'Eliminate unnecessary data scans',
        'Use column pruning and predicate pushdown',
        'Optimize join strategies'
    ]
}
}
}

return strategies

def implement_cost_monitoring(self):
    """Implement comprehensive cost monitoring and alerting"""

    monitoring_setup = """
    -- Snowflake cost monitoring queries

    -- Daily compute cost by warehouse
    SELECT
        start_time::date as usage_date,
        warehouse_name,
        SUM(credits_used) as total_credits,
        SUM(credits_used) * 3.00 as estimated_cost_usd    -- $3
    """

```

per credit

```

FROM snowflake.account_usage.warehouse_metering_history
WHERE start_time >= dateadd('day', -30, current_timestamp())
GROUP BY 1, 2
ORDER BY 1 DESC, 3 DESC;

-- Storage cost analysis
SELECT
    usage_date,
    storage_bytes / (1024*1024*1024) as storage_gb,
    stage_bytes / (1024*1024*1024) as stage_gb,
    failsafe_bytes / (1024*1024*1024) as failsafe_gb

```

The corresponding SQL query:

```
FROM snowflake.account_usage.storage_usage
  WHERE usage_date >= dateadd('day', -30, current_timestamp())
  ORDER BY usage_date DESC;
```

```
-- Query cost analysis - identify expensive queries
SELECT
  query_id,
  user_name,
  warehouse_name,
  database_name,
  query_type,
  total_elapsed_time / 1000 as execution_time_seconds,
  credits_used_cloud_services,
  bytes_scanned / (1024*1024*1024) as gb_scanned
FROM snowflake.account_usage.query_history
WHERE start_time >= dateadd('hour', -24,
```

```
current_timestamp())
```

```
AND total_elapsed_time > 60000 -- Queries taking more
```

than 1 minute

```
ORDER BY credits_used_cloud_services DESC
LIMIT 50;
"""

    return monitoring_setup

def create_cost_alert_system(self):
    """Create automated cost alerts and budget controls"""

    alert_system = {
        'budget_alerts': [
            {
                'threshold': '70% of monthly budget',
                'action': 'Email notification to data team',
                'frequency': 'Daily check'
            },
            {
                'threshold': '90% of monthly budget',
                'action': 'Slack alert + email to management',
                'frequency': 'Real-time check'
            },
            {
                'threshold': '100% of monthly budget',
                'action': 'Auto-suspend non-critical
```

warehouses',

```

'frequency': 'Real-time check'
    }
],
'anomaly_detection': [
    {
        'metric': 'Daily compute costs',
        'threshold': '200% of 7-day average',
        'action': 'Investigate and alert'
    },
    {
        'metric': 'Storage growth rate',
        'threshold': '>50% week-over-week growth',
        'action': 'Review data retention policies'
    },
    {
        'metric': 'Query duration',
        'threshold': '>10x normal execution time',
        'action': 'Kill long-running queries and alert'
    }
]
}

return alert_system

```

Warehouse Security and Governance

```
-- Comprehensive security and governance setup
```

Role-based access control hierarchy:

```

-- Role-based access control hierarchy
CREATE ROLE data_engineers;
CREATE ROLE data_analysts;
CREATE ROLE data_scientists;
CREATE ROLE executives;
CREATE ROLE service_accounts;

-- Grant appropriate privileges
GRANT CREATE SCHEMA ON DATABASE analytics_db TO data_engineers;
GRANT CREATE TABLE ON SCHEMA analytics_db.processed TO

```

SQL

data_engineers;

```
GRANT SELECT ON ALL TABLES IN SCHEMA analytics_db.processed TO
```

SQL

data_analysts;

```
GRANT SELECT ON ALL VIEWS IN SCHEMA analytics_db.marts TO
```

SQL

executives;

```
-- Row-level security for multi-tenant data
CREATE OR REPLACE ROW ACCESS POLICY customer_data_policy
AS (customer_id) RETURNS BOOLEAN ->
  'ANALYST' = CURRENT_ROLE()
  OR customer_id IN (
    SELECT allowed_customer_id
    FROM user_access_control
    WHERE username = CURRENT_USER()
  );

-- Apply row access policy
ALTER TABLE customer_data ADD ROW ACCESS POLICY customer_data_policy
```

SQL

```
ON (customer_id);
```

```
-- Column-level security for PII protection
CREATE OR REPLACE MASKING POLICY email_mask AS (val STRING) RETURNS
```

STRING ->

```
CASE
  WHEN CURRENT_ROLE() IN ('DATA_PROTECTION_OFFICER',
```

'COMPLIANCE_TEAM') THEN val

```

ELSE REGEXP_REPLACE(val, '.*@(.+)', '*****@\1')
END;

-- Apply masking policy
ALTER TABLE customers MODIFY COLUMN email SET MASKING POLICY

```

SQL

email_mask;

```

-- Data classification and tagging
ALTER TABLE customers SET TAG (
  'data_classification' = 'confidential',
  'contains_pii' = 'true',
  'retention_period' = '7_years'
);

-- Audit and compliance monitoring
CREATE OR REPLACE VIEW audit_access_log AS
SELECT
  query_start_time,
  user_name,
  role_name,
  database_name,
  schema_name,
  query_type,
  query_text,
  bytes_scanned
FROM snowflake.account_usage.access_history
WHERE query_start_time >= dateadd('day', -90, current_timestamp());

-- Data lineage tracking
CREATE OR REPLACE VIEW data_lineage AS
SELECT
  target_table_name,
  source_table_name,
  query_start_time,
  user_name,
  query_type

```

SQL

The corresponding SQL query:

```

FROM snowflake.account_usage.access_history
WHERE query_type IN ('INSERT', 'UPDATE', 'MERGE',

```

SQL

'CREATE_TABLE_AS_SELECT');

Pro Tip Modern data warehouses are not just about storage

and compute - they're platforms for data collaboration. Design your architecture with governance, security, and cost management as first-class concerns, not afterthoughts.

PYTHON

□ Checkpoint

1. What's the main advantage of separating storage and compute in modern data warehouses? a. Better security b. Independent scaling and cost optimization c. Faster queries d. Simpler architecture
2. When should you use a data lakehouse architecture? a. When you need only structured data analytics b. When you want to combine warehouse performance with lake

flexibility for diverse data types

c. When you have a small data team d. When cost is not a concern 3. What's the most effective way to reduce data warehouse costs? a. Use smaller warehouse sizes b. Implement auto-scaling, query optimization, and data lifecycle

management

c. Store less data d. Use fewer users

Answers: 1-b, 2-b, 3-b

5.2 Advanced Partitioning & Clustering

TL;DR

Partitioning dramatically improves query performance by eliminating unnecessary data scans - Clustering organizes data within partitions for faster access to related records - Time- based partitioning is most common, but consider multi- dimensional strategies - Auto-clustering maintains performance as data grows without manual intervention - Wrong partitioning strategy can hurt performance more than no partitioning

Understanding Partitioning Strategies

Partitioning is the foundation of warehouse performance. It determines which data gets scanned for every query, directly impacting both performance and cost.

```
class PartitioningStrategy:
    """Advanced partitioning strategies for different use cases"""

    def __init__(self, table_name, data_characteristics):
        self.table_name = table_name
        self.data_characteristics = data_characteristics

    def analyze_query_patterns(self, query_logs):
        """Analyze historical query patterns to determine optimal
```

PYTHON

partitioning"""

```
analysis = {
    'filter_columns': {},    # Which columns are filtered most
```

often

```
'join_columns': {},      # Which columns are used for
```

SQL

joins

```
'time_granularity': {}, # What time ranges are queried  
'data_volume_by_column': {} # Data distribution across
```

columns

```
}
```

The corresponding SQL query:

```
for query in query_logs:  
    # Extract WHERE clause patterns  
    filters = self.extract_filter_columns(query['sql'])  
    for column in filters:  
        analysis['filter_columns'][column] =
```

SQL

```
analysis['filter_columns'].get(column, 0) + 1
```

```
# Extract JOIN patterns  
joins = self.extract_join_columns(query['sql'])  
for column in joins:  
    analysis['join_columns'][column] =
```

SQL

```
analysis['join_columns'].get(column, 0) + 1
```

```
# Analyze time ranges  
time_ranges = self.extract_time_ranges(query['sql'])  
for time_range in time_ranges:  
    granularity =
```

PYTHON

```
self.determine_time_granularity(time_range)
```

```
analysis['time_granularity'][granularity] =
```

```
analysis['time_granularity'].get(granularity, 0) + 1
```

```
return analysis
```

PYTHON

```
def recommend_partitioning_strategy(self, analysis_result):  
    """Recommend optimal partitioning strategy based on
```

```
analysis"""
```

```
recommendations = []
```

```
# Primary partitioning recommendation  
most_filtered_column =
```

```
max(analysis_result['filter_columns'].items(), key=lambda x: x[1]) [0]
```

```
if 'date' in most_filtered_column.lower() or 'timestamp' in
```

```
most_filtered_column.lower():
```

```
# Time-based partitioning  
time_granularity =
```

```
self.determine_optimal_time_granularity(analysis_result)
```

```
recommendations.append({  
    'type': 'time_partitioning',  
    'column': most_filtered_column,  
    'granularity': time_granularity,  
    'reason': f'Most queries filter by
```

```
{most_filtered_column}',
```

```
'expected_improvement':
```

```
self.estimate_performance_improvement('time', analysis_result)
```

```
)  
else:  
    # Value-based partitioning  
    recommendations.append({  
        'type': 'value_partitioning',  
        'column': most_filtered_column,  
        'strategy': 'hash_partitioning' if
```

```
self.is_high_cardinality(most_filtered_column) else 'range_partitioning',
```

```
'reason': f'High filter frequency on
```

```
{most_filtered_column}',
```

```
'expected_improvement':
```

```
self.estimate_performance_improvement('value', analysis_result)
```

```
)  
  
# Secondary partitioning/clustering recommendation  
secondary_columns =
```

```
sorted(analysis_result['filter_columns'].items(), key=lambda x: x[1],  
reverse=True)[1:3]
```

```
if secondary_columns:  
    recommendations.append({  
        'type': 'clustering',  
        'columns': [col[0] for col in secondary_columns],  
        'reason': 'Secondary filter and join columns',
```

SQL

```
'expected_improvement': '20-50% for filtered
```

queries'

```
    })

    return recommendations

# Time-based partitioning patterns
class TimeBasedPartitioning:
    """Implement various time-based partitioning strategies"""

    @staticmethod
    def daily_partitioning_ddl(table_name, time_column):
        """Generate DDL for daily partitioning"""

        snowflake_ddl = f"""
        -- Snowflake automatic clustering by date
        CREATE OR REPLACE TABLE {table_name} (
            id NUMBER,
            {time_column} TIMESTAMP_NTZ,
            customer_id NUMBER,
            product_id NUMBER,
            amount DECIMAL(10,2),
            -- other columns
        )
        CLUSTER BY (DATE({time_column}));
        """

        bigquery_ddl = f"""
        -- BigQuery daily partitioning
        CREATE OR REPLACE TABLE `project.dataset.{table_name}` (
            id INT64,
            {time_column} TIMESTAMP,
            customer_id INT64,
            product_id INT64,
            amount NUMERIC,
            -- other columns
        )
        PARTITION BY DATE({time_column})
        OPTIONS(
            partition_expiration_days=2555, -- 7 years
            require_partition_filter=true   -- Force partition
        )
        """
```

PYTHON

filtering

```
);
"""

return {'snowflake': snowflake_ddl, 'bigquery':
```

bigquery_ddl}

```
@staticmethod
def monthly_partitioning_with_subpartitions(table_name):
    """Monthly partitioning with daily subpartitioning"""

    # This pattern is useful for very large tables where daily
```

SQL

partitions would be too small

```
# but yearly partitions would be too large

advanced_partitioning = f"""
-- BigQuery monthly partitioning with clustering for daily
```

access

```
CREATE OR REPLACE TABLE `project.dataset.{table_name}` (
    id INT64,
    event_timestamp TIMESTAMP,
    user_id INT64,
    event_type STRING,
    properties JSON
)
PARTITION BY DATE_TRUNC(DATE(event_timestamp), MONTH)
CLUSTER BY DATE(event_timestamp), user_id;

-- Query example that benefits from this structure
SELECT user_id, COUNT(*) as event_count
FROM `project.dataset.{table_name}`
```

SQL

The corresponding SQL query:

```
WHERE DATE(event_timestamp) BETWEEN '2024-01-15' AND '2024-
```

SQL

01-20'

```
AND user_id IN (SELECT user_id FROM
```

SQL

high_value_customers)

```
GROUP BY user_id;
"""

    return advanced_partitioning

@staticmethod
def integer_range_partitioning(table_name, partition_column):
    """Integer range partitioning for non-time columns"""

    # Useful for ID ranges, geographic regions, etc.
    range_partitioning = f"""
    -- PostgreSQL/Greenplum range partitioning example
    CREATE TABLE {table_name} (
        customer_id BIGINT,
        order_date DATE,
        amount DECIMAL(10,2)
    ) PARTITION BY RANGE (customer_id);

    -- Create partitions for customer ID ranges
    CREATE TABLE {table_name}_p1 PARTITION OF {table_name}
        FOR VALUES FROM (1) TO (1000000);
    CREATE TABLE {table_name}_p2 PARTITION OF {table_name}
        FOR VALUES FROM (1000000) TO (2000000);
    CREATE TABLE {table_name}_p3 PARTITION OF {table_name}
        FOR VALUES FROM (2000000) TO (3000000);

    -- Auto-create future partitions
    SELECT partman.create_parent(
        p_parent_table => 'public.{table_name}',
        p_control => 'customer_id',
        p_type => 'range',
        p_interval => 1000000
    );
    """

    return range_partitioning
```

SQL

Multi-Dimensional Partitioning

```
class MultiDimensionalPartitioning:
    """Handle complex partitioning scenarios with multiple
```

PYTHON

```
dimensions"""
```

```
def __init__(self):
    self.partition_strategies = {}

def design_geography_time_partitioning(self):
    """Partition by both geography and time for global
```

PYTHON

```
applications"""
```

```
strategy = {
    'primary_partition': 'time_based',
    'secondary_partition': 'geography_based',
    'implementation': {
        'snowflake': """
            -- Snowflake: Use clustering for multi-
```

```
dimensional optimization
```

```
CREATE TABLE global_events (
    event_timestamp TIMESTAMP_NTZ,
    user_id NUMBER,
    country_code STRING,
    event_type STRING,
    properties VARIANT
)
CLUSTER BY (DATE(event_timestamp),
```

SQL

```
country_code);
```

```
""",
    'bigquery': """
```



```
-- BigQuery: Time partitioning + geography
```

clustering

```
CREATE TABLE `project.dataset.global_events` (  
    event_timestamp TIMESTAMP,  
    user_id INT64,  
    country_code STRING,  
    event_type STRING,  
    properties JSON  
)  
PARTITION BY DATE(event_timestamp)  
CLUSTER BY country_code, event_type;  
""",  
'redshift': ""  
    -- Redshift: Compound sort key for multi-
```

SQL

dimensional access

```
CREATE TABLE global_events (  
    event_timestamp TIMESTAMP,  
    user_id BIGINT,  
    country_code VARCHAR(3),  
    event_type VARCHAR(50),  
    properties JSON  
)  
COMPOUND SORTKEY (event_timestamp,
```

SQL

country_code);

```
""  
    }  
}  
  
return strategy  
  
def implement_customer_tier_partitioning(self):  
    """Partition by customer tier for SaaS applications"""  
  
    # Common pattern for multi-tenant SaaS where different
```

SQL

customer tiers

```
# have different query patterns and performance requirements
```

```
tiered_partitioning = """
-- Logical partitioning by customer tier
CREATE VIEW enterprise_customer_events AS
SELECT * FROM all_customer_events
WHERE customer_tier = 'enterprise';

CREATE VIEW standard_customer_events AS
SELECT * FROM all_customer_events
WHERE customer_tier = 'standard';

-- Physical partitioning using separate tables
CREATE TABLE enterprise_events (
    event_timestamp TIMESTAMP,
    customer_id BIGINT,
    event_data JSON
) CLUSTER BY (customer_id, DATE(event_timestamp));

CREATE TABLE standard_events (
    event_timestamp TIMESTAMP,
    customer_id BIGINT,
    event_data JSON
) CLUSTER BY (DATE(event_timestamp));    -- Different
```

clustering strategy

```
"""

    return tiered_partitioning

def calculate_partition_sizes(self, table_stats):
    """Calculate optimal partition sizes based on data
```

characteristics"""

```
calculations = {
    'target_partition_size': '1GB - 10GB per partition',
    'considerations': [
        'Too small: excessive metadata overhead',
```

```
'Too large: poor query pruning',  
'Sweet spot: 1-10GB allows good pruning without
```

overhead'

```
] }  
  
total_size_gb = table_stats['total_size_gb']  
daily_growth_gb = table_stats['daily_growth_gb']  
query_time_range_days =
```

```
table_stats['typical_query_range_days']
```

```
# Calculate optimal partitioning granularity  
if daily_growth_gb < 0.1: # Less than 100MB per day  
    recommendation = 'monthly_partitioning'  
    partition_size_gb = daily_growth_gb * 30  
elif daily_growth_gb < 1: # Less than 1GB per day  
    recommendation = 'weekly_partitioning'  
    partition_size_gb = daily_growth_gb * 7  
else: # More than 1GB per day  
    recommendation = 'daily_partitioning'  
    partition_size_gb = daily_growth_gb  
  
calculations['recommendation'] = recommendation  
calculations['estimated_partition_size_gb'] =
```

PYTHON

partition_size_gb

```
calculations['number_of_partitions'] = total_size_gb /
```

partition_size_gb

```
return calculations
```

Clustering Strategies

SQL

```
-- Advanced clustering patterns for different use cases

-- 1. Time-series clustering for event data
CREATE OR REPLACE TABLE user_events_clustered (
    event_timestamp TIMESTAMP_NTZ,
    user_id NUMBER,
    session_id STRING,
    event_type STRING,
    page_url STRING,
    properties VARIANT
)
CLUSTER BY (event_timestamp, user_id);

-- Benefits: Fast time-range queries and user-specific analysis
-- Query example that benefits:
SELECT user_id, COUNT(*) as events
FROM user_events_clustered
WHERE event_timestamp >= '2024-01-01'::timestamp
    AND event_timestamp < '2024-01-02'::timestamp
    AND user_id IN (SELECT user_id FROM premium_users)
GROUP BY user_id;

-- 2. Multi-level clustering for complex access patterns
CREATE OR REPLACE TABLE sales_transactions (
    transaction_date DATE,
    customer_id NUMBER,
    product_category STRING,
    store_region STRING,
    amount DECIMAL(10,2)
)
CLUSTER BY (transaction_date, customer_id, product_category);

-- Query patterns this optimizes:
-- - Daily sales reports
-- - Customer purchase history
-- - Category performance analysis

-- 3. Geographic clustering for location-based applications
CREATE OR REPLACE TABLE ride_sharing_trips (
    trip_timestamp TIMESTAMP_NTZ,
```

```

pickup_latitude DECIMAL(10,6),
    pickup_longitude DECIMAL(10,6),
    dropoff_latitude DECIMAL(10,6),
    dropoff_longitude DECIMAL(10,6),
    driver_id NUMBER,
    rider_id NUMBER
)
CLUSTER BY (trip_timestamp, pickup_latitude, pickup_longitude);

-- Optimizes queries for:
-- - Time-based trip analysis
-- - Geographic hotspot identification
-- - Driver/rider activity in specific areas

-- 4. Clustering with derived columns for complex analytics
CREATE OR REPLACE TABLE financial_transactions (
    transaction_timestamp TIMESTAMP_NTZ,
    account_id NUMBER,
    transaction_type STRING,
    amount DECIMAL(15,2),
    merchant_category STRING,
    risk_score DECIMAL(5,2)
)
CLUSTER BY (
    DATE(transaction_timestamp),
    account_id,
    CASE
        WHEN risk_score > 0.8 THEN 'high_risk'
        WHEN risk_score > 0.5 THEN 'medium_risk'
        ELSE 'low_risk'
    END
);

-- Enables fast queries for:
-- - Daily transaction summaries
-- - Account-specific transaction history
-- - Risk-based transaction analysis

```

Auto-Clustering and Maintenance

```
class AutoClusteringManager:
    """Manage automatic clustering and maintenance operations"""

    def __init__(self, warehouse_client):
        self.client = warehouse_client
        self.clustering_metrics = {}

    def analyze_clustering_effectiveness(self, table_name):
        """Analyze how well current clustering is working"""

        # Snowflake clustering information query
        clustering_analysis = f"""
        -- Check clustering depth and efficiency
        SELECT
            table_name,
            clustering_key,
            total_partition_count,
            total_constant_partition_count,
            average_overlaps,
            average_depth,
            partition_depth_histogram
        FROM
            TABLE(
                INFORMATION_SCHEMA.CLUSTERING_INFORMATION('{table_name}')
            );
        """
```

SQL

```
-- Check clustering ratio over time
SELECT
    start_time,
    table_name,
    schema_name,
    clustering_key,
    rows_clustered,
    bytes_clustered,
```

SQL

The corresponding SQL query:

SQL

```
credits_used
FROM SNOWFLAKE.ACCOUNT_USAGE.AUTOMATIC_CLUSTERING_HISTORY
WHERE table_name = '{table_name}'
      AND start_time >= dateadd('day', -30,
```

```
current_timestamp())
```

PYTHON

```
ORDER BY start_time DESC;
"""

    results = self.client.execute(clustering_analysis)
    return self.interpret_clustering_metrics(results)

def interpret_clustering_metrics(self, metrics):
    """Interpret clustering metrics and provide
    recommendations"""

    interpretation = {
        'clustering_health': 'unknown',
        'recommendations': [],
        'maintenance_needed': False
    }

    # Analyze average overlaps (lower is better)
    avg_overlaps = metrics.get('average_overlaps', 0)
    if avg_overlaps > 10:
        interpretation['clustering_health'] = 'poor'
        interpretation['recommendations'].append(
            'High overlap detected - consider re-clustering or
```

changing clustering key'

```
)
    interpretation['maintenance_needed'] = True
elif avg_overlaps > 5:
    interpretation['clustering_health'] = 'fair'
    interpretation['recommendations'].append(
        'Moderate overlap - monitor clustering performance'
    )
else:
    interpretation['clustering_health'] = 'good'

# Analyze clustering depth (lower is better for point
```

queries)

```
avg_depth = metrics.get('average_depth', 0)
if avg_depth > 50:
    interpretation['recommendations'].append(
        'High clustering depth - queries scan many
```

partitions'


```

)

return interpretation

def setup_clustering_monitoring(self):
    """Set up automated monitoring for clustering performance"""

    monitoring_sql = """
    -- Create monitoring view for clustering metrics
    CREATE OR REPLACE VIEW clustering_health_monitor AS
    SELECT
        table_catalog,
        table_schema,
        table_name,
        clustering_key,
        CASE
            WHEN average_overlaps <= 5 THEN 'Excellent'
            WHEN average_overlaps <= 10 THEN 'Good'
            WHEN average_overlaps <= 20 THEN 'Fair'
            ELSE 'Poor'
        END as clustering_health,
        average_overlaps,
        average_depth,
        total_partition_count,
        current_timestamp() as last_checked
    FROM INFORMATION_SCHEMA.TABLES t
    LEFT JOIN TABLE(INFORMATION_SCHEMA.CLUSTERING_INFORMATION(

```

```

't.' || t.table_catalog || '.' || t.table_schema ||

```

```

'.' || t.table_name || '

```

```

)) ci ON 1=1
WHERE t.table_type = 'BASE TABLE'
      AND t.clustering_key IS NOT NULL;

-- Alert query for poor clustering performance
SELECT
    table_schema,
    table_name,
    clustering_health,
    average_overlaps,
    'Re-cluster recommended' as action
FROM clustering_health_monitor
WHERE clustering_health IN ('Fair', 'Poor')
ORDER BY average_overlaps DESC;
"""

return monitoring_sql

def implement_maintenance_automation(self):
    """Implement automated clustering maintenance"""

    maintenance_strategy = {
        'automatic_clustering': {
            'enable': 'ALTER TABLE table_name RESUME RECLUSTER',
            'disable': 'ALTER TABLE table_name SUSPEND

```

RECLUSTER',

```
'monitor': 'SELECT * FROM
```

```
SNOWFLAKE.ACCOUNT_USAGE.AUTOMATIC_CLUSTERING_HISTORY'
```

```

},
'scheduled_maintenance': {
    'frequency': 'Weekly during low-usage hours',
    'actions': [
        'Analyze clustering effectiveness',
        'Re-cluster tables with poor clustering health',
        'Update clustering keys if query patterns

```

changed'

```

]
},
'cost_controls': {
    'max_credits_per_day': 10,
    'suspend_during_peak_hours': True,
    'priority_tables': ['fact_sales', 'user_events',

```

'financial_transactions']

```

}
}

return maintenance_strategy

```

Partitioning Anti-Patterns

```

class PartitioningAntiPatterns:
    """Common partitioning mistakes to avoid"""

    def identify_common_mistakes(self):
        """Document common partitioning anti-patterns"""

        anti_patterns = {
            'over_partitioning': {
                'problem': 'Too many small partitions create

```

PYTHON

metadata overhead',

```

'example': 'Hourly partitions with only 10MB of data

```

per hour',

```

'solution': 'Use daily or weekly partitions

```

instead',

SQL

```
'code_example': ""
-- BAD: Over-partitioned table
CREATE TABLE events_hourly (...)
PARTITION BY DATE_TRUNC('hour', event_timestamp);
-- Results in 8760 partitions per year!
```

GOOD: Right-sized partitions:

SQL

```
-- GOOD: Right-sized partitions
CREATE TABLE events_daily (...)
PARTITION BY DATE(event_timestamp);
-- Results in 365 partitions per year
""
},
'wrong_partition_key': {
  'problem': 'Partitioning on columns that queries
```

don't filter by',

```
'example': 'Partitioning by user_id when queries
```

filter by date',

```
'solution': 'Analyze query patterns to choose the
```

right key',

```
'code_example': ""
-- BAD: Partition by user_id but queries filter by
```

date

SQL

```
CREATE TABLE user_events (...)
PARTITION BY user_id;

SELECT * FROM user_events
WHERE event_date = '2024-01-01'; -- Scans all
```

partitions!

```
-- GOOD: Partition by date since that's how queries
```

filter

```
CREATE TABLE user_events (...)  
    PARTITION BY DATE(event_timestamp);  
    ""  
},  
'high_cardinality_partitioning': {  
    'problem': 'Partitioning on high-cardinality columns
```

SQL

creates too many partitions',

```
'example': 'Partitioning by customer_id with
```

millions of customers',

```
'solution': 'Use clustering or hash partitioning
```

instead',

```
'code_example': ""  
-- BAD: Millions of partitions  
CREATE TABLE orders (...)  
PARTITION BY customer_id;  
  
-- GOOD: Use clustering instead  
CREATE TABLE orders (...)  
CLUSTER BY customer_id;  
""  
},  
'no_partition_pruning': {  
    'problem': 'Queries don\'t include partition key in
```

SQL

```
WHERE clause',
```

```
'example': 'Queries that don\'t filter by the
```

partition column',

```
'solution': 'Enforce partition filtering or redesign'
```

partition strategy',

```
'code_example': """
-- BAD: Query doesn't use partition key
SELECT SUM(amount) FROM sales_partitioned_by_date
WHERE product_id = 123; -- Scans all date
"""
```

SQL

partitions!

```
-- GOOD: Include partition key in filter
        SELECT SUM(amount) FROM sales_partitioned_by_date
        WHERE product_id = 123
              AND sale_date >= '2024-01-01'
              AND sale_date < '2024-02-01';
        """
    }
}

return anti_patterns

def create_partitioning_health_check(self):
    """Create health check queries to identify partitioning
    issues"""
```

SQL

```
health_checks = """
-- 1. Identify over-partitioned tables (too many small
partitions)
```

```

WITH partition_stats AS (
    SELECT
        table_name,
        COUNT(*) as partition_count,
        AVG(row_count) as avg_rows_per_partition,
        AVG(bytes) as avg_bytes_per_partition
    FROM INFORMATION_SCHEMA.TABLE_STORAGE_METRICS
    WHERE partition_count IS NOT NULL
    GROUP BY table_name
)
SELECT
    table_name,
    partition_count,
    avg_rows_per_partition,
    avg_bytes_per_partition / (1024*1024) as

```

avg_mb_per_partition,

```

CASE
    WHEN partition_count > 1000 AND

```

avg_bytes_per_partition < 100*1024*1024

THEN 'Over-partitioned'

ELSE 'OK'

```

END as health_status
FROM partition_stats
WHERE partition_count > 100
ORDER BY partition_count DESC;

-- 2. Identify unused partitions (partitions that are never

```

queried)

SQL

```

WITH query_patterns AS (
    SELECT
        tables_scanned,
        partitions_scanned,
        COUNT(*) as query_count
    FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
    WHERE start_time >= dateadd('day', -30,

```

```
current_timestamp())
```

SQL

```

AND partitions_scanned IS NOT NULL
    GROUP BY tables_scanned, partitions_scanned
),
all_partitions AS (
    SELECT
        table_schema,
        table_name,
        partition_key,
        COUNT(*) as total_partitions
    FROM INFORMATION_SCHEMA.TABLE_STORAGE_METRICS
    GROUP BY table_schema, table_name, partition_key
)
SELECT
    ap.table_schema,
    ap.table_name,
    ap.total_partitions,
    COALESCE(SUM(qp.query_count), 0) as

```

```
queries_last_30_days,
```

```

CASE
    WHEN COALESCE(SUM(qp.query_count), 0) = 0
    THEN 'Unused table'
    WHEN COALESCE(SUM(qp.query_count), 0) <

```

```
ap.total_partitions * 0.1
```


THEN 'Many unused partitions'

ELSE 'OK'

```
END as partition_usage_health
FROM all_partitions ap
LEFT JOIN query_patterns qp ON qp.tables_scanned LIKE '%' ||
```

SQL

```
ap.table_name || '%'
```

```
GROUP BY ap.table_schema, ap.table_name, ap.total_partitions
ORDER BY queries_last_30_days ASC;
```

SQL

```
""
```

```
return health_checks
```

⚠ Common Mistake

⚠ Common Mistake Don't partition just because you can. Analyze your query patterns first. Poor partitioning can hurt performance more than no partitioning. Always measure before and after implementing partitioning strategies.

□ Checkpoint

1. When is partitioning most beneficial? a. For all tables regardless of size b. When queries consistently filter by the partition key c. When tables have many columns d. When you want to save storage space
2. What's the main difference between partitioning and clustering? a. Partitioning is for small tables, clustering for large tables b. Partitioning physically separates data, clustering organizes

data within partitions

c. There's no difference - they're the same thing d. Clustering is only available in newer databases
3. What indicates over-partitioning? a. Queries are very fast b. Many small partitions with high metadata overhead c. Too few partitions d. High storage costs

Answers: 1-b, 2-b, 3-b

5.3 Materialized Views & Pre-Aggregation

⚡ TL;DR

Materialized views trade storage for query performance by pre-computing results - Incremental refresh strategies keep MVs fresh without full rebuilds - Cube and rollup patterns handle hierarchical aggregations efficiently - Auto-refresh materialized views reduce maintenance overhead in modern warehouses - Design MV hierarchies to serve different query patterns and latencies

Materialized View Design Patterns

Materialized views are pre-computed query results stored as tables. They're for achieving sub-second response times on complex analytical queries.

```
class MaterializedViewStrategy:
    """Design and manage materialized view strategies"""

    def __init__(self, warehouse_platform):
        self.platform = warehouse_platform
        self.view_hierarchy = {}
        self.refresh_strategies = {}

    def design_view_hierarchy(self, fact_table, dimensions):
        """Design hierarchy of materialized views for different
```

PYTHON

```
aggregation levels"""
```

```

hierarchy = {
    'raw_fact_view': {
        'purpose': 'Cleaned and enriched fact data',
        'refresh_frequency': 'Real-time/Streaming',
        'storage_multiplier': '1x',
        'query_types': 'Detailed analysis, debugging'
    },
    'daily_aggregates': {
        'purpose': 'Daily rollups for common metrics',
        'refresh_frequency': 'Daily (2 AM)',

```

```

        'storage_multiplier': '0.1x',
        'query_types': 'Daily dashboards, trend analysis'
    },
    'weekly_aggregates': {
        'purpose': 'Weekly rollups for executive reporting',
        'refresh_frequency': 'Weekly (Sunday)',
        'storage_multiplier': '0.01x',
        'query_types': 'Executive dashboards, weekly

```

reports'

```

    },
    'monthly_aggregates': {
        'purpose': 'Monthly rollups for historical

```

analysis',

```

        'refresh_frequency': 'Monthly (1st of month)',
        'storage_multiplier': '0.001x',
        'query_types': 'Historical trends, year-over-year

```

analysis'

```

    }

    return hierarchy

def generate_mv_ddl(self, aggregation_level, fact_table,

```

PYTHON

dimensions, metrics):

```
"""Generate DDL for materialized views at different
```

aggregation levels"""

```
if self.platform == 'snowflake':  
    return self._generate_snowflake_mv(aggregation_level,
```

PYTHON

fact_table, dimensions, metrics)

```
elif self.platform == 'bigquery':  
    return self._generate_bigquery_mv(aggregation_level,
```

PYTHON

fact_table, dimensions, metrics)

```
elif self.platform == 'redshift':  
    return self._generate_redshift_mv(aggregation_level,
```

PYTHON

fact_table, dimensions, metrics)

```
def _generate_snowflake_mv(self, level, fact_table, dimensions,
```

PYTHON

metrics):

SQL

```

"""Generate Snowflake materialized view DDL"""

if level == 'daily':
    return f"""
CREATE OR REPLACE MATERIALIZED VIEW daily_sales_mv
CLUSTER BY (sale_date, {dimensions[0]})
AS
SELECT
    DATE({fact_table}.sale_timestamp) as sale_date,
    {' '.join(dimensions)},
    COUNT(*) as transaction_count,
    SUM({metrics['revenue']}) as total_revenue,
    AVG({metrics['revenue']}) as avg_transaction_value,
    COUNT(DISTINCT {fact_table}.customer_id) as

```

unique_customers,

SQL

```

MIN({fact_table}.sale_timestamp) as first_sale,
    MAX({fact_table}.sale_timestamp) as last_sale
FROM {fact_table}
INNER JOIN dim_product ON {fact_table}.product_id =

```

dim_product.product_id

SQL

```

INNER JOIN dim_store ON {fact_table}.store_id =

```

dim_store.store_id

SQL

```

WHERE {fact_table}.sale_timestamp >= '2020-01-01'
GROUP BY 1, {' '.join([f'{i+2}' for i in

```

range(len(dimensions))]]}

SQL

```

"""

elif level == 'hourly':
    return f"""
    CREATE OR REPLACE MATERIALIZED VIEW hourly_sales_mv
    CLUSTER BY (sale_hour, store_region)
    AS
    SELECT

```

```
DATE_TRUNC('hour', {fact_table}.sale_timestamp) as
```

```
sale_hour,
```

```

dim_store.region as store_region,
dim_product.category as product_category,
COUNT(*) as transaction_count,
SUM({metrics['revenue']}) as total_revenue,
COUNT(DISTINCT {fact_table}.customer_id) as

```

```
unique_customers
```

SQL

```

FROM {fact_table}
INNER JOIN dim_product ON {fact_table}.product_id =

```

```
dim_product.product_id
```

SQL

```
INNER JOIN dim_store ON {fact_table}.store_id =
```

```
dim_store.store_id
```

SQL

```
WHERE {fact_table}.sale_timestamp >= CURRENT_DATE - 90
```

```
-- Last 90 days
```

```
GROUP BY 1, 2, 3
      """
```

SQL

```
def _generate_bigquery_mv(self, level, fact_table, dimensions,
```

metrics):

```
        """Generate BigQuery materialized view DDL"""
```

```
        if level == 'daily':
            return f"""
            CREATE MATERIALIZED VIEW
```

```
`project.dataset.daily_sales_mv`
```

```
    PARTITION BY sale_date
    CLUSTER BY {dimensions[0]}, {dimensions[1] if
```

```
len(dimensions) > 1 else dimensions[0]}
```

```
    OPTIONS(
        enable_refresh = true,
        refresh_interval_minutes = 60
    )
    AS
    SELECT
        DATE({fact_table}.sale_timestamp) as sale_date,
        {'', '.join(dimensions)},
        COUNT(*) as transaction_count,
        SUM({metrics['revenue']}) as total_revenue,
        AVG({metrics['revenue']}) as avg_transaction_value,
        COUNT(DISTINCT {fact_table}.customer_id) as
```

SQL

unique_customers

SQL

```
FROM `project.dataset.{fact_table}` {fact_table}
INNER JOIN `project.dataset.dim_product` dim_product
    ON {fact_table}.product_id = dim_product.product_id
INNER JOIN `project.dataset.dim_store` dim_store
    ON {fact_table}.store_id = dim_store.store_id
WHERE DATE({fact_table}.sale_timestamp) >=
```

```
DATE_SUB(CURRENT_DATE(), INTERVAL 2 YEAR)
```

SQL

```
GROUP BY 1, {'', '.join([f'{i+2}' for i in
```

```
range(len(dimensions))]]}
```

PYTHON

```
"""

# Real-world materialized view examples
class EcommerceAnalyticsMVs:
    """Materialized views for e-commerce analytics platform"""

    def create_customer_analytics_mvs(self):
        """Create customer-focused materialized views"""

        customer_mvs = {
            'customer_daily_summary': """
                CREATE MATERIALIZED VIEW customer_daily_summary AS
                SELECT
                    customer_id,
                    DATE(order_timestamp) as order_date,
                    COUNT(*) as order_count,
                    SUM(order_total) as total_spent,
                    AVG(order_total) as avg_order_value,
                    COUNT(DISTINCT product_category) as
```

```
categories_purchased,
```

```
FIRST_VALUE(order_timestamp) OVER (
```

```
PARTITION BY customer_id,
```

```
DATE(order_timestamp)
```

```
ORDER BY order_timestamp  
) as first_order_of_day,  
LAST_VALUE(order_timestamp) OVER (  
    PARTITION BY customer_id,
```

SQL

```
DATE(order_timestamp)
```

```
ORDER BY order_timestamp  
ROWS BETWEEN UNBOUNDED PRECEDING AND
```

SQL

UNBOUNDED FOLLOWING

```
) as last_order_of_day  
FROM fact_orders fo  
INNER JOIN dim_product dp ON fo.product_id =
```

SQL

```
dp.product_id
```

```
WHERE order_timestamp >= CURRENT_DATE - 730    -- 2
```

SQL

years

```
GROUP BY customer_id, DATE(order_timestamp);  
""",  
  
'customer_lifetime_metrics': ""  
    CREATE MATERIALIZED VIEW customer_lifetime_metrics
```

SQL

AS

```

SELECT
    customer_id,
    MIN(order_date) as first_order_date,
    MAX(order_date) as last_order_date,
    COUNT(DISTINCT order_date) as active_days,
    SUM(total_spent) as lifetime_value,
    AVG(total_spent) as avg_daily_spend,
    SUM(order_count) as total_orders,
    AVG(avg_order_value) as overall_avg_order_value,
    -- Customer segment classification
    CASE
        WHEN SUM(total_spent) > 10000 THEN 'VIP'
        WHEN SUM(total_spent) > 1000 THEN 'Premium'
        WHEN SUM(total_spent) > 100 THEN 'Standard'
        ELSE 'New'
    END as customer_segment,
    -- Recency calculation
    DATEDIFF('day', MAX(order_date), CURRENT_DATE)

```

as days_since_last_order

```

FROM customer_daily_summary
    GROUP BY customer_id;
""",

'customer_cohort_analysis': ""
CREATE MATERIALIZED VIEW customer_cohort_analysis AS
WITH customer_cohorts AS (
    SELECT
        customer_id,
        DATE_TRUNC('month', first_order_date) as

```

cohort_month,

```

first_order_date,
    last_order_date
FROM customer_lifetime_metrics
),
cohort_data AS (
    SELECT
        cc.cohort_month,
        DATE_TRUNC('month', cds.order_date) as

```

order_month,

```
COUNT(DISTINCT cc.customer_id) as
```

active_customers,

```
SUM(cds.total_spent) as cohort_revenue  
FROM customer_cohorts cc  
INNER JOIN customer_daily_summary cds ON
```

SQL

```
cc.customer_id = cds.customer_id
```

```
WHERE cds.order_date >= cc.first_order_date  
GROUP BY cc.cohort_month, DATE_TRUNC('month',
```

SQL

```
cds.order_date)
```

```
)  
SELECT
```

SQL

```
cohort_month,  
order_month,  
active_customers,  
cohort_revenue,  
DATEDIFF('month', cohort_month, order_month) as
```

month_number,

```
active_customers / FIRST_VALUE(active_customers)
```

```
OVER (
```

```

PARTITION BY cohort_month
            ORDER BY order_month
        ) as retention_rate
    FROM cohort_data
    ORDER BY cohort_month, order_month;
    """
}

return customer_mvs

def create_product_analytics_mvs(self):
    """Create product-focused materialized views"""

    product_mvs = {
        'product_performance_daily': """
            CREATE MATERIALIZED VIEW product_performance_daily

```

AS

```

SELECT
    dp.product_id,
    dp.product_name,
    dp.category,
    dp.subcategory,
    DATE(fo.order_timestamp) as sale_date,
    COUNT(*) as units_sold,
    SUM(fo.quantity * fo.unit_price) as

```

gross_revenue,

```

    SUM(fo.quantity * (fo.unit_price -

```

```

    dp.cost_price)) as gross_profit,

```

```

    AVG(fo.unit_price) as avg_selling_price,
    COUNT(DISTINCT fo.customer_id) as unique_buyers,
    -- Ranking within category
    RANK() OVER (
        PARTITION BY dp.category,

```

```

    DATE(fo.order_timestamp)

```

```
ORDER BY COUNT(*) DESC
) as category_rank_by_volume,
RANK() OVER (
    PARTITION BY dp.category,
```

SQL

```
DATE(fo.order_timestamp)
```

```
ORDER BY SUM(fo.quantity * fo.unit_price)
```

SQL

DESC

```
) as category_rank_by_revenue
FROM fact_orders fo
INNER JOIN dim_product dp ON fo.product_id =
```

SQL

```
dp.product_id
```

```
WHERE fo.order_timestamp >= CURRENT_DATE - 365
GROUP BY
    dp.product_id, dp.product_name, dp.category,
```

SQL

```
dp.subcategory,
```

```
DATE(fo.order_timestamp);
""",

'trending_products': ""
CREATE MATERIALIZED VIEW trending_products AS
WITH recent_performance AS (
    SELECT
        product_id,
        sale_date,
        units_sold,
        gross_revenue,
        -- 7-day moving averages
        AVG(units_sold) OVER (
            PARTITION BY product_id
            ORDER BY sale_date
```

SQL

The corresponding SQL query:

SQL

```
ROWS 6 PRECEDING
    ) as avg_units_7_day,
    AVG(gross_revenue) OVER (
        PARTITION BY product_id
        ORDER BY sale_date
        ROWS 6 PRECEDING
    ) as avg_revenue_7_day,
    -- 30-day moving averages
    AVG(units_sold) OVER (
        PARTITION BY product_id
        ORDER BY sale_date
        ROWS 29 PRECEDING
    ) as avg_units_30_day
FROM product_performance_daily
WHERE sale_date >= CURRENT_DATE - 60
)
SELECT
    product_id,
    sale_date,
    units_sold,
    avg_units_7_day,
    avg_units_30_day,
    -- Trending calculation
    CASE
        WHEN avg_units_7_day > avg_units_30_day *
```

1.5

THEN 'Trending Up'

```
WHEN avg_units_7_day < avg_units_30_day *
```

0.7 THEN 'Trending Down'

ELSE 'Stable'

```
END as trend_direction,
(avg_units_7_day / NULLIF(avg_units_30_day, 0) -
```

1) * 100 as trend_percentage

```
FROM recent_performance
WHERE sale_date = CURRENT_DATE - 1      -- Yesterday's
```

SQL

trends

```
ORDER BY trend_percentage DESC;
    """
}

return product_mvs
```

SQL

Incremental Refresh Strategies

```
class IncrementalRefreshManager:
    """Manage incremental refresh strategies for materialized
```

PYTHON

views"""


```
def __init__(self):
    self.refresh_strategies = {}

def design_incremental_refresh_pattern(self, mv_type,
```

fact_table):

```
"""Design incremental refresh strategy based on data
```

patterns"""

```
if mv_type == 'append_only_facts':
    return self._append_only_refresh_strategy(fact_table)
elif mv_type == 'slowly_changing_dimensions':
    return self._scd_refresh_strategy(fact_table)
elif mv_type == 'mixed_inserts_updates':
    return self._mixed_refresh_strategy(fact_table)

def _append_only_refresh_strategy(self, fact_table):
    """Strategy for append-only fact tables (most common)"""

    strategy = f"""
    -- Incremental refresh for append-only data
    -- Use high water mark tracking

    CREATE TABLE mv_refresh_log (
        mv_name VARCHAR(255),
        last_refresh_timestamp TIMESTAMP,
```

The corresponding SQL query:

```

high_water_mark TIMESTAMP,
    rows_processed INTEGER,
    refresh_duration_seconds INTEGER
);

-- Incremental refresh procedure
CREATE OR REPLACE PROCEDURE refresh_daily_sales_mv()
RETURNS STRING
LANGUAGE SQL
AS $$
DECLARE
    last_refresh TIMESTAMP;
    current_refresh TIMESTAMP := CURRENT_TIMESTAMP();
    rows_added INTEGER;
BEGIN
    -- Get last refresh timestamp
    SELECT COALESCE(MAX(high_water_mark), '1900-01-

```

01'::TIMESTAMP)

```

    INTO last_refresh
    FROM mv_refresh_log
    WHERE mv_name = 'daily_sales_mv';

-- Delete data that might have been updated
DELETE FROM daily_sales_mv
WHERE sale_date >= DATE(last_refresh) - 1; -- 1 day

```

buffer for late arrivals

```

-- Insert new/updated data
INSERT INTO daily_sales_mv
SELECT
    DATE(sale_timestamp) as sale_date,
    store_id,
    product_category,
    COUNT(*) as transaction_count,
    SUM(amount) as total_revenue,
    COUNT(DISTINCT customer_id) as unique_customers
FROM {fact_table}
WHERE sale_timestamp > last_refresh - INTERVAL '1 DAY'
    AND sale_timestamp <= current_refresh
GROUP BY DATE(sale_timestamp), store_id,

```

product_category;

```
GET DIAGNOSTICS rows_added = ROW_COUNT;

-- Log refresh completion
INSERT INTO mv_refresh_log VALUES (
    'daily_sales_mv',
    current_refresh,
    current_refresh,
    rows_added,
    EXTRACT(EPOCH FROM (CURRENT_TIMESTAMP() -
```

SQL

current_refresh))

```
);

RETURN 'Refresh completed. Rows processed: ' ||
```

rows_added;

```
END;
$$;

-- Schedule incremental refresh
CREATE TASK refresh_daily_sales_task
WAREHOUSE = ETL_WH
SCHEDULE = '15 2 * * *' -- 2:15 AM daily, after ETL
```

completes

```
AS
    CALL refresh_daily_sales_mv();
    """

    return strategy

def _mixed_refresh_strategy(self, fact_table):
    """Strategy for tables with both inserts and updates"""
```

PYTHON

The corresponding SQL query:

```
strategy = f"""\n-- Strategy for handling both inserts and updates\n-- Use change data capture or modification timestamps\n\nCREATE OR REPLACE PROCEDURE refresh_customer_summary_mv()\nRETURNS STRING\nLANGUAGE SQL\nAS $$\nDECLARE\n    last_refresh TIMESTAMP;\n    affected_customers ARRAY;\nBEGIN\n    -- Get last refresh timestamp\n    SELECT COALESCE(MAX(high_water_mark), '1900-01-
```

01'::TIMESTAMP)

```

INTO last_refresh
FROM mv_refresh_log
WHERE mv_name = 'customer_summary_mv';

-- Find customers affected by changes
SELECT ARRAY_AGG(DISTINCT customer_id)
INTO affected_customers
FROM {fact_table}
WHERE last_modified_timestamp > last_refresh;

-- Remove existing data for affected customers
DELETE FROM customer_summary_mv
WHERE customer_id = ANY(affected_customers);

-- Recalculate summary for affected customers
INSERT INTO customer_summary_mv
SELECT
    customer_id,
    COUNT(*) as total_orders,
    SUM(order_amount) as lifetime_value,
    AVG(order_amount) as avg_order_value,
    MIN(order_date) as first_order_date,
    MAX(order_date) as last_order_date,
    CURRENT_TIMESTAMP() as last_calculated
FROM {fact_table}
WHERE customer_id = ANY(affected_customers)
GROUP BY customer_id;

-- Log refresh completion
INSERT INTO mv_refresh_log VALUES (
    'customer_summary_mv',
    CURRENT_TIMESTAMP(),
    CURRENT_TIMESTAMP(),
    ARRAY_SIZE(affected_customers),
    EXTRACT(EPOCH FROM (CURRENT_TIMESTAMP() -

```

```

:start_time))

```

```

);

```

```

RETURN 'Refresh completed for ' ||

```

```

ARRAY_SIZE(affected_customers) || ' customers';

```

```

END;
$$;
"""

    return strategy

def create_mv_dependency_graph(self, materialized_views):
    """Create dependency graph for cascading refreshes"""

    dependency_graph = {
        'base_views': [
            'daily_sales_facts_mv',          # Level 1: Direct from

```

fact tables

```

'customer_daily_summary_mv',
'product_daily_summary_mv'
],

```

The corresponding SQL query:

```

'derived_views': [
    'weekly_sales_summary_mv',    # Level 2: Built from

```

daily views

```

'customer_lifetime_mv',
'product_performance_mv'
],
'aggregated_views': [
    'monthly_executive_summary_mv', # Level 3: Built

```

```

from weekly/monthly

```

```

    'customer_segmentation_mv',
        'category_performance_mv'
    ]
}

# Define refresh order based on dependencies
refresh_sequence = """
-- Refresh sequence to maintain data consistency

-- Level 1: Base materialized views (can run in parallel)
CALL refresh_daily_sales_facts_mv();
CALL refresh_customer_daily_summary_mv();
CALL refresh_product_daily_summary_mv();

-- Level 2: Derived views (depend on Level 1)
CALL refresh_weekly_sales_summary_mv();
CALL refresh_customer_lifetime_mv();
CALL refresh_product_performance_mv();

-- Level 3: Aggregated views (depend on Level 2)
CALL refresh_monthly_executive_summary_mv();
CALL refresh_customer_segmentation_mv();
CALL refresh_category_performance_mv();
"""

return {
    'dependency_graph': dependency_graph,
    'refresh_sequence': refresh_sequence
}

```

Cube and Rollup Patterns

SQL

```
-- Advanced OLAP cube and rollup patterns for hierarchical data

-- 1. Geographic hierarchy rollups
CREATE MATERIALIZED VIEW geographic_sales_cube AS
WITH base_data AS (
    SELECT
        fs.sale_date,
        ds.country,
        ds.region,
        ds.city,
        dp.category,
        dp.subcategory,
        dp.product_name,
        SUM(fs.amount) as revenue,
        COUNT(*) as transaction_count
    FROM fact_sales fs
    INNER JOIN dim_store ds ON fs.store_id = ds.store_id
    INNER JOIN dim_product dp ON fs.product_id = dp.product_id
    WHERE fs.sale_date >= CURRENT_DATE - 730 -- 2 years
    GROUP BY 1,2,3,4,5,6,7
)
SELECT * FROM base_data
UNION ALL
-- Country level rollup
SELECT
    sale_date, country, 'ALL_REGIONS' as region, 'ALL_CITIES' as
```

city,

SQL

```
category, subcategory, product_name, revenue, transaction_count
FROM base_data
UNION ALL
-- Category level rollup
```

The corresponding SQL query:

SQL

```
SELECT
    sale_date, country, region, city,
    category, 'ALL_SUBCATEGORIES' as subcategory, 'ALL_PRODUCTS' as
```


product_name,

```
revenue, transaction_count
FROM base_data
UNION ALL
-- Grand total rollup
SELECT
    sale_date, 'ALL_COUNTRIES' as country, 'ALL_REGIONS' as region,
```

SQL

'ALL_CITIES' as city,

```
'ALL_CATEGORIES' as category, 'ALL_SUBCATEGORIES' as
```

subcategory, 'ALL_PRODUCTS' as product_name,

```
revenue, transaction_count
FROM base_data;

-- 2. Time hierarchy rollups
CREATE MATERIALIZED VIEW time_hierarchy_sales AS
WITH daily_sales AS (
    SELECT
        sale_date,
        DATE_TRUNC('week', sale_date) as sale_week,
        DATE_TRUNC('month', sale_date) as sale_month,
        DATE_TRUNC('quarter', sale_date) as sale_quarter,
        DATE_TRUNC('year', sale_date) as sale_year,
        store_id,
        SUM(amount) as daily_revenue,
        COUNT(*) as daily_transactions
    FROM fact_sales
    WHERE sale_date >= CURRENT_DATE - 1095 -- 3 years
    GROUP BY sale_date, store_id
)
-- Daily level
SELECT 'daily' as time_level, sale_date::VARCHAR as time_period,
    sale_week, sale_month, sale_quarter, sale_year,
    store_id, daily_revenue as revenue, daily_transactions as
```

SQL

transactions

SQL

```
FROM daily_sales
UNION ALL
-- Weekly level
SELECT 'weekly' as time_level, sale_week::VARCHAR as time_period,
      sale_week, sale_month, sale_quarter, sale_year,
      store_id, SUM(daily_revenue) as revenue,
```

```
SUM(daily_transactions) as transactions
```

SQL

```
FROM daily_sales
GROUP BY sale_week, sale_month, sale_quarter, sale_year, store_id
UNION ALL
-- Monthly level
SELECT 'monthly' as time_level, sale_month::VARCHAR as time_period,
      NULL as sale_week, sale_month, sale_quarter, sale_year,
      store_id, SUM(daily_revenue) as revenue,
```

```
SUM(daily_transactions) as transactions
```

SQL

```
FROM daily_sales
GROUP BY sale_month, sale_quarter, sale_year, store_id
UNION ALL
-- Quarterly level
SELECT 'quarterly' as time_level, sale_quarter::VARCHAR as
```

```
time_period,
```

```
NULL as sale_week, NULL as sale_month, sale_quarter,
```

```
sale_year,
```

```
store_id, SUM(daily_revenue) as revenue,
```

```
SUM(daily_transactions) as transactions
```

```

FROM daily_sales
GROUP BY sale_quarter, sale_year, store_id;

-- 3. Customer dimension hierarchy
CREATE MATERIALIZED VIEW customer_hierarchy_analysis AS
WITH customer_base AS (
    SELECT
        fo.customer_id,
        dc.customer_segment,
        dc.customer_tier,

```

The corresponding SQL query:

```

dc.geographic_region,
    DATE(fo.order_timestamp) as order_date,
    SUM(fo.order_amount) as daily_spend,
    COUNT(*) as daily_orders
FROM fact_orders fo
INNER JOIN dim_customer dc ON fo.customer_id = dc.customer_id
WHERE fo.order_timestamp >= CURRENT_DATE - 365
GROUP BY 1,2,3,4,5
)
-- Individual customer level
SELECT 'customer' as hierarchy_level,
    customer_id::VARCHAR as entity_id,
    customer_segment, customer_tier, geographic_region,
    order_date, daily_spend, daily_orders
FROM customer_base
UNION ALL
-- Customer segment level
SELECT 'segment' as hierarchy_level,
    customer_segment as entity_id,
    customer_segment, customer_tier, geographic_region,
    order_date, SUM(daily_spend) as daily_spend,

```

```

SUM(daily_orders) as daily_orders

```

```

FROM customer_base
GROUP BY customer_segment, customer_tier, geographic_region,

```

order_date

UNION ALL

```
-- Customer tier level
SELECT 'tier' as hierarchy_level,
       customer_tier as entity_id,
       'ALL_SEGMENTS' as customer_segment, customer_tier,
```

SQL

geographic_region,

```
order_date, SUM(daily_spend) as daily_spend,
```

```
SUM(daily_orders) as daily_orders
```

```
FROM customer_base
GROUP BY customer_tier, geographic_region, order_date;
```

SQL

Performance Optimization for Materialized Views

SQL

```
class MVPerformanceOptimizer:
    """Optimize materialized view performance and maintenance"""

    def __init__(self):
        self.optimization_strategies = {}

    def analyze_mv_usage_patterns(self, mv_name):
        """Analyze how materialized views are being used"""

        usage_analysis = f"""
        -- Analyze query patterns against materialized views
        WITH mv_queries AS (
            SELECT
                query_id,
                query_text,
                start_time,
                total_elapsed_time,
                rows_examined,
                bytes_scanned,
                warehouse_name
            FROM snowflake.account_usage.query_history
            WHERE query_text ILIKE '%{mv_name}%'
            AND start_time >= DATEADD('day', -30,
```

CURRENT_TIMESTAMP())

SQL

```
),
usage_stats AS (
    SELECT
        COUNT(*) as query_count,
        AVG(total_elapsed_time) as avg_execution_time_ms,
        MEDIAN(total_elapsed_time) as
```

median_execution_time_ms,

```
MAX(total_elapsed_time) as max_execution_time_ms,
AVG(bytes_scanned) as avg_bytes_scanned,
COUNT(DISTINCT warehouse_name) as warehouses_used
```

The corresponding SQL query:

```
FROM mv_queries
)
SELECT
    '{mv_name}' as materialized_view,
    query_count,
    avg_execution_time_ms / 1000 as
```

avg_execution_time_seconds,

```
median_execution_time_ms / 1000 as
```

median_execution_time_seconds,

```
max_execution_time_ms / 1000 as
```

max_execution_time_seconds,

```
avg_bytes_scanned / (1024*1024*1024) as avg_gb_scanned,
warehouses_used,
CASE
    WHEN query_count = 0 THEN 'Unused - Consider
```

dropping'

```
    WHEN avg_execution_time_ms > 30000 THEN 'Slow
```

queries - Needs optimization'

```
    WHEN query_count > 1000 THEN 'High usage - Monitor
```

refresh cost'

ELSE 'Normal usage'

```
END as optimization_recommendation
FROM usage_stats;
"""

return usage_analysis

def optimize_mv_refresh_scheduling(self, materialized_views):
    """Optimize refresh scheduling based on usage patterns and
```

SQL

dependencies"""

```
optimization_plan = {
    'high_frequency_mvs': {
        'criteria': 'Updated multiple times per day, high
```

query volume',

```
'refresh_strategy': 'Auto-refresh or streaming
```

refresh',

```
'examples': ['real_time_dashboard_mv',
```

'customer_activity_mv'],

```
'implementation': """
    ALTER MATERIALIZED VIEW real_time_dashboard_mv
    SET AUTO_REFRESH = TRUE;

    -- Or use streaming refresh for near real-time
```

updates

```
CREATE STREAM fact_orders_stream ON TABLE
```

fact_orders;

```
CREATE TASK refresh_dashboard_mv_task
WAREHOUSE = REFRESH_WH
SCHEDULE = '1 minute'
WHEN
```

```
SYSTEM$STREAM_HAS_DATA('fact_orders_stream')
```

```
AS
    CALL refresh_dashboard_mv_incremental();
    ""
},
'medium_frequency_mvs': {
    'criteria': 'Daily refresh, moderate query volume',
    'refresh_strategy': 'Scheduled daily refresh during
```

off-peak hours',

```
'examples': ['daily_sales_summary_mv',
```

'customer_metrics_mv'],

```
'implementation': ""
    CREATE TASK refresh_daily_mvs
    WAREHOUSE = ETL_WH
    SCHEDULE = 'USING CRON 0 3 * * * UTC' -- 3 AM
```

daily

```
AS
BEGIN
    CALL refresh_daily_sales_summary_mv();
    CALL refresh_customer_metrics_mv();
END;
```



```

"""
},
'low_frequency_mvs': {
    'criteria': 'Weekly/monthly refresh, low query

```

volume',

```

'refresh_strategy': 'Scheduled periodic refresh',
'examples': ['monthly_executive_summary_mv',

```

'annual_trends_mv'],

```

'implementation': """
    CREATE TASK refresh_monthly_mvs
    WAREHOUSE = SMALL_WH -- Use smaller warehouse

```

for cost efficiency

```

SCHEDULE = 'USING CRON 0 4 1 * * UTC' -- 1st of

```

month at 4 AM

```

AS
                                CALL refresh_monthly_executive_summary_mv();
                                """
                                }
                                }

    return optimization_plan

def implement_mv_monitoring(self):
    """Implement comprehensive monitoring for materialized

```

PYTHON

views"""

```

monitoring_setup = ""
-- Create MV health monitoring view
CREATE OR REPLACE VIEW mv_health_monitor AS
WITH mv_info AS (
    SELECT
        table_catalog,
        table_schema,
        table_name,
        table_type,
        row_count,
        bytes,
        last_altered,
        auto_clustering_on,
        is_transient,
        comment
    FROM information_schema.tables
    WHERE table_type = 'MATERIALIZED VIEW'
),
mv_usage AS (
    SELECT
        REGEXP_SUBSTR(query_text, 'FROM\\s+

```

```

(\\w+\\.\\w+\\.\\w+)' , 1, 1, 'i', 1) as table_reference,

```

```

COUNT(*) as query_count_30_days,
    AVG(total_elapsed_time) as avg_query_time_ms,
    MAX(start_time) as last_queried
FROM snowflake.account_usage.query_history
WHERE start_time >= DATEADD('day', -30,

```

```

CURRENT_TIMESTAMP())

```

```

AND query_text ILIKE '%materialized%view%'
      GROUP BY 1
),
refresh_stats AS (
  SELECT
    name as mv_name,
    state,
    definition,
    owner,
    last_refresh_start_time,
    last_refresh_end_time,
    DATEDIFF('minute', last_refresh_start_time,

```

last_refresh_end_time) as last_refresh_duration_minutes

```

FROM snowflake.account_usage.materialized_views
)
SELECT
  mi.table_catalog || '.' || mi.table_schema || '.' ||

```

```

mi.table_name as full_mv_name,

```

```

mi.table_name as mv_name,
mi.row_count,
mi.bytes / (1024*1024*1024) as size_gb,
mi.last_altered,
rs.last_refresh_start_time,
rs.last_refresh_duration_minutes,
COALESCE(mu.query_count_30_days, 0) as

```

queries_last_30_days,

```

mu.avg_query_time_ms / 1000 as avg_query_time_seconds,
mu.last_queried,
-- Health score calculation
CASE
  WHEN COALESCE(mu.query_count_30_days, 0) = 0 THEN

```

'Unused'

```
WHEN DATEDIFF('day', rs.last_refresh_start_time,
```

```
CURRENT_TIMESTAMP()) > 7 THEN 'Stale'
```

```
WHEN rs.last_refresh_duration_minutes > 60 THEN
```

'Slow Refresh'

```
WHEN mu.avg_query_time_ms > 10000 THEN 'Slow
```

Queries'

ELSE 'Healthy'

```
END as health_status,  
-- Cost efficiency estimate  
(COALESCE(mu.query_count_30_days, 0) *
```

(mu.avg_query_time_ms / 1000)) as total_query_seconds_saved,

```
rs.last_refresh_duration_minutes * 60 as
```

refresh_cost_seconds

```
FROM mv_info mi  
LEFT JOIN mv_usage mu ON mu.table_reference =
```

SQL

mi.table_catalog || '.' || mi.table_schema || '.' || mi.table_name

```
LEFT JOIN refresh_stats rs ON rs.mv_name = mi.table_name
ORDER BY queries_last_30_days DESC;
```

```
-- Alert queries for MV issues
SELECT
    mv_name,
    health_status,
    'Action Required' as alert_type,
    CASE
        WHEN health_status = 'Unused' THEN 'Consider
```

dropping this materialized view'

```
WHEN health_status = 'Stale' THEN 'Materialized view
```

needs refresh'

```
WHEN health_status = 'Slow Refresh' THEN 'Optimize
```

refresh logic or schedule'

```
WHEN health_status = 'Slow Queries' THEN 'Review MV
```

design and indexing'

```
END as recommended_action
FROM mv_health_monitor
WHERE health_status != 'Healthy'
ORDER BY
    CASE health_status
        WHEN 'Stale' THEN 1
        WHEN 'Slow Refresh' THEN 2
        WHEN 'Slow Queries' THEN 3
        WHEN 'Unused' THEN 4
    END;
"""

return monitoring_setup
```

Pro Tip Materialized views are not free - they consume

storage and compute for refreshes. Only create MVs that provide significant query performance improvements. Monitor usage patterns and drop unused MVs. A good MV should be queried at least 10x more often than it's refreshed.

□ Checkpoint

1. When should you use materialized views?
 - a. For all tables in your warehouse
 - b. When query performance gains justify the storage and refresh

costs

c. Only for small tables d. When you need real-time data 2. What's the main challenge with incremental MV refresh?

- a. It's too slow
- b. Handling late-arriving data and updates to historical data
- c. It uses too much storage
- d. It's not supported by modern warehouses

3. How should you organize materialized views in a hierarchy?

- a. All MVs at the same level
- b. Base MVs from fact tables, derived MVs from base MVs,

aggregated MVs from derived MVs

SQL

c. Random organization d. Only use one MV per table

Answers: 1-b, 2-b, 3-b

⚡ TL;DR

Query optimization starts with understanding execution plans and identifying bottlenecks - Join order and join strategies have massive impact on performance - Column pruning, predicate pushdown, and partition elimination are optimizations - Window functions and analytical queries need special optimization techniques - Automated query optimization in modern warehouses handles many optimizations, but understanding the principles helps you write better queries

Understanding Query Execution Plans

Query execution plans are the roadmap to optimization. Modern data warehouses provide detailed execution plans that show exactly how queries are executed.

```
class QueryOptimizationAnalyzer:
    """Analyze and optimize query execution patterns"""

    def __init__(self, warehouse_client):
        self.client = warehouse_client
        self.optimization_patterns = {}

    def analyze_query_execution_plan(self, query_sql):
        """Analyze execution plan to identify optimization
        opportunities"""
```

PYTHON

PYTHON

```
# Get execution plan (Snowflake example)
plan_query = f"EXPLAIN USING TABULAR {query_sql}"
execution_plan = self.client.execute(plan_query)

analysis = {
    'performance_issues': [],
    'optimization_opportunities': [],
    'estimated_cost': 0
}

# Analyze each operation in the plan
for operation in execution_plan:
    if operation['operation'] == 'TableScan':
        analysis.update(self._analyze_table_scan(operation))
    elif operation['operation'] == 'Join':
```

```
analysis.update(self._analyze_join_operation(operation))
```

PYTHON

```
elif operation['operation'] == 'Aggregate':
```

```
analysis.update(self._analyze_aggregation(operation))
```

PYTHON

```
elif operation['operation'] == 'Sort':
```

```
analysis.update(self._analyze_sort_operation(operation))
```

```
return analysis
```

The `_analyze_table_scan` implementation:

PYTHON

```
def _analyze_table_scan(self, operation):
    """Analyze table scan operations for optimization
```

```
opportunities"""
```



```

issues = []
opportunities = []

partitions_scanned = operation.get('partitions_scanned', 0)
partitions_total = operation.get('partitions_total', 0)
bytes_scanned = operation.get('bytes_scanned', 0)

# Check partition pruning efficiency
if partitions_scanned == partitions_total and

```

partitions_total > 1:

```

issues.append({
    'type': 'poor_partition_pruning',
    'severity': 'high',
    'message': f'Scanning all {partitions_total}

```

partitions - add partition filters',

```

    'table': operation['table_name']
})
opportunities.append({
    'type': 'add_partition_filter',
    'potential_improvement': '50-90% reduction in scan

```

time'

```

    })

# Check for large table scans without filtering
if bytes_scanned > 1024**3: # > 1GB
    issues.append({
        'type': 'large_table_scan',
        'severity': 'medium',
        'message': f'Scanning {bytes_scanned /

```

```

(1024**3):.1f}GB - consider adding filters',

```

```

        'table': operation['table_name']
    })

    return {'performance_issues': issues,

```

'optimization_opportunities': opportunities}

```

def _analyze_join_operation(self, operation):
    """Analyze join operations for optimization opportunities"""

    issues = []
    opportunities = []

    join_type = operation.get('join_type', '')
    join_method = operation.get('join_method', '')
    left_rows = operation.get('left_input_rows', 0)
    right_rows = operation.get('right_input_rows', 0)

    # Check for cross joins (usually unintentional)
    if join_type.upper() == 'CROSS':
        issues.append({
            'type': 'cross_join_detected',
            'severity': 'critical',
            'message': 'Cross join detected - may produce

```

SQL

cartesian product',

```

        'estimated_rows': left_rows * right_rows
    })

    # Check for hash join vs nested loop
    if join_method == 'NESTED_LOOP' and left_rows * right_rows >

```

SQL

1000000:

```

    issues.append({
        'type': 'inefficient_join_method',
        'severity': 'high',
        'message': 'Nested loop join on large tables -

```

SQL

consider hash join',

```
'estimated_cost': left_rows * right_rows
})
```

The corresponding SQL query:

```
opportunities.append({
    'type': 'optimize_join_conditions',
    'suggestion': 'Ensure join conditions use indexed
```

SQL

columns or enable hash joins'

```
})

# Check for potential broadcast join optimization
if right_rows < 100000 and left_rows > 1000000:
    opportunities.append({
        'type': 'broadcast_join_candidate',
        'suggestion': f'Consider broadcast join - right
```

SQL

table has only {right_rows} rows'

```
})

return {'performance_issues': issues,
```

'optimization_opportunities': opportunities}

```

# Query rewriting patterns for optimization
class QueryRewriteOptimizer:
    """Common query rewriting patterns for better performance"""

    def optimize_exists_vs_in(self, original_query):
        """Optimize EXISTS vs IN subqueries"""

        patterns = {
            'inefficient_in_subquery': """
                -- INEFFICIENT: IN with subquery
                SELECT customer_id, customer_name
                FROM customers c
                WHERE customer_id IN (
                    SELECT customer_id
                    FROM orders
                    WHERE order_date >= '2024-01-01'
                );
            """,
            'optimized_exists': """
                -- OPTIMIZED: EXISTS (often faster)
                SELECT customer_id, customer_name
                FROM customers c
                WHERE EXISTS (
                    SELECT 1
                    FROM orders o
                    WHERE o.customer_id = c.customer_id
                    AND o.order_date >= '2024-01-01'
                );
            """,
            'optimized_join': """
                -- OFTEN BEST: Inner join with DISTINCT
                SELECT DISTINCT c.customer_id, c.customer_name
                FROM customers c
                INNER JOIN orders o ON c.customer_id = o.customer_id
                WHERE o.order_date >= '2024-01-01';
            """
        }

        return patterns

    def optimize_window_functions(self):
        """Optimize window function patterns"""

        patterns = {
            'inefficient_multiple_windows': """
                -- INEFFICIENT: Multiple window functions with
            """
        }

```

different partitions

```
SELECT
    customer_id,
    order_date,
    amount,
    ROW_NUMBER() OVER (PARTITION BY customer_id
```

SQL

ORDER BY order_date) as order_sequence,

```
SUM(amount) OVER (PARTITION BY customer_id ORDER
```

BY order_date) as running_total,

```
AVG(amount) OVER (PARTITION BY
```

```
DATE_TRUNC('month', order_date)) as monthly_avg,
```

```
RANK() OVER (PARTITION BY DATE_TRUNC('year',
```

order_date) ORDER BY amount DESC) as yearly_rank

```
FROM orders;
""",
'optimized_cte_approach': ""
    -- OPTIMIZED: Use CTEs to reduce window function
```

SQL

complexity

```
WITH monthly_stats AS (
    SELECT
        DATE_TRUNC('month', order_date) as
```

SQL

order_month,

```

AVG(amount) as monthly_avg
  FROM orders
  GROUP BY DATE_TRUNC('month', order_date)
),
yearly_ranks AS (
  SELECT
    customer_id,
    order_date,
    amount,
    RANK() OVER (PARTITION BY DATE_TRUNC('year',

```

SQL

order_date) ORDER BY amount DESC) as yearly_rank

FROM orders

```

)
SELECT
  o.customer_id,
  o.order_date,
  o.amount,
  ROW_NUMBER() OVER (PARTITION BY o.customer_id

```

SQL

ORDER BY o.order_date) as order_sequence,

```

SUM(o.amount) OVER (PARTITION BY o.customer_id

```

```

ORDER BY o.order_date) as running_total,

```

```

ms.monthly_avg,
  yr.yearly_rank
FROM orders o
LEFT JOIN monthly_stats ms ON DATE_TRUNC('month',

```

SQL

```

o.order_date) = ms.order_month

```

```

LEFT JOIN yearly_ranks yr ON o.customer_id =

```

SQL

```
yr.customer_id AND o.order_date = yr.order_date;
```

```
"""
    }

    return patterns

def optimize_aggregation_queries(self):
    """Optimize aggregation query patterns"""

    patterns = {
        'avoid_distinct_in_aggregates': """
            -- INEFFICIENT: DISTINCT inside aggregates
            SELECT
                store_id,
                COUNT(DISTINCT customer_id) as unique_customers,
                COUNT(DISTINCT product_id) as unique_products,
                COUNT(DISTINCT order_id) as unique_orders
            FROM sales_fact
            GROUP BY store_id;
        """,
        'optimized_separate_aggregation': """
            -- OPTIMIZED: Separate aggregations to avoid

```

SQL

multiple DISTINCTs

```
WITH unique_customers AS (
    SELECT store_id, COUNT(DISTINCT customer_id) as
```

SQL

unique_customers

```
FROM sales_fact GROUP BY store_id
),
unique_products AS (
    SELECT store_id, COUNT(DISTINCT product_id) as
```

SQL

unique_products

```
FROM sales_fact GROUP BY store_id
```

SQL

The corresponding SQL query:

SQL

```
),
unique_orders AS (
    SELECT store_id, COUNT(DISTINCT order_id) as
```

unique_orders

SQL

```
FROM sales_fact GROUP BY store_id
)
SELECT
    uc.store_id,
    uc.unique_customers,
    up.unique_products,
    uo.unique_orders
FROM unique_customers uc
JOIN unique_products up ON uc.store_id = up.store_id
JOIN unique_orders uo ON uc.store_id = uo.store_id;
""",
'use_approximate_functions': ""
    -- ALTERNATIVE: Use approximate functions for large
```

datasets

SQL

```
SELECT
    store_id,
    APPROX_COUNT_DISTINCT(customer_id) as
```

approx_unique_customers,

```
APPROX_COUNT_DISTINCT(product_id) as
```

approx_unique_products

SQL

```
FROM sales_fact
    GROUP BY store_id;
    -- Trade precision for speed on large datasets
    ""
}

return patterns
```


Join Optimization Strategies

SQL

```
-- Advanced join optimization patterns

-- 1. Join order optimization (inner joins can be reordered)
-- INEFFICIENT: Large table first
SELECT
    o.order_id,
    c.customer_name,
    p.product_name
FROM large_orders_table o -- 100M rows
INNER JOIN small_customers_table c ON o.customer_id = c.customer_id
```

-- 1M rows

SQL

```
INNER JOIN tiny_products_table p ON o.product_id = p.product_id;
```

-- 10K rows

SQL

```
-- OPTIMIZED: Start with smallest table and most selective joins
SELECT
    o.order_id,
    c.customer_name,
    p.product_name
FROM tiny_products_table p      -- Start with smallest
INNER JOIN large_orders_table o ON p.product_id = o.product_id
INNER JOIN small_customers_table c ON o.customer_id = c.customer_id
WHERE p.category = 'Electronics' -- Most selective filter first
      AND o.order_date >= '2024-01-01';

-- 2. Join condition optimization
-- INEFFICIENT: Function calls in join conditions
SELECT o.*, c.*
FROM orders o
JOIN customers c ON UPPER(o.customer_email) = UPPER(c.email);

-- OPTIMIZED: Use computed columns or pre-process
ALTER TABLE orders ADD COLUMN customer_email_upper AS
```

```
UPPER(customer_email);
```

```
ALTER TABLE customers ADD COLUMN email_upper AS UPPER(email);
```

```
SELECT o.*, c.*  
FROM orders o
```

The corresponding SQL query:

```

JOIN customers c ON o.customer_email_upper = c.email_upper;

-- 3. Semi-join vs EXISTS vs IN optimization
-- Find customers who have placed orders in the last 30 days

-- Option 1: EXISTS (usually fastest for large datasets)
SELECT customer_id, customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
        AND o.order_date >= CURRENT_DATE - 30
);

-- Option 2: IN with subquery (can be slow if subquery is large)
SELECT customer_id, customer_name
FROM customers
WHERE customer_id IN (
    SELECT DISTINCT customer_id
    FROM orders
    WHERE order_date >= CURRENT_DATE - 30
);

-- Option 3: SEMI JOIN (often optimal)
SELECT DISTINCT c.customer_id, c.customer_name
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= CURRENT_DATE - 30;

-- 4. Broadcast join hints for small dimension tables
-- Snowflake example
SELECT /*+ USE_BROADCAST_JOIN(d) */
    f.sale_date,
    d.product_name,
    SUM(f.amount) as total_sales
FROM large_fact_table f
JOIN small_dimension_table d ON f.product_id = d.product_id
GROUP BY f.sale_date, d.product_name;

-- 5. Avoiding cartesian products
-- DANGEROUS: Missing join condition creates cartesian product
SELECT o.order_id, c.customer_name, p.product_name
FROM orders o, customers c, products p
WHERE o.order_date >= '2024-01-01'; -- Missing join conditions!

-- SAFE: Explicit join conditions
SELECT o.order_id, c.customer_name, p.product_name
FROM orders o

```

```
INNER JOIN customers c ON o.customer_id = c.customer_id
INNER JOIN products p ON o.product_id = p.product_id
WHERE o.order_date >= '2024-01-01';

-- 6. Join elimination through denormalization
-- Instead of joining dimension tables repeatedly, consider
```

denormalization

```
-- BEFORE: Multiple joins for each query
CREATE VIEW enriched_orders AS
SELECT
    o.order_id,
    o.order_date,
    o.amount,
    c.customer_name,
    c.customer_segment,
    p.product_name,
    p.category,
    s.store_name,
    s.region
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN products p ON o.product_id = p.product_id
JOIN stores s ON o.store_id = s.store_id;
```

SQL

AFTER: Denormalized fact table (trade storage for performance):

```
-- AFTER: Denormalized fact table (trade storage for performance)
CREATE TABLE fact_orders_denormalized AS
SELECT
    o.order_id,
    o.order_date,
    o.amount,
    o.customer_id,
    c.customer_name,
    c.customer_segment,
    o.product_id,
    p.product_name,
    p.category,
    o.store_id,
    s.store_name,
    s.region
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN products p ON o.product_id = p.product_id
JOIN stores s ON o.store_id = s.store_id;

-- Queries now scan single table instead of joining multiple tables
SELECT store_name, SUM(amount) as total_sales
FROM fact_orders_denormalized
WHERE order_date >= '2024-01-01'
GROUP BY store_name;
```

Column Pruning and Predicate Pushdown

```
class QueryOptimizationTechniques:
    """Advanced query optimization techniques"""

    def demonstrate_column_pruning(self):
        """Show impact of column pruning on performance"""

        examples = {
            'inefficient_select_star': """
                -- INEFFICIENT: SELECT * scans all columns
                SELECT *
                FROM large_table
                WHERE date_column >= '2024-01-01'
                   AND status = 'active';
                -- Scans ALL columns even if you only need a few
            """,
            'optimized_column_selection': """
                -- OPTIMIZED: Only select needed columns
                SELECT
                    id,
                    customer_id,
                    amount,
                    created_date
                FROM large_table
                WHERE date_column >= '2024-01-01'
                   AND status = 'active';
                -- Scans only selected columns (massive savings in
```

SQL

columnar storage)

```
""",
    'cte_column_pruning': """
        -- GOOD: Column pruning in CTEs
        WITH filtered_data AS (
            SELECT
                customer_id,
                order_amount,
                order_date -- Only select columns needed
```

SQL

downstream

FROM orders

```
WHERE order_date >= '2024-01-01'
      AND order_status = 'completed'
),
customer_totals AS (
  SELECT
    customer_id,
```

SQL

The corresponding SQL query:

```
SUM(order_amount) as total_amount
      FROM filtered_data
      GROUP BY customer_id
    )
    SELECT * FROM customer_totals;
"""

}

return examples

def demonstrate_predicate_pushdown(self):
    """Show predicate pushdown optimization patterns"""

    examples = {
        'inefficient_late_filtering': """
        -- INEFFICIENT: Filtering after JOIN
        SELECT
            o.order_id,
            c.customer_name,
            p.product_name
        FROM orders o
        JOIN customers c ON o.customer_id = c.customer_id
        JOIN products p ON o.product_id = p.product_id
        WHERE o.order_date >= '2024-01-01' -- Filter
```

SQL

applied after all joins

SQL

```

AND c.customer_segment = 'premium'
    AND p.category = 'electronics';
""",
'optimized_early_filtering': ""
-- OPTIMIZED: Filter each table before joining
SELECT
    o.order_id,
    c.customer_name,
    p.product_name
FROM (
    SELECT order_id, customer_id, product_id
    FROM orders
    WHERE order_date >= '2024-01-01' -- Filter

```

orders early

SQL

```

) o
JOIN (
    SELECT customer_id, customer_name
    FROM customers
    WHERE customer_segment = 'premium' -- Filter

```

customers early

SQL

```

) c ON o.customer_id = c.customer_id
JOIN (
    SELECT product_id, product_name
    FROM products
    WHERE category = 'electronics' -- Filter

```

products early

SQL

```

) p ON o.product_id = p.product_id;
""",
'optimized_with_ctes': """
    -- OPTIMIZED: Using CTEs for clarity and performance
    WITH filtered_orders AS (
        SELECT order_id, customer_id, product_id
        FROM orders
        WHERE order_date >= '2024-01-01'
    ),
    premium_customers AS (
        SELECT customer_id, customer_name
        FROM customers
        WHERE customer_segment = 'premium'
    ),
    electronics_products AS (
        SELECT product_id, product_name
        FROM products
        WHERE category = 'electronics'
    )
    SELECT
        o.order_id,

```

The corresponding SQL query:

SQL

```

c.customer_name,
    p.product_name
FROM filtered_orders o
JOIN premium_customers c ON o.customer_id =

```

```

c.customer_id

```

SQL

```

JOIN electronics_products p ON o.product_id =

```

```

p.product_id;

```

```

"""
}

return examples

```

Advanced Analytical Query Optimization

```
-- Complex analytical query optimization patterns
```

SQL

```
-- 1. Window function optimization
```

```
-- INEFFICIENT: Multiple window functions with same partitioning
```

```
SELECT
```

```
    customer_id,
```

```
    order_date,
```

```
    amount,
```

```
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date)
```

as order_number,

```
LAG(amount) OVER (PARTITION BY customer_id ORDER BY order_date)
```

SQL

as previous_amount,

```
LEAD(amount) OVER (PARTITION BY customer_id ORDER BY order_date)
```

SQL

as next_amount,

```
SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date)
```

SQL

as running_total,

```
AVG(amount) OVER (PARTITION BY customer_id ORDER BY order_date
```

SQL

ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING) as moving_avg

```
FROM orders;
```

```
-- OPTIMIZED: Use WINDOW clause to define partitioning once
SELECT
    customer_id,
    order_date,
    amount,
    ROW_NUMBER() OVER w as order_number,
    LAG(amount) OVER w as previous_amount,
    LEAD(amount) OVER w as next_amount,
    SUM(amount) OVER w as running_total,
    AVG(amount) OVER (w ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING) as
```

moving_avg

FROM orders

```
WINDOW w AS (PARTITION BY customer_id ORDER BY order_date);

-- 2. Recursive CTE optimization
-- Calculate customer hierarchy (referrals)
WITH RECURSIVE customer_hierarchy AS (
    -- Base case: top-level customers (no referrer)
    SELECT
        customer_id,
        customer_name,
        referred_by_customer_id,
        0 as hierarchy_level,
        customer_id as root_customer_id
    FROM customers
    WHERE referred_by_customer_id IS NULL

    UNION ALL

    -- Recursive case: customers referred by someone
    SELECT
        c.customer_id,
        c.customer_name,
        c.referred_by_customer_id,
        ch.hierarchy_level + 1,
        ch.root_customer_id
    FROM customers c
    INNER JOIN customer_hierarchy ch ON c.referred_by_customer_id =
```

ch.customer_id

```
WHERE ch.hierarchy_level < 10 -- Prevent infinite recursion
)
SELECT
    root_customer_id,
    hierarchy_level,
    COUNT(*) as customers_at_level,
    SUM(COUNT(*)) OVER (PARTITION BY root_customer_id ORDER BY
```

SQL

hierarchy_level) as cumulative_referrals

```
FROM customer_hierarchy
GROUP BY root_customer_id, hierarchy_level
ORDER BY root_customer_id, hierarchy_level;

-- 3. Pivot table optimization
-- INEFFICIENT: Multiple conditional aggregates
SELECT
    product_category,
    SUM(CASE WHEN EXTRACT(QUARTER FROM order_date) = 1 THEN amount
```

SQL

ELSE 0 END) as Q1_sales,

```
SUM(CASE WHEN EXTRACT(QUARTER FROM order_date) = 2 THEN amount
```

SQL

ELSE 0 END) as Q2_sales,

```
SUM(CASE WHEN EXTRACT(QUARTER FROM order_date) = 3 THEN amount
```

SQL

ELSE 0 END) as Q3_sales,

```
SUM(CASE WHEN EXTRACT(QUARTER FROM order_date) = 4 THEN amount
```

SQL

ELSE 0 END) as Q4_sales

FROM orders o

SQL

```
JOIN products p ON o.product_id = p.product_id
WHERE EXTRACT(YEAR FROM order_date) = 2024
GROUP BY product_category;

-- OPTIMIZED: Use native PIVOT function (where available)
SELECT *
FROM (
    SELECT
        p.product_category,
        EXTRACT(QUARTER FROM o.order_date) as quarter,
        o.amount
    FROM orders o
    JOIN products p ON o.product_id = p.product_id
    WHERE EXTRACT(YEAR FROM order_date) = 2024
)
PIVOT (
    SUM(amount)
    FOR quarter IN (1 as Q1_sales, 2 as Q2_sales, 3 as Q3_sales, 4
```

as Q4_sales)

SQL

```
);

-- 4. Percentile and ranking optimization
-- Calculate percentiles efficiently for large datasets
WITH sales_metrics AS (
    SELECT
        customer_id,
        SUM(amount) as total_sales,
        COUNT(*) as order_count,
        AVG(amount) as avg_order_value
    FROM orders
    WHERE order_date >= '2024-01-01'
    GROUP BY customer_id
),
percentile_calculations AS (
    SELECT
        customer_id,
        total_sales,
        order_count,
        avg_order_value,
        -- Use PERCENT_RANK for efficiency
        PERCENT_RANK() OVER (ORDER BY total_sales) as
```

sales_percentile,

SQL

```
NTILE(10) OVER (ORDER BY total_sales) as sales_decile,
-- More efficient than PERCENTILE_CONT for discrete
```

percentiles

SQL

```
PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY total_sales)
```

```
OVER () as median_sales,
```

SQL

```
PERCENTILE_DISC(0.9) WITHIN GROUP (ORDER BY total_sales)
```

```
OVER () as p90_sales
```

```

FROM sales_metrics
)
SELECT
    sales_decile,
    COUNT(*) as customers_in_decile,
    MIN(total_sales) as min_sales,
    MAX(total_sales) as max_sales,
    AVG(total_sales) as avg_sales,
    FIRST_VALUE(median_sales) OVER () as overall_median
FROM percentile_calculations
GROUP BY sales_decile, median_sales
ORDER BY sales_decile;

-- 5. Time-series gap filling optimization
-- Fill missing dates in time series data efficiently
WITH date_range AS (
    SELECT DATE_ADD('2024-01-01', INTERVAL seq DAY) as calendar_date
    FROM (
        SELECT ROW_NUMBER() OVER () - 1 as seq
        FROM orders
        LIMIT 365 -- Generate 365 consecutive dates
    ) sequence
    WHERE seq < 365
),
daily_sales AS (
    SELECT
        DATE(order_date) as sale_date,
        SUM(amount) as daily_revenue,
        COUNT(*) as daily_orders
    FROM orders
    WHERE order_date >= '2024-01-01'
        AND order_date < '2025-01-01'
    GROUP BY DATE(order_date)
)
SELECT
    dr.calendar_date,
    COALESCE(ds.daily_revenue, 0) as revenue,
    COALESCE(ds.daily_orders, 0) as orders,
    -- Use window functions to calculate moving averages over gaps
    AVG(ds.daily_revenue) OVER (
        ORDER BY dr.calendar_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) as seven_day_avg_revenue
FROM date_range dr
LEFT JOIN daily_sales ds ON dr.calendar_date = ds.sale_date
ORDER BY dr.calendar_date;

```

Query Performance Monitoring

PYTHON

```
class QueryPerformanceMonitor:
    """Monitor and analyze query performance patterns"""

    def create_query_performance_dashboard(self):
        """Create comprehensive query performance monitoring"""

        monitoring_queries = {
            'slowest_queries': """
                -- Identify slowest queries in last 24 hours
                SELECT
                    query_id,
                    user_name,
                    warehouse_name,
                    database_name,
                    TRUNCATE(query_text, 100) as query_snippet,
                    start_time,
                    total_elapsed_time / 1000 as
```

execution_time_seconds,

```
queued_provisioning_time / 1000 as
```

queue_time_seconds,

```
compilation_time / 1000 as
```

compilation_time_seconds,

```
execution_time / 1000 as
```

actual_execution_seconds,

SQL

```

bytes_scanned / (1024*1024*1024) as gb_scanned,
    rows_produced,
    credits_used_cloud_services,
    partitions_scanned,
    partitions_total
FROM snowflake.account_usage.query_history
WHERE start_time >= DATEADD('hour', -24,

```

```
CURRENT_TIMESTAMP())
```

SQL

```

AND execution_status = 'SUCCESS'
    AND total_elapsed_time > 30000 -- > 30 seconds
ORDER BY total_elapsed_time DESC
LIMIT 50;
""",

'query_patterns_analysis': ""
-- Analyze common query patterns and performance
WITH query_patterns AS (
    SELECT
        query_type,
        database_name,
        schema_name,
        warehouse_name,
        COUNT(*) as query_count,
        AVG(total_elapsed_time / 1000) as

```

```
avg_execution_time,
```

```
MEDIAN(total_elapsed_time / 1000) as
```

```
median_execution_time,
```

```
MAX(total_elapsed_time / 1000) as
```

```
max_execution_time,
```

```
AVG(bytes_scanned / (1024*1024*1024)) as
```

```
avg_gb_scanned,
```

```
SUM(credits_used_cloud_services) as
```

total_credits_used

```
FROM snowflake.account_usage.query_history  
WHERE start_time >= DATEADD('day', -7,
```

SQL

```
CURRENT_TIMESTAMP())
```

```
AND execution_status = 'SUCCESS'  
GROUP BY query_type, database_name, schema_name,
```

SQL

warehouse_name

```
)  
SELECT  
    query_type,  
    database_name,  
    query_count,  
    avg_execution_time,  
    median_execution_time,  
    total_credits_used,  
    -- Identify optimization opportunities  
    CASE  
        WHEN avg_execution_time > 60 AND query_count
```

SQL

> 100

THEN 'High impact optimization target'

```
WHEN avg_execution_time > 30 AND
```

avg_gb_scanned > 10

THEN 'Review query efficiency'

```
WHEN query_count > 1000 AND
```

```
total_credits_used > 100
```

THEN 'High frequency queries - consider caching'

ELSE 'Normal performance'

```
END as optimization_priority
FROM query_patterns
WHERE query_count >= 10
ORDER BY
    CASE optimization_priority
        WHEN 'High impact optimization target' THEN
```

SQL

1

```
WHEN 'Review query efficiency' THEN 2
```

WHEN 'High frequency queries - consider caching' THEN 3

```
ELSE 4
    END,
    total_credits_used DESC;
""",

'resource_utilization_analysis': ""
-- Analyze warehouse resource utilization
SELECT
    warehouse_name,
    DATE(start_time) as usage_date,
    COUNT(*) as total_queries,
    SUM(total_elapsed_time) / 1000 / 60 as
```

SQL

```
total_execution_minutes,
```

```
AVG(total_elapsed_time / 1000) as
```

avg_query_time_seconds,

```
SUM(credits_used) as total_credits,  
SUM(bytes_scanned) / (1024*1024*1024*1024) as
```

total_tb_scanned,

```
-- Identify efficiency metrics  
SUM(credits_used) / COUNT(*) as
```

credits_per_query,

```
SUM(bytes_scanned) / SUM(credits_used) as
```

bytes_per_credit,

```
COUNT(CASE WHEN total_elapsed_time > 60000 THEN
```

1 END) as slow_queries,

```
COUNT(CASE WHEN total_elapsed_time > 60000 THEN
```

```
1 END) * 100.0 / COUNT(*) as slow_query_percentage
```

```
FROM snowflake.account_usage.query_history  
WHERE start_time >= DATEADD('day', -30,
```

SQL

```
CURRENT_TIMESTAMP())
```

```

AND execution_status = 'SUCCESS'
        AND warehouse_name IS NOT NULL
        GROUP BY warehouse_name, DATE(start_time)
        HAVING total_queries >= 10
        ORDER BY usage_date DESC, total_credits DESC;
    """
}

return monitoring_queries

def create_automated_optimization_suggestions(self):
    """Generate automated optimization suggestions"""

    optimization_rules = """
    -- Automated optimization suggestion engine
    WITH query_analysis AS (
        SELECT
            query_id,
            query_text,
            total_elapsed_time,
            bytes_scanned,
            partitions_scanned,
            partitions_total,
            rows_examined,
            rows_produced,
            -- Extract patterns from query text
            CASE
                WHEN query_text ILIKE '%SELECT *%' THEN

```

'uses_select_star'

```

ELSE 'selective_columns'
END as column_selection_pattern,
CASE
    WHEN partitions_scanned = partitions_total AND

```

partitions_total > 1

```

THEN 'full_table_scan'
    ELSE 'partition_pruning'
END as partition_pattern,
CASE
    WHEN query_text ILIKE '%ORDER BY%' AND

```

query_text NOT ILIKE '%LIMIT%'

```
THEN 'sort_without_limit'
    ELSE 'normal_sort'
END as sort_pattern
FROM snowflake.account_usage.query_history
WHERE start_time >= DATEADD('day', -7,
```

SQL

CURRENT_TIMESTAMP())

```
AND execution_status = 'SUCCESS'
AND total_elapsed_time > 10000 -- Focus on queries
```

> 10 seconds

```
)
SELECT
    query_id,
    LEFT(query_text, 200) as query_preview,
    total_elapsed_time / 1000 as execution_time_seconds,
    bytes_scanned / (1024*1024*1024) as gb_scanned,
    -- Generate optimization suggestions
    ARRAY_CONSTRUCT_COMPACT(
        CASE WHEN column_selection_pattern =
```

SQL

'uses_select_star'

```
THEN 'Replace SELECT * with specific columns'
```

SQL

END,

```
CASE WHEN partition_pattern = 'full_table_scan'
    THEN 'Add partition filters to WHERE clause'
```

SQL

END,

```
CASE WHEN sort_pattern = 'sort_without_limit'
      THEN 'Add LIMIT clause to ORDER BY queries'
```

SQL

END,

```
CASE WHEN bytes_scanned / (1024*1024*1024) > 100
      THEN 'Query scans >100GB - consider pre-
```

aggregation' END,

```
CASE WHEN rows_examined > rows_produced * 100
      THEN 'Low selectivity - improve WHERE
```

SQL

conditions' END

```
) as optimization_suggestions,
-- Estimate potential improvement
CASE
  WHEN column_selection_pattern = 'uses_select_star'
    AND partition_pattern = 'full_table_scan'
  THEN 'High (50-90% improvement possible)'
  WHEN partition_pattern = 'full_table_scan'
  THEN 'Medium (30-60% improvement possible)'
  WHEN column_selection_pattern = 'uses_select_star'
  THEN 'Low-Medium (20-40% improvement possible)'
  ELSE 'Low (10-20% improvement possible)'
END as improvement_potential
FROM query_analysis
WHERE ARRAY_SIZE(
  ARRAY_CONSTRUCT_COMPACT(
    CASE WHEN column_selection_pattern =
```

SQL

'uses_select_star'

```
THEN 'Replace SELECT * with specific columns'
```

SQL

END,

```
CASE WHEN partition_pattern = 'full_table_scan'
      THEN 'Add partition filters to WHERE clause'
```

SQL

END,

```
CASE WHEN sort_pattern = 'sort_without_limit'
      THEN 'Add LIMIT clause to ORDER BY queries'
```

SQL

END,

```
CASE WHEN bytes_scanned / (1024*1024*1024) > 100
      THEN 'Query scans >100GB - consider pre-
```

aggregation' END,

```
CASE WHEN rows_examined > rows_produced * 100
      THEN 'Low selectivity - improve WHERE
```

SQL

conditions' END

```
)
) > 0
ORDER BY
  CASE improvement_potential
    WHEN 'High (50-90% improvement possible)' THEN 1
    WHEN 'Medium (30-60% improvement possible)' THEN 2
    WHEN 'Low-Medium (20-40% improvement possible)' THEN
```

SQL

3

```
ELSE 4
  END,
  total_elapsed_time DESC;
""""

return optimization_rules
```


⚠ Common Mistake

⚠ Common Mistake Don't optimize queries in isolation. A 10-second query that runs once per day might not need optimization, while a 1-second query that runs 10,000 times per day definitely does. Focus optimization efforts on queries with the highest total impact (execution time × frequency).

□ Checkpoint

1. What's the most impactful optimization for analytical queries?
a. Adding more indexes
b. Improving join order and eliminating unnecessary data scans
c. Using more expensive hardware
d. Running queries at off-peak hours
2. When should you use approximate functions like APPROX_COUNT_DISTINCT?
a. Never - they're inaccurate
b. For large datasets where small accuracy trade-offs provide

significant performance gains

c. Only for small datasets
d. When exact results are required

3. What does predicate pushdown accomplish?
a. It pushes queries to different servers
b. It filters data early in the execution plan to reduce processing
c. It makes queries run in parallel
d. It improves data quality

Answers: 1-b, 2-b, 3-b

5.5 Multi-Tenant Analytics Architecture

🔗 TL;DR

Multi-tenancy in analytics requires balancing isolation, performance, and cost efficiency - Row-level security and data masking enable shared table approaches - Separate databases/schemas provide stronger isolation but increase complexity - Tenant-aware partitioning strategies optimize for both isolation and performance - Cross-tenant analytics require careful design to preserve tenant boundaries

Multi-Tenant Architecture Patterns

Multi-tenant analytics architectures must balance competing requirements: tenant isolation, query performance, cost efficiency, and operational simplicity.

```
class MultiTenantArchitectureDesigner:
    """Design multi-tenant analytics architectures"""

    def __init__(self, tenant_count, data_volume_per_tenant):
        self.tenant_count = tenant_count
        self.data_volume_per_tenant = data_volume_per_tenant
        self.isolation_requirements = {}

    def design_architecture_pattern(self, requirements):
        """Choose multi-tenant pattern based on requirements"""

        patterns = {
            'shared_database_shared_schema': {
                'isolation_level': 'Low',
                'operational_complexity': 'Low',
                'cost_efficiency': 'High',
                'performance_impact': 'Low',
                'suitable_for': 'SaaS applications with similar
```

PYTHON

tenants, low security requirements',

```
'implementation': self._design_shared_everything()  
,  
'shared_database_separate_schema': {  
    'isolation_level': 'Medium',  
    'operational_complexity': 'Medium',  
    'cost_efficiency': 'Medium',  
    'performance_impact': 'Medium',  
    'suitable_for': 'Medium security requirements,
```

PYTHON

moderate tenant count (<1000)',

```
'implementation': self._design_schema_per_tenant()  
,  
'separate_database_per_tenant': {  
    'isolation_level': 'High',  
    'operational_complexity': 'High',  
    'cost_efficiency': 'Low',  
    'performance_impact': 'High (for cross-tenant
```

PYTHON

queries)',

```
'suitable_for': 'High security/compliance
```

requirements, enterprise customers',

```
'implementation': self._design_database_per_tenant()  
,  
'hybrid_approach': {  
    'isolation_level': 'Variable',  
    'operational_complexity': 'High',  
    'cost_efficiency': 'Medium',  
    'performance_impact': 'Variable',  
    'suitable_for': 'Mixed tenant types (enterprise +
```

PYTHON

SMB)',

```

'implementation': self._design_hybrid_architecture()
    }
}

# Recommendation logic based on requirements
if requirements['tenant_count'] > 10000:
    return patterns['shared_database_shared_schema']
elif requirements['security_level'] == 'high':
    return patterns['separate_database_per_tenant']
elif requirements['cross_tenant_analytics'] == True:
    return patterns['shared_database_shared_schema']
else:
    return patterns['shared_database_separate_schema']

def _design_shared_everything(self):
    """Design shared database, shared schema with row-level
security"""

```

security"""

```

architecture = {
    'data_model': """
        -- Shared tables with tenant_id column
        CREATE TABLE shared_orders (
            order_id BIGINT,
            tenant_id VARCHAR(50) NOT NULL,
            customer_id BIGINT,
            order_date DATE,
            amount DECIMAL(10,2),
            status VARCHAR(20),
            created_at TIMESTAMP
        )
        CLUSTER BY (tenant_id, order_date);

        CREATE TABLE shared_customers (
            customer_id BIGINT,
            tenant_id VARCHAR(50) NOT NULL,
            customer_name VARCHAR(255),
            email VARCHAR(255),
            created_at TIMESTAMP
        )
        CLUSTER BY (tenant_id, customer_id);
    """,
    'security_model': """

```

Row-level security policy:

SQL

```
-- Row-level security policy
CREATE ROW ACCESS POLICY tenant_isolation_policy
AS (tenant_id VARCHAR) RETURNS BOOLEAN ->
    tenant_id = CURRENT_TENANT_ID()
    OR IS_ROLE_IN_SESSION('ADMIN_ROLE');

-- Apply to all tables
ALTER TABLE shared_orders ADD ROW ACCESS POLICY
```

```
tenant_isolation_policy ON (tenant_id);
```

SQL

```
ALTER TABLE shared_customers ADD ROW ACCESS POLICY
```

```
tenant_isolation_policy ON (tenant_id);
```

```
-- Column masking for cross-tenant analytics role
CREATE MASKING POLICY email_masking_policy AS (val
```

```
STRING) RETURNS STRING ->
```

```
CASE
    WHEN CURRENT_ROLE() =
```

```
'CROSS_TENANT_ANALYTICS'
```

```
THEN REGEXP_REPLACE(val, '.*@(.+)',
```

```
'***@\\1')
```

ELSE val

```
END;
```

SQL

```
    ALTER TABLE shared_customers MODIFY COLUMN email
    SET MASKING POLICY email_masking_policy;
    """,
    'partitioning_strategy': ""
    -- Tenant-aware partitioning
    CREATE TABLE shared_orders_partitioned (
        order_id BIGINT,
        tenant_id VARCHAR(50) NOT NULL,
        order_date DATE,
        amount DECIMAL(10,2)
    )
    PARTITION BY (tenant_id, DATE_TRUNC('month',
```

```
order_date))
```

```

    CLUSTER BY (tenant_id, order_date);
    """
    'benefits': [
        'Lowest operational overhead',
        'Easy cross-tenant analytics',
        'Efficient resource utilization',
        'Simple backup and maintenance'
    ],
    'drawbacks': [
        'Risk of data leakage',
        'Limited customization per tenant',
        'Noisy neighbor problems',
        'Complex security implementation'
    ]
}

return architecture

def _design_schema_per_tenant(self):
    """Design separate schema per tenant"""

    architecture = {
        'schema_organization': """
            -- Create schemas per tenant
            CREATE SCHEMA tenant_1001_data;
            CREATE SCHEMA tenant_1002_data;
            CREATE SCHEMA tenant_1003_data;

            -- Each schema has identical table structure
            USE SCHEMA tenant_1001_data;
            CREATE TABLE orders (
                order_id BIGINT,
                customer_id BIGINT,
                order_date DATE,
                amount DECIMAL(10,2)
            );
        """
    }

```

Creating the customers table:

```
CREATE TABLE customers (  
    customer_id BIGINT,  
    customer_name VARCHAR(255),  
    email VARCHAR(255)  
);  
""",  
'access_control': """  
    -- Role per tenant  
    CREATE ROLE tenant_1001_role;  
    CREATE ROLE tenant_1002_role;  
  
    -- Grant access to tenant-specific schema only  
    GRANT USAGE ON SCHEMA tenant_1001_data TO
```

tenant_1001_role;

GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES

IN SCHEMA tenant_1001_data TO tenant_1001_role;

```
-- Cross-tenant analytics role  
CREATE ROLE cross_tenant_analytics_role;  
GRANT USAGE# Module 6: Data Lake & Lakehouse
```

Architecture

□ What you'll learn

> **What you'll learn:** How to build flexible, scalable data lakes

that don't become swamps. You'll master lakehouse architecture patterns, understand when to use Delta Lake vs Iceberg, and learn to design data lakes that scale from gigabytes to petabytes while maintaining governance and performance.


```
---
```

6.1 Data Lake Organization Patterns

- ```
> **TL;DR**
> - Organize your data lake in zones: Raw → Refined → Curated
> - Use consistent naming conventions and directory structures
> - Implement metadata management from day one, not as an
```

afterthought

- ```
> - Data lakes fail without governance - swamps are real
```

Most data lakes fail. Not because the technology is bad, but because

teams treat them like dumping grounds. "Just throw all the data in S3 and figure it out later."

Six months later, you have 10,000 directories with names like

`data\_export\_final\_v2\_ACTUAL\_FINAL.csv` and nobody knows what anything contains. Your data lake has become a data swamp.

The secret to successful data lakes isn't fancy technology - it's

****organization and governance from day one**.**

The Zone-Based Architecture Pattern

[DIAGRAM: Data lake zones shown as three connected areas flowing

left to right:

****Raw Zone (Bronze)**:** Various data sources (APIs, databases, files,

streams) flowing into structured folders with labels like "orders/2024/03/15/batch_001.parquet", "users/streaming/hour=14/", "products/weekly/export.json"

****Refined Zone (Silver)****: Cleaned and validated data with

consistent schemas, labeled "orders_cleaned/year=2024/month=03/", "users_enriched/", "products_standardized/"

****Curated Zone (Gold)****: Business-ready datasets and aggregations,

labeled "fact_orders/", "dim_customers/", "daily_metrics/", "ml_features/"

Arrows show data flow between zones with transformation processes at each boundary.]

****Raw Zone (Bronze) - The Firehose****

- ****Purpose:**** Exact copy of source data, no modifications
- ****Format:**** Source format (JSON, CSV, Avro, whatever came in)
- ****Schema:**** Schema-on-read, may be inconsistent
- ****Retention:**** Keep forever (disk is cheap, rebuilding is

expensive)

```

- **Users:** Data engineers for investigation and reprocessing

**Refined Zone (Silver) - The Clean Room**
- **Purpose:** Cleaned, validated, schema-consistent data
- **Format:** Parquet/Iceberg with consistent schemas
- **Schema:** Enforced schema with evolution tracking
- **Retention:** Long-term (2-5 years typically)
- **Users:** Data engineers and advanced analysts

**Curated Zone (Gold) - The Business Layer**
- **Purpose:** Business-ready datasets and aggregations
- **Format:** Optimized for consumption (star schema, feature sets)
- **Schema:** Stable business schemas with documentation
- **Retention:** Medium-term (1-3 years, depending on use)
- **Users:** Business analysts, data scientists, BI tools

> **Pro Tip**
> The zone model isn't just about data quality - it's about **blast

```

radius******. When something breaks in Raw, it doesn't affect Curated data consumption. When business logic changes, you only rebuild from Silver forward, not from source.

Directory Structure Patterns

****The Netflix Pattern (Partition by Time and Source):****

```

s3://data-lake/ └─ raw/ │ └─ orders/ │ │ └─ year=2024/month=03/day=15/hour=14/
│ │ │ │ └─ batch_20240315_140000.parquet │ │ │ │ └─
batch_20240315_143000.parquet │ │ └─ _metadata/ │ │ └─ schema_v1.json │ │ └─
lineage_info.json │ └─ users/ │ └─ streaming/ │ │ └─ year=2024/month=03/day=15/
hour=14/ │ └─ batch/ │ └─ year=2024/month=03/day=15/ └─ refined/ │ └─ or-
ders_cleaned/ │ │ └─ year=2024/month=03/day=15/ │ └─ users_enriched/ │ └─
year=2024/month=03/day=15/ └─ curated/ └─ analytics/ │ │ └─
daily_sales_by_region/ │ └─ monthly_cohort_analysis/ │ └─ customer_lifetime_value/
└─ ml_features/ │ │ └─ user_behavior_features/ │ │ └─
product_recommendation_features/ └─ data_products/ └─ customer_360/ └─ finan-
cial_reporting/

```

****The Uber Pattern (Partition by Domain and Environment):**** s3://data-lake/ └─ do-
mains/ │ └─ rides/ │ │ └─ raw/ │ │ └─ refined/ │ │ └─ curated/ │ └─ eats/ │ │ └─

raw/ | | └─ refined/ | | └─ curated/ | └─ freight/ └─ shared/ | └─ dimensions/ | |
└─ geography/ | | └─ time/ | └─ metrics/ └─ environments/ └─ dev/ └─ staging/
└─ prod/

Naming Conventions That Scale

****Bad Naming (Data Swamp Style):**** user_data_export_march.csv or-
ders_final_v2.parquet customer_analysis_FIXED.json temp_data_for_sarah.txt

****Good Naming (Lakehouse Style):****

Format: {domain}{entity}{type} {version}{timestamp} user_behavi-
or_events_v1_20240315_140000 order_transactions_refined_v2_20240315 custom-
er_segments_curated_v1_20240315

Include critical metadata in names or-
ders_raw_cdc_mysql_replica_v1_20240315_140000
users_batch_salesforce_export_v1_20240315
products_streaming_kafka_events_v1_20240315_140000

Metadata Management Patterns

The difference between a data lake and a data swamp is ****metadata****.

You need to track:

- - ****Schema evolution:**** How data structures change over time
- ****Data lineage:**** Where data comes from and where it goes
- ****Quality metrics:**** Freshness, completeness, accuracy
- ****Access patterns:**** Who uses what data when
- ****Business context:**** What the data means

****Metadata Store Example (Using Apache Atlas or AWS Glue):****

```
```python # Example metadata registration for a new dataset dataset_metadata = {  
"name": "orders_raw_mysql_v1",
"location": "s3://data-lake/raw/orders/year=2024/month=03/day=15/",
"format": "parquet",
"schema": {
```

```
"order_id": "string",
"customer_id": "string",
"order_total": "decimal(10,2)",
"order_date": "timestamp",
"status": "string"
```

```
},
```

```
"partitions": ["year", "month", "day"],
```

```
"source": {
```

```
"system": "mysql_orders_db",
"table": "orders",
"extraction_method": "cdc",
"last_updated": "2024-03-15T14:00:00Z"
```

```
},
```

```
"quality_metrics": {
```

```
"completeness_score": 0.98,
"freshness_sla": "< 5 minutes",
"row_count": 125000,
"null_percentage": 0.02
```

```
},
```

```
"business_context": {
```

```
"owner": "orders_team",
"description": "Real-time order transactions from main e-
```

SQL

```
commerce system",
```

```
"pii_fields": ["customer_email", "shipping_address"],
"retention_policy": "7 years"
```

```
} }
```

# Register with metadata service  
metadata\_service.register\_dataset(dataset\_metadata)

## Data Cataloging and Discovery

Apache Hive Metastore Pattern:

```
-- Register table in Hive Metastore for SQL access
CREATE TABLE orders_raw (
 order_id STRING,
```

SQL

```
 customer_id STRING,
 order_total DECIMAL(10,2),
 order_date TIMESTAMP,
 status STRING
)
USING PARQUET
PARTITIONED BY (year INT, month INT, day INT)
LOCATION 's3://data-lake/raw/orders/'
```

Self-Describing Data Pattern:

```
Include metadata in the data itself
def write_with_metadata(df, path, metadata):
 # Write data
 df.write.parquet(path)

 # Write metadata sidecar file
 metadata_path = f"{path}/_metadata.json"
 with open(metadata_path, 'w') as f:
 json.dump({
 "schema": df.schema.json(),
 "row_count": df.count(),
 "created_at": datetime.utcnow().isoformat(),
 "source": metadata["source"],
 "quality_checks": metadata["quality_checks"]
 }, f)
```

PYTHON

### ***Real-World Example: Spotify's Data Lake***

Organization

Spotify processes 100+ billion events per day across hundreds of datasets. Their lakehouse organization:

#### Zone Strategy:

- Raw (Eventhouse): Kafka events stored in Avro format, partitioned by date and hour
- Derived (Feature Store): Pre- aggregated features for ML models
- Curated (Analytics): Business metrics and dimensional models

#### Domain Organization:

/music/ /streaming\_events/ /playlist\_interactions/ /recommendation\_signals/ /podcasts/ /episode\_plays/ /creator\_metrics/ /ads/ /impression\_events/ /campaign\_performance/

#### Metadata Strategy:

- Schema Registry: Confluent Schema Registry for Kafka topics
- Data Catalog: Custom-built catalog with lineage tracking
- Quality Monitoring: Automated data quality checks at zone boundaries

#### ⚠ Common Mistake

⚠ Common Mistake Don't try to build the metadata system from day one. Start with basic organization (zones + naming conventions) and add metadata gradually. A simple, enforced naming convention beats a complex metadata system that nobody uses.

#### □ Checkpoint

1. Explain the purpose of each zone in the Bronze/Silver/Gold pattern
2. Why is the zone model better than dumping everything in one location?
3. What are the key elements of effective data lake naming conventions?
4. How would you organize a data lake for a company with 3 business domains?

06

# **Data Lake & Lakehouse Architecture**



## 6.2 Lakehouse Implementation Strategies

---

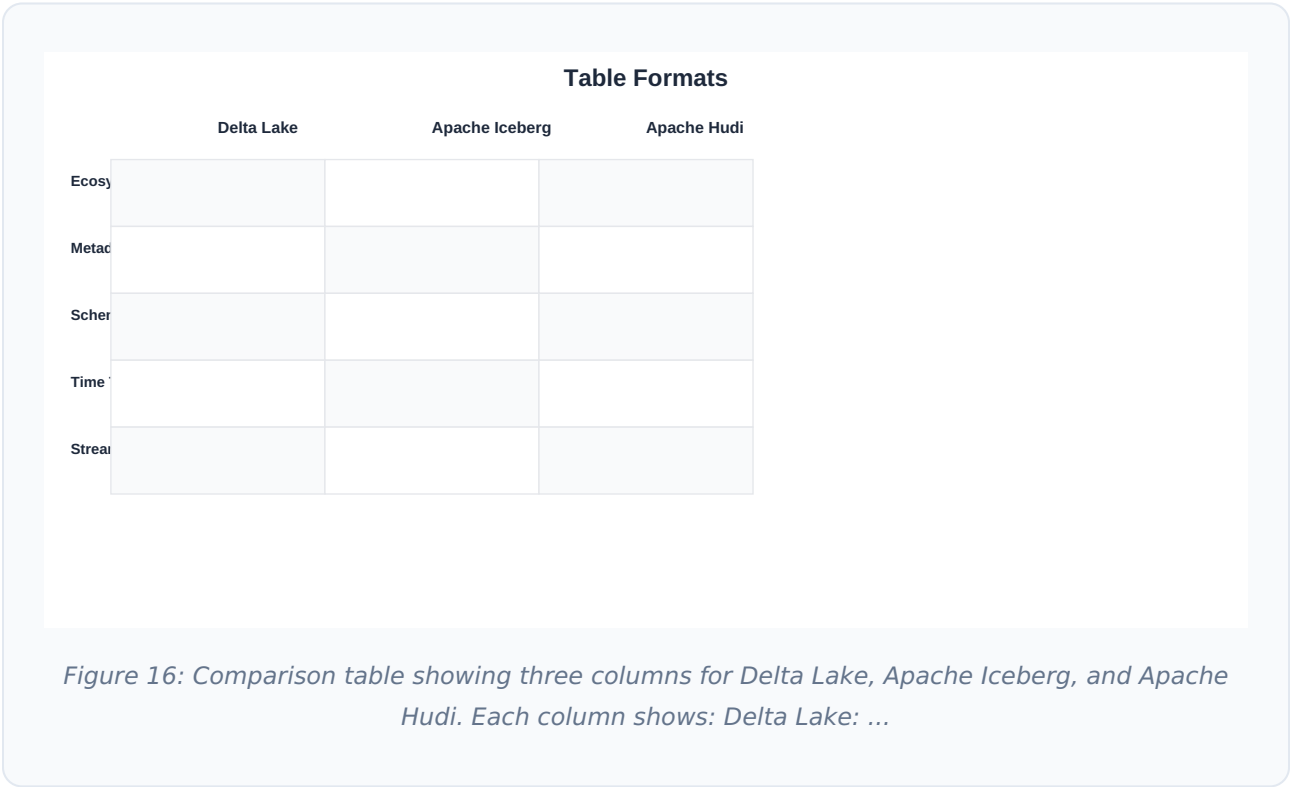
### ⚡ TL;DR

Lakehouse = Data Lake + Data Warehouse capabilities (ACID, schema evolution, time travel) - Delta Lake vs Iceberg vs Hudi - pick based on your ecosystem, not features - ACID transactions prevent data corruption but add complexity - Time travel is game-changing for debugging and compliance

Traditional data lakes are just file storage. You can't update files atomically, can't handle concurrent writers safely, and can't easily rollback bad data. You need a separate data warehouse for reliable analytics.

**Lakehouse architecture solves this by bringing database** capabilities to data lakes: ACID transactions, schema evolution, and time travel, all while storing data in open formats on cheap object storage.

# Lakehouse Table Format Comparison



## Delta Lake Implementation Pattern

Basic Delta Lake Setup:

```
Create a Delta table
from delta.tables import DeltaTable

Initial write
df.write.format("delta").save("s3://lakehouse/orders/")

Read as Delta table
orders = spark.read.format("delta").load("s3://lakehouse/orders/")

ACID updates and merges
deltaTable = DeltaTable.forPath(spark, "s3://lakehouse/orders/")

Upsert new data
deltaTable.alias("target").merge(
 new_orders_df.alias("source"),
 "target.order_id = source.order_id"
).whenMatchedUpdateAll() \
 .whenNotMatchedInsertAll() \
 .execute()
```

### Schema Evolution Example:

```
Original schema
original_orders = ["order_id", "customer_id", "amount",
```

"order\_date"]

```
Add new column without breaking existing pipelines
new_orders_df = original_df.withColumn("discount_amount", lit(0.0))
```

```
Delta Lake handles schema evolution automatically
new_orders_df.write.format("delta") \
 .option("mergeSchema", "true") \
 .mode("append") \
 .save("s3://lakehouse/orders/")

Historical data will have null for new columns
New data will have the new schema
```

### Time Travel for Debugging:

PYTHON

```
See what the data looked like yesterday
yesterday_orders = spark.read.format("delta") \
 .option("timestampAsOf", "2024-03-
```

```
14T00:00:00Z") \
```

PYTHON

```
.load("s3://lakehouse/orders/")

Or version-based time travel
v5_orders = spark.read.format("delta") \
 .option("versionAsOf", 5) \
 .load("s3://lakehouse/orders/")

Compare versions to debug data issues
current_count =
```

```
spark.read.format("delta").load("s3://lakehouse/orders/").count()
```

PYTHON

```
yesterday_count = yesterday_orders.count()
print(f"Row count change: {current_count - yesterday_count}")
```

## Apache Iceberg Implementation Pattern

---

Multi-Engine Compatibility:

```
Write with Spark
df.write.format("iceberg").save("catalog.db.orders")

Read with Trino
SELECT order_id, customer_id, amount
FROM catalog.db.orders
WHERE order_date >= DATE '2024-03-01'

Read with Flink for streaming
CREATE TABLE orders (
 order_id STRING,
 customer_id STRING,
 amount DECIMAL(10,2),
 order_date TIMESTAMP
) WITH (
 'connector' = 'iceberg',
 'catalog-name' = 'hadoop_catalog',
 'warehouse' = 's3://lakehouse/',
 'database-name' = 'ecommerce',
 'table-name' = 'orders'
)
```

## Advanced Schema Evolution:

```
Iceberg handles complex schema changes
Add nested columns
ALTER TABLE catalog.db.orders
ADD COLUMN shipping_info STRUCT<
 address: STRING,
 city: STRING,
 state: STRING,
 zip_code: STRING
>

Rename columns without rewriting data
ALTER TABLE catalog.db.orders
RENAME COLUMN customer_id TO user_id

Change column types (with compatibility checks)
```

*The corresponding SQL query:*

```
ALTER TABLE catalog.db.orders
ALTER COLUMN amount TYPE DECIMAL(12,2)
```

SQL

## Snapshot Management:

```
-- View snapshots
SELECT * FROM catalog.db.orders.snapshots

-- Create branch for experimentation
ALTER TABLE catalog.db.orders
CREATE BRANCH experiment_branch
AS OF VERSION 12

-- Work on branch without affecting main
INSERT INTO catalog.db.orders.branch_experiment_branch
VALUES (...)

-- Merge branch back to main (if experiments worked)
ALTER TABLE catalog.db.orders
REPLACE BRANCH main
WITH BRANCH experiment_branch
```

SQL

# ACID Transaction Implementation

## The Challenge Without ACID:

```
This can create inconsistent data!
Writer 1 starts writing partition 2024-03-15
orders_batch_1.write.mode("append").parquet("s3://lake/orders/date=2024-
```

03-15/")

```
Writer 2 starts writing same partition simultaneously
orders_batch_2.write.mode("append").parquet("s3://lake/orders/date=2024-
```

03-15/")

```
Reader might see partial data from both writers
Result: inconsistent, corrupted reads
```

### The Solution With Lakehouse:

```
Delta Lake ensures atomic writes
@retry(stop=stop_after_attempt(3), wait=wait_exponential())
def atomic_upsert(new_data, table_path):
 delta_table = DeltaTable.forPath(spark, table_path)

 # This entire operation is atomic
 delta_table.alias("target").merge(
 new_data.alias("source"),
 "target.order_id = source.order_id"
).whenMatchedUpdateAll() \
 .whenNotMatchedInsertAll() \
 .execute()

 # Either all data is committed or none is
 # Readers always see consistent state

Multiple writers can safely write concurrently
atomic_upsert(orders_batch_1, "s3://lakehouse/orders/")
atomic_upsert(orders_batch_2, "s3://lakehouse/orders/") # Won't
```

conflict

## Performance Optimization Strategies

### File Sizing Optimization:

```
Problem: Too many small files
Solution: Optimize file sizes during writes
df.coalesce(200).write.format("delta").save("s3://lakehouse/orders/")

Better: Use Delta Lake's auto-optimization
deltaTable.optimize().executeCompaction()
```

*Best: Automatic optimization on writes:*

```
Best: Automatic optimization on writes
spark.conf.set("delta.autoOptimize.optimizeWrite", "true")
spark.conf.set("delta.autoOptimize.autoCompact", "true")
```

## Z-Order Clustering:

```
Optimize for common query patterns
deltaTable.optimize().where("date >= '2024-03-01'") \
 .zOrderBy("customer_id", "product_category") \
 .execute()

Now queries filtering by customer_id and product_category
will skip irrelevant files
```

## Partition Evolution:

```
Start with daily partitions
orders_df.write.format("delta") \
 .partitionBy("year", "month", "day") \
 .save("s3://lakehouse/orders/")

As data grows, consider hourly partitions for recent data
current_month_df.write.format("delta") \
 .partitionBy("year", "month", "day", "hour") \
 .mode("append") \
 .save("s3://lakehouse/orders/")

Iceberg supports partition evolution without rewriting
ALTER TABLE catalog.db.orders
SET PARTITION SPEC (year, month, day, hour)
```

## Real-World Example: Netflix's Iceberg

### Implementation

Netflix processes 2.5 trillion events per day using Iceberg. Their implementation strategy:

### Multi-Engine Architecture:

Data Sources → Kafka → Flink (streaming) → Iceberg Tables



↘  
Spark (batch) → Iceberg Tables  
↘

Trino (analytics) ← Iceberg Tables ← Iceberg Catalog

Table Management:

```
Automatic compaction for streaming writes
CREATE TABLE netflix.events.user_interactions (
 user_id BIGINT,
 content_id STRING,
 event_type STRING,
 timestamp TIMESTAMP,
 session_id STRING
)
USING iceberg
PARTITIONED BY (days(timestamp))
TBLPROPERTIES (
 'write.target-file-size-bytes' = '134217728', # 128MB files
 'write.delete.mode' = 'merge-on-read',
 'commit.retry.num-retries' = '3'
)

Automatic maintenance
-- Daily compaction job
CALL
```

SQL

```
catalog.system.rewrite_data_files('netflix.events.user_interactions')
```

```
-- Weekly partition optimization
CALL
```

catalog.system.rewrite\_manifests('netflix.events.user\_interactions')

#### 📌 Pro Tip

Don't migrate your entire data lake to lakehouse format at once. Start with your most critical, frequently updated datasets. The ones where data quality issues cause the most pain. Then gradually expand coverage.

# Choosing Your Lakehouse Technology

---

Choose Delta Lake When: - You're already using Databricks or

**Spark heavily - You need mature streaming integration - You want the** easiest getting-started experience - Your team is comfortable with Databricks ecosystem

**Choose Apache Iceberg When: - You need true multi-engine**

- support (Spark + Trino + Flink)
- You want vendor independence
- Schema evolution is critical for your use case
- You're building for 5+ year timeline

Choose Apache Hudi When:

- You need frequent updates/upserts (IoT, CDC)
- You want incremental processing capabilities
- You're comfortable with higher complexity
- Your primary use case is stream processing

## □ Checkpoint

1. What are the three main benefits that lakehouse brings over traditional data lakes?
2. Compare Delta Lake, Iceberg, and Hudi - what are the key differentiators?
3. Why are ACID transactions important for data lakes?
4. How would you design a migration strategy from data lake to lakehouse?

## ⚡ TL;DR

Governance isn't about control, it's about enabling safe self-service - Implement access controls, data classification, and lineage tracking from day one - Automate quality gates between zones to prevent bad data propagation - Privacy and compliance are design requirements, not afterthoughts

## ***The biggest difference between a successful data lake and a data***

swamp isn't technology—it's governance. Without governance, your data lake becomes a liability: ungoverned access, unknown data quality, privacy violations, and compliance nightmares.

But governance done wrong kills productivity. The goal is governed self-service: teams can access and use data safely without waiting for approvals or risking violations.

### Data Classification and Security Model

#### Classification Framework:

```
Example data classification schema
DATA_CLASSIFICATIONS = {
 "PUBLIC": {
 "sensitivity": "Low",
 "access_level": "All employees",
 "retention": "Unlimited",
 "examples": ["Product catalogs", "Marketing materials"]
 },
 "INTERNAL": {
 "sensitivity": "Medium",
 "access_level": "Employees only",
 "retention": "7 years",
 "examples": ["Sales data", "Business metrics"]
 },
}
```

```

"CONFIDENTIAL": {
 "sensitivity": "High",
 "access_level": "Need-to-know basis",
 "retention": "3 years",
 "examples": ["Customer PII", "Financial data"]
},
"RESTRICTED": {
 "sensitivity": "Critical",
 "access_level": "Legal/Security approval",
 "retention": "As required by law",
 "examples": ["Payment info", "Health records"]
}
}

```

## Role-Based Access Control (RBAC) Implementation:

```

Lake Formation or similar access control
roles:
 data_analysts:
 permissions:
 - database: ecommerce
 table: orders_curated
 access: SELECT
 columns: [order_id, customer_id, amount, order_date]
 row_filter: "order_date >= current_date - interval '2

```

SQL

years"

```

- database: ecommerce
 table: customers_pii
 access: DENIED

data_scientists:
 permissions:
 - database: ecommerce
 table: "*"
 access: SELECT
 columns: "*"
 row_filter: "data_classification != 'RESTRICTED'"

finance_team:
 permissions:
 - database: ecommerce
 table: financial_*
 access: SELECT, UPDATE
 columns: "*"

data_engineers:
 permissions:
 - database: "*"
 table: "*"
 access: ALL
 conditions: ["approved_by_data_governance_team"]

```

## PII Detection and Handling

Automated PII Detection:

```

import re
from typing import Dict, List

class PIIDetector:
 def __init__(self):
 self.patterns = {
 'email': re.compile(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-
]+\.[A-Z|a-z]{2,}\b'),

```

```
'ssn': re.compile(r'\b\d{3}-\d{2}-\d{4}\b'),
'phone': re.compile(r'\b\d{3}-\d{3}-\d{4}\b'),
'credit_card': re.compile(r'\b\d{4}[\s-]?\d{4}[\s-]?
```

```
\d{4}[\s-]?\d{4}\b')
```

```
}
```

PYTHON

```
def scan_dataset(self, df: DataFrame) -> Dict[str, List[str]]:
 pii_findings = {}
```

*The corresponding SQL query:*

```
for column in df.columns:
 # Sample data to avoid scanning entire dataset
 sample_data = df.select(column).sample(0.1).collect()

 for pii_type, pattern in self.patterns.items():
 matches = []
 for row in sample_data:
 if pattern.search(str(row[column])):
 matches.append(column)
 break

 if matches:
 pii_findings[pii_type] =
```

SQL

```
pii_findings.get(pii_type, []) + matches
```

```
return pii_findings
```

**PYTHON**

```
Auto-classify datasets based on PII content
def classify_dataset(dataset_path: str) -> str:
 df = spark.read.parquet(dataset_path)
 pii_detector = PIIDetector()
 pii_findings = pii_detector.scan_dataset(df)

 if any(['ssn', 'credit_card'] in pii_findings.keys()):
 return "RESTRICTED"
 elif any(['email', 'phone'] in pii_findings.keys()):
 return "CONFIDENTIAL"
 else:
 return "INTERNAL"
```

## Data Masking Patterns:

```
from pyspark.sql.functions import regexp_replace, sha2, concat, lit
```

**PYTHON**

```
def apply_data_masking(df, classification, user_role):
 """Apply appropriate masking based on data classification and
```

```
user role"""
```

```
if classification == "RESTRICTED" and user_role not in ["legal",
```

```
"security"]:
```

```
Complete denial of access
raise PermissionError(f"User role {user_role} cannot access
```

```
RESTRICTED data")
```

```
elif classification == "CONFIDENTIAL" and user_role not in
```

**PYTHON**

```
["data_scientist", "analyst_senior"]:
```

```
Mask PII fields
masked_df = df.withColumn("email",
 regexp_replace("email", "(.{3})(.*)
```

```
(@.*)", "$1***$3")) \
```

```
.withColumn("phone",
 regexp_replace("phone", "(\\d{3})
```

```
(\\d{3})(\\d{4})", "$1-***-$3")) \
```

```
.withColumn("customer_id",
 sha2(concat(lit("salt_"),
```

```
col("customer_id")), 256))
```

```
return masked_df
```

```
else:
 # Full access for appropriate roles
 return df
```

## Data Lineage Tracking

Lineage Metadata Schema:

```
Data lineage tracking
class LineageTracker:
 def __init__(self):
 self.metadata_store = {}
```

PYTHON

*The track\_transformation implementation:*

```
def track_transformation(self, job_name: str, input_datasets:
```

PYTHON



```
List[str],
```

```
output_dataset: str,
```

transformation\_logic: str):

```
lineage_record = {
 "job_name": job_name,
 "timestamp": datetime.utcnow(),
 "inputs": input_datasets,
 "output": output_dataset,
 "transformation": transformation_logic,
 "job_id": str(uuid.uuid4())
}

Store in metadata system (Atlas, DataHub, etc.)
self.metadata_store[output_dataset] = lineage_record

def get_upstream_dependencies(self, dataset: str, max_depth: int
```

PYTHON

= 5) -> Dict:

```
"""Get all upstream datasets that contribute to this
```

dataset"""

```
dependencies = {}

def traverse(current_dataset, depth=0):
 if depth > max_depth or current_dataset not in
```

PYTHON

self.metadata\_store:

```

return

 record = self.metadata_store[current_dataset]
 dependencies[current_dataset] = record

 for input_dataset in record["inputs"]:
 traverse(input_dataset, depth + 1)

 traverse(dataset)
 return dependencies

Example usage in ETL job
lineage = LineageTracker()

def daily_customer_metrics():
 # Track what we're building
 input_datasets = [
 "s3://lake/raw/orders/2024/03/15/",
 "s3://lake/raw/customers/2024/03/15/",
 "s3://lake/refined/products/"
]

 orders = spark.read.parquet(input_datasets[0])
 customers = spark.read.parquet(input_datasets[1])
 products = spark.read.parquet(input_datasets[2])

 # Business logic
 customer_metrics = orders.join(customers, "customer_id") \
 .join(products, "product_id") \
 .groupBy("customer_id") \
 .agg(

```

```
sum("order_amount").alias("total_spent"),
```

```
count("*").alias("order_count"),
```

```
max("order_date").alias("last_order_date")
```

```
)
```

```
output_path =
```

```
"s3://lake/curated/customer_daily_metrics/2024/03/15/"
```

```
customer_metrics.write.parquet(output_path)

Track lineage
lineage.track_transformation(
 job_name="daily_customer_metrics",
 input_datasets=input_datasets,
 output_dataset=output_path,
```

*The corresponding SQL query:*

```
transformation_logic="""
 Join orders + customers + products
 Group by customer_id
 Calculate total_spent, order_count, last_order_date
 """

)
```

SQL

## Quality Gates Between Zones

Automated Quality Checks:

```
from great_expectations import DataContext
import pandas as pd

class QualityGate:
 def __init__(self, zone_name: str):
 self.zone = zone_name
 self.ge_context = DataContext()

 def validate_raw_to_refined(self, dataset_path: str) -> bool:
 """Quality checks when moving from raw to refined zone"""

 df = spark.read.parquet(dataset_path)

 # Convert to pandas for Great Expectations
 sample_df = df.sample(0.1).toPandas()

 # Define expectations for refined zone
 expectations = [
 # Data completeness
 ("expect_table_row_count_to_be_between", {"min_value":
```

PYTHON

```
1000})),
```

```
Key columns should not be null
("expect_column_values_to_not_be_null", {"column":
```

```
"order_id"})),
```

```
("expect_column_values_to_not_be_null", {"column":
```

```
"customer_id"})),
```

```
Data types should be correct
("expect_column_values_to_be_of_type", {"column":
```

```
"order_date", "type_": "datetime64"})),
```

```
Business logic validation
("expect_column_values_to_be_between", {"column":
```

```
"order_amount", "min_value": 0})),
```

```
Referential integrity (if customer dim exists)
 ("expect_column_values_to_be_in_set", {
 "column": "customer_id",
 "value_set": self.get_valid_customer_ids()
 })
]

Run validations
validation_results = []
for expectation_type, kwargs in expectations:
 result = getattr(sample_df, expectation_type)(**kwargs)
 validation_results.append(result.success)

All checks must pass for data to proceed
all_passed = all(validation_results)

if not all_passed:
 self.log_quality_failure(dataset_path,
```

PYTHON

validation\_results)

```
return all_passed

def validate_refined_to_curated(self, dataset_path: str) ->
```

PYTHON

bool:

```
"""Quality checks when moving from refined to curated
```

SQL

zone"""

```
More stringent checks for business-ready data
df = spark.read.parquet(dataset_path)

checks = [
 # Schema stability
 self.validate_schema_consistency(df),

 # Business logic validation
 self.validate_business_rules(df),

 # Freshness validation
 self.validate_data_freshness(df),

 # Completeness validation
 self.validate_completeness(df)
]

return all(checks)

Example integration in ETL pipeline
quality_gate = QualityGate("refined")

def raw_to_refined_pipeline():
 raw_data =
```

PYTHON

```
spark.read.parquet("s3://lake/raw/orders/2024/03/15/")
```

PYTHON

```
Clean and transform data
refined_data = raw_data.filter(col("order_amount") > 0) \
 .withColumn("order_date",
```

```
to_timestamp("order_date")) \
```

SQL

```
.dropDuplicates(["order_id"])

Write to staging location first
staging_path = "s3://lake/staging/orders_refined/2024/03/15/"
refined_data.write.mode("overwrite").parquet(staging_path)

Quality gate validation
if quality_gate.validate_raw_to_refined(staging_path):
 # Move to refined zone
 final_path = "s3://lake/refined/orders/2024/03/15/"
 refined_data.write.mode("overwrite").parquet(final_path)

 # Clean up staging
 spark.sql(f"DROP TABLE IF EXISTS staging.orders_temp")
else:
 # Alert on quality failure
 send_alert(f"Quality gate failed for {staging_path}")
 raise Exception("Data quality validation failed")
```

## Compliance and Auditing

### GDPR Right to be Forgotten Implementation:

PYTHON

```
class GDPRCompliance:
 def __init__(self, lakehouse_path: str):
 self.lakehouse_path = lakehouse_path

 def forget_customer(self, customer_id: str):
 """Remove all data for a customer across the entire
```

```
lakehouse"""
```

```

affected_tables = [
 "orders", "order_items", "customer_interactions",
 "marketing_events", "support_tickets"
]

deletion_log = []

for table in affected_tables:
 table_path = f"{self.lakehouse_path}/{table}/"

```

The *PYTHON* code:

```

if self.table_uses_delta(table_path):
 # Delta Lake deletion
 delta_table = DeltaTable.forPath(spark, table_path)

 rows_before = delta_table.toDF().count()
 delta_table.delete(f"customer_id = '{customer_id}'")
 rows_after = delta_table.toDF().count()

 deletion_log.append({
 "table": table,
 "rows_deleted": rows_before - rows_after,
 "timestamp": datetime.utcnow()
 })

else:
 # For non-Delta tables, need to rewrite partitions
 self.rewrite_partitions_without_customer(table_path,

```

customer\_id)

```

Log deletion for audit trail
self.log_gdpr_deletion(customer_id, deletion_log)

def anonymize_customer(self, customer_id: str):
 """Anonymize rather than delete (for analytics that need row

```

counts)"""

```

 anonymized_id =

```

```
f"anon_{hashlib.sha256(customer_id.encode()).hexdigest()[:8]}"
```

```
affected_tables = ["orders", "customer_interactions"]
```

```
for table in affected_tables:
 delta_table = DeltaTable.forPath(spark, f"
```

```
{self.lakehouse_path}/{table}/")
```

```
delta_table.update(
 condition = f"customer_id = '{customer_id}'",
 set = {
 "customer_id": f"'{anonymized_id}'",
 "anonymized_at": "current_timestamp()",
 "pii_removed": "true"
 }
)

Audit trail implementation
def log_data_access(user_id: str, dataset_path: str, query: str):
 """Log all data access for compliance auditing"""

 access_log = {
 "timestamp": datetime.utcnow(),
 "user_id": user_id,
 "dataset": dataset_path,
 "query": query,
 "ip_address": request.remote_addr,
 "classification": get_dataset_classification(dataset_path)
 }

 # Write to audit log (separate from business data)
 audit_df = spark.createDataFrame([access_log])
```

PYTHON

```
audit_df.write.mode("append").parquet("s3://governance/audit_logs/data_access/")
```

## ***Real-World Example: Airbnb's Data Governance***

Airbnb manages 150+ petabytes of data with strict governance requirements:

Data Classification System:



```
Airbnb's data classification tiers
classifications = {
```

```
"L1_PUBLIC": "Marketing data, public metrics",
"L2_INTERNAL": "Business metrics, aggregated user data",
"L3_CONFIDENTIAL": "Individual user data, host information",
"L4_RESTRICTED": "Payment data, government IDs, private
```

communications"

```
}
```

PYTHON

```
Automated classification based on column names
def auto_classify_table(schema):
 restricted_columns = ['ssn', 'payment', 'credit_card',
```

'government\_id']

```
confidential_columns = ['email', 'phone', 'address', 'user_id']

if any(col in schema.names for col in restricted_columns):
 return "L4_RESTRICTED"
elif any(col in schema.names for col in confidential_columns):
 return "L3_CONFIDENTIAL"
else:
 return "L2_INTERNAL"
```

PYTHON

Self-Service with Guardrails:

```
-- Users can create derived tables but with automatic governance
CREATE TABLE my_analysis AS
SELECT
 anonymized_user_id, -- Auto-anonymized
 city,
 booking_count,
 avg_rating
FROM bookings_curated
WHERE created_date >= '2024-01-01'
 AND data_classification <=
```

SQL

```
user_clearance_level('john.doe@airbnb.com')
```

**SQL**

```
-- Governance system automatically:
-- 1. Validates user has access to source tables
-- 2. Inherits highest classification from sources
-- 3. Applies appropriate retention policies
-- 4. Tracks lineage automatically
```

Pro Tip Start governance simple but enforce it consistently. A

basic classification system that everyone follows beats a complex system that everyone ignores. Add sophistication gradually as your team matures.

#### □ Checkpoint

1. What are the four levels of data classification and what access controls apply to each?
2. How would you implement automated PII detection in a data lake?
3. Why are quality gates between zones important for governance?
4. How would you handle a GDPR deletion request in a lakehouse architecture?

## 6.4 Performance Optimization in Data Lakes

### ⚡ TL;DR

File size matters: target 128MB-1GB per file for optimal performance - Partitioning is powerful but can backfire - avoid small partitions - Column pruning and predicate pushdown are your best friends - Caching and materialization can speed up queries 100x

Query performance in data lakes is fundamentally different from databases. No indexes, no query optimizer that knows your data distribution, no buffer pool. Just you, your files, and the laws of physics.

**The good news: you can achieve incredible performance if you** organize your data for how it's actually queried.

### File Size Optimization Strategies

The Small Files Problem:

```
This creates terrible performance
for hour in range(24):
 hourly_data = generate_hourly_data(hour)
 # Creates 24 tiny files, each ~10MB
```

```
hourly_data.write.mode("append").parquet(f"s3://lake/orders/hour= {hour}/")
```

```
Reading this is extremely slow
df = spark.read.parquet("s3://lake/orders/")
Spark has to open 24 files, each requiring S3 API calls
Each file is too small to utilize parallelism effectively
```

PYTHON

The Solution - Optimal File Sizing:

```
Collect hourly data and write optimal-sized files
daily_data = []
for hour in range(24):
 hourly_data = generate_hourly_data(hour)
 daily_data.append(hourly_data)

Combine and write as optimally-sized files
combined_df = reduce(DataFrame.union, daily_data)

Target 128MB-1GB per file
optimal_partitions = max(1, combined_df.count() // 1000000) # ~1M
```

rows per partition

```
combined_df.coalesce(optimal_partitions) \
 .write.mode("overwrite") \
 .parquet("s3://lake/orders/date=2024-03-15/")
```

Dynamic File Sizing Based on Data Volume:

```
def optimal_file_count(df: DataFrame) -> int:
 """Calculate optimal file count based on data size"""

 # Estimate data size (rough calculation)
 row_count = df.count()
 avg_row_size = 500 # bytes - adjust based on your schema
 total_size_mb = (row_count * avg_row_size) / (1024 * 1024)

 # Target 256MB per file
 target_file_size_mb = 256
 optimal_files = max(1, int(total_size_mb / target_file_size_mb))

 return optimal_files

Use in your ETL pipeline
processed_data = transform_data(raw_data)
file_count = optimal_file_count(processed_data)
processed_data.coalesce(file_count).write.parquet(output_path)
```

PYTHON

## Intelligent Partitioning Strategies

The Partitioning Sweet Spot:

```
Bad: Too many small partitions
Creates 365 * 24 = 8,760 partitions per year!
df.write.partitionBy("year", "month", "day", "hour").parquet(path)

Bad: Too few large partitions
Each partition contains entire year of data
df.write.partitionBy("year").parquet(path)

Good: Balanced partitioning based on query patterns
Most queries filter by day, some by hour
df.write.partitionBy("year", "month", "day").parquet(path)
```

*Even better: Dynamic partitioning based on data volume:*

```
Even better: Dynamic partitioning based on data volume
def smart_partition_strategy(df, base_columns=["year", "month"]):
 row_count = df.count()

 if row_count < 1_000_000:
 # Small data - minimal partitioning
 return base_columns
 elif row_count < 100_000_000:
 # Medium data - add day partitioning
 return base_columns + ["day"]
 else:
 # Large data - add hour partitioning for recent data only
 return base_columns + ["day", "hour"]

partition_columns = smart_partition_strategy(daily_orders)
daily_orders.write.partitionBy(*partition_columns).parquet(output_path)
```

PYTHON

Partition Pruning Optimization:

```
-- Query that benefits from partition pruning
SELECT customer_id, SUM(order_amount)
FROM orders
WHERE order_date >= '2024-03-01'
 AND order_date < '2024-04-01'
 AND customer_region = 'US'
GROUP BY customer_id

-- If partitioned by (year, month), Spark only reads March 2024
```

SQL

files

```
-- But still scans all files for customer_region filter

-- Better partitioning strategy for this query pattern:
-- Partition by date, cluster by customer_region
```

## Z-Order Clustering (Delta Lake):

```
from delta.tables import DeltaTable

Optimize table for common filter combinations
delta_table = DeltaTable.forPath(spark, "s3://lakehouse/orders/")

Z-order by columns frequently used together in WHERE clauses
delta_table.optimize().where("order_date >= '2024-03-01'") \
 .zOrderBy("customer_region",
```

SQL

```
"product_category") \
```

```
.execute()

Now queries filtering by customer_region + product_category
will skip most files even within the same partition
```

## Column Pruning and Predicate Pushdown

### Schema Evolution for Performance:

```
Bad: Wide table with many rarely-used columns
orders_schema = StructType([
 StructField("order_id", StringType()),
 StructField("customer_id", StringType()),
 StructField("product_id", StringType()),
 StructField("order_amount", DecimalType(10, 2)),
 StructField("order_date", TimestampType()),
 StructField("shipping_address", StringType()), # Rarely queried
 StructField("billing_address", StringType()), # Rarely queried
 StructField("customer_notes", StringType()), # Rarely queried
 StructField("internal_notes", StringType()), # Rarely queried
 StructField("audit_log", StringType()) # Very large,
```

rarely queried

```
])

Better: Separate frequently-queried from rarely-queried columns
core_orders_schema = StructType([
 StructField("order_id", StringType()),
```

SQL

*The corresponding SQL query:*

```
StructField("customer_id", StringType()),
 StructField("product_id", StringType()),
 StructField("order_amount", DecimalType(10, 2)),
 StructField("order_date", TimestampType())
])

extended_orders_schema = StructType([
 StructField("order_id", StringType()),
 StructField("shipping_address", StringType()),
 StructField("billing_address", StringType()),
 StructField("customer_notes", StringType()),
 StructField("internal_notes", StringType()),
 StructField("audit_log", StringType())
])

Store as separate datasets, join when needed
core_df.write.parquet("s3://lake/orders_core/")
extended_df.write.parquet("s3://lake/orders_extended/")
```

SQL

Optimizing Column Layout:

```
Column order matters for columnar formats
Put commonly filtered columns first

Bad: Random column order
df.select("order_id", "notes", "customer_id", "order_date",
```

SQL

"status", "amount")

```
Good: Filter columns first, then select columns
df.select("order_date", "status", "customer_region", # Common
```

SQL

filters

```
"order_id", "customer_id", "amount", # Common
```

selects

```
"notes", "metadata") # Rare access
```

## Caching and Materialization Strategies

Intelligent Caching:

```
Cache frequently-accessed intermediate results
def get_customer_metrics(start_date, end_date):
 cache_key = f"customer_metrics_{start_date}_{end_date}"

 # Check if cached version exists and is fresh
 if cache_exists(cache_key) and cache_age(cache_key) <
```

PYTHON

timedelta(hours=6):

```
return spark.read.parquet(f"s3://cache/{cache_key}/")

Compute fresh metrics
orders = spark.read.parquet("s3://lake/orders/") \
 .filter(col("order_date").between(start_date,
```

PYTHON

end\_date))



PYTHON

```
customers = spark.read.parquet("s3://lake/customers/")

metrics = orders.join(customers, "customer_id") \
 .groupBy("customer_id", "customer_segment") \
 .agg(
 sum("order_amount").alias("total_spent"),
 count("*").alias("order_count"),
 avg("order_amount").alias("avg_order_value")
)

Cache result for reuse
```

```
metrics.write.mode("overwrite").parquet(f"s3://cache/{cache_key}/")
```

```
return metrics

Usage in multiple queries/dashboards reuses cached computation
customer_analysis = get_customer_metrics("2024-03-01", "2024-03-31")
```

## Materialized Views Pattern:

PYTHON

```
For frequently-run expensive queries, pre-compute results
class MaterializedView:
 def __init__(self, name: str, query: str, refresh_schedule:
```

```
str):
```

```

self.name = name
 self.query = query
 self.refresh_schedule = refresh_schedule
 self.path = f"s3://lake/materialized_views/{name}/"

def refresh(self):
 """Refresh the materialized view"""
 result = spark.sql(self.query)
 result.write.mode("overwrite").parquet(self.path)

 # Update metadata
 metadata = {
 "last_refreshed": datetime.utcnow(),
 "row_count": result.count(),
 "refresh_duration_seconds": time.time() - start_time
 }
 self.update_metadata(metadata)

def read(self):
 """Read from the materialized view"""
 return spark.read.parquet(self.path)

Define common materialized views
daily_sales_by_region = MaterializedView(
 name="daily_sales_by_region",
 query="""
 SELECT
 order_date,
 customer_region,
 product_category,
 COUNT(*) as order_count,
 SUM(order_amount) as total_sales,
 AVG(order_amount) as avg_order_value
 FROM orders_curated o
 JOIN customers_curated c ON o.customer_id = c.customer_id
 JOIN products_curated p ON o.product_id = p.product_id
 WHERE order_date >= current_date - interval '90 days'
 GROUP BY 1, 2, 3
 """,
 refresh_schedule="0 2 * * *" # Daily at 2 AM
)

Refresh on schedule
daily_sales_by_region.refresh()

Queries use pre-computed results instead of scanning raw data

```

```
dashboard_data = daily_sales_by_region.read() \
 .filter(col("order_date") >=
```

```
"2024-03-15")
```

## Query Optimization Techniques

Join Optimization:

```
Bad: Large table to large table join without optimization
orders = spark.read.parquet("s3://lake/orders/") # 1B rows
customers = spark.read.parquet("s3://lake/customers/") # 100M rows

This will be slow - both sides shuffled across network
result = orders.join(customers, "customer_id")

Better: Broadcast small table if it fits in memory
from pyspark.sql.functions import broadcast

customers = spark.read.parquet("s3://lake/customers/")
if customers.count() < 1_000_000: # <200MB estimated
 result = orders.join(broadcast(customers), "customer_id")
else:
```

SQL

*Use bucketing for large-large joins:*

```
Use bucketing for large-large joins
result = orders.join(customers, "customer_id")
```

SQL

Bucketing for Repeated Joins:

```
Pre-bucket frequently-joined tables
orders.write \
 .bucketBy(50, "customer_id") \
 .sortBy("order_date") \
 .saveAsTable("orders_bucketed")

customers.write \
 .bucketBy(50, "customer_id") \
 .saveAsTable("customers_bucketed")

Joins between bucketed tables avoid shuffle
result = spark.table("orders_bucketed").join(
 spark.table("customers_bucketed"),
 "customer_id"
)
```

## Real-World Example: Uber's Query Optimization

Uber runs 100,000+ queries daily on their data lake. Their optimization strategies:

Multi-Layer Caching:

```
Layer 1: Hot data in memory (Redis/Alluxio)
Layer 2: Warm data pre-computed (Hive tables)
Layer 3: Cold data on demand (S3)

def get_trip_metrics(city, date_range):
 # Try hot cache first
 hot_key = f"trips_{city}_{date_range.start}_{date_range.end}"
 if redis.exists(hot_key):
 return redis.get(hot_key)

 # Try warm cache (pre-computed daily aggregates)
 warm_data = spark.sql(f"""
 SELECT * FROM trip_metrics_daily
 WHERE city = '{city}'
 AND date BETWEEN '{date_range.start}' AND
 '{date_range.end}'
```

```

"""
)

if warm_data.count() > 0:
 redis.setex(hot_key, 3600, warm_data.toPandas().to_json())
 return warm_data

Fall back to raw data computation
return compute_from_raw(city, date_range)

```

### Adaptive Partitioning:

```

Uber adjusts partitioning based on city size
def partition_strategy(city_data):
 daily_trips = city_data.groupBy("date").count().avg()

 if daily_trips < 1000:
 # Small city: monthly partitions
 return ["year", "month"]
 elif daily_trips < 50000:
 # Medium city: daily partitions
 return ["year", "month", "day"]
 else:
 # Large city: hourly partitions
 return ["year", "month", "day", "hour"]

```

Pro Tip Performance optimization is about understanding

your query patterns first, then optimizing storage for those patterns. Don't optimize randomly—measure query performance and optimize the slowest, most frequent queries first.

#### □ Checkpoint

1. What's the optimal file size range for data lake performance and why?
2. How would you decide between daily vs hourly partitioning?
3. What's the difference between column pruning and predicate pushdown?
4. When would you use a materialized view vs real-time computation?

# Multi-Format Data Handling

## ⚡ TL;DR

Structured data (Parquet) for analytics, semi-structured (JSON) for flexibility, unstructured (text/images) for AI - Schema- on-read enables flexibility but requires governance - Text and image data need specialized processing pipelines - Hybrid approaches combine the best of structured and unstructured

Most data lakes start simple: CSV files from databases, maybe some JSON from APIs. But modern applications generate much more: product images, customer support chat logs, sensor data, social media posts, video content, mobile app events with nested JSON.

A mature data lake needs to handle all formats efficiently while still enabling fast analytics.

## Structured Data Optimization

Columnar Storage for Analytics:

```
Convert row-based formats to columnar for better analytics
```

performance

```
def optimize_for_analytics(source_format: str, data_path: str,
```

**PYTHON**

```
output_path: str):
```

**PYTHON**

```
"""Convert various formats to optimized Parquet"""
```

```
if source_format == "csv":
 # CSV optimization
 df = spark.read.option("header", "true") \
 .option("inferSchema", "true") \
 .csv(data_path)

 # Clean up data types
 optimized_df = df.withColumn("order_date",
```

```
to_timestamp("order_date")) \
```

```
.withColumn("order_amount",
```

```
col("order_amount").cast("decimal(10,2)"))
```

**PYTHON**

```
elif source_format == "json":
 # JSON optimization - flatten nested structures for
```

## analytics

**SQL**

```
df = spark.read.json(data_path)

Flatten nested JSON for better columnar compression
optimized_df = df.select(
 col("order.order_id").alias("order_id"),
 col("order.customer.id").alias("customer_id"),
```

```
col("order.amount").cast("decimal(10,2)").alias("order_amount"),
```

```
col("order.items").alias("order_items_json") # Keep
```

## complex nested data as JSON string

```
)

Write with optimal compression and partitioning
optimized_df.write \
 .option("compression", "snappy") \
 .partitionBy("year", "month", "day") \
 .mode("overwrite") \
 .parquet(output_path)
```

## Delta Lake for ACID Operations:

```
Handle slowly changing dimensions with Delta Lake
def update_customer_dimension(new_customer_data):
 """Update customer dimension with Type 2 SCD pattern"""

 delta_table = DeltaTable.forPath(spark,
```

**PYTHON**

```
"s3://lake/curated/dim_customers/")
```

```
Type 2 SCD: Close current records and insert new ones
delta_table.alias("target").merge(
 new_customer_data.alias("source"),
 "target.customer_id = source.customer_id AND
```

```
target.is_current = true"
```

```
).whenMatchedUpdate(set = {
 "is_current": "false",
 "effective_end_date": "current_date()",
 "updated_at": "current_timestamp()"
}).whenNotMatchedInsert(values = {
 "customer_id": "source.customer_id",
 "customer_name": "source.customer_name",
 "customer_email": "source.customer_email",
 "customer_segment": "source.customer_segment",
 "is_current": "true",
 "effective_start_date": "current_date()",
 "effective_end_date": "date '9999-12-31'",
 "created_at": "current_timestamp()",
 "updated_at": "current_timestamp()"
}).execute()
```



# Semi-Structured Data Patterns

Schema-on-Read with JSON:

```
Flexible JSON processing for varying schemas
class JSONProcessor:
 def __init__(self, schema_registry_path: str):
 self.schema_registry_path = schema_registry_path
 self.known_schemas = self.load_schemas()

 def process_json_batch(self, json_files_path: str):
 """Process JSON files with varying schemas"""

 # Read all JSON files
 raw_df = spark.read.option("multiline",
```

PYTHON

```
"true").json(json_files_path)
```

```
Detect schema variations
schema_variants = self.detect_schema_variants(raw_df)

processed_dfs = []

for variant_id, variant_schema in schema_variants.items():
 # Filter records matching this schema variant
 variant_filter =
```

PYTHON

```
self.build_schema_filter(variant_schema)
```

```
variant_df = raw_df.filter(variant_filter)

Normalize to canonical schema
normalized_df = self.normalize_schema(variant_df,
```

PYTHON

```
variant_schema)
```

```
Add metadata about schema variant
normalized_df =
```

```
normalized_df.withColumn("schema_variant", lit(variant_id)) \
```

```
.withColumn("processed_at",
```

```
current_timestamp())
```

```
processed_dfs.append(normalized_df)
```

```
Union all variants
return reduce(DataFrame.union, processed_dfs)
```

*The normalize\_schema implementation:*

```
def normalize_schema(self, df: DataFrame, variant_schema: dict):
 """Convert variant schema to canonical form"""

 canonical_columns = []

 for canonical_field, field_config in
```

PYTHON

```
self.get_canonical_schema().items():
```

```
Map from variant field name to canonical
variant_field = variant_schema.get(canonical_field)

if variant_field:
 # Direct mapping
```

SQL

```
canonical_columns.append(col(variant_field).alias(canonical_field))
```

```
elif field_config.get("required", False):
 # Required field missing - use default
```

PYTHON

```
canonical_columns.append(lit(field_config["default"]).alias(canonical_field))
```

```
else:
 # Optional field missing - use null
```

```
canonical_columns.append(lit(None).cast(field_config["type"]).alias(canonical_f
ield))
```

```
return df.select(*canonical_columns)
```

**SQL**

```
Example usage for e-commerce events with varying schemas
json_processor =
```

```
JSONProcessor("s3://lake/schemas/ecommerce_events/")
```

```
Schema variant 1: Mobile app events
mobile_events = {
 "event_id": "event_id",
 "user_id": "user_id",
 "event_type": "action",
 "product_id": "item_id",
 "timestamp": "event_time",
 "device_info": "device" # Extra field
}

Schema variant 2: Web events
web_events = {
 "event_id": "id",
 "user_id": "customer_id",
 "event_type": "event_type",
 "product_id": "product_id",
 "timestamp": "created_at",
 "session_id": "session" # Different extra field
}

Process both variants into unified schema
unified_events =
```

```
json_processor.process_json_batch("s3://lake/raw/user_events/2024/03/15/")
Complex JSON Flattening:
```

Complex JSON Flattening:

```

from pyspark.sql.functions import explode, col, struct

def flatten_complex_json(df: DataFrame, max_depth: int = 3):
 """Flatten nested JSON structures for analytics"""

 flattened_df = df

 for depth in range(max_depth):
 # Get schema of current DataFrame
 schema = flattened_df.schema

 # Find nested struct columns
 struct_columns = [field for field in schema.fields
 if isinstance(field.dataType, StructType)]

```

*The corresponding SQL query:*

```

if not struct_columns:
 break # No more nested structures

Flatten each struct column
select_expressions = []

for field in schema.fields:
 if isinstance(field.dataType, StructType):
 # Flatten struct by selecting all sub-fields
 for sub_field in field.dataType.fields:
 select_expressions.append(
 col(f"{field.name}.

```

```

{sub_field.name}").alias(f"{field.name}_{sub_field.name}")

```

```

)
elif isinstance(field.dataType, ArrayType):
 # Keep arrays as-is for now (could explode

```

separately)

```

select_expressions.append(col(field.name))
 else:
 # Keep primitive columns as-is
 select_expressions.append(col(field.name))

 flattened_df = flattened_df.select(*select_expressions)

 return flattened_df

Example: Flatten e-commerce order JSON
order_json = spark.read.json("s3://lake/raw/orders_json/")

Original nested structure:
{
"order_id": "12345",
"customer": {
"id": "cust_123",
"profile": {
"name": "John Doe",
"email": "john@example.com"
}
},
"items": [
{"product_id": "prod_1", "quantity": 2, "price": 29.99}
]
}

flattened_orders = flatten_complex_json(order_json)

Results in columns:
order_id, customer_id, customer_profile_name,

```

customer\_profile\_email, items

## Unstructured Data Processing

Text Data Processing Pipeline:

PYTHON

```
import re
from textblob import TextBlob

class TextDataProcessor:
 def __init__(self):
 self.cleanup_patterns = [
 (r'http\S+', ''), # Remove URLs
 (r'@\w+', ''), # Remove @mentions
 (r'#\w+', ''), # Remove hashtags
 (r'\d+', 'NUMBER'), # Replace numbers
 (r'^\w\s', '') # Remove punctuation
]

 def process_text_dataset(self, text_df: DataFrame, text_column:
```

str):

```
"""Process large text datasets for analytics"""
```

*The PYTHON code:*

PYTHON

```
Clean text
cleaned_df = text_df
for pattern, replacement in self.cleanup_patterns:
 cleaned_df = cleaned_df.withColumn(
 f"{text_column}_cleaned",
 regexp_replace(col(text_column), pattern,
```

replacement)

PYTHON

```
)

Extract features using UDF
extract_features_udf = udf(self.extract_text_features,
```

```
StructType([
```

```

StructField("word_count", IntegerType()),
 StructField("sentence_count", IntegerType()),
 StructField("sentiment_score", DoubleType()),
 StructField("key_phrases", ArrayType(StringType())))
]))

features_df = cleaned_df.withColumn(
 "text_features",
 extract_features_udf(col(f"{text_column}_cleaned"))
)

Flatten features for analytics
return features_df.select(
 "*",
 col("text_features.word_count").alias("word_count"),
 col("text_features.sentiment_score").alias("sentiment"),
 col("text_features.key_phrases").alias("key_phrases")
)

def extract_text_features(self, text: str) -> dict:
 """Extract features from individual text"""
 if not text:
 return {"word_count": 0, "sentence_count": 0,

```

```

"sentence_count": 0, "key_phrases": []}

```

```

blob = TextBlob(text)

 return {
 "word_count": len(blob.words),
 "sentence_count": len(blob.sentences),
 "sentiment_score": blob.sentiment.polarity,
 "key_phrases": self.extract_key_phrases(text)
 }

Process customer support tickets
support_tickets = spark.read.text("s3://lake/raw/support_tickets/")
text_processor = TextDataProcessor()

processed_tickets =

```

```

text_processor.process_text_dataset(support_tickets, "value")

```

```
Save with partitioning for analytics
processed_tickets.write \
 .partitionBy("year", "month", "day") \
```

```
.parquet("s3://lake/curated/support_tickets_analyzed/")
```

Image Data Metadata Extraction:

Image Data Metadata Extraction:

```
from PIL import Image
import pandas as pd

class ImageProcessor:
 def process_image_batch(self, image_paths: list):
 """Extract metadata from image files for data lake"""

 metadata_records = []

 for image_path in image_paths:
 try:
 with Image.open(image_path) as img:
```

PYTHON

```
 metadata = {
 "file_path": image_path,
 "file_name": os.path.basename(image_path),
 "file_size_mb": os.path.getsize(image_path)
```

```
 / (1024 * 1024),
```

```
 "width": img.width,
 "height": img.height,
 "format": img.format,
 "mode": img.mode,
 "aspect_ratio": img.width / img.height,
 "megapixels": (img.width * img.height) /
```

```
1_000_000,
```

```
 "has_transparency": img.mode in ("RGBA",
```



"LA"),

```
"processing_timestamp": datetime.utcnow()
}

Extract EXIF data if available
if hasattr(img, '_getexif'):
 exif = img._getexif()
 if exif:
 metadata.update({
 "camera_make": exif.get(271,
```

"Unknown"),

```
"camera_model": exif.get(272,
```

"Unknown"),

```
"date_taken": exif.get(306, None)
 })

 metadata_records.append(metadata)

 except Exception as e:
 # Log error but continue processing
 metadata_records.append({
 "file_path": image_path,
 "error": str(e),
 "processing_timestamp": datetime.utcnow()
 })

 return pd.DataFrame(metadata_records)

Process product images
image_processor = ImageProcessor()

Get list of new image files
new_images =
```

```
get_s3_file_list("s3://lake/raw/product_images/2024/03/15/")
```

PYTHON

```
Extract metadata
image_metadata_df = image_processor.process_image_batch(new_images)

Convert to Spark DataFrame and save
spark_metadata = spark.createDataFrame(image_metadata_df)
spark_metadata.write \
 .partitionBy("year", "month", "day") \
 .parquet("s3://lake/curated/product_image_metadata/")
```

## Hybrid Processing Patterns

Combining Structured and Unstructured Data:

```
Example: E-commerce product analysis combining structured sales
```

data

PYTHON

```
with unstructured product reviews
def analyze_product_performance():
 """Combine sales metrics with review sentiment"""

 # Structured sales data
 sales_data =
```

```
spark.read.parquet("s3://lake/curated/product_sales_daily/")
```

```
Unstructured review data (processed)
```

```
review_data =
```

```
spark.read.parquet("s3://lake/curated/product_reviews_analyzed/")
```

PYTHON

```
Aggregate review metrics
review_metrics = review_data.groupBy("product_id") \
 .agg(
```

```
avg("sentiment_score").alias("avg_sentiment"),
```

```
count("*").alias("review_count"),
 avg("rating").alias("avg_rating")
)
```

SQL

```
Combine structured and unstructured insights
product_analysis = sales_data.join(review_metrics, "product_id")
```

\

```
.select(
 "product_id",
 "total_sales",
 "units_sold",
 "avg_sentiment",
 "review_count",
 "avg_rating"
) \
```

SQL

```
.withColumn(
 "performance_score",
 col("total_sales") * 0.4 +
 col("avg_sentiment") * 1000 *
```

```
col("avg_rating") * 200 * 0.3
)

 return product_analysis

Create unified product insights
product_insights = analyze_product_performance()
product_insights.write \
 .mode("overwrite") \
```

```
.parquet("s3://lake/curated/product_performance_analysis/")
```

Real-World Example: Pinterest's Multi-Format Data

Lake

Pinterest handles 20+ billion pins (images + metadata) daily across multiple formats:

Format Strategy:

## ***Real-World Example: Pinterest's Multi-Format Data***

Lake

Pinterest handles 20+ billion pins (images + metadata) daily across multiple formats:

Format Strategy:

```
Pinterest's multi-format processing
data_formats = {
 "user_events": "structured", # Parquet for analytics
 "pin_metadata": "semi_structured", # JSON with schema evolution
 "images": "unstructured", # Binary files with extracted
```

metadata

```

"user_content": "text" # Descriptions, comments
}

Unified processing pipeline
class PinterestDataProcessor:
 def process_daily_batch(self, date: str):
 # Process each format with appropriate strategy

 # Structured: Direct to curated
 user_events = self.process_structured(f"events/{date}")

 # Semi-structured: Schema evolution handling
 pin_metadata =

```

```

self.process_json_with_evolution(f"pins/{date}")

```

```

Unstructured: Extract features
image_features =

```

```

self.extract_image_features(f"images/{date}")

```

```

Text: NLP processing
text_features = self.process_text_content(f"content/{date}")

Combine for unified analytics
unified_dataset = user_events.join(pin_metadata, "pin_id") \
 .join(image_features, "pin_id") \
 .join(text_features, "pin_id")

return unified_dataset

```

Pro Tip Don't try to structure everything immediately. Keep

raw unstructured data in its original format, but extract structured features for analytics. This gives you the flexibility to re-process with better algorithms later.

### □ Checkpoint

1. When would you choose Parquet vs JSON vs raw text storage?
2. How would you handle JSON data with evolving schemas?
3. What metadata would you extract from image files for analytics?
4. How would you combine sentiment analysis with sales data for product insights?

Module 6 Project: Design a Lakehouse

## Architecture for Media Company

---

Objective

**Design a comprehensive lakehouse architecture for a streaming media** company (similar to Netflix or Spotify) that handles multiple data types and supports various use cases.

## Business Context

---

StreamCorp is a video streaming service with:

- 100M subscribers worldwide
- 50,000 hours of video content
- 10B viewing events per day
- 1M new user-generated reviews/ratings per day
- Content includes movies, TV shows, documentaries, user-generated content

Requirements

Data Sources:

- Viewing events (JSON from mobile/web apps)
- Content metadata (MySQL database)
- Video files and thumbnails (binary files)

- User reviews and ratings (text)
- Customer support chats (text)
- Payment transactions (database)

### **Use Cases: - Real-time recommendation engine - Business analytics**

- and reporting
- Content performance analysis
- Fraud detection
- Customer support analytics
- Compliance reporting (GDPR, content licensing)

#### Scale Requirements:

- 10B events/day growing to 50B in 3 years
- 500TB total content storage
- <1 second latency for recommendations
- 99.9% availability for analytics
- 7-year retention for compliance

#### Deliverables

Create a comprehensive design document including:

##### 1. Data Lake Organization (25 points)

Zone-based architecture (Raw/Refined/Curated)

## **Directory structure and naming conventions**

---

Data classification scheme

## **Metadata management strategy**

---

##### 2. Lakehouse Implementation (25 points)

Technology choice (Delta Lake vs Iceberg vs Hudi) with justification

## Schema evolution strategy

---

## ACID transaction patterns

---

## Time travel use cases

---

### 3. Multi-Format Processing (20 points)

Structured data optimization (viewing events, transactions) Semi-structured data handling (JSON from multiple app versions) Unstructured data processing (video metadata, reviews, images)

## Integration patterns between formats

---

### 4. Performance Optimization (15 points)

## Partitioning strategy

---

## File sizing optimization

---

## Caching and materialization patterns

---

## Query optimization techniques

---

### 5. Governance and Security (15 points)



## Access control model

---

## PII detection and handling

---

## Data lineage tracking

---

## Quality gates between zones

---

## GDPR compliance implementation

---

## Design Template

---

```
StreamCorp Lakehouse Architecture Design

Executive Summary
[2-paragraph overview of the solution]

1. Data Lake Organization

Zone Architecture
- **Raw Zone (Bronze)**: [Description, what data, format, retention]
- **Refined Zone (Silver)**: [Description, transformations, quality
```

checks]

```
- **Curated Zone (Gold)**: [Description, business-ready datasets]

Directory Structure
```

```
s3://streamcorp-lakehouse/ └─ raw/ | └─ viewing_events/ | └─ content_metadata/ |
└─ ... └─ refined/ └─ curated/
```

```
Naming Conventions [Specific naming patterns with examples]
```

## ## 2. Lakehouse Technology Implementation

### Technology Choice: [Delta Lake/Iceberg/Hudi] **\*\*Rationale\*\***: [Why this choice over alternatives]

### Schema Evolution Strategy [How you'll handle changing schemas over time]

### ACID Transaction Patterns [Examples of how you'll use ACID transactions]

## ## 3. Multi-Format Data Processing

### Structured Data (Viewing Events, Transactions) [Processing pipeline design]

### Semi-Structured Data (App JSON Events)

[Schema-on-read strategy]

### Unstructured Data (Videos, Reviews, Images) [Feature extraction and processing]

## ## 4. Performance Optimization

### Partitioning Strategy [Table-by-table partitioning decisions]

### Query Optimization [Materialized views, caching, indexing strategies]

## ## 5. Governance and Security

### Access Control [RBAC model and implementation]

### Data Quality [Quality gates and monitoring]

### Compliance [GDPR and data retention implementation]

## ## 6. Implementation Plan

### Phase 1 (Months 1-3): [Foundation] ### Phase 2 (Months 4-6): [Advanced Features] ### Phase 3 (Months 7-12): [Optimization]

## ## 7. Success Metrics

- ### Technical Metrics
  - Query performance (p95 latency < 5 seconds)
  - Data freshness (< 15 minutes for real-time use cases)
  - Availability (99.9% uptime)
- ### Business Metrics

- Cost per TB stored
- Time to insight for new analytics
- Developer productivity metrics

## Evaluation Criteria

---

Technical Accuracy (40%) - Appropriate technology choices -

## Realistic scale estimates - Sound architectural decisions

---

Practical Implementation (30%) - Detailed implementation steps -

**Consideration of real-world constraints - Migration and operational** strategies

- Business Value (20%)
- Clear connection between technical choices and business needs
- Cost-benefit analysis
- Success metrics definition

Communication (10%) - Clear, well-structured documentation -

## Appropriate use of diagrams - Executive summary quality

---

## Bonus Challenges

---

Real-Time ML Pipeline (+5 points) Design real-time feature computation for recommendation engine

Cost Optimization (+5 points) Detailed cost analysis and optimization strategies for 50B events/day

Disaster Recovery (+5 points) Multi-region disaster recovery strategy

Advanced Analytics (+5 points) Design for advanced analytics like content similarity analysis using video features

## Expected Time Investment

---

Research and Planning: 4-6 hours Architecture Design: 6-8 hours Documentation: 3-4 hours Total: 13-18 hours

## Tips for Success

---

- 1. Start with requirements
- understand the business needs before choosing technologies
- 2. Consider growth
- design for 3-year projected scale, not current scale
- 3. Think operationally
- how will this be deployed, monitored, and maintained?
- 4. Show trade-offs
- acknowledge what you're optimizing for and what you're giving up
- 5. Be specific
- avoid generic statements, give concrete examples and numbers

This project simulates the type of lakehouse design you'd do at a major tech company. The complexity and scale require you to apply all the concepts from this module in a realistic scenario.

## Further Reading

---

## Papers and Articles

---

### Lakehouse Architecture: - Lakehouse: A New Generation of Open

---

- Platforms
- Original paper defining lakehouse concept
- Apache Iceberg: The Definitive Guide

- Comprehensive Iceberg documentation
- Delta Lake: High-Performance ACID Table Storage

Data Lake Best Practices:

- Building a Data Lake at Uber
- Real- world data lake implementation
- Netflix's Data Platform
- Scale and architecture insights
- Airbnb's Data Infrastructure
- Data governance and democratization

## **Performance Optimization: - Optimizing Apache Spark for Large**

---

- Scale Workloads
- Parquet vs ORC vs Avro
- Format comparison
- Z-

## **Ordering and Data Clustering**

---

Books

Data Architecture:

- "Designing Data-Intensive Applications" by Martin Kleppmann
- concepts
- "Data Lake Architecture" by John Mertic
- Practical implementation guide
- "Building Analytics Applications" by Sarang Shah
- End-to-end data platform design

Big Data Technologies:

- "Spark: The Definitive Guide" by Bill Chambers
- Deep dive into Spark optimization
- "Learning Spark" by

## **Jules Damji - Modern Spark patterns and best practices**

---

### **Tools and Technologies to Explore**

---

Lakehouse Formats:

- Delta Lake
- Try the open-source version -
- Apache Iceberg
- Multi-engine compatibility
- Apache Hudi -

### **Incremental processing focus**

---

**Metadata and Governance: - Apache Atlas - Open-source data**

- governance
- DataHub
- Modern data catalog
- Amundsen
- Lyft's data discovery platform

Query Engines:

- Trino (formerly Presto)
- Distributed query engine -

## Online Resources

---

### **Blogs and Communities: - Databricks Engineering Blog - Latest**

- lakehouse developments
- Netflix Tech Blog
- Real-world scale challenges
- Data Engineering Weekly
- Industry news and trends

### Training and Certification:

- Databricks Certified Data Engineer Professional
- Snowflake SnowPro Advanced Data Engineer
- AWS Certified Data Analytics
- Specialty

### Conferences:

- Strata Data Conference
- O'Reilly's data conference -

DataEngConf - Community-driven data engineering conference -

Spark + AI Summit - Apache Spark and AI applications

**This module has covered the concepts for building modern** data lakes and lakehouses. The key is to start simple with good organization principles, then add sophisticated features like ACID transactions and advanced performance optimizations as your use cases mature.

**Remember: the best data lake is the one your team actually uses** successfully, not the most technically advanced one. Focus on solving real business problems with appropriate technology choices.

# Module 7: ML Platform & Feature Store Design

## ***What you'll learn***

How to build data infrastructure that powers machine learning at scale. You'll design feature stores that solve the training-serving skew problem, architect model serving systems for real-time and batch inference, and implement ML monitoring that catches problems before they impact users.



07

# ML Platform & Feature Store Design

Design Design ML platforms from data pipeline to model serving. Master feature stores, vector databases, and ML monitoring.

## ⚡ TL;DR

ML pipelines are fundamentally different from analytics pipelines - they need reproducible, point-in-time data - Training and serving have different requirements: batch vs real-time, historical vs current - Feature engineering is the most important part - good features beat fancy algorithms - Versioning everything (data, features, models) is critical for debugging and compliance

Most data engineers think they understand ML data needs. "It's just another analytics use case, right? Train on historical data, predict on new data."

Then they join an ML team and discover their mental models are wrong. ML data pipelines are harder than analytics pipelines because they have additional constraints that traditional ETL doesn't care about.

## ***The ML Data Pipeline Difference***

Traditional Analytics Pipeline:

- Source Data → Clean → Transform → Aggregate → Business Metrics
- Latest data is best data
- Schema changes are manageable
- Occasional errors acceptable
- Human-readable outputs

ML Pipeline:

- Source Data → Clean → Feature Engineering → Training/Serving Split → Models
- Point-in-time data consistency required
- Schema changes break models
- Errors compound through predictions
- Model-readable features

	Table Formats		
	Delta Lake	Apache Iceberg	Apache Hudi
Ecosystem			
Metadata			
Schema			
Time			
Stream			

Figure 17: Side-by-side comparison showing:

- Analytics Pipeline (left side): Data Sources → ETL → Data Warehouse → BI Dashboard
- Arrow labeled "Latest data wins"
- Arrow labeled "Human interpretation"
- Simple linear flow
- ML Pipeline (right side): Data Sources → Feature Engineering → Training Set (historical) / Serving Set (real-time)
- Training Set → Model Training → Model Registry
- Serving Set → Model Serving →

**Predictions - Multiple feedback loops and versioning points - Arrow** labeled "Point-in-time consistency" - Arrow labeled "Reproducible features"]

## Training vs Serving Data Requirements

### Training Data Requirements: - Historical perspective: Features

- as they existed at prediction time

- Large volume: Millions of examples spanning months/years
- Batch processing: Can take hours to generate training sets
- accuracy: Training on wrong data ruins model quality

### **Serving Data Requirements: - Real-time: Features computed from**

- current data
- Low latency: Features ready in <100ms for online inference
- High availability: 99.9%+ uptime for production models -

## **Consistent computation: Exact same logic as training**

**The training-serving skew problem is when these two pipelines** diverge, causing models to fail in production despite good training metrics.

## **Feature Engineering Architecture**

```
Example: Customer churn prediction features
class ChurnFeatureEngineering:
 def __init__(self, spark_session):
 self.spark = spark_session

 def compute_historical_features(self, as_of_date: str,
```

**PYTHON**

```
lookback_days: int = 90):
```

```
 """Compute features as they would have been available at
```

```
as_of_date"""
```

SQL

```
Point-in-time customer data (as it existed at as_of_date)
customers = self.spark.sql(f"""
 SELECT customer_id, signup_date, plan_type, geography
 FROM customers_scd -- Slowly Changing Dimension
 WHERE effective_start_date <= '{as_of_date}'
 AND effective_end_date > '{as_of_date}'
""")

Behavioral features (looking back from as_of_date)
behavior_window_start = (datetime.strptime(as_of_date, '%Y-
```

```
%m-%d') - timedelta(days=lookback_days)).strftime('%Y-%m-%d')
```

PYTHON

```
behavioral_features = self.spark.sql(f"""
 SELECT
 customer_id,
 COUNT(*) as logins_90d,
 SUM(session_duration_minutes) as total_usage_90d,
 AVG(session_duration_minutes) as
```

avg\_session\_duration,

SQL

```
COUNT(DISTINCT DATE(login_time)) as active_days_90d,
 MAX(login_time) as last_login_date,
 COUNT(*) / 90.0 as avg_logins_per_day
FROM user_sessions
WHERE login_time >= '{behavior_window_start}'
 AND login_time < '{as_of_date}'
GROUP BY customer_id
""")

Transaction features (also point-in-time)
```

*The PYTHON code:*

PYTHON

```
transaction_features = self.spark.sql(f"""
 SELECT
 customer_id,
 COUNT(*) as transactions_90d,
 SUM(amount) as total_spent_90d,
 AVG(amount) as avg_transaction_amount,
 COUNT(DISTINCT DATE(transaction_date)) as
```

transaction\_days\_90d

## FROM transactions

SQL

```
WHERE transaction_date >= '{behavior_window_start}'
 AND transaction_date < '{as_of_date}'
 GROUP BY customer_id
 """)

Join all features with proper null handling
feature_set = customers.join(behavioral_features,
```

"customer\_id", "left") \

SQL

```
.join(transaction_features,
```

"customer\_id", "left") \

```
.fillna({
 'logins_90d': 0,
 'total_usage_90d': 0,
 'transactions_90d': 0,
 'total_spent_90d': 0
})

Add derived features
feature_set = feature_set.withColumn(
 "days_since_signup",
 datediff(lit(as_of_date), col("signup_date"))
).withColumn(
 "days_since_last_login",
 datediff(lit(as_of_date), col("last_login_date"))
).withColumn(
 "usage_trend",
 when(col("active_days_90d") > 0,
 col("total_usage_90d") / col("active_days_90d"))
 .otherwise(0)
)

return feature_set

def compute_realtime_features(self, customer_id: str):
 """Compute the exact same features for real-time serving"""
 current_date = datetime.now().strftime('%Y-%m-%d')

 # Use the same logic as historical computation
 features = self.compute_historical_features(current_date)
 return features.filter(col("customer_id") == customer_id)
```

## The Feature Store Pattern:

```
class FeatureStore:
 def __init__(self, offline_store, online_store):
 self.offline_store = offline_store # For training
```

(Snowflake, BigQuery)

```
self.online_store = online_store # For serving (Redis,
```

DynamoDB)

```
self.feature_definitions = {}

def register_feature_view(self, name: str,
```

PYTHON

feature\_computation\_logic):

```
 """Register a feature computation that works for both
```

training and serving"""

```
self.feature_definitions[name] = {
 "computation": feature_computation_logic,
 "registered_at": datetime.utcnow(),
 "schema": feature_computation_logic.output_schema,
 "version":
```

PYTHON

```
self.generate_version_hash(feature_computation_logic)
```

```
}
```

*The get\_historical\_features implementation:*

```
def get_historical_features(self, feature_views: list,
```

PYTHON

entity\_df, as\_of\_timestamp\_column: str):

```
 """Get point-in-time correct features for training"""

 result_dfs = [entity_df]

 for feature_view_name in feature_views:
 feature_view =
```

```
self.feature_definitions[feature_view_name]
```



```
Compute features for each point in time
feature_df =
```

```
feature_view["computation"].compute_historical(
```

```
entity_df.select("entity_id",
```

SQL

```
as_of_timestamp_column).distinct()
```

```
)
```

SQL

```
Join with point-in-time correctness
result_df = entity_df.join(
 feature_df,
 (entity_df.entity_id == feature_df.entity_id) &
 (entity_df[as_of_timestamp_column] ==
```

```
feature_df.as_of_timestamp),
```

```
"left"
)
```

SQL

```
result_dfs.append(result_df)
```

```
Union all features
return reduce(lambda df1, df2: df1.join(df2, ["entity_id",
```

```
as_of_timestamp_column]), result_dfs)
```

```
def get_online_features(self, feature_views: list, entity_ids:
```

PYTHON

```
list):
```

```
"""Get current features for real-time serving"""
```

```
features = {}
```

```
for feature_view_name in feature_views:
 # Try online store first (fast path)
 online_features =
```

```
self.online_store.get_features(feature_view_name, entity_ids)
```

```
if online_features.is_stale() or
```

```
online_features.is_missing():
```

```
Fallback to computing fresh features
feature_view =
```

```
self.feature_definitions[feature_view_name]
```

```
fresh_features =
```

```
feature_view["computation"].compute_realtime(entity_ids)
```

```
Update online store
self.online_store.put_features(feature_view_name,
```

PYTHON

fresh\_features)

```
features[feature_view_name] = fresh_features
else:
 features[feature_view_name] = online_features

return features
```

# Data Lineage for ML

**ML systems need more sophisticated lineage tracking because model performance depends on data quality in ways that business dashboards don't.**

PYTHON

```
class MLLineageTracker:
 def __init__(self):
 self.lineage_store = {}

 def track_training_job(self, model_id: str, training_config:
```

dict):

```
 """Track what data was used to train a model"""

 lineage_record = {
 "model_id": model_id,
 "training_start": datetime.utcnow(),
 "data_sources": {
 "feature_store_version":
```

```
training_config["feature_store_version"],
```

```
 "training_data_range": {
 "start_date": training_config["start_date"],
 "end_date": training_config["end_date"]
 },
 "feature_views": training_config["feature_views"],
 "label_source": training_config["label_source"]
},
 "data_validation": {
 "schema_validation": "passed",
 "data_quality_score": 0.95,
 "missing_value_percentage": 0.02,
 "outlier_percentage": 0.001
 },
 "training_dataset_hash":
```

```
self.compute_dataset_hash(training_config),
```

```

"model_metrics": {} # Will be filled after training
 }

 self.lineage_store[model_id] = lineage_record
 return lineage_record

def track_prediction_job(self, model_id: str, prediction_config:

```

dict):

```

"""Track what data was used for predictions"""

prediction_record = {
 "model_id": model_id,
 "prediction_timestamp": datetime.utcnow(),
 "serving_data": {
 "feature_store_version":

```

```

prediction_config["feature_store_version"],

```

```

"feature_views": prediction_config["feature_views"],
"serving_data_timestamp":

```

```

prediction_config["data_timestamp"]

```

```

},
"feature_consistency_check":

```

```

self.validate_serving_features(model_id, prediction_config)

```

```

}

 return prediction_record

def validate_serving_features(self, model_id: str,

```

serving\_config: dict):

```

"""Validate that serving features match training features"""

training_record = self.lineage_store[model_id]

Check feature view versions match
training_features = set(training_record["data_sources"]

```

["feature\_views"])

```

serving_features = set(serving_config["feature_views"])

if training_features != serving_features:
 return {
 "status": "WARNING",
 "message": f"Feature views differ: training=

```

```

{training_features}, serving={serving_features}"

```

```

}

Check feature store version compatibility
training_version = training_record["data_sources"]

```

["feature\_store\_version"]

```

serving_version = serving_config["feature_store_version"]

```

*The PYTHON code:*

```

if not self.are_versions_compatible(training_version,

```

serving\_version):

```

return {
 "status": "ERROR",
 "message": f"Incompatible feature store versions:

```

```
training={training_version}, serving={serving_version}"
```

```
}
```

```
return {"status": "OK", "message": "Features are
```

```
consistent"}
```

```
def debug_model_performance_drop(self, model_id: str):
 """Help debug when model performance degrades"""

 training_record = self.lineage_store[model_id]
 recent_predictions =
```

PYTHON

```
self.get_recent_prediction_records(model_id, days=7)
```

```
analysis = {
 "training_data_age": (datetime.utcnow() -
```

```
training_record["training_start"]).days,
```

```
"feature_drift": self.detect_feature_drift(model_id),
"schema_changes": self.detect_schema_changes(model_id),
"data_quality_degradation":
```

PYTHON

```
self.detect_quality_issues(model_id)
```

```
}
```

```
recommendations = []
```

```
if analysis["training_data_age"] > 90:
 recommendations.append("Consider retraining - model is
```

```
>90 days old")
```

```
if analysis["feature_drift"]["high_drift_features"]:
 recommendations.append(f"Feature drift detected in:
```

```
{analysis['feature_drift']['high_drift_features']})
```

```
if analysis["schema_changes"]:
 recommendations.append("Schema changes detected - model
```

may be receiving unexpected inputs")

```
return {
 "analysis": analysis,
 "recommendations": recommendations,
 "suggested_actions": [
 "Retrain model with recent data",
 "Investigate feature engineering pipeline",
 "Check data source quality"
]
}
```

## ***Real-World Example: Spotify's ML Data Platform***

Spotify trains thousands of ML models for music recommendations, ad targeting, and content understanding. Their ML data architecture:

Multi-Modal Feature Engineering:

```
Spotify processes multiple data types for recommendations
feature_types = {
 "user_behavioral": {
 "source": "listening_events",
 "features": [
 "songs_played_7d", "skip_rate", "replay_rate",
 "genre_diversity_score", "discovery_vs_familiar_ratio"
],
 "latency": "realtime", # Updated on every play/skip
 "retention": "1 year"
 },
 "content_features": {
 "source": "audio_analysis",
```

The PYTHON code:

```
"features": [
 "tempo", "energy", "valence", "acousticness",
 "audio_fingerprint_vector", "genre_embeddings"
],
"latency": "batch", # Pre-computed for all tracks
"retention": "permanent"
},
"contextual_features": {
 "source": "session_context",
 "features": [
 "time_of_day", "device_type", "location_type",
 "weather", "activity_mode", "social_listening"
],
 "latency": "realtime", # Context changes frequently
 "retention": "30 days"
}
}

Feature computation pipeline
class SpotifyFeatureComputation:
 def compute_user_behavioral_features(self, user_id: str,
```

PYTHON

as\_of\_timestamp: datetime):

```
"""Real-time behavioral features from listening events"""

7-day lookback window
lookback_start = as_of_timestamp - timedelta(days=7)

listening_events = self.get_user_events(user_id,
```

SQL

lookback\_start, as\_of\_timestamp)

```
features = {
 "songs_played_7d": len(listening_events),
 "unique_artists_7d": len(set(event.artist_id for event
```

in listening\_events)),



```
"skip_rate": sum(1 for e in listening_events if
```

```
e.skipped) / len(listening_events),
```

```
"replay_rate": sum(1 for e in listening_events if
```

```
e.replayed) / len(listening_events),
```

```
"avg_completion_rate": np.mean([e.completion_percentage
```

```
for e in listening_events]),
```

```
"genre_diversity_score":
```

```
self.calculate_genre_diversity(listening_events)
```

```
}
```

```
return features
```

Real-Time Feature Serving:

```
class RealtimeFeatureService:
 def __init__(self):
 self.redis_client = redis.Redis()
 self.feature_computation = SpotifyFeatureComputation()

 def get_recommendation_features(self, user_id: str,
```

PYTHON

candidate\_track\_ids: list):

PYTHON

```

"""Get features for real-time recommendation scoring"""

User features (cached in Redis, updated on events)
user_features_key = f"user_features:{user_id}"
user_features = self.redis_client.hgetall(user_features_key)

if not user_features or self.is_stale(user_features):
 # Compute fresh user features
 fresh_features =

```

```

self.feature_computation.compute_user_behavioral_features(

```

PYTHON

```

 user_id, datetime.utcnow()
)
Cache for 5 minutes
self.redis_client.hmset(user_features_key,

```

```

fresh_features)

```

PYTHON

```

self.redis_client.expire(user_features_key, 300)
user_features = fresh_features

```

```

Track features (pre-computed, stored in fast lookup)
track_features = {}
for track_id in candidate_track_ids:
 track_key = f"track_features:{track_id}"
 track_features[track_id] =

```

```

self.redis_client.hgetall(track_key)

```

```
Contextual features (computed fresh each time)
context_features = self.get_context_features(user_id)

return {
 "user": user_features,
 "tracks": track_features,
 "context": context_features
}
```

Pro Tip Design your feature engineering pipeline to work

identically for training and serving from day one. The most expensive ML mistakes come from training-serving skew where features are computed differently in the two environments.

#### □ Checkpoint

1. What makes ML data pipelines different from traditional analytics pipelines?
2. Why is point-in-time correctness important for ML training data?
3. How would you design a feature store that avoids training-serving skew?
4. What data lineage information is critical for debugging ML model performance?

# Feature Store Design Patterns

## ⚡ TL;DR

Feature stores solve the "feature reuse" and "training- serving skew" problems -  
Online store (Redis/DynamoDB) for low-latency serving, offline store (Snowflake/BigQuery) for training - Point-in-time joins are the hardest technical challenge -  
Feature versioning and backward compatibility are critical for production stability

**A feature store is a database optimized for machine learning** features. It's not just storage - it's a complete system for feature computation, storage, serving, discovery, and governance.

Without a feature store, every ML team reinvents feature engineering, creating silos and inconsistencies. With a feature store, features become reusable assets that improve over time.

# Feature Store Architecture Patterns



## Online vs Offline Feature Stores

### Offline Store (Training):

```
class OfflineFeatureStore:
 def __init__(self, warehouse_connection):
 self.warehouse = warehouse_connection

 def get_historical_features(self, entity_df, feature_views,
```

PYTHON

timestamp\_column="event\_timestamp"):

```
"""Get point-in-time correct features for training"""

result_query_parts = [f"SELECT * FROM ({entity_df.to_sql()})
```

SQL

entities"]

```
for feature_view in feature_views:
 # Point-in-time join with feature view
 feature_query = f"""
 LEFT JOIN (
 SELECT
 entity_id,
 {' '.join(feature_view.features)},
 ROW_NUMBER() OVER (
 PARTITION BY entity_id
 ORDER BY created_timestamp DESC
) as rn
 FROM {feature_view.table_name}
 WHERE created_timestamp <= entities.
```

SQL

{timestamp\_column}

```
) {feature_view.name} ON entities.entity_id =
```

```
{feature_view.name}.entity_id
```

```
AND {feature_view.name}.rn = 1
 """
 result_query_parts.append(feature_query)

 # Combine all joins
 full_query = " ".join(result_query_parts)
 return self.warehouse.execute(full_query)

def materialize_incremental_features(self, feature_view,
```

PYTHON

start\_time, end\_time):

```
"""Materialize features for a time range (used for batch
```

```
inference)"""
```

```
Compute features for the time range
feature_computation_query =
```

```
feature_view.feature_computation_logic.format(
```

```
 start_time=start_time,
 end_time=end_time
)

computed_features =
```

```
self.warehouse.execute(feature_computation_query)
```

```
Upsert into feature table
upsert_query = f"""
 MERGE INTO {feature_view.table_name} target
 USING ({computed_features.to_sql()}) source
 ON target.entity_id = source.entity_id
 AND target.created_timestamp =
```

```
source.created_timestamp
```

```
 WHEN MATCHED THEN UPDATE SET *
 WHEN NOT MATCHED THEN INSERT *
 """

self.warehouse.execute(upsert_query)
```

PYTHON

## Online Store (Serving):

```
import redis
import json
from datetime import datetime, timedelta

class OnlineFeatureStore:
 def __init__(self):
```

PYTHON

*The PYTHON code:*

PYTHON

```

self.redis_client = redis.Redis(
 host='feature-store-redis.cluster.local',
 port=6379,
 decode_responses=True
)
self.default_ttl = 3600 # 1 hour

def get_features(self, feature_view_name: str, entity_ids:

```

list):

SQL

```

"""Get latest features for real-time serving"""

if not entity_ids:
 return {}

Batch get from Redis for efficiency
keys = [f"{feature_view_name}:{entity_id}" for entity_id in

```

entity\_ids]

PYTHON

```

values = self.redis_client.mget(keys)

result = {}
for entity_id, value in zip(entity_ids, values):
 if value:
 feature_data = json.loads(value)

 # Check if features are fresh enough
 created_at =

```

```
datetime.fromisoformat(feature_data['_created_at'])
```

```
age_minutes = (datetime.utcnow() -
```

```
created_at).total_seconds() / 60
```

```
if age_minutes < feature_data.get('_ttl_minutes',
```



60):

```
result[entity_id] = feature_data
 else:
 # Mark as stale for refresh
 result[entity_id] = None
 else:
 result[entity_id] = None

 return result

def put_features(self, feature_view_name: str, entity_features:
```

PYTHON

dict, ttl\_seconds: int = None):

```
"""Store features in online store"""

ttl = ttl_seconds or self.default_ttl

pipeline = self.redis_client.pipeline()

for entity_id, features in entity_features.items():
 key = f"{feature_view_name}:{entity_id}"

 # Add metadata
 features_with_metadata = {
 **features,
 '_created_at': datetime.utcnow().isoformat(),
 '_ttl_minutes': ttl // 60,
 '_feature_view': feature_view_name
 }

 pipeline.setex(key, ttl,
```

PYTHON

```
json.dumps(features_with_metadata))
```

```
pipeline.execute()

def refresh_stale_features(self, feature_view_name: str,
```

PYTHON

entity\_ids: list, offline\_store):

```
"""Refresh features that are stale in the online store"""
```

PYTHON

```
Get current features and identify stale ones
current_features = self.get_features(feature_view_name,
```

entity\_ids)

```
stale_entity_ids = [eid for eid, features in
```

```
current_features.items() if features is None]
```

```
if not stale_entity_ids:
 return current_features
```

SQL

```
Get fresh features from offline store
fresh_features =
```

```
offline_store.get_latest_features(feature_view_name, stale_entity_ids)
```

```
Update online store
self.put_features(feature_view_name, fresh_features)

Return combined fresh and current features
result = {**current_features}
result.update(fresh_features)
return {eid: features for eid, features in result.items() if
```

PYTHON

features is not None}

## Point-in-Time Join Implementation

**The hardest technical challenge in feature stores is point-in-time** joins - joining features with their values as they existed at a specific timestamp.

PYTHON

```
class PointInTimeJoiner:
 def __init__(self, spark_session):
 self.spark = spark_session

 def point_in_time_join(self, entity_df, feature_views,
```

timestamp\_column="event\_timestamp"):

SQL

```
"""
Join entities with features as they existed at the entity
```

timestamp

```
Args:
 entity_df: DataFrame with entity_id and event_timestamp
```

columns

```
feature_views: List of feature view configurations
timestamp_column: Column name for the timestamp in
```

entity\_df

PYTHON

```
"""

 result_df = entity_df

 for feature_view in feature_views:
 result_df = self._join_single_feature_view(
 result_df, feature_view, timestamp_column
)

 return result_df

 def _join_single_feature_view(self, entity_df, feature_view,
```

timestamp\_column):

```
"""Join a single feature view with point-in-time
```

SQL

```
semantics"""
```

```
Load feature data
feature_df = self.spark.read.table(feature_view.table_name)

Create window function to get the latest feature value
```

PYTHON

before each entity timestamp

```
window_spec =
```

```
Window.partitionBy("entity_id").orderBy(col("created_timestamp").desc())
```

```
Add row numbers to identify the most recent feature value
feature_df_windowed = feature_df.withColumn("rn",
```

```
row_number().over(window_spec))
```

```
Join condition: entity_id matches and feature timestamp <=
```

SQL

entity timestamp

```
join_condition = [
 entity_df.entity_id == feature_df_windowed.entity_id,
 feature_df_windowed.created_timestamp <=
```

SQL

```
entity_df[timestamp_column],
```

SQL

```

feature_df_windowed.rn == 1
]

 # Perform the join
 joined_df = entity_df.join(
 feature_df_windowed.select(
 col("entity_id").alias("f_entity_id"),
 "created_timestamp",
 *feature_view.feature_columns,
 "rn"
),
 join_condition,
 "left"
).drop("f_entity_id", "created_timestamp", "rn")

 return joined_df

def optimized_point_in_time_join(self, entity_df, feature_views,

```

timestamp\_column="event\_timestamp"):

SQL

```

"""Optimized version using range joins for better

```

performance"""

SQL

```

Sort both DataFrames by timestamp for range join

```

optimization

```

entity_df_sorted = entity_df.sort(timestamp_column)

result_df = entity_df_sorted

for feature_view in feature_views:
 feature_df =

```

self.spark.read.table(feature\_view.table\_name).sort("created\_timestamp")

SQL

```

Use range join for better performance on large

```

## datasets

```
joined_df = entity_df_sorted.join(
 broadcast(feature_df), # Broadcast if feature data
```

SQL

is small

```
[
 entity_df_sorted.entity_id ==
```

```
feature_df.entity_id,
```

```
feature_df.created_timestamp <=
```

```
entity_df_sorted[timestamp_column]
```

```
]
) .select(
 entity_df_sorted["*"],
 *[feature_df[col].alias(f"
```

SQL

```
{feature_view.name}_{col}")
```

```
for col in feature_view.feature_columns]
)

 result_df = joined_df

return result_df
```

SQL

# Feature Versioning and Schema Evolution

```
class FeatureVersion:
 def __init__(self, feature_view_name: str, version: str):
 self.name = feature_view_name
 self.version = version
 self.schema_hash = None
 self.created_at = datetime.utcnow()

class FeatureRegistry:
 def __init__(self):
 self.versions = {} # {feature_view_name: [FeatureVersion]}
```

PYTHON

*The PYTHON code:*

```
self.active_versions = {} # {feature_view_name: version}

def register_feature_version(self, feature_view_name: str,
```

PYTHON

schema: dict, computation\_logic: str):

```
"""Register a new version of a feature view"""

Compute schema hash for version detection
schema_hash = hashlib.sha256(json.dumps(schema,
```

```
sort_keys=True).encode()).hexdigest()
```

```
Check if this schema already exists
existing_versions = self.versions.get(feature_view_name, [])
for existing_version in existing_versions:
 if existing_version.schema_hash == schema_hash:
 return existing_version.version # Schema unchanged,
```

PYTHON

```
return existing version
```

```
Create new version
version_number = f"v{len(existing_versions) + 1}"
new_version = FeatureVersion(feature_view_name,
```

version\_number)

```
new_version.schema_hash = schema_hash
new_version.schema = schema
new_version.computation_logic = computation_logic

Add to registry
if feature_view_name not in self.versions:
 self.versions[feature_view_name] = []
self.versions[feature_view_name].append(new_version)

Set as active version
self.active_versions[feature_view_name] = version_number

return version_number

def get_feature_schema(self, feature_view_name: str, version:
```

PYTHON

str = None):

```
"""Get the schema for a specific feature version"""

if not version:
 version = self.active_versions.get(feature_view_name)

versions = self.versions.get(feature_view_name, [])
for v in versions:
 if v.version == version:
 return v.schema

raise ValueError(f"Version {version} not found for
```

PYTHON

{feature\_view\_name})

```
def validate_backward_compatibility(self, feature_view_name:
```

PYTHON

str, new\_schema: dict):



```

"""Validate that new schema is backward compatible"""

if feature_view_name not in self.versions:
 return True # First version, no compatibility concerns

current_version = self.versions[feature_view_name][-1]
current_schema = current_version.schema

compatibility_issues = []

Check for removed fields
current_fields = set(current_schema.keys())
new_fields = set(new_schema.keys())
removed_fields = current_fields - new_fields

if removed_fields:
 compatibility_issues.append(f"Removed fields:

```

```

{removed_fields}")

```

```

Check for type changes

```

```

for field_name in current_fields.intersection(new_fields):
 current_type = current_schema[field_name].get("type")
 new_type = new_schema[field_name].get("type")

 if current_type != new_type:
 compatibility_issues.append(f"Type change for

```

```

{field_name}: {current_type} -> {new_type}")

```

```

return len(compatibility_issues) == 0, compatibility_issues

def migrate_models_to_version(self, feature_view_name: str,

```

```

target_version: str):

```

```

"""Help migrate models to a new feature version"""

target_schema = self.get_feature_schema(feature_view_name,

```

target\_version)

```
Find all models using this feature view
affected_models =
```

```
self.find_models_using_feature_view(feature_view_name)
```

```
migration_plan = []

for model_id in affected_models:
 model_metadata = self.get_model_metadata(model_id)
 current_feature_version =
```

PYTHON

```
model_metadata.get("feature_versions", {}).get(feature_view_name)
```

```
if current_feature_version != target_version:
 migration_plan.append({
 "model_id": model_id,
 "current_version": current_feature_version,
 "target_version": target_version,
 "schema_changes": self.compare_schemas(
 self.get_feature_schema(feature_view_name,
```

PYTHON

current\_feature\_version),

```
target_schema
)
 })

return migration_plan
```

# Feature Discovery and Governance

PYTHON

```
class FeatureDiscoveryEngine:
 def __init__(self, feature_registry):
 self.registry = feature_registry
 self.usage_tracker = FeatureUsageTracker()

 def search_features(self, query: str, filters: dict = None):
 """Search features by name, description, or tags"""

 results = []

 for feature_view_name, versions in
```

```
self.registry.versions.items():
```

```
 latest_version = versions[-1]

 # Text matching
 if (query.lower() in feature_view_name.lower() or
 query.lower() in
```

```
 latest_version.schema.get("description", "").lower()):
```

PYTHON

```
 # Apply filters
 if self._matches_filters(latest_version, filters):

 # Get usage statistics
 usage_stats =
```

```
self.usage_tracker.get_usage_stats(feature_view_name)
```

```
 results.append({
 "feature_view": feature_view_name,
```

```
 "version": latest_version.version,
 "description":
```

```
latest_version.schema.get("description"),
```

```
"features":
```

```
list(latest_version.schema.keys()),
```

```
"usage_stats": usage_stats,
"quality_score":
```

```
self._calculate_quality_score(feature_view_name),
```

```
"freshness":
```

```
self._get_freshness_info(feature_view_name)
```

```
})
```

PYTHON

```
Sort by relevance and quality
results.sort(key=lambda x: (x["usage_stats"]
```

```
["weekly_users"], x["quality_score"]), reverse=True)
```

```
return results
```

PYTHON

```
def suggest_features_for_use_case(self, use_case_description:
```

```
str, target_entity: str):
```

```
"""Suggest relevant features for a new ML use case"""
```

```
Find features for the same entity type
entity_features =
```

```
self._get_features_for_entity(target_entity)
```

```
Use text similarity to find relevant features
use_case_keywords =
```

```
self._extract_keywords(use_case_description)
```

```
suggestions = []

for feature_view_name in entity_features:
 latest_version =
```

```
self.registry.versions[feature_view_name][-1]
```

```
feature_keywords = self._extract_keywords(
 latest_version.schema.get("description", "") + " " +
 " ".join(latest_version.schema.keys())
)

similarity_score =
```

SQL

```
self._calculate_similarity(use_case_keywords, feature_keywords)
```

```
if similarity_score > 0.3: # Threshold for relevance
 usage_stats =
```

```
self.usage_tracker.get_usage_stats(feature_view_name)
```

```
suggestions.append({
 "feature_view": feature_view_name,
 "similarity_score": similarity_score,
 "usage_popularity": usage_stats["weekly_users"],
 "success_rate":
```

```
usage_stats.get("model_success_rate", 0),
```

```
"description":
```

```
latest_version.schema.get("description")
```

```
)
```

**PYTHON**

```
Sort by combined score of similarity and proven success
suggestions.sort(
 key=lambda x: x["similarity_score"] * 0.6 +
 (x["success_rate"] / 100) * 0.4,
 reverse=True
)
```

```
return suggestions[:10] # Top 10 suggestions
```

```
class FeatureUsageTracker:
```

```
 def __init__(self):
```

```
 self.usage_events = []
```

```
 def track_feature_usage(self, model_id: str, feature_view_name:
```

```
str, usage_type: str):
```

```
 """Track when features are used by models"""
```

*The PYTHON code:*

```
self.usage_events.append({
 "timestamp": datetime.utcnow(),
 "model_id": model_id,
 "feature_view": feature_view_name,
 "usage_type": usage_type # training, serving,
```

**PYTHON**

validation

```

}))

def get_usage_stats(self, feature_view_name: str):
 """Get usage statistics for a feature view"""

 recent_events = [
 event for event in self.usage_events
 if event["feature_view"] == feature_view_name and
 (datetime.utcnow() - event["timestamp"]).days <= 30
]

 unique_models = set(event["model_id"] for event in

```

recent\_events)

```

 return {
 "monthly_usage_count": len(recent_events),
 "monthly_users": len(unique_models),
 "training_usage": len([e for e in recent_events if

```

```

e["usage_type"] == "training"]),

```

```

 "serving_usage": len([e for e in recent_events if

```

```

e["usage_type"] == "serving"])

```

```

 }

```

## ***Real-World Example: Uber's Michelangelo Feature***

Store

Uber's ML platform serves millions of predictions per second across hundreds of models. Their feature store design:

Feature Store Scale:

```

Uber's feature store handles massive scale
uber_feature_store_stats = {
 "feature_views": 10000,
 "total_features": 50000,
 "daily_feature_computations": "10 billion",
 "peak_serving_qps": "1 million",
 "feature_freshness_sla": "< 1 minute for critical features",
 "storage": {
 "online_store": "Cassandra (100TB)",
 "offline_store": "Hive/HDFS (10PB)"
 }
}

Multi-layer caching for performance
class UberFeatureServing:
 def __init__(self):
 self.l1_cache = {} # In-memory (fastest)
 self.l2_cache = CassandraClient() # Distributed (fast)
 self.l3_store = HiveClient() # Batch (complete)

 def get_features(self, feature_requests):
 """Multi-layer feature serving with fallbacks"""

 results = {}

 # Try L1 cache first
 l1_hits, l1_misses = self._try_l1_cache(feature_requests)
 results.update(l1_hits)

 if l1_misses:
 # Try L2 cache
 l2_hits, l2_misses = self._try_l2_cache(l1_misses)
 results.update(l2_hits)

```

*Update L1 cache:*



```
Update L1 cache
 self._update_l1_cache(l2_hits)

 if l2_misses:
 # Compute fresh features
 fresh_features = self._compute_fresh_features(l2_misses)
 results.update(fresh_features)

 # Update both caches
 self._update_l2_cache(fresh_features)
 self._update_l1_cache(fresh_features)

 return results
```

Pro Tip Start your feature store simple - online store for

serving, offline store for training, basic versioning. Add sophisticated features like automated feature discovery only after the basics are working reliably in production.

#### □ Checkpoint

1. What's the difference between online and offline feature stores?
2. Why are point-in-time joins technically challenging to implement?
3. How would you design feature versioning to support model rollbacks?
4. What information would you track to help teams discover relevant features?

## ⚡ TL;DR

Online inference needs <100ms latency, batch inference optimizes for throughput - Model versioning and A/B testing are critical for safe deployments - Real-time feature computation is often the bottleneck, not model execution - Canary deployments and shadow testing reduce model deployment risk

Model serving is where ML meets production systems. Your model might achieve 99% accuracy in training, but if it takes 10 seconds to return a prediction or crashes under load, it's useless.

**Model serving infrastructure must be as reliable as any other** production system - with the added complexity of dealing with evolving models, feature dependencies, and ML-specific monitoring requirements.

## Online vs Batch Inference Architecture

---

Online Inference (Real-Time):

```
Real-time model serving API
from flask import Flask, jsonify, request
import pickle
import redis
import time

class OnlineModelService:
 def __init__(self):
 self.model_cache = {}
 self.feature_store = FeatureStoreClient()
 self.metrics_collector = MetricsCollector()

 def load_model(self, model_id: str, version: str):
 """Load model into memory cache"""
 cache_key = f"{model_id}_{version}"

 if cache_key not in self.model_cache:
 model_path =
```

```
f"s3://models/{model_id}/{version}/model.pkl"
```

```
with open(model_path, 'rb') as f:
 model = pickle.load(f)
 self.model_cache[cache_key] = model

 return self.model_cache[cache_key]

def predict(self, model_id: str, version: str, entity_ids: list,
```

feature\_views: list):

```
"""Serve real-time predictions"""

start_time = time.time()

try:
 # 1. Load model (cached)
 model = self.load_model(model_id, version)

 # 2. Get features (this is often the bottleneck)
 feature_start = time.time()
 features =
```

```
self.feature_store.get_online_features(feature_views, entity_ids)
```

```
feature_latency = time.time() - feature_start
```

PYTHON

```
3. Prepare input data
input_data = self._prepare_model_input(features,
```

entity\_ids)

```
4. Run inference
inference_start = time.time()
predictions = model.predict(input_data)
inference_latency = time.time() - inference_start
```

PYTHON

```
5. Format response
result = self._format_predictions(predictions,
```

entity\_ids)

```
6. Record metrics
total_latency = time.time() - start_time
self.metrics_collector.record_prediction(
 model_id=model_id,
 version=version,
 total_latency=total_latency,
 feature_latency=feature_latency,
 inference_latency=inference_latency,
 num_predictions=len(entity_ids)
)

return {"predictions": result, "model_version": version}

except Exception as e:
 self.metrics_collector.record_error(model_id, version,
```

PYTHON

```
str(e))
```

```

raise

FastAPI version for better performance
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List
import asyncio

class PredictionRequest(BaseModel):
 entity_ids: List[str]
 feature_views: List[str]

class FastAPIModelService:
 def __init__(self):
 self.app = FastAPI()
 self.model_service = OnlineModelService()
 self.setup_routes()

 def setup_routes(self):
 @self.app.post("/predict/{model_id}/{version}")
 async def predict(model_id: str, version: str, request:

```

PredictionRequest):

```

"""Async prediction endpoint for better concurrency"""

try:
 # Run in executor to avoid blocking async loop
 loop = asyncio.get_event_loop()
 result = await loop.run_in_executor(
 None,
 self.model_service.predict,
 model_id, version, request.entity_ids,

```

```

request.feature_views

```

PYTHON

```
)
 return result
except Exception as e:
 raise HTTPException(status_code=500, detail=str(e))

@self.app.get("/models/{model_id}/health")
async def health_check(model_id: str):
 """Health check endpoint"""
 # Quick health check - maybe a simple prediction on
```

dummy data

```
return {"status": "healthy", "timestamp": time.time()}
```

Batch Inference (High Throughput):

PYTHON

```
Batch inference for high-volume prediction jobs
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, udf
from pyspark.sql.types import DoubleType
import mlflow

class BatchInferenceEngine:
 def __init__(self):
 self.spark =
```

```
SparkSession.builder.appName("BatchInference").getOrCreate()
```

PYTHON

```
def run_batch_inference(self, model_uri: str, input_data_path:
```

str,

PYTHON

```
output_path: str, feature_views: list):
 """Run batch inference on large datasets"""

 # 1. Load model
 model = mlflow.pyfunc.spark_udf(spark=self.spark,
```

```
model_uri=model_uri)
```

**PYTHON**

```
2. Load input data (entity IDs and timestamps)
entity_df = self.spark.read.parquet(input_data_path)

3. Get historical features (point-in-time correct)
feature_df = self.feature_store.get_historical_features(
 entity_df=entity_df,
 feature_views=feature_views,
 timestamp_column="prediction_timestamp"
)

4. Apply model to all rows
predictions_df = feature_df.withColumn("prediction",
```

```
model(*feature_views))
```

**SQL**

```
5. Save results
predictions_df.select("entity_id", "prediction_timestamp",
```

```
"prediction") \
```

**PYTHON**

```
.write.mode("overwrite") \
 .partitionBy("date") \
 .parquet(output_path)

def distributed_batch_inference(self, model_uri: str,
```

```
input_data_path: str,
```

```
output_path: str, batch_size: int
```

```
= 100000):
```

```
"""Process very large datasets in chunks"""

Read input data
```

*The PYTHON code:*

PYTHON

```
input_df = self.spark.read.parquet(input_data_path)
total_rows = input_df.count()
num_batches = (total_rows + batch_size - 1) // batch_size

Process in batches
for batch_idx in range(num_batches):
 batch_start = batch_idx * batch_size
 batch_end = min((batch_idx + 1) * batch_size,
```

total\_rows)

```
batch_df = input_df.limit(batch_end -
```

```
batch_start).offset(batch_start)
```

PYTHON

```
Process batch
self.run_batch_inference(
 model_uri=model_uri,
```

```
input_data_path=batch_df.write.mode("overwrite").parquet(f"temp/
batch_{batch_idx}"),
```

PYTHON

```
output_path=f"{output_path}/batch_{batch_idx}/",
 feature_views=feature_views
)
```

```
Combine all batch results
batch_results = []
for batch_idx in range(num_batches):
 batch_result = self.spark.read.parquet(f"
```

```
{output_path}/batch_{batch_idx}/")
```



```

batch_results.append(batch_result)

final_result = batch_results[0]
for batch_result in batch_results[1:]:
 final_result = final_result.union(batch_result)

final_result.write.mode("overwrite").parquet(f"

```

{output\_path}/final/")

## Model Versioning and Deployment Patterns

Model Registry Implementation:

```

import mlflow
from dataclasses import dataclass
from typing import Dict, List
from enum import Enum

class ModelStage(Enum):
 DEVELOPMENT = "development"
 STAGING = "staging"
 PRODUCTION = "production"
 ARCHIVED = "archived"

@dataclass
class ModelVersion:
 name: str
 version: str
 stage: ModelStage
 metrics: Dict[str, float]
 feature_views: List[str]
 model_uri: str
 created_at: datetime
 created_by: str

class ModelRegistry:
 def __init__(self):
 self.mlflow_client = mlflow.tracking.MlflowClient()

 def register_model(self, model_name: str, model_uri: str,
 metrics: dict, feature_views: list,

```

PYTHON

description: str = ""):

```
"""Register a new model version"""
```

*Register in MLflow:*

```
Register in MLflow
model_version = self.mlflow_client.create_model_version(
 name=model_name,
 source=model_uri,
 description=description
)

Add custom metadata
self.mlflow_client.set_model_version_tag(
 name=model_name,
 version=model_version.version,
 key="feature_views",
 value=json.dumps(feature_views)
)

for metric_name, metric_value in metrics.items():
 self.mlflow_client.set_model_version_tag(
 name=model_name,
 version=model_version.version,
 key=f"metric_{metric_name}",
 value=str(metric_value)
)

return model_version

def promote_model(self, model_name: str, version: str,
```

PYTHON

target\_stage: ModelStage):

```
"""Promote a model to a new stage"""
```

```
Validate promotion
if not self._validate_promotion(model_name, version,
```

PYTHON

target\_stage):

```
raise ValueError(f"Model {model_name}:{version} failed
```

promotion validation")

```
Update stage
self.mlflow_client.transition_model_version_stage(
 name=model_name,
 version=version,
 stage=target_stage.value
)

If promoting to production, demote current production
```

PYTHON

model

```
if target_stage == ModelStage.PRODUCTION:
 self._demote_current_production(model_name)

def _validate_promotion(self, model_name: str, version: str,
```

PYTHON

target\_stage: ModelStage) -> bool:

```
"""Validate that model meets requirements for target
```

stage"""

```
model_version =
```

```
self.mlflow_client.get_model_version(model_name, version)
```

```

tags = model_version.tags

if target_stage == ModelStage.PRODUCTION:
 # Production requirements
 required_metrics = ["accuracy", "precision", "recall"]

 for metric in required_metrics:
 metric_key = f"metric_{metric}"
 if metric_key not in tags:
 return False

 metric_value = float(tags[metric_key])
 if metric_value < 0.8: # Minimum threshold
 return False

 # Check that model has been tested in staging
 model_versions =

```

```

self.mlflow_client.get_latest_versions(model_name, stages=
["Staging"])

```

["Staging"])

```

if not model_versions or model_versions[0].version !=

```

version:

```

return False

 return True

A/B Testing and Canary Deployments
class ModelDeploymentManager:
 def __init__(self, model_registry):
 self.registry = model_registry
 self.deployment_configs = {}

 def create_ab_test(self, model_name: str, control_version: str,
 treatment_version: str, traffic_split:

```

PYTHON

float):

```

"""Create A/B test between two model versions"""

ab_test_config = {
 "type": "ab_test",
 "model_name": model_name,
 "control": {
 "version": control_version,
 "traffic_percentage": 1.0 - traffic_split
 },
 "treatment": {
 "version": treatment_version,
 "traffic_percentage": traffic_split
 },
 "created_at": datetime.utcnow(),
 "status": "active"
}

deployment_id = f"

```

```

{model_name}_ab_{datetime.utcnow().strftime('%Y%m%d_%H%M%S')}"

```

```

self.deployment_configs[deployment_id] = ab_test_config

 return deployment_id

def route_prediction_request(self, deployment_id: str,

```

**PYTHON**

entity\_id: str):

```

"""Route prediction request based on deployment

```

configuration"""

```

config = self.deployment_configs[deployment_id]

if config["type"] == "ab_test":
 # Deterministic routing based on entity_id hash
 hash_value = hash(entity_id) % 100
 threshold = config["treatment"]["traffic_percentage"] *

```

**PYTHON**

100

```

if hash_value < threshold:
 return config["treatment"]["version"]
else:
 return config["control"]["version"]

elif config["type"] == "canary":
 # Route small percentage to new version
 hash_value = hash(entity_id) % 100
 canary_percentage = config["canary_percentage"]

 if hash_value < canary_percentage:
 return config["canary_version"]
 else:
 return config["stable_version"]

else:
 return config["default_version"]

def monitor_ab_test_results(self, deployment_id: str):
 """Monitor A/B test performance and recommend actions"""

```

The *PYTHON* code:

```

config = self.deployment_configs[deployment_id]

Collect metrics for both versions
control_metrics = self._collect_version_metrics(
 config["model_name"],
 config["control"]["version"],
 hours=24
)

treatment_metrics = self._collect_version_metrics(
 config["model_name"],
 config["treatment"]["version"],
 hours=24
)

Statistical significance testing
results = {
 "control_performance": control_metrics,
 "treatment_performance": treatment_metrics,
 "significant_difference":

self._test_significance(control_metrics, treatment_metrics),

```

```
"recommendation": "continue" # continue, promote,
```

rollback

```
}

Decision logic
if results["significant_difference"]["p_value"] < 0.05:
 if treatment_metrics["accuracy"] >
```

```
control_metrics["accuracy"]:
```

```
results["recommendation"] = "promote"
 else:
 results["recommendation"] = "rollback"
elif (datetime.utcnow() - config["created_at"]).days > 7:
 results["recommendation"] = "extend_test" # Need more
```

PYTHON

data

```
return results
```

## Real-Time Feature Computation

Often the bottleneck in model serving isn't the model itself, but computing features in real-time.

```
class RealtimeFeatureComputation:
 def __init__(self):
 self.cache = redis.Redis()
 self.feature_processors = {}

 def register_realtime_feature(self, feature_name: str,
```

PYTHON

```
computation_func, cache_ttl: int = 300):
```

```

"""Register a feature that can be computed in real-time"""

 self.feature_processors[feature_name] = {
 "func": computation_func,
 "cache_ttl": cache_ttl
 }

def compute_features(self, entity_id: str, required_features:

```

list):

```

"""Compute features with caching for performance"""

results = {}

for feature_name in required_features:
 cache_key = f"feature:{feature_name}:{entity_id}"

 # Try cache first
 cached_value = self.cache.get(cache_key)
 if cached_value:
 results[feature_name] = json.loads(cached_value)
 continue

```

*Compute fresh:*

```

Compute fresh
if feature_name in self.feature_processors:
 processor = self.feature_processors[feature_name]
 feature_value = processor["func"](entity_id)

 # Cache result
 self.cache.setex(
 cache_key,
 processor["cache_ttl"],
 json.dumps(feature_value)
)

 results[feature_name] = feature_value
else:
 results[feature_name] = None # Feature not

```

available



```

return results

Example: Real-time user behavior features
def compute_user_session_features(user_id: str):
 """Compute real-time session features for recommendation"""

 # Get recent user activity (last 30 minutes)
 recent_activity = get_user_activity(user_id, minutes=30)

 return {
 "sessions_last_30min": len(recent_activity),
 "pages_viewed_last_30min": sum(len(session.pages) for

```

```

session in recent_activity),

```

```

 "avg_session_duration": np.mean([session.duration for

```

```

session in recent_activity]),

```

```

 "current_device_type": recent_activity[-1].device_type if

```

```

recent_activity else "unknown",

```

```

 "is_mobile": recent_activity[-1].device_type == "mobile" if

```

```

recent_activity else False

```

```

}

Register real-time feature computation
feature_computer = RealtimeFeatureComputation()
feature_computer.register_realtime_feature(
 "user_session_features",
 compute_user_session_features,
 cache_ttl=180 # 3 minutes
)

```

## Real-World Example: Netflix's Model Serving

Netflix serves billions of recommendations daily with strict latency requirements. Their serving architecture:

Multi-Layer Model Serving:

```
class NetflixModelServing:
 """Simplified version of Netflix's recommendation serving"""

 def __init__(self):
 self.model_layers = {
 "candidate_generation": CandidateGenerationService(),
 "ranking": RankingService(),
 "personalization": PersonalizationService()
 }

 def get_recommendations(self, user_id: str, device_context:
```

PYTHON

dict):

```
 """Multi-stage recommendation pipeline"""
```

```
 # Stage 1: Generate candidate set (fast, broad)
 candidates =
```

```
 self.model_layers["candidate_generation"].generate_candidates(
```

```
 user_id=user_id,
```

```
 num_candidates=1000,
 timeout_ms=50
)
```

```
 # Stage 2: Rank candidates (more expensive, precise)
 ranked_candidates =
```

```
 self.model_layers["ranking"].rank_candidates(
```

```

user_id=user_id,
 candidates=candidates,
 context=device_context,
 timeout_ms=100
)

Stage 3: Personalize final results
final_recommendations =

```

```

self.model_layers["personalization"].personalize(

```

```

user_id=user_id,
 ranked_items=ranked_candidates[:100],
 context=device_context,
 timeout_ms=50
)

 return final_recommendations[:20] # Return top 20

class CandidateGenerationService:
 """Fast candidate generation using pre-computed similarities"""

 def __init__(self):
 self.similarity_index = load_precomputed_similarities()
 self.user_profiles = UserProfileCache()

 def generate_candidates(self, user_id: str, num_candidates: int,

```

PYTHON

timeout\_ms: int):

```
with timeout(timeout_ms):
 # Get user's viewing history
 user_profile = self.user_profiles.get(user_id)

 # Find similar content based on viewing history
 candidates = set()
 for viewed_item in user_profile.recent_views:
 similar_items = self.similarity_index.get_similar(
 viewed_item,
 top_k=100
)
 candidates.update(similar_items)

 # Remove already watched content
 candidates -= set(user_profile.watched_items)

 return list(candidates)[:num_candidates]
```

Pro Tip Design your model serving for graceful degradation.

If feature computation is slow, serve with cached features. If the main model is down, serve with a simpler fallback model. Never return empty recommendations or error pages to users.

### □ Checkpoint

1. What are the key differences between online and batch inference requirements?
2. How would you implement A/B testing for model deployments?
3. Why is real-time feature computation often the bottleneck in model serving?
4. How would you design a model serving system that handles 100,000 requests per second?

## ⚡ TL;DR

Data drift detection catches when input features change distribution - Model performance drift catches when predictions become less accurate - Feature freshness monitoring ensures features are computed on time - Automated alerting on anomalies prevents silent model degradation

**Traditional application monitoring focuses on system health:** CPU usage, response times, error rates. ML monitoring adds a new dimension: model health. Your system can be running perfectly but your model can be failing silently due to data drift, feature bugs, or changing user behavior.

**ML monitoring is about detecting when models stop working** before users notice.

## Data Drift Detection

Statistical Drift Detection:

```
import numpy as np
from scipy import stats
from typing import Dict, List, Tuple
import pandas as pd

class DataDriftDetector:
 def __init__(self):
 self.baseline_distributions = {}
 self.drift_thresholds = {
 "ks_test": 0.05, # Kolmogorov-Smirnov test p-value
 "chi2_test": 0.05, # Chi-square test p-value
 "psi": 0.1, # Population Stability Index
 "js_divergence": 0.1 # Jensen-Shannon divergence
 }

 def establish_baseline(self, feature_name: str, baseline_data:
```

**PYTHON**

np.array):

```
"""Establish baseline distribution for a feature"""

self.baseline_distributions[feature_name] = {
 "mean": np.mean(baseline_data),
 "std": np.std(baseline_data),
 "quantiles": np.percentile(baseline_data, [10, 25, 50,
```

PYTHON

75, 90]),

```
"histogram": np.histogram(baseline_data, bins=20),
 "sample_size": len(baseline_data)
}

def detect_drift(self, feature_name: str, current_data:
```

PYTHON

np.array) -> Dict:

```
"""Detect if current data has drifted from baseline"""

if feature_name not in self.baseline_distributions:
 return {"error": "No baseline established for this
```

SQL

feature"}

```
baseline = self.baseline_distributions[feature_name]
baseline_hist = baseline["histogram"]

1. Kolmogorov-Smirnov test
Generate sample from baseline distribution (approximation)
baseline_sample = np.random.normal(
 baseline["mean"],
 baseline["std"],
 size=min(1000, baseline["sample_size"])
)
ks_stat, ks_p_value = stats.ks_2samp(baseline_sample,
```

SQL

current\_data)

```
2. Population Stability Index
psi_score = self._calculate_psi(baseline_hist, current_data)

3. Basic statistics comparison
```

PYTHON

The PYTHON code:

```
current_mean = np.mean(current_data)
current_std = np.std(current_data)

drift_detected = (
 ks_p_value < self.drift_thresholds["ks_test"] or
 psi_score > self.drift_thresholds["psi"]
)

return {
 "feature_name": feature_name,
 "drift_detected": drift_detected,
 "metrics": {
 "ks_test": {"statistic": ks_stat, "p_value":
```

PYTHON

ks\_p\_value},

```
"psi_score": psi_score,
"mean_shift": abs(current_mean - baseline["mean"]) /
```

```
baseline["std"],
```

```
"std_ratio": current_std / baseline["std"]
 },
 "recommendations": self._generate_recommendations(
 ks_p_value, psi_score, current_mean, baseline
)
}

def _calculate_psi(self, baseline_hist: Tuple, current_data:
```

PYTHON

np.array) -> float:

```
"""Calculate Population Stability Index"""
```

```
baseline_counts, bin_edges = baseline_hist
baseline_probs = baseline_counts / np.sum(baseline_counts)
```

```
Get current data distribution using same bins
current_counts, _ = np.histogram(current_data,
```

```
bins=bin_edges)
```

```
current_probs = current_counts / np.sum(current_counts)
```

SQL

```
Avoid division by zero
baseline_probs = np.where(baseline_probs == 0, 1e-6,
```

```
baseline_probs)
```

```
current_probs = np.where(current_probs == 0, 1e-6,
```

SQL

```
current_probs)
```

```
PSI calculation
psi = np.sum((current_probs - baseline_probs) *
```

```
np.log(current_probs / baseline_probs))
```

```
return psi
```

PYTHON

```
def monitor_batch_drift(self, features_df: pd.DataFrame,
```

```
baseline_date: str):
```



```
"""Monitor drift across multiple features in a batch"""
```

```
 drift_results = {}
```

```
 for feature_name in features_df.columns:
```

```
 if feature_name in ['entity_id', 'timestamp']:
 continue
```

```
 current_data = features_df[feature_name].dropna().values
```

```
 if len(current_data) > 0:
```

```
 drift_result = self.detect_drift(feature_name,
```

```
current_data)
```

```
 drift_results[feature_name] = drift_result
```

```
 # Overall drift summary
```

```
 features_with_drift = [
```

```
 name for name, result in drift_results.items()
```

```
 if result.get("drift_detected", False)
```

```
]
```

```
 return {
```

```
 "timestamp": datetime.utcnow(),
```

```
 "features_analyzed": len(drift_results),
```

```
 "features_with_drift": len(features_with_drift),
```

```
 "drift_features": features_with_drift,
```

```
 "individual_results": drift_results,
```

```
 "overall_health": "healthy" if len(features_with_drift)
```

```
 == 0 else "degraded"
```

```

}

Real-time drift monitoring
class RealTimeDriftMonitor:
 def __init__(self, window_size: int = 1000):
 self.window_size = window_size
 self.feature_windows = {}
 self.drift_detector = DataDriftDetector()

 def add_sample(self, feature_values: Dict[str, float]):
 """Add a new sample to the monitoring windows"""

 for feature_name, value in feature_values.items():
 if feature_name not in self.feature_windows:
 self.feature_windows[feature_name] = []

 # Add new value and maintain window size
 self.feature_windows[feature_name].append(value)
 if len(self.feature_windows[feature_name]) >

```

self.window\_size:

```

self.feature_windows[feature_name].pop(0)

def check_drift(self) -> Dict:
 """Check for drift in current windows"""

 drift_alerts = []

 for feature_name, window_data in

```

self.feature\_windows.items():

```

if len(window_data) >= self.window_size:
 drift_result = self.drift_detector.detect_drift(
 feature_name, np.array(window_data)
)

 if drift_result.get("drift_detected", False):
 drift_alerts.append({
 "feature": feature_name,
 "severity":

```

```
self._assess_drift_severity(drift_result),
```

```
 "metrics": drift_result["metrics"]
 })

 return {
 "timestamp": datetime.utcnow(),
 "alerts": drift_alerts,
 "monitoring_status": "alert" if drift_alerts else
```

```
"normal"
```

```
}
```

## Model Performance Monitoring

Prediction Quality Tracking:

```
class ModelPerformanceMonitor:
 def __init__(self):
 self.prediction_cache = {}
 self.metrics_history = {}

 def log_prediction(self, model_id: str, entity_id: str,
 prediction: float, features: Dict, timestamp:
```

PYTHON

datetime):

```
 """Log a prediction for later evaluation"""

 prediction_record = {
 "model_id": model_id,
```

```
"entity_id": entity_id,
"prediction": prediction,
"features": features,
"prediction_timestamp": timestamp,
"actual_value": None, # Will be filled when ground
```

truth arrives

```
"evaluation_timestamp": None
}

cache_key = f"{model_id}:{entity_id}:"
```

```
{timestamp.isoformat()}"
```

```
self.prediction_cache[cache_key] = prediction_record

def log_actual_outcome(self, model_id: str, entity_id: str,
 actual_value: float, outcome_timestamp:
```

PYTHON

datetime):

```
"""Log actual outcome to evaluate prediction quality"""

Find the corresponding prediction (within reasonable time
```

window)

```
matching_predictions = []

for cache_key, record in self.prediction_cache.items():
 if (record["model_id"] == model_id and
 record["entity_id"] == entity_id and
 record["actual_value"] is None):

 time_diff = outcome_timestamp -
```

PYTHON

```
record["prediction_timestamp"]
```

```
if timedelta(0) <= time_diff <= timedelta(days=1):
```

# Reasonable window

```
matching_predictions.append((cache_key, record,
```

```
time_diff))
```

```
if matching_predictions:
 # Use the closest prediction in time
 best_match = min(matching_predictions, key=lambda x:
```

PYTHON

```
x[2])
```

```
cache_key, record, _ = best_match

 # Update with actual outcome
 record["actual_value"] = actual_value
 record["evaluation_timestamp"] = outcome_timestamp

 # Calculate error metrics
 error = actual_value - record["prediction"]
 record["absolute_error"] = abs(error)
 record["squared_error"] = error ** 2

 self.prediction_cache[cache_key] = record

def evaluate_model_performance(self, model_id: str,
```

PYTHON

```
time_window_hours: int = 24):
```

```
"""Evaluate model performance over a time window"""

cutoff_time = datetime.utcnow() -
```

```
timedelta(hours=time_window_hours)
```

```
Get evaluated predictions within time window
evaluated_predictions = [
 record for record in self.prediction_cache.values()
 if (record["model_id"] == model_id and
 record["actual_value"] is not None and
 record["prediction_timestamp"] >= cutoff_time)
]

if not evaluated_predictions:
 return {"error": "No evaluated predictions in time
```

window"}

```
Calculate metrics
predictions = [r["prediction"] for r in
```

evaluated\_predictions]

```
actuals = [r["actual_value"] for r in evaluated_predictions]

mae = np.mean([r["absolute_error"] for r in
```

evaluated\_predictions])

```
rmse = np.sqrt(np.mean([r["squared_error"] for r in
```

evaluated\_predictions]))

```

Correlation
correlation = np.corrcoef(predictions, actuals)[0, 1]

Bias (systematic over/under prediction)
bias = np.mean(predictions) - np.mean(actuals)

return {
 "model_id": model_id,
 "evaluation_window": f"{time_window_hours} hours",
 "sample_size": len(evaluated_predictions),
 "metrics": {
 "mae": mae,
 "rmse": rmse,
 "correlation": correlation,
 "bias": bias,
 "prediction_coverage": len(evaluated_predictions) /

```

```

self._count_total_predictions(model_id, time_window_hours)

```

```

},
 "timestamp": datetime.utcnow()
}

def detect_performance_degradation(self, model_id: str):
 """Detect if model performance is degrading over time"""

 # Compare recent performance to baseline
 recent_metrics = self.evaluate_model_performance(model_id,

```

PYTHON

```

time_window_hours=24)

```

```

baseline_metrics = self.get_baseline_metrics(model_id)

if not baseline_metrics:
 return {"warning": "No baseline metrics available"}

Calculate performance changes
mae_change = (recent_metrics["metrics"]["mae"] -

```

PYTHON

```

baseline_metrics["mae"]) / baseline_metrics["mae"]

```

```
correlation_change = baseline_metrics["correlation"] -
```

```
recent_metrics["metrics"]["correlation"]
```

```
Degradation thresholds
significant_degradation = (
 mae_change > 0.15 or # MAE increased by 15%
 correlation_change > 0.1 # Correlation dropped by 0.1
)

return {
 "model_id": model_id,
 "degradation_detected": significant_degradation,
 "performance_changes": {
 "mae_change_percentage": mae_change * 100,
 "correlation_change": correlation_change,
 "sample_size": recent_metrics["sample_size"]
 },
 "recommendation": "retrain_model" if
```

```
significant_degradation else "continue_monitoring"
```

```
}
```

## Feature Freshness and Quality Monitoring

Feature Pipeline Health:

```
class FeatureHealthMonitor:
 def __init__(self):
 self.feature_metrics = {}
 self.freshness_slas = {}
```

PYTHON

*The register\_feature\_sla implementation:*

```
def register_feature_sla(self, feature_name: str,
```

PYTHON

```
max_age_minutes: int):
```



PYTHON

```

"""Register SLA for feature freshness"""
self.freshness_slas[feature_name] = max_age_minutes

def check_feature_freshness(self, feature_name: str,

```

last\_update\_time: datetime):

PYTHON

```

"""Check if feature is fresh according to its SLA"""

if feature_name not in self.freshness_slas:
 return {"warning": "No SLA defined for this feature"}

age_minutes = (datetime.utcnow() -

```

```
last_update_time).total_seconds() / 60
```

PYTHON

```

max_age = self.freshness_slas[feature_name]

is_fresh = age_minutes <= max_age

return {
 "feature_name": feature_name,
 "is_fresh": is_fresh,
 "age_minutes": age_minutes,
 "sla_minutes": max_age,
 "sla_breach_minutes": max(0, age_minutes - max_age),
 "severity":

```

```
self._calculate_freshness_severity(age_minutes, max_age)
```

PYTHON

```

}

def monitor_feature_quality(self, feature_name: str,

```

feature\_values: List[float]):

```

"""Monitor feature quality metrics"""

if not feature_values:
 return {"error": "No feature values provided"}

values_array = np.array(feature_values)

quality_metrics = {
 "null_percentage": np.sum(np.isnan(values_array)) /

```

```

len(values_array),

```

```

 "zero_percentage": np.sum(values_array == 0) /

```

```

len(values_array),

```

```

 "negative_percentage": np.sum(values_array < 0) /

```

```

len(values_array),

```

```

 "outlier_percentage":

```

```

self._calculate_outlier_percentage(values_array),

```

```

 "mean": np.nanmean(values_array),
 "std": np.nanstd(values_array),
 "min": np.nanmin(values_array),
 "max": np.nanmax(values_array)
}

Quality assessment
quality_issues = []

if quality_metrics["null_percentage"] > 0.05: # >5% null
 quality_issues.append("high_null_rate")

if quality_metrics["outlier_percentage"] > 0.1: # >10%

```

## outliers

```
quality_issues.append("high_outlier_rate")

if np.isnan(quality_metrics["mean"]):
 quality_issues.append("invalid_statistics")

return {
 "feature_name": feature_name,
 "sample_size": len(feature_values),
 "quality_metrics": quality_metrics,
 "quality_issues": quality_issues,
 "quality_score":
```

```
self._calculate_quality_score(quality_metrics, quality_issues),
```

```
"timestamp": datetime.utcnow()
}
```

PYTHON

```
def _calculate_outlier_percentage(self, values: np.array) ->
```

## float:

```
"""Calculate percentage of outliers using IQR method"""
```

PYTHON

```
q1 = np.percentile(values, 25)
q3 = np.percentile(values, 75)
iqr = q3 - q1

lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr

outliers = (values < lower_bound) | (values > upper_bound)
return np.sum(outliers) / len(values)
```

```
def comprehensive_health_check(self, feature_store_client):
 """Run comprehensive health check across all monitored
```

## features"""

**PYTHON**

```
health_report = {
 "timestamp": datetime.utcnow(),
 "features_checked": 0,
 "healthy_features": 0,
 "stale_features": [],
 "poor_quality_features": [],
 "overall_health": "healthy"
}

for feature_name in self.freshness_slas.keys():
 # Check freshness
 feature_metadata =
```

```
feature_store_client.get_feature_metadata(feature_name)
```

**PYTHON**

```
freshness_check = self.check_feature_freshness(
 feature_name,
 feature_metadata.last_updated
)

Check quality
recent_values =
```

```
feature_store_client.sample_feature_values(feature_name, limit=1000)
```

```
quality_check =
```

```
self.monitor_feature_quality(feature_name, recent_values)
```

```

health_report["features_checked"] += 1

Assess overall feature health
feature_healthy = (
 freshness_check["is_fresh"] and
 quality_check["quality_score"] > 0.8
)

if feature_healthy:
 health_report["healthy_features"] += 1
else:
 if not freshness_check["is_fresh"]:
 health_report["stale_features"].append({
 "feature": feature_name,
 "age_minutes":

```

```

freshness_check["age_minutes"],

```

```

"sla_breach":

```

```

freshness_check["sla_breach_minutes"]

```

```

})

if quality_check["quality_score"] <= 0.8:
 health_report["poor_quality_features"].append({
 "feature": feature_name,
 "quality_score":

```

```

quality_check["quality_score"],

```

```

"issues": quality_check["quality_issues"]

```

```

})

Overall health assessment
healthy_percentage = health_report["healthy_features"] /

```

```
health_report["features_checked"]
```

```
if healthy_percentage >= 0.95:
 health_report["overall_health"] = "healthy"
elif healthy_percentage >= 0.80:
 health_report["overall_health"] = "degraded"
else:
 health_report["overall_health"] = "critical"

return health_report
```

PYTHON

## Automated Alerting and Response

```
class MAlertingSystem:
 def __init__(self):
 self.alert_channels = {}
 self.alert_history = []
 self.suppression_rules = {}

 def register_alert_channel(self, channel_name: str, webhook_url:
```

PYTHON

```
str, severity_threshold: str = "warning"):
```

```
"""Register alert channel (Slack, PagerDuty, etc.)"""

self.alert_channels[channel_name] = {
 "webhook_url": webhook_url,
 "severity_threshold": severity_threshold,
 "enabled": True
}

def send_alert(self, alert_type: str, severity: str, model_id:
```

PYTHON

```
str,
```

```
message: str, metadata: Dict):
 """Send alert through appropriate channels"""
```

```
 alert_record = {
 "timestamp": datetime.utcnow(),
 "alert_type": alert_type,
 "severity": severity,
 "model_id": model_id,
 "message": message,
 "metadata": metadata,
 "alert_id": f"
```

```
{alert_type}_{model_id}_{int(time.time())}"
```

```
}
```

PYTHON

```
Check suppression rules
if self._is_suppressed(alert_record):
 return {"status": "suppressed", "reason":
```

"duplicate\_alert"}

```
Send to appropriate channels
channels_notified = []

for channel_name, channel_config in
```

```
self.alert_channels.items():
```

```
 if (channel_config["enabled"] and
 self._severity_meets_threshold(severity,
```

PYTHON

```
channel_config["severity_threshold"])):
```

PYTHON

```

self._send_to_channel(channel_name, alert_record)
 channels_notified.append(channel_name)

Log alert
self.alert_history.append(alert_record)

return {
 "status": "sent",
 "alert_id": alert_record["alert_id"],

```

The *PYTHON* code:

PYTHON

```

"channels_notified": channels_notified
 }

def _send_to_channel(self, channel_name: str, alert_record:

```

Dict):

PYTHON

```

"""Send alert to specific channel"""

channel_config = self.alert_channels[channel_name]

Format message for channel
if "slack" in channel_name.lower():
 payload = self._format_slack_message(alert_record)
elif "pagerduty" in channel_name.lower():
 payload = self._format_pagerduty_message(alert_record)
else:
 payload = self._format_generic_message(alert_record)

Send webhook
import requests
response = requests.post(channel_config["webhook_url"],

```

```

json=payload)

```



```
return response.status_code == 200

def create_model_monitoring_workflow(self, model_id: str):
 """Create automated monitoring workflow for a model"""

 workflow = {
 "model_id": model_id,
 "monitoring_tasks": [
 {
 "task": "data_drift_check",
 "frequency": "hourly",
 "alert_on": ["high_drift", "medium_drift"]
 },
 {
 "task": "performance_check",
 "frequency": "daily",
 "alert_on": ["degraded_performance",
```

```
"low_prediction_volume"]
```

```
 },
 {
 "task": "feature_freshness_check",
 "frequency": "every_15_minutes",
 "alert_on": ["stale_features",
```

```
"missing_features"]
```

```

 }

],
 "escalation_policy": {
 "warning": ["slack_data_team"],
 "critical": ["slack_data_team", "pagerduty_oncall"]
 }
}

return workflow

Integration with monitoring systems
class MLObservabilityDashboard:
 def __init__(self):
 self.drift_monitor = DataDriftDetector()
 self.performance_monitor = ModelPerformanceMonitor()
 self.feature_monitor = FeatureHealthMonitor()
 self.alerting = MLAlertingSystem()

 def get_model_health_summary(self, model_id: str):
 """Get comprehensive health summary for a model"""

 # Recent performance
 performance =

```

```
self.performance_monitor.evaluate_model_performance(model_id)
```

```
Feature health
```

```
feature_health =
```

```
self.feature_monitor.comprehensive_health_check(
```

```

feature_store_client=get_feature_store_client()
)

Recent alerts
recent_alerts = [
 alert for alert in self.alerting.alert_history
 if alert["model_id"] == model_id and
 (datetime.utcnow() - alert["timestamp"]).days <= 7
]

return {
 "model_id": model_id,
 "overall_health":

```

```

self._assess_overall_health(performance, feature_health, recent_alerts),

```

```

"performance_metrics": performance,
"feature_health": feature_health,
"recent_alerts": recent_alerts,
"recommendations":

```

```

self._generate_health_recommendations(performance, feature_health,
recent_alerts)

```

```

}

```

## Real-World Example

- Airbnb's ML Monitoring

**Airbnb monitors hundreds of ML models in production with** sophisticated observability:

Multi-Level Monitoring:

```

Airbnb's monitoring levels
monitoring_levels = {
 "infrastructure": {
 "metrics": ["latency", "throughput", "error_rate",

```

"resource\_utilization"],

```
"tools": ["Datadog", "Grafana"],
 "alert_threshold": "immediate"
},
"data_quality": {
 "metrics": ["feature_drift", "data_freshness",
```

"schema\_compliance"],

```
"tools": ["Great Expectations", "custom pipeline"],
 "alert_threshold": "15_minutes"
},
"model_quality": {
 "metrics": ["prediction_accuracy", "bias_detection",
```

"fairness\_metrics"],

```
"tools": ["MLflow", "custom evaluation"],
 "alert_threshold": "1_hour"
},
"business_impact": {
 "metrics": ["booking_conversion", "user_satisfaction",
```

"revenue\_impact"],

```
"tools": ["custom dashboards"],
 "alert_threshold": "daily"
}

Airbnb's alerting strategy
def airbnb_alerting_strategy():
 """Multi-tier alerting based on impact severity"""

 alert_tiers = {
 "P0": {
 "criteria": "Model serving completely down OR critical
```

PYTHON

business metric drop >20%",

```

"response": "Immediate page to on-call engineer",
 "escalation": "Manager after 15 minutes"
},
"P1": {

```

```

"criteria": "Model performance degraded >10% OR

```

significant data drift",

```

"response": "Slack alert to ML team",
 "escalation": "Email to team leads after 2 hours"
},
"P2": {
 "criteria": "Feature freshness SLA breach OR minor

```

performance degradation",

```

"response": "Dashboard notification",
 "escalation": "Daily summary email"
}
}

return alert_tiers

```

Pro Tip Monitor the user experience, not just the model. A

model with 99% accuracy but 5-second latency is worse for business than a 95% accurate model with 100ms latency. Always connect ML metrics to business outcomes.

- ☐ Checkpoint

1. What's the difference between data drift and model performance drift?
2. How would you detect when model predictions are becoming less accurate?
3. Why is feature freshness monitoring important for ML systems?

4. How would you design an alerting system that minimizes false positives?

## 7.5 Vector Database & Embedding Systems

### TL;DR

Vector databases enable similarity search at scale for recommendations and RAG systems - Embedding generation pipelines need to handle both batch and real-time workloads - Vector indexing (FAISS, Annoy, HNSW) trades accuracy for speed - RAG systems combine vector search with traditional databases for context-aware AI

Vector databases are the infrastructure behind modern AI. Every time you get a relevant recommendation on Netflix, ChatGPT finds relevant context for your question, or Google shows you visually similar images, there's a vector database doing similarity search in the background.

Vectors are how machines understand similarity. Text, images, user behavior, products - everything can be converted to vectors where similar items cluster together in multi-dimensional space.

## Vector Embeddings and Similarity Search

Understanding Vector Embeddings:

```
import numpy as np
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import faiss

class EmbeddingGenerator:
 def __init__(self):
 # Load pre-trained models for different data types
 self.text_model = SentenceTransformer('all-MiniLM-L6-v2') #
```

PYTHON

384 dimensions

```
self.image_model = None # Would load ResNet or CLIP model

def generate_text_embeddings(self, texts: List[str]) ->
```

PYTHON

np.ndarray:

```
"""Generate embeddings for text data"""
```

*Clean and preprocess text:*

```
Clean and preprocess text
cleaned_texts = [self._clean_text(text) for text in texts]

Generate embeddings
embeddings = self.text_model.encode(
 cleaned_texts,
 batch_size=32,
 show_progress_bar=True,
 convert_to_numpy=True
)

Normalize embeddings for cosine similarity
embeddings = embeddings / np.linalg.norm(embeddings, axis=1,
```

PYTHON

```
keepdims=True)
```

```
return embeddings

def generate_user_behavior_embeddings(self, user_interactions:
```

PYTHON

```
List[Dict]) -> np.ndarray:
```

```
"""Generate embeddings from user behavior data"""

embeddings = []

for user_data in user_interactions:
 # Create behavior vector from user actions
 behavior_vector = np.zeros(256) # 256-dimensional
```

SQL



## behavior space

```
Encode different types of interactions
if 'purchases' in user_data:
 behavior_vector[0:50] =
```

```
self._encode_purchases(user_data['purchases'])
```

```
if 'views' in user_data:
 behavior_vector[50:100] =
```

```
self._encode_views(user_data['views'])
```

```
if 'searches' in user_data:
 behavior_vector[100:150] =
```

```
self._encode_searches(user_data['searches'])
```

```
if 'demographics' in user_data:
 behavior_vector[150:200] =
```

```
self._encode_demographics(user_data['demographics'])
```

```
Time-based features
behavior_vector[200:256] =
```

```
self._encode_temporal_patterns(user_data)
```

```
Normalize
behavior_vector = behavior_vector /
```

```
np.linalg.norm(behavior_vector)
```

```

embeddings.append(behavior_vector)

 return np.array(embeddings)

def _encode_purchases(self, purchases: List[Dict]) ->

```

np.ndarray:

```

"""Encode purchase history into vector representation"""

Category preferences (50 dimensions)
category_vector = np.zeros(50)

category_mapping = {
 'electronics': 0, 'clothing': 1, 'books': 2, 'home': 3,
 # ... map categories to vector positions
}

for purchase in purchases:
 category = purchase.get('category')
 amount = purchase.get('amount', 0)

```

```

if category in category_mapping:
 idx = category_mapping[category]
 category_vector[idx] += np.log(1 + amount) # Log

```

transform for scaling

```

return category_vector

def find_similar_items(self, query_embedding: np.ndarray,
 item_embeddings: np.ndarray, top_k: int =

```

10) -> List[Tuple[int, float]]:

```

"""Find most similar items to query"""

Calculate cosine similarities
similarities = cosine_similarity(
 query_embedding.reshape(1, -1),
 item_embeddings
)[0]

Get top-k most similar items
top_indices = np.argpartition(similarities, -top_k)[-top_k:]
top_indices =

```

```

top_indices[np.argsort(similarities[top_indices])][::-1]

```

```

results = [(idx, similarities[idx]) for idx in top_indices]
return results

Batch embedding generation pipeline
class BatchEmbeddingPipeline:
 def __init__(self, spark_session):
 self.spark = spark_session
 self.embedding_generator = EmbeddingGenerator()

 def process_text_embeddings(self, input_path: str, output_path:

```

PYTHON

str):

```

"""Process large-scale text embedding generation"""

Read text data
text_df = self.spark.read.parquet(input_path)

Process in batches to avoid memory issues
def generate_embeddings_udf(texts):
 embeddings =

```

PYTHON

```

self.embedding_generator.generate_text_embeddings(texts)

```

```
return embeddings.tolist()
```

**PYTHON**

```
Register UDF
from pyspark.sql.types import ArrayType, FloatType
embeddings_udf = udf(generate_embeddings_udf,
```

```
ArrayType(ArrayType(FloatType()))))
```

```
Apply embedding generation
embeddings_df = text_df.withColumn(
 "embedding",
 embeddings_udf(col("text"))
)

Save embeddings
embeddings_df.write.mode("overwrite").parquet(output_path)

return embeddings_df
```

## Vector Database Implementation

FAISS-Based Vector Store:

```
import faiss
import pickle
from typing import List, Tuple, Optional
import numpy as np

class VectorDatabase:
```

**PYTHON**

*The `__init__` implementation:*

```
def __init__(self, dimension: int, index_type: str = "flat"):
 self.dimension = dimension
 self.index_type = index_type
 self.index = None
 self.metadata_store = {} # Store associated metadata
 self.id_to_index = {} # Map external IDs to internal
```

**PYTHON**

indices

```
self.index_to_id = {} # Map internal indices to external
```

PYTHON

IDs

```
self.next_index = 0
```

```
 self._initialize_index()
```

```
def _initialize_index(self):
 """Initialize FAISS index based on type and dimension"""
```

```
 if self.index_type == "flat":
 # Exact search, good for small datasets
 self.index = faiss.IndexFlatIP(self.dimension) # Inner
```

PYTHON

```
product (cosine similarity)
```

```
elif self.index_type == "ivf":
 # Inverted file index for faster approximate search
 nlist = 100 # Number of clusters
 quantizer = faiss.IndexFlatIP(self.dimension)
 self.index = faiss.IndexIVFFlat(quantizer,
```

PYTHON

```
self.dimension, nlist)
```

```
elif self.index_type == "hnsw":
 # Hierarchical Navigable Small World - good balance of
```

PYTHON

speed/accuracy

```
self.index = faiss.IndexHNSWFlat(self.dimension, 32) #
```

PYTHON

32 connections per node

```
else:
 raise ValueError(f"Unsupported index type:
```

```
{self.index_type}")
```

```
def add_vectors(self, vectors: np.ndarray, ids: List[str],
```

PYTHON

metadata: List[Dict] = None):

```
"""Add vectors to the database"""
```

PYTHON

```
if vectors.shape[1] != self.dimension:
 raise ValueError(f"Vector dimension {vectors.shape[1]}
```

```
doesn't match index dimension {self.dimension}")
```

```
Normalize vectors for cosine similarity
normalized_vectors = vectors / np.linalg.norm(vectors,
```

```
axis=1, keepdims=True)
```

```
Train index if needed (for IVF)
if self.index_type == "ivf" and not self.index.is_trained:
 self.index.train(normalized_vectors)

Add vectors to index
start_idx = self.next_index
self.index.add(normalized_vectors.astype('float32'))

Update ID mappings
for i, external_id in enumerate(ids):
 internal_idx = start_idx + i
 self.id_to_index[external_id] = internal_idx
 self.index_to_id[internal_idx] = external_id

Store metadata if provided
if metadata and i < len(metadata):
 self.metadata_store[external_id] = metadata[i]

self.next_index += len(ids)

def search(self, query_vectors: np.ndarray, k: int = 10,
 return_metadata: bool = True) -> List[List[Dict]]:
```

PYTHON

```
"""Search for similar vectors"""

Normalize query vectors
normalized_queries = query_vectors /
```

```
np.linalg.norm(query_vectors, axis=1, keepdims=True)
```

```
Search
similarities, indices =
```

```
self.index.search(normalized_queries.astype('float32'), k)
```

PYTHON

```
results = []
for query_idx in range(len(normalized_queries)):
 query_results = []

 for rank in range(k):
 internal_idx = indices[query_idx][rank]
 similarity = similarities[query_idx][rank]

 # Skip if no match found
 if internal_idx == -1:
 continue

 external_id = self.index_to_id[internal_idx]

 result = {
 "id": external_id,
 "similarity": float(similarity),
 "rank": rank + 1
 }

 # Add metadata if requested
 if return_metadata and external_id in
```

self.metadata\_store:

```
result["metadata"] =
```

```
self.metadata_store[external_id]
```

```

query_results.append(result)

 results.append(query_results)

 return results

def get_vector_by_id(self, external_id: str) ->

```

Optional[np.ndarray]:

```

"""Retrieve vector by external ID"""

 if external_id not in self.id_to_index:
 return None

 internal_idx = self.id_to_index[external_id]
 return self.index.reconstruct(internal_idx)

def update_metadata(self, external_id: str, metadata: Dict):
 """Update metadata for an item"""
 self.metadata_store[external_id] = metadata

def save(self, index_path: str, metadata_path: str):
 """Save index and metadata to disk"""

 # Save FAISS index
 faiss.write_index(self.index, index_path)

 # Save metadata and mappings
 with open(metadata_path, 'wb') as f:
 pickle.dump({
 'metadata_store': self.metadata_store,
 'id_to_index': self.id_to_index,
 'index_to_id': self.index_to_id,
 'next_index': self.next_index,
 'dimension': self.dimension,
 'index_type': self.index_type
 }, f)

```

*The load implementation:*



PYTHON

```
def load(self, index_path: str, metadata_path: str):
 """Load index and metadata from disk"""

 # Load FAISS index
 self.index = faiss.read_index(index_path)

 # Load metadata and mappings
 with open(metadata_path, 'rb') as f:
 data = pickle.load(f)
 self.metadata_store = data['metadata_store']
 self.id_to_index = data['id_to_index']
 self.index_to_id = data['index_to_id']
 self.next_index = data['next_index']
 self.dimension = data['dimension']
 self.index_type = data['index_type']

 # Production vector database service
 class VectorDatabaseService:
 def __init__(self):
 self.databases = {} # Multiple databases for different use
```

cases

PYTHON

```
def create_database(self, db_name: str, dimension: int,
```

index\_type: str = "hnsw"):

PYTHON

```
"""Create a new vector database"""

self.databases[db_name] = VectorDatabase(dimension,
```

index\_type)

PYTHON

```
return self.databases[db_name]

def batch_update_database(self, db_name: str, embeddings_df):
 """Update database with new embeddings from Spark
```

DataFrame"""

SQL

```

if db_name not in self.databases:
 raise ValueError(f"Database {db_name} not found")

db = self.databases[db_name]

Collect data from Spark DataFrame
data = embeddings_df.select("id", "embedding",

```

```

"metadata").collect()

```

```

vectors = np.array([row["embedding"] for row in data])
ids = [row["id"] for row in data]
metadata = [row["metadata"] for row in data]

Add to vector database
db.add_vectors(vectors, ids, metadata)

return len(data)

```

## Real-Time Embedding Computation

Streaming Embedding Pipeline:

PYTHON

```

from kafka import KafkaConsumer, KafkaProducer
import json

class RealTimeEmbeddingService:
 def __init__(self):
 self.embedding_generator = EmbeddingGenerator()
 self.vector_db_service = VectorDatabaseService()

 # Kafka setup
 self.consumer = KafkaConsumer(
 'new_content_topic',
 bootstrap_servers=['localhost:9092'],
 value_deserializer=lambda x: json.loads(x.decode('utf-

```

```

8'))

```

```
)
```

The *PYTHON* code:

```
self.producer = KafkaProducer(
 bootstrap_servers=['localhost:9092'],
 value_serializer=lambda x: json.dumps(x).encode('utf-8')
)

def process_streaming_embeddings(self):
 """Process embedding generation in real-time"""

 batch_size = 10
 batch = []

 for message in self.consumer:
 content_data = message.value
 batch.append(content_data)

 # Process batch when full
 if len(batch) >= batch_size:
 self._process_batch(batch)
 batch = []

def _process_batch(self, content_batch: List[Dict]):
 """Process a batch of content for embedding generation"""

 try:
 # Extract text content
 texts = [item['text'] for item in content_batch]
 ids = [item['id'] for item in content_batch]

 # Generate embeddings
 embeddings =
```

PYTHON

```
self.embedding_generator.generate_text_embeddings(texts)
```

```
Add to vector database
metadata = [{'title': item.get('title', ''), 'category':
```

```
item.get('category', '')}]
```

```
for item in content_batch]
```

```
db =
```

```
self.vector_db_service.databases.get('content_embeddings')
```

```
if db:
 db.add_vectors(embeddings, ids, metadata)

Publish embeddings to downstream services
for i, embedding in enumerate(embeddings):
 embedding_message = {
 'id': ids[i],
 'embedding': embedding.tolist(),
 'metadata': metadata[i],
 'timestamp': datetime.utcnow().isoformat()
 }

 self.producer.send('embeddings_topic',
```

PYTHON

embedding\_message)

```
except Exception as e:
 print(f"Error processing batch: {e}")
 # Could implement retry logic or dead letter queue here

Caching layer for embeddings
class EmbeddingCache:
 def __init__(self):
 self.redis_client = redis.Redis()
 self.cache_ttl = 3600 # 1 hour

 def get_embedding(self, content_id: str) ->
```

PYTHON

```
Optional[np.ndarray]:
```

```
"""Get cached embedding"""

cached_value = self.redis_client.get(f"embedding:
```

PYTHON

{content\_id}")

```
if cached_value:
```

The *PYTHON* code:

```
return np.frombuffer(cached_value, dtype=np.float32)
 return None

def set_embedding(self, content_id: str, embedding: np.ndarray):
 """Cache embedding"""

 self.redis_client.setex(
 f"embedding:{content_id}",
 self.cache_ttl,
 embedding.astype(np.float32).tobytes()
)

def get_or_compute_embedding(self, content_id: str, text: str) -
```

PYTHON

> np.ndarray:

```
"""Get embedding from cache or compute if not found"""

Try cache first
cached_embedding = self.get_embedding(content_id)
if cached_embedding is not None:
 return cached_embedding

Compute fresh embedding
embedding_generator = EmbeddingGenerator()
embedding =
```

SQL

```
embedding_generator.generate_text_embeddings([text])[0]
```

```
Cache result
self.set_embedding(content_id, embedding)

return embedding
```

PYTHON

## RAG (Retrieval-Augmented Generation) System Design

### RAG System Architecture:

```

class RAGSystem:
 def __init__(self):
 self.vector_db = VectorDatabaseService()
 self.embedding_generator = EmbeddingGenerator()
 self.llm_client = LLMClient() # OpenAI, Anthropic, etc.

 def add_knowledge_documents(self, documents: List[Dict]):
 """Add documents to the RAG knowledge base"""

 # Create knowledge base if it doesn't exist
 if 'knowledge_base' not in self.vector_db.databases:
 self.vector_db.create_database('knowledge_base', 384,

```

'hsw')

```

kb_db = self.vector_db.databases['knowledge_base']

Process documents in chunks for better retrieval
document_chunks = []
chunk_ids = []
chunk_metadata = []

for doc in documents:
 chunks = self._chunk_document(doc['text'])

 for i, chunk in enumerate(chunks):
 chunk_id = f"{doc['id']}_chunk_{i}"
 chunk_ids.append(chunk_id)
 document_chunks.append(chunk)
 chunk_metadata.append({
 'document_id': doc['id'],
 'chunk_index': i,
 'title': doc.get('title', ''),
 'source': doc.get('source', ''),
 'chunk_text': chunk
 })

```

```

Generate embeddings for chunks
chunk_embeddings =

```

```

self.embedding_generator.generate_text_embeddings(document_chunks)

```

```
Add to vector database
kb_db.add_vectors(chunk_embeddings, chunk_ids,
```

chunk\_metadata)

```
return len(document_chunks)

def _chunk_document(self, text: str, chunk_size: int = 500,
```

PYTHON

overlap: int = 50) -> List[str]:

```
"""Split document into overlapping chunks"""

words = text.split()
chunks = []

for i in range(0, len(words), chunk_size - overlap):
 chunk_words = words[i:i + chunk_size]
 chunk = ' '.join(chunk_words)
 chunks.append(chunk)

 if i + chunk_size >= len(words):
 break

return chunks

def retrieve_relevant_context(self, query: str, top_k: int = 5)
```

SQL

-> List[Dict]:

```
"""Retrieve relevant document chunks for a query"""

Generate query embedding
query_embedding =
```

```
self.embedding_generator.generate_text_embeddings([query])[0]
```

```

Search knowledge base
kb_db = self.vector_db.databases['knowledge_base']
search_results = kb_db.search(
 query_embedding.reshape(1, -1),
 k=top_k,
 return_metadata=True
)[0]

Format results
context_chunks = []
for result in search_results:
 context_chunks.append({
 'text': result['metadata']['chunk_text'],
 'source': result['metadata']['source'],
 'title': result['metadata']['title'],
 'similarity': result['similarity'],
 'document_id': result['metadata']['document_id']
 })

return context_chunks

def generate_answer(self, question: str, context_chunks:

```

List[Dict]) -> str:

```

"""Generate answer using retrieved context"""

Combine context chunks
context_text = "\n\n".join([
 f"Source: {chunk['title']}\n{chunk['text']}"
 for chunk in context_chunks
])

Create prompt for LLM
prompt = f"""Answer the following question based on the

```

provided context. If the context doesn't contain enough information to answer the question, say so clearly.

Context:

*The data structure:*



```
{context_text}
```

```
Question: {question}
```

```
Answer: ""
```

```
 # Generate response
```

```
 response = self.llm_client.generate(
 prompt=prompt,
 max_tokens=500,
 temperature=0.3
)
```

```
 return response
```

```
def ask_question(self, question: str) -> Dict:
```

```
 """Complete RAG pipeline: retrieve + generate"""
```

```
 # Retrieve relevant context
```

```
 context_chunks = self.retrieve_relevant_context(question,
```

```
top_k=5)
```

```

Generate answer
answer = self.generate_answer(question, context_chunks)

Return full response with sources
return {
 "question": question,
 "answer": answer,
 "sources": [
 {
 "title": chunk["title"],
 "source": chunk["source"],
 "similarity": chunk["similarity"]
 }
 for chunk in context_chunks
],
 "context_used": len(context_chunks)
}

Example usage
rag_system = RAGSystem()

Add knowledge documents
documents = [
 {
 "id": "doc_1",
 "title": "Machine Learning Best Practices",
 "text": "Machine learning model deployment requires careful

```

consideration of data drift, model monitoring, and A/B testing...",

```

 "source": "internal_docs"
 },
 # ... more documents
]

rag_system.add_knowledge_documents(documents)

Ask questions
response = rag_system.ask_question("How should I monitor ML models

```

in production?")

```
print(response["answer"])
print("Sources:", [source["title"] for source in
```

PYTHON

```
response["sources"]])
```

Real-World Example: Pinterest's Visual Search

Pinterest uses vector embeddings to power visual search across

400+ billion pins:

Pinterest's Vector Architecture:

## Real-World Example: Pinterest's Visual Search

# Pinterest uses vector embeddings to power visual search across

400+ billion pins:

Pinterest's Vector Architecture:

```
Simplified Pinterest visual search architecture
```

*The PinterestVisualSearch implementation:*

```
class PinterestVisualSearch:
 def __init__(self):
 self.image_encoder = CLIPImageEncoder() # CLIP model for
```

PYTHON

images

```
self.text_encoder = CLIPTextEncoder() # CLIP model for
```

PYTHON

text

```
self.pin_vector_db = PinVectorDatabase() # Custom vector DB

def index_pin(self, pin_id: str, image_url: str, description:
```

PYTHON

str, metadata: dict):

```
"""Index a new pin for visual search"""

Generate multi-modal embeddings
image_embedding = self.image_encoder.encode_image(image_url)
text_embedding = self.text_encoder.encode_text(description)

Combine embeddings (weighted average)
combined_embedding = 0.7 * image_embedding + 0.3 *
```

PYTHON

text\_embedding

```
Add to vector database
self.pin_vector_db.add_pin(pin_id, combined_embedding,
```

PYTHON

metadata)

```
def visual_search(self, query_image: str, num_results: int =
```

PYTHON

50):

```
"""Find visually similar pins"""

Encode query image
query_embedding =
```

```
self.image_encoder.encode_image(query_image)
```

```
Search for similar pins
similar_pins = self.pin_vector_db.search(
 query_embedding,
 k=num_results * 2, # Get extra for filtering
 filter_criteria={"status": "active"}
)

Post-process results (diversity, quality filtering)
filtered_results = self._filter_results(similar_pins,
```

PYTHON

num\_results)

```
return filtered_results
```

PYTHON

```
def text_to_visual_search(self, text_query: str, num_results:
```

```
int = 50):
```

```
"""Find pins matching text description"""
```

PYTHON

```
Encode text query
```

```
query_embedding = self.text_encoder.encode_text(text_query)
```

```
Search for matching pins
```

```
matching_pins = self.pin_vector_db.search(query_embedding,
```

```
k=num_results)
```

```
return matching_pins
```

```
Pinterest's scale challenges
```

```
pinterest_stats = {
```

```
 "total_pins": 400_000_000_000,
```

```
 "daily_new_pins": 100_000_000,
```

```
 "vector_dimension": 512,
```

```
 "index_update_frequency": "real-time",
```

```
 "search_latency_p99": "< 100ms",
```

```
 "embedding_computation_throughput": "1M vectors/second"
```

```
}
```

Pro Tip Start with simple vector similarity before building complex multi-modal systems. A well-tuned single-modality vector search often outperforms a poorly implemented multi-

modal system. Focus on embedding quality first, then add complexity.

- □ Checkpoint
1. How do vector embeddings enable similarity search for different data types?
  2. What are the trade-offs between different vector index types (flat, IVF, HNSW)?
  3. How would you design a real-time embedding computation pipeline?
  4. What are the key components of a RAG system and how do they work together?

## Module 7 Project: Design a Complete ML Platform

### Objective

**Design a comprehensive ML platform for a food delivery company that** supports the entire ML lifecycle: data pipelines, feature stores, model serving, and monitoring.

## Business Context

---

**FoodFast is a food delivery service competing with DoorDash and**

Uber Eats with:

- 5M customers across 50 cities
- 100K restaurant partners
- 50K delivery drivers
- 500K orders per day, growing 50% annually
- Need to optimize delivery times, predict demand, prevent fraud, and personalize recommendations

## ML Use Cases

---

- Primary Models: 1. Delivery Time Prediction
- Real-time ETA for customers and driver routing 2. Demand Forecasting
- Predict order volume by restaurant/area/time for driver positioning 3. Fraud Detection
- Real-time transaction and account fraud detection 4.

**Restaurant Recommendations - Personalized restaurant** suggestions for customers  
5. Dynamic Pricing - Surge pricing during high demand periods

Requirements

Scale Requirements:

- 500K orders/day → 2M orders/day in 2 years
- <100ms latency for real-time predictions (delivery time, fraud)
- 99.9% availability for customer-facing features
- Support for 50+ ML models across teams

Data Sources:

- Order transactions (real-time stream)
- Driver GPS locations (real-time stream)
- Restaurant data (batch updates)
- Customer app events (real-time stream)
- Weather and traffic APIs -

## Historical performance data

---

Deliverables

Create a comprehensive ML platform design including:

1. ML Data Pipeline Architecture (25 points)

Data ingestion from multiple sources (batch + streaming)

## **Feature engineering pipelines for each ML use case**

---

### **Point-in-time correctness for training data**

---

### **Data lineage tracking for model debugging**

---

#### 2. Feature Store Design (25 points)

Online/offline store architecture Feature definitions for each use case

### **Real-time feature computation**

---

### **Feature versioning and schema evolution**

---

#### 3. Model Serving Infrastructure (20 points)

### **Online inference for real-time predictions**

---

### **Batch inference for bulk operations**

---

Model versioning and A/B testing

### **Canary deployments and rollbacks**

---

#### 4. ML Monitoring & Observability (15 points)



## Data drift detection

---

## Model performance monitoring

---

## Feature freshness alerts

---

## Business impact tracking

---

5. Vector Database System (15 points)

## Restaurant recommendation embeddings

---

## Customer behavior similarity search

---

## Real-time embedding computation

---

## Scaling for millions of entities

---

## Design Template

---

```
FoodFast ML Platform Design
```

```
Executive Summary
```

```
[3-paragraph overview: business context, technical approach,
```

```
expected outcomes]
```

## ## 1. ML Data Pipeline Architecture

### ### Data Sources & Ingestion

#### \*\*Real-time Streams:\*\*

- Order events: [volume, schema, processing requirements]
- Driver locations: [frequency, accuracy needs]
- Customer interactions: [types, business value]

#### \*\*Batch Sources:\*\*

- Restaurant data: [update frequency, consistency requirements]
- Historical performance: [retention, aggregation needs]

### ### Feature Engineering Pipelines

#### #### Delivery Time Prediction Features

```
```sql
```

```
-- Example feature computation
```

```
SELECT
```

```
    order_id,
    restaurant_id,
    customer_address_hash,
    -- Distance features
    ST_Distance(restaurant_location, customer_location) as
```

distance_km,

```
-- Historical performance
```

```
    AVG(delivery_time_minutes) OVER (
        PARTITION BY restaurant_id, hour_of_day
        ROWS BETWEEN 10 PRECEDING AND 1 PRECEDING
    ) as restaurant_avg_delivery_time,
```

```
-- Real-time context
```

```
    active_drivers_within_5km,
    current_weather_condition,
    traffic_congestion_score
```

```
FROM orders o
```

```
JOIN restaurants r ON o.restaurant_id = r.restaurant_id
```

```
JOIN real_time_context c ON o.order_time = c.timestamp
```

[Continue with other use cases...]

Point-in-Time Correctness Implementation

[How you ensure training data reflects reality at prediction time]

2. Feature Store Design

Architecture Overview

[Online store technology choice, offline store choice, sync strategy]

Feature View Definitions

User Behavioral Features

SQL

```
@feature_view(  
    entities=["customer_id"],  
    ttl=timedelta(hours=1),  
    batch_source=customer_orders_source,  
    stream_source=customer_events_stream  
)  
  
def customer_behavior_features(df):  
    return df.select(  
        col("customer_id"),  
        col("orders_last_30d"),  
        col("avg_order_value"),  
        col("preferred_cuisines"),  
        col("time_of_day_preference")  
    )
```

Real-time Feature Computation

[How you compute features in <100ms for serving]

3. Model Serving Infrastructure

Online Inference Architecture

[API design, caching strategy, latency requirements]

Model Registry & Versioning

[How you manage 50+ models, promote between environments]

A/B Testing Framework

[How you safely test new models, traffic splitting strategy]

4. Monitoring & Observability

Data Drift Detection

[Which features to monitor, alert thresholds, remediation]

Model Performance Tracking

[Metrics per use case, ground truth collection strategy]

Business Impact Monitoring

[KPIs that matter: delivery accuracy, conversion rates, fraud caught]

5. Vector Database System

Restaurant Recommendation Embeddings

[How you generate and serve restaurant similarity vectors]

Customer Embeddings

[Behavioral embeddings for personalization]

Scaling Strategy

[How you handle millions of customers and restaurants]

6. Implementation Roadmap

Phase 1 (Months 1-3): MVP

Basic feature store

Delivery time prediction model

Simple monitoring

Phase 2 (Months 4-6): Scale

Fraud detection

Demand forecasting

Advanced monitoring

Phase 3 (Months 7-12): Optimization

Recommendations Dynamic pricing

Vector similarity systems

7. Success Metrics & ROI

Technical Metrics

Model latency: [targets for each use case] Availability: [uptime requirements]

Feature freshness: [SLAs by feature type]

Business Metrics

Delivery time accuracy improvement: [% improvement target] Fraud detection

rate: [current vs target] Order conversion from recommendations: [% of orders]

Cost Analysis

[Infrastructure costs, development costs, expected savings]

Evaluation Criteria

****Architecture Depth (40%)****

Detailed technical design for each component

Appropriate technology choices with justification

Consideration of scale and performance requirements

Integration between components

****ML Engineering Best Practices (25%)****

Proper handling of training-serving skew

Feature versioning and schema evolution

- Model deployment and monitoring strategies - Data quality and drift detection

****Business Alignment (20%)****

Clear connection between technical design and business value

Realistic success metrics and ROI projections

Consideration of operational constraints
Scalability for business growth

****Implementation Realism (15%)****

Practical implementation roadmap
Risk mitigation strategies
Team structure and skill requirements
Technology migration considerations

Bonus Challenges

****Real-time Feature Serving (+5 points)**** Design sub-100ms feature serving for all real-time use cases

****Multi-city Deployment (+5 points)****

Design for geographic distribution and city-specific models

****Cost Optimization (+5 points)****

Detailed cost analysis and optimization strategies

****Advanced RAG System (+5 points)****

Design knowledge base for internal ML team collaboration

Sample Questions to Consider

1. How would you handle the cold start problem for new restaurants? 2. What happens when GPS data from drivers is delayed or inaccurate? 3. How would you detect when fraud patterns change and models need retraining? 4. What's your strategy for handling seasonal demand patterns (holidays, weather)? 5. How would you ensure feature consistency between cities with different data characteristics?

Expected Time Investment

- ****Architecture Design:**** 8-10 hours
- **Technical Deep Dives:**** 6-8 hours
- **Documentation:**** 4-6 hours
- **Total:**** 18-24 hours

This project requires applying all ML platform concepts in a realistic, complex scenario similar to what you'd encounter at a major tech company building production ML systems.

Further Reading

Papers

****Feature Store Fundamentals:****

[Feature Stores for ML] (<https://www.logicalclocks.com/blog/feature-stores-the-missing-data-layer-in-ml-pipelines>)

Comprehensive overview

[Michelangelo: Uber's ML Platform] (<https://eng.uber.com/michelangelo-machine-learning-platform/>) -

Production ML at scale

- [Zipline: Airbnb's Feature Store] (<https://medium.com/airbnb-engineering/zipline-airbnbs-machine-learning-data-management-platform-7100c31abca2>)

****Model Serving:****

[TFX: TensorFlow Extended] (<https://www.tensorflow.org/tfx>)

End-to-end ML platform

[MLflow Model Registry] (<https://mlflow.org/docs/latest/model-registry.html>)

Model versioning patterns

[Clipper: Low-Latency Online Prediction] (<https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw>)

Berkeley's serving system

****Vector Databases & Embeddings:****

[FAISS: A Library for Efficient Similarity Search] (<https://arxiv.org/abs/1702.08734>)

Facebook's vector search

[Milvus: Purpose-Built Vector Database] (<https://milvus.io/docs>) -

Production vector database

- [CLIP: Connecting Text and Images] (<https://openai.com/research/clip>) -

Multi-modal embeddings

Books

****ML Engineering:****

"Designing Machine Learning Systems" by Chip Huyen

Complete ML systems design

"Building Machine Learning Powered Applications" by Emmanuel

Ameisen - Practical ML development

- "Machine Learning Design Patterns" by Valliappa Lakshmanan -

Common ML architecture patterns

****Vector Search & Information Retrieval:****

"Introduction to Information Retrieval" by Manning, Raghavan & Schütze
Search fundamentals
"Deep Learning for Search" by Tommaso Teofili
Modern search with neural networks

Tools and Frameworks

****Feature Stores:****

[Feast](https://feast.dev/)
Open-source feature store
[Tecton](https://www.tecton.ai/)
Enterprise feature platform
[Hopsworks](https://www.hopsworks.ai/)
ML platform with feature store

****Model Serving:****

[Seldon Core](https://www.seldon.io/)
K8s-native ML deployment
[BentoML](https://bentoml.org/)
Model serving made easy
[Ray Serve](https://docs.ray.io/en/latest/serve/)
Scalable model serving

****Vector Databases:****

[Pinecone](https://www.pinecone.io/)
Managed vector database
[Weaviate](https://weaviate.io/)
Open-source vector search engine
[Chroma](https://www.trychroma.com/)
AI-native open-source embedding database

Online Communities and Resources

****ML Engineering Communities:****

[MLOps Community](https://mlops.community/)
Slack community for ML practitioners
[Weights & Biases Community](https://wandb.ai/community)
ML experiment tracking discussions
[Reddit r/MachineLearning] (https://www.reddit.com/r/MachineLearning/)

Academic and industry ML

****Conferences:****

MLSys Conference

Systems for machine learning

MLOps World

Production ML practices

Vector Search Summit

Vector database and embedding systems

****Blogs and Tutorials:****

[Netflix Tech Blog](https://netflixtechblog.com/)

ML at scale case studies

[Uber Engineering](https://eng.uber.com/)

Production ML systems

[Spotify Engineering](https://engineering.atspotify.com/)

Audio

ML and recommendations

This module has covered the infrastructure for machine learning at scale. The insight is that ****ML systems are data systems first**** - get the data pipelines, feature engineering, and serving infrastructure right, and the ML models will follow. Focus on solving real business problems with appropriate technology choices, not on using the latest ML frameworks.

Module 8: Event-Driven & Microservices Data Architecture

> ****What you'll learn:**** How to design data systems for distributed architectures. You'll master event sourcing, CQRS, and change data capture patterns, learn to build APIs that scale, and understand data mesh principles for large organizations.

💡 In Plain English

Data mesh is an organizational approach where each team owns their own data as a product, rather than one central team managing everything. Think of it like microservices, but for data.

8.1 Data in Microservices Architecture

> ****TL;DR**** >

Database per service prevents coupling but creates data consistency challenges >
Shared databases kill microservice independence but simplify data access >
Eventual consistency is reality
design for it, don't fight it >
Distributed data requires distributed thinking about transactions and queries

Traditional monolithic applications have one database. All components share the same data model, transactions are ACID, and joins are fast. Life is simple. Then you move to microservices and discover that ****data is the hardest part of distributed systems****. Every decision about data architecture becomes a trade-off between service independence, data consistency, and operational complexity.

The Database-Per-Service Pattern

****Pure Microservices Data Architecture:****

User Service → User Database (PostgreSQL) Order Service → Order Database (PostgreSQL) Payment Service → Payment Database (PostgreSQL) Inventory Service → Inventory Database (Redis + PostgreSQL) Notification Service → Message Queue (RabbitMQ) Analytics Service → Data Warehouse (Snowflake)

Microservices Architecture

Service 1
+ DB

Service 2
+ DB

Service 3
+ DB

Service 4
+ DB

Service 5

+ DB

Service 6

+ DB

API Calls & Event Streams

Figure 19: Microservices architecture showing 6 separate services, each with its own database. Services communicate through API cal...

****Benefits:****

****Service Independence:**** Teams can choose optimal database technology

****Scalability:**** Scale databases independently based on service needs

****Failure Isolation:**** Database issues don't cascade across services

****Development Velocity:**** Teams can evolve schemas without coordination

****Challenges:****

****No Cross-Service Joins:**** Can't JOIN orders with users in SQL

****Distributed Transactions:**** No ACID guarantees across services

****Data Duplication:**** Same entity data appears in multiple services

****Eventual Consistency:**** Data changes propagate over time

Database-Per-Service Implementation

```python

```
User Service - Owns user identity and profile data
class UserService:
 def __init__(self):
```

## Point-in-Time Correctness Implementation

---

[How you ensure training data reflects reality at prediction time]

### 2. Feature Store Design

## Architecture Overview

---

[Online store technology choice, offline store choice, sync strategy]

Feature View Definitions

## User Behavioral Features

---

```
@feature_view(
 entities=["customer_id"],
 ttl=timedelta(hours=1),
 batch_source=customer_orders_source,
 stream_source=customer_events_stream
)
def customer_behavior_features(df):
 return df.select(
 col("customer_id"),
 col("orders_last_30d"),
 col("avg_order_value"),
 col("preferred_cuisines"),
 col("time_of_day_preference")
)
```

SQL

## Real-time Feature Computation

---

[How you compute features in <100ms for serving]

### 3. Model Serving Infrastructure

## Online Inference Architecture

---

[API design, caching strategy, latency requirements]

## Model Registry & Versioning

---

[How you manage 50+ models, promote between environments]

A/B Testing Framework

[How you safely test new models, traffic splitting strategy]

4. Monitoring & Observability

## Data Drift Detection

---

[Which features to monitor, alert thresholds, remediation]

## Model Performance Tracking

---

[Metrics per use case, ground truth collection strategy]

## Business Impact Monitoring

---

[KPIs that matter: delivery accuracy, conversion rates, fraud caught]

5. Vector Database System

## Restaurant Recommendation Embeddings

---

[How you generate and serve restaurant similarity vectors]

## Customer Embeddings

---

[Behavioral embeddings for personalization]

## Scaling Strategy

---

[How you handle millions of customers and restaurants]

## 6. Implementation Roadmap

Phase 1 (Months 1-3): MVP

### **Basic feature store**

---

### **Delivery time prediction model**

---

### **Simple monitoring**

---

Phase 2 (Months 4-6): Scale

### **Fraud detection**

---

### **Demand forecasting**

---

### **Advanced monitoring**

---

Phase 3 (Months 7-12): Optimization

Recommendations Dynamic pricing

### **Vector similarity systems**

---

## 7. Success Metrics & ROI

### **Technical Metrics**

---

Model latency: [targets for each use case] Availability: [uptime requirements] Feature freshness: [SLAs by feature type]

## Business Metrics

---

Delivery time accuracy improvement: [% improvement target] Fraud detection rate: [current vs target] Order conversion from recommendations: [% of orders]

## Cost Analysis

---

[Infrastructure costs, development costs, expected savings]

### ### Evaluation Criteria

- **Architecture Depth (40%)**
  - Detailed technical design for each component
  - Appropriate technology choices with justification
  - Consideration of scale and performance requirements
  - Integration between components
- **ML Engineering Best Practices (25%)**
  - Proper handling of training-serving skew
  - Feature versioning and schema evolution
- Model deployment and monitoring strategies - Data quality and drift detection
  - **Business Alignment (20%)**
    - Clear connection between technical design and business value
    - Realistic success metrics and ROI projections
    - Consideration of operational constraints
    - Scalability for business growth
  - **Implementation Realism (15%)**
    - Practical implementation roadmap
    - Risk mitigation strategies
    - Team structure and skill requirements
    - Technology migration considerations

### ### Bonus Challenges

**\*\*Real-time Feature Serving (+5 points)\*\*** Design sub-100ms feature serving for all real-time use cases

**\*\*Multi-city Deployment (+5 points)\*\***

## **Design for geographic distribution and city-specific models**

---

**\*\*Cost Optimization (+5 points)\*\***

## **Detailed cost analysis and optimization strategies**

---

**\*\*Advanced RAG System (+5 points)\*\***

## **Design knowledge base for internal ML team collaboration**

---

### ### Sample Questions to Consider

1. How would you handle the cold start problem for new restaurants? 2. What happens when GPS data from drivers is delayed or inaccurate? 3. How would you detect when fraud patterns change and models need retraining? 4. What's your strategy for handling seasonal demand patterns (holidays, weather)? 5. How would you ensure feature consistency between cities with different data characteristics?

### ### Expected Time Investment

- - **\*\*Architecture Design:\*\*** 8-10 hours
- **\*\*Technical Deep Dives:\*\*** 6-8 hours
- **\*\*Documentation:\*\*** 4-6 hours
- **\*\*Total:\*\*** 18-24 hours

**This project requires applying all ML platform concepts in a** realistic, complex scenario similar to what you'd encounter at a major tech company building production ML systems.

---

## ## Further Reading

### ### Papers

- **Feature Store Fundamentals:**
- [Feature Stores for ML] (<https://www.logicalclocks.com/blog/feature-stores-the-missing-data-layer-in-ml-pipelines>)
- Comprehensive overview
- [Michelangelo: Uber's ML Platform] (<https://eng.uber.com/michelangelo-machine-learning-platform/>) -

## Production ML at scale

---

- [Zipline: Airbnb's Feature Store](<https://medium.com/airbnb-engineering/zipline-airbnbs-machine-learning-data-management-platform-7100c31abca2>)

- **Model Serving:**
- [TFX: TensorFlow Extended](<https://www.tensorflow.org/tfx>)
- End-to-end ML platform
- [MLflow Model Registry](<https://mlflow.org/docs/latest/model-registry.html>)
- Model versioning patterns
- [Clipper: Low-Latency Online Prediction] (<https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw>)
- Berkeley's serving system
- **Vector Databases & Embeddings:**
- [FAISS: A Library for Efficient Similarity Search] (<https://arxiv.org/abs/1702.08734>)
- Facebook's vector search
- [Milvus: Purpose-Built Vector Database](<https://milvus.io/docs>) -



## Production vector database

---

- [CLIP: Connecting Text and Images] (<https://openai.com/research/clip>) - Multi-modal embeddings

### ### Books

- **ML Engineering:**
- "Designing Machine Learning Systems" by Chip Huyen
- Complete ML systems design
- "Building Machine Learning Powered Applications" by Emmanuel

## Ameisen - Practical ML development

---

- "Machine Learning Design Patterns" by Valliappa Lakshmanan -

## Common ML architecture patterns

---

- **Vector Search & Information Retrieval:**
- "Introduction to Information Retrieval" by Manning, Raghavan & Schütze
- Search fundamentals
- "Deep Learning for Search" by Tommaso Teofili
- Modern search with neural networks

### ### Tools and Frameworks

- **Feature Stores:**
- [Feast](<https://feast.dev/>)
- Open-source feature store
- [Tecton](<https://www.tecton.ai/>)
- Enterprise feature platform
- [Hopsworx](<https://www.hopsworx.ai/>)
- ML platform with feature store
- **Model Serving:**

- [Seldon Core](https://www.seldon.io/)
- K8s-native ML deployment
- [BentoML](https://bentoml.org/)
- Model serving made easy
- [Ray Serve](https://docs.ray.io/en/latest/serve/)
- Scalable model serving
- **Vector Databases:**
- [Pinecone](https://www.pinecone.io/)
- Managed vector database
- [Weaviate](https://weaviate.io/)
- Open-source vector search engine
- [Chroma](https://www.trychroma.com/)
- AI-native open-source embedding database

### ### Online Communities and Resources

- **ML Engineering Communities:**
- [MLOps Community](https://mlops.community/)
- Slack community for ML practitioners
- [Weights & Biases Community](https://wandb.ai/community)
- ML experiment tracking discussions
- [Reddit r/MachineLearning] (https://www.reddit.com/r/MachineLearning/)
- Academic and industry ML
- **Conferences:**
- MLSys Conference
- Systems for machine learning
- MLOps World
- Production ML practices
- Vector Search Summit

- Vector database and embedding systems
- **Blogs and Tutorials:**
- [Netflix Tech Blog](https://netflixtechblog.com/)
- ML at scale case studies
- [Uber Engineering](https://eng.uber.com/)
- Production ML systems
- [Spotify Engineering](https://engineering.atspotify.com/)
- Audio

## ML and recommendations

---

**This module has covered the infrastructure for machine learning at scale.** The insight is that **ML systems are data systems first** - get the data pipelines, feature engineering, and serving infrastructure right, and the ML models will follow. Focus on solving real business problems with appropriate technology choices, not on using the latest ML frameworks.

# Module 8: Event-

## Driven & Microservices Data Architecture

---

> **What you'll learn:** How to design data systems for distributed architectures. You'll master event sourcing, CQRS, and change data capture patterns, learn to build APIs that scale, and understand data mesh principles for large organizations.

### In Plain English

**Data mesh** is an organizational approach where each team owns their own data as a product, rather than one central team managing everything. Think of it like microservices, but for data.

---

### ## 8.1 Data in Microservices Architecture

- > **TL;DR** >
- Database per service prevents coupling but creates data consistency challenges >

- Shared databases kill microservice independence but simplify data access >
- Eventual consistency is reality
- design for it, don't fight it >
- Distributed data requires distributed thinking about transactions and queries

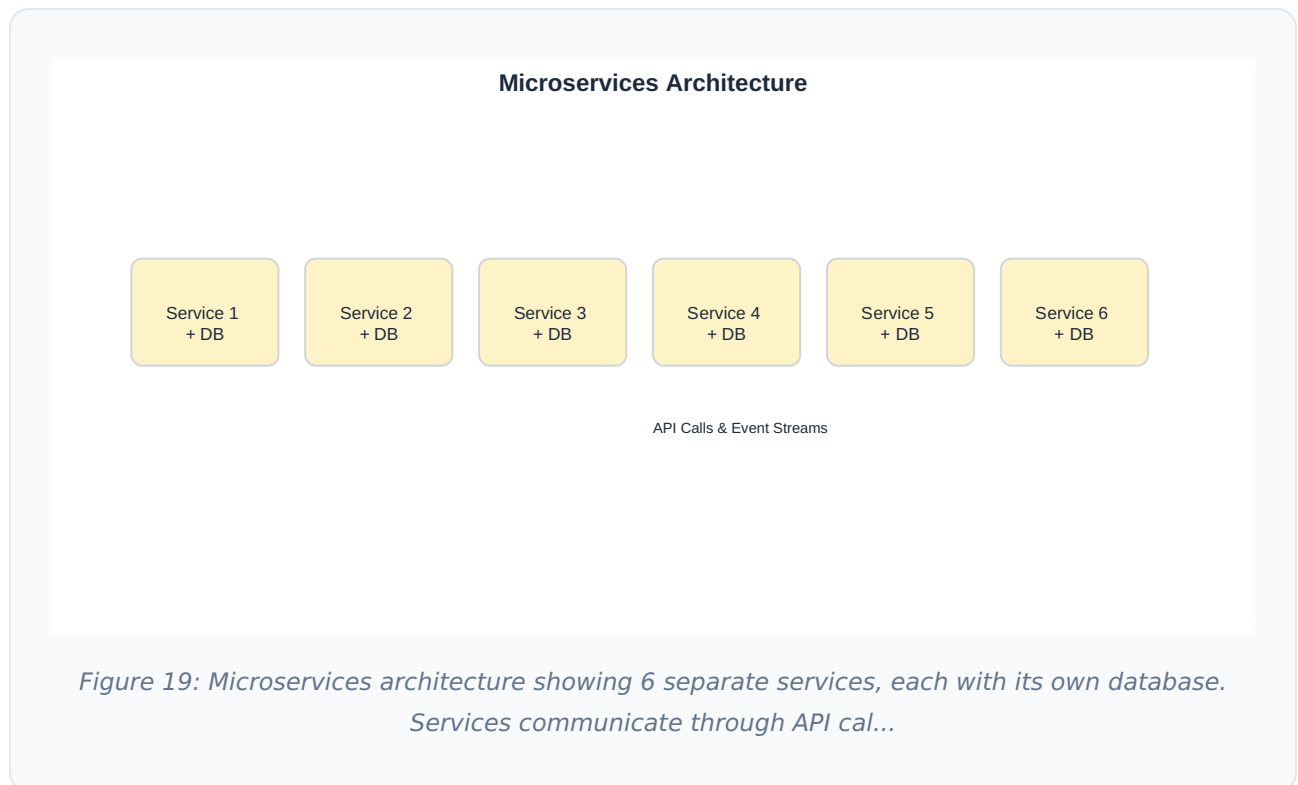
Traditional monolithic applications have one database. All components share the same data model, transactions are ACID, and joins are fast. Life is simple.

Then you move to microservices and discover that **\*\*data is the hardest part of distributed systems\*\***. Every decision about data architecture becomes a trade-off between service independence, data consistency, and operational complexity.

### ### The Database-Per-Service Pattern

**\*\*Pure Microservices Data Architecture:\*\***

User Service → User Database (PostgreSQL) Order Service → Order Database (PostgreSQL) Payment Service → Payment Database (PostgreSQL) Inventory Service → Inventory Database (Redis + PostgreSQL) Notification Service → Message Queue (RabbitMQ) Analytics Service → Data Warehouse (Snowflake)



- **Benefits:**
- **Service Independence:** Teams can choose optimal database technology
- **Scalability:** Scale databases independently based on service needs
- **Failure Isolation:** Database issues don't cascade across services
- **Development Velocity:** Teams can evolve schemas without coordination
- **Challenges:**
- **No Cross-Service Joins:** Can't JOIN orders with users in SQL
- **Distributed Transactions:** No ACID guarantees across services
- **Data Duplication:** Same entity data appears in multiple services
- **Eventual Consistency:** Data changes propagate over time

### Database-Per-Service Implementation

```
```python
```

```
# User Service - Owns user identity and profile data class UserService: def __init__(self):
```

```
self.db = PostgreSQLConnection("users_db")
self.event_publisher = EventPublisher()
```

PYTHON

def create_user(self, user_data):

```
    """Create new user and publish event"""

    # Store in local database
    user = User(
        user_id=generate_uuid(),
        email=user_data['email'],
        name=user_data['name'],
        created_at=datetime.utcnow()
    )

    self.db.save(user)

    # Publish event for other services
    self.event_publisher.publish('user.created', {
        'user_id': user.user_id,
        'email': user.email,
        'name': user.name,
        'created_at': user.created_at.isoformat()
    })

    return user
```

PYTHON

def update_user_profile(self, user_id, updates):

PYTHON

```

"""Update user profile and notify other services"""

user = self.db.get_user(user_id)
if not user:
    raise UserNotFound(user_id)

# Update local data
for field, value in updates.items():
    if hasattr(user, field):
        setattr(user, field, value)

user.updated_at = datetime.utcnow()
self.db.save(user)

# Publish change event
self.event_publisher.publish('user.updated', {
    'user_id': user_id,
    'changes': updates,
    'updated_at': user.updated_at.isoformat()
})

```

Order Service - Owns order data but needs user information class OrderService: def __init__(self):

PYTHON

```

self.db = PostgreSQLConnection("orders_db")
self.event_publisher = EventPublisher()
self.event_subscriber = EventSubscriber()
self.user_cache = UserCache() # Local cache of user data

# Subscribe to user events
self.event_subscriber.subscribe('user.created',

```

```

self.on_user_created)

```

PYTHON

```

self.event_subscriber.subscribe('user.updated',

```

self.on_user_updated) def create_order(self, order_data):

SQL

```

"""Create order with user information"""

user_id = order_data['user_id']

# Get user info from local cache (populated by events)
user_info = self.user_cache.get(user_id)

```

```

if not user_info:
    raise UserInfoNotAvailable(user_id)

order = Order(
    order_id=generate_uuid(),
    user_id=user_id,
    user_email=user_info['email'], # Denormalized for

```

efficiency

```

user_name=user_info['name'],      # Denormalized for

```

efficiency

PYTHON

```

items=order_data['items'],
    total_amount=order_data['total_amount'],
    created_at=datetime.utcnow()
)

self.db.save(order)

self.event_publisher.publish('order.created', {
    'order_id': order.order_id,
    'user_id': order.user_id,
    'total_amount': order.total_amount,
    'created_at': order.created_at.isoformat()
})

return order

```

def on_user_created(self, event_data):

PYTHON

```

"""Handle user creation - add to local cache"""
self.user_cache.set(event_data['user_id'], {
    'email': event_data['email'],
    'name': event_data['name']
})

```

def on_user_updated(self, event_data):

PYTHON

```

"""Handle user updates - update local cache"""
user_id = event_data['user_id']
changes = event_data['changes']

current_info = self.user_cache.get(user_id)
if current_info:
    current_info.update(changes)
    self.user_cache.set(user_id, current_info)

```

Shared cache implementation for eventual consistency class UserCache: def __init__(self):

PYTHON

```

self.redis_client = redis.Redis()
self.ttl = 3600 # 1 hour cache

```

def get(self, user_id):

SQL

```

"""Get user info from cache"""
cached_data = self.redis_client.get(f"user:{user_id}")
if cached_data:
    return json.loads(cached_data)
return None

```

def set(self, user_id, user_info):

PYTHON

```

"""Cache user info with TTL"""
self.redis_client.setex(
    f"user:{user_id}",
    self.ttl,
    json.dumps(user_info)
)

```

Handling Cross-Service Queries

When you need data from multiple services, you have several patterns:

1. API Composition Pattern:

```
class OrderSummaryService:
```

PYTHON

Compose data from multiple services for complex queries:

```
"""Compose data from multiple services for complex queries"""
```

PYTHON

```
def __init__(self):
    self.user_service_client = UserServiceClient()
    self.order_service_client = OrderServiceClient()
    self.payment_service_client = PaymentServiceClient()

    async def get_user_order_summary(self, user_id, date_range):
        """Get comprehensive order summary requiring multiple
```

```
services"""
```

```
    # Parallel calls to multiple services
    user_task = asyncio.create_task(
        self.user_service_client.get_user(user_id)
    )
    orders_task = asyncio.create_task(
        self.order_service_client.get_orders_by_user(user_id,
```

PYTHON

```
date_range)
```

```
    )

    # Wait for basic data
    user, orders = await asyncio.gather(user_task, orders_task)

    if not orders:
        return {"user": user, "summary": {"order_count": 0,
```

```
"total_spent": 0}}
```

```
# Get payment details for orders
order_ids = [order['order_id'] for order in orders]
payments_task = asyncio.create_task(
```

```
self.payment_service_client.get_payments_for_orders(order_ids)
```

```
)

payments = await payments_task

# Compose final response
return {
    "user": user,
    "summary": {
        "order_count": len(orders),
        "total_spent": sum(payment['amount'] for payment in
```

```
payments),
```

```
"avg_order_value": sum(payment['amount'] for payment
```

```
in payments) / len(orders),
```

```
"date_range": date_range
    },
    "recent_orders": orders[:5] # Last 5 orders
}
```

2. Data Denormalization Pattern:

```

class OrderWithUserData:
    """Order service stores denormalized user data for efficiency"""

    def __init__(self):
        self.db = PostgreSQLConnection("orders_db")

    def get_orders_with_user_info(self, filters):
        """Get orders with user info without cross-service calls"""

        # This works because we denormalized user data in orders
        query = """
            SELECT
                o.order_id,
                o.user_id,
                o.user_email,
                o.user_name,
                o.total_amount,
                o.created_at,
                o.status
            FROM orders o
            WHERE o.created_at >= %s

```

The corresponding SQL query:

```

        AND o.created_at < %s
        AND o.status = %s
        ORDER BY o.created_at DESC
    """

    return self.db.execute(query, [
        filters['start_date'],
        filters['end_date'],
        filters['status']
    ])

```

3. CQRS with Read Models Pattern:

PYTHON

```
class OrderUserReadModel:
    """Dedicated read model combining order and user data"""

    def __init__(self):
        self.read_db = PostgreSQLConnection("read_models_db")
        self.event_subscriber = EventSubscriber()

        # Subscribe to events from multiple services
        self.event_subscriber.subscribe('order.created',
```

```
self.on_order_created)
```

PYTHON

```
self.event_subscriber.subscribe('user.updated',
```

```
self.on_user_updated)
```

```

def on_order_created(self, event_data):
    """Build read model when order is created"""

    order_user_record = {
        'order_id': event_data['order_id'],
        'user_id': event_data['user_id'],
        'user_email': event_data.get('user_email'),
        'user_name': event_data.get('user_name'),
        'order_amount': event_data['total_amount'],
        'order_date': event_data['created_at'],
        'last_updated': datetime.utcnow()
    }

    self.read_db.upsert('order_user_view', order_user_record)

def on_user_updated(self, event_data):
    """Update read model when user data changes"""

    user_id = event_data['user_id']
    changes = event_data['changes']

    # Update all order records for this user
    if 'email' in changes or 'name' in changes:
        self.read_db.update_where(
            'order_user_view',
            {'user_id': user_id},
            {
                'user_email': changes.get('email'),
                'user_name': changes.get('name'),
                'last_updated': datetime.utcnow()
            }
        )

def query_orders_by_user(self, user_filters):
    """Fast queries on pre-built read model"""

    return self.read_db.query("""
        SELECT * FROM order_user_view
        WHERE user_email LIKE %s
        ORDER BY order_date DESC
    """, [f"%{user_filters['email_pattern']}%"])

```

Distributed Transaction Patterns

Saga Pattern for Distributed Transactions:

```

from enum import Enum
import json

class SagaStatus(Enum):
    PENDING = "pending"
    COMPLETED = "completed"
    FAILED = "failed"
    COMPENSATING = "compensating"

class OrderProcessingSaga:
    """Manage distributed transaction across multiple services"""

    def __init__(self):
        self.saga_store = SagaStore()
        self.user_service = UserServiceClient()
        self.inventory_service = InventoryServiceClient()
        self.payment_service = PaymentServiceClient()
        self.shipping_service = ShippingServiceClient()

    async def process_order(self, order_request):
        """Execute order processing saga"""

        saga = Saga(
            saga_id=generate_uuid(),
            order_data=order_request,
            status=SagaStatus.PENDING,
            steps=[
                {'service': 'user', 'action': 'validate_user',

```

```

'status': 'pending'},

```

```

    {'service': 'inventory', 'action': 'reserve_items',

```

```

'status': 'pending'},

```

```

    {'service': 'payment', 'action': 'charge_payment',

```

```

'status': 'pending'},

```

```

    {'service': 'shipping', 'action':

```

```

'schedule_delivery', 'status': 'pending'}

```

```

],
    created_at=datetime.utcnow()
)

self.saga_store.save(saga)

try:
    # Step 1: Validate user
    await self._execute_step(saga, 0, self._validate_user)

    # Step 2: Reserve inventory
    await self._execute_step(saga, 1,

```

```
self._reserve_inventory)
```

```

# Step 3: Process payment
await self._execute_step(saga, 2, self._process_payment)

# Step 4: Schedule shipping
await self._execute_step(saga, 3,

```

```
self._schedule_shipping)
```

```

# Mark saga as completed
saga.status = SagaStatus.COMPLETED
self.saga_store.save(saga)

    return {"status": "success", "saga_id": saga.saga_id}

except SagaStepFailure as e:
    # Start compensation process
    await self._compensate_saga(saga, e.failed_step_index)
    return {"status": "failed", "reason": str(e)}

async def _execute_step(self, saga, step_index, step_function):
    """Execute a single step in the saga"""

    try:
        result = await step_function(saga.order_data)

```

Mark step as completed:


```

# Mark step as completed
saga.steps[step_index]['status'] = 'completed'
saga.steps[step_index]['result'] = result
self.saga_store.save(saga)

return result

except Exception as e:
    saga.steps[step_index]['status'] = 'failed'
    saga.steps[step_index]['error'] = str(e)
    self.saga_store.save(saga)

    raise SagaStepFailure(step_index, str(e))

async def _compensate_saga(self, saga, failed_step_index):
    """Execute compensation actions in reverse order"""

    saga.status = SagaStatus.COMPENSATING
    self.saga_store.save(saga)

    # Compensate completed steps in reverse order
    for step_index in range(failed_step_index - 1, -1, -1):
        step = saga.steps[step_index]

        if step['status'] == 'completed':
            try:
                await self._compensate_step(saga, step_index)
                step['compensation_status'] = 'completed'
            except Exception as e:
                step['compensation_status'] = 'failed'
                step['compensation_error'] = str(e)
                # Log critical error - manual intervention

```

needed

```

self._alert_ops_team(saga.saga_id, step_index,

```

e)

```
saga.status = SagaStatus.FAILED
self.saga_store.save(saga)

async def _compensate_step(self, saga, step_index):
    """Execute compensation for a specific step"""

    step = saga.steps[step_index]

    if step['service'] == 'inventory':
        await self.inventory_service.release_reservation(
            step['result']['reservation_id']
        )
    elif step['service'] == 'payment':
        await self.payment_service.refund_payment(
            step['result']['transaction_id']
        )
    elif step['service'] == 'shipping':
        await self.shipping_service.cancel_delivery(
            step['result']['delivery_id']
        )
```

Real-World Example

Netflix operates 700+ microservices with sophisticated data management:

Netflix's Service Boundaries:

```
# Netflix's microservices data architecture (simplified)
netflix_services = {
    "user_service": {
        "data": ["user_profiles", "preferences", "subscriptions"],
        "database": "Cassandra",
        "cache": "EVCache (Redis)",
```

```
"events": ["user.registered", "user.preferences.updated"]
},
"content_service": {
  "data": ["movies", "shows", "metadata", "licensing"],
  "database": "MongoDB + MySQL",
  "cache": "EVCache",
  "events": ["content.added", "content.licensed"]
},
"viewing_service": {
  "data": ["watch_history", "progress", "ratings"],
  "database": "Cassandra",
  "stream": "Kafka",
  "events": ["viewing.started", "viewing.completed"]
},
"recommendation_service": {
  "data": ["recommendation_models", "user_vectors",
```

"content_vectors"],

```

"database": "Cassandra + S3",
    "ml_pipeline": "Spark + TensorFlow",
    "events": ["recommendations.generated"]
}
}

# Netflix's eventual consistency approach
class NetflixEventualConsistency:
    """How Netflix handles data consistency across services"""

    def handle_user_subscription_change(self, user_id, new_plan):
        """Example of eventual consistency in action"""

        # 1. User service updates subscription immediately
        user_service.update_subscription(user_id, new_plan)

        # 2. Event published to multiple services
        event_bus.publish("user.subscription.changed", {
            "user_id": user_id,
            "new_plan": new_plan,
            "changed_at": datetime.utcnow()
        })

        # 3. Services update asynchronously:
        # - Billing service: Update billing cycle
        # - Content service: Update content access rules
        # - Recommendation service: Refresh recommendations
        # - Analytics service: Update user segments

        # Result: User sees new plan immediately, but

```

recommendations

```
# and content access may take a few seconds to update
```

Pro Tip Don't start with microservices if you can avoid it.

Begin with a well-structured monolith and extract services when team boundaries and domain boundaries become clear.

Premature microservices adoption leads to distributed complexity without the benefits.

□ Checkpoint

1. What are the main trade-offs of database-per-service vs shared database patterns?
2. How would you handle a query that needs data from 5 different microservices?
3. Why is the saga pattern better than distributed transactions for microservices?
4. How does eventual consistency affect user experience design?

08

Event-Driven & Distributed Systems

Event Sourcing & CQRS Patterns

⚡ TL;DR

Event sourcing stores events, not current state - enables audit trails and time travel - CQRS separates read and write models for performance and complexity management - Event stores are append-only logs that become the source of truth - Snapshots prevent replay performance issues for entities with many events

Traditional databases store current state. When you update a bank account balance from \$100 to \$150, you overwrite \$100 with \$150. The history of how you got to \$150 is lost unless you explicitly track it.

Event sourcing flips this around: instead of storing current state, you store the sequence of events that led to that state. The current state is derived by replaying events.

This pattern is powerful for domains where change history matters: financial systems, audit requirements, debugging complex workflows, or any system where "how did we get here?" is an important question.

Event Sourcing Fundamentals

Traditional State Storage vs Event Sourcing:

```

# Traditional approach - store current state
class BankAccount:
    def __init__(self, account_id):
        self.account_id = account_id
        self.balance = 0.0
        self.status = "active"

    def deposit(self, amount):
        # Update current state, lose history
        self.balance += amount
        # Save to database overwrites previous balance

# Event sourcing approach - store events
class BankAccountEventSourced:
    def __init__(self, account_id):
        self.account_id = account_id
        self.events = []
        self.version = 0

    def deposit(self, amount):
        # Create event
        event = DepositEvent(
            account_id=self.account_id,
            amount=amount,
            timestamp=datetime.utcnow(),
            event_id=generate_uuid()
        )

        # Apply event to current state
        self._apply_event(event)

        # Store event (append-only)
        self.events.append(event)

        return event

    def _apply_event(self, event):
        """Apply event to current state"""
        if isinstance(event, AccountOpenedEvent):
            self.balance = event.initial_balance
            self.status = "active"
        elif isinstance(event, DepositEvent):
            self.balance += event.amount
        elif isinstance(event, WithdrawalEvent):
            self.balance -= event.amount
        elif isinstance(event, AccountClosedEvent):
            self.status = "closed"

```



```
self.version += 1
```

The `get_balance_at_time` implementation:

```
def get_balance_at_time(self, target_timestamp):
    """Time travel - get balance at any point in history"""
    balance = 0.0

    for event in self.events:
        if event.timestamp <= target_timestamp:
            if isinstance(event, DepositEvent):
                balance += event.amount
            elif isinstance(event, WithdrawalEvent):
                balance -= event.amount
        else:
            break

    return balance

def get_transaction_history(self):
    """Perfect audit trail"""
    return [
        {
            "timestamp": event.timestamp,
            "type": event.__class__.__name__,
            "amount": getattr(event, 'amount', None),
            "balance_after": self.calculate_balance_after(event)
        }
        for event in self.events
    ]
```

PYTHON

Event Store Implementation

Core Event Store:

```
from abc import ABC, abstractmethod
from typing import List, Optional
import json

class Event(ABC):
    def __init__(self, aggregate_id: str, event_id: str = None,
```

PYTHON

timestamp: datetime = None):

```
self.aggregate_id = aggregate_id
    self.event_id = event_id or generate_uuid()
    self.timestamp = timestamp or datetime.utcnow()
    self.event_type = self.__class__.__name__

class EventStore:
    def __init__(self, connection):
        self.connection = connection
        self._setup_tables()

    def _setup_tables(self):
        """Create event store tables"""

        create_events_table = """
            CREATE TABLE IF NOT EXISTS events (
                event_id VARCHAR(36) PRIMARY KEY,
                aggregate_id VARCHAR(36) NOT NULL,
                event_type VARCHAR(100) NOT NULL,
                event_data JSONB NOT NULL,
                event_version INTEGER NOT NULL,
                timestamp TIMESTAMP NOT NULL,
                metadata JSONB
            );

            CREATE INDEX IF NOT EXISTS idx_aggregate_id
            ON events (aggregate_id, event_version);

            CREATE INDEX IF NOT EXISTS idx_timestamp
            ON events (timestamp);
        """

        self.connection.execute(create_events_table)
```

PYTHON

The append_events implementation:

```
def append_events(self, aggregate_id: str, events: List[Event],
                  expected_version: int) -> None:
    """Append events with optimistic concurrency control"""

    # Check current version
    current_version = self._get_current_version(aggregate_id)

    if current_version != expected_version:
        raise ConcurrencyError(
            f"Expected version {expected_version}, but current
```

version is {current_version}"

```
)

# Insert events atomically
with self.connection.transaction():
    for i, event in enumerate(events):
        event_data = self._serialize_event(event)

        self.connection.execute("""
            INSERT INTO events (
                event_id, aggregate_id, event_type,
```

event_data,

```
event_version, timestamp, metadata
        ) VALUES (%s, %s, %s, %s, %s, %s, %s)
    """, [
        event.event_id,
        aggregate_id,
        event.event_type,
        json.dumps(event_data),
        expected_version + i + 1,
        event.timestamp,
        json.dumps(getattr(event, 'metadata', {}))
    ])

def get_events(self, aggregate_id: str, from_version: int = 0) -
```

> List[Event]:

```

"""Get events for an aggregate starting from a version"""

rows = self.connection.execute("""
    SELECT event_type, event_data, event_version, timestamp
    FROM events
    WHERE aggregate_id = %s
        AND event_version > %s
    ORDER BY event_version
""", [aggregate_id, from_version])

events = []
for row in rows:
    event = self._deserialize_event(
        row['event_type'],
        json.loads(row['event_data']),
        row['timestamp']
    )
    events.append(event)

return events

def _get_current_version(self, aggregate_id: str) -> int:
    """Get current version for an aggregate"""

    result = self.connection.execute_one("""
        SELECT COALESCE(MAX(event_version), 0) as

```

current_version

FROM events

```

WHERE aggregate_id = %s
""", [aggregate_id])

return result['current_version']

def _serialize_event(self, event: Event) -> dict:
    """Convert event object to dictionary"""

```

The PYTHON code:

```
event_data = {}
    for key, value in event.__dict__.items():
        if not key.startswith('_'):
            event_data[key] = value

    return event_data

def _deserialize_event(self, event_type: str, event_data: dict,
```

timestamp: datetime) -> Event:

```
"""Convert dictionary back to event object"""

# Registry of event types (in production, use proper class
```

registry)

```

event_classes = {
    'AccountOpenedEvent': AccountOpenedEvent,
    'DepositEvent': DepositEvent,
    'WithdrawalEvent': WithdrawalEvent,
    'AccountClosedEvent': AccountClosedEvent
}

if event_type not in event_classes:
    raise ValueError(f"Unknown event type: {event_type}")

event_class = event_classes[event_type]

# Create event instance
event = event_class(**event_data)
event.timestamp = timestamp

return event

# Aggregate root with event sourcing
class EventSourcedAggregate:
    def __init__(self, aggregate_id: str):
        self.aggregate_id = aggregate_id
        self.version = 0
        self.uncommitted_events = []

    def apply_event(self, event: Event):
        """Apply event and add to uncommitted events"""

        self._when(event)
        self.uncommitted_events.append(event)
        self.version += 1

    def mark_events_as_committed(self):
        """Clear uncommitted events after successful save"""
        self.uncommitted_events.clear()

    def load_from_history(self, events: List[Event]):
        """Rebuild aggregate state from event history"""

        for event in events:
            self._when(event)
            self.version += 1

    @abstractmethod
    def _when(self, event: Event):
        """Apply event to aggregate state - implement in

```

subclasses"""

pass

PYTHON

Example aggregate implementation

class BankAccountAggregate(EventSourcedAggregate):

def __init__(self, account_id: str):

super().__init__(account_id)

self.balance = 0.0

self.status = None

self.overdraft_limit = 0.0

def open_account(self, initial_balance: float, overdraft_limit:

float = 0.0):

```

"""Open new bank account"""

    if self.status is not None:
        raise ValueError("Account already exists")

    event = AccountOpenedEvent(
        aggregate_id=self.aggregate_id,
        initial_balance=initial_balance,
        overdraft_limit=overdraft_limit
    )

    self.apply_event(event)

def deposit(self, amount: float):
    """Deposit money"""

    if amount <= 0:
        raise ValueError("Deposit amount must be positive")

    if self.status != "active":
        raise ValueError("Cannot deposit to inactive account")

    event = DepositEvent(
        aggregate_id=self.aggregate_id,
        amount=amount
    )

    self.apply_event(event)

def withdraw(self, amount: float):
    """Withdraw money with overdraft checking"""

    if amount <= 0:
        raise ValueError("Withdrawal amount must be positive")

    if self.status != "active":
        raise ValueError("Cannot withdraw from inactive

```

account")


```
# Check if withdrawal would exceed overdraft limit
new_balance = self.balance - amount
min_allowed_balance = -self.overdraft_limit

if new_balance < min_allowed_balance:
    raise ValueError(f"Insufficient funds. Available
```

```
balance: {self.balance + self.overdraft_limit}")
```

```
event = WithdrawalEvent(
    aggregate_id=self.aggregate_id,
    amount=amount
)

self.apply_event(event)

def _when(self, event: Event):
    """Apply events to aggregate state"""

    if isinstance(event, AccountOpenedEvent):
        self.balance = event.initial_balance
        self.status = "active"
        self.overdraft_limit = event.overdraft_limit

    elif isinstance(event, DepositEvent):
        self.balance += event.amount

    elif isinstance(event, WithdrawalEvent):
        self.balance -= event.amount

    elif isinstance(event, AccountClosedEvent):
        self.status = "closed"
```

CQRS (Command Query Responsibility Segregation)

Separating Read and Write Models:

```
# Command side - optimized for writes
class AccountCommandHandler:
    def __init__(self, event_store: EventStore):
        self.event_store = event_store

    def handle_open_account(self, command: OpenAccountCommand):
        """Handle account opening command"""

        # Load aggregate (will be empty for new account)
        account = self._load_account(command.account_id)

        # Execute business logic
        account.open_account(
            initial_balance=command.initial_balance,
            overdraft_limit=command.overdraft_limit
        )

        # Save events
        self.event_store.append_events(
            aggregate_id=command.account_id,
            events=account.uncommitted_events,
            expected_version=account.version -
```

```
len(account.uncommitted_events)
```

```
)

    account.mark_events_as_committed()

    return {"success": True, "account_id": command.account_id}

    def handle_deposit(self, command: DepositCommand):
        """Handle deposit command"""

        account = self._load_account(command.account_id)
        account.deposit(command.amount)

        self.event_store.append_events(
            aggregate_id=command.account_id,
            events=account.uncommitted_events,
            expected_version=account.version -
```

```
len(account.uncommitted_events)
```

PYTHON

```
)

    account.mark_events_as_committed()

def _load_account(self, account_id: str) ->
```

BankAccountAggregate:

SQL

```
"""Load account from event store"""

account = BankAccountAggregate(account_id)

# Load events from snapshot if available
snapshot = self._load_snapshot(account_id)
if snapshot:
    account = self._apply_snapshot(account, snapshot)
    from_version = snapshot.version
else:
    from_version = 0

# Load events since snapshot
events = self.event_store.get_events(account_id,
```

from_version)

PYTHON

```
account.load_from_history(events)

    return account

# Query side - optimized for reads
class AccountQueryHandler:
    def __init__(self, read_db_connection):
        self.read_db = read_db_connection
        self.event_subscriber = EventSubscriber()
```

Subscribe to events to update read models:

PYTHON

```
# Subscribe to events to update read models
self.event_subscriber.subscribe('AccountOpenedEvent',
```

```
self.on_account_opened)
```

```
self.event_subscriber.subscribe('DepositEvent',
```

PYTHON

```
self.on_money_deposited)
```

```
self.event_subscriber.subscribe('WithdrawalEvent',
```

PYTHON

```
self.on_money_withdrawn)
```

```
def get_account_summary(self, account_id: str) -> dict:
    """Get account summary from read model"""

    account = self.read_db.execute_one("""
        SELECT account_id, balance, status, overdraft_limit,
               created_at, last_transaction_date
        FROM account_summary
        WHERE account_id = %s
    """, [account_id])

    if not account:
        raise AccountNotFound(account_id)

    return account

def get_transaction_history(self, account_id: str, limit: int =
```

SQL

50) -> List[dict]:

```
"""Get transaction history from read model"""

return self.read_db.execute("""
    SELECT transaction_id, transaction_type, amount,
```

SQL

balance_after,

```

timestamp, description
    FROM account_transactions
    WHERE account_id = %s
    ORDER BY timestamp DESC
    LIMIT %s
    """ , [account_id, limit])

def get_accounts_by_balance_range(self, min_balance: float,

```

SQL

max_balance: float) -> List[dict]:

```

"""Complex query optimized for reads"""

return self.read_db.execute("""
    SELECT account_id, balance, status,

```

SQL

last_transaction_date

```

FROM account_summary
    WHERE balance BETWEEN %s AND %s
    AND status = 'active'
    ORDER BY balance DESC
    """ , [min_balance, max_balance])

# Event handlers to maintain read models
def on_account_opened(self, event_data):
    """Update read model when account is opened"""

    self.read_db.execute("""
        INSERT INTO account_summary (
            account_id, balance, status, overdraft_limit,

```

SQL

created_at

```

) VALUES (%s, %s, %s, %s, %s)
        """ , [
            event_data['aggregate_id'],
            event_data['initial_balance'],
            'active',
            event_data['overdraft_limit'],
            event_data['timestamp']
        ])

def on_money_deposited(self, event_data):
    """Update read model when money is deposited"""

    # Update account summary

```

The corresponding SQL query:

```

self.read_db.execute("""
    UPDATE account_summary
    SET balance = balance + %s,
        last_transaction_date = %s
    WHERE account_id = %s
""", [
    event_data['amount'],
    event_data['timestamp'],
    event_data['aggregate_id']
])

# Add transaction record
self.read_db.execute("""
    INSERT INTO account_transactions (
        transaction_id, account_id, transaction_type,

```

amount,

```

timestamp, balance_after
) VALUES (%s, %s, %s, %s, %s,
        (SELECT balance FROM account_summary WHERE

```

```

account_id = %s))

```

```

"""", [
    event_data['event_id'],
    event_data['aggregate_id'],
    'deposit',
    event_data['amount'],
    event_data['timestamp'],
    event_data['aggregate_id']
])

```

Snapshotting for Performance

Snapshot Implementation:

```

class SnapshotStore:
    def __init__(self, connection):
        self.connection = connection
        self._setup_tables()

    def _setup_tables(self):
        """Create snapshot tables"""

        create_snapshots_table = """
            CREATE TABLE IF NOT EXISTS snapshots (
                aggregate_id VARCHAR(36) PRIMARY KEY,
                aggregate_type VARCHAR(100) NOT NULL,
                snapshot_data JSONB NOT NULL,
                version INTEGER NOT NULL,
                timestamp TIMESTAMP NOT NULL
            );
        """

        self.connection.execute(create_snapshots_table)

    def save_snapshot(self, aggregate_id: str, aggregate_type: str,
                     snapshot_data: dict, version: int):
        """Save snapshot of aggregate state"""

        self.connection.execute("""
            INSERT INTO snapshots (
                aggregate_id, aggregate_type, snapshot_data,

```

SQL

version, timestamp

```

) VALUES (%s, %s, %s, %s, %s)
ON CONFLICT (aggregate_id)
DO UPDATE SET
    snapshot_data = EXCLUDED.snapshot_data,
    version = EXCLUDED.version,
    timestamp = EXCLUDED.timestamp
"""', [
    aggregate_id,
    aggregate_type,
    json.dumps(snapshot_data),
    version,

```

The corresponding SQL query:

```

datetime.utcnow()
    ])

def get_snapshot(self, aggregate_id: str) -> Optional[dict]:
    """Get latest snapshot for aggregate"""

    result = self.connection.execute_one("""
        SELECT snapshot_data, version, timestamp
        FROM snapshots
        WHERE aggregate_id = %s
    """, [aggregate_id])

    if result:
        return {
            'data': json.loads(result['snapshot_data']),
            'version': result['version'],
            'timestamp': result['timestamp']
        }

    return None

# Automatic snapshotting
class SnapshotManager:
    def __init__(self, event_store: EventStore, snapshot_store:

```

SQL

SnapshotStore):

PYTHON

```

self.event_store = event_store
    self.snapshot_store = snapshot_store
    self.snapshot_frequency = 50 # Snapshot every 50 events

def save_aggregate_with_snapshotting(self, aggregate:

```

EventSourcedAggregate):

PYTHON

```

"""Save aggregate and create snapshot if needed"""

# Save events
self.event_store.append_events(
    aggregate_id=aggregate.aggregate_id,
    events=aggregate.uncommitted_events,
    expected_version=aggregate.version -

```

```

len(aggregate.uncommitted_events)

```

PYTHON

```

)

# Check if snapshot is needed
if aggregate.version % self.snapshot_frequency == 0:
    self._create_snapshot(aggregate)

aggregate.mark_events_as_committed()

def _create_snapshot(self, aggregate: EventSourcedAggregate):
    """Create snapshot of current aggregate state"""

    # Serialize aggregate state
    snapshot_data = {
        'balance': getattr(aggregate, 'balance', None),
        'status': getattr(aggregate, 'status', None),
        'overdraft_limit': getattr(aggregate, 'overdraft_limit',

```

None)

```

    }

    self.snapshot_store.save_snapshot(
        aggregate_id=aggregate.aggregate_id,
        aggregate_type=aggregate.__class__.__name__,
        snapshot_data=snapshot_data,
        version=aggregate.version
    )

    def load_aggregate_with_snapshot(self, aggregate_id: str,

```

aggregate_class) -> EventSourcedAggregate:

```

    """Load aggregate using snapshot + subsequent events"""

    aggregate = aggregate_class(aggregate_id)

    # Try to load snapshot

```

The corresponding SQL query:

```

snapshot = self.snapshot_store.get_snapshot(aggregate_id)

if snapshot:
    # Apply snapshot to aggregate
    self._apply_snapshot(aggregate, snapshot)
    from_version = snapshot['version']
else:
    from_version = 0

# Load events since snapshot
events = self.event_store.get_events(aggregate_id,

```

from_version)

```

    aggregate.load_from_history(events)

    return aggregate

    def _apply_snapshot(self, aggregate: EventSourcedAggregate,

```

snapshot: dict):

```

"""Apply snapshot data to aggregate"""

data = snapshot['data']

# Set aggregate state from snapshot
if 'balance' in data:
    aggregate.balance = data['balance']
if 'status' in data:
    aggregate.status = data['status']
if 'overdraft_limit' in data:
    aggregate.overdraft_limit = data['overdraft_limit']

aggregate.version = snapshot['version']

```

Real-World Example: Event Sourcing at Eventbrite

Eventbrite uses event sourcing for their order processing system to handle complex ticketing scenarios:

Eventbrite's Event Sourcing Architecture:

```

# Eventbrite's ticket order events (simplified)
class EventbriteTicketOrder:
    """Event sourced ticket order handling complex scenarios"""

    def __init__(self, order_id: str):
        super().__init__(order_id)
        self.tickets = []
        self.status = None
        self.total_amount = 0.0
        self.payment_status = None

    def create_order(self, event_id: str, tickets_requested:

```

```
List[dict]):
```

```

"""Start order process"""

event = OrderCreatedEvent(
    aggregate_id=self.aggregate_id,
    event_id=event_id,
    tickets_requested=tickets_requested
)

self.apply_event(event)

def reserve_tickets(self, available_tickets: List[dict]):
    """Reserve specific tickets"""

    event = TicketsReservedEvent(
        aggregate_id=self.aggregate_id,
        reserved_tickets=available_tickets,
        expires_at=datetime.utcnow() + timedelta(minutes=15)
    )

    self.apply_event(event)

```

The apply_pricing implementation:

```

def apply_pricing(self, ticket_prices: dict, fees: dict):
    """Apply dynamic pricing and fees"""

    event = PricingAppliedEvent(
        aggregate_id=self.aggregate_id,
        ticket_prices=ticket_prices,
        fees=fees,
        total_amount=self._calculate_total(ticket_prices, fees)
    )

    self.apply_event(event)

def process_payment(self, payment_method: str, payment_result:

```

dict):

```

"""Process payment with various outcomes"""

if payment_result['status'] == 'success':
    event = PaymentSucceededEvent(
        aggregate_id=self.aggregate_id,
        payment_method=payment_method,
        transaction_id=payment_result['transaction_id'],
        amount=payment_result['amount']
    )
else:
    event = PaymentFailedEvent(
        aggregate_id=self.aggregate_id,
        payment_method=payment_method,
        failure_reason=payment_result['error'],
        retry_allowed=payment_result.get('retry_allowed',

```

True)

```

)

    self.apply_event(event)

def handle_ticket_transfer(self, from_user_id: str, to_user_id:

```

str, ticket_ids: List[str]):

```

"""Handle post-purchase ticket transfers"""

    event = TicketsTransferredEvent(
        aggregate_id=self.aggregate_id,
        from_user_id=from_user_id,
        to_user_id=to_user_id,
        ticket_ids=ticket_ids,
        transferred_at=datetime.utcnow()
    )

    self.apply_event(event)

# The power of event sourcing for Eventbrite:
# 1. Perfect audit trail for financial transactions
# 2. Easy refund processing by replaying events
# 3. Complex business rule debugging
# 4. Temporal queries: "Show me all orders in progress at 2 PM

```

yesterday"

5. A/B testing by replaying events with different business logic

SQL

Pro Tip Event sourcing adds complexity, so only use it where the benefits justify the cost. Good candidates: financial systems, audit requirements, complex business processes where you need to understand “why” not just “what”. Bad candidates: simple CRUD applications, read-heavy systems without complex business logic.

□ Checkpoint

1. What are the main advantages of event sourcing over traditional state storage?
2. How does CQRS help manage complexity in read vs write operations?
3. Why are snapshots necessary in event sourcing systems?
4. What types of business domains benefit most from event sourcing?

8.3 Change Data Capture (CDC) Patterns

⚡ TL;DR

CDC captures database changes in real-time without application changes - Outbox pattern ensures reliable event publishing with database transactions - CDC enables real-time data replication, event streaming, and microservice synchronization - Debezium is the gold standard for database CDC in production

Most applications change data in databases constantly. But other systems need to know about these changes: search indices need updates, analytics systems need fresh data, other microservices need to stay synchronized.

Traditional approaches don't work well:

- Polling: Inefficient, high latency, can miss deletes
- Application triggers: Couples business logic to integration concerns
- Database triggers: Hard to maintain, poor error handling

Change Data Capture (CDC) solves this by reading database transaction logs directly, capturing every change as it happens without impacting application performance.

Database Transaction Log Processing

Understanding Transaction Logs:

```

# What happens in a database transaction (PostgreSQL example)
class PostgreSQLTransaction:
    def __init__(self):
        self.wal_entries = [] # Write-Ahead Log entries

    def execute_update(self, table, old_values, new_values, lsn):
        """Simulate a database update operation"""

        # Database writes to WAL before updating data
        wal_entry = {
            "lsn": lsn, # Log Sequence Number
            "operation": "UPDATE",
            "table": table,
            "old_values": old_values,
            "new_values": new_values,
            "timestamp": datetime.utcnow(),
            "transaction_id": self.transaction_id
        }

        # WAL entry is written first (durability guarantee)
        self.wal_entries.append(wal_entry)

        # Then data is updated in place
        # CDC tools read these WAL entries to detect changes

# CDC reader implementation (simplified Debezium-style)
class PostgreSQLCDCReader:
    def __init__(self, connection_config):
        self.connection = psycopg2.connect(**connection_config)
        self.replication_slot = "cdc_slot"
        self.last_lsn = None

    def start_cdc_stream(self):
        """Start reading CDC events from PostgreSQL WAL"""

        # Create replication slot if it doesn't exist
        self._create_replication_slot()

        # Start consuming WAL changes
        cursor = self.connection.cursor()
        cursor.start_replication(
            slot_name=self.replication_slot,
            decode=True,

```

The PYTHON code:


```

start_lsn=self.last_lsn
    )

    for msg in cursor:
        change_event = self._parse_wal_message(msg)

        if change_event:
            yield change_event

        # Acknowledge processing
        msg.cursor.send_feedback(flush_lsn=msg.data_start)

def _parse_wal_message(self, msg):
    """Parse WAL message into change event"""

    # Parse logical decoding output (simplified)
    change_data = json.loads(msg.payload)

    if change_data.get("change"):
        change = change_data["change"][0]

    return {
        "lsn": msg.data_start,
        "timestamp": msg.send_time,
        "operation": change.get("kind"),    # INSERT, UPDATE,

```

DELETE

```

"schema": change.get("schema"),
"table": change.get("table"),
"old_values": change.get("oldkeys",

```

```

{}).get("keyvalues", []),

```

```

    "new_values": change.get("columnvalues", []),
        "transaction_id": change.get("xid")
    }

    return None

def _create_replication_slot(self):
    """Create logical replication slot"""

    cursor = self.connection.cursor()

    try:
        cursor.execute(f"""
            SELECT pg_create_logical_replication_slot(
                '{self.replication_slot}',
                'wal2json'
            )
        """)
    except psycopg2.errors.DuplicateObject:
        # Slot already exists
        pass

```

CDC Event Processing Pipeline:

```

class CDCEventProcessor:
    def __init__(self):
        self.event_handlers = {}
        self.event_publisher = KafkaEventPublisher()
        self.metrics = MetricsCollector()

    def register_table_handler(self, table_name: str, handler_func):
        """Register handler for specific table changes"""
        self.event_handlers[table_name] = handler_func

    def process_cdc_stream(self, cdc_reader: PostgreSQLCDCReader):
        """Process CDC events and publish to event streams"""

        for change_event in cdc_reader.start_cdc_stream():
            try:
                self._process_single_event(change_event)
                self.metrics.increment('cdc.events.processed')

            except Exception as e:

```

The PYTHON code:

PYTHON

```
self.metrics.increment('cdc.events.failed')
    self._handle_processing_error(change_event, e)

def _process_single_event(self, change_event):
    """Process a single CDC event"""

    table_name = f"{change_event['schema']}.
{change_event['table']}"
```

PYTHON

```
# Table-specific processing
if table_name in self.event_handlers:
    processed_event = self.event_handlers[table_name]
```

```
(change_event)
```

```

if processed_event:
    self.event_publisher.publish(
        topic=f"cdc.{table_name}",
        event=processed_event
    )

    # Publish raw CDC event for generic consumers
    self.event_publisher.publish(
        topic="cdc.raw",
        event=change_event
    )

# Example: E-commerce order processing with CDC
def process_order_changes(cdc_event):
    """Process order table changes"""

    if cdc_event['operation'] == 'INSERT':
        # New order created
        order_data = dict(zip(
            ['order_id', 'customer_id', 'total_amount', 'status'],
            cdc_event['new_values']
        ))

        return {
            "event_type": "order.created",
            "order_id": order_data['order_id'],
            "customer_id": order_data['customer_id'],
            "total_amount": order_data['total_amount'],
            "timestamp": cdc_event['timestamp'],
            "source": "cdc.orders"
        }

    elif cdc_event['operation'] == 'UPDATE':
        # Order status changed
        old_values = dict(zip(
            ['order_id', 'customer_id', 'total_amount', 'status'],
            cdc_event['old_values']
        ))
        new_values = dict(zip(
            ['order_id', 'customer_id', 'total_amount', 'status'],
            cdc_event['new_values']
        ))

        if old_values['status'] != new_values['status']:
            return {
                "event_type": "order.status_changed",
                "order_id": new_values['order_id'],
                "old_status": old_values['status'],

```

```

        "new_status": new_values['status'],
        "timestamp": cdc_event['timestamp'],
        "source": "cdc.orders"
    }

    return None    # No event to publish

# Setup CDC processing
cdc_processor = CDCEventProcessor()
cdc_processor.register_table_handler("public.orders",

```

process_order_changes)

```
cdc_processor.register_table_handler("public.customers",
```

process_customer_changes)

```

# Start processing
cdc_reader = PostgreSQLCDCReader(database_config)
cdc_processor.process_cdc_stream(cdc_reader)

```

The Outbox Pattern

Reliable Event Publishing with Database Transactions:

```

class OutboxPattern:
    """Ensure event publishing is consistent with database

```

PYTHON

transactions"""

```

def __init__(self, db_connection):
    self.db = db_connection
    self.event_publisher = KafkaEventPublisher()
    self._setup_outbox_table()

def _setup_outbox_table(self):
    """Create outbox table for event storage"""

    create_outbox = """
        CREATE TABLE IF NOT EXISTS outbox_events (
            id SERIAL PRIMARY KEY,
            event_id VARCHAR(36) UNIQUE NOT NULL,
            event_type VARCHAR(100) NOT NULL,
            aggregate_id VARCHAR(36) NOT NULL,
            event_payload JSONB NOT NULL,
            created_at TIMESTAMP DEFAULT NOW(),
            published_at TIMESTAMP NULL,
            status VARCHAR(20) DEFAULT 'pending'
        );

        CREATE INDEX IF NOT EXISTS idx_outbox_status
        ON outbox_events (status, created_at);
    """

    self.db.execute(create_outbox)

def execute_business_operation_with_events(self,

```

business_operation, events_to_publish):

```

    """Execute business logic and store events in same

```

transaction"""

```

with self.db.transaction():
    # Execute business operation
    business_result = business_operation()

    # Store events in outbox table
    for event in events_to_publish:
        self.db.execute("""
            INSERT INTO outbox_events (
                event_id, event_type, aggregate_id,

```

event_payload

```
) VALUES (%s, %s, %s, %s)
        """
        [
            event['event_id'],
            event['event_type'],
            event['aggregate_id'],
            json.dumps(event['payload'])
        ])

        return business_result

def publish_pending_events(self):
    """Publish events from outbox table (run as separate
```

SQL

process)"""

```
# Get unpublished events
```

The corresponding SQL query:

```
pending_events = self.db.execute("""
    SELECT id, event_id, event_type, aggregate_id,
```

SQL

event_payload

```

FROM outbox_events
    WHERE status = 'pending'
    ORDER BY created_at
    LIMIT 100
    """
)

for event_row in pending_events:
    try:
        # Publish event
        self.event_publisher.publish(
            topic=f"events.{event_row['event_type']}",
            key=event_row['aggregate_id'],
            value=json.loads(event_row['event_payload'])
        )

        # Mark as published
        self.db.execute("""
            UPDATE outbox_events
            SET status = 'published', published_at = NOW()
            WHERE id = %s
            """, [event_row['id']])

    except Exception as e:
        # Mark as failed for retry
        self.db.execute("""
            UPDATE outbox_events
            SET status = 'failed'
            WHERE id = %s
            """, [event_row['id']])

        self._log_publishing_error(event_row, e)

# Example usage: Order service with outbox pattern
class OrderService:
    def __init__(self):
        self.db = PostgreSQLConnection()
        self.outbox = OutboxPattern(self.db)

    def create_order(self, customer_id, items, payment_info):
        """Create order ensuring event publishing consistency"""

        def business_operation():
            # Business logic
            order_id = generate_uuid()
            total_amount = sum(item['price'] * item['quantity'] for

```

item in items)

SQL

```
# Insert order
self.db.execute("""
    INSERT INTO orders (order_id, customer_id,
```

total_amount, status, created_at)

```
VALUES (%s, %s, %s, %s, %s)
""", [order_id, customer_id, total_amount, 'pending',
```

```
datetime.utcnow()]])
```

SQL

```
# Insert order items
for item in items:
    self.db.execute("""
        INSERT INTO order_items (order_id, product_id,
```

quantity, price)

```
VALUES (%s, %s, %s, %s)
""", [order_id, item['product_id'],
```

```
item['quantity'], item['price']]])
```

```
return {'order_id': order_id, 'total_amount':
```

total_amount}

```
# Events to publish
events_to_publish = [
```

```
{
    'event_id': generate_uuid(),
    'event_type': 'order.created',
    'aggregate_id': customer_id, # Will be set in
```

business_operation

```
'payload': {
    'customer_id': customer_id,
    'items': items,
    'payment_info': payment_info,
    'created_at': datetime.utcnow().isoformat()
}
]

# Execute with outbox pattern
result = self.outbox.execute_business_operation_with_events(
    business_operation,
    events_to_publish
)

# Update event with actual order_id
events_to_publish[0]['aggregate_id'] = result['order_id']
events_to_publish[0]['payload']['order_id'] =
```

```
result['order_id']
```

```
events_to_publish[0]['payload']['total_amount'] =
```

```
result['total_amount']
```

```

return result

# Separate process to publish events
class OutboxEventPublisher:
    """Dedicated service to publish events from outbox"""

    def __init__(self):
        self.outbox = OutboxPattern(PostgreSQLConnection())

    def run(self):
        """Run event publishing loop"""

        while True:
            try:
                self.outbox.publish_pending_events()
                time.sleep(1) # Check for new events every second

            except Exception as e:
                logging.error(f"Error publishing events: {e}")
                time.sleep(5) # Wait longer on errors

```

Multi-Database CDC Synchronization

Keeping Multiple Databases in Sync:

```

class MultiDatabaseCDCSync:
    """Synchronize data across multiple databases using CDC"""

    def __init__(self):
        self.source_readers = {}
        self.target_writers = {}
        self.transformation_rules = {}

    def register_source(self, source_name: str, cdc_reader):
        """Register a source database for CDC"""
        self.source_readers[source_name] = cdc_reader

    def register_target(self, target_name: str, writer):
        """Register target database"""
        self.target_writers[target_name] = writer

    def register_transformation(self, source_table: str,

```

target_config: dict):

```
"""Register transformation rules for table replication"""
```

The *PYTHON* code:

```
self.transformation_rules[source_table] = target_config

def start_sync(self):
    """Start synchronization across all registered sources"""

    for source_name, reader in self.source_readers.items():
        threading.Thread(
            target=self._sync_from_source,
            args=(source_name, reader),
            daemon=True
        ).start()

def _sync_from_source(self, source_name: str, cdc_reader):
    """Sync changes from a specific source"""

    for change_event in cdc_reader.start_cdc_stream():
        table_key = f"{change_event['schema']}"
```

PYTHON

{change_event['table']}"

```
if table_key in self.transformation_rules:
    config = self.transformation_rules[table_key]

    # Transform event for target
    transformed_event =
```

PYTHON

```
self._transform_event(change_event, config)
```

```
# Write to target databases
    for target_name in config['targets']:
        writer = self.target_writers[target_name]
        writer.apply_change(transformed_event)

def _transform_event(self, change_event, config):
    """Transform CDC event for target database"""

    transformed = {
        'operation': change_event['operation'],
        'target_table': config['target_table'],
        'timestamp': change_event['timestamp']
    }

    # Field mapping
    if change_event['operation'] in ['INSERT', 'UPDATE']:
        new_values = {}
        field_mapping = config.get('field_mapping', {})

        for i, field_name in enumerate(config['source_fields']):
            target_field = field_mapping.get(field_name,
```

field_name)

```
new_values[target_field] =
```

```
change_event['new_values'][i]
```

```
transformed['new_values'] = new_values

if change_event['operation'] in ['UPDATE', 'DELETE']:
    old_values = {}
    field_mapping = config.get('field_mapping', {})

    for i, field_name in enumerate(config['source_fields']):
        target_field = field_mapping.get(field_name,
```

field_name)

```
old_values[target_field] =
```

```
change_event['old_values'][i]
```

```
transformed['old_values'] = old_values
```

PYTHON

```
    return transformed
```

```
# Target database writer
```

```
class TargetDatabaseWriter:
```

```
    def __init__(self, connection,
```

```
conflict_resolution='last_write_wins'):
```

```
    self.db = connection
```

PYTHON

The corresponding SQL query:

```

self.conflict_resolution = conflict_resolution

def apply_change(self, change_event):
    """Apply change to target database"""

    try:
        if change_event['operation'] == 'INSERT':
            self._insert(change_event)
        elif change_event['operation'] == 'UPDATE':
            self._update(change_event)
        elif change_event['operation'] == 'DELETE':
            self._delete(change_event)

    except Exception as e:
        self._handle_sync_error(change_event, e)

def _insert(self, change_event):
    """Handle INSERT operations"""

    table = change_event['target_table']
    values = change_event['new_values']

    columns = list(values.keys())
    placeholders = [f"%s" for _ in columns]

    query = f"""
        INSERT INTO {table} ({', '.join(columns)})
        VALUES ({', '.join(placeholders)})
        ON CONFLICT (id) DO UPDATE SET
            {', '.join([f"{col} = EXCLUDED.{col}" for col in
                columns if col!= 'id'])}
    """

```

```

"""

self.db.execute(query, list(values.values()))

def _update(self, change_event):
    """Handle UPDATE operations"""

    table = change_event['target_table']
    new_values = change_event['new_values']
    old_values = change_event['old_values']

    # Use primary key from old values for WHERE clause
    primary_key = 'id' # Assume 'id' is primary key
    where_value = old_values[primary_key]

    # Build SET clause
    set_clauses = [f"{col} = %s" for col in new_values.keys() if

```

```
col!= primary_key]
```

```

query = f"""
    UPDATE {table}
    SET {'', '.join(set_clauses)}
    WHERE {primary_key} = %s
"""

values = [new_values[col] for col in new_values.keys() if

```

```
col!= primary_key]
```



```

values.append(where_value)

        self.db.execute(query, values)

# Example: Synchronize order data from PostgreSQL to Elasticsearch
def setup_order_sync():
    """Setup CDC sync from PostgreSQL to Elasticsearch"""

    sync = MultiDatabaseCDCSync()

    # Source: PostgreSQL
    postgres_reader = PostgreSQLCDCReader(postgres_config)
    sync.register_source("main_db", postgres_reader)

```

```

# Target: Elasticsearch
elasticsearch_writer = ElasticsearchWriter(elasticsearch_config)
sync.register_target("search", elasticsearch_writer)

# Transformation: orders table to search index
sync.register_transformation("public.orders", {
    'target_table': 'orders_index',
    'source_fields': ['order_id', 'customer_id', 'total_amount',

```

```

'status', 'created_at'],

```

```

    'field_mapping': {
        'order_id': 'id',
        'customer_id': 'customer',
        'total_amount': 'amount',
        'created_at': 'timestamp'
    },
    'targets': ['search']
})

sync.start_sync()

```

Real-World Example

- LinkedIn's Databus

LinkedIn built Databus to handle CDC at massive scale for their platform:

LinkedIn's CDC Architecture:

```

# LinkedIn Databus architecture (simplified)
class LinkedInDatabus:
    """LinkedIn's approach to CDC at scale"""

    def __init__(self):
        self.sources = {
            "member_db": MySQLBinlogReader(),
            "connections_db": OracleCDCReader(),
            "messaging_db": PostgreSQLCDCReader()
        }

        self.relay_cluster = DatabusRelayCluster()
        self.consumers = []

    def setup_cdc_pipeline(self):
        """Setup CDC pipeline for member data"""

        # Source: Member database changes
        member_cdc = self.sources["member_db"]

        # Relay: Buffer and serve CDC events
        self.relay_cluster.register_source("members", member_cdc)

        # Consumers: Various downstream systems
        consumers = [
            SearchIndexUpdater("member_search"),
            RecommendationModelUpdater("member_features"),
            AnalyticsDataWriter("member_analytics"),
            ProfileCacheUpdater("member_cache")
        ]

        for consumer in consumers:
            consumer.subscribe_to_relay(self.relay_cluster)

    def handle_member_profile_change(self, cdc_event):
        """Process member profile changes"""

        if cdc_event['table'] == 'member_profiles':
            if cdc_event['operation'] == 'UPDATE':
                # Update search index
                self._update_search_index(cdc_event)

                # Invalidate recommendation cache

                self._invalidate_recommendations(cdc_event['new_values']
                ['member_id'])

```

['member_id'])

```
# Update real-time features
self._update_member_features(cdc_event)

# LinkedIn processes 1+ trillion CDC events per day across:
# - Member profiles and connections
# - Job postings and applications
# - Content posts and interactions
# - Messaging and notifications
linkedin_scale = {
    "cdc_events_per_day": 1_000_000_000_000,
    "databases_monitored": 50,
    "downstream_consumers": 200,
    "peak_events_per_second": 10_000_000,
    "lag_target": "< 10 seconds"
}
```

PYTHON

Pro Tip Start with simple CDC use cases (like cache

invalidation or search indexing) before building complex multi- database synchronization. CDC can introduce significant operational complexity, so make sure the benefits justify the cost. Consider managed CDC services like Debezium Connect or AWS DMS for production use.

- □ Checkpoint

1. How does CDC differ from traditional polling or trigger-based approaches?
2. What is the outbox pattern and why is it important for reliable event publishing?
3. How would you handle schema evolution in a CDC system?
4. What are the main challenges of multi-database CDC synchronization?

⚡ TL;DR

REST APIs for CRUD operations, GraphQL for flexible data fetching, gRPC for high-performance service-to-service - Pagination, filtering, and caching are critical for data APIs at scale - Rate limiting and authentication prevent abuse of expensive data operations - API versioning strategy must support backward compatibility for data consumers

Data services need APIs that can handle complex queries, large result sets, and diverse client needs. Unlike simple web APIs that return small payloads, data APIs often deal with:

Large datasets that need efficient pagination Complex filtering across multiple dimensions

Aggregations that can be computationally expensive

Real-time data with freshness requirements

Multiple data formats for different consumers

Good data API design balances flexibility with performance.

REST API Patterns for Data Services

Resource-Based API Design:

```
from flask import Flask, request, jsonify
from typing import Dict, List, Optional
import math
```

PYTHON

```
class DataServiceAPI:
    def __init__(self):
        self.app = Flask(__name__)
        self.data_service = DataService()
        self.cache = RedisCache()
```

The *PYTHON* code:

```
self.rate_limiter = RateLimiter()

self._setup_routes()

def _setup_routes(self):
    """Setup REST API routes for data service"""

    # Basic CRUD operations
    self.app.route('/api/v1/orders', methods=['GET'])
```

PYTHON

```
(self.get_orders)
```

```
self.app.route('/api/v1/orders/<order_id>', methods=['GET'])
```

PYTHON

```
(self.get_order)
```

```
self.app.route('/api/v1/orders', methods=['POST'])
```

PYTHON

```
(self.create_order)
```

```
# Analytics and aggregations
self.app.route('/api/v1/analytics/sales', methods=['GET'])
```

PYTHON

```
(self.get_sales_analytics)
```

```
self.app.route('/api/v1/analytics/customers', methods=
```

PYTHON

```
['GET'])(self.get_customer_analytics)
```

```
# Data export endpoints
```

```
self.app.route('/api/v1/exports/orders', methods=['POST'])
```

BASH

```
(self.export_orders)
```

```
def get_orders(self):
```

```
    """Get orders with advanced filtering, pagination, and
```

PYTHON

```
    sorting"""
```

```
# Rate limiting
```

```
client_id = self._get_client_id()
```

```
if not self.rate_limiter.allow_request(client_id,
```

PYTHON

```
requests_per_minute=1000):
```

```
    return jsonify({"error": "Rate limit exceeded"}), 429
```

PYTHON

```
    try:
```

```
        # Parse query parameters
```

```
        filters = self._parse_filters(request.args)
```

```
        pagination = self._parse_pagination(request.args)
```

```
        sorting = self._parse_sorting(request.args)
```

```
        # Build cache key
```

```
        cache_key = self._build_cache_key('orders', filters,
```

```
        pagination, sorting)
```

```

# Check cache
cached_result = self.cache.get(cache_key)
if cached_result:
    return jsonify(cached_result)

# Query data
result = self.data_service.get_orders(
    filters=filters,
    pagination=pagination,
    sorting=sorting
)

# Format response with pagination metadata
response = {
    "data": result.items,
    "pagination": {
        "page": pagination.page,
        "per_page": pagination.per_page,
        "total_items": result.total_count,
        "total_pages": math.ceil(result.total_count /

```

```

pagination.per_page),

```

```

"has_next": result.has_next,
    "has_prev": result.has_prev
},
"meta": {
    "query_time_ms": result.query_time_ms,
    "cached": False

```

The PYTHON code:

```

}
}

# Cache response (5 minutes for orders data)
self.cache.set(cache_key, response, ttl=300)

return jsonify(response)

except ValidationError as e:
    return jsonify({"error": "Invalid parameters",

```

```
"details": str(e)}), 400
```

PYTHON

```
except Exception as e:
    return jsonify({"error": "Internal server error"}), 500

def _parse_filters(self, args) -> Dict:
    """Parse filtering parameters"""

    filters = {}

    # Date range filtering
    if 'start_date' in args:
        filters['start_date'] =
```

```
self._parse_date(args['start_date'])
```



```

if 'end_date' in args:
    filters['end_date'] = self._parse_date(args['end_date'])

    # Status filtering (multiple values)
    if 'status' in args:
        filters['status'] = args.getlist('status')

    # Range filtering
    if 'min_amount' in args:
        filters['min_amount'] = float(args['min_amount'])
    if 'max_amount' in args:
        filters['max_amount'] = float(args['max_amount'])

    # Text search
    if 'search' in args:
        filters['search'] = args['search']

    # Customer filtering
    if 'customer_id' in args:
        filters['customer_id'] = args.getlist('customer_id')

    return filters

def _parse_pagination(self, args) -> 'Pagination':
    """Parse pagination parameters"""

    page = int(args.get('page', 1))
    per_page = int(args.get('per_page', 20))

    # Limit max page size to prevent abuse
    per_page = min(per_page, 1000)

    return Pagination(page=page, per_page=per_page)

def _parse_sorting(self, args) -> List[Dict]:
    """Parse sorting parameters"""

    sort_fields = []

    if 'sort' in args:
        for sort_param in args.getlist('sort'):
            if sort_param.startswith('-'):
                field = sort_param[1:]
                direction = 'desc'
            else:
                field = sort_param
                direction = 'asc'

```

```
# Validate field names to prevent injection
```

```
if field in ['created_at', 'updated_at',
```

```
'total_amount', 'customer_id']:
```

```
    sort_fields.append({'field': field, 'direction':
```

```
direction})
```

```
    # Default sorting
```

```
    if not sort_fields:
```

```
        sort_fields = [{'field': 'created_at', 'direction':
```

```
'desc'}]
```

```
    return sort_fields
```

```
    # Example API calls:
```

```
    # GET /api/v1/orders?
```

```
status=pending&status=processing&start_date=2024-01-
```

```
01&min_amount=100&sort=-created_at&page=1&per_page=50
```

```
    # GET /api/v1/orders?
```

```
customer_id=123&customer_id=456&search=laptop&sort=total_amount&sort=- cre-
ated_at
```

```
class AnalyticsAPI:
    """Advanced analytics endpoints"""

    def get_sales_analytics(self):
        """Get sales analytics with aggregations"""

        # Parse time grouping (hour, day, week, month)
        group_by = request.args.get('group_by', 'day')
        start_date =
```

```
self._parse_date(request.args.get('start_date'))
```

```
end_date = self._parse_date(request.args.get('end_date'))

# Parse dimensions for grouping
dimensions = request.args.getlist('dimension')    # e.g.,
```

product_category, region

```
# Parse metrics to calculate
metrics = request.args.getlist('metric')    # e.g., revenue,
```

order_count, avg_order_value

```
if not metrics:
    metrics = ['revenue', 'order_count']

# Build cache key for analytics query
cache_key = f"analytics:sales:{group_by}:{start_date}:"
```

```
{end_date}:{':'.join(dimensions)}:{':'.join(metrics)}"
```

```
# Check cache (analytics can be cached longer)
cached_result = self.cache.get(cache_key)
if cached_result:
    return jsonify(cached_result)

# Query analytics data
result = self.data_service.get_sales_analytics(
    group_by=group_by,
    start_date=start_date,
    end_date=end_date,
    dimensions=dimensions,
    metrics=metrics
)

response = {
    "data": result.data,
    "meta": {
        "group_by": group_by,
        "date_range": {"start": start_date.isoformat(),
"end": end_date.isoformat()}},
```

```
        "dimensions": dimensions,
        "metrics": metrics,
        "query_time_ms": result.query_time_ms,
        "data_freshness": result.data_freshness.isoformat()
    }
}
```

Cache analytics for 1 hour:

```
# Cache analytics for 1 hour
self.cache.set(cache_key, response, ttl=3600)

return jsonify(response)

# Example analytics API calls:
# GET /api/v1/analytics/sales?group_by=day&start_date=2024-01-
```

01&end_date=2024-01-
 sion=product_category&metric=revenue&metric=order_count

31&dimen-

```
# GET /api/v1/analytics/sales?group_by=hour&start_date=2024-01-
```

01&dimension=region&dimension=payment_method&metric=avg_order_value

Streaming and Export APIs:

```
import csv
import io
from flask import Response

class DataExportAPI:
    """APIs for large data exports"""

    def export_orders(self):
        """Export orders as streaming CSV"""

        # Parse export parameters
        filters = self._parse_filters(request.json)
        format = request.json.get('format', 'csv')

        # Validate request size
        estimated_rows =
```

PYTHON

```
self.data_service.estimate_export_size(filters)
```

```
if estimated_rows > 1_000_000:
    return jsonify({
        "error": "Export too large",
        "estimated_rows": estimated_rows,
        "max_allowed": 1_000_000,
        "suggestion": "Use more specific filters or request
```

a batch export"

```

    }), 400

    if format == 'csv':
        return self._stream_csv_export(filters)
    elif format == 'json':
        return self._stream_json_export(filters)
    else:
        return jsonify({"error": "Unsupported format"}), 400

def _stream_csv_export(self, filters):
    """Stream CSV export to avoid memory issues"""

    def generate_csv():
        # CSV header
        yield

```

"order_id,customer_id,total_amount,status,created_at\n"

```

# Stream data in chunks
for chunk in self.data_service.stream_orders(filters,

```

chunk_size=1000):

```

    for order in chunk:
        yield f"{order.order_id},{order.customer_id},

```

```

{order.total_amount},{order.status},{order.created_at}\n"

```

```

    return Response(
        generate_csv(),
        mimetype='text/csv',
        headers={
            'Content-Disposition': 'attachment;

```

```

filename=orders_export.csv',

```

```

        'Cache-Control': 'no-cache'
    }
)

```

The `_stream_json_export` implementation:

PYTHON

```
def _stream_json_export(self, filters):
    """Stream JSON export"""

    def generate_json():
        yield '{"orders": ['

        first_chunk = True
        for chunk in self.data_service.stream_orders(filters,
```

`chunk_size=1000)`:

```
    for i, order in enumerate(chunk):
        if not first_chunk or i > 0:
            yield ", "

            yield json.dumps({
                "order_id": order.order_id,
                "customer_id": order.customer_id,
                "total_amount": float(order.total_amount),
                "status": order.status,
                "created_at": order.created_at.isoformat()
            })

            first_chunk = False

        yield ']'

    return Response(
        generate_json(),
        mimetype='application/json',
        headers={'Cache-Control': 'no-cache'}
    )
```

GraphQL for Flexible Data Fetching

GraphQL Schema for Data Service:

```

import graphene
from graphene import ObjectType, String, Int, Float, List, Field
from graphene_sqlalchemy import SQLAlchemyObjectType

class Order(SQLAlchemyObjectType):
    class Meta:
        model = OrderModel

class Customer(SQLAlchemyObjectType):
    class Meta:
        model = CustomerModel

    # Computed fields
    total_orders = Int()
    lifetime_value = Float()

    def resolve_total_orders(self, info):
        return self.orders.count()

    def resolve_lifetime_value(self, info):
        return sum(order.total_amount for order in self.orders)

class OrderConnection(graphene.Connection):
    class Meta:
        node = Order

    total_count = Int()

    def resolve_total_count(self, info):
        return self.iterable.count()

class Query(ObjectType):
    # Single record queries
    order = Field(Order, order_id=String(required=True))
    customer = Field(Customer, customer_id=String(required=True))

    # Collection queries with filtering and pagination

```

The PYTHON code:


```

orders = graphene.ConnectionField(
    OrderConnection,
    customer_id=String(),
    status=String(),
    start_date=String(),
    end_date=String(),
    min_amount=Float(),
    max_amount=Float()
)

# Analytics queries
sales_by_period = List(
    lambda: SalesByPeriod,
    period=String(required=True),
    start_date=String(required=True),
    end_date=String(required=True),
    group_by=List(String)
)

def resolve_order(self, info, order_id):
    """Get single order"""
    return Order.get_query(info).filter(OrderModel.order_id ==

```

```
order_id).first()
```

```

def resolve_orders(self, info, **filters):
    """Get orders with filtering"""
    query = Order.get_query(info)

    # Apply filters
    if filters.get('customer_id'):
        query = query.filter(OrderModel.customer_id ==

```

```
filters['customer_id'])
```

```

if filters.get('status'):
    query = query.filter(OrderModel.status ==

```

```
filters['status'])
```

```
if filters.get('start_date'):
    start_date =
```

```
datetime.fromisoformat(filters['start_date'])
```

```
query = query.filter(OrderModel.created_at >=
```

PYTHON

start_date)

```
if filters.get('end_date'):
    end_date = datetime.fromisoformat(filters['end_date'])
    query = query.filter(OrderModel.created_at <= end_date)
```

SQL

```
if filters.get('min_amount'):
    query = query.filter(OrderModel.total_amount >=
```

```
filters['min_amount'])
```

```
if filters.get('max_amount'):
    query = query.filter(OrderModel.total_amount <=
```

PYTHON

```
filters['max_amount'])
```

```
return query

# Example GraphQL queries:
"""
# Get orders with customer info
query GetOrdersWithCustomers {
  orders(first: 20, customerIds: ["123", "456"]) {
    edges {
      node {
        orderId
        totalAmount
        status
        createdAt
        customer {
          customerId
          name
          email
        }
      }
    }
  }
}
```

The PYTHON code:

```

totalOrders
    lifetimeValue
    }
    items {
        productId
        quantity
        price
    }
    }
    }
    totalCount
    pageInfo {
        hasNextPage
        hasPreviousPage
        startCursor
        endCursor
    }
    }
}

# Analytics query
query GetSalesAnalytics {
    salesByPeriod(
        period: "day"
        startDate: "2024-01-01"
        endDate: "2024-01-31"
        groupBy: ["productCategory", "region"]
    ) {
        date
        productCategory
        region
        totalRevenue
        orderCount
        avgOrderValue
    }
}
"""

# GraphQL execution with caching and performance monitoring
class GraphQLDataService:
    def __init__(self):
        self.schema = graphene.Schema(query=Query)
        self.cache = RedisCache()
        self.metrics = MetricsCollector()

    def execute_query(self, query: str, variables: dict = None,

```

context: dict = None):

```
"""Execute GraphQL query with caching and monitoring"""

# Generate cache key
cache_key = self._generate_cache_key(query, variables)

# Check cache
cached_result = self.cache.get(cache_key)
if cached_result:
    self.metrics.increment('graphql.cache.hit')
    return cached_result

# Execute query
start_time = time.time()

result = self.schema.execute(
    query,
    variables=variables,
    context=context
)

execution_time = time.time() - start_time

# Record metrics
self.metrics.record('graphql.execution_time',
```

PYTHON

execution_time)

```
if result.errors:
    self.metrics.increment('graphql.errors')
    return {"errors": [str(error) for error in
```

PYTHON

```
result.errors]]
```

```
# Cache successful results
response = {"data": result.data}
cache_ttl = self._determine_cache_ttl(query)
self.cache.set(cache_key, response, ttl=cache_ttl)

self.metrics.increment('graphql.cache.miss')

return response
```

gRPC for High-Performance APIs gRPC Service Definition:

```
// data_service.proto
syntax = "proto3";

package dataservice;

// Data service definition
service DataService {
    rpc GetOrder(GetOrderRequest) returns (Order);
    rpc GetOrders(GetOrdersRequest) returns (GetOrdersResponse);
    rpc GetSalesAnalytics(GetSalesAnalyticsRequest) returns
```

```
(GetSalesAnalyticsResponse);
```

```
rpc StreamOrders(StreamOrdersRequest) returns (stream Order);
}
```

```
// Messages
```

```
message Order {
    string order_id = 1;
    string customer_id = 2;
    double total_amount = 3;
    string status = 4;
    int64 created_at = 5;
    repeated OrderItem items = 6;
}
```

```
message OrderItem {
    string product_id = 1;
    int32 quantity = 2;
    double price = 3;
}
```

```
message GetOrderRequest {
    string order_id = 1;
}
```

```
message GetOrdersRequest {
    repeated string customer_ids = 1;
    repeated string statuses = 2;
    int64 start_date = 3;
    int64 end_date = 4;
    double min_amount = 5;
    double max_amount = 6;
    Pagination pagination = 7;
}
```

```
message Pagination {
    int32 page = 1;
    int32 per_page = 2;
}
```

```
message GetOrdersResponse {
    repeated Order orders = 1;
    int32 total_count = 2;
    bool has_next = 3;
```

```
    bool has_prev = 4;
}
```

gRPC Service Implementation:

```
import grpc
from concurrent import futures
import data_service_pb2
import data_service_pb2_grpc

class DataServiceGRPC(data_service_pb2_grpc.DataServiceServicer):
    def __init__(self):
        self.data_service = DataService()
        self.cache = RedisCache()

    def GetOrder(self, request, context):
        """Get single order"""

        try:
            order = self.data_service.get_order(request.order_id)

            if not order:
                context.set_code(grpc.StatusCode.NOT_FOUND)
                context.set_details(f"Order {request.order_id} not
```

found")

```
        return data_service_pb2.Order()

        return self._convert_order_to_proto(order)

    except Exception as e:
        context.set_code(grpc.StatusCode.INTERNAL)
        context.set_details(str(e))
        return data_service_pb2.Order()

    def GetOrders(self, request, context):
        """Get orders with filtering and pagination"""

        try:
            # Convert protobuf request to internal filters
            filters = {
                'customer_ids': list(request.customer_ids),
                'statuses': list(request.statuses),
                'start_date':
```

```
datetime.fromtimestamp(request.start_date) if request.start_date else None,
```

PYTHON

PYTHON

SQL

```
'end_date': datetime.fromtimestamp(request.end_date)
```

```
if request.end_date else None,
```

```
'min_amount': request.min_amount if
```

```
request.min_amount else None,
```

```
'max_amount': request.max_amount if
```

```
request.max_amount else None
```

PYTHON

```
}
```

```
    pagination = Pagination(  
        page=request.pagination.page or 1,  
        per_page=request.pagination.per_page or 20  
    )
```

```
    # Query data
```

```
    result = self.data_service.get_orders(filters=filters,
```

```
    pagination=pagination)
```

```
    # Convert to protobuf response
```

```
    response = data_service_pb2.GetOrdersResponse(  
        total_count=result.total_count,  
        has_next=result.has_next,  
        has_prev=result.has_prev  
    )
```

```
    for order in result.items:
```

```
    response.orders.append(self._convert_order_to_proto(order))
```

PYTHON

```
return response

    except Exception as e:
        context.set_code(grpc.StatusCode.INTERNAL)
        context.set_details(str(e))
        return data_service_pb2.GetOrdersResponse()

def StreamOrders(self, request, context):
    """Stream orders for large datasets"""

    try:
        filters =
```

```
self._convert_stream_request_to_filters(request)
```

PYTHON

```
for chunk in self.data_service.stream_orders(filters,
```

```
chunk_size=100):
```

```

for order in chunk:
    yield self._convert_order_to_proto(order)

    except Exception as e:
        context.set_code(grpc.StatusCode.INTERNAL)
        context.set_details(str(e))

def _convert_order_to_proto(self, order):
    """Convert internal order object to protobuf"""

    proto_order = data_service_pb2.Order(
        order_id=order.order_id,
        customer_id=order.customer_id,
        total_amount=float(order.total_amount),
        status=order.status,
        created_at=int(order.created_at.timestamp())
    )

    # Add order items
    for item in order.items:
        proto_item = data_service_pb2.OrderItem(
            product_id=item.product_id,
            quantity=item.quantity,
            price=float(item.price)
        )
        proto_order.items.append(proto_item)

    return proto_order

# gRPC server setup
def serve_grpc():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=50))

    data_service_pb2_grpc.add_DataServiceServicer_to_server(
        DataServiceGRPC(),
        server
    )

    server.add_insecure_port('[::]:50051')
    server.start()

    print("gRPC server started on port 50051")
    server.wait_for_termination()

# gRPC client example
def grpc_client_example():
    channel = grpc.insecure_channel('localhost:50051')
    stub = data_service_pb2_grpc.DataServiceStub(channel)

```

```
# Get single order
request = data_service_pb2.GetOrderRequest(order_id="123")
order = stub.GetOrder(request)
print(f"Order: {order.order_id}, Amount: {order.total_amount}")

# Get orders with filtering
orders_request = data_service_pb2.GetOrdersRequest(
```

The PYTHON code:

```
customer_ids=["customer_123"],
    statuses=["completed"],
    pagination=data_service_pb2.Pagination(page=1, per_page=50)
)
orders_response = stub.GetOrders(orders_request)

print(f"Found {orders_response.total_count} orders")
for order in orders_response.orders:
    print(f" Order {order.order_id}: ${order.total_amount}")
```

PYTHON

Real-World Example

Shopify's API Strategy:

```
# Shopify's multi-protocol API approach
shopify_apis = {
    "rest_api": {
        "use_case": "Simple CRUD operations, third-party
```

integrations",

```

"rate_limits": {
    "basic": "2 requests/second",
    "shopify_plus": "40 requests/second"
},
"pagination": "Cursor-based for large datasets"
},
"graphql_api": {
    "use_case": "Flexible data fetching for admin interfaces",
    "query_complexity_limit": 1000,
    "rate_limits": "Query complexity based",
    "caching": "Field-level caching",
    "real_time": "Subscriptions for inventory updates"
},
"webhook_api": {
    "use_case": "Real-time event notifications",
    "events": ["order/created", "product/updated",

```

"customer/updated"],

```

"delivery_guarantee": "At least once",
    "retry_policy": "Exponential backoff, 48 hour window"
},
"bulk_api": {
    "use_case": "Large data exports/imports",
    "max_file_size": "1GB",
    "formats": ["CSV", "JSON", "XML"],
    "async_processing": "Webhook notification on completion"
}
}

```

PYTHON

```

class ShopifyDataAPI:
    """Example of Shopify's data API patterns"""

    def get_orders_with_intelligent_caching(self, shop_id, filters):
        """Smart caching based on data patterns"""

        # Real-time data (no caching)
        if filters.get('status') == 'processing':
            return self._query_orders_realtime(shop_id, filters)

        # Recent data (short cache)
        elif filters.get('created_at') and

```

```
self._is_recent(filters['created_at']):
```

```
    cache_key = f"orders:recent:{shop_id}:"
```

```
    {hash(str(filters))})"
```

```
    ttl = 60 # 1 minute
```

```
    # Historical data (long cache)
```

```
    else:
```

```
        cache_key = f"orders:historical:{shop_id}:"
```

```
    {hash(str(filters))})"
```

```
    ttl = 3600 # 1 hour
```

PYTHON

```
    return self._cached_query(cache_key, ttl, lambda:
```

```
self._query_orders(shop_id, filters))
```

□ Pro Tip

Choose your API style based on your clients' needs, not what's trendy. REST for simple integrations, GraphQL for complex UIs, gRPC for service-to-service communication. Many successful companies offer multiple API styles for different use cases.

□ Checkpoint

What are the key differences between designing APIs for simple web apps vs data services?

How would you implement effective pagination for a dataset with 100 million records?

When would you choose GraphQL over REST for a data service API?

What caching strategies work best for different types of data queries?

8.5

Data Mesh Architecture Principles

≤ TL;DR

Data mesh treats data as a product owned by domain teams, not a centralized IT function - Self-serve data infrastructure enables domain teams to manage their own data products - Federated governance ensures consistency while preserving domain autonomy - Data mesh works best for large organizations with multiple business domains

Traditional data architecture centralizes everything: one data team, one data warehouse, one set of tools. This works well for small organizations but breaks down as companies grow.

Data mesh flips this model: instead of centralized data management, each business domain owns and manages their data as products. Marketing owns marketing data, Sales owns sales data, Engineering owns product data.

The data mesh is organizational, not just technical - it's about changing how teams work with data, not just the technology stack.

Data Mesh Core Principles

1. Domain-Oriented Decentralized Data Ownership:

📌 Pro Tip

Choose your API style based on your clients' needs, not what's trendy. REST for simple integrations, GraphQL for complex UIs, gRPC for service-to-service communication. Many successful companies offer multiple API styles for different use cases.

□ Checkpoint

1. What are the key differences between designing APIs for simple web apps vs data services?
2. How would you implement effective pagination for a dataset with 100 million records?
3. When would you choose GraphQL over REST for a data service API?
4. What caching strategies work best for different types of data queries?

8.5 Data Mesh Architecture Principles

⚡ TL;DR

Data mesh treats data as a product owned by domain teams, not a centralized IT function - Self-serve data infrastructure enables domain teams to manage their own data products - Federated governance ensures consistency while preserving domain autonomy - Data mesh works best for large organizations with multiple business domains

Traditional data architecture centralizes everything: one data team, one data warehouse, one set of tools. This works well for small organizations but breaks down as companies grow.

Data mesh flips this model: instead of centralized data management, each business domain owns and manages their data as products. Marketing owns marketing data, Sales owns sales data, Engineering owns product data.

The data mesh is organizational, not just technical - it's about

changing how teams work with data, not just the technology stack.

Data Mesh Core Principles

1. Domain-Oriented Decentralized Data Ownership:

```
# Traditional centralized approach
class CentralizedDataTeam:
    """One team manages all data across the company"""

    def __init__(self):
        self.data_warehouse = SnowflakeWarehouse()
        self.data_pipelines = AirflowOrchestrator()
        self.data_catalog = DataCatalog()

    def manage_all_data(self):
        """Central team handles everything"""
        domains = [
            "marketing", "sales", "product", "finance",
            "support", "hr", "operations"
        ]

        for domain in domains:
            # Central team builds and maintains pipelines for every
```

domain

```
self.build_domain_pipelines(domain)
self.maintain_domain_schemas(domain)
self.handle_domain_requests(domain)
```

The PYTHON code:

```
# Problems:
    # 1. Central team becomes bottleneck
    # 2. Domain knowledge stays in business teams
    # 3. Data team doesn't understand business context
    # 4. Long wait times for data requests

# Data mesh approach
class DomainDataTeam:
    """Each domain owns their data as a product"""

    def __init__(self, domain_name: str):
        self.domain = domain_name
        self.data_products = []
        self.infrastructure = SelfServeDataPlatform()

    def create_data_product(self, product_name: str, data_sources:
```

```
List[str]):
```

PYTHON

```
"""Domain team creates their own data products"""

    data_product = DataProduct(
        name=f"{self.domain}.{product_name}",
        owner_team=self.domain,
        sources=data_sources,
        infrastructure=self.infrastructure
    )

    # Domain team owns the full lifecycle
    data_product.define_schema()
    data_product.build_pipelines()
    data_product.setup_access_controls()
    data_product.monitor_quality()
    data_product.provide_documentation()

    self.data_products.append(data_product)

    return data_product

# Example domain data products
marketing_team = DomainDataTeam("marketing")
customer_journey_product = marketing_team.create_data_product(
    "customer_journey",
    ["web_analytics", "email_campaigns", "ad_campaigns"]
)

sales_team = DomainDataTeam("sales")
sales_performance_product = sales_team.create_data_product(
    "sales_performance",
    ["crm_data", "deal_pipeline", "quota_tracking"]
)
```

2. Data as a Product:

```

from abc import ABC, abstractmethod
from typing import List, Dict

class DataProduct(ABC):
    """Data product with clear ownership and SLAs"""

    def __init__(self, name: str, owner_team: str):
        self.name = name
        self.owner_team = owner_team
        self.consumers = []
        self.sla = DataProductSLA()
        self.metrics = DataProductMetrics()

    @abstractmethod
    def get_schema(self) -> Dict:
        """Well-defined schema for consumers"""
        pass

    @abstractmethod
    def get_data(self, filters: Dict = None) -> 'DataSet':

```

Primary interface for data consumption:

```

"""Primary interface for data consumption"""
pass

def register_consumer(self, consumer_info: Dict):
    """Track who uses this data product"""
    self.consumers.append(consumer_info)

def get_health_metrics(self) -> Dict:
    """Product health for owner and consumers"""
    return {
        "uptime": self.metrics.get_uptime(),
        "freshness": self.metrics.get_freshness(),
        "quality_score": self.metrics.get_quality_score(),
        "consumer_satisfaction":

```

```

self.metrics.get_consumer_satisfaction()

```

```

}

class CustomerJourneyDataProduct(DataProduct):
    """Marketing team's customer journey data product"""

    def __init__(self):
        super().__init__("marketing.customer_journey",

```

"marketing_team")

```

self.infrastructure = self._setup_infrastructure()

# Product SLA
self.sla = DataProductSLA(
    freshness("< 1 hour",
    uptime="99.5%",
    response_time("< 5 seconds",
    quality_threshold=95
)

def get_schema(self) -> Dict:
    """Customer journey schema"""
    return {
        "customer_id": {"type": "string", "required": True},
        "journey_stage": {"type": "string", "enum":

```

["awareness", "consideration", "purchase", "retention"]},

```

"touchpoints": {
    "type": "array",
    "items": {
        "channel": {"type": "string"},
        "timestamp": {"type": "datetime"},
        "action": {"type": "string"},
        "value": {"type": "number"}
    }
},
"conversion_probability": {"type": "number", "min": 0,

```

"max": 1},

PYTHON

```
"lifetime_value": {"type": "number"},
    "last_updated": {"type": "datetime"}
}

def get_data(self, filters: Dict = None) ->
```

'CustomerJourneyDataSet':

PYTHON

```
"""Get customer journey data with filtering"""

# Validate filters against schema
self._validate_filters(filters)

# Apply access controls
```

```
self._check_access_permissions(filters.get('requesting_user'))
```

SQL

```
# Query data from product storage
data = self.infrastructure.query_data_product(
    product_name=self.name,
    filters=filters
)

# Track usage metrics
```

The PYTHON code:

PYTHON

```
self.metrics.record_access(
    consumer=filters.get('requesting_user'),
    query_complexity=self._calculate_complexity(filters),
    response_time=data.query_time
)

return data

def _setup_infrastructure(self):
    """Setup data product infrastructure"""

    # Each data product can choose its own storage
    return DataProductInfrastructure(
        storage_type="lakehouse", # Could be warehouse, lake,
```

etc.

PYTHON

```
compute_type="serverless",
        access_method="api",
        monitoring=True
    )

# Data product catalog and discovery
class DataProductCatalog:
    """Registry of all data products across domains"""

    def __init__(self):
        self.products = {}

    def register_product(self, product: DataProduct):
        """Register new data product"""

        product_metadata = {
            "name": product.name,
            "owner_team": product.owner_team,
            "schema": product.get_schema(),
            "sla": product.sla.__dict__,
            "access_patterns": product.get_access_patterns(),
            "documentation_url": product.get_documentation_url(),
            "contact": product.get_owner_contact()
        }

        self.products[product.name] = product_metadata

    def discover_products(self, search_criteria: Dict) ->
```

```
List[Dict]:
```

PYTHON

```

"""Help teams discover relevant data products"""

results = []

for product_name, metadata in self.products.items():
    # Search by domain
    if search_criteria.get('domain') and \
        search_criteria['domain'] in product_name:
        results.append(metadata)

    # Search by schema fields
    if search_criteria.get('contains_field'):
        schema = metadata['schema']
        if search_criteria['contains_field'] in schema:
            results.append(metadata)

    # Search by description
    if search_criteria.get('description_keywords'):
        description = metadata.get('description', '')
        if any(keyword in description for keyword in

```

```
search_criteria['description_keywords'])):
```

```

results.append(metadata)

return results

```

3. Self-Serve Data Infrastructure:

PYTHON

```

class SelfServeDataPlatform:
    """Platform that enables domain teams to create data products

```

```
independently"""
```

PYTHON

```

def __init__(self):
    self.infrastructure_templates = {}
    self.governance_policies = GovernancePolicies()
    self.monitoring = PlatformMonitoring()

def register_template(self, template_name: str, template:

```


'InfrastructureTemplate'):

```
"""Register infrastructure templates for common patterns"""
    self.infrastructure_templates[template_name] = template

def create_data_product_infrastructure(self, product_spec: Dict)
```

PYTHON

-> 'DataProductInfrastructure':

```
"""Self-service infrastructure creation"""

# Choose appropriate template
template_name = self._recommend_template(product_spec)
template = self.infrastructure_templates[template_name]

# Create infrastructure from template
infrastructure =
```

SQL

```
template.create_infrastructure(product_spec)
```

```
# Apply governance policies
```

```
self.governance_policies.apply_to_infrastructure(infrastructure)
```

```
# Setup monitoring
    self.monitoring.setup_monitoring(infrastructure)

    return infrastructure

def _recommend_template(self, product_spec: Dict) -> str:
    """Recommend best infrastructure template based on
```

PYTHON

requirements"""

```
data_volume = product_spec.get('expected_volume', 'medium')
latency_requirement =
```

```
product_spec.get('latency_requirement', 'batch')
```

```
access_pattern = product_spec.get('access_pattern',
```

'analytical')

```
if latency_requirement == 'realtime' and access_pattern ==
```

'operational':

```
    return 'realtime_operational'
    elif data_volume == 'large' and access_pattern ==
```

PYTHON

'analytical':

```
    return 'large_scale_analytics'
    else:
        return 'standard_batch'

# Infrastructure templates
class RealtimeOperationalTemplate:
    """Template for real-time operational data products"""

    def create_infrastructure(self, spec: Dict) ->
```

PYTHON

'DataProductInfrastructure':

```
    """Create real-time infrastructure"""

    return DataProductInfrastructure(
        ingestion=KafkaStreams(
            topics=spec['source_topics'],
            parallelism=spec.get('parallelism', 10)
        ),
        processing=FlinkJobs(
            job_definitions=spec['transformations']
        ),
        storage=Redis(
            cluster_mode=True,
            replication=3
        ),
    ),
```

The PYTHON code:

```
serving=RESTApi(
    rate_limit="1000/minute",
    caching=True
),
monitoring=RealtimeMonitoring(
    latency_sla("< 100ms",
    throughput_sla("> 10k events/sec"
)
)

class LargeScaleAnalyticsTemplate:
    """Template for large-scale analytical data products"""

    def create_infrastructure(self, spec: Dict) ->
```

PYTHON

'DataProductInfrastructure':

```
"""Create analytics infrastructure"""

return DataProductInfrastructure(
    ingestion=SparkBatch(
        schedule=spec.get('schedule', 'daily'),
        resources=spec.get('compute_resources', 'large')
    ),
    storage=DataLakehouse(
        format="delta",
        partitioning=spec.get('partitioning_keys'),
        optimization=True
    ),
    serving=SQLApi(
        query_engine="presto",
        result_caching=True
    ),
    monitoring=BatchMonitoring(
        freshness_sla=spec.get('freshness_sla', '< 24
```

hours'),

```
quality_checks=spec.get('quality_checks')
    )
)
```

4. Federated Computational Governance:

```
class FederatedGovernance:
    """Governance that balances consistency with domain autonomy"""

    def __init__(self):
        self.global_policies = GlobalDataPolicies()
        self.domain_policies = {}

    def register_domain(self, domain_name: str, policies:
```

PYTHON

'DomainDataPolicies'):

```
    """Register domain-specific policies"""

    # Validate domain policies don't conflict with global
```

policies

```
        conflicts = self.global_policies.check_conflicts(policies)
        if conflicts:
            raise PolicyConflictError(f"Domain policies conflict
```

PYTHON

with global: {conflicts}")

```
        self.domain_policies[domain_name] = policies

    def get_effective_policies(self, domain: str, data_product: str)
```

PYTHON

-> 'EffectivePolicies':

```
    """Combine global and domain policies"""

    global_rules = self.global_policies.get_rules()
    domain_rules = self.domain_policies.get(domain,
```

PYTHON

```
{}).get_rules()
```

```
# Domain policies can be more restrictive than global
effective_policies = EffectivePolicies(
```

```
data_classification=max(global_rules.data_classification,          do-
main_rules.data_classification),                                ac-
cess_controls=self._merge_access_controls(global_rules.access_controls, do-
main_rules.access_controls),
```

```
quality_standards=max(global_rules.quality_standards,
```

```
domain_rules.quality_standards),
```

```
retention_policies=min(global_rules.retention_policies,
```

```
domain_rules.retention_policies)
```

```
)
```

PYTHON

```
    return effective_policies
```

```
class GlobalDataPolicies:
    """Company-wide data policies"""
```

```
    def __init__(self):
        self.policies = {
            "data_classification": {
                "public": {"access": "unrestricted", "retention":
```

```
"unlimited"},
```

```
            "internal": {"access": "employees_only",
```

```
            "retention": "7_years"},
```

```
            "confidential": {"access": "need_to_know",
```

```
"retention": "3_years"},
```

```
"restricted": {"access": "legal_approval",
```

```
"retention": "as_required_by_law"}
```

```
},
"privacy_requirements": {
    "pii_detection": "required",
    "anonymization": "required_for_analytics",
    "gdpr_compliance": "required",
    "data_subject_rights": "must_support"
},
"quality_standards": {
    "minimum_quality_score": 0.90,
    "required_checks": ["completeness", "validity",
```

```
"consistency"],
```

```
"sla_compliance": 0.95
    },
    "security_requirements": {
        "encryption_at_rest": "required",
        "encryption_in_transit": "required",
        "access_logging": "required",
        "regular_access_reviews": "quarterly"
    }
}

class DomainDataPolicies:
    """Domain-specific policies (can be more restrictive)"""

    def __init__(self, domain: str):
        self.domain = domain
        self.policies = {}

    def set_finance_domain_policies(self):
        """Example: Finance domain has stricter policies"""
        self.policies = {
            "data_classification": "confidential", # More
```

PYTHON

restrictive than global minimum

```
"quality_standards": {  
    "minimum_quality_score": 0.99, # Higher than global
```

minimum

```
"required_checks": ["completeness", "validity",
```

"consistency", "accuracy", "timeliness"],

```
"financial_controls": "sox_compliant"  
    },  
    "access_controls": {  
        "approval_required": True,  
        "access_duration": "90_days",  
        "justification_required": True  
    }  
}
```

The set_marketing_domain_policies implementation:

```
def set_marketing_domain_policies(self):  
    """Example: Marketing domain focuses on privacy"""  
    self.policies = {  
        "privacy_requirements": {  
            "consent_tracking": "required",  
            "opt_out_support": "immediate",  
            "purpose_limitation": "marketing_only",  
            "data_minimization": "required"  
        },  
        "quality_standards": {  
            "minimum_quality_score": 0.85, # Lower than
```

PYTHON

finance, higher than global

```
"customer_data_validation": "real_time"  
    }  
}
```

Data Product Lifecycle Management

```
class DataProductLifecycle:
    """Manage data product from creation to retirement"""

    def __init__(self):
        self.catalog = DataProductCatalog()
        self.governance = FederatedGovernance()
        self.platform = SelfServeDataPlatform()

    def create_data_product(self, domain: str, product_spec: Dict) -
```

PYTHON

> DataProduct:

```
"""Full data product creation workflow"""

# 1. Validate against governance policies
policies = self.governance.get_effective_policies(domain,
```

PYTHON

```
product_spec['name'])
```

```
self._validate_spec_against_policies(product_spec, policies)

# 2. Create infrastructure
infrastructure =
```

PYTHON

```
self.platform.create_data_product_infrastructure(product_spec)
```

```
# 3. Build data product
product = self._build_data_product(product_spec,
```

PYTHON

```
infrastructure, policies)
```



```
# 4. Register in catalog
self.catalog.register_product(product)

# 5. Setup monitoring and alerting
self._setup_product_monitoring(product)

return product

def evolve_data_product(self, product: DataProduct, changes:
```

Dict):

```
"""Handle data product evolution"""

# Check backward compatibility
compatibility_check =
```

```
self._check_backward_compatibility(product, changes)
```

```
if compatibility_check.breaking_changes:
    # Breaking changes require consumer migration
    migration_plan = self._create_migration_plan(product,
```

changes)

```
self._notify_consumers(product, migration_plan)

# Create new version
new_version = self._create_new_version(product, changes)

# Dual-run old and new versions during migration
self._setup_dual_deployment(product, new_version)

else:
```

Non-breaking changes can be deployed directly:

PYTHON

```
# Non-breaking changes can be deployed directly
    self._apply_changes(product, changes)

def retire_data_product(self, product: DataProduct,
```

retirement_date: datetime):

```
"""Manage data product retirement"""

# 1. Notify all consumers with timeline
retirement_plan = RetirementPlan(
    product=product,
    retirement_date=retirement_date,
```

```
alternative_products=self._find_alternative_products(product),
```

PYTHON

```
migration_support=True
)

self._notify_consumers(product, retirement_plan)

# 2. Freeze feature development
product.freeze_features()

# 3. Provide migration assistance
for consumer in product.consumers:
    self._provide_migration_assistance(consumer,
```

retirement_plan)

```
# 4. Gradual shutdown
shutdown_phases = [
    {"date": retirement_date - timedelta(days=90), "action":
```

"stop_new_consumers"},

JSON

```
{"date": retirement_date - timedelta(days=30), "action":
```

"read_only_mode"},

JSON

```
{
  "date": retirement_date,
  "action": "shutdown"
}
```

```
self._schedule_shutdown(product, shutdown_phases)
```

```
class DataProductMetrics:
    """Metrics for data product management"""

    def __init__(self):
        self.metrics_store = MetricsStore()

    def track_product_health(self, product: DataProduct):
        """Track key product health metrics"""

        health_metrics = {
            "uptime": self._calculate_uptime(product),
            "freshness": self._calculate_freshness(product),
            "quality_score": self._calculate_quality_score(product),
            "usage_growth": self._calculate_usage_growth(product),
            "consumer_satisfaction":
```

```
self._survey_consumer_satisfaction(product),
```

```
"cost_per_query":
```

```
self._calculate_cost_efficiency(product)
```

```
}
```

PYTHON

```
self.metrics_store.record_metrics(product.name,
```

```
health_metrics)
```

```
# Alert on threshold breaches
self._check_sla_compliance(product, health_metrics)
```

PYTHON

```
def generate_product_report(self, product: DataProduct, period:
```

```
str = "month"):
```

```
"""Generate comprehensive product health report"""
```

PYTHON

```
    return {  
        "product_name": product.name,  
        "owner": product.owner_team,  
        "period": period,  
        "health_summary": self._get_health_summary(product,
```

```
period),
```

```
        "usage_analytics": self._get_usage_analytics(product,
```

PYTHON

```
period),
```

```
        "cost_analysis": self._get_cost_analysis(product,
```

PYTHON

```
period),
```

```
        "consumer_feedback":
```

```
            self._get_consumer_feedback(product, period),
```

```
        "improvement_recommendations":
```

```
            self._generate_recommendations(product)
```

```
    }
```

Real-World Example: Netflix's Data Mesh

Implementation

Netflix operates one of the most successful data mesh implementations at scale:

Netflix's Data Mesh Architecture:

```
# Netflix data mesh (simplified representation)
class NetflixDataMesh:
    """Netflix's approach to decentralized data ownership"""

    def __init__(self):
        self.domains = {
            "content": ContentDataDomain(),
            "personalization": PersonalizationDataDomain(),
            "streaming": StreamingDataDomain(),
            "billing": BillingDataDomain(),
            "growth": GrowthDataDomain()
        }

        self.shared_platform = NetflixDataPlatform()
        self.governance = NetflixDataGovernance()

    def get_domain_products(self, domain_name: str) ->
```

```
List[DataProduct]:
```

```
"""Get data products for a domain"""

    domain = self.domains[domain_name]
    return domain.get_data_products()

class ContentDataDomain:
    """Content domain owns all content-related data"""

    def __init__(self):
        self.data_products = [
            ContentMetadataProduct(),
            ViewingDataProduct(),
            ContentPerformanceProduct(),
            RecommendationSignalsProduct()
        ]

        # Domain team structure
        self.team = {
            "product_manager": "Defines what data products to
```

```
build",
```

```

"data_engineers": "Build and maintain data pipelines",
    "data_scientists": "Define analytics and ML features",
    "domain_experts": "Provide business context"
}

class ContentMetadataProduct(DataProduct):
    """Content metadata as a data product"""

    def get_schema(self):
        return {
            "content_id": "string",
            "title": "string",
            "genre": "array<string>",
            "release_date": "date",
            "director": "string",

```

The PYTHON code:

```

"cast": "array<string>",
    "rating": "string",
    "duration_minutes": "integer",
    "content_type": "enum[movie, series, documentary]",
    "availability_regions": "array<string>",
    "licensing_end_date": "date"
}

def get_data(self, filters=None):
    """Serve content metadata with SLA guarantees"""

    # Netflix SLA: 99.99% availability, < 100ms response time
    start_time = time.time()

    data = self.content_store.query(filters)

    response_time = time.time() - start_time
    self.metrics.record_response_time(response_time)

    if response_time > 0.1: # 100ms SLA
        self.alerts.send_sla_breach_alert("response_time",

```

response_time)

```

return data

# Netflix's data platform capabilities
netflix_platform_stats = {
    "data_products": 500,
    "domain_teams": 20,
    "daily_data_volume": "2.5 petabytes",
    "data_engineers_per_domain": "5-15",
    "platform_team_size": "50 engineers",
    "self_service_adoption": "95% of data needs served without

```

platform team"

```

}

```

Pro Tip Data mesh is an organizational transformation, not

just a technology change. Start with clear domain boundaries, establish product thinking around data, and build self-service capabilities gradually. Don't try to implement all principles at once - begin with one domain and expand from there.

□ Checkpoint

1. How does data mesh differ from traditional centralized data architecture?
2. What does it mean to treat “data as a product”?
3. Why is federated governance important in a data mesh architecture?
4. What organizational changes are needed to successfully implement data mesh?

Module 8 Project: Design Event-Driven

Data Architecture for Global E-commerce

Platform

Objective

Design a comprehensive event-driven data architecture for a global e-commerce platform that supports microservices, handles complex business processes, and enables real-time analytics across multiple business domains.

Business Context

GlobalShop is an international e-commerce platform competing with

Amazon and Alibaba:

- 100M active customers across 50 countries
- 500K merchant partners
- 50M product catalog
- 10M orders per day growing 100% annually
- Complex business processes: inventory management, pricing, fulfillment, payments, returns
- Multiple business domains: Marketplace, Payments, Logistics, Marketing, Customer Service

Technical Challenges

Current Pain Points: - **Monolithic database creating bottlenecks** across teams -
Difficult to maintain data consistency across regions -

Complex business processes spanning multiple systems - Real-time

- analytics requirements for fraud detection and recommendations
- Multiple teams stepping on each other's data changes
- Compliance requirements varying by country (GDPR, PCI-DSS, local regulations)

Requirements

Functional Requirements:

- Support complex order fulfillment workflows across multiple services
- Real-time inventory updates across global warehouses
- Event-driven price optimization and dynamic pricing

- Cross-domain analytics for business intelligence
- Audit trails for all financial transactions
- Customer data synchronization across regions

Non-Functional Requirements:

- 10M orders/day → 50M orders/day in 3 years
- 99.99% uptime for core business processes
- <100ms latency for real-time operations
- Multi-region deployment (US, EU, Asia)
- GDPR and regional compliance
- Event ordering guarantees for critical workflows

Deliverables

Create a comprehensive event-driven architecture design:

1. Microservices Data Architecture (25 points)

Service boundaries with database-per-service pattern

Cross-service communication patterns

Data consistency strategies

Distributed transaction management

2. Event Sourcing & CQRS Implementation (20 points)

Event store design for critical business processes

CQRS patterns for read/write optimization

Event schema evolution strategy

Snapshot and replay mechanisms

3. Change Data Capture & Integration (20 points)

CDC implementation for real-time data synchronization

Outbox pattern for reliable event publishing

Cross-region data replication strategy

Legacy system integration patterns

4. API Design for Data Services (15 points)

REST, GraphQL, and gRPC API strategy

Real-time data serving architecture

Rate limiting and caching strategies

API versioning and backward compatibility

5. Data Mesh Implementation (20 points)

Domain-oriented data ownership model

Self-serve data platform design

Federated governance framework

Data product lifecycle management

Design Template

```
# GlobalShop Event-Driven Data Architecture
```

```
## Executive Summary
```

```
[4-paragraph overview: business context, architecture approach, key
```

innovations, expected business impact]

```
## 1. Microservices Data Architecture
```

```
### Service Domain Boundaries
```

User Management Service → User Database (PostgreSQL) Product Catalog Service → Product Database (MongoDB + Elasticsearch) Inventory Service → Inventory Database (Redis + PostgreSQL) Order Service → Order Database (PostgreSQL) + Event Store Payment Service → Payment Database (PostgreSQL) + HSM Logistics Service → Logistics Database (PostgreSQL + PostGIS) Pricing Service → Pricing Database (Redis + PostgreSQL) Analytics Service → Data Warehouse (Snowflake)

```
### Cross-Service Data Patterns
```

```
#### Order Processing Workflow [Detailed design of how order placement involves multiple services]
```

Inventory Synchronization [Real-time inventory updates across warehouses and services]

Distributed Transaction Management [Saga pattern implementation for complex workflows]

2. Event Sourcing & CQRS

Event Store Design [Technical architecture for event storage and replay]

- ### Critical Business Processes Using Event Sourcing
- Order lifecycle management
- Payment processing
- Inventory adjustments
- Pricing changes

CQRS Read Models [Optimized read models for different query patterns]

Example: Order Event Sourcing ```python class OrderAggregate: def place_order(self, customer_id, items):

```
# Business logic and event creation
```

def cancel_order(self, reason):

```
# Cancellation logic and events
```

def process_payment(self, payment_info):

```
# Payment processing events
```

3. Change Data Capture & Integration

CDC Architecture

[How you capture changes from operational databases]

Cross-Region Replication

[Strategy for keeping data synchronized across US/EU/Asia]

Integration with Legacy Systems

[Patterns for integrating existing systems]

Example: Product Catalog CDC

```
-- Example CDC pipeline for product updates
CREATE PUBLICATION product_changes FOR TABLE products;
```

4. API Design Strategy

Multi-Protocol API Strategy

REST APIs: Simple CRUD operations, third-party integrations

GraphQL APIs: Complex data fetching for admin interfaces gRPC APIs: High-performance service-to-service communication WebSocket APIs: Real-time notifications

Real-Time Data Serving

[Architecture for serving real-time inventory, pricing, recommendations]

API Gateway Architecture

[How you handle routing, authentication, rate limiting]

5. Data Mesh Implementation

Domain Data Products

Marketplace Domain

Product catalog data product

Merchant analytics data product

Search and discovery data product

Customer Domain

Customer profile data product

Customer journey data product

Customer support data product

Self-Serve Data Platform

[Infrastructure and tools for domain teams]

Federated Governance Model

[Policies that balance consistency with autonomy]

6. Event Streaming Infrastructure

Kafka Cluster Design

[Topic design, partitioning strategy, retention policies]

Event Schema Management

[Schema evolution, compatibility policies]

Dead Letter Queue Handling

[How you handle failed event processing]

7. Regional Architecture & Compliance

Multi-Region Deployment

[How data flows between US, EU, and Asia regions]

GDPR Compliance Implementation

[Data localization, right to be forgotten, consent management]

Data Sovereignty

[Ensuring data stays within required geographic boundaries]

8. Monitoring & Operations

Event-Driven System Monitoring

[How you monitor distributed event flows]

Data Quality Across Services

[Ensuring consistency and quality in distributed data]

Disaster Recovery

[Cross-region failover and data recovery strategies]

9. Implementation Roadmap

Phase 1 (Months 1-6): Foundation

Microservices decomposition

Event streaming infrastructure

Core CDC implementation

Phase 2 (Months 7-12): Scale

Event sourcing for critical processes

CQRS implementation

Multi-region deployment

Phase 3 (Months 13-18): Optimization

Data mesh implementation

Advanced analytics

ML platform integration

10. Success Metrics & Business Impact

Technical Metrics

Event processing latency: [targets] System availability: [uptime goals] Data consistency: [consistency guarantees]

Business Metrics

Order processing efficiency: [% improvement] Real-time inventory accuracy: [target %]
Cross-domain analytics speed: [query performance]

Cost Analysis

[Infrastructure costs, development costs, operational savings]

Evaluation Criteria

- ****Architecture Completeness (30%)****
- Comprehensive coverage of all required components
- Clear service boundaries and data ownership
- Realistic approach to distributed system challenges
- Consideration of both technical and business requirements
- ****Event-Driven Design Excellence (25%)****
- Appropriate use of event sourcing, CQRS, and CDC
- Well-designed event schemas and evolution strategies

- Proper handling of event ordering and consistency
- Scalable event streaming architecture
- ****Data Mesh Implementation (20%)****
- Clear domain boundaries and data product definitions
- Practical self-service platform design
- Realistic governance model
- Consideration of organizational change management
- ****Global Scale Considerations (15%)****
- Multi-region architecture design
- Compliance and data sovereignty handling
- Performance optimization across geographic regions
- Disaster recovery and high availability
- ****Implementation Realism (10%)****
- Practical migration strategy from current state
- Risk mitigation and rollback plans
- Team structure and skill requirements
- Technology choices with proper justification

Bonus Challenges

****Real-Time ML Integration (+5 points)****

Design real-time feature computation from event streams for fraud detection

****Blockchain Integration (+5 points)****

Design blockchain-based audit trail for high-value transactions

****Advanced Compliance (+5 points)****

Design for PCI-DSS compliance in payment processing

****Event Stream Analytics (+5 points)****

Design complex event processing for business intelligence

Sample Architecture Decisions to Address

1. ****Event Ordering****: How do you ensure events are processed in the correct order across services? 2. ****Saga Compensation****: How do you handle failed distributed transactions in the order workflow? 3. ****Schema Evolution****: How do you evolve event schemas without breaking downstream consumers? 4. ****Multi-Region Consistency****: How do you handle inventory updates when warehouses are in different regions? 5. ****GDPR Right to Erasure****: How do you implement "right to be forgotten" in an event-sourced system?

Expected Time Investment

- ****Architecture Research and Planning**** 8-12 hours
- - ****Detailed Component Design**** 10-15 hours
- ****Documentation and Diagrams**** 6-10 hours
- ****Total**** 24-37 hours

This project requires applying all the event-driven architecture patterns in a complex, realistic scenario that mirrors what you'd encounter when building a global-scale e-commerce platform with modern distributed architecture patterns.

Further Reading

Papers and Case Studies

- ****Microservices Data Management****

- [Microservices Data Management] (<https://microservices.io/patterns/data/database-per-service.html>)
- Chris Richardson's comprehensive guide
- [Data Management in Microservices] (<https://martinfowler.com/articles/microservice-trade-offs.html>) -

Martin Fowler on trade-offs

- [Distributed Data Management for Microservices] (<https://www.nginx.com/blog/microservices-data-management/>) - NGINX architecture guide
- **Event Sourcing & CQRS:**
- [Event Sourcing Pattern] (<https://martinfowler.com/eaDev/EventSourcing.html>)
- Martin Fowler's definitive guide
- [CQRS Documents](<https://cQRS.wordpress.com/>)
- Greg Young's original CQRS resources
- [Versioning in an Event Sourced System] (<https://leanpub.com/esversioning>)
- Comprehensive guide to schema evolution
- Change Data Capture:** - [Debezium Architecture] (<https://debezium.io/documentation/reference/architecture.html>) -

CDC implementation patterns

- - [Building Event-Driven Microservices] (<https://www.oreilly.com/library/view/building-event-driven-microservices/9781492057888/>)
- O'Reilly book on event-driven architectures
- [Change Data Capture at Scale] (<https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying>)
- LinkedIn's approach
- Data Mesh:** - [How to# Module 9: Monitoring, Observability & Data Quality

> **What you'll learn:** How to build production-ready data systems that stay healthy, catch problems early, and maintain high data quality at scale. You'll master SLA design, monitoring strategies, observability patterns, and cost optimization techniques used by the best data teams.

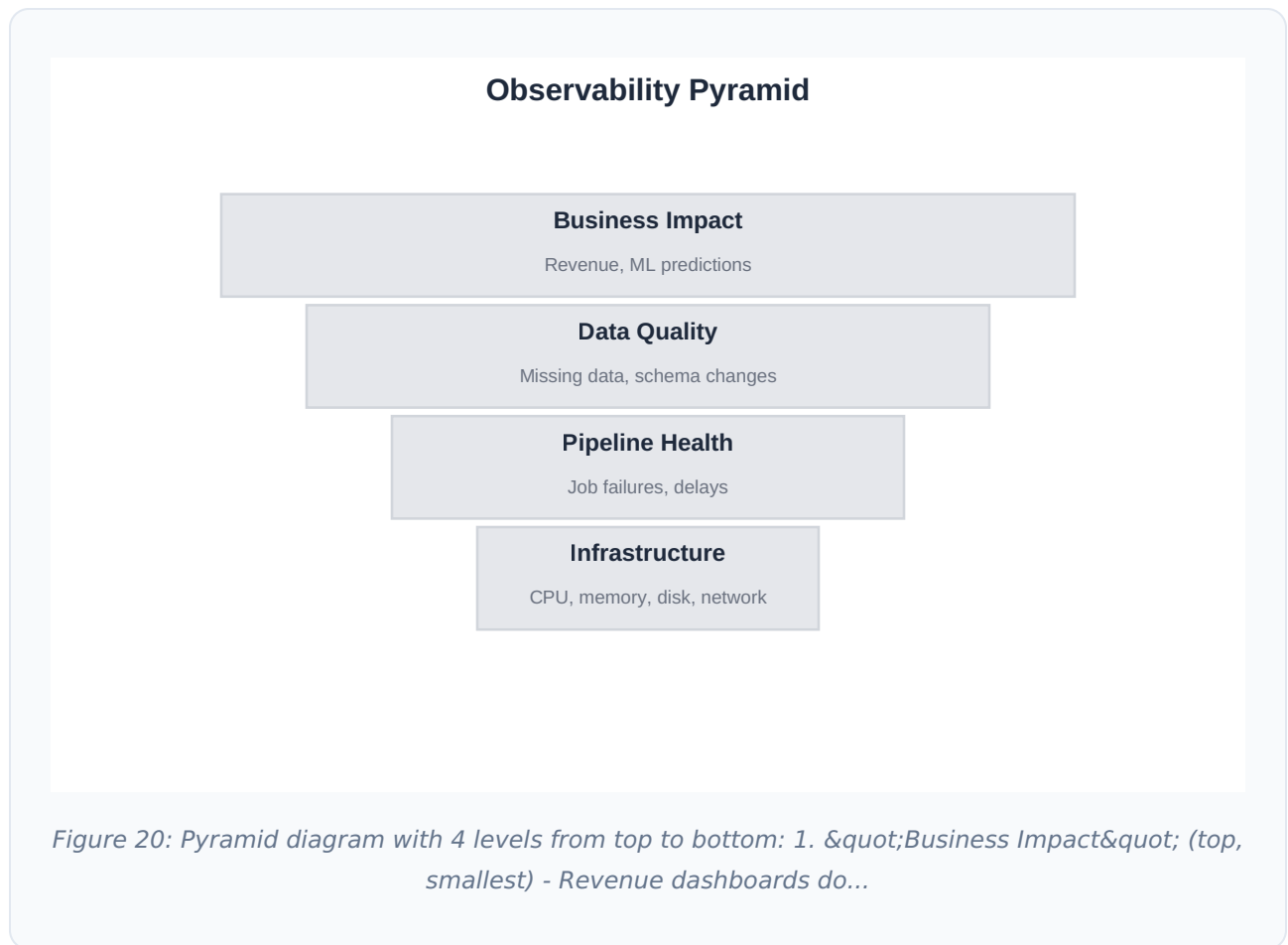
9.1 Data Pipeline Monitoring

- > **TL;DR** >
- Design SLAs around business impact, not technical metrics >
- Monitor the entire data flow, not just individual jobs >
- Alert on what matters: data freshness, completeness, and quality >
- Build dashboards that tell a story, not just show metrics

Most data teams monitor the wrong things. They track job success rates and resource utilization while missing the signals that actually matter to the business: Is the data fresh? Is it complete? Is it accurate?

After spending three years on-call for data infrastructure at two different companies, in practice, good monitoring isn't about collecting more metrics — it's about collecting the *right* metrics and alerting on what actually impacts users.

The Hierarchy of Data Pipeline Monitoring



****Level 1: Infrastructure Monitoring**** This is where most teams start and stop. CPU usage, memory consumption, disk space. Important for capacity planning, but tells you nothing about whether your business is getting value from the data.

****Level 2: Pipeline Health Monitoring**** Job success/failure rates, processing times, dependency tracking. This catches operational issues but not data issues.

****Level 3: Data Quality Monitoring**** Schema validation, completeness checks, anomaly detection. This is where you start monitoring the *data* instead of just the *jobs*.

****Level 4: Business Impact Monitoring**** Dashboard availability, model prediction accuracy, executive report delivery. This is what actually matters to the company.

> ****Pro Tip**** > Start monitoring from the top down, not bottom up. Figure out what business metrics you need to protect, then work backwards to identify the data and infrastructure signals that impact them.

SLA Design for Data Pipelines

****Bad SLA:**** "All jobs must complete successfully 99.5% of the time" ****Good SLA:****
"Customer-facing analytics data must be fresh within 2 hours of source system updates, with 99.9% availability during business hours"

Here's how to design SLAs that actually matter:

****1. Business Context First**** ```python # Example: E-commerce analytics SLAs slas = {

"revenue_dashboard": {

```
"freshness": "1 hour", # Executives check this hourly
"availability": "99.95%", # High-visibility, revenue-
```

critical

```
"completeness": "99%", # Some missing orders acceptable
"business_hours_only": False # Revenue happens 24/7
```

},

"weekly_cohort_analysis": {

```
"freshness": "24 hours", # Weekly cadence, daily refresh
```

sufficient

```
"availability": "99%", # Lower stakes than revenue
"completeness": "99.9%", # Historical analysis needs
```

complete data

```
"business_hours_only": True # Only checked during work
```

hours

},

"ml_feature_pipeline": {

```
"freshness": "15 minutes", # Real-time recommendations
"availability": "99.99%", # Directly impacts user
```

experience

```
"completeness": "95%", # Graceful degradation acceptable
"latency": "p95 < 500ms" # Real-time serving requirement
```

```
} }
```

- 2. Measurable Metrics
- Freshness: Maximum age of data (e.g., "2 hours old")
- Completeness: Percentage of expected records (e.g., "99% of orders")
- Availability: Uptime of data endpoints (e.g., "99.9%")
- Accuracy: Deviation from expected values (e.g., " $\pm 5\%$ of known truth")

3. Error Budgets and Alerting

```
# SLA monitoring implementation
class SLAMonitor:
    def __init__(self, sla_config):
        self.sla_config = sla_config

    def check_freshness(self, table_name):
        """Check if data is fresh enough"""
        sla = self.sla_config[table_name]
        last_update = get_table_last_update(table_name)
        max_age = datetime.now() -
```

PYTHON

```
timedelta(hours=sla['freshness_hours'])
```



```

if last_update < max_age:
    self.alert(f"FRESHNESS_SLA_BREACH", {
        'table': table_name,
        'last_update': last_update,
        'max_age': max_age,
        'severity': sla['severity']
    })

def check_completeness(self, table_name, expected_count):
    """Check if we received expected volume of data"""
    actual_count = get_table_row_count(table_name,

```

PYTHON

```

last_hour=True)

```

```

expected_min = expected_count * self.sla_config[table_name]

```

PYTHON

```

['completeness_threshold']

```

```

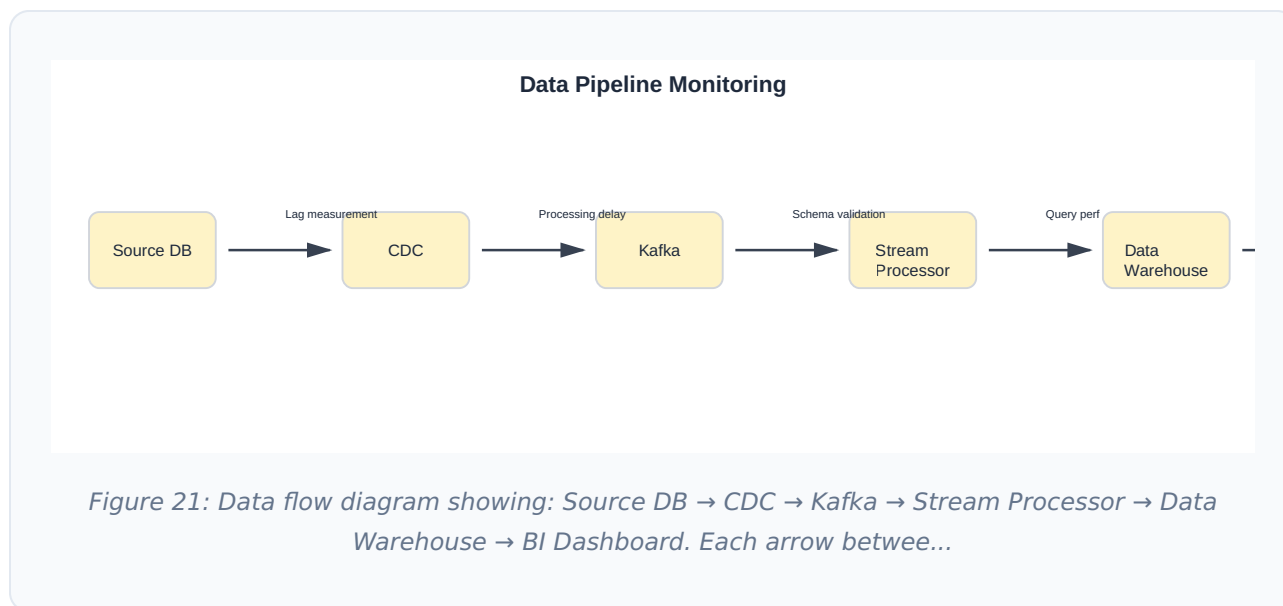
if actual_count < expected_min:
    self.alert(f"COMPLETENESS_SLA_BREACH", {
        'table': table_name,
        'actual': actual_count,
        'expected_min': expected_min,
        'completion_rate': actual_count / expected_count
    })

```

PYTHON

End-to-End Data Flow Monitoring

Most monitoring focuses on individual jobs. But data flows through multiple systems, and problems often manifest in the handoffs between systems.



Monitor the Handoffs:

```
-- Data lag monitoring across the pipeline
WITH pipeline_timing AS (
  SELECT
    'source_to_kafka' as stage,
    AVG(EXTRACT(EPOCH FROM kafka_timestamp - source_timestamp))
```

SQL

as avg_lag_seconds

```
FROM cdc_monitoring
WHERE timestamp > NOW() - INTERVAL '1 hour'
```

SQL

UNION ALL

```
SELECT
  'kafka_to_warehouse' as stage,
  AVG(EXTRACT(EPOCH FROM warehouse_timestamp -
```

SQL

kafka_timestamp)) as avg_lag_seconds

SQL

```
FROM stream_processing_monitoring
WHERE timestamp > NOW() - INTERVAL '1 hour'

UNION ALL

SELECT
    'warehouse_to_dashboard' as stage,
    AVG(EXTRACT(EPOCH FROM dashboard_refresh -
```

warehouse_timestamp)) as avg_lag_seconds

SQL

```
FROM dashboard_monitoring
    WHERE timestamp > NOW() - INTERVAL '1 hour'
)
SELECT
    stage,
    avg_lag_seconds,
    CASE
        WHEN avg_lag_seconds > 3600 THEN 'CRITICAL'
        WHEN avg_lag_seconds > 1800 THEN 'WARNING'
        ELSE 'OK'
    END as status
FROM pipeline_timing
ORDER BY avg_lag_seconds DESC;
```

Dependency Tracking

PYTHON

```
# Track pipeline dependencies and impact analysis
class DependencyGraph:
    def __init__(self):
        self.graph = nx.DiGraph()

    def add_pipeline(self, name, depends_on=None):
        self.graph.add_node(name)
        if depends_on:
            for dep in depends_on:
                self.graph.add_edge(dep, name)

    def get_downstream_impact(self, failed_pipeline):
        """Find all pipelines that could be impacted by a failure"""
        return list(nx.descendants(self.graph, failed_pipeline))

    def get_critical_path(self, target_pipeline):
        """Find the longest dependency chain to a pipeline"""
        paths = []
        for source in self.graph.nodes():
            if self.graph.in_degree(source) == 0: # Root nodes
                try:
                    path = nx.shortest_path(self.graph, source,
```

target_pipeline)

```
        paths.append(path)
        except nx.NetworkXNoPath:
            continue
        return max(paths, key=len) if paths else []

# Usage example
dep_graph = DependencyGraph()
dep_graph.add_pipeline("raw_orders", [])
dep_graph.add_pipeline("clean_orders", ["raw_orders"])
dep_graph.add_pipeline("customer_ltv", ["clean_orders",
```

"customer_dim"])

```
dep_graph.add_pipeline("executive_dashboard", ["customer_ltv",
```

```
"revenue_metrics"]])
```

```
# If raw_orders fails, what's impacted?  
impact = dep_graph.get_downstream_impact("raw_orders")  
# Result: ["clean_orders", "customer_ltv", "executive_dashboard"]
```

Real-World Alerting Strategies

The Golden Rule: Alert on symptoms, not causes.

❑ Bad Alert: "Kafka consumer lag is high" ❑ Good Alert: "Customer dashboard is 3 hours stale (SLA: 1 hour)"

Alert Categories and Response Times:

```

# Alert severity and escalation matrix
ALERT_MATRIX = {
  'P0_CRITICAL': {
    'description': 'Revenue-impacting data outage',
    'response_time': '15 minutes',
    'escalation': 'Immediate page + exec notification',
    'examples': [
      'Customer-facing analytics down',
      'Payment processing ML models failing',
      'Real-time fraud detection offline'
    ]
  },
  'P1_HIGH': {
    'description': 'Internal analytics disrupted',
    'response_time': '1 hour',
    'escalation': 'Slack alert + manager notification',
    'examples': [
      'Daily executive dashboard delayed',
      'Marketing attribution data missing',
      'A/B test results unavailable'
    ]
  },
  'P2_MEDIUM': {
    'description': 'Non-critical pipeline issues',
    'response_time': '4 hours',
    'escalation': 'Team Slack channel',
    'examples': [
      'Weekly cohort analysis delayed',
      'Data quality anomaly detected',
      'Non-essential dashboard queries slow'
    ]
  },
  'P3_LOW': {
    'description': 'Information and trending',
    'response_time': 'Next business day',
    'escalation': 'Email summary',
    'examples': [
      'Storage costs trending up',
      'Query optimization opportunities',
      'Capacity planning alerts'
    ]
  }
}

```

Intelligent Alerting with Context:

```
def generate_alert(metric_name, current_value, threshold, context):  
    """Generate contextual alerts that help with triage"""  
  
    alert = {  
        'metric': metric_name,  
        'value': current_value,  
        'threshold': threshold,  
        'timestamp': datetime.now(),  
        'severity': determine_severity(metric_name, current_value),  
        'context': {}  
    }  
  
    # Add business context  
    if metric_name == 'data_freshness':  
        alert['context']['business_impact'] =
```

```
get_dependent_dashboards(context['table'])
```

```
alert['context']['stakeholders'] =
```

```
get_table_stakeholders(context['table'])
```

```
alert['context']['historical_incidents'] =
```

```
get_similar_incidents(context['table'])
```

```
# Add technical context  
if context.get('pipeline_id'):  
    alert['context']['recent_changes'] =
```

```
get_recent_deployments(context['pipeline_id'])
```

```
alert['context']['dependencies'] =
```

```
get_failed_dependencies(context['pipeline_id'])
```

```
alert['context']['resource_usage'] =
```

```
get_current_resource_usage(context['pipeline_id'])
```

```
# Add suggested actions  
alert['suggested_actions'] =
```

```
generate_runbook_suggestions(metric_name, context)
```

```
return alert
```

□ Checkpoint

1. What's the difference between monitoring infrastructure vs monitoring business impact?
2. Why should SLAs be defined in terms of business requirements rather than technical metrics?
3. What are the key components of end-to-end data flow monitoring?

09

Monitoring, Observability & Data Quality

Build comprehensive monitoring and observability.
Master data quality frameworks, cost monitoring,
and security compliance. SLA Design · Data Quality
Framework · Observability · Cost Monitoring ·
Security & Compliance

Data Quality Framework Design

⚡ TL;DR

Implement expectation-based testing: define what good data looks like, then validate continuously - Catch data quality issues at ingestion time, not when reports break - Build quality scores that roll up from field-level to dataset-level to platform-level - Automate anomaly detection but keep humans in the loop for business context

I once spent a weekend debugging why our customer acquisition cost metrics were off by 40%. The issue? A schema change in our advertising platform API that changed cost from dollars to cents, but our pipeline didn't catch it. \$100 campaigns suddenly looked like \$1 campaigns.

That incident taught me that data quality isn't just about technical validation — it's about understanding your data's business semantics and monitoring for deviations from expected patterns.

The Pyramid of Data Quality



Level 1: Syntax Quality (Schema validation)

```
# Basic syntax validation
syntax_checks = {
    'order_id': 'NOT NULL AND LENGTH > 0',
    'order_amount': 'NUMERIC AND > 0',
    'order_date': 'DATE AND <= CURRENT_DATE',
    'customer_email': 'EMAIL_FORMAT OR NULL'
}
```

Level 2: Semantic Quality (Business rules)

```
# Semantic validation - business logic
semantic_checks = {
    'orders': [
        'order_amount <= customer_credit_limit + 1000',      # Allow
```

small overages

```
'product_id EXISTS IN products_dim',
'shipping_cost < order_amount * 0.5', # Shipping shouldn't
```

exceed 50% of order

```
],
    'customers': [
        'registration_date >= company_founding_date',
        'country_code IN (approved_countries)',
        'age BETWEEN 13 AND 120' # Reasonable human age range
    ]
}
```

Level 3: Temporal Quality (Time-based patterns)

```
# Temporal validation - patterns over time
temporal_checks = {
    'daily_order_volume': {
        'min_expected': 'AVG(last_7_days) * 0.3', # At least 30% of
```

recent average

```
'max_expected': 'AVG(last_7_days) * 3.0', # No more than 3x
```

recent average

```
'freshness': 'MAX_DELAY 2 hours'
},
'revenue_per_day': {
    'weekend_adjustment': 0.6, # Weekends typically 60% of
```

weekday volume

```
'holiday_adjustment': 0.3, # Holidays much lower
  'trend_analysis': 'WEEKLY_GROWTH between -20% and +50%'
}
```

Level 4: Contextual Quality (External validation)

```
# Contextual validation - cross-system checks
contextual_checks = {
  'advertising_spend': {
    'reconcile_with': ['facebook_ads_api', 'google_ads_api'],
    'tolerance': 0.05, # 5% variance acceptable
    'currency_consistency': True
  },
  'product_inventory': {
    'reconcile_with': ['warehouse_management_system'],
    'tolerance': 0.02, # 2% variance acceptable (some in-
```

transit)

```
'negative_check': 'inventory_count >= 0'
}
```

Expectation-Driven Data Quality

Instead of writing custom validation logic for every dataset, define expectations that can be applied systematically:

PYTHON

```
# Great Expectations style framework
class DataExpectations:
    def __init__(self, dataset_name):
        self.dataset_name = dataset_name
        self.expectations = []

    def expect_column_values_to_not_be_null(self, column):
        self.expectations.append({
            'type': 'not_null',
            'column': column,
            'description': f'{column} should never be null'
        })

    def expect_column_values_to_be_in_set(self, column, value_set):
        self.expectations.append({
            'type': 'in_set',
            'column': column,
            'value_set': value_set,
            'description': f'{column} should be one of {value_set}'
```

The PYTHON code:

PYTHON

```
    })

    def expect_column_values_to_match_regex(self, column,
```

regex_pattern):

PYTHON

```
        self.expectations.append({
            'type': 'regex',
            'column': column,
            'pattern': regex_pattern,
            'description': f'{column} should match pattern
```

{regex_pattern}'

PYTHON

```
    })

    def expect_table_row_count_to_be_between(self, min_value,
```

max_value):

```
self.expectations.append({
    'type': 'row_count_range',
    'min_value': min_value,
    'max_value': max_value,
    'description': f'Table should have between {min_value}
```

```
and {max_value} rows'
```

```
})
```

```
# Define expectations for orders table
orders_expectations = DataExpectations('orders')
orders_expectations.expect_column_values_to_not_be_null('order_id')
orders_expectations.expect_column_values_to_not_be_null('customer_id')

orders_expectations.expect_column_values_to_be_in_set('status',
```

```
['pending', 'shipped', 'delivered', 'cancelled'])
```

```
orders_expectations.expect_column_values_to_match_regex('email',
```

```
r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$')
```

```
# Validate batch
validation_result = orders_expectations.validate(orders_df)
```

Automated Anomaly Detection

Statistical Anomaly Detection:

```
import numpy as np
from scipy import stats
from sklearn.ensemble import IsolationForest

class DataAnomalyDetector:
    def __init__(self):
        self.models = {}
        self.historical_stats = {}

    def fit_normal_patterns(self, table_name, metric_history):
        """Learn what normal patterns look like for a metric"""
        # Fit statistical distribution
        mu, sigma = stats.norm.fit(metric_history)
        self.historical_stats[table_name] = {'mu': mu, 'sigma':
```

sigma}

```
# Fit isolation forest for multivariate anomaly detection
if len(metric_history.shape) > 1:
    iso_forest = IsolationForest(contamination=0.1,
```

```
random_state=42)
```

```
iso_forest.fit(metric_history)
    self.models[table_name] = iso_forest

def detect_anomalies(self, table_name, current_metrics):
    """Detect if current metrics are anomalous"""
    anomalies = []

    # Statistical outlier detection
    if table_name in self.historical_stats:
        stats_info = self.historical_stats[table_name]
        z_score = abs(current_metrics['value'] -
```

```
stats_info['mu']) / stats_info['sigma']
```



```

if z_score > 3: # 3-sigma rule
    anomalies.append({
        'type': 'statistical_outlier',
        'metric': current_metrics['name'],
        'z_score': z_score,
        'severity': 'high' if z_score > 4 else 'medium'
    })

# Machine learning anomaly detection
if table_name in self.models:
    anomaly_score =

```

```
self.models[table_name].decision_function([current_metrics['features']]) [0]
```

```

if anomaly_score < -0.5: # Threshold for anomaly
    anomalies.append({
        'type': 'multivariate_anomaly',
        'metric': current_metrics['name'],
        'anomaly_score': anomaly_score,
        'severity': 'high' if anomaly_score < -0.7 else

```

```
'medium'
```

```

    })

    return anomalies

# Example usage
detector = DataAnomalyDetector()

# Train on historical data
daily_orders_history = get_daily_order_metrics(last_90_days=True)
detector.fit_normal_patterns('daily_orders', daily_orders_history)

# Detect anomalies in today's data
today_metrics = {
    'name': 'daily_order_count',
    'value': get_today_order_count(),
    'features': [order_count, avg_order_value, unique_customers,

```

```
return_rate]
```

```
}

anomalies = detector.detect_anomalies('daily_orders', today_metrics)
```

Business-Aware Anomaly Detection:

```
def business_aware_anomaly_detection(table_name, current_date,
```

PYTHON

metrics):

```
"""Anomaly detection that understands business context"""
anomalies = []

# Check if it's a known special day
special_days = get_special_business_days(current_date)
if special_days:
    # Adjust expectations for holidays, sales events, etc.
    expected_variance = get_special_day_variance(special_days)
    threshold_multiplier = expected_variance
else:
    threshold_multiplier = 1.0

# Day-of-week patterns
day_of_week = current_date.strftime('%A')
historical_pattern = get_day_of_week_pattern(table_name,
```

day_of_week)

```
    expected_range = (
        historical_pattern['mean'] - (historical_pattern['std'] * 2
```

* threshold_multiplier),

```
        historical_pattern['mean'] + (historical_pattern['std'] * 2
```

* threshold_multiplier)

```
)  
  
if metrics['value'] < expected_range[0] or metrics['value'] >
```

```
expected_range[1]:
```

```
anomalies.append({
```

```
    'type': 'business_pattern_deviation',  
    'expected_range': expected_range,  
    'actual_value': metrics['value'],  
    'context': {  
        'day_of_week': day_of_week,  
        'special_days': special_days,  
        'historical_pattern': historical_pattern  
    }  
})  
  
return anomalies
```

Data Quality Scoring and Reporting

Multi-Level Quality Scores:

```
class DataQualityScore:  
    def __init__(self):  
        self.weights = {  
            'completeness': 0.3,    # How much data is missing?  
            'accuracy': 0.3,        # How correct is the data?  
            'consistency': 0.2,     # Is data consistent across
```

PYTHON

sources?

```
'timeliness': 0.2      # How fresh is the data?
    }

def calculate_field_score(self, field_name, field_data):
    """Calculate quality score for individual field"""
    scores = {}

    # Completeness: % of non-null values
    scores['completeness'] = (field_data.count() /
```

```
len(field_data)) * 100
```

```
# Accuracy: % of values passing validation rules
validation_rules = get_field_validation_rules(field_name)
valid_count = sum(1 for value in field_data if
```

```
validate_field(value, validation_rules))
```

```
scores['accuracy'] = (valid_count / len(field_data)) * 100

# Consistency: % of values matching expected format/pattern
pattern_match_count = sum(1 for value in field_data if
```

```
matches_expected_pattern(value, field_name))
```

```
scores['consistency'] = (pattern_match_count /
```

```
len(field_data)) * 100
```

```
# Timeliness: Age of data relative to expectations
data_age = get_field_data_age(field_name)
expected_freshness = get_expected_freshness(field_name)
scores['timeliness'] = max(0, 100 - (data_age /
```

```
expected_freshness * 100))
```

```
# Weighted overall score
overall_score = sum(scores[dim] * self.weights[dim] for dim
```

in scores)

```
return {
    'field_name': field_name,
    'overall_score': round(overall_score, 2),
    'dimension_scores': scores,
    'sample_issues': get_field_quality_issues(field_name,
```

field_data)

```
}

def calculate_table_score(self, table_name):
    """Roll up field scores to table level"""
    table_data = get_table_data(table_name)
    field_scores = []

    for field in table_data.columns:
        field_score = self.calculate_field_score(field,
```

table_data[field])

```
field_scores.append(field_score)

# Weight field scores by business importance
field_weights = get_field_business_weights(table_name)

weighted_score = sum(
    score['overall_score'] *
```

field_weights.get(score['field_name'], 1.0)

```

for score in field_scores
) / sum(field_weights.values())

return {
    'table_name': table_name,
    'overall_score': round(weighted_score, 2),
    'field_scores': field_scores,
    'record_count': len(table_data),
    'critical_issues': [score for score in field_scores if

```

```
score['overall_score'] < 80]
```

```

}

def calculate_platform_score(self):
    """Roll up table scores to platform level"""
    all_tables = get_monitored_tables()
    table_scores = []

    for table in all_tables:
        table_score = self.calculate_table_score(table)
        table_scores.append(table_score)

    # Weight by business criticality
    table_weights = get_table_business_criticality()

    platform_score = sum(
        score['overall_score'] *

```

PYTHON

```
table_weights.get(score['table_name'], 1.0)
```

```

for score in table_scores
) / sum(table_weights.values())

return {
    'platform_score': round(platform_score, 2),
    'table_scores': table_scores,
    'total_tables': len(all_tables),
    'tables_below_threshold': len([s for s in table_scores

```

```
if s['overall_score'] < 85])
```

```
}
```

Quality Dashboard Generation:

```
-- Executive data quality dashboard query
WITH table_quality_summary AS (
    SELECT
        table_name,
        AVG(quality_score) as avg_quality_score,
        COUNT(CASE WHEN quality_score < 80 THEN 1 END) as
```

SQL

critical_issues,

```
COUNT(CASE WHEN quality_score < 90 THEN 1 END) as
```

warning_issues,

```
MAX(last_updated) as last_quality_check
FROM data_quality_metrics
WHERE date >= CURRENT_DATE - INTERVAL '7 days'
GROUP BY table_name
),
business_impact AS (
    SELECT
        t.table_name,
        t.avg_quality_score,
        t.critical_issues,
        b.business_criticality,
        b.stakeholder_teams,
        CASE
            WHEN t.avg_quality_score < 80 AND b.business_criticality
```

SQL

```
= 'HIGH' THEN 'URGENT'
```

```
WHEN t.avg_quality_score < 85 AND b.business_criticality
```

```
= 'MEDIUM' THEN 'IMPORTANT'
```

ELSE 'MONITORING'

```
END as priority_level
FROM table_quality_summary t
JOIN business_metadata b ON t.table_name = b.table_name
)
SELECT
    priority_level,
    COUNT(*) as table_count,
    AVG(avg_quality_score) as avg_score,
    STRING_AGG(table_name, ', ') as affected_tables
FROM business_impact
GROUP BY priority_level
ORDER BY
    CASE priority_level
        WHEN 'URGENT' THEN 1
        WHEN 'IMPORTANT' THEN 2
        ELSE 3
    END;

```

SQL

Circuit Breakers and Quality Gates

Data Quality Circuit Breaker:

```
class DataQualityCircuitBreaker:
    def __init__(self, table_name, failure_threshold=5,
```

PYTHON

reset_timeout=300):

```
self.table_name = table_name
self.failure_threshold = failure_threshold
self.reset_timeout = reset_timeout
self.failure_count = 0
self.last_failure_time = None
self.state = 'CLOSED' # CLOSED -> OPEN -> HALF_OPEN ->
```

PYTHON

CLOSED


```
def call_pipeline(self, pipeline_function, data):
    """Execute pipeline with circuit breaker protection"""
    if self.state == 'OPEN':
        if time.time() - self.last_failure_time >
```

PYTHON

self.reset_timeout:

```
self.state = 'HALF_OPEN'
else:
    raise CircuitBreakerOpen(f"Circuit breaker open for
```

PYTHON

```
{self.table_name}")
```

```
try:
    # Validate data quality before processing
    quality_check = self.validate_data_quality(data)

    if quality_check['overall_score'] < 70:    # Quality
```

PYTHON

threshold

```
raise DataQualityException(f"Data quality too low:
```

```
{quality_check}")
```

```
# Execute the pipeline
result = pipeline_function(data)

# Reset failure count on success
self.failure_count = 0
if self.state == 'HALF_OPEN':
    self.state = 'CLOSED'

return result

except (DataQualityException, PipelineException) as e:
    self.failure_count += 1
    self.last_failure_time = time.time()

    if self.failure_count >= self.failure_threshold:
        self.state = 'OPEN'
        self.send_circuit_breaker_alert()
```

The PYTHON code:

```

raise e

def validate_data_quality(self, data):
    """Quick data quality validation"""
    return {
        'completeness': calculate_completeness(data),
        'row_count': len(data),
        'schema_valid': validate_schema(data),
        'overall_score': calculate_quick_quality_score(data)
    }

def send_circuit_breaker_alert(self):
    """Alert that circuit breaker has opened"""
    alert = {
        'type': 'CIRCUIT_BREAKER_OPEN',
        'table': self.table_name,
        'failure_count': self.failure_count,
        'last_failure': self.last_failure_time,
        'action': 'Pipeline stopped to prevent data corruption'
    }
    send_alert(alert)

# Usage in data pipeline
breaker = DataQualityCircuitBreaker('customer_orders')

def process_daily_orders():
    raw_orders = extract_orders_from_source()

    # Circuit breaker protects downstream systems
    processed_orders = breaker.call_pipeline(
        pipeline_function=clean_and_transform_orders,
        data=raw_orders
    )

    load_to_warehouse(processed_orders)

```

□ Checkpoint

1. What's the difference between syntax, semantic, temporal, and contextual data quality?
2. How does expectation-driven testing differ from traditional data validation?
3. Why might you want a circuit breaker pattern in a data pipeline?

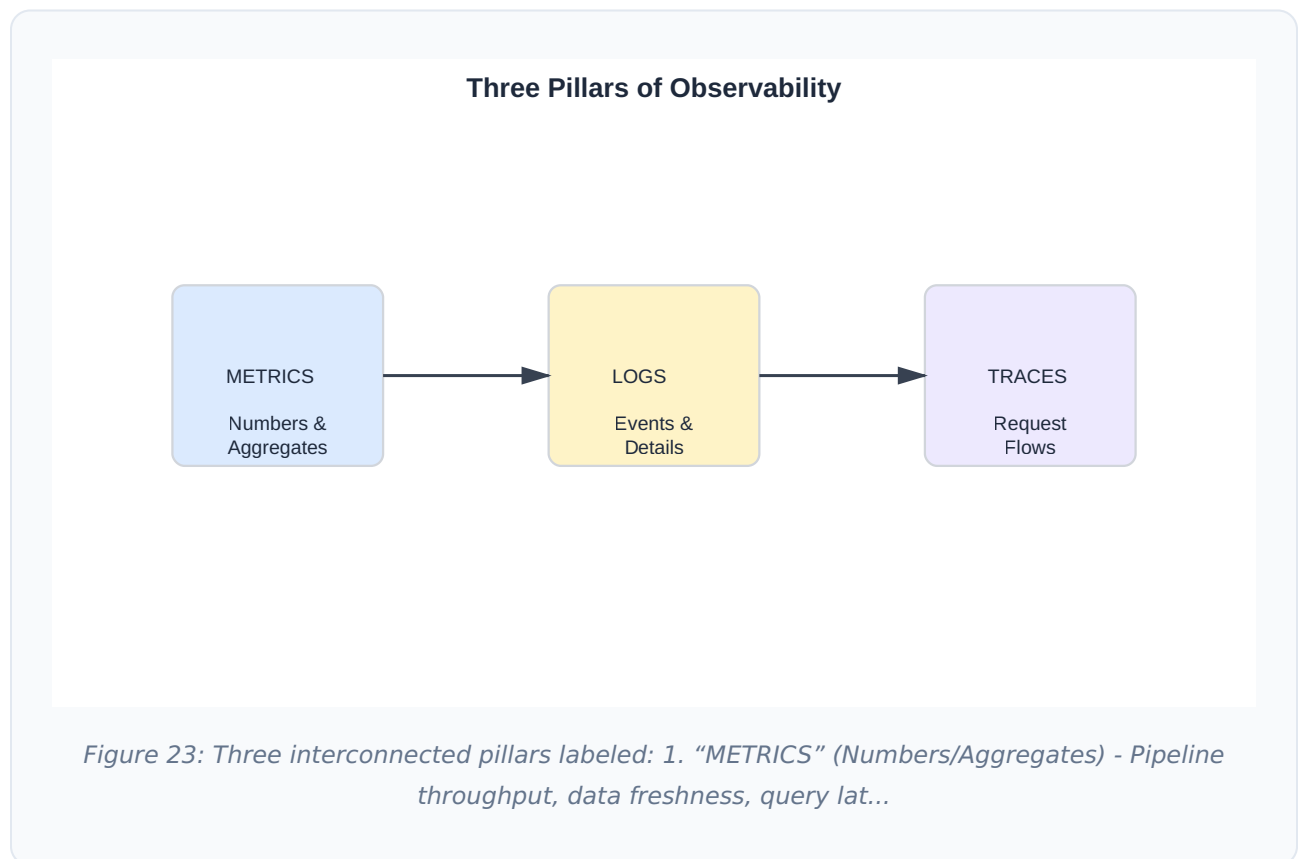
⚡ TL;DR

Implement the three pillars: Metrics, Logs, and Traces for complete visibility - Focus on data lineage tracking to understand end-to-end impact - Build performance monitoring that connects query patterns to infrastructure costs - Use distributed tracing to debug complex data pipeline issues

Traditional monitoring tells you what happened. Observability tells you why it happened. For data systems, this means being able to trace a data quality issue from a dashboard all the way back to its source, understand the performance impact of different query patterns, and see how changes propagate through your entire data infrastructure.

After debugging countless data issues across distributed systems, in practice, good observability isn't about having more dashboards — it's about having the right context when things go wrong.

The Three Pillars of Data Observability



transformations, user queries, system errors 3. "TRACES" (Relationships/Flow) - Data lineage, cross-system dependencies, end-to-end request flow]

Metrics: The Numbers That Matter

```
# Key data pipeline metrics to track
DATA_PIPELINE_METRICS = {
    'throughput': {
        'records_per_second': 'rate(records_processed[5m])',
        'bytes_per_second': 'rate(bytes_processed[5m])',
        'pipelines_per_hour': 'rate(pipeline_executions[1h])'
    },
    'latency': {
        'end_to_end_latency': 'histogram_quantile(0.95,
```

pipeline_duration_seconds)',

```
'processing_latency': 'histogram_quantile(0.95,
```

job_duration_seconds)',

```
'data_freshness': 'time() - max(last_update_timestamp)'
},
'errors': {
  'failure_rate': 'rate(pipeline_failures[5m]) /
```

```
rate(pipeline_executions[5m])',
```

```
'data_quality_violations':
```

```
'sum(quality_check_failures[1h])',
```

```
'schema_validation_errors': 'rate(schema_errors[5m])'
},
'saturation': {
  'cpu_utilization': 'avg(cpu_usage_percent)',
  'memory_utilization': 'avg(memory_usage_percent)',
  'queue_depth': 'avg(kafka_consumer_lag)',
  'storage_utilization': 'storage_used_bytes /
```

storage_total_bytes'

```
}
}
```

Logs: The Context Behind the Numbers

```
import structlog
import json

# Structured logging for data pipelines
logger = structlog.get_logger()

class DataPipelineLogger:
    def __init__(self, pipeline_name, run_id):
        self.pipeline_name = pipeline_name
        self.run_id = run_id
        self.context = {
            'pipeline_name': pipeline_name,
            'run_id': run_id,
            'component': 'data_pipeline'
        }

    def log_pipeline_start(self, input_sources, expected_output):
        logger.info(
            "Pipeline execution started",
            **self.context,
            input_sources=input_sources,
            expected_output=expected_output,
            start_time=time.time()
        )

    def log_data_quality_check(self, check_name, result, details):
        logger.info(
            "Data quality check completed",
            **self.context,
            check_name=check_name,
            passed=result['passed'],
            score=result.get('score'),
            violations=result.get('violations', []),
            sample_records=details.get('sample_records', [])
        )
```

PYTHON

The log_transformation_step implementation:

```
def log_transformation_step(self, step_name, input_count,
                             output_count, duration):
```

PYTHON

PYTHON

```

logger.info(
    "Transformation step completed",
    **self.context,
    step_name=step_name,
    input_record_count=input_count,
    output_record_count=output_count,
    duration_seconds=duration,
    throughput_rps=output_count / duration if duration > 0

```

else 0

PYTHON

```

)

def log_pipeline_error(self, error_type, error_message,

```

stack_trace, recovery_action):

PYTHON

```

logger.error(
    "Pipeline execution failed",
    **self.context,
    error_type=error_type,
    error_message=error_message,
    stack_trace=stack_trace,
    recovery_action=recovery_action,
    retry_count=getattr(self, 'retry_count', 0)
)

# Usage example
pipeline_logger = DataPipelineLogger('customer_ltv_calculation',

```

```
run_id='run_20240315_143022')
```

```

pipeline_logger.log_pipeline_start(
    input_sources=['orders', 'customers', 'product_purchases'],
    expected_output={'table': 'customer_ltv', 'estimated_rows':

```

50000}


```

)

try:
    # Execute pipeline steps with logging
    raw_data = extract_source_data()
    pipeline_logger.log_transformation_step(
        step_name='data_extraction',
        input_count=0, # Source system
        output_count=len(raw_data),
        duration=extraction_time
    )

    cleaned_data = clean_and_validate(raw_data)
    quality_result = run_quality_checks(cleaned_data)
    pipeline_logger.log_data_quality_check('standard_validation',

```

quality_result, {})

```

except Exception as e:
    pipeline_logger.log_pipeline_error(
        error_type=type(e).__name__,
        error_message=str(e),
        stack_trace=traceback.format_exc(),
        recovery_action='retry_with_reduced_batch_size'
    )

```

Traces: End-to-End Request Flow

```

import opentelemetry
from opentelemetry import trace
from opentelemetry.exporter.jaeger.thrift import JaegerExporter
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor

# Set up distributed tracing for data pipelines
tracer = trace.get_tracer(__name__)

class DataPipelineTracer:
    def __init__(self, pipeline_name):
        self.pipeline_name = pipeline_name

```

PYTHON

The trace_data_flow implementation:

```

def trace_data_flow(self, trace_id=None):
    """Context manager for tracing data flow through pipeline"""
    return tracer.start_as_current_span(
        f"data_pipeline.{self.pipeline_name}",
        attributes={
            'pipeline.name': self.pipeline_name,
            'pipeline.type': 'batch_processing'
        }
    )

def trace_transformation(self, step_name, input_data):
    """Trace individual transformation steps"""
    with tracer.start_as_current_span(f"transform.{step_name}")

```

as span:

```

span.set_attributes({
    'transform.step_name': step_name,
    'transform.input_size': len(input_data),
    'transform.input_schema':

```

```

get_schema_fingerprint(input_data)

```

```

}))

try:
    result = execute_transformation(step_name,

```

input_data)

```

span.set_attributes({
    'transform.output_size': len(result),
    'transform.success': True
})
return result

except Exception as e:
    span.record_exception(e)
    span.set_attributes({'transform.success': False})
    raise

# Usage with full end-to-end tracing
def process_customer_data_with_tracing():
    tracer = DataPipelineTracer('customer_enrichment')

    with tracer.trace_data_flow() as main_span:
        # Extract data with child span
        with tracer.trace_transformation('extract_customers', []) as

```

extract_span:

```

customers = extract_from_database()
extract_span.set_attribute('source.table', 'customers')
extract_span.set_attribute('source.record_count',

```

```

len(customers))

```

```

# Transform data with tracing
enriched_customers =

```

```

tracer.trace_transformation('enrich_with_orders', customers)

```

```

final_customers =

```

```

tracer.trace_transformation('calculate_ltv', enriched_customers)

```

```

# Load with tracing
with tracer.trace_transformation('load_to_warehouse',

```

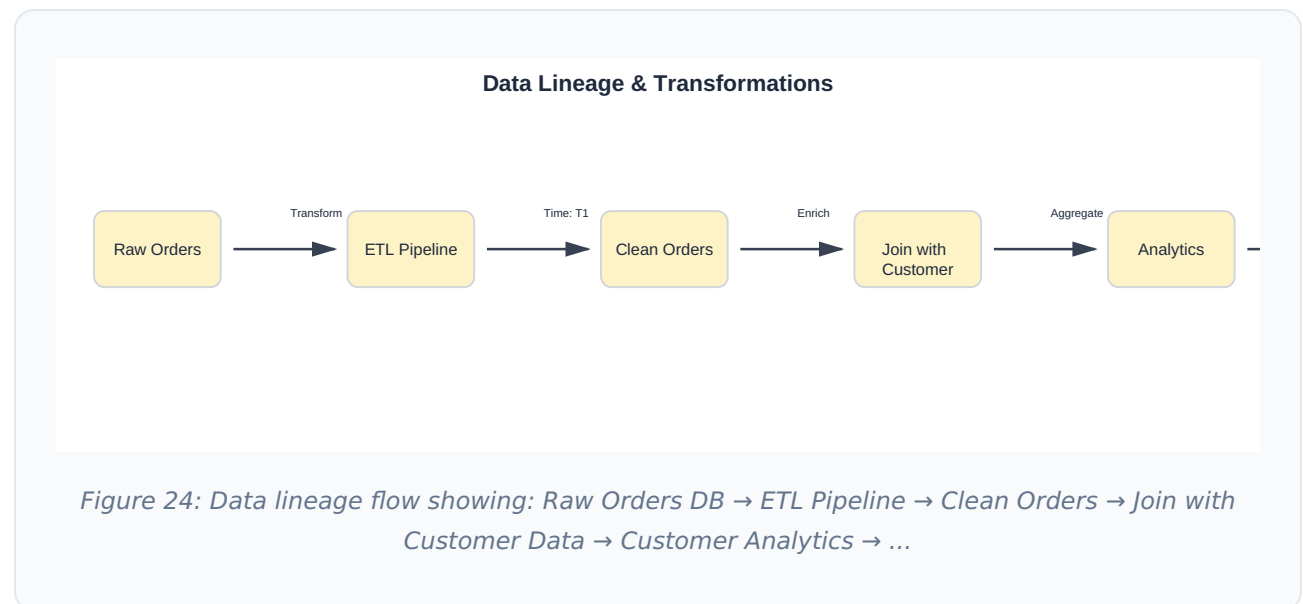
final_customers) as load_span:

```
load_to_target(final_customers)
load_span.set_attribute('target.table',
```

```
'customer_analytics')
```

Data Lineage Tracking

Understanding how data flows through your system is for impact analysis, debugging, and compliance.



Automatic Lineage Capture:

```

class DataLineageTracker:
    def __init__(self, graph_storage):
        self.graph = graph_storage # Neo4j, NetworkX, etc.

    def track_data_read(self, source_name, query, record_count):
        """Track when data is read from a source"""
        lineage_event = {
            'event_type': 'DATA_READ',
            'timestamp': datetime.now(),
            'source': source_name,
            'query': query,
            'record_count': record_count,
            'pipeline_run_id': get_current_run_id(),
            'user': get_current_user()
        }
        self.graph.add_lineage_edge(source_name,

```

```

get_current_run_id(), lineage_event)

```

```

def track_transformation(self, input_datasets, output_dataset,

```

transformation_code):

```

"""Track data transformation with lineage"""
lineage_event = {
    'event_type': 'TRANSFORMATION',
    'timestamp': datetime.now(),
    'inputs': input_datasets,
    'output': output_dataset,
    'transformation': {
        'type': 'sql_transform',
        'code_hash':

```

```

hashlib.md5(transformation_code.encode()).hexdigest(),

```

SQL

```

'code': transformation_code
    },
    'pipeline_run_id': get_current_run_id()
}

# Create lineage edges from inputs to output
for input_ds in input_datasets:
    self.graph.add_lineage_edge(input_ds, output_dataset,

```

lineage_event)

PYTHON

```

def track_data_write(self, target_name, record_count, schema):
    """Track when data is written to a target"""
    lineage_event = {
        'event_type': 'DATA_WRITE',
        'timestamp': datetime.now(),
        'target': target_name,
        'record_count': record_count,
        'schema': schema,
        'pipeline_run_id': get_current_run_id()
    }
    self.graph.add_lineage_edge(get_current_run_id(),

```

target_name, lineage_event)

PYTHON

```

def get_downstream_impact(self, dataset_name):
    """Find all datasets that depend on this one"""
    return self.graph.find_downstream_dependencies(dataset_name)

def get_data_ancestry(self, dataset_name, max_depth=5):
    """Trace data back to its original sources"""
    return self.graph.find_upstream_dependencies(dataset_name,

```

max_depth)

```
# Integration with SQL execution
class LineageAwareSQL:
    def __init__(self, lineage_tracker):
        self.lineage_tracker = lineage_tracker

    def execute_with_lineage(self, query, output_table=None):
        """Execute SQL and automatically capture lineage"""
```

Parse SQL to extract source tables:

```
# Parse SQL to extract source tables
parsed = sqlparse.parse(query)[0]
source_tables = self.extract_source_tables(parsed)

# Track data reads
for table in source_tables:
    record_count = self.get_table_count(table)
    self.lineage_tracker.track_data_read(table, query,
```

record_count)

```
# Execute query
result = self.execute_query(query)

# Track transformation and write
if output_table:
    self.lineage_tracker.track_transformation(
        input_datasets=source_tables,
        output_dataset=output_table,
        transformation_code=query
    )
    self.lineage_tracker.track_data_write(
        target_name=output_table,
        record_count=len(result),
        schema=self.get_result_schema(result)
    )

return result
```

Lineage Query Interface:

```
-- Neo4j Cypher queries for lineage analysis
```

```
-- Find all downstream datasets impacted by a source change  
MATCH (source:Dataset {name: 'raw_orders'})-[:FLOWS_TO*]->
```

```
(downstream:Dataset)
```

```
RETURN downstream.name, downstream.last_updated,
```

```
downstream.owner_team
```

```
-- Find the complete ancestry of a dataset  
MATCH path = (origin:Dataset)-[:FLOWS_TO*]->(target:Dataset {name:
```

```
'customer_ltv_model'})
```

```
WHERE NOT ()-[:FLOWS_TO]->(origin) // origin has no upstream
```

SQL

dependencies

RETURN path

```
-- Find datasets that haven't been updated recently but feed
```

critical dashboards

```
MATCH (dataset:Dataset)-[:FLOWS_TO*]->(dashboard:Dashboard
```

```
{criticality: 'HIGH'})
```

```
WHERE dataset.last_updated < datetime() - duration({days: 2})  
RETURN dataset.name, dataset.last_updated, collect(dashboard.name)
```

SQL

as impacted_dashboards

Performance Monitoring and Cost Attribution

Query Pattern Analysis:

```
class QueryPerformanceMonitor:
    def __init__(self, warehouse_connection):
        self.warehouse = warehouse_connection

    def analyze_query_patterns(self, time_window_hours=24):
        """Analyze query patterns and performance"""
        query = """
        SELECT
            query_hash,
            COUNT(*) as execution_count,
            AVG(execution_time_ms) as avg_duration,
            MAX(execution_time_ms) as max_duration,
            SUM(bytes_scanned) as total_bytes_scanned,
            SUM(credits_used) as total_cost,
            AVG(credits_used) as avg_cost_per_query,
```

PYTHON

The corresponding SQL query:

```
user_name,
    warehouse_size,
    query_type
FROM query_history
WHERE start_time >= CURRENT_TIMESTAMP - INTERVAL
```

SQL

'{time_window_hours} HOURS'

```

GROUP BY query_hash, user_name, warehouse_size, query_type
HAVING execution_count > 1 -- Focus on repeated queries
ORDER BY total_cost DESC, execution_count DESC
"".format(time_window_hours=time_window_hours)

return self.warehouse.execute(query)

def identify_optimization_opportunities(self):
    """Find queries that could benefit from optimization"""
    opportunities = []

    query_stats = self.analyze_query_patterns()

    for query in query_stats:
        # High-cost, frequently run queries
        if query['total_cost'] > 10 and query['execution_count']

```

> 50:

```

opportunities.append({
    'type': 'high_cost_frequent',
    'query_hash': query['query_hash'],
    'potential_savings': query['total_cost'] * 0.5,

```

Assume 50% optimization

```

'recommendation': 'Consider materialized view or

```

result caching'

```

}))

# Queries with high variance in execution time
if query['max_duration'] > query['avg_duration'] * 3:
    opportunities.append({
        'type': 'inconsistent_performance',
        'query_hash': query['query_hash'],
        'recommendation': 'Review clustering keys and

```

query patterns'

```

}))

# Queries scanning excessive data
avg_scan_ratio = query['total_bytes_scanned'] /

```

```

(query['execution_count'] * 1_000_000) # MB per query

```

```

if avg_scan_ratio > 1000: # More than 1GB average
    opportunities.append({
        'type': 'excessive_data_scan',
        'query_hash': query['query_hash'],
        'avg_scan_gb': avg_scan_ratio / 1000,
        'recommendation': 'Add partition pruning or

```

column selection optimization'

```

}))

    return opportunities

# Cost attribution by team/project
def generate_cost_attribution_report():
    """Break down data platform costs by team and project"""

    query = """
    WITH user_costs AS (
        SELECT
            u.user_name,
            u.team,
            u.department,
            SUM(qh.credits_used) as total_credits,
            COUNT(DISTINCT qh.query_hash) as unique_queries,
            SUM(qh.execution_time_ms) / 1000 / 3600 as compute_hours
        FROM query_history qh
        JOIN users u ON qh.user_name = u.user_name
        WHERE qh.start_time >= CURRENT_DATE - INTERVAL '30 days'
        GROUP BY u.user_name, u.team, u.department
    """

```

SQL

The corresponding SQL query:

SQL

```

),
storage_costs AS (
    SELECT
        table_owner_team as team,
        SUM(table_size_gb * 23.0) as storage_cost_usd    --

```

\$23/TB/month

SQL

```

FROM table_metadata
    GROUP BY table_owner_team
)
SELECT
    uc.team,
    uc.department,
    SUM(uc.total_credits * 3.0) as compute_cost_usd, --

```

\$3/credit estimate

```

COALESCE(sc.storage_cost_usd, 0) as storage_cost_usd,
SUM(uc.total_credits * 3.0) + COALESCE(sc.storage_cost_usd,

```

0) as total_cost_usd,

SQL

```

SUM(uc.compute_hours) as total_compute_hours,
    COUNT(DISTINCT uc.user_name) as active_users
FROM user_costs uc
LEFT JOIN storage_costs sc ON uc.team = sc.team
GROUP BY uc.team, uc.department, sc.storage_cost_usd
ORDER BY total_cost_usd DESC
"""

return execute_cost_query(query)

```

□ Checkpoint

1. How do the three pillars of observability (metrics, logs, traces) work together?
2. Why is data lineage tracking important for debugging and compliance?
3. How can you use query performance monitoring to optimize data platform costs?

⚡ TL;DR

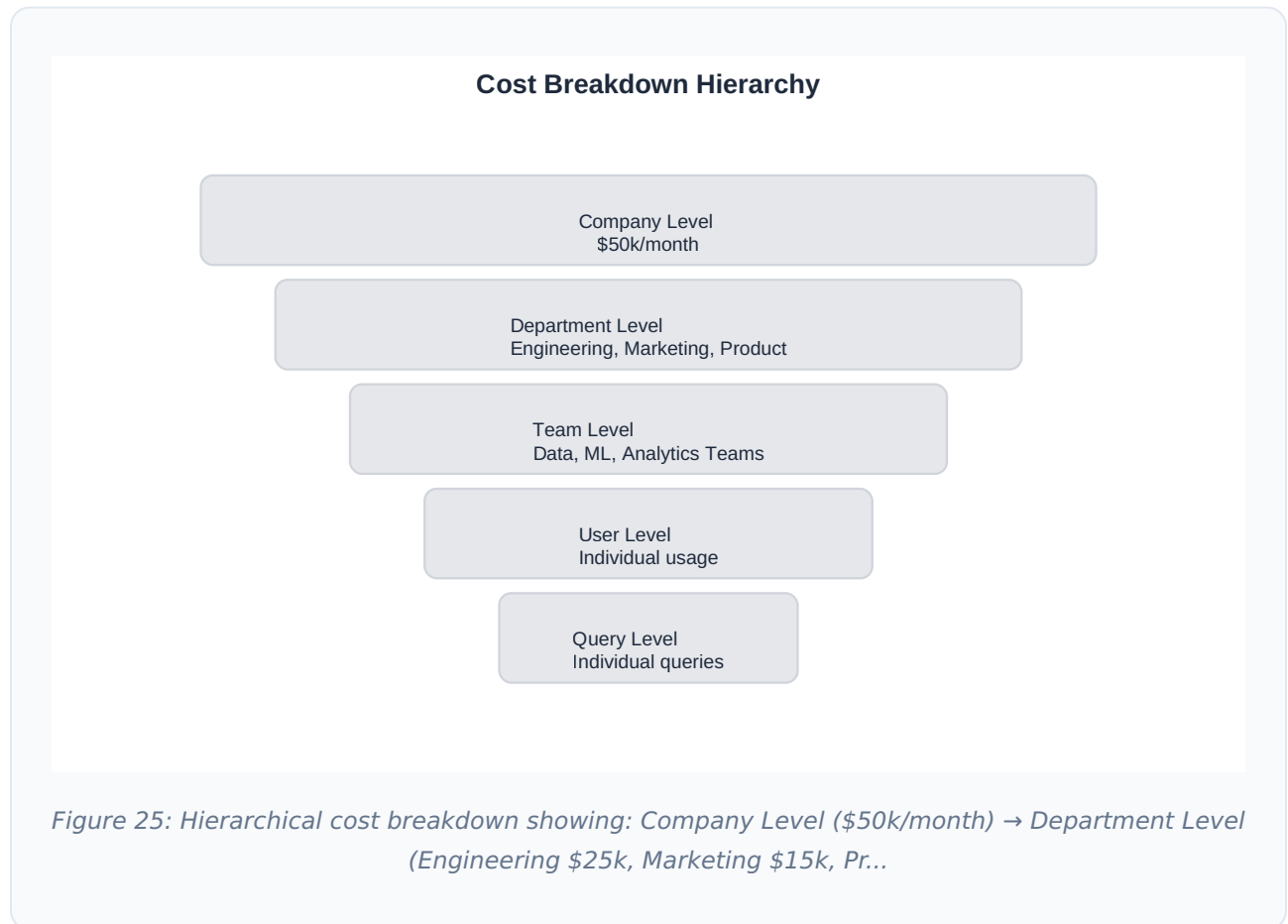
- Track costs at multiple levels: query, user, team, project to enable accountability
- Implement cost guardrails before problems happen (query timeout, resource limits)
- Use automated optimization: result caching, materialized views, workload scheduling
- Build cost visibility into everyday workflows, not just monthly reports

Data platform costs can spiral out of control fast. it's common to see monthly bills jump from \$10k to \$100k because someone ran an unoptimized query against a petabyte table, or forgot to turn off a warehouse that was auto-scaling.

The key to cost control isn't restricting access — it's building cost

awareness into your platform so teams can make informed trade-offs between speed, convenience, and cost.

Multi-Level Cost Tracking



Query-Level Cost Attribution:

```
class QueryCostTracker:
    def __init__(self, warehouse_type='snowflake'):
        self.warehouse_type = warehouse_type
        self.cost_models = {
            'snowflake': {
```

PYTHON

The PYTHON code:

```

'compute_credit_cost': 3.0, # $3 per credit
    'storage_cost_per_gb_month': 0.023, # $23/TB/month
},
'bigquery': {
    'on_demand_per_tb': 5.0, # $5 per TB scanned
    'storage_cost_per_gb_month': 0.020, # $20/TB/month
}
}

def calculate_query_cost(self, query_metadata):
    """Calculate actual cost of a single query"""
    costs = {}

    if self.warehouse_type == 'snowflake':
        # Compute cost based on warehouse size and duration
        credits_used = (
            query_metadata['execution_time_seconds'] / 3600 #

```

hours

```

* query_metadata['warehouse_credits_per_hour']
)
costs['compute'] = credits_used *

```

```

self.cost_models['snowflake']['compute_credit_cost']

```

```

# Storage scanning cost (if applicable)
if query_metadata.get('bytes_scanned'):
    # Snowflake doesn't charge for scanning, but track

```

for optimization

```

costs['data_scanned_gb'] =

```

```

query_metadata['bytes_scanned'] / (1024**3)

```

```

elif self.warehouse_type == 'bigquery':
    # On-demand pricing based on bytes processed
    tb_scanned = query_metadata['bytes_processed'] /

```



```
(1024**4) # TB
```

```
costs['compute'] = tb_scanned *
```

```
self.cost_models['bigquery']['on_demand_per_tb']
```

PYTHON

```
costs['total'] = sum(costs.values())
costs['query_hash'] = query_metadata['query_hash']
costs['user'] = query_metadata['user_name']
costs['timestamp'] = query_metadata['start_time']

return costs

def track_query_cost(self, query_metadata):
    """Store query cost for analysis"""
    cost_info = self.calculate_query_cost(query_metadata)

    # Store in cost tracking table
    insert_query = """
    INSERT INTO query_costs (
        query_hash, user_name, team, department,
        execution_time, bytes_processed, compute_cost,
        total_cost, warehouse_size, timestamp
    ) VALUES (
        %(query_hash)s, %(user)s, %(team)s, %(department)s,
        %(execution_time)s, %(bytes_processed)s, %
```

```
(compute_cost)s,
```

```
%(total_cost)s, %(warehouse_size)s, %(timestamp)s
    )
    """

    execute_query(insert_query, {
        'query_hash': cost_info['query_hash'],
        'user': cost_info['user'],
        'team': get_user_team(cost_info['user']),
        'department': get_user_department(cost_info['user']),
        'execution_time':
```

```
query_metadata['execution_time_seconds'],
```

```
'bytes_processed': query_metadata.get('bytes_processed',
```

```
0),
```

```
'compute_cost': cost_info.get('compute', 0),
```

```
'total_cost': cost_info['total'],  
'warehouse_size': query_metadata.get('warehouse_size',
```

```
'unknown'),
```

```
'timestamp': cost_info['timestamp']  
    })
```

PYTHON

```
# Real-time cost monitoring  
def monitor_running_queries():  
    """Monitor currently running queries and their projected
```

```
costs"""
```

```
running_queries = get_running_queries()  
  
for query in running_queries:  
    estimated_cost = estimate_query_cost(  
        query['warehouse_size'],  
        query['elapsed_time_seconds'],  
        query.get('bytes_scanned_so_far', 0)  
    )  
  
    # Alert on expensive queries  
    if estimated_cost > 100: # $100 threshold  
        send_cost_alert({  
            'query_id': query['query_id'],  
            'user': query['user_name'],  
            'estimated_cost': estimated_cost,  
            'elapsed_time': query['elapsed_time_seconds'],  
            'action': 'Consider cancelling if cost exceeds
```

```
budget'
```

```
})
```

Team and Project Cost Aggregation:

```
-- Team cost summary with trends
WITH daily_costs AS (
    SELECT
        DATE(timestamp) as cost_date,
        team,
        SUM(total_cost) as daily_cost,
        COUNT(DISTINCT user_name) as active_users,
        COUNT(*) as query_count
    FROM query_costs
    WHERE timestamp >= CURRENT_DATE - INTERVAL '30 days'
    GROUP BY DATE(timestamp), team
),
team_trends AS (
    SELECT
        team,
        AVG(daily_cost) as avg_daily_cost,
        STDDEV(daily_cost) as cost_volatility,
        SUM(daily_cost) as total_monthly_cost,
        MAX(daily_cost) as peak_daily_cost,
        AVG(active_users) as avg_daily_users
    FROM daily_costs
    GROUP BY team
)
SELECT
    team,
    total_monthly_cost,
    avg_daily_cost,
    peak_daily_cost,
    cost_volatility,
    avg_daily_users,
    total_monthly_cost / avg_daily_users as cost_per_user_per_month,
    CASE
        WHEN cost_volatility / avg_daily_cost > 0.5 THEN
```

SQL

'HIGH_VARIANCE'

```

WHEN total_monthly_cost > 5000 THEN 'HIGH_COST'
    ELSE 'NORMAL'
END as cost_risk_category
FROM team_trends
ORDER BY total_monthly_cost DESC;

```

SQL

Cost Guardrails and Prevention

Query Cost Limits:

```

class QueryCostGuardrails:
    def __init__(self):
        self.cost_limits = {
            'individual_query_max': 50, # $50 per query
            'user_daily_limit': 500,    # $500 per user per day
            'team_daily_limit': 2000,   # $2000 per team per day
            'warehouse_hourly_limit': 300 # $300 per hour per

```

PYTHON

warehouse

```

}

def check_query_approval(self, user, team, estimated_cost,

```

PYTHON

warehouse_size):

```

"""Check if query should be allowed to run"""
checks = []

# Individual query limit
if estimated_cost >

```

self.cost_limits['individual_query_max']:

```

checks.append({
    'type': 'individual_query_limit',
    'current': estimated_cost,
    'limit': self.cost_limits['individual_query_max'],
    'action': 'require_approval'
})

# User daily limit
user_daily_spend = get_user_daily_spend(user)
if user_daily_spend + estimated_cost >

```

`self.cost_limits['user_daily_limit']:`

```

checks.append({
    'type': 'user_daily_limit',
    'current': user_daily_spend + estimated_cost,
    'limit': self.cost_limits['user_daily_limit'],
    'action': 'block_query'
})

# Team daily limit
team_daily_spend = get_team_daily_spend(team)
if team_daily_spend + estimated_cost >

```

`self.cost_limits['team_daily_limit']:`

```

checks.append({
    'type': 'team_daily_limit',
    'current': team_daily_spend + estimated_cost,
    'limit': self.cost_limits['team_daily_limit'],
    'action': 'notify_team_lead'
})

return {
    'approved': len([c for c in checks if c['action'] ==

```

```

'block_query']) == 0,

```

```

'warnings': checks,
'requires_approval': len([c for c in checks if

```

```
c['action'] == 'require_approval')) > 0
```

```
}
```

PYTHON

```
def estimate_query_cost(self, query_text, warehouse_size):
    """Estimate query cost before execution"""

    # Get query plan and statistics
    explain_result = execute_explain_plan(query_text)

    # Extract estimated scan size and complexity
    estimated_bytes =
```

```
extract_estimated_scan_size(explain_result)
```

```
estimated_complexity = analyze_query_complexity(query_text)

# Calculate estimated execution time based on historical
```

patterns

```
similar_queries = find_similar_queries(query_text,
```

estimated_bytes)

```
if similar_queries:
    estimated_time_seconds = np.median([q['execution_time']
```

for q in similar_queries])

```
else:
    # Fallback estimation based on data size and complexity
    estimated_time_seconds = (
        (estimated_bytes / (1024**3)) # GB
        * complexity_multiplier[estimated_complexity]
        * warehouse_performance[warehouse_size]
```

['seconds_per_gb']

```
)

# Calculate cost based on warehouse pricing
warehouse_cost_per_hour =
```

```
get_warehouse_cost_per_hour(warehouse_size)
```

```
estimated_cost = (estimated_time_seconds / 3600) *
```

warehouse_cost_per_hour

```
return {
    'estimated_cost': estimated_cost,
    'estimated_time_seconds': estimated_time_seconds,
    'estimated_bytes_scanned': estimated_bytes,
    'confidence_level':
```

```
calculate_estimation_confidence(similar_queries)
```

```
}

# Query interceptor with cost controls
def execute_query_with_cost_controls(query_text, user,
```

PYTHON

warehouse_size):

```
"""Execute query with cost guardrails"""

guardrails = QueryCostGuardrails()

# Estimate cost
cost_estimate = guardrails.estimate_query_cost(query_text,
```

warehouse_size)

```

# Check guardrails
approval_check = guardrails.check_query_approval(
    user, get_user_team(user),
    cost_estimate['estimated_cost'],
    warehouse_size
)

if not approval_check['approved']:
    blocked_reasons = [w['type'] for w in

```

```

approval_check['warnings'] if w['action'] == 'block_query']

```

```

raise QueryBlockedException(f"Query blocked:

```

```

{blocked_reasons}"))

```

```

if approval_check['requires_approval']:
    # Send approval request
    approval_id = request_query_approval(user, query_text,

```

```

cost_estimate)

```

```

return {'status': 'pending_approval', 'approval_id':

```

```

approval_id}

```

```

# Execute with monitoring
start_time = time.time()
try:
    result = execute_query(query_text, warehouse_size)
    execution_time = time.time() - start_time

    # Track actual vs estimated cost
    actual_cost = calculate_actual_query_cost(query_text,

```

```

execution_time, warehouse_size)

```



```

track_cost_estimation_accuracy(cost_estimate, actual_cost)

return result

```

```

except Exception as e:
    # Track failed query costs too
    execution_time = time.time() - start_time
    partial_cost = calculate_partial_query_cost(execution_time,

```

warehouse_size)

```

track_query_failure_cost(query_text, partial_cost, str(e))
raise

```

Resource Auto-Scaling with Cost Awareness:

```

class CostAwareAutoScaling:
    def __init__(self):
        self.scaling_policies = {
            'scale_up_threshold': 0.8,    # CPU/queue utilization
            'scale_down_threshold': 0.3,
            'cost_per_hour_limit': 300,   # Don't auto-scale above

```

PYTHON

\$300/hour

```

'scale_down_delay_minutes': 15 # Wait before scaling

```

down

```

}

def should_scale_up(self, warehouse_name, current_utilization,

```

PYTHON

queue_depth):

```
"""Determine if warehouse should scale up"""
```

```
# Check utilization thresholds  
if current_utilization <
```

```
self.scaling_policies['scale_up_threshold']:
```

```
    return False
```

```
# Check queue depth  
if queue_depth < 5:    # Not enough queued queries to justify
```

```
scale-up
```

```
    return False
```

```
# Check cost limits  
current_cost_per_hour =
```

```
    get_warehouse_cost_per_hour(warehouse_name)
```

```
    next_size_cost =
```

```
    get_next_warehouse_size_cost(warehouse_name)
```

```
    if next_size_cost >
```

```
self.scaling_policies['cost_per_hour_limit']:
```

```
    log_scaling_blocked(warehouse_name,
```

```
'cost_limit_exceeded', {
```

```
'current_cost': current_cost_per_hour,  
'next_size_cost': next_size_cost,  
'limit':
```

```
self.scaling_policies['cost_per_hour_limit']
```

```
)  
    return False  
  
# Check team budget  
team = get_warehouse_owner_team(warehouse_name)  
team_remaining_budget =
```

```
get_team_remaining_daily_budget(team)
```

```
estimated_additional_cost = (next_size_cost -
```

```
current_cost_per_hour) * 1 # 1 hour estimate
```

```
if estimated_additional_cost > team_remaining_budget:  
    notify_team_budget_limit(team, warehouse_name,
```

```
estimated_additional_cost)
```

```
return False  
  
    return True  
  
def execute_scaling_decision(self, warehouse_name, action,
```

PYTHON

```
reason):
```

```
"""Execute scaling action with cost tracking"""  
  
old_size = get_warehouse_size(warehouse_name)  
old_cost_per_hour =
```

```
get_warehouse_cost_per_hour(warehouse_name)
```

PYTHON

```
if action == 'scale_up':
    new_size = scale_warehouse_up(warehouse_name)
elif action == 'scale_down':
    new_size = scale_warehouse_down(warehouse_name)

new_cost_per_hour =
```

```
get_warehouse_cost_per_hour(warehouse_name)
```

```
# Log scaling event with cost impact
log_scaling_event({
    'warehouse_name': warehouse_name,
    'action': action,
    'reason': reason,
    'old_size': old_size,
    'new_size': new_size,
    'cost_change_per_hour': new_cost_per_hour -
```

```
old_cost_per_hour,
```

```
    'team': get_warehouse_owner_team(warehouse_name),
    'timestamp': datetime.now()
})

# Alert stakeholders of significant cost changes
if abs(new_cost_per_hour - old_cost_per_hour) > 50:    #
```

```
$50/hour change
```

```
notify_cost_change(warehouse_name, old_cost_per_hour,
```

```
new_cost_per_hour)
```

Automated Cost Optimization

Result Caching Strategy:

```
class IntelligentResultCaching:
    def __init__(self):
        self.cache_policies = {
            'expensive_query_threshold': 10, # Cache queries
```

costing >\$10

```
'frequent_query_threshold': 5, # Cache queries run >5
```

times/day

```
'cache_duration_hours': 24, # Default cache TTL
'max_cache_size_gb': 100 # Limit cache storage
```

costs

```
}

def should_cache_result(self, query_hash, query_cost,
```

execution_count):

```
"""Determine if query result should be cached"""

# Cache expensive queries
if query_cost >
```

self.cache_policies['expensive_query_threshold']:

```
return True, 'expensive_query'

# Cache frequently run queries
daily_executions =
```

```
get_query_daily_execution_count(query_hash)
```

```
if daily_executions >
```

self.cache_policies['frequent_query_threshold']:

```
    return True, 'frequent_query'

    # Don't cache if storage costs would exceed savings
    estimated_result_size = estimate_result_size(query_hash)
    storage_cost_per_day = (estimated_result_size / (1024**3)) *
```

```
0.023 / 30 # $23/TB/month
```

```
potential_savings = query_cost * daily_executions

if storage_cost_per_day > potential_savings * 0.1:    #
```

Storage cost >10% of savings

```
    return False, 'storage_cost_too_high'

    return False, 'no_caching_criteria_met'
```

The optimize_cache_storage implementation:

```
def optimize_cache_storage(self):
    """Remove least valuable cached results to manage storage
```

PYTHON

costs"""

```
    cached_queries = get_cached_query_statistics()

    # Calculate value score for each cached result
    for query in cached_queries:
        hit_rate = query['cache_hits'] / (query['cache_hits'] +
```

```
query['cache_misses'])
```

```
savings_per_hit = query['average_query_cost']
storage_cost = query['result_size_gb'] * 0.023 / 30 #
```

Daily storage cost

```
query['value_score'] = (
    hit_rate
    * savings_per_hit
    * query['daily_hits']
) - storage_cost

# Remove lowest value cached results if over storage limit
total_cache_size = sum(q['result_size_gb'] for q in
```

cached_queries)

```
if total_cache_size >
```

self.cache_policies['max_cache_size_gb']:

```
# Sort by value score and remove lowest value items
sorted_queries = sorted(cached_queries, key=lambda x:
```

PYTHON

```
x['value_score'])
```

```
for query in sorted_queries:
    if total_cache_size <=
```

self.cache_policies['max_cache_size_gb']:

```

break

remove_cached_result(query['query_hash'])
total_cache_size -= query['result_size_gb']

log_cache_eviction({
    'query_hash': query['query_hash'],
    'reason': 'low_value_score',
    'value_score': query['value_score'],
    'storage_savings_gb': query['result_size_gb']
})

```

Materialized View Recommendations:

```

def identify_materialization_opportunities():
    """Identify queries that would benefit from materialized

```

SQL

views"""

```

# Query patterns that are good candidates for materialization
candidate_queries = """
WITH query_patterns AS (
    SELECT
        query_hash,
        COUNT(*) as execution_count,
        SUM(total_cost) as total_cost,
        AVG(execution_time_seconds) as avg_execution_time,
        STRING_AGG(DISTINCT user_name, ',') as users,
        MIN(timestamp) as first_seen,
        MAX(timestamp) as last_seen,
        -- Extract common patterns
        CASE
            WHEN LOWER(query_text) LIKE '%group by%' AND

```

SQL

```

        LOWER(query_text) LIKE '%sum%' THEN 'aggregation'

```

```

        WHEN LOWER(query_text) LIKE '%join%' AND

```

SQL

```

        LOWER(query_text) LIKE '%where%' THEN 'filtered_join'

```



```
WHEN LOWER(query_text) LIKE '%window%' THEN
```

'window_function'

ELSE 'other'

```
END as query_pattern  
FROM query_costs
```

SQL

The corresponding SQL query:

```
WHERE timestamp >= CURRENT_DATE - INTERVAL '30 days'  
GROUP BY query_hash, query_text  
HAVING execution_count >= 10 -- Frequently executed  
AND total_cost >= 50         -- Expensive enough to justify
```

SQL

materialization

```
)  
SELECT  
    query_hash,  
    execution_count,  
    total_cost,  
    avg_execution_time,  
    query_pattern,  
    users,  
    -- Estimate materialization benefit  
    total_cost * 0.8 as potential_savings, -- Assume 80% cost
```

SQL

reduction

```

CASE
    WHEN query_pattern = 'aggregation' THEN 'HIGH'
    WHEN query_pattern = 'filtered_join' THEN 'MEDIUM'
    ELSE 'LOW'
END as materialization_suitability
FROM query_patterns
ORDER BY potential_savings DESC
"""

opportunities = execute_query(candidate_queries)

recommendations = []
for opp in opportunities:
    if opp['materialization_suitability'] in ['HIGH', 'MEDIUM']:
        recommendations.append({
            'query_hash': opp['query_hash'],
            'estimated_monthly_savings':

```

```

opp['potential_savings'],

```

```

'execution_frequency': opp['execution_count'],
'users_impacted': len(opp['users'].split(',')),
'recommendation':

```

```

generate_materialization_recommendation(opp),

```

```

'implementation_effort':

```

```

estimate_implementation_effort(opp['query_pattern'])

```

PYTHON

```

}))

    return recommendations

def generate_materialization_recommendation(query_opportunity):
    """Generate specific materialization recommendations"""

    if query_opportunity['query_pattern'] == 'aggregation':
        return {
            'type': 'aggregate_table',
            'refresh_frequency': 'hourly',
            'estimated_storage_cost':

```

```
estimate_aggregate_table_size(query_opportunity),
```

SQL

```
'implementation': 'CREATE TABLE AS SELECT with scheduled
```

refresh'

SQL

```

}

elif query_opportunity['query_pattern'] == 'filtered_join':
    return {
        'type': 'denormalized_table',
        'refresh_frequency': 'daily',
        'estimated_storage_cost':

```

```
estimate_denormalized_table_size(query_opportunity),
```

SQL

```

'implementation': 'Pre-join frequently accessed tables'
    }

else:
    return {
        'type': 'result_cache',
        'refresh_frequency': 'on_demand',
        'estimated_storage_cost': 'minimal',
        'implementation': 'Intelligent caching with TTL'

```

```
}
```

Cost Visibility and Reporting

Executive Cost Dashboard:

```
-- Executive summary of data platform costs
WITH cost_summary AS (
    SELECT
        DATE_TRUNC('week', timestamp) as week_start,
        SUM(total_cost) as weekly_cost,
        COUNT(DISTINCT user_name) as active_users,
        COUNT(*) as query_count,
        AVG(total_cost) as avg_cost_per_query
    FROM query_costs
    WHERE timestamp >= CURRENT_DATE - INTERVAL '12 weeks'
    GROUP BY DATE_TRUNC('week', timestamp)
),
cost_trends AS (
    SELECT
        week_start,
        weekly_cost,
        active_users,
        LAG(weekly_cost) OVER (ORDER BY week_start) as
```

SQL

previous_week_cost,

```
(weekly_cost - LAG(weekly_cost) OVER (ORDER BY week_start))
```

SQL

```
/ LAG(weekly_cost) OVER (ORDER BY week_start) * 100 as week_over_week_change
```

```

FROM cost_summary
)
SELECT
    week_start,
    weekly_cost,
    active_users,
    ROUND(week_over_week_change, 1) as wow_change_percent,
    CASE
        WHEN week_over_week_change > 20 THEN '    High Growth'
        WHEN week_over_week_change > 10 THEN '    Moderate Growth'
        WHEN week_over_week_change < -10 THEN '    Cost Reduction'
        ELSE 'o Stable'
    END as trend_indicator
FROM cost_trends
WHERE week_start >= CURRENT_DATE - INTERVAL '8 weeks'
ORDER BY week_start;

```

Team Cost Accountability Dashboard:

```

def generate_team_cost_report(team_name, time_period_days=30):
    """Generate detailed cost report for a team"""

    query = """
    WITH team_costs AS (
        SELECT
            user_name,
            DATE(timestamp) as cost_date,
            SUM(total_cost) as daily_cost,
            COUNT(*) as query_count,
            SUM(CASE WHEN total_cost > 10 THEN 1 ELSE 0 END) as

```

expensive_queries

```

FROM query_costs
  WHERE team = %(team_name)s
  AND timestamp >= CURRENT_DATE - INTERVAL '%(days)s days'
  GROUP BY user_name, DATE(timestamp)
),
user_summaries AS (
  SELECT
    user_name,
    SUM(daily_cost) as total_cost,
    AVG(daily_cost) as avg_daily_cost,
    MAX(daily_cost) as peak_daily_cost,
    SUM(query_count) as total_queries,
    SUM(expensive_queries) as total_expensive_queries

```

The corresponding SQL query:

```

FROM team_costs
  GROUP BY user_name
)
SELECT
  user_name,
  total_cost,
  avg_daily_cost,
  peak_daily_cost,
  total_queries,
  total_expensive_queries,
  total_cost / total_queries as avg_cost_per_query,
  RANK() OVER (ORDER BY total_cost DESC) as cost_rank
FROM user_summaries
ORDER BY total_cost DESC
"""

user_costs = execute_query(query, {'team_name': team_name,

```

'days': time_period_days})

```

# Generate insights and recommendations
insights = {
  'team_total_cost': sum(user['total_cost'] for user in

```

```

user_costs),

```

```
'top_spender': user_costs[0]['user_name'] if user_costs else
```

None,

```
'cost_concentration': user_costs[0]['total_cost'] /
```

```
sum(user['total_cost'] for user in user_costs) if user_costs else 0,
```

```
'avg_cost_per_user': np.mean([user['total_cost'] for user in
```

```
user_costs]),
```

```
'users_with_expensive_queries': len([u for u in user_costs
```

if u['total_expensive_queries'] > 0]),

```
'recommendations': []  
}
```

```
# Generate recommendations  
if insights['cost_concentration'] > 0.5:  
    insights['recommendations'].append({  
        'type': 'cost_concentration',  
        'message': f"One user accounts for
```

```
{insights['cost_concentration']*100:.1f}% of team costs",
```

```
'action': 'Review top spender query patterns for
```

optimization opportunities'

```
})
```

```
if insights['users_with_expensive_queries'] > len(user_costs) *
```

0.3:

```
insights['recommendations'].append({
    'type': 'expensive_queries',
    'message': f"{insights['users_with_expensive_queries']}
```

users ran expensive queries",

```
'action': 'Implement query cost training or approval
```

workflows'

```
    })

    return {
        'team_name': team_name,
        'period_days': time_period_days,
        'user_costs': user_costs,
        'insights': insights
    }
```

Pro Tip Make cost information visible in real-time, not just in monthly reports. Show estimated costs before queries run, display running totals in BI tools, and send weekly cost summaries to team leads. Cost awareness prevents expensive mistakes better than cost restrictions.

□ Checkpoint

1. Why is it important to track costs at multiple levels (query, user, team, project)?
2. How do cost guardrails differ from cost monitoring, and why do you need both?
3. What factors should you consider when deciding whether to cache a query result?

9.5 Security & Compliance Monitoring

🔗 TL;DR

Implement defense-in-depth: access monitoring, data classification, audit logging - Automate PII detection and handling — manual processes don't scale - Build compliance reporting into your regular workflows, not as an afterthought - Focus on behavioral anomaly detection, not just rule-based access controls

Security for data systems isn't just about preventing unauthorized access — it's about understanding who accesses what data, detecting anomalous behavior, and proving compliance with regulations like GDPR, HIPAA, and SOX. It's common to see companies pass security audits only to discover later that they had no idea which users were accessing customer PII, or whether that data was being copied to unauthorized systems. Good security monitoring isn't just about keeping bad actors out — it's about having visibility into how your data is actually being used.

Data Access Monitoring and Behavioral Analysis

Comprehensive Access Logging:

```
class DataAccessMonitor:
    def __init__(self, audit_storage):
        self.audit_storage = audit_storage

    def log_data_access(self, user, table_name, query, access_type,
```

PYTHON

```
result_count=None):
```

```
"""Log all data access with context"""
```

```
access_event = {
    'timestamp': datetime.utcnow(),
    'user_id': user['user_id'],
    'user_email': user['email'],
    'user_role': user['role'],
    'user_department': user['department'],
    'session_id': user.get('session_id'),
    'ip_address': user.get('ip_address'),
    'user_agent': user.get('user_agent'),

    'table_name': table_name,
    'database_name': get_table_database(table_name),
    'data_classification':
```

```
get_table_classification(table_name),
```

```
'contains_pii': check_table_contains_pii(table_name),

'access_type': access_type,    # SELECT, INSERT, UPDATE,
```

SQL

DELETE, EXPORT

```
'query_hash': hashlib.md5(query.encode()).hexdigest(),
'query_pattern': extract_query_pattern(query),
'columns_accessed': extract_columns_from_query(query,
```

SQL

```
table_name),
```

```
'where_conditions': extract_where_conditions(query),
'result_count': result_count,
'bytes_returned': estimate_bytes_returned(result_count,
```

SQL

```
table_name),
```

```
'compliance_context': {  
    'data_residency_region':
```

```
get_table_region(table_name),
```

```
'retention_policy':
```

```
get_table_retention_policy(table_name),
```

```
'consent_required':
```

```
check_consent_requirement(table_name, user),
```

```
}
```

```
}
```

```
# Store in audit log
```

```
self.audit_storage.store(access_event)
```

```
# Real-time analysis
```

```
self.analyze_access_patterns(access_event)
```

```
def analyze_access_patterns(self, access_event):
```

```
    """Real-time analysis of access patterns for anomaly
```

```
detection"""
```

PYTHON

```
# Check for suspicious patterns
anomalies = []

# Unusual time access
if self.is_unusual_time_access(access_event):
    anomalies.append({
        'type': 'unusual_time_access',
        'severity': 'medium',
        'details': 'Access outside normal business hours'
    })

# Large data extraction
if access_event['result_count'] and
```

```
access_event['result_count'] > 100000:
```

```
anomalies.append({
    'type': 'large_data_extraction',
    'severity': 'high',
    'details': f"User extracted
```

```
{access_event['result_count']} records"
```

```
})

# Access to sensitive data by non-authorized role
if (access_event['contains_pii'] and
    access_event['user_role'] not in
```

```
get_authorized_pii_roles()):
```

```
anomalies.append({
    'type': 'unauthorized_pii_access',
    'severity': 'critical',
    'details': 'PII access by unauthorized role'
})

# First-time access to sensitive table
if (access_event['data_classification'] in ['CONFIDENTIAL',
```

```
'RESTRICTED'] and
```

```
self.is_first_time_access(access_event['user_id'],
```

PYTHON

```
access_event['table_name']))):
```

```
anomalies.append({
    'type': 'first_time_sensitive_access',
    'severity': 'medium',
    'details': 'First access to sensitive data'
})

# Send alerts for high/critical severity anomalies
for anomaly in anomalies:
    if anomaly['severity'] in ['high', 'critical']:
        self.send_security_alert(access_event, anomaly)

def is_unusual_time_access(self, access_event):
    """Check if access is happening at unusual time"""
    access_time = access_event['timestamp']
    user_timezone = get_user_timezone(access_event['user_id'])
    local_time =
```

PYTHON

```
access_time.replace(tzinfo=timezone.utc).astimezone(user_timezone)
```

```
# Define business hours (9 AM - 6 PM)
if local_time.weekday() < 5: # Monday-Friday
    business_hours = 9 <= local_time.hour < 18
else: # Weekend
    business_hours = False
```

The PYTHON code:

```
return not business_hours
```

```
def send_security_alert(self, access_event, anomaly):
    """Send security alert to SOC team"""
    alert = {
        'alert_type': 'DATA_ACCESS_ANOMALY',
        'severity': anomaly['severity'],
        'user_id': access_event['user_id'],
        'user_email': access_event['user_email'],
        'table_name': access_event['table_name'],
        'anomaly_type': anomaly['type'],
        'anomaly_details': anomaly['details'],
        'timestamp': access_event['timestamp'],
        'investigation_context': {
            'recent_user_activity':
```

```
get_recent_user_activity(access_event['user_id']),
```

```
'table_sensitivity':
```

```
access_event['data_classification'],
```

```
    'access_pattern': access_event['query_pattern']
    }
}
```

```
    send_to_security_team(alert)
```

Behavioral Anomaly Detection:

PYTHON

```
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
import numpy as np

class UserBehaviorAnalyzer:
    def __init__(self):
        self.models = {}
        self.scalers = {}

    def build_user_behavior_model(self, user_id):
        """Build behavioral model for a user based on historical
```

```
access patterns"""
```

PYTHON

```
# Get user's historical access patterns
historical_data = self.get_user_historical_access(user_id,
```

```
days=90)
```

```

if len(historical_data) < 50:    # Need sufficient data
    return None

# Extract behavioral features
features = []
for day_data in historical_data:
    features.append([
        day_data['queries_per_day'],
        day_data['unique_tables_accessed'],
        day_data['total_records_accessed'],
        day_data['avg_query_complexity'],
        day_data['pii_access_count'],
        day_data['off_hours_access_count'],
        day_data['weekend_access_count'],
        day_data['new_table_access_count']
    ])

features = np.array(features)

# Scale features
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features)

# Train isolation forest
model = IsolationForest(contamination=0.1, random_state=42)
model.fit(features_scaled)

# Store model and scaler
self.models[user_id] = model

```

The *PYTHON* code:

```

self.scalers[user_id] = scaler

    return model

def detect_behavioral_anomaly(self, user_id,

```

current_day_features):


```
"""Detect if current behavior is anomalous for this user"""
```

```
    if user_id not in self.models:
        self.build_user_behavior_model(user_id)
    if user_id not in self.models:
        return None # Insufficient data
```

```
    model = self.models[user_id]
    scaler = self.scalers[user_id]
```

```
# Scale current features
current_features_scaled =
```

```
    scaler.transform([current_day_features])
```

```
# Get anomaly score
anomaly_score =
```

```
    model.decision_function(current_features_scaled)[0]
```

```
is_anomaly = model.predict(current_features_scaled)[0] == -1
```

```
if is_anomaly:
    # Identify which features are most anomalous
    feature_names = [
        'queries_per_day', 'unique_tables_accessed',
```

```
'total_records_accessed',
```

```
'avg_query_complexity', 'pii_access_count',
```

```
'off_hours_access_count',
```

```
'weekend_access_count', 'new_table_access_count'
    ]
```

```
# Compare current features to historical average
historical_avg =
```

```
np.mean(scaler.inverse_transform(model.score_samples( scaler.transform(self.get
_user_historical_features(user_id))
```

```
).reshape(-1, 1)), axis=0)
```

```
anomalous_features = []
for i, (current, historical) in
```

```
enumerate(zip(current_day_features, historical_avg)):
```

```
    if abs(current - historical) / (historical + 1) >
```

```
        0.5: # >50% deviation
```

```
            anomalous_features.append({
                'feature': feature_names[i],
                'current_value': current,
                'historical_average': historical,
                'deviation_percent': abs(current -
```

```
                historical) / (historical + 1) * 100
```

```
            })
```

```
        return {
            'is_anomaly': True,
            'anomaly_score': anomaly_score,
            'anomalous_features': anomalous_features,
            'risk_level': 'HIGH' if anomaly_score < -0.5 else
```

```
'MEDIUM'
```

```
        }
```

```
        return {'is_anomaly': False, 'anomaly_score': anomaly_score}
```

Automated PII Detection and Classification

Dynamic PII Detection:

```
import re
from typing import List, Dict, Tuple
```

SQL

The PIIDetector implementation:

```
class PIIDetector:
    def __init__(self):
        self.patterns = {
            'ssn': re.compile(r'\b\d{3}-\d{2}-\d{4}\b|\b\d{9}\b'),
            'credit_card': re.compile(r'\b(?:\d{4}[-\s]?)
```

PYTHON

```
{3}\d{4}\b'),
```

```
'email': re.compile(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-
```

```
] + \. [A-Z|a-z]{2,} \b'),
```

```
'phone': re.compile(r'\b\d{3}-\d{3}-\d{4}\b|\b\
```

```
(\d{3}) \s? \d{3}-\d{4} \b'),
```

```
'ip_address': re.compile(r'\b(?:[0-9]{1,3}\.){3}[0-9]
```

```
{1,3}\b'),
```

```
'passport': re.compile(r'\b[A-Z]{1,2}\d{6,9}\b'),
'drivers_license': re.compile(r'\b[A-Z]{1,2}\d{6,8}\b')
}
```

PYTHON

```
self.column_name_indicators = {
    'ssn': ['ssn', 'social_security',
```

```
'social_security_number'],
```

```
'credit_card': ['credit_card', 'cc_number',
```

```
'card_number', 'payment_card'],
```

```
'email': ['email', 'email_address', 'email_addr'],  
'phone': ['phone', 'telephone', 'mobile', 'cell_phone'],  
'name': ['first_name', 'last_name', 'full_name',
```

```
'customer_name'],
```

```
'address': ['address', 'street_address', 'home_address',
```

```
'billing_address']
```

```
}
```

PYTHON

```
def scan_table_for_pii(self, table_name: str, sample_size: int =
```

```
1000) -> Dict:
```

```
"""Scan a table for PII and return classification results"""
```

```
# Get table schema and sample data
schema = get_table_schema(table_name)
sample_data = get_table_sample(table_name, sample_size)

pii_findings = {
    'table_name': table_name,
    'scan_timestamp': datetime.utcnow(),
    'total_columns': len(schema),
    'columns_with_pii': [],
    'pii_summary': {},
    'confidence_scores': {},
    'recommended_classification': 'PUBLIC'
}

for column in schema:
    column_name = column['name'].lower()
    column_type = column['type']

    # Check column name indicators
    name_based_pii =
```

```
self.check_column_name_for_pii(column_name)
```

```
# Check sample data content
if column_name in sample_data.columns:
    content_based_pii =
```

```
self.check_column_content_for_pii(
```

```
sample_data[column_name].astype(str)
    )
else:
    content_based_pii = []

    # Combine findings
    pii_types = list(set(name_based_pii +
```

```
content_based_pii))
```

```

if pii_types:
    confidence = self.calculate_pii_confidence(
        column_name, column_type, pii_types,
        name_based_pii, content_based_pii

```

PYTHON

The PYTHON code:

```

)

    pii_findings['columns_with_pii'].append({
        'column_name': column_name,
        'pii_types': pii_types,
        'confidence': confidence,
        'detection_method': {
            'name_based': name_based_pii,
            'content_based': content_based_pii
        }
    })

# Calculate overall table classification
if pii_findings['columns_with_pii']:
    highest_risk_pii = max(
        pii_findings['columns_with_pii'],
        key=lambda x: x['confidence']
    )

    if highest_risk_pii['confidence'] > 0.8:
        pii_findings['recommended_classification'] =

```

PYTHON

'RESTRICTED'

```

elif highest_risk_pii['confidence'] > 0.5:
    pii_findings['recommended_classification'] =

```

PYTHON

'CONFIDENTIAL'

```

else:
    pii_findings['recommended_classification'] =

```

'INTERNAL'

PYTHON

```

return pii_findings

def check_column_name_for_pii(self, column_name: str) ->

```

```
List[str]:
```

```

"""Check if column name suggests PII content"""
detected_pii = []

for pii_type, indicators in

```

```
self.column_name_indicators.items():
```

PYTHON

```

for indicator in indicators:
    if indicator in column_name:
        detected_pii.append(pii_type)

return detected_pii

def check_column_content_for_pii(self, column_data) ->

```

```
List[str]:
```

SQL

```

"""Check actual data content for PII patterns"""
detected_pii = []

# Sample non-null values
non_null_data = column_data.dropna().head(100)
combined_text = ' '.join(non_null_data.astype(str))

for pii_type, pattern in self.patterns.items():
    matches = pattern.findall(combined_text)
    if len(matches) > len(non_null_data) * 0.1:    # >10% of

```

values match

```

detected_pii.append(pii_type)

    return detected_pii

def calculate_pii_confidence(self, column_name, column_type,

```

pii_types,

```

name_based, content_based):
    """Calculate confidence score for PII detection"""

    confidence = 0.0

    # High confidence if both name and content indicate PII
    if name_based and content_based:
        confidence = 0.9

```

Medium-high confidence if content strongly suggests PII:

```

# Medium-high confidence if content strongly suggests PII
elif content_based:
    confidence = 0.7

# Medium confidence if name strongly suggests PII
elif name_based:
    confidence = 0.6

# Adjust based on data type compatibility
compatible_types = {
    'ssn': ['varchar', 'char', 'string'],
    'credit_card': ['varchar', 'char', 'string'],
    'email': ['varchar', 'char', 'string'],
    'phone': ['varchar', 'char', 'string', 'bigint']
}

for pii_type in pii_types:
    if column_type.lower() in compatible_types.get(pii_type,

```

[]):


```

confidence += 0.1

    return min(confidence, 1.0)

# Automated table classification workflow
def classify_all_tables():
    """Scan all tables for PII and update classification metadata"""

    detector = PIIDetector()
    all_tables = get_all_tables()

    classification_results = []

    for table in all_tables:
        try:
            pii_findings =

```

```

detector.scan_table_for_pii(table['name'])

```

```

# Update table metadata
    update_table_classification(
        table['name'],
        pii_findings['recommended_classification'],
        pii_findings
    )

    # Store detailed findings
    store_pii_scan_results(pii_findings)

    classification_results.append(pii_findings)

except Exception as e:
    log_classification_error(table['name'], str(e))

# Generate summary report
generate_classification_summary_report(classification_results)

return classification_results

```

Compliance Reporting and Audit Trails

GDPR Compliance Monitoring:

```
class GDPRComplianceMonitor:
    def __init__(self):
        self.consent_storage = ConsentStorage()
        self.audit_logger = AuditLogger()

    def track_personal_data_processing(self, user_id,
```

data_subject_id,

```
        processing_activity,
```

legal_basis):

```
        """Track processing of personal data for GDPR compliance"""
```

```
        processing_record = {
            'timestamp': datetime.utcnow(),
            'processor_user_id': user_id,
            'data_subject_id': data_subject_id,
            'processing_activity': processing_activity,
            'legal_basis': legal_basis, # consent, contract,
```

legal_obligation, etc.

```
            'purpose': get_processing_purpose(processing_activity),
            'data_categories':
```

```
                get_data_categories(processing_activity),
```

```
                'retention_period':
```

```
                    get_retention_period(processing_activity),
```

```
                'third_party_sharing':
```

```
check_third_party_sharing(processing_activity)
```

```
}
```

```
# Validate legal basis
if legal_basis == 'consent':
    consent_status =
```

```
self.consent_storage.get_consent_status(
```

```
data_subject_id, processing_activity
)
processing_record['consent_valid'] =
```

```
consent_status['valid']
```

```
processing_record['consent_timestamp'] =
```

```
consent_status['timestamp']
```

```
self.audit_logger.log_processing_activity(processing_record)
```

PYTHON

```
    return processing_record
```

```
def handle_data_subject_request(self, request_type,
```

```
data_subject_id, requestor_email):
```

```
    """Handle GDPR data subject requests (access, portability,
```

```
erasure)"""
```

```

request_id = generate_request_id()

request_record = {
    'request_id': request_id,
    'request_type': request_type, # access, portability,

```

erasure, rectification

```

'data_subject_id': data_subject_id,
'requestor_email': requestor_email,
'request_timestamp': datetime.utcnow(),
'status': 'received',
'completion_deadline': datetime.utcnow() +

```

```

timedelta(days=30)

```

```

}

# Store request
store_gdpr_request(request_record)

# Execute request based on type
if request_type == 'access':
    self.process_access_request(request_id, data_subject_id)
elif request_type == 'erasure':
    self.process_erasure_request(request_id,

```

PYTHON

data_subject_id)

```

elif request_type == 'portability':
    self.process_portability_request(request_id,

```

PYTHON

data_subject_id)

```
return request_id
```

```
def process_erasure_request(self, request_id, data_subject_id):
    """Process right to be forgotten request"""
```

```
    # Find all tables containing data for this subject
    tables_with_data =
```

```
find_tables_with_personal_data(data_subject_id)
```

```
erasure_plan = []
```

```
for table in tables_with_data:
    # Check if erasure is legally permissible
    retention_policy =
```

```
get_table_retention_policy(table['name'])
```

```
if retention_policy['legal_hold'] or
```

```
retention_policy['regulatory_requirement']:
```

```
erasure_plan.append({
    'table_name': table['name'],
    'action': 'CANNOT_DELETE',
    'reason': retention_policy['reason'],
    'record_count': table['record_count']
})
else:
    erasure_plan.append({
        'table_name': table['name'],
        'action': 'DELETE',
        'record_count': table['record_count']
    })

# Execute erasure plan
for action in erasure_plan:
    if action['action'] == 'DELETE':
        delete_personal_data(action['table_name'],
```

data_subject_id)

```
# Log erasure
        self.audit_logger.log_data_erasure({
            'request_id': request_id,
            'table_name': action['table_name'],
            'data_subject_id': data_subject_id,
            'records_deleted': action['record_count'],
            'timestamp': datetime.utcnow()
        })

    # Update request status
    update_gdpr_request_status(request_id, 'completed', {
        'erasure_plan': erasure_plan,
        'completion_timestamp': datetime.utcnow()
    })

    return erasure_plan

def generate_compliance_report(self, start_date, end_date):
    """Generate GDPR compliance report for audit purposes"""

    report = {
        'report_period': {'start': start_date, 'end': end_date},
        'processing_activities':
```

PYTHON

```
self.get_processing_activities_summary(start_date, end_date),
```

```
    'data_subject_requests':
```

```
self.get_dsr_summary(start_date, end_date),
```

```
    'consent_management':
```

```
self.get_consent_summary(start_date, end_date),
```

```
    'data_breaches': self.get_breach_summary(start_date,
```

PYTHON

end_date),

```
'compliance_metrics':
```

```
self.calculate_compliance_metrics(start_date, end_date)
```

```
}
```

PYTHON

```
    return report
```

```
# SOX Compliance for Financial Data
```

```
class SOXComplianceMonitor:
```

```
    def __init__(self):
```

```
        self.financial_data_tables = get_financial_data_tables()
```

```
    def monitor_financial_data_access(self, access_event):
```

```
        """Monitor access to financial data for SOX compliance"""
```

The PYTHON code:

```
if access_event['table_name'] in self.financial_data_tables:
```

PYTHON

```
    # Check if user is authorized for financial data access
```

```
    is_authorized = check_financial_data_authorization(
```

```
        access_event['user_id'],
```

```
        access_event['table_name']
```

```
    )
```

```
    sox_audit_record = {
```

```
        'access_timestamp': access_event['timestamp'],
```

```
        'user_id': access_event['user_id'],
```

```
        'user_email': access_event['user_email'],
```

```
        'table_name': access_event['table_name'],
```

```
        'access_type': access_event['access_type'],
```

```
        'authorized_access': is_authorized,
```

```
        'query_hash': access_event['query_hash'],
```

```
        'business_justification':
```

```
get_access_justification(access_event),
```

```
'supervisor_approval':
```

```
check_supervisor_approval(access_event),
```

```
'financial_period':
```

```
determine_financial_period(access_event['timestamp'])
```

```
}
```

SQL

```
store_sox_audit_record(sox_audit_record)
```

```
# Alert on unauthorized access
```

```
if not is_authorized:
```

```
    send_sox_compliance_alert(sox_audit_record)
```

```
def generate_sox_audit_report(self, financial_period):
```

```
    """Generate SOX audit report for a financial period"""
```

```
    query = """
```

```
    SELECT
```

```
        user_email,
```

```
        COUNT(*) as access_count,
```

```
        COUNT(DISTINCT table_name) as unique_tables_accessed,
```

```
        COUNT(CASE WHEN authorized_access = false THEN 1 END) as
```

```
unauthorized_access,
```

```
MAX(access_timestamp) as last_access,
```

```
    STRING_AGG(DISTINCT table_name, ', ') as tables_accessed
```

```
FROM sox_audit_records
```

```
WHERE financial_period = %(period)s
```

```
GROUP BY user_email
```

```
ORDER BY access_count DESC
```

```
"""
```

```
access_summary = execute_query(query, {'period':
```

SQL

```
financial_period}})
```



```
return {  
    'financial_period': financial_period,  
    'total_users': len(access_summary),  
    'total_accesses': sum(row['access_count'] for row in
```

```
access_summary),
```

```
    'unauthorized_accesses': sum(row['unauthorized_access']
```

```
for row in access_summary),
```

```
    'user_access_summary': access_summary,  
    'compliance_score':
```

```
calculate_sox_compliance_score(access_summary)
```

```
}
```

□ Checkpoint

1. What's the difference between rule-based access controls and behavioral anomaly detection?
2. Why is automated PII detection important for data governance and compliance?
3. How do GDPR and SOX compliance requirements differ in terms of data monitoring?

Try It Yourself Exercises

Exercise 1: Design a Data Quality Framework (45 minutes)

You're building a data quality framework for an e-commerce company with these tables:

- orders (5M rows/day): order details, amounts, customer info
- products (500K rows): product catalog with prices, categories
- customers (10M rows): customer profiles, addresses, preferences
- inventory (1M rows): stock levels, updated hourly

Task: Design expectations and monitoring for each table. Include: 1. Completeness checks 2. Referential integrity validation 3. Business rule validation 4. Anomaly detection thresholds

Deliverable: Python code defining expectations and SQL queries for validation.

Exercise 2: Build a Cost Optimization Strategy (60 minutes)

Your data platform costs have grown from \$5k/month to \$50k/month over 6 months. Analysis shows:

- 70% of costs from 5 "expensive" queries run daily
- Marketing team runs 500+ ad-hoc queries/day -

Data science team experiments with large datasets frequently -

Executive dashboard refreshes every 15 minutes

Task: Design a cost optimization strategy including: 1. Cost attribution and budgeting 2. Query optimization opportunities 3. Caching and materialization strategy 4. Usage policies and guardrails

Deliverable: Cost optimization plan with specific recommendations and implementation timeline.

Exercise 3: Security Monitoring Dashboard (30 minutes)

Design a security monitoring dashboard for data platform administrators that shows: - Real-time access to sensitive tables -

Unusual user behavior patterns - PII access by non-authorized users -

Cross-system data movement tracking

Task: Define the metrics, visualizations, and alerting rules.

Deliverable: Dashboard mockup and metric definitions.

Module 9 Final Project: Production Monitoring Platform

Objective Design a complete monitoring, observability, and data quality platform for a fintech company processing 50M transactions/day. The system needs to ensure data quality, monitor costs, detect security issues, and maintain compliance with financial regulations.

Prerequisites

Understanding of data pipeline architecture

Basic knowledge of monitoring tools (Prometheus, Grafana, etc.)

Familiarity with SQL and Python

Expected Time

4-6 hours

Problem Statement

Company Context: FinanceFlow is a digital payment processor handling:

- 50M transactions/day across credit cards, ACH, wire transfers
- Real-time fraud detection with <100ms latency requirement
- Regulatory compliance (PCI DSS, SOX, GDPR)
- 24/7 operations with 99.95% availability SLA

Current Pain Points:

- Data quality issues causing \$2M/month in failed payment processing
- Lack of cost visibility leading to 300% year-over-year infrastructure cost growth
- Manual compliance reporting taking 2 weeks per quarter
- Security incidents detected hours after they occur

Your Mission: Design a comprehensive monitoring and data quality platform that addresses these issues while scaling to 200M transactions/day over the next 2 years.

Requirements

- Functional Requirements: 1. Data Quality Monitoring
- Real-time validation of transaction data
- Detection of schema changes and data anomalies
- Automated data quality scoring and reporting
- Integration with existing fraud detection systems

2. Cost Monitoring & Optimization

Real-time cost tracking by team/project/query
Automated cost alerts and budget enforcement
Query optimization recommendations
Resource auto-scaling based on cost constraints

3. Security & Compliance

Real-time access monitoring and anomaly detection

Automated PII detection and classification
Compliance reporting for PCI DSS and SOX
Data lineage tracking for audit purposes

4. Observability & Performance

End-to-end transaction tracing

Performance monitoring and bottleneck identification
Predictive capacity planning
Integration with existing monitoring stack

Non-Functional Requirements:

- Support 200M transactions/day (4x current scale)
- <1 second latency for data quality checks
- 99.9% availability for monitoring platform
- GDPR-compliant data retention and processing

Deliverables

Create a comprehensive design document including:

1. Architecture Overview (2 pages)

High-level system architecture with major components

Data flow diagrams showing monitoring data collection and processing Integration points with existing fintech infrastructure

Technology stack recommendations

2. Data Quality Framework (3 pages)

Quality dimensions and metrics definition

Real-time validation strategy for transaction data

Anomaly detection algorithms and thresholds

Quality scoring methodology and reporting

3. Cost Management System (2 pages)

Cost attribution model (query/user/team/project level)

Real-time cost tracking and alerting strategy

Automated optimization recommendations

Budget enforcement and approval workflows

4. Security & Compliance Monitoring (3 pages)

Access monitoring and behavioral analysis

PII detection and classification strategy Compliance reporting automation (PCI DSS, SOX)

Data lineage tracking and audit trail design

5. Implementation Plan (2 pages)

6-month implementation roadmap

Team requirements and skill development

Risk mitigation strategies

Success metrics and KPIs

Evaluation Criteria

- Architecture & Design (25 points)
 - ☐ Clear, scalable system architecture
 - ☐ Appropriate technology choices with justification
 - ☐ Realistic integration with existing systems
 - ☐ Consideration of performance and scalability requirements
- Data Quality Framework (20 points)
 - ☐ Comprehensive quality dimension coverage
 - ☐ Real-time validation strategy
 - ☐ Appropriate anomaly detection approaches
 - ☐ Business-aligned quality scoring
- Cost Management (15 points)
 - ☐ Multi-level cost attribution design
 - ☐ Practical optimization strategies
 - ☐ Realistic cost control mechanisms
- Security & Compliance (20 points)
 - ☐ Comprehensive security monitoring approach
 - ☐ Automated compliance reporting strategy
 - ☐

Appropriate regulatory requirement coverage

- Implementation & Operations (20 points)
- ☐ Realistic implementation timeline
- ☐ Consideration of operational complexity -

☐ Checkpoint

1. ☐ Risk identification and mitigation - ☐ Success metrics and monitoring

Sample Questions to Guide Your Design

Architecture Questions:

- How will you handle 200M events/day without impacting transaction processing performance?
- What's your strategy for monitoring distributed transaction processing across multiple data centers?
- How will you ensure the monitoring platform itself meets 99.9% availability?

Data Quality Questions: - How will you detect schema changes in real-time transaction streams?

- What's your approach for validating transaction data without blocking payment processing?
- How will you handle false positives in anomaly detection for financial data?

Cost Questions: - How will you attribute infrastructure costs to business units for a shared platform?

- What cost optimization opportunities exist for batch vs. real-time processing?
- How will you balance cost optimization with regulatory compliance requirements?

Security Questions:

- How will you detect insider threats in financial data access patterns?

- What's your strategy for automated PII detection in unstructured transaction data?
- How will you ensure audit trails are tamper-proof for regulatory compliance?

Success Criteria

Your design should demonstrate: 1. Technical Feasibility: Realistic architecture that can scale to requirements 2. Business Alignment: Solutions that address actual pain points with measurable impact 3.

Operational Practicality: Implementation plan that accounts for team capabilities and constraints 4. Regulatory Compliance: Design that meets strict financial industry requirements

Bonus Challenges

For extra credit, address these advanced scenarios:

1. Multi-Cloud Compliance: How would your design change if the company operates across AWS, Azure, and on-premises data centers?
2. Real-Time Regulatory Reporting: Design a system that can generate SOX compliance reports in real-time rather than quarterly.
3. Machine Learning Integration: How would you integrate ML- based anomaly detection while maintaining explainability for regulators?
4. Zero-Trust Security: Design access monitoring for a zero-trust security model where every data access is authenticated and authorized.

Further Reading

Resources

Books: - Designing Data-Intensive Applications by Martin Kleppmann

- - Deep dive into distributed systems observability
- Building Secure and Reliable Systems by Google SRE team
- Security and reliability patterns
- The Data Warehouse Toolkit by Ralph Kimball
- Data quality patterns for dimensional models

Research Papers:

- "Deequ: Efficient Data Quality Validation for Large Datasets" (Amazon)
- Production data quality patterns
- "Continuous Data Quality Assessment" (Netflix)
- Streaming data quality approaches
- "Data Lineage at Scale" (Airbnb)
- Large-scale lineage tracking implementation

Industry Resources: - Great Expectations documentation - Open

- source data quality framework
- OpenLineage project
- Open standard for data lineage
- Datadog's "Monitoring Best Practices"
- Real-world monitoring patterns

Advanced Topics

Data Quality:

- Statistical process control for data pipelines
- ML- based data drift detection
- Data quality in real-time streaming systems

Cost Optimization: - **FinOps principles for data platforms** - **Query** optimization in cloud warehouses - Automated resource management patterns

Security & Compliance: - Zero-trust architecture for data systems -

Privacy-preserving analytics techniques - **Automated compliance** monitoring patterns

Observability: - **Distributed tracing in data pipelines** - **Chaos** engineering for data systems - Performance optimization through observability

Remember: The best monitoring is the monitoring that helps you prevent problems, not just detect them after they happen. Focus on building systems that give you the context you need to understand not just what went wrong, but why it went wrong and how to prevent it next time. # Module 10: Cost Optimization & Capacity Planning

What you'll learn

- How to build data systems that scale

economically, predict capacity needs accurately, and optimize costs without sacrificing performance. You'll master cloud pricing models, automated optimization techniques, and capacity planning methodologies that keep your data platform both powerful and affordable.

10

Cost Optimization & Capacity Planning

and queries. Master capacity planning for growing systems.

Cloud Data Platform Cost Models

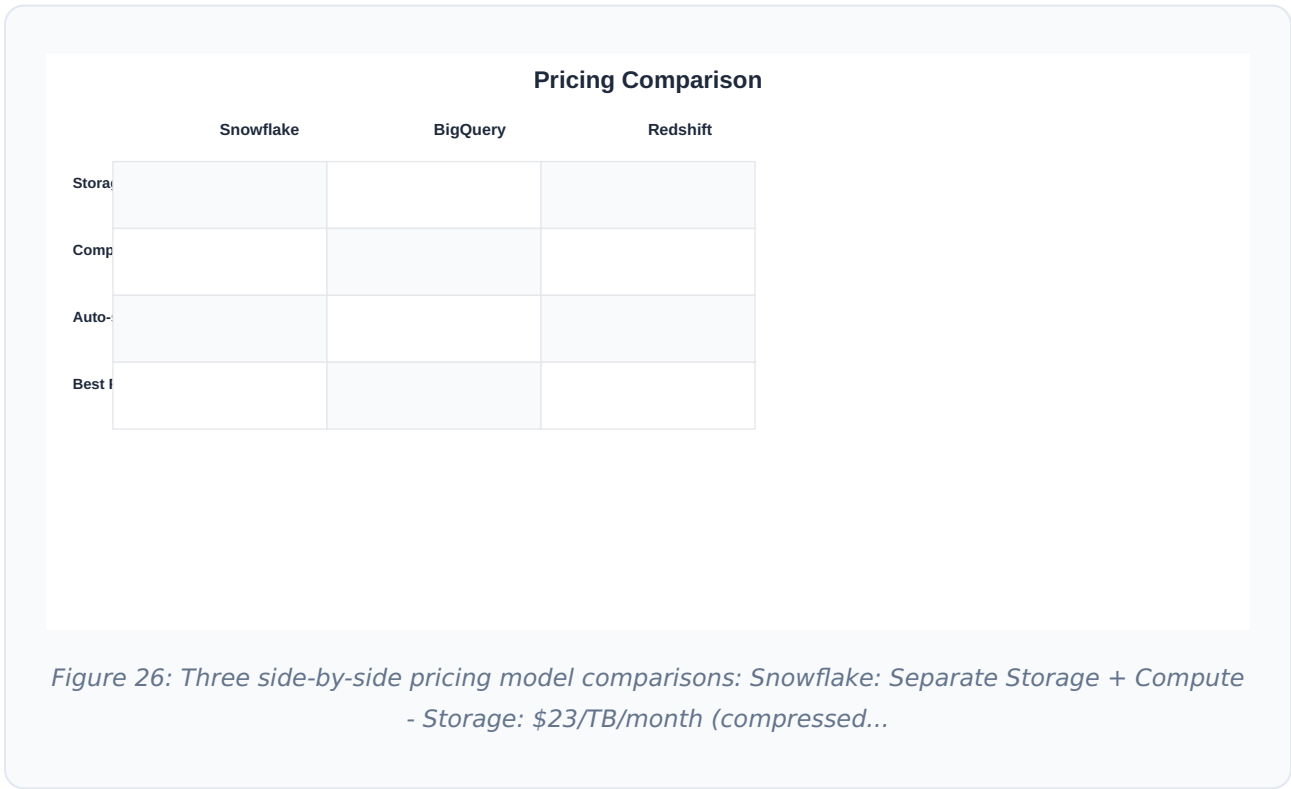
TL;DR

Master the pricing models: understand when to use on- demand vs reserved capacity vs spot instances - Snowflake charges for compute time, BigQuery charges for data scanned, Redshift charges for nodes - Multi-cloud strategies can reduce costs but add operational complexity - Cost optimization is about making informed trade-offs, not just spending less money

After helping three different companies reduce their data platform costs by 40-60% while improving performance, in practice, most teams don't fail at cost optimization because they don't know the technical tactics — they fail because they don't understand the economic models they're operating in.

Every cloud data platform has different pricing psychology. Snowflake wants you to use more compute. BigQuery wants you to scan less data. Redshift wants you to commit to reserved capacity. Understanding these incentives is the first step to optimizing costs.

The Big Three: Pricing Model Deep Dive



Snowflake Economic Model Deep Dive:

PYTHON

```
# Snowflake cost calculation framework
class SnowflakeCostCalculator:
    def __init__(self):
        self.warehouse_costs = {
            'X-Small': {'credits_per_hour': 1, 'cost_per_credit':
2.0},
            'Small': {'credits_per_hour': 2, 'cost_per_credit':
2.0},
            'Medium': {'credits_per_hour': 4, 'cost_per_credit':
```

```
2.0},
```

```
'Large': {'credits_per_hour': 8, 'cost_per_credit':
```

```
2.0},
```

```
'X-Large': {'credits_per_hour': 16, 'cost_per_credit':
```

```
2.0},
```

```
'2X-Large': {'credits_per_hour': 32, 'cost_per_credit':
```

```
2.0},
```

```
'3X-Large': {'credits_per_hour': 64, 'cost_per_credit':
```

```
2.0},
```

```
'4X-Large': {'credits_per_hour': 128, 'cost_per_credit':
```

```
2.0}
```

```
}
```

PYTHON

```
        self.storage_cost_per_tb_month = 23.0    # After compression

    def calculate_query_cost(self, warehouse_size,
```

```
execution_time_seconds):
```

PYTHON

```

"""Calculate cost for a single query"""
warehouse_config = self.warehouse_costs[warehouse_size]

hours = execution_time_seconds / 3600
credits_used = hours * warehouse_config['credits_per_hour']
cost = credits_used * warehouse_config['cost_per_credit']

return {
    'cost_usd': cost,
    'credits_used': credits_used,
    'warehouse_size': warehouse_size,
    'execution_time_seconds': execution_time_seconds
}

def optimize_warehouse_size(self, query_runtime_seconds,

```

target_sla_seconds):

```

"""Find the most cost-effective warehouse size for a

```

query"""

PYTHON

```

# Snowflake scales roughly linearly with warehouse size
options = []

for size, config in self.warehouse_costs.items():
    # Estimate runtime scaling (rough approximation)
    base_runtime = query_runtime_seconds # Assume measured

```

on X-Small

```

scaling_factor = config['credits_per_hour'] # More

```

```

credits = more power

```

PYTHON

```

estimated_runtime = base_runtime / scaling_factor
cost = self.calculate_query_cost(size,

```

estimated_runtime)

PYTHON

```

meets_sla = estimated_runtime <= target_sla_seconds

    options.append({
        'warehouse_size': size,
        'estimated_runtime': estimated_runtime,
        'cost_usd': cost['cost_usd'],
        'credits_used': cost['credits_used'],
        'meets_sla': meets_sla
    })

# Find cheapest option that meets SLA
valid_options = [opt for opt in options if opt['meets_sla']]

if valid_options:
    best_option = min(valid_options, key=lambda x:

```

```

x['cost_usd'])

```

PYTHON

```

return best_option
else:
    # No option meets SLA - return fastest option
    return min(options, key=lambda x:

```

```

x['estimated_runtime'])

```

```

# Usage example
calculator = SnowflakeCostCalculator()

# Optimize warehouse size for a 30-minute query that needs to

```

complete in 10 minutes

```

optimization = calculator.optimize_warehouse_size(
    query_runtime_seconds=1800, # 30 minutes
    target_sla_seconds=600      # 10 minutes
)

```

The PYTHON code:

```
print(f"Recommended: {optimization['warehouse_size']}")
print(f"Estimated cost: ${optimization['cost_usd']:.2f}")
print(f"Estimated runtime:
```

PYTHON

```
{optimization['estimated_runtime']/60:.1f} minutes")
```

BigQuery Cost Optimization Strategy:

```
class BigQueryCostOptimizer:
    def __init__(self):
        self.on_demand_cost_per_tb = 5.0 # $5 per TB scanned
        self.storage_cost_per_tb_month = 20.0 # Active storage
        self.long_term_storage_cost_per_tb_month = 10.0 # 90+ days
```

PYTHON

old

```
# BigQuery Slot pricing (reserved capacity)
self.slot_cost_per_year = 2400 # $2,400 per slot per year
self.slots_per_tb_hour = 1000 # Rough estimate: 1000 slots
```

PYTHON

can process 1TB/hour

```
def calculate_on_demand_vs_slots(self, monthly_tb_scanned):
    """Compare on-demand vs reserved slots pricing"""

    # On-demand pricing
    on_demand_monthly = monthly_tb_scanned *
```

PYTHON

```
self.on_demand_cost_per_tb
```

```
# Reserved slots pricing
# Assume we need to process the monthly volume in 20
```

```
business days (160 hours)
```

```
tb_per_hour = monthly_tb_scanned / 160
required_slots = tb_per_hour * self.slots_per_tb_hour

slots_monthly = (required_slots * self.slot_cost_per_year) /
```

PYTHON

12

```
savings = on_demand_monthly - slots_monthly
breakeven_tb = (self.slot_cost_per_year / 12) /
```

PYTHON

```
self.on_demand_cost_per_tb
```

```
return {
    'monthly_tb_scanned': monthly_tb_scanned,
    'on_demand_cost': on_demand_monthly,
    'reserved_slots_cost': slots_monthly,
    'savings_amount': savings,
    'savings_percent': (savings / on_demand_monthly) * 100
```

if on_demand_monthly > 0 else 0,

```
'breakeven_tb_monthly': breakeven_tb,
'recommendation': 'Reserved Slots' if savings > 0 else
```

'On-Demand',

```
'required_slots': required_slots
}

def optimize_query_for_cost(self, query_analysis):
    """Generate cost optimization recommendations for a BigQuery
```

PYTHON

query"""

```

recommendations = []
potential_savings = 0

# Partition pruning optimization
if not query_analysis.get('uses_partition_pruning'):
    scan_reduction = query_analysis['bytes_scanned'] * 0.8

```

Assume 80% reduction

```

cost_savings = (scan_reduction / (1024**4)) *

```

```

self.on_demand_cost_per_tb # Convert to TB

```

```

recommendations.append({
    'optimization': 'Partition Pruning',
    'description': 'Add partition filters to reduce data

```

scanned',

```

    'potential_savings_monthly': cost_savings *

```

```

query_analysis['monthly_executions'],

```

```

    'implementation_effort': 'Low',
    'bytes_savings': scan_reduction
})
potential_savings += cost_savings *

```

```

query_analysis['monthly_executions']

```

```

# Clustering optimization
if query_analysis.get('has_where_clauses') and not

```

SQL

```

query_analysis.get('uses_clustering'):

```

```
scan_reduction = query_analysis['bytes_scanned'] * 0.4
```

Assume 40% reduction

```
cost_savings = (scan_reduction / (1024**4)) *
```

```
self.on_demand_cost_per_tb
```

```
recommendations.append({  
    'optimization': 'Table Clustering',  
    'description': 'Cluster tables on frequently
```

filtered columns',

```
'potential_savings_monthly': cost_savings *
```

```
query_analysis['monthly_executions'],
```

```
'implementation_effort': 'Medium',  
    'bytes_savings': scan_reduction  
})  
potential_savings += cost_savings *
```

```
query_analysis['monthly_executions']
```

```
# Materialized view opportunity  
if query_analysis['monthly_executions'] > 100:    # Frequently
```

run query

```

current_monthly_cost = (
    (query_analysis['bytes_scanned'] / (1024**4))
    * self.on_demand_cost_per_tb
    * query_analysis['monthly_executions']
)

# Materialized view cost: storage + refresh cost
estimated_mv_size_tb =

```

```

query_analysis['result_size_bytes'] / (1024**4)

```

```

mv_storage_cost = estimated_mv_size_tb *

```

```

self.storage_cost_per_tb_month

```

```

mv_refresh_cost = current_monthly_cost * 0.1 # Assume

```

10% of original cost for refresh

```

mv_total_cost = mv_storage_cost + mv_refresh_cost
mv_savings = current_monthly_cost - mv_total_cost

if mv_savings > 0:
    recommendations.append({
        'optimization': 'Materialized View',
        'description': 'Pre-compute frequently accessed

```

results',

```

'potential_savings_monthly': mv_savings,
    'implementation_effort': 'High',
    'storage_cost': mv_storage_cost,
    'refresh_cost': mv_refresh_cost
})
potential_savings += mv_savings

return {
    'total_potential_savings': potential_savings,
    'recommendations': sorted(recommendations, key=lambda x:

```

```
x['potential_savings_monthly'], reverse=True)
```

```
}
```

```
# Example usage
```

```
optimizer = BigQueryCostOptimizer()
```

```
# Analyze whether to use reserved slots
```

```
usage_analysis =
```

```
optimizer.calculate_on_demand_vs_slots(monthly_tb_scanned=50)
```

```
print(f"Recommendation: {usage_analysis['recommendation']}")
```

```
print(f"Potential savings:
```

PYTHON

```
${usage_analysis['savings_amount']:.2f}/month")
```

```
# Optimize a specific query
```

```
query_analysis = {
```

```
    'bytes_scanned': 1024**4 * 2, # 2 TB
```

```
    'result_size_bytes': 1024**3 * 100, # 100 GB
```

```
    'monthly_executions': 200,
```

```
    'uses_partition_pruning': False,
```

```
    'uses_clustering': False,
```

```
    'has_where_clauses': True
```

```
}
```

```
query_optimization =
```

SQL

```
optimizer.optimize_query_for_cost(query_analysis)
```

```
print(f"Total potential savings:
```

PYTHON

```
${query_optimization['total_potential_savings']:.2f}/month")
```

```
for rec in query_optimization['recommendations']:
```

```
    print(f"- {rec['optimization']}:")
```

PYTHON

```
${rec['potential_savings_monthly']:.2f}/month")
```

Multi-Cloud Cost Optimization Strategies

When Multi-Cloud Makes Financial Sense:

PYTHON

```
class MultiCloudCostAnalyzer:
    def __init__(self):
        self.platform_costs = {
            'snowflake_aws': {
                'storage_tb_month': 23.0,
                'compute_credit': 2.0,
                'data_transfer_gb': 0.09 # Egress from AWS
            },
            'bigquery_gcp': {
                'storage_tb_month': 20.0,
                'query_tb': 5.0,
                'data_transfer_gb': 0.12 # Egress from GCP
            },
            'redshift_aws': {
                'node_hour': 0.25, # ra3.xlplus
                'storage_tb_month': 24.0, # Managed storage
                'data_transfer_gb': 0.09
            },
            'synapse_azure': {
                'dwu_hour': 1.20, # DW100c
                'storage_tb_month': 23.0,
                'data_transfer_gb': 0.087
            }
        }

        # Cross-cloud data transfer costs
        self.cross_cloud_transfer_costs = {
            ('aws', 'gcp'): 0.09,
            ('gcp', 'aws'): 0.12,
            ('aws', 'azure'): 0.09,
            ('azure', 'aws'): 0.087,
            ('gcp', 'azure'): 0.12,
            ('azure', 'gcp'): 0.087
        }

    def analyze_workload_placement(self, workloads):
        """Determine optimal cloud placement for different
```

```
workloads"""
```



```

recommendations = []

for workload in workloads:
    placement_costs = {}

    for platform, costs in self.platform_costs.items():
        cloud = platform.split('_')[1] # Extract cloud

```

provider

```

# Calculate workload cost on this platform
if 'snowflake' in platform or 'redshift' in platform

```

```

or 'synapse' in platform:

```

```

# Compute + storage model
compute_cost = workload['compute_hours'] *

```

```

costs.get('compute_credit', costs.get('node_hour', costs.get('dwu_hour')))

```

```

storage_cost = workload['storage_tb'] *

```

```

costs['storage_tb_month']

```

```

elif 'bigquery' in platform:
    # Scan + storage model
    compute_cost = workload['tb_scanned_monthly'] *

```

```

costs['query_tb']

```

```

storage_cost = workload['storage_tb'] *

```

```

costs['storage_tb_month']

```

```
# Add data transfer costs
transfer_cost = 0
for source_cloud, transfer_gb in
```

```
workload.get('data_sources', {}).items():
```

```
    if source_cloud != cloud:
        transfer_rate =
```

```
        self.cross_cloud_transfer_costs.get((source_cloud, cloud), 0.10)
```

```
        transfer_cost += transfer_gb * transfer_rate
```

```
total_cost = compute_cost + storage_cost +
```

transfer_cost

```
placement_costs[platform] = {
    'compute_cost': compute_cost,
    'storage_cost': storage_cost,
    'transfer_cost': transfer_cost,
    'total_cost': total_cost
}
```

```
# Find cheapest placement
best_placement = min(placement_costs.items(), key=lambda
```

x: x[1]['total_cost'])

```

recommendations.append({
    'workload_name': workload['name'],
    'recommended_platform': best_placement[0],
    'monthly_cost': best_placement[1]['total_cost'],
    'cost_breakdown': best_placement[1],
    'alternatives': placement_costs
})

return recommendations

def calculate_multi_cloud_tax(self, current_setup,

```

workload_distribution):

```

"""Calculate the operational overhead of multi-cloud

```

setup"""

```

clouds_used = set()
for workload, platform in workload_distribution.items():
    cloud = platform.split('_')[1]
    clouds_used.add(cloud)

num_clouds = len(clouds_used)

operational_overhead = {
    1: 0,      # Single cloud - no overhead
    2: 0.15,   # 15% overhead for dual-cloud
    3: 0.30    # 30% overhead for tri-cloud
}

base_costs = sum(
    current_setup[workload]['total_cost']
    for workload in workload_distribution.keys()
)

```

```

overhead_multiplier = operational_overhead.get(num_clouds,

```

```

0.30)

```

```

overhead_cost = base_costs * overhead_multiplier

return {
    'clouds_used': list(clouds_used),
    'operational_overhead_percent': overhead_multiplier *

```

100,

```

'monthly_overhead_cost': overhead_cost,
'total_monthly_cost': base_costs + overhead_cost,
'recommendation': 'Single Cloud' if overhead_cost >

```

```

base_costs * 0.1 else 'Multi-Cloud'

```

```

}

# Example: Analyze workload placement across clouds
analyzer = MultiCloudCostAnalyzer()

workloads = [
    {
        'name': 'real_time_analytics',
        'compute_hours': 720, # 24/7 processing
        'storage_tb': 50,
        'tb_scanned_monthly': 200,
        'data_sources': {'aws': 1000, 'gcp': 500} # GB transferred

```

from each cloud

```

    },
    {
        'name': 'batch_reporting',
        'compute_hours': 100, # Nightly batches
        'storage_tb': 200,
        'tb_scanned_monthly': 50,
        'data_sources': {'aws': 2000}
    },
    {
        'name': 'ml_experimentation',
        'compute_hours': 200,
        'storage_tb': 30,
        'tb_scanned_monthly': 150,
        'data_sources': {'gcp': 500, 'azure': 300}
    }
]

placement_recommendations =

```

```
analyzer.analyze_workload_placement(workloads)
```

```

for rec in placement_recommendations:
    print(f"\nWorkload: {rec['workload_name']}")
    print(f"Recommended: {rec['recommended_platform']}")
    print(f"Monthly cost: ${rec['monthly_cost']:.2f}")

```

PYTHON

Pro Tip Multi-cloud isn't always about cost savings – sometimes it's about avoiding vendor lock-in or meeting compliance requirements. When evaluating multi-cloud, include the “operational tax” of managing multiple platforms. A 20% cost savings can quickly disappear if you need to hire additional engineers to manage complexity.

Reserved Capacity vs On-Demand Optimization

Capacity Planning for Reserved Instances:

```
import numpy as np
from scipy.optimize import minimize

class ReservedCapacityOptimizer:
    def __init__(self, platform='redshift'):
        self.platform = platform

        if platform == 'redshift':
            self.on_demand_hourly = {
                'ra3.xlplus': 3.26,
```

The PYTHON code:

```
'ra3.4xlarge': 13.04,
    'ra3.16xlarge': 52.16
    }
    self.reserved_discounts = {
        '1_year': 0.42, # 42% discount
        '3_year': 0.64 # 64% discount
    }

    elif platform == 'bigquery':
        self.on_demand_tb = 5.0
        self.slot_cost_annual = 2400 # per slot

    def analyze_usage_patterns(self, historical_usage_hours):
        """Analyze historical usage to recommend reserved
```

capacity"""

```
usage_stats = {
    'mean_daily_hours': np.mean(historical_usage_hours),
    'median_daily_hours': np.median(historical_usage_hours),
    'p95_daily_hours': np.percentile(historical_usage_hours,
```

95),

```
'min_daily_hours': np.min(historical_usage_hours),
'max_daily_hours': np.max(historical_usage_hours),
'variability': np.std(historical_usage_hours) /
```

```
np.mean(historical_usage_hours)
```

PYTHON

```
}

# Determine base capacity for reserved instances
# Conservative approach: reserve for median usage
base_capacity_hours = usage_stats['median_daily_hours']

# Calculate cost scenarios
scenarios = self.calculate_cost_scenarios(
    base_capacity_hours,
    historical_usage_hours
)

return {
    'usage_statistics': usage_stats,
    'recommended_reserved_hours': base_capacity_hours,
    'cost_scenarios': scenarios,
    'savings_analysis': self.compare_savings(scenarios)
}

def calculate_cost_scenarios(self, reserved_hours_daily,
```

actual_usage_hours):

```
"""Calculate costs for different reserved capacity
```

scenarios"""

```
scenarios = {}

# All on-demand
all_on_demand_cost = np.sum(actual_usage_hours) *
```

```
self.on_demand_hourly['ra3.xlplus'] * 30 # Monthly
```

```

scenarios['all_on_demand'] = {
    'monthly_cost': all_on_demand_cost,
    'reserved_cost': 0,
    'on_demand_cost': all_on_demand_cost
}

# Reserved capacity scenarios
for term, discount in self.reserved_discounts.items():
    reserved_hourly_cost =

```

```

self.on_demand_hourly['ra3.xlplus'] * (1 - discount)

```

```

reserved_monthly_cost = reserved_hours_daily *

```

```

reserved_hourly_cost * 30

```

```

# Calculate overage (when usage exceeds reserved

```

```

capacity)

```

```

overage_hours = np.maximum(0, actual_usage_hours -

```

```

reserved_hours_daily)

```

```

overage_monthly_cost = np.sum(overage_hours) *

```

```

self.on_demand_hourly['ra3.xlplus']

```

```

total_monthly_cost = reserved_monthly_cost +

```

```

overage_monthly_cost

```



```

scenarios[f'reserved_{term}'] = {
    'monthly_cost': total_monthly_cost,
    'reserved_cost': reserved_monthly_cost,
    'on_demand_cost': overage_monthly_cost,
    'reserved_hours_daily': reserved_hours_daily,
    'discount_percent': discount * 100
}

return scenarios

def compare_savings(self, scenarios):
    """Compare savings across different capacity strategies"""

    baseline_cost = scenarios['all_on_demand']['monthly_cost']

    savings_comparison = []

    for scenario_name, scenario in scenarios.items():
        if scenario_name != 'all_on_demand':
            savings_amount = baseline_cost -

```

```
scenario['monthly_cost']
```

```
savings_percent = (savings_amount / baseline_cost) *
```

100

```

savings_comparison.append({
    'scenario': scenario_name,
    'monthly_cost': scenario['monthly_cost'],
    'savings_amount': savings_amount,
    'savings_percent': savings_percent,
    'break_even_utilization':

```

```
self.calculate_break_even(scenario_name)
```

```
})
```

```

# Sort by savings amount
savings_comparison.sort(key=lambda x: x['savings_amount'],

```

```
reverse=True)
```

```
return savings_comparison
```

PYTHON

```
def calculate_break_even(self, scenario_name):  
    """Calculate minimum utilization needed to break even on
```

```
reserved capacity"""
```

```
if 'reserved_1_year' in scenario_name:  
    discount = self.reserved_discounts['1_year']  
elif 'reserved_3_year' in scenario_name:  
    discount = self.reserved_discounts['3_year']  
else:  
    return None  
  
# Break-even occurs when reserved cost savings equal
```

PYTHON

```
commitment cost
```

```
break_even_utilization = 1 - discount  
  
    return {  
        'daily_hours_needed': 24 * break_even_utilization,  
        'utilization_percent': break_even_utilization * 100  
    }  
  
# Usage example: Analyze 90 days of historical usage  
np.random.seed(42)  
# Simulate variable usage: higher during business days, lower on
```

```
weekends
```

```
business_day_usage = np.random.normal(18, 3, 65) # Average 18 hours
```

```
on business days
```

```
weekend_usage = np.random.normal(8, 2, 25) # Average 8 hours on
```

weekends

```
historical_usage = np.concatenate([business_day_usage,
```

weekend_usage])

```
historical_usage = np.clip(historical_usage, 0, 24) # Ensure valid
```

hours

```
optimizer = ReservedCapacityOptimizer('redshift')
analysis = optimizer.analyze_usage_patterns(historical_usage)

print("Usage Statistics:")
for stat, value in analysis['usage_statistics'].items():
    print(f" {stat}: {value:.2f}")

print(f"\nRecommended reserved capacity:
```

PYTHON

```
{analysis['recommended_reserved_hours']:.1f} hours/day")
```

```
print("\nCost Scenarios:")
for scenario_name, scenario in analysis['cost_scenarios'].items():
    print(f" {scenario_name}:
```

PYTHON

```
${scenario['monthly_cost']:.2f}/month")
```

```
print("\nSavings Analysis:")
for savings in analysis['savings_analysis']:
    print(f" {savings['scenario']}:  
${savings['savings_amount']:.2f}/month ({savings['savings_percent']:.1f}% savings)")
```

PYTHON

- □ Checkpoint
 1. How do Snowflake, BigQuery, and Redshift pricing models differ, and what does this mean for optimization strategies?
 2. When does multi-cloud make financial sense, and what are the hidden costs?
 3. How do you determine the optimal amount of reserved capacity to purchase?

⚡ TL;DR

Auto-scaling saves money but adds complexity — design scaling policies based on business patterns - Spot instances can reduce costs by 50-70% for fault-tolerant workloads like batch processing - Workload scheduling (running non-critical jobs during low-cost hours) can cut compute costs dramatically - Right-sizing is more important than auto-scaling — most teams over-provision by 2-3x

The biggest compute cost optimization isn't technical — it's operational. Most data teams provision for peak capacity 24/7 because it's easier than building systems that scale intelligently. But peak capacity might only be needed 10% of the time.

•
After implementing compute cost optimization across different data platforms, I've found that the biggest savings come from three areas: intelligent auto-scaling, spot instance strategies, and workload scheduling.

Intelligent Auto-Scaling Strategies

Business-Aware Auto-Scaling:

```
import pandas as pd
from datetime import datetime, timedelta
import numpy as np

class BusinessAwareAutoScaler:
    def __init__(self, platform='snowflake'):
        self.platform = platform
        self.business_patterns = self.load_business_patterns()

    def load_business_patterns(self):
```

PYTHON

```
"""Load known business patterns that affect compute needs"""  
return {  
    'daily_patterns': {  
        'business_hours': {'start': 8, 'end': 18,
```

```
'timezone': 'US/Pacific'},
```

```
    'peak_hours': {'start': 9, 'end': 11, 'multiplier':
```

```
2.0},
```

```
    'evening_batch': {'start': 20, 'end': 23,
```

```
    'multiplier': 1.5}
```

```
},  
    'weekly_patterns': {  
        'weekdays': {'multiplier': 1.0},  
        'saturday': {'multiplier': 0.3},  
        'sunday': {'multiplier': 0.2}  
    },  
    'seasonal_patterns': {  
        'black_friday': {'date_range': ['11-24', '11-29'],
```

```
    'multiplier': 5.0},
```

```
    'end_of_quarter': {'days_before_quarter_end': 5,
```

```
    'multiplier': 2.0},
```

PYTHON

```

'holidays': {'multiplier': 0.1}
    }
}

def predict_required_capacity(self, timestamp, base_capacity):
    """Predict required capacity based on business patterns"""

    # Start with base capacity
    predicted_capacity = base_capacity

    # Apply daily patterns
    hour = timestamp.hour
    if self.business_patterns['daily_patterns']

```

```

['business_hours']['start']    <=    hour    <=    self.business_patterns['daily_patterns']
['business_hours']['end']:

```

PYTHON

```

# During business hours
if self.business_patterns['daily_patterns']

```

```

['peak_hours']['start']    <=    hour    <=    self.business_patterns['daily_patterns']
['peak_hours']['end']:

```

```

    predicted_capacity *=

```

```

self.business_patterns['daily_patterns']['peak_hours']['multiplier']

```

PYTHON

```

elif self.business_patterns['daily_patterns']

```

```

['evening_batch']['start']    <=    hour    <=    self.business_patterns['daily_patterns']
['evening_batch']['end']:

```

```

# Evening batch processing
predicted_capacity *=

```

```

self.business_patterns['daily_patterns']['evening_batch'] ['multiplier']

```

```

else:
    # Off hours - reduce capacity
    predicted_capacity *= 0.5

# Apply weekly patterns
weekday = timestamp.weekday()
if weekday == 5: # Saturday
    predicted_capacity *=

```

`self.business_patterns['weekly_patterns']['saturday']['multiplier']`

```

elif weekday == 6: # Sunday
    predicted_capacity *=

```

PYTHON

`self.business_patterns['weekly_patterns']['sunday']['multiplier']`

```

# Apply seasonal patterns
predicted_capacity *=

```

```

self.get_seasonal_multiplier(timestamp)

```

```

return max(predicted_capacity, base_capacity * 0.1)    # Never

```

go below 10% of base

```

def get_seasonal_multiplier(self, timestamp):
    """Get seasonal capacity multiplier"""

    # Check for specific date ranges

```

PYTHON

The PYTHON code:

```

month_day = timestamp.strftime('%m-%d')

# Black Friday period
if '11-24' <= month_day <= '11-29':
    return self.business_patterns['seasonal_patterns']

```

PYTHON

['black_friday']['multiplier']

```
# End of quarter (approximate)
if timestamp.day >= 26 and timestamp.month in [3, 6, 9, 12]:
    return self.business_patterns['seasonal_patterns']
```

PYTHON

['end_of_quarter']['multiplier']

```
# Major holidays (simplified)
holidays = ['01-01', '07-04', '12-25', '12-26']
if month_day in holidays:
    return self.business_patterns['seasonal_patterns']
```

PYTHON

['holidays']['multiplier']

```
return 1.0 # Default multiplier

def generate_scaling_schedule(self, start_date, days=30):
    """Generate a scaling schedule for the next N days"""

    schedule = []
    current_date = start_date

    for day in range(days):
        daily_schedule = []

        # Generate hourly capacity predictions
        for hour in range(24):
            timestamp = current_date.replace(hour=hour,
```

PYTHON

```
minute=0, second=0)
```

```
predicted_capacity =
```

```
self.predict_required_capacity(timestamp, base_capacity=1.0)
```

```
daily_schedule.append({
    'timestamp': timestamp,
    'predicted_capacity_multiplier':
```

predicted_capacity,

```
'recommended_action':
```

```
self.get_scaling_action(predicted_capacity)
```

```
    })

    schedule.extend(daily_schedule)
    current_date += timedelta(days=1)

    return schedule

def get_scaling_action(self, capacity_multiplier):
    """Determine scaling action based on capacity prediction"""

    if capacity_multiplier >= 2.0:
        return 'scale_up_aggressive'
    elif capacity_multiplier >= 1.5:
        return 'scale_up_moderate'
    elif capacity_multiplier <= 0.3:
        return 'scale_down_aggressive'
    elif capacity_multiplier <= 0.7:
        return 'scale_down_moderate'
    else:
        return 'maintain'

# Snowflake-specific auto-scaling implementation
class SnowflakeAutoScaler(BusinessAwareAutoScaler):
    def __init__(self):
        super().__init__('snowflake')
        self.warehouse_sizes = [
            'X-Small', 'Small', 'Medium', 'Large',
            'X-Large', '2X-Large', '3X-Large', '4X-Large'
        ]
```

PYTHON

The scale_warehouse implementation:

```
def scale_warehouse(self, warehouse_name,
```

PYTHON

target_capacity_multiplier):

```
    """Scale Snowflake warehouse based on predicted capacity
```

needs"""

```
    current_size =
```

```
    self.get_current_warehouse_size(warehouse_name)
```

```
    current_index = self.warehouse_sizes.index(current_size)
```

PYTHON

```
    # Calculate target size index
    if target_capacity_multiplier >= 2.0:
        target_index = min(current_index + 2,
```

```
len(self.warehouse_sizes) - 1)
```

```
    elif target_capacity_multiplier >= 1.5:
        target_index = min(current_index + 1,
```

PYTHON

```
len(self.warehouse_sizes) - 1)
```

```

elif target_capacity_multiplier <= 0.3:
    target_index = max(current_index - 2, 0)
elif target_capacity_multiplier <= 0.7:
    target_index = max(current_index - 1, 0)
else:
    target_index = current_index

target_size = self.warehouse_sizes[target_index]

if target_size != current_size:
    scaling_decision = {
        'warehouse_name': warehouse_name,
        'current_size': current_size,
        'target_size': target_size,
        'capacity_multiplier': target_capacity_multiplier,
        'timestamp': datetime.now(),
        'cost_impact':

```

```

self.calculate_cost_impact(current_size, target_size)

```

```

}

    # Execute scaling (would call Snowflake API)
    self.execute_warehouse_scaling(scaling_decision)

    return scaling_decision

return None

def calculate_cost_impact(self, current_size, target_size):
    """Calculate cost impact of scaling decision"""

    warehouse_costs = {
        'X-Small': 1, 'Small': 2, 'Medium': 4, 'Large': 8,
        'X-Large': 16, '2X-Large': 32, '3X-Large': 64, '4X-

```

Large': 128

```

    }

    current_credits_hour = warehouse_costs[current_size]
    target_credits_hour = warehouse_costs[target_size]

    hourly_cost_change = (target_credits_hour -

```

```

current_credits_hour) * 2.0 # $2/credit

```

```

    return {
        'hourly_cost_change': hourly_cost_change,
        'daily_cost_change': hourly_cost_change * 24,
        'scaling_direction': 'up' if hourly_cost_change > 0 else

```

'down'

```

    }

    def get_current_warehouse_size(self, warehouse_name):
        """Get current warehouse size (mock implementation)"""
        # In reality, this would call Snowflake API
        return 'Medium'

    def execute_warehouse_scaling(self, scaling_decision):
        """Execute warehouse scaling (mock implementation)"""

```

PYTHON

In reality, this would call Snowflake API:

```

# In reality, this would call Snowflake API
print(f"Scaling {scaling_decision['warehouse_name']} from

```

SQL

{scaling_decision['current_size']} to {scaling_decision['target_size']}")

```
# Usage example
scaler = SnowflakeAutoScaler()

# Generate scaling schedule for next 7 days
start_date = datetime.now()
schedule = scaler.generate_scaling_schedule(start_date, days=7)

# Show peak and off-peak periods
peak_periods = [s for s in schedule if
```

```
s['predicted_capacity_multiplier'] >= 1.5]
```

```
off_peak_periods = [s for s in schedule if
```

```
s['predicted_capacity_multiplier'] <= 0.5]
```

```
print(f"Peak periods (next 7 days): {len(peak_periods)} hours")
print(f"Off-peak periods: {len(off_peak_periods)} hours")
print(f"Potential cost savings from right-sizing:
```

SQL

```
{len(off_peak_periods) * 0.5 * 24:.1f}% compute cost reduction")
```

Spot Instance Strategies for Batch Processing

Fault-Tolerant Batch Processing with Spot Instances:

```
import boto3
import time
from typing import List, Dict, Optional

class SpotInstanceBatchProcessor:
    def __init__(self):
        self.ec2 = boto3.client('ec2')
        self.batch = boto3.client('batch')

        self.spot_configurations = {
            'data_processing': {
                'instance_types': ['m5.large', 'm5.xlarge',
```

PYTHON

```
'c5.large', 'c5.xlarge'],
```

```
    'max_price': '0.05', # 50% of on-demand price
    'min_instances': 2,
    'max_instances': 20,
    'target_capacity': 10
},
'ml_training': {
    'instance_types': ['p3.2xlarge', 'p3.8xlarge',
```

```
'g4dn.xlarge'],
```

```
    'max_price': '0.80', # 60% of on-demand price
    'min_instances': 1,
    'max_instances': 5,
    'target_capacity': 2
}
}
```

PYTHON

```
def get_spot_price_history(self, instance_types: List[str],
```

hours_back: int = 168):

```
    """Get spot price history to inform bidding strategy"""

    end_time = time.time()
    start_time = end_time - (hours_back * 3600)

    price_history = self.ec2.describe_spot_price_history(
        InstanceTypes=instance_types,
        ProductDescriptions=['Linux/UNIX'],
        StartTime=start_time,
        EndTime=end_time
    )

    # Analyze price patterns
    prices_by_type = {}
    for price_record in price_history['SpotPriceHistory']:
```

PYTHON

```
instance_type = price_record['InstanceType']
if instance_type not in prices_by_type:
    prices_by_type[instance_type] = []
```

```
prices_by_type[instance_type].append(float(price_record['SpotPrice']))
```

```
price_analysis = {}
for instance_type, prices in prices_by_type.items():
    price_analysis[instance_type] = {
        'current_price': prices[0] if prices else 0,
        'average_price': sum(prices) / len(prices),
        'min_price': min(prices),
        'max_price': max(prices),
        'volatility':
```

```
self.calculate_price_volatility(prices),
```

```
'recommended_bid':
```

```
self.calculate_optimal_bid(prices)
```

```
}
```

```
    return price_analysis
```

```
def calculate_price_volatility(self, prices: List[float]) ->
```

PYTHON

float:

```
"""Calculate price volatility as coefficient of variation"""
if len(prices) < 2:
    return 0.0

mean_price = sum(prices) / len(prices)
variance = sum((p - mean_price) ** 2 for p in prices) /
```

```
(len(prices) - 1)
```



```

std_dev = variance ** 0.5

    return std_dev / mean_price if mean_price > 0 else 0.0

def calculate_optimal_bid(self, prices: List[float]) -> float:
    """Calculate optimal bid price based on historical data"""
    if not prices:
        return 0.0

    # Sort prices to find percentiles
    sorted_prices = sorted(prices)

    # Bid at 80th percentile to balance cost and availability
    percentile_80 = sorted_prices[int(len(sorted_prices) * 0.8)]

    return percentile_80

def create_spot_fleet_request(self, workload_type: str,

```

job_queue_arn: str):

```

"""Create spot fleet for batch processing workload"""

config = self.spot_configurations[workload_type]
price_analysis =

```

```

self.get_spot_price_history(config['instance_types'])

```

```

# Build launch specifications for different instance types
launch_specifications = []
for instance_type in config['instance_types']:
    if instance_type in price_analysis:
        bid_price = min(
            price_analysis[instance_type]

```

['recommended_bid'],

```
float(config['max_price'])
)

launch_specifications.append({
    'ImageId': 'ami-0abcdef1234567890',      # Your
```

batch processing AMI

```
'InstanceType': instance_type,
'SpotPrice': str(bid_price),
'SecurityGroups': [{'GroupId': 'sg-batch-
```

processing'}],

```
'IamInstanceProfile': {'Name':
```

'BatchInstanceProfile'},

```
'UserData':
```

```
self.get_user_data_script(workload_type),
```

```
'WeightedCapacity':
```

```
self.get_instance_weight(instance_type)
```

```
})
```

```
spot_fleet_config = {
    'IamFleetRole': 'arn:aws:iam::account:role/aws-ec2-spot-
```

fleet-role',

```
'AllocationStrategy': 'diversified', # Spread across
```

AZs and instance types

PYTHON

```
'TargetCapacity': config['target_capacity'],
    'SpotPrice': config['max_price'],
    'LaunchSpecifications': launch_specifications,
    'TerminateInstancesWithExpiration': True,
    'Type': 'maintain'
}

response = self.ec2.request_spot_fleet(
    SpotFleetRequestConfig=spot_fleet_config
)

return {
    'spot_fleet_id': response['SpotFleetRequestId'],
    'workload_type': workload_type,
    'target_capacity': config['target_capacity'],
    'estimated_savings':
```

```
self.calculate_spot_savings(launch_specifications)
```

```

}

def get_instance_weight(self, instance_type: str) -> int:
    """Get instance weight for capacity planning"""

    # Simplified weighting based on vCPUs
    instance_weights = {
        'm5.large': 1, 'm5.xlarge': 2,
        'c5.large': 1, 'c5.xlarge': 2,
        'p3.2xlarge': 4, 'p3.8xlarge': 16,
        'g4dn.xlarge': 2
    }

    return instance_weights.get(instance_type, 1)

def get_user_data_script(self, workload_type: str) -> str:
    """Get user data script for instance initialization"""

    if workload_type == 'data_processing':
        return """#!/bin/bash

yum update -y
yum install -y docker
service docker start
usermod -a -G docker ec2-user

# Install AWS Batch agent
curl -o /tmp/ecs-init.rpm https://amazon-ssm-

```

[downloads.s3.amazonaws.com/amazon-linux/latest/amazon-ssm-agent.rpm](https://s3.amazonaws.com/amazon-linux/latest/amazon-ssm-agent.rpm)

```

yum install -y /tmp/ecs-init.rpm
start ecs

# Set up data processing environment
docker pull your-data-processing-image:latest
"""

    elif workload_type == 'ml_training':
        return """#!/bin/bash

yum update -y
yum install -y nvidia-docker2
service docker start

```

Install GPU drivers:

BASH

```
# Install GPU drivers
aws s3 cp --recursive s3://nvidia-drivers/latest/ /tmp/nvidia/
chmod +x /tmp/nvidia/install.sh
/tmp/nvidia/install.sh

# Set up ML training environment
docker pull your-ml-training-image:latest
"""

    return ""

    def calculate_spot_savings(self, launch_specifications:
```

```
List[Dict]) -> Dict:
```

SQL

```
"""Calculate estimated cost savings from using spot
```

```
instances"""
```

PYTHON

```
# Get on-demand prices for comparison
on_demand_prices = self.get_on_demand_prices([
    spec['InstanceType'] for spec in launch_specifications
])

total_spot_cost = 0
total_on_demand_cost = 0

for spec in launch_specifications:
    instance_type = spec['InstanceType']
    spot_price = float(spec['SpotPrice'])
    weight = spec['WeightedCapacity']

    on_demand_price = on_demand_prices.get(instance_type, 0)

    total_spot_cost += spot_price * weight
    total_on_demand_cost += on_demand_price * weight

savings_percent = ((total_on_demand_cost - total_spot_cost)
```

```
/ total_on_demand_cost) * 100 if total_on_demand_cost > 0 else 0
```

```

return {
    'hourly_spot_cost': total_spot_cost,
    'hourly_on_demand_cost': total_on_demand_cost,
    'hourly_savings': total_on_demand_cost -

```

total_spot_cost,

```

'savings_percent': savings_percent
}

```

PYTHON

```

def get_on_demand_prices(self, instance_types: List[str]) ->

```

Dict[str, float]:

```

"""Get current on-demand prices (mock implementation)"""

# In reality, you'd use AWS Pricing API
mock_prices = {
    'm5.large': 0.096,
    'm5.xlarge': 0.192,
    'c5.large': 0.085,
    'c5.xlarge': 0.17,
    'p3.2xlarge': 3.06,
    'p3.8xlarge': 12.24,
    'g4dn.xlarge': 0.526
}

return {instance_type: mock_prices.get(instance_type, 0) for

```

instance_type in instance_types}

```

def monitor_spot_interruptions(self, spot_fleet_id: str):
    """Monitor spot instance interruptions and handle

```

PYTHON

gracefully"""

PYTHON

```
# Check spot fleet status
response = self.ec2.describe_spot_fleet_requests(
    SpotFleetRequestIds=[spot_fleet_id]
)
```

```
fleet_state = response['SpotFleetRequestConfigs'][0]
```

```
['SpotFleetRequestState']
```

```
interruption_handling = {
    'active': 'Fleet is running normally',
    'modifying': 'Fleet is being modified',
    'cancelled_running': 'Fleet cancelled but instances
```

```
still running',
```

```
'cancelled_terminating': 'Fleet cancelled and instances
```

```
terminating'
```

```
}

if fleet_state in ['cancelled_running',
```

```
'cancelled_terminating']:
```

PYTHON

```
# Handle interruption gracefully
return self.handle_spot_interruption(spot_fleet_id)

return {'status': interruption_handling.get(fleet_state,
```

```
'Unknown'), 'action_needed': False}
```

PYTHON

```
def handle_spot_interruption(self, spot_fleet_id: str):
    """Handle spot instance interruption by moving work to on-
```

```
demand instances"""
```

```
# Get currently running instances
running_instances =
```

```
self.get_spot_fleet_instances(spot_fleet_id)
```

```
# Launch replacement on-demand instances if needed
if running_instances:
    replacement_capacity = len(running_instances)
    on_demand_backup =
```

```
self.launch_on_demand_backup(replacement_capacity)
```

```
return {
    'interrupted_instances': len(running_instances),
    'replacement_instances':
```

```
on_demand_backup['instance_count'],
```

```
'cost_impact':
```

```
on_demand_backup['hourly_cost_increase'],
```

```
'action_taken': 'Launched on-demand backup
```

```
instances'
```

```
}
```

```
return {'status': 'No action needed', 'action_needed':
```

```
False}
```



```
# Usage example
processor = SpotInstanceBatchProcessor()

# Create spot fleet for data processing workload
spot_fleet = processor.create_spot_fleet_request(
    workload_type='data_processing',
    job_queue_arn='arn:aws:batch:region:account:job-queue/data-
```

processing'

```
)

print(f"Created spot fleet: {spot_fleet['spot_fleet_id']}")
print(f"Estimated savings: {spot_fleet['estimated_savings']
```

PYTHON

```
['savings_percent']:.1f}%")
```

```
print(f"Hourly cost: ${spot_fleet['estimated_savings']
```

PYTHON

['hourly_spot_cost':.2f} (vs \${spot_fleet['estimated_savings']
['hourly_on_demand_cost':.2f} on-demand))"

```
# Monitor for interruptions
interruption_status =
```

```
processor.monitor_spot_interruptions(spot_fleet['spot_fleet_id'])
```

```
print(f"Fleet status: {interruption_status['status']}")
```

PYTHON

Workload Scheduling for Cost Optimization

Time-Based Cost Optimization:

```
import pandas as pd
from datetime import datetime, timedelta
import pytz

class WorkloadScheduler:
    def __init__(self):
        self.cost_schedules = {
            'aws': {
                'peak_hours': {'start': 8, 'end': 20, 'multiplier':
```

```
1.0},
```

```
'off_peak_hours': {'start': 20, 'end': 8,
```

```
'multiplier': 0.7},
```

```
'weekend_discount': 0.8
},
'bigquery': {
    'flat_rate_slots': {
        'cost_per_slot_hour': 0.04,    # $40/slot/month /
```

730 hours

```
'peak_demand_multiplier': 2.0
    },
    'on_demand': {
        'cost_per_tb': 5.0,
        'no_time_discount': True
    }
}

self.workload_priorities = {
    'critical': {'max_delay_hours': 0, 'cost_tolerance':
```

'high'},

```
'high': {'max_delay_hours': 4, 'cost_tolerance':
```

'medium'},

```
'medium': {'max_delay_hours': 12, 'cost_tolerance':
```

'low'},

```
'low': {'max_delay_hours': 48, 'cost_tolerance':
```

'minimal'}

```
}
```

PYTHON

```
def calculate_time_based_costs(self, platform, workload_hours,
```

execution_time):

```
"""Calculate costs for different execution times"""
```

PYTHON

```
time_slots = []
current_time = datetime.now()

# Generate 48-hour time slots (hourly granularity)
for hour_offset in range(48):
    slot_time = current_time + timedelta(hours=hour_offset)
    cost_multiplier = self.get_cost_multiplier(platform,
```

slot_time)

```
time_slots.append({
    'start_time': slot_time,
    'end_time': slot_time +
```

```
timedelta(hours=execution_time),
```

```

'cost_multiplier': cost_multiplier,
    'estimated_cost': workload_hours * cost_multiplier,
    'is_business_hours': 8 <= slot_time.hour <= 18,
    'is_weekend': slot_time.weekday() >= 5
})

return sorted(time_slots, key=lambda x: x['estimated_cost'])

def get_cost_multiplier(self, platform, timestamp):
    """Get cost multiplier for specific time"""

    if platform not in self.cost_schedules:
        return 1.0

    schedule = self.cost_schedules[platform]
    hour = timestamp.hour
    is_weekend = timestamp.weekday() >= 5

```

```

# Base multiplier
if 'peak_hours' in schedule:
    if schedule['peak_hours']['start'] <= hour <=

```

```

schedule['peak_hours']['end']:

```

```

multiplier = schedule['peak_hours']['multiplier']
else:
    multiplier = schedule['off_peak_hours']

```

['multiplier']

```

else:
    multiplier = 1.0

    # Weekend discount
    if is_weekend and 'weekend_discount' in schedule:
        multiplier *= schedule['weekend_discount']

    return multiplier

def schedule_workloads(self, workloads, platform='aws'):
    """Schedule workloads to minimize costs while meeting

```

SLAs"""

```
scheduled_workloads = []

for workload in workloads:
    # Get possible time slots
    time_slots = self.calculate_time_based_costs(
        platform,
        workload['compute_hours'],
        workload['execution_time_hours']
    )

    # Filter slots that meet priority constraints
    priority_config =
```

PYTHON

```
self.workload_priorities[workload['priority']]
```

```
max_delay =
```

```
timedelta(hours=priority_config['max_delay_hours'])
```

```
valid_slots = [
    slot for slot in time_slots
    if slot['start_time'] <= datetime.now() + max_delay
]

if not valid_slots:
    # Use earliest available slot if no slots meet delay
```

constraint

```
valid_slots = [time_slots[0]]

# Select best slot based on cost and priority
if workload['priority'] == 'critical':
    # Schedule immediately regardless of cost
    best_slot = min(valid_slots, key=lambda x:
```

SQL

```
x['start_time'])
```

```

else:
    # Select cheapest available slot
    best_slot = min(valid_slots, key=lambda x:

```

SQL

```

x['estimated_cost'])

```

```

scheduled_workloads.append({
    'workload_name': workload['name'],
    'priority': workload['priority'],
    'scheduled_start': best_slot['start_time'],
    'scheduled_end': best_slot['end_time'],
    'estimated_cost': best_slot['estimated_cost'],
    'cost_savings':

```

```

self.calculate_cost_savings(workload, best_slot, time_slots[0]),

```

```

'delay_from_immediate': (best_slot['start_time'] -

```

SQL

```

datetime.now()).total_seconds() / 3600

```

```

}))

return sorted(scheduled_workloads, key=lambda x:

```

PYTHON

```

x['scheduled_start'])

```

```

def calculate_cost_savings(self, workload, scheduled_slot,

```

PYTHON

immediate_slot):

```

"""Calculate cost savings from scheduling vs immediate

```

SQL

execution"""

```
immediate_cost = workload['compute_hours'] *
```

```
immediate_slot['cost_multiplier']
```

```
scheduled_cost = scheduled_slot['estimated_cost']
```

```
savings_amount = immediate_cost - scheduled_cost
```

```
savings_percent = (savings_amount / immediate_cost) * 100 if
```

immediate_cost > 0 else 0

```
return {
    'amount': savings_amount,
    'percent': savings_percent,
    'immediate_cost': immediate_cost,
    'scheduled_cost': scheduled_cost
}

def generate_monthly_schedule(self, recurring_workloads):
    """Generate optimized schedule for recurring workloads"""

    monthly_schedule = {}
    total_savings = 0

    # Process each day of the month
    for day in range(1, 32):
        try:
            schedule_date = datetime.now().replace(day=day,
```

PYTHON

```
hour=0, minute=0, second=0)
```

```
daily_workloads = []
```

```
for workload in recurring_workloads:
    if self.should_run_on_date(workload,
```

PYTHON

schedule_date):

```

daily_workloads.append({
    **workload,
    'scheduled_date': schedule_date
})

if daily_workloads:
    daily_schedule =

```

```

self.schedule_workloads(daily_workloads)

```

```

monthly_schedule[schedule_date.strftime('%Y-%m-

```

```

%d')]] = daily_schedule

```

```

# Calculate daily savings
daily_savings = sum(
    wl['cost_savings']['amount'] for wl in

```

daily_schedule

```

)
    total_savings += daily_savings

except ValueError:
    # Skip invalid dates (e.g., Feb 30)
    continue

return {
    'schedule': monthly_schedule,
    'total_monthly_savings': total_savings,
    'optimization_summary':

```

```

self.generate_optimization_summary(monthly_schedule)

```



```

}

def should_run_on_date(self, workload, date):
    """Determine if workload should run on specific date"""

    frequency = workload.get('frequency', 'daily')

    if frequency == 'daily':

```

The PYTHON code:

```

return True
    elif frequency == 'weekdays':
        return date.weekday() < 5
    elif frequency == 'weekly':
        return date.weekday() == workload.get('day_of_week', 0)
    elif frequency == 'monthly':
        return date.day == workload.get('day_of_month', 1)

    return False

def generate_optimization_summary(self, monthly_schedule):
    """Generate summary of optimization opportunities"""

    all_workloads = []
    for daily_schedule in monthly_schedule.values():
        all_workloads.extend(daily_schedule)

    if not all_workloads:
        return {}

    summary = {
        'total_workloads': len(all_workloads),
        'workloads_delayed': len([w for w in all_workloads if

```

```

w['delay_from_immediate'] > 0]),

```

```

    'average_delay_hours':

```

```

np.mean([w['delay_from_immediate'] for w in all_workloads]),

```

```
'total_cost_savings': sum(w['cost_savings']['amount']
```

```
for w in all_workloads),
```

```
'average_cost_savings_percent':
```

```
np.mean([w['cost_savings']['percent'] for w in all_workloads]),
```

```
'priority_breakdown': {}  
}
```

```
# Priority breakdown  
for priority in ['critical', 'high', 'medium', 'low']:  
    priority_workloads = [w for w in all_workloads if
```

```
w['priority'] == priority]
```

```
if priority_workloads:  
    summary['priority_breakdown'][priority] = {  
        'count': len(priority_workloads),  
        'average_delay':
```

```
np.mean([w['delay_from_immediate'] for w in priority_workloads]),
```

```
'total_savings': sum(w['cost_savings']['amount']
```

```
for w in priority_workloads)
```

```

}

    return summary

# Usage example
scheduler = WorkloadScheduler()

# Define sample workloads
workloads = [
    {
        'name': 'daily_etl_pipeline',
        'priority': 'high',
        'compute_hours': 50,
        'execution_time_hours': 2,
        'frequency': 'daily'
    },
    {
        'name': 'ml_model_training',
        'priority': 'medium',
        'compute_hours': 200,
        'execution_time_hours': 8,
        'frequency': 'weekly'
    },
    {
        'name': 'data_quality_checks',
        'priority': 'critical',
        'compute_hours': 10,
        'execution_time_hours': 0.5,

```

The PYTHON code:

PYTHON

```
'frequency': 'daily'
    },
    {
        'name': 'monthly_reporting',
        'priority': 'low',
        'compute_hours': 100,
        'execution_time_hours': 4,
        'frequency': 'monthly'
    }
]

# Schedule workloads for immediate optimization
immediate_schedule = scheduler.schedule_workloads(workloads)

print("Immediate Workload Schedule:")
for workload in immediate_schedule:
    print(f" {workload['workload_name']}:

```

```
{workload['scheduled_start'].strftime('%H:%M')} "

```

```
f"({workload['estimated_cost']:.2f},

```

```
{workload['cost_savings']['percent']:.1f}% savings)")

```

```
# Generate monthly optimization schedule
monthly_optimization =

```

```
scheduler.generate_monthly_schedule(workloads)

```

PYTHON

```
print(f"\nMonthly Optimization Summary:")
print(f" Total potential savings:

```

```
${monthly_optimization['total_monthly_savings']:.2f}")

```

PYTHON

```
print(f" Average cost savings:

```

```
{monthly_optimization['optimization_summary'] ['average_cost_savings_percent']:.1f}
%")

```

```
print(f" Workloads delayed for savings:
```

PYTHON

```
{monthly_optimization['optimization_summary'] ['workloads_delayed']})
```

📌 Pro Tip

The biggest compute cost optimization isn't technical — it's cultural. Train your team to think about cost when they write queries and design pipelines. A 30-second conversation about "do we need this to run every hour or is daily sufficient?" can save thousands of dollars.

📌 Checkpoint

1. What are the key components of business-aware auto-scaling, and why is it better than simple metric-based scaling?
2. When should you use spot instances, and how do you handle interruptions gracefully?
3. How can workload scheduling reduce compute costs without impacting business operations?

Storage Cost Optimization

⚡ TL;DR

Implement intelligent data lifecycle policies: hot → warm → cold → archive based on access patterns - Use compression and column-oriented formats (Parquet, ORC) for 5- 10x storage savings - Partition data strategically — wrong partitioning can increase costs and hurt performance - Delete unused data aggressively — most companies store 30-50% data they never access

Storage costs seem small compared to compute costs, but they add up fast when you're storing petabytes. More importantly, inefficient storage patterns lead to higher compute costs because you're scanning more data than necessary.

After optimizing storage costs across multiple data platforms, in practice, the biggest wins come from three areas: intelligent lifecycle management, compression optimization, and aggressive data pruning.

Intelligent Data Lifecycle Management

Automated Tiering Based on Access Patterns:

```
import pandas as pd
from datetime import datetime, timedelta
import boto3
from typing import Dict, List, Optional

class DataLifecycleManager:
    def __init__(self):
        self.s3 = boto3.client('s3')

        # Define storage tiers and costs (per GB per month)
        self.storage_tiers = {
            'hot': {'cost': 0.023, 'retrieval_cost': 0,
'retrieval_time': 'immediate'},
```

PYTHON

```
'warm': {'cost': 0.0125, 'retrieval_cost': 0.01,
```

```
'retrieval_time': 'minutes'},
```

```
'cold': {'cost': 0.004, 'retrieval_cost': 0.03,
```

```
'retrieval_time': 'hours'},
```

```
'archive': {'cost': 0.00099, 'retrieval_cost': 0.05,
```

```
'retrieval_time': '12 hours'},
```

```
'deep_archive': {'cost': 0.00099, 'retrieval_cost':
```

```
0.10, 'retrieval_time': '48 hours'}
```

```
}
```

PYTHON

```
# Business-defined access patterns
```

```
self.access_patterns = {
```

```
    'transactional': {'hot_days': 7, 'warm_days': 30,
```

```
'cold_days': 365},
```

```
'analytics': {'hot_days': 30, 'warm_days': 90,
```

```
'cold_days': 730},
```

```
'compliance': {'hot_days': 90, 'warm_days': 365,
```

```
'cold_days': 2555}, # 7 years
```

```
'experimental': {'hot_days': 7, 'warm_days': 30,
```

'cold_days': 180}

```
}
```

PYTHON

```
def analyze_access_patterns(self, s3_bucket: str, prefix: str =
```

") -> Dict:

```
"""Analyze historical access patterns for S3 objects"""
```

PYTHON

```
# Get object metadata and access logs
objects = self.get_s3_objects_with_metadata(s3_bucket,
```

prefix)

```
access_logs = self.get_s3_access_logs(s3_bucket, prefix,
```

PYTHON

```
days_back=365)
```

```
access_analysis = {}
```

```
for obj in objects:
    object_key = obj['Key']
    last_modified = obj['LastModified']
    size_bytes = obj['Size']
```

```
# Analyze access patterns
object_accesses = [log for log in access_logs if
```

```
log['key'] == object_key]
```

```
if object_accesses:
    last_access = max(access['timestamp'] for access in
```

object_accesses)


```

access_frequency = len(object_accesses)
    access_recency = (datetime.now() - last_access).days
else:
    last_access = last_modified
    access_frequency = 0
    access_recency = (datetime.now() -

```

```

last_modified).days

```

```

# Calculate access pattern metrics

```

```

access_analysis[object_key] = {
    'size_gb': size_bytes / (1024**3),
    'age_days': (datetime.now() - last_modified).days,
    'last_access_days': access_recency,
    'access_count_365d': access_frequency,
    'access_frequency_monthly': access_frequency / 12 if

```

```

access_frequency > 0 else 0,

```

```

    'current_storage_class': obj.get('StorageClass',

```

```

'STANDARD'),

```

```

    'recommended_tier':

```

```

self.recommend_storage_tier(access_recency, access_frequency, size_bytes)

```

```

}

```

```

    return access_analysis

```

```

def recommend_storage_tier(self, days_since_access: int,

```

```

access_frequency: int, size_bytes: int) -> str:

```

PYTHON

```

"""Recommend storage tier based on access patterns"""

# Size-based rules (very small files stay in hot tier)
if size_bytes < 128 * 1024: # < 128 KB
    return 'hot'

# Access recency rules
if days_since_access <= 7:
    return 'hot'
elif days_since_access <= 30:
    if access_frequency >= 10: # Frequently accessed
        return 'warm'
    else:
        return 'cold'
elif days_since_access <= 90:
    if access_frequency >= 5:
        return 'warm'
    else:
        return 'cold'
elif days_since_access <= 365:
    return 'cold'
else:
    return 'archive'

def calculate_tiering_savings(self, access_analysis: Dict) ->

```

Dict:

```

"""Calculate cost savings from optimal tiering"""

current_costs = {}
optimized_costs = {}
potential_savings = {}

for object_key, analysis in access_analysis.items():
    size_gb = analysis['size_gb']
    current_tier =

self.map_storage_class_to_tier(analysis['current_storage_class'])

```

```
recommended_tier = analysis['recommended_tier']
```

```
# Calculate monthly costs  
current_monthly_cost = size_gb *
```

```
self.storage_tiers[current_tier]['cost']
```

```
optimized_monthly_cost = size_gb *
```

```
self.storage_tiers[recommended_tier]['cost']
```

```
current_costs[object_key] = current_monthly_cost  
optimized_costs[object_key] = optimized_monthly_cost  
potential_savings[object_key] = current_monthly_cost -
```

optimized_monthly_cost

```
total_current_cost = sum(current_costs.values())  
total_optimized_cost = sum(optimized_costs.values())  
total_savings = sum(potential_savings.values())  
  
return {
```

```
'total_current_monthly_cost': total_current_cost,  
'total_optimized_monthly_cost': total_optimized_cost,  
'monthly_savings': total_savings,  
'savings_percentage': (total_savings /
```

total_current_cost) * 100 if total_current_cost > 0 else 0,

```

'object_level_savings': potential_savings
    }

def map_storage_class_to_tier(self, storage_class: str) -> str:
    """Map S3 storage class to internal tier"""

    mapping = {
        'STANDARD': 'hot',
        'STANDARD_IA': 'warm',
        'ONEZONE_IA': 'warm',
        'GLACIER': 'cold',
        'DEEP_ARCHIVE': 'deep_archive'
    }

    return mapping.get(storage_class, 'hot')

def create_lifecycle_policy(self, bucket_name: str,

```

access_analysis: Dict, data_type: str) -> Dict:

```

"""Create S3 lifecycle policy based on access analysis"""

pattern_config = self.access_patterns.get(data_type,

```

```

self.access_patterns['analytics'])

```

```

lifecycle_rules = []

# Group objects by common prefixes
prefix_groups = {}
for object_key, analysis in access_analysis.items():
    # Extract prefix (everything before the last '/')
    prefix = '/'.join(object_key.split('/')[:-1])
    if prefix not in prefix_groups:
        prefix_groups[prefix] = []
    prefix_groups[prefix].append(analysis)

# Create rules for each prefix group
for prefix, objects in prefix_groups.items():
    # Calculate average access patterns for this prefix
    avg_access_recency = np.mean([obj['last_access_days']

```

for obj in objects])

```

total_size_gb = sum(obj['size_gb'] for obj in objects)

# Only create rule if there's significant storage
if total_size_gb > 1: # > 1 GB
    rule = {
        'ID': f'lifecycle_rule_{prefix.replace("/",

```

```

"_"}}',

```

```

        'Status': 'Enabled',
        'Filter': {'Prefix': prefix},
        'Transitions': [
            {
                'Days': pattern_config['hot_days'],
                'StorageClass': 'STANDARD_IA'
            },
            {
                'Days': pattern_config['warm_days'],
                'StorageClass': 'GLACIER'
            },
            {
                'Days': pattern_config['cold_days'],
                'StorageClass': 'DEEP_ARCHIVE'
            }
        ]
    }

    lifecycle_rules.append(rule)

```

```

# Create lifecycle configuration

```

```

lifecycle_config = {
    'Rules': lifecycle_rules
}

return {
    'bucket_name': bucket_name,
    'lifecycle_configuration': lifecycle_config,
    'estimated_monthly_savings':

```

```

self.estimate_lifecycle_savings(access_analysis, pattern_config),

```

```
'affected_prefixes': list(prefix_groups.keys())
}
```

PYTHON

```
def estimate_lifecycle_savings(self, access_analysis: Dict,
```

pattern_config: Dict) -> float:

```
"""Estimate savings from lifecycle policy implementation"""

total_savings = 0

for object_key, analysis in access_analysis.items():
    age_days = analysis['age_days']
    size_gb = analysis['size_gb']

    # Determine what tier object would be in with lifecycle
```

SQL

policy

```
if age_days > pattern_config['cold_days']:
    target_tier = 'archive'
elif age_days > pattern_config['warm_days']:
    target_tier = 'cold'
elif age_days > pattern_config['hot_days']:
    target_tier = 'warm'
else:
    target_tier = 'hot'

current_tier =
```

PYTHON

```
self.map_storage_class_to_tier(analysis['current_storage_class'])
```

```
current_cost = size_gb *
```

```
self.storage_tiers[current_tier]['cost']
```

```
target_cost = size_gb * self.storage_tiers[target_tier]
```

PYTHON

['cost']

```
total_savings += current_cost - target_cost

    return total_savings

def get_s3_objects_with_metadata(self, bucket: str, prefix: str)
```

-> List[Dict]:

```
"""Get S3 objects with metadata (mock implementation)"""
# In reality, this would paginate through S3 objects
return [
    {
        'Key': 'data/2024/01/transactions.parquet',
        'LastModified': datetime.now() - timedelta(days=90),
        'Size': 1024**3, # 1 GB
        'StorageClass': 'STANDARD'
    },
    {
        'Key': 'data/2023/12/transactions.parquet',
        'LastModified': datetime.now() -
```

```
timedelta(days=180),
```

```
'Size': 1024**3 * 2, # 2 GB
    'StorageClass': 'STANDARD'
},
{
    'Key': 'data/2023/01/transactions.parquet',
    'LastModified': datetime.now() -
```

```
timedelta(days=450),
```

```
'Size': 1024**3, # 1 GB
    'StorageClass': 'STANDARD'
}
]
```

The get_s3_access_logs implementation:

```
def get_s3_access_logs(self, bucket: str, prefix: str,
```

PYTHON

days_back: int) -> List[Dict]:

```
"""Get S3 access logs (mock implementation)"""
# In reality, this would query CloudTrail or S3 access logs
return [
    {
        'key': 'data/2024/01/transactions.parquet',
        'timestamp': datetime.now() - timedelta(days=5),
        'operation': 'GET'
    },
    {
        'key': 'data/2023/12/transactions.parquet',
        'timestamp': datetime.now() - timedelta(days=30),
        'operation': 'GET'
    }
]

# Usage example
lifecycle_manager = DataLifecycleManager()

# Analyze access patterns for a bucket
access_analysis = lifecycle_manager.analyze_access_patterns(
    s3_bucket='my-data-bucket',
    prefix='data/'
)

# Calculate potential savings
savings_analysis =
```

```
lifecycle_manager.calculate_tiering_savings(access_analysis)
```

```
print(f"Current monthly storage cost:
```

PYTHON

```
${savings_analysis['total_current_monthly_cost']:.2f}")
```

```
print(f"Optimized monthly cost:
```

PYTHON

```
${savings_analysis['total_optimized_monthly_cost']:.2f}")
```



```
print(f"Potential monthly savings:
```

PYTHON

```
${savings_analysis['monthly_savings']:.2f} ({savings_analysis['savings_percentage']:.1f}%))")
```

```
# Create lifecycle policy
lifecycle_policy = lifecycle_manager.create_lifecycle_policy(
    bucket_name='my-data-bucket',
    access_analysis=access_analysis,
    data_type='analytics'
)

print(f"Lifecycle policy would save an estimated
```

PYTHON

```
${lifecycle_policy['estimated_monthly_savings']:.2f}/month")
```

Compression and Format Optimization

Storage Format Cost Comparison:

```
import pandas as pd
import numpy as np
from typing import Dict, List

class StorageFormatOptimizer:
    def __init__(self):
        # Compression ratios and query performance characteristics
        self.format_characteristics = {
            'csv_uncompressed': {
                'compression_ratio': 1.0,
                'query_performance': 0.1, # Relative performance
```

PYTHON

score

```
'storage_cost_multiplier': 1.0,
'compute_cost_multiplier': 10.0    # More compute
```

needed to scan

```

},
'csv_gzipped': {
  'compression_ratio': 0.3, # 70% size reduction
  'query_performance': 0.3,
  'storage_cost_multiplier': 0.3,

```

```

'compute_cost_multiplier': 8.0
},
'parquet_uncompressed': {
  'compression_ratio': 0.4, # Columnar storage

```

benefit

```

'query_performance': 1.0, # Baseline for comparison
  'storage_cost_multiplier': 0.4,
  'compute_cost_multiplier': 1.0
},
'parquet_snappy': {
  'compression_ratio': 0.2, # 80% size reduction
  'query_performance': 0.9, # Slight decompression

```

overhead

```

'storage_cost_multiplier': 0.2,
  'compute_cost_multiplier': 1.1
},
'parquet_gzip': {
  'compression_ratio': 0.15, # 85% size reduction
  'query_performance': 0.7, # More decompression

```

overhead

```

'storage_cost_multiplier': 0.15,
  'compute_cost_multiplier': 1.4
},
'orc_zlib': {
  'compression_ratio': 0.18,
  'query_performance': 0.95,
  'storage_cost_multiplier': 0.18,
  'compute_cost_multiplier': 1.05
},
'delta_lake': {
  'compression_ratio': 0.25, # Includes metadata

```

overhead

```
'query_performance': 1.1, # Benefits from
```

SQL

optimization

```

'storage_cost_multiplier': 0.25,
  'compute_cost_multiplier': 0.9
}

# Base costs per GB per month
self.base_storage_cost = 0.023 # S3 Standard
self.base_compute_cost = 0.05 # Cost per GB scanned

def analyze_current_storage(self, dataset_info: Dict) -> Dict:
    """Analyze current storage format efficiency"""

    current_format = dataset_info['current_format']
    raw_size_gb = dataset_info['raw_size_gb']
    monthly_scan_gb = dataset_info['monthly_scan_gb']

    current_chars = self.format_characteristics[current_format]

    # Calculate current costs
    actual_storage_gb = raw_size_gb *

```

PYTHON

```
current_chars['compression_ratio']
```

```
storage_cost_monthly = actual_storage_gb *
```

```
self.base_storage_cost * current_chars['storage_cost_multiplier']
```

```
effective_scan_gb = monthly_scan_gb *
```

```
current_chars['compute_cost_multiplier']
```

```
compute_cost_monthly = effective_scan_gb *
```

```
self.base_compute_cost
```

```
return {  
    'current_format': current_format,  
    'raw_size_gb': raw_size_gb,  
    'actual_storage_gb': actual_storage_gb,  
    'monthly_storage_cost': storage_cost_monthly,  
    'monthly_compute_cost': compute_cost_monthly,  
    'total_monthly_cost': storage_cost_monthly +
```

compute_cost_monthly,

```
'compression_ratio': current_chars['compression_ratio'],
```

```
'query_performance_score':
```

```
current_chars['query_performance']
```

```
}
```

PYTHON

```
def compare_storage_formats(self, dataset_info: Dict) ->
```

```
List[Dict]:
```

```
"""Compare costs across different storage formats"""
```

```
raw_size_gb = dataset_info['raw_size_gb']  
monthly_scan_gb = dataset_info['monthly_scan_gb']
```

```
format_comparisons = []
```

```
for format_name, characteristics in
```

```
self.format_characteristics.items():
```

```
# Calculate storage costs  
compressed_size_gb = raw_size_gb *
```

```
characteristics['compression_ratio']
```

```
storage_cost = compressed_size_gb *
```

```
self.base_storage_cost * characteristics['storage_cost_multiplier']
```

```
# Calculate compute costs  
effective_scan_gb = monthly_scan_gb *
```

```
characteristics['compute_cost_multiplier']
```

```
compute_cost = effective_scan_gb *
```

```
self.base_compute_cost
```

```
total_cost = storage_cost + compute_cost

format_comparisons.append({
    'format': format_name,
    'storage_size_gb': compressed_size_gb,
    'monthly_storage_cost': storage_cost,
    'monthly_compute_cost': compute_cost,
    'total_monthly_cost': total_cost,
    'compression_ratio':
```

```
characteristics['compression_ratio'],
```

```
'query_performance_score':
```

```
characteristics['query_performance'],
```

```
'relative_performance':
```

```
characteristics['query_performance'] /
characteristics['compute_cost_multiplier']
```

```
})

# Sort by total cost
return sorted(format_comparisons, key=lambda x:
```

PYTHON

```
x['total_monthly_cost'])
```

```
def recommend_optimal_format(self, dataset_info: Dict,
```

PYTHON

priorities: Dict) -> Dict:

```
"""Recommend optimal format based on priorities"""

format_comparisons =
```

```
self.compare_storage_formats(dataset_info)
```

```
current_analysis =
```

```
self.analyze_current_storage(dataset_info)
```

```
# Score formats based on priorities
weighted_scores = []

for comparison in format_comparisons:
    # Normalize metrics for scoring
    cost_score = 1 - (comparison['total_monthly_cost'] /
```

```
max(c['total_monthly_cost'] for c in format_comparisons))
```

```
performance_score =
```

```
comparison['query_performance_score']
```

```
compression_score = 1 - comparison['compression_ratio']
```

```
# Better compression = higher score
```

```
# Apply weights
total_score = (
    cost_score * priorities.get('cost_weight', 0.4) +
```

```
performance_score *
```

```
priorities.get('performance_weight', 0.4) +
```

```
compression_score *
```

```
priorities.get('compression_weight', 0.2)
```

```
)

    weighted_scores.append({
        **comparison,
        'cost_score': cost_score,
        'performance_score': performance_score,
        'compression_score': compression_score,
        'total_score': total_score
    })

# Find best format
best_format = max(weighted_scores, key=lambda x:
```

PYTHON

```
x['total_score'])
```

```
# Calculate migration benefits
current_monthly_cost =
```

```
current_analysis['total_monthly_cost']
```

```
optimal_monthly_cost = best_format['total_monthly_cost']
monthly_savings = current_monthly_cost -
```

optimal_monthly_cost

```
return {
    'current_format': dataset_info['current_format'],
    'recommended_format': best_format['format'],
    'monthly_savings': monthly_savings,
    'savings_percentage': (monthly_savings /
```

current_monthly_cost) * 100 if current_monthly_cost > 0 else 0,

```
'migration_benefits': {
    'storage_reduction_gb':
```



```
current_analysis['actual_storage_gb'] - best_format['storage_size_gb'],
```

```
'storage_savings':
```

```
current_analysis['monthly_storage_cost'] - best_format['monthly_storage_cost'],
```

```
'compute_savings':
```

```
current_analysis['monthly_compute_cost'] - best_format['monthly_compute_cost'],
```

```
'performance_impact':
```

```
best_format['query_performance_score'] -  
current_analysis['query_performance_score']
```

```
},  
    'all_format_comparisons': weighted_scores  
}  
  
def estimate_migration_cost(self, dataset_info: Dict,
```

PYTHON

target_format: str) -> Dict:

```
"""Estimate cost of migrating to new format"""  
  
raw_size_gb = dataset_info['raw_size_gb']  
  
# Migration compute costs (rough estimates)  
migration_costs = {  
    'cpu_hours_per_gb': {  
        'parquet_snappy': 0.1,      # Fast compression  
        'parquet_gzip': 0.3,       # Slower compression  
        'orc_zlib': 0.2,  
        'delta_lake': 0.4          # Additional optimization
```

overhead

```
},  
    'cost_per_cpu_hour': 0.10    # EC2 compute cost  
}  
  
cpu_hours_needed = raw_size_gb *
```

```
migration_costs['cpu_hours_per_gb'].get(target_format, 0.2)
```

```
migration_compute_cost = cpu_hours_needed *
```

```
migration_costs['cost_per_cpu_hour']
```

```
# Storage costs during migration (temporary duplication)
```

```
target_characteristics =
```

```
self.format_characteristics[target_format]
```

```
target_size_gb = raw_size_gb *
```

```
target_characteristics['compression_ratio']
```

```
temporary_storage_cost = (raw_size_gb + target_size_gb) *
```

```
self.base_storage_cost # One month of duplication
```

```
total_migration_cost = migration_compute_cost +
```

temporary_storage_cost

```
# Calculate payback period  
current_analysis =
```

```
self.analyze_current_storage(dataset_info)
```

```
target_monthly_cost = target_size_gb *
```

```
self.base_storage_cost + dataset_info['monthly_scan_gb'] * target_characteristics['compute_cost_multiplier'] * self.base_compute_cost
```

```
monthly_savings = current_analysis['total_monthly_cost'] -
```

```
target_monthly_cost
```

```
payback_months = total_migration_cost / monthly_savings if
```

```
monthly_savings > 0 else float('inf')
```

```
return {  
    'total_migration_cost': total_migration_cost,  
    'compute_cost': migration_compute_cost,  
    'temporary_storage_cost': temporary_storage_cost,  
    'monthly_savings': monthly_savings,  
    'payback_period_months': payback_months,  
    'roi_12_months': (monthly_savings * 12 -
```

```
total_migration_cost) / total_migration_cost if total_migration_cost > 0 else 0
```

```

}

# Usage example
optimizer = StorageFormatOptimizer()

# Analyze a dataset
dataset = {
    'current_format': 'csv_uncompressed',
    'raw_size_gb': 1000, # 1 TB dataset
    'monthly_scan_gb': 5000 # Scanned 5 times per month
}

# Get format recommendations
priorities = {
    'cost_weight': 0.5,
    'performance_weight': 0.3,
    'compression_weight': 0.2
}

recommendation = optimizer.recommend_optimal_format(dataset,

```

priorities)

```

print(f"Current format: {recommendation['current_format']}")
print(f"Recommended format: {recommendation['recommended_format']}")
print(f"Monthly savings: ${recommendation['monthly_savings']:.2f}

```

PYTHON

```

({recommendation['savings_percentage']:.1f}%)")

```

```

print("\nMigration benefits:")
benefits = recommendation['migration_benefits']
print(f" Storage reduction: {benefits['storage_reduction_gb']:.0f}

```

PYTHON

GB")

```

print(f" Storage cost savings:

```

PYTHON

```

${benefits['storage_savings']:.2f}/month")

```

```
print(f" Compute cost savings:
```

PYTHON

```
${benefits['compute_savings']:.2f}/month")
```

```
# Estimate migration cost
migration_analysis = optimizer.estimate_migration_cost(dataset,
```

```
recommendation['recommended_format'])
```

```
print(f"\nMigration analysis:")
```

PYTHON

The PYTHON code:

```
print(f" Total migration cost:
```

PYTHON

```
${migration_analysis['total_migration_cost']:.2f}")
```

```
print(f" Payback period:
```

PYTHON

```
{migration_analysis['payback_period_months']:.1f} months")
```

```
print(f" 12-month ROI:
```

PYTHON

```
{migration_analysis['roi_12_months']*100:.1f}%")
```

Intelligent Data Pruning and Retention

Data Value Assessment Framework:

```

from datetime import datetime, timedelta
from typing import Dict, List, Tuple

class DataRetentionOptimizer:
    def __init__(self):
        # Define data value depreciation models
        self.value_models = {
            'transactional': {
                'initial_value': 1.0,
                'half_life_days': 30,          # Value halves every 30

```

days

```

        'minimum_value': 0.1,          # Retains 10% value long-

```

term

```

        'legal_retention_days': 2555 # 7 years
    },
    'behavioral': {
        'initial_value': 1.0,
        'half_life_days': 90,          # Behavioral patterns

```

stable longer

```

        'minimum_value': 0.05,
        'legal_retention_days': 1095   # 3 years
    },
    'experimental': {
        'initial_value': 1.0,
        'half_life_days': 7,           # Experiments lose value

```

quickly

```

        'minimum_value': 0.01,
        'legal_retention_days': 90     # 3 months
    },
    'compliance': {
        'initial_value': 0.3,          # Lower business value

```

initially

```
'half_life_days': 365,      # Stable compliance value
'minimum_value': 0.3,      # Maintains value for
```

legal reasons

```
'legal_retention_days': 2555  # 7 years
    }
}

self.storage_cost_per_gb_month = 0.023
self.query_cost_per_gb = 0.05

def calculate_data_value(self, data_type: str, age_days: int,
```

PYTHON

size_gb: float, monthly_access_count: int) -> Dict:

```
"""Calculate current business value of data"""

if data_type not in self.value_models:
    data_type = 'transactional' # Default model

model = self.value_models[data_type]

# Calculate time-based value depreciation
time_value = max(
    model['minimum_value'],
    model['initial_value'] * (0.5 ** (age_days /
```

PYTHON

```
model['half_life_days'])))
```

```
)

# Calculate access-based value (recent access increases
```

value)

```
access_multiplier = min(2.0, 1 + (monthly_access_count /
```

10)) # Up to 2x value for high access

```
current_value = time_value * access_multiplier

# Calculate costs
monthly_storage_cost = size_gb *
```

```
self.storage_cost_per_gb_month
```

```
monthly_query_cost = size_gb * monthly_access_count *
```

```
self.query_cost_per_gb
```

```
total_monthly_cost = monthly_storage_cost +
```

monthly_query_cost

```
# Value-to-cost ratio
value_cost_ratio = current_value / total_monthly_cost if
```

total_monthly_cost > 0 else 0

```
return {
    'data_type': data_type,
    'age_days': age_days,
    'size_gb': size_gb,
    'current_value': current_value,
    'time_value': time_value,
    'access_multiplier': access_multiplier,
    'monthly_storage_cost': monthly_storage_cost,
    'monthly_query_cost': monthly_query_cost,
    'total_monthly_cost': total_monthly_cost,
    'value_cost_ratio': value_cost_ratio,
    'legal_retention_days': model['legal_retention_days']
}

def identify_deletion_candidates(self, data_inventory:
```

PYTHON


```
List[Dict]) -> List[Dict]:
```

```
"""Identify data that can be safely deleted"""

deletion_candidates = []

for dataset in data_inventory:
    value_analysis = self.calculate_data_value(
        dataset['data_type'],
        dataset['age_days'],
        dataset['size_gb'],
        dataset['monthly_access_count']
    )

    # Determine if deletion is recommended
    can_delete_legally = dataset['age_days'] >
```

PYTHON

```
value_analysis['legal_retention_days']
```

```
should_delete_economically =
```

```
value_analysis['value_cost_ratio'] < 0.1 # Very low value
```

```
if can_delete_legally and should_delete_economically:
    recommendation = 'DELETE'
elif value_analysis['value_cost_ratio'] < 0.3 and
```

PYTHON

```
dataset['age_days'] > 365:
```

```
recommendation = 'ARCHIVE' # Move to cheaper
```

storage

```
elif value_analysis['value_cost_ratio'] < 0.5 and
```

PYTHON

```
dataset['monthly_access_count'] == 0:
```

```
recommendation = 'COLD_STORAGE' # Move to cold
```

storage

```
else:
    recommendation = 'KEEP'

deletion_candidates.append({
    **dataset,
    **value_analysis,
    'recommendation': recommendation,
    'potential_monthly_savings':
```

```
value_analysis['total_monthly_cost'] if recommendation == 'DELETE' else 0
```

```
})
```

The PYTHON code:

```
return sorted(deletion_candidates, key=lambda x:
```

PYTHON

```
x['potential_monthly_savings'], reverse=True)
```

```
def create_retention_policy(self, data_inventory: List[Dict]) ->
```

PYTHON

Dict:

```

"""Create comprehensive data retention policy"""

# Group data by type and analyze patterns
data_by_type = {}
for dataset in data_inventory:
    data_type = dataset['data_type']
    if data_type not in data_by_type:
        data_by_type[data_type] = []
    data_by_type[data_type].append(dataset)

retention_policy = {}
total_potential_savings = 0

for data_type, datasets in data_by_type.items():
    # Analyze this data type
    total_size = sum(d['size_gb'] for d in datasets)
    total_cost = sum(d['size_gb'] *

```

```

self.storage_cost_per_gb_month for d in datasets)

```

```

# Find optimal retention thresholds
value_analyses = [
    self.calculate_data_value(d['data_type'],

```

PYTHON

```

d['age_days'], d['size_gb'], d['monthly_access_count'])

```

```

for d in datasets
]

# Determine retention tiers
deletion_candidates =

```

```

self.identify_deletion_candidates(datasets)

```

```

savings_by_action = {
    'DELETE': sum(d['potential_monthly_savings'] for d

```

```

in deletion_candidates if d['recommendation'] == 'DELETE'),

```

```
'ARCHIVE': sum(d['total_monthly_cost'] * 0.8 for d
```

in deletion_candidates if d['recommendation'] == 'ARCHIVE'), # 80% savings

```
'COLD_STORAGE': sum(d['total_monthly_cost'] * 0.6
```

for d in deletion_candidates if d['recommendation'] == 'COLD_STORAGE') # 60% savings

```
}
```

PYTHON

```
retention_policy[data_type] = {  
    'total_datasets': len(datasets),  
    'total_size_gb': total_size,  
    'total_monthly_cost': total_cost,  
    'legal_retention_days': self.value_models[data_type]
```

['legal_retention_days'],

```
'recommended_actions': {  
    'delete_after_days':
```

```
self.value_models[data_type]['legal_retention_days'],
```

```
'archive_after_days': max(365,
```

```
self.value_models[data_type]['legal_retention_days'] // 2),
```

```
'cold_storage_after_days': 90  
},  
'potential_monthly_savings':
```

```
sum(savings_by_action.values()),
```

```
'datasets_to_delete': len([d for d in
```

```
deletion_candidates if d['recommendation'] == 'DELETE']),
```

```
'datasets_to_archive': len([d for d in
```

```
deletion_candidates if d['recommendation'] == 'ARCHIVE']),
```

```
'datasets_to_cold_storage': len([d for d in
```

```
deletion_candidates if d['recommendation'] == 'COLD_STORAGE'])
```

```
}
```

```
total_potential_savings += retention_policy[data_type]
```

```
['potential_monthly_savings']
```

```
return {  
    'retention_policy': retention_policy,  
    'total_potential_monthly_savings':
```

```
total_potential_savings,
```

```

'total_datasets_analyzed': len(data_inventory),
    'implementation_priority': sorted(
        retention_policy.items(),
        key=lambda x: x[1]['potential_monthly_savings'],
        reverse=True
    )
}

def generate_retention_rules(self, policy: Dict) -> List[Dict]:
    """Generate specific retention rules for implementation"""

    rules = []

    for data_type, type_policy in

```

```

policy['retention_policy'].items():

```

```

# Deletion rule
rules.append({
    'rule_name': f'delete_{data_type}_after_retention',
    'data_type': data_type,
    'action': 'DELETE',
    'age_threshold_days':

```

```

type_policy['recommended_actions']['delete_after_days'],

```

```

    'estimated_monthly_savings':

```

```

    type_policy['potential_monthly_savings'] * 0.4, # Rough allocation

```

```

    'implementation_complexity': 'Low',
    'risk_level': 'High' if data_type == 'compliance'

```

```

else 'Medium'

```

```

}))

# Archive rule
rules.append({
    'rule_name': f'archive_{data_type}_after_period',
    'data_type': data_type,
    'action': 'ARCHIVE',
    'age_threshold_days':

```

```

type_policy['recommended_actions']['archive_after_days'],

```

```

'estimated_monthly_savings':

```

```

type_policy['potential_monthly_savings'] * 0.4,

```

```

    'implementation_complexity': 'Low',
    'risk_level': 'Low'
}))

```

```

# Cold storage rule
rules.append({
    'rule_name':

```

```

f'cold_storage_{data_type}_after_period',

```

```

    'data_type': data_type,
    'action': 'COLD_STORAGE',
    'age_threshold_days':

```

```

type_policy['recommended_actions']['cold_storage_after_days'],

```

```

'estimated_monthly_savings':

```

```

type_policy['potential_monthly_savings'] * 0.2,

```

```
'implementation_complexity': 'Medium',  
    'risk_level': 'Low'  
}))  
  
return sorted(rules, key=lambda x:
```

```
x['estimated_monthly_savings'], reverse=True)
```

```
# Usage example  
optimizer = DataRetentionOptimizer()  
  
# Sample data inventory  
data_inventory = [  
    {  
        'dataset_name': 'transaction_logs_2022',  
        'data_type': 'transactional',  
        'age_days': 500,
```



```

'size_gb': 1000,
    'monthly_access_count': 2
},
{
    'dataset_name': 'user_behavior_2021',
    'data_type': 'behavioral',
    'age_days': 800,
    'size_gb': 500,
    'monthly_access_count': 0
},
{
    'dataset_name': 'experiment_data_q1_2024',
    'data_type': 'experimental',
    'age_days': 120,
    'size_gb': 200,
    'monthly_access_count': 0
},
{
    'dataset_name': 'audit_logs_2020',
    'data_type': 'compliance',
    'age_days': 1200,
    'size_gb': 300,
    'monthly_access_count': 1
}
]

# Analyze deletion candidates
deletion_analysis =

```

```
optimizer.identify_deletion_candidates(data_inventory)
```

```

print("Deletion Analysis:")
for candidate in deletion_analysis:
    print(f" {candidate['dataset_name']}:

```

PYTHON

```
{candidate['recommendation']] "
```

```

f"(Value/Cost: {candidate['value_cost_ratio']:.2f}, "
f"Potential savings:

```

```

${candidate['potential_monthly_savings']:.2f}/month)")

```

PYTHON

```
# Create retention policy
retention_policy = optimizer.create_retention_policy(data_inventory)

print(f"\nTotal potential monthly savings:
```

```
${retention_policy['total_potential_monthly_savings']:.2f}")
```

PYTHON

```
print("\nRetention policy by data type:")
for data_type, policy in
```

```
retention_policy['retention_policy'].items():
```

PYTHON

```
print(f" {data_type}: Delete after
```

```
{policy['recommended_actions']['delete_after_days']} days, "
```

```
f"Archive after {policy['recommended_actions']
```

```
['archive_after_days']} days")
```

PYTHON

```
# Generate implementation rules
rules = optimizer.generate_retention_rules(retention_policy)

print("\nTop 3 implementation priorities:")
for rule in rules[:3]:
    print(f" {rule['rule_name']}:
```

```
${rule['estimated_monthly_savings']:.2f}/month savings, "
```

```
f"{rule['risk_level']} risk")
```

```
Pro Tip The biggest storage cost optimization isn't technical
```

— it's organizational. Create a "data expiration date" policy where every dataset must have an explicit retention period.

Most teams store data "just in case" without ever quantifying the value. Make data retention a conscious decision, not a default.

□ **Checkpoint**

1. How do you balance storage costs, access costs, and retrieval times when designing lifecycle policies?
2. What factors should you consider when choosing between different compression formats?
3. How do you calculate the business value of data to make retention decisions?

Query Cost Optimization

⚡ TL;DR

Focus on the 80/20 rule: optimize the 20% of queries that generate 80% of costs -
 Use materialized views and result caching strategically — not everything needs to be cached - Workload management and query queues prevent expensive queries from impacting others - Education and governance often save more money than technical optimizations

Query optimization is where the biggest cost savings live. A single poorly written query can cost thousands of dollars and slow down the entire data platform. But optimization isn't just about making queries faster — it's about making them cost-effective.

After optimizing query costs across multiple platforms, in practice, the most effective approach combines technical optimization (better SQL, smarter indexing, result caching) with organizational changes (query governance, cost visibility, education).

Query Cost Analysis and Identification

Identifying Cost-Heavy Queries:

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import hashlib
from typing import Dict, List, Optional

class QueryCostAnalyzer:
    def __init__(self, platform='snowflake'):
        self.platform = platform
        self.cost_models = {
            'snowflake': {'credit_cost': 2.0, 'storage_cost_gb':
```

PYTHON

```
0.023},
```

```
'bigquery': {'tb_scan_cost': 5.0, 'storage_cost_gb':
```

```
0.020},
```

```
'redshift': {'node_hour_cost': 0.25, 'storage_cost_gb':
```

```
0.024}
```

```
}
```

PYTHON

```
def analyze_query_costs(self, query_history_days=30) -> Dict:
    """Analyze query costs and identify optimization
```

```
opportunities"""
```

```
# Get query history from data warehouse
query_history = self.get_query_history(query_history_days)

# Aggregate by query pattern
query_patterns =
```

SQL

```
self.group_queries_by_pattern(query_history)
```

```
# Calculate costs and identify high-impact queries
cost_analysis = []

for pattern, queries in query_patterns.items():
    total_executions = len(queries)
    total_cost = sum(q['cost'] for q in queries)
    avg_execution_time =
```

```
np.mean([q['execution_time_seconds'] for q in queries])
```

```
avg_cost_per_execution = total_cost / total_executions
```

```
if total_executions > 0 else 0
```

```
# Calculate data scanning patterns
total_bytes_scanned = sum(q.get('bytes_scanned', 0) for
```

q in queries)

```
avg_bytes_per_execution = total_bytes_scanned /
```

total_executions if total_executions > 0 else 0

```
# Identify cost drivers
cost_drivers = self.identify_cost_drivers(queries)

cost_analysis.append({
    'query_pattern': pattern,
    'total_executions': total_executions,
    'total_cost': total_cost,
    'avg_cost_per_execution': avg_cost_per_execution,
    'avg_execution_time_seconds': avg_execution_time,
    'total_bytes_scanned': total_bytes_scanned,
    'avg_bytes_per_execution': avg_bytes_per_execution,
    'cost_drivers': cost_drivers,
    'optimization_opportunities':
```

PYTHON

```
self.identify_optimization_opportunities(queries),
```

```
'sample_query': queries[0]['query_text'][:500] +
```

```
'...' if queries else ''
```

```
})

# Sort by total cost impact
cost_analysis.sort(key=lambda x: x['total_cost'],
```

PYTHON

```
reverse=True)
```

```

return {
    'analysis_period_days': query_history_days,
    'total_queries_analyzed': len(query_history),
    'unique_patterns': len(query_patterns),
    'total_cost': sum(q['total_cost'] for q in

```

cost_analysis),

```

'cost_by_pattern': cost_analysis,
'summary_stats':

```

```

self.calculate_summary_stats(cost_analysis)

```

```

}

```

PYTHON

```

def group_queries_by_pattern(self, query_history: List[Dict]) ->

```

Dict:

```

"""Group similar queries together for pattern analysis"""

patterns = {}

for query in query_history:
    # Normalize query to identify patterns
    normalized_query =

```

```

self.normalize_query(query['query_text'])

```

```

pattern_hash =

```

```

hashlib.md5(normalized_query.encode()).hexdigest()[:8]

```

```

if pattern_hash not in patterns:
    patterns[pattern_hash] = []
    patterns[pattern_hash].append(query)

return patterns

def normalize_query(self, query_text: str) -> str:
    """Normalize query text to identify patterns"""

    import re

    # Convert to lowercase
    normalized = query_text.lower()

    # Replace literals with placeholders
    normalized = re.sub(r"'[^']*'", 'STRING_LITERAL',

```

normalized)

```
normalized = re.sub(r'\b\d+\b', 'NUMBER_LITERAL',
```

normalized)

```
normalized = re.sub(r'\b\d{4}-\d{2}-\d{2}\b',
```

'DATE_LITERAL', normalized)

```
# Replace common variations
```

```
normalized = re.sub(r'\s+', ' ', normalized) # Multiple
```

spaces to single

```
normalized = re.sub(r'(where|and|or)\s+\w+\s*(=|>|<|>=|
```

```
<=|!=)\s*\w+', r'\1 CONDITION', normalized)
```



```
return normalized.strip()
```

```
def identify_cost_drivers(self, queries: List[Dict]) ->
```

```
List[Dict]:
```

```
"""Identify what's driving costs in a group of queries"""
```

```
drivers = []
```

```
# Analyze data volume scanned
```

```
avg_bytes_scanned = np.mean([q.get('bytes_scanned', 0) for q
```

```
in queries])
```

```
if avg_bytes_scanned > 1024**4: # > 1 TB
```

```
    drivers.append({
        'type': 'large_data_scan',
        'description': f'Scans
```

```
{avg_bytes_scanned/(1024**4):.1f} TB on average',
```

```
    'impact': 'high',
```

```
    'optimization': 'Add partition pruning or column
```

```
selection'
```

```
    })
```

```
# Analyze execution time
```

```
avg_execution_time = np.mean([q['execution_time_seconds']
```

```
for q in queries])
```

```

if avg_execution_time > 600: # > 10 minutes
    drivers.append({
        'type': 'long_execution_time',
        'description': f'Average execution time:

```

```

{avg_execution_time/60:.1f} minutes',

```

```

'impact': 'high',
'optimization': 'Optimize joins, add indexes, or use

```

SQL

materialized views'

```

}))

# Analyze query complexity
sample_query = queries[0]['query_text'].lower()
if sample_query.count('join') > 5:
    drivers.append({
        'type': 'complex_joins',
        'description': f'Multiple joins detected

```

SQL

```

({sample_query.count("join")} joins)',

```

```

'impact': 'medium',
'optimization': 'Consider denormalization or pre-

```

computed aggregates'

```

}))

# Analyze frequency
if len(queries) > 100: # Run more than 100 times in the

```

period

```
drivers.append({
    'type': 'high_frequency',
    'description': f'Executed {len(queries)} times in
```

analysis period',

```
    'impact': 'high',
    'optimization': 'Cache results or use materialized
```

views'

```
    })

    return drivers

def identify_optimization_opportunities(self, queries:
```

PYTHON

```
List[Dict]) -> List[Dict]:
```

```
    """Identify specific optimization opportunities"""

    opportunities = []

    # Result caching opportunity
```

The PYTHON code:

```
    if len(queries) >= 10: # Frequently executed
        cache_savings = self.estimate_cache_savings(queries)
        opportunities.append({
            'type': 'result_caching',
            'estimated_monthly_savings': cache_savings,
            'implementation_effort': 'Low',
            'description': f'Cache results for {len(queries)}
```

PYTHON

similar executions'

```
})

# Materialized view opportunity
if len(queries) >= 50 and
```

```
self.is_materialization_candidate(queries[0]):
```

```
mv_savings =
```

```
self.estimate_materialization_savings(queries)
```

```
opportunities.append({
    'type': 'materialized_view',
    'estimated_monthly_savings': mv_savings,
    'implementation_effort': 'Medium',
    'description': 'Create materialized view for
```

frequent aggregation'

```
})

# Query optimization opportunity
optimization_potential =
```

```
self.estimate_query_optimization_savings(queries)
```

```
if optimization_potential > 0:
    opportunities.append({
        'type': 'query_optimization',
        'estimated_monthly_savings': optimization_potential,
        'implementation_effort': 'High',
        'description': 'Optimize SQL structure and
```

predicates'

PYTHON

```
})

return sorted(opportunities, key=lambda x:
```

```
x['estimated_monthly_savings'], reverse=True)
```

SQL

```
def estimate_cache_savings(self, queries: List[Dict]) -> float:
    """Estimate savings from result caching"""

    # Assume 80% cache hit rate after first execution
    cache_hit_rate = 0.8
    total_cost = sum(q['cost'] for q in queries)

    # Savings = cost avoided by cache hits
    cache_savings = total_cost * cache_hit_rate * (len(queries)
```

```
- 1) / len(queries)
```

```
# Subtract cache storage costs (rough estimate)
avg_result_size_gb = np.mean([q.get('result_size_bytes',
```

```
1024**2) for q in queries]) / (1024**3)
```

```
cache_storage_cost = avg_result_size_gb *
```

```
self.cost_models[self.platform]['storage_cost_gb']
```

PYTHON

```
return max(0, cache_savings - cache_storage_cost)

def estimate_materialization_savings(self, queries: List[Dict])
```

```
-> float:
```

SQL

```

"""Estimate savings from materialized views"""

total_cost = sum(q['cost'] for q in queries)

# Assume materialized view reduces query cost by 90%
cost_reduction = 0.9
savings_from_mv = total_cost * cost_reduction

# Estimate materialization costs
avg_result_size_gb = np.mean([q.get('result_size_bytes',

```

```

1024**3) for q in queries]) / (1024**3)

```

```

mv_storage_cost = avg_result_size_gb *

```

```

self.cost_models[self.platform]['storage_cost_gb']

```

```

mv_refresh_cost = total_cost * 0.1    # 10% of original cost

```

for refresh

SQL

```

mv_total_cost = mv_storage_cost + mv_refresh_cost

    return max(0, savings_from_mv - mv_total_cost)

def is_materialization_candidate(self, query: Dict) -> bool:
    """Determine if query is a good candidate for

```

materialization"""

SQL

```

query_text = query['query_text'].lower()

# Good candidates: aggregations, complex joins, frequently

```

used tables

```

has_aggregation = any(keyword in query_text for keyword in

```

['group by', 'sum(', 'count(', 'avg(', 'max(', 'min(')]

```
has_joins = 'join' in query_text
is_complex = len(query_text.split()) > 50

return has_aggregation or (has_joins and is_complex)

def estimate_query_optimization_savings(self, queries:
```

SQL

List[Dict]) -> float:

```
"""Estimate savings from SQL optimization"""

total_cost = sum(q['cost'] for q in queries)
avg_execution_time = np.mean([q['execution_time_seconds']
```

SQL

for q in queries])

```
# Estimate optimization potential based on execution time
```

and complexity

```
if avg_execution_time > 300: # > 5 minutes
    optimization_potential = 0.3 # 30% improvement possible
elif avg_execution_time > 60: # > 1 minute
    optimization_potential = 0.2 # 20% improvement possible
else:
    optimization_potential = 0.1 # 10% improvement possible

return total_cost * optimization_potential

def calculate_summary_stats(self, cost_analysis: List[Dict]) ->
```

PYTHON

Dict:

```

"""Calculate summary statistics across all queries"""

total_cost = sum(q['total_cost'] for q in cost_analysis)
total_executions = sum(q['total_executions'] for q in

```

cost_analysis)

```

# Find cost concentration
top_10_patterns = cost_analysis[:10]
top_10_cost = sum(q['total_cost'] for q in top_10_patterns)
cost_concentration = (top_10_cost / total_cost) * 100 if

```

total_cost > 0 else 0

```

# Calculate optimization potential
total_optimization_potential = sum(
    sum(opp['estimated_monthly_savings'] for opp in

```

```

q['optimization_opportunities']))

```

```

for q in cost_analysis
)

return {
    'total_cost': total_cost,
    'total_executions': total_executions,
    'avg_cost_per_execution': total_cost / total_executions

```

if total_executions > 0 else 0,

```

'cost_concentration_top_10': cost_concentration,
'total_optimization_potential':

```

total_optimization_potential,

```

'optimization_potential_percent':

```



```
(total_optimization_potential / total_cost) * 100 if total_cost > 0 else 0
```

```
}
```

SQL

```
def get_query_history(self, days: int) -> List[Dict]:  
    """Get query history from warehouse (mock implementation)"""  
  
    # In reality, this would query the warehouse's query history  
    # For example, Snowflake's QUERY_HISTORY view or BigQuery's
```

JOBS view

```
mock_queries = []  
base_date = datetime.now() - timedelta(days=days)  
  
for day in range(days):  
    date = base_date + timedelta(days=day)  
  
    # Simulate daily query patterns  
    daily_queries = [  
        {  
            'query_id': f'q_{day}_{i}',  
            'query_text': self.generate_mock_query(i % 5),
```

PYTHON

5 different patterns

```
'execution_time_seconds': np.random.normal(180,
```

60),

```
'bytes_scanned': np.random.normal(1024**4,
```

1024**3 * 200), # ~1TB ± 200GB

```
'cost': np.random.normal(25, 10),
'user_name': f'user_{i % 10}',
'timestamp': date,
'result_size_bytes': np.random.normal(1024**2 *
```

100, 1024**2 * 50) # ~100MB ± 50MB

```
}
    for i in range(20) # 20 queries per day
    ]

    mock_queries.extend(daily_queries)

    return mock_queries

def generate_mock_query(self, pattern_id: int) -> str:
    """Generate mock queries for testing"""

    patterns = [
        """
        SELECT customer_id, SUM(order_amount), COUNT(*)
        FROM orders
        WHERE order_date >= '2024-01-01'
        GROUP BY customer_id
        """,
        """
        SELECT p.product_name, SUM(ol.quantity),
```

SQL

```
SUM(ol.total_amount)
```

```
FROM order_lines ol
JOIN products p ON ol.product_id = p.product_id
WHERE ol.created_date >= CURRENT_DATE - INTERVAL '30
```

SQL

days'

SQL

```

GROUP BY p.product_name
ORDER BY SUM(ol.total_amount) DESC
""",
""",
SELECT
    DATE_TRUNC('month', order_date) as month,
    customer_tier,
    AVG(order_amount) as avg_order_value,
    COUNT(DISTINCT customer_id) as unique_customers
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE order_date >= '2023-01-01'
GROUP BY DATE_TRUNC('month', order_date), customer_tier
""",
""",
WITH customer_ltv AS (

```

The corresponding SQL query:

SQL

```

SELECT customer_id, SUM(order_amount) as total_value
FROM orders
WHERE order_date >= CURRENT_DATE - INTERVAL '365

```

days'

```

GROUP BY customer_id
    )
    SELECT c.customer_tier, AVG(ltv.total_value), COUNT(*)
    FROM customer_ltv ltv
    JOIN customers c ON ltv.customer_id = c.customer_id
    GROUP BY c.customer_tier
    """
    """

    SELECT * FROM large_table
    WHERE some_column = 'some_value'
    AND created_date BETWEEN '2024-01-01' AND '2024-12-31'
    """

]

return patterns[pattern_id]

# Usage example
analyzer = QueryCostAnalyzer('snowflake')

# Analyze query costs over the last 30 days
cost_analysis = analyzer.analyze_query_costs(query_history_days=30)

print(f"Query Cost Analysis Summary:")
print(f" Total cost: ${cost_analysis['total_cost']:, .2f}")
print(f" Total queries:

```

```
{cost_analysis['total_queries_analyzed'],})")
```

```

print(f" Unique patterns: {cost_analysis['unique_patterns']}")
print(f" Optimization potential: ${cost_analysis['summary_stats']

```

```
['total_optimization_potential']:, .2f]    ({cost_analysis['summary_stats']    ['optimiza-
tion_potential_percent']:.1f}%))")
```

```

print(f"\nTop 5 Most Expensive Query Patterns:")
for i, pattern in enumerate(cost_analysis['cost_by_pattern'][:5]):
    print(f" {i+1}. Pattern {pattern['query_pattern']}")
    print(f"      Total cost: ${pattern['total_cost']:, .2f}")
    print(f"      Executions: {pattern['total_executions']}")
    print(f"      Avg cost/execution:

```

```
    ${pattern['avg_cost_per_execution']:, .2f}")
```

```

if pattern['optimization_opportunities']:
    best_opp = pattern['optimization_opportunities'][0]
    print(f"        Best optimization: {best_opp['type']}

```

PYTHON

```

({best_opp['estimated_monthly_savings']:,.2f} savings)")

```

```

print()

```

PYTHON

Materialized Views and Result Caching Strategies

Intelligent Caching Strategy:

```

from datetime import datetime, timedelta
import hashlib
from typing import Dict, List, Optional, Tuple

class IntelligentCachingStrategy:
    def __init__(self, platform='snowflake'):
        self.platform = platform
        self.cache_storage = {} # Mock cache storage
        self.cache_stats = {}   # Cache hit/miss statistics

    # Cache configuration
    self.cache_policies = {
        'result_cache': {
            'ttl_hours': 24,           # Time to live
            'max_size_gb': 100,       # Maximum cache size
            'min_query_cost': 1.0,    # Only cache expensive

```

PYTHON

queries

```

'min_frequency': 3           # Must be run at least 3

```

times

```
},  
  'materialized_view': {  
    'refresh_frequency': 'daily',  
    'min_query_frequency': 10, # Must be run at least
```

10 times

```
'min_data_staleness_tolerance': 3600, # 1 hour in
```

seconds

```
'max_storage_cost': 50 # Maximum monthly storage
```

cost

```
}  
  }  
  
  def analyze_caching_opportunities(self, query_history:
```

PYTHON

```
List[Dict]) -> Dict:
```

```
"""Analyze which queries should be cached or materialized"""
```

```
# Group queries by similarity
query_groups = self.group_similar_queries(query_history)
```

```
caching_recommendations = []
```

```
for group_id, queries in query_groups.items():
    if len(queries) < 2: # Skip one-off queries
        continue
```

```
    # Analyze this query group
    total_cost = sum(q['cost'] for q in queries)
    avg_cost = total_cost / len(queries)
    frequency = len(queries)
```

```
    # Calculate time patterns
    timestamps = [q['timestamp'] for q in queries]
    time_spread_hours = (max(timestamps) -
```

```
min(timestamps)).total_seconds() / 3600
```

```
frequency_per_day = frequency / (time_spread_hours / 24)
```

```
if time_spread_hours > 0 else 0
```

```
# Determine best caching strategy
cache_recommendations = self.recommend_caching_strategy(
    queries, total_cost, frequency, frequency_per_day
)
```

```
if cache_recommendations:
    caching_recommendations.append({
        'query_group': group_id,
        'sample_query': queries[0]['query_text'][:200] +
```

```
    '...',
```

```

'frequency': frequency,
    'total_cost': total_cost,
    'avg_cost': avg_cost,
    'frequency_per_day': frequency_per_day,
    'recommendations': cache_recommendations
})

# Sort by potential savings
caching_recommendations.sort(
    key=lambda x: max(r['estimated_savings'] for r in

```

```

x['recommendations'])),

```

```

reverse=True
)

return {
    'total_query_groups': len(query_groups),
    'cacheable_groups': len(caching_recommendations),
    'recommendations': caching_recommendations,
    'total_potential_savings': sum(
        max(r['estimated_savings'] for r in

```

```

rec['recommendations'])

```

```

for rec in caching_recommendations
    )
}

```

The group_similar_queries implementation:

```

def group_similar_queries(self, query_history: List[Dict]) ->

```

Dict:


```
"""Group similar queries for caching analysis"""
```

```
query_groups = {}
```

```
for query in query_history:
    # Create a signature for query similarity
    signature =
```

```
self.create_query_signature(query['query_text'])
```

```
if signature not in query_groups:
    query_groups[signature] = []
    query_groups[signature].append(query)

return query_groups

def create_query_signature(self, query_text: str) -> str:
    """Create a signature for query similarity matching"""

    import re

    # Normalize query
    normalized = query_text.lower().strip()

    # Remove literals and replace with placeholders
    normalized = re.sub(r"'[^']*'", "'?'", normalized)
    normalized = re.sub(r'\b\d+\b', '?', normalized)
    normalized = re.sub(r'\b\d{4}-\d{2}-\d{2}
```

PYTHON

```
(?:\s+\d{2}:\d{2}:\d{2})?\b', '?', normalized)
```

```
# Normalize whitespace
normalized = re.sub(r'\s+', ' ', normalized)

# Create hash
return hashlib.md5(normalized.encode()).hexdigest()[:12]

def recommend_caching_strategy(self, queries: List[Dict],
```

PYTHON

total_cost: float,

```
frequency: int, frequency_per_day:
```

float) -> List[Dict]:

```
"""Recommend specific caching strategies for a query
```

group"""

```
recommendations = []  
  
# Result caching recommendation  
if self.should_use_result_cache(queries, total_cost,
```

PYTHON

frequency):

```
cache_savings =
```

```
self.estimate_result_cache_savings(queries)
```

```
recommendations.append({  
    'strategy': 'result_cache',  
    'estimated_savings': cache_savings,  
    'implementation_effort': 'Low',  
    'cache_duration_hours':
```

```
self.recommend_cache_duration(queries),
```

```
'storage_overhead_gb':
```

```
self.estimate_cache_storage_size(queries),
```

```
'description': f'Cache results for {frequency}
```

similar queries'

```

}))

# Materialized view recommendation
if self.should_use_materialized_view(queries,

```

frequency_per_day):

```
mv_savings =
```

```
self.estimate_materialized_view_savings(queries)
```

```

recommendations.append({
    'strategy': 'materialized_view',
    'estimated_savings': mv_savings,

```

```

    'implementation_effort': 'Medium',
    'refresh_frequency':

```

```
self.recommend_mv_refresh_frequency(queries),
```

```
'storage_overhead_gb':
```

```
self.estimate_mv_storage_size(queries),
```

```
'description': f'Create materialized view for
```

frequent aggregation pattern'

```

}))

# Pre-aggregated table recommendation
if self.should_use_preaggregation(queries,

```

frequency_per_day):

```
preagg_savings =
```

```
self.estimate_preaggregation_savings(queries)
```

```
recommendations.append({
    'strategy': 'pre_aggregated_table',
    'estimated_savings': preagg_savings,
    'implementation_effort': 'High',
    'refresh_frequency': 'hourly',
    'storage_overhead_gb':
```

```
self.estimate_preagg_storage_size(queries),
```

```
'description': f'Create pre-aggregated summary
```

```
tables'
```

```
})
```

PYTHON

```
return sorted(recommendations, key=lambda x:
```

```
x['estimated_savings'], reverse=True)
```

```
def should_use_result_cache(self, queries: List[Dict],
```

PYTHON

total_cost: float, frequency: int) -> bool:

```
"""Determine if result caching is appropriate"""
```

PYTHON

```
min_cost = self.cache_policies['result_cache']
```

```
['min_query_cost']
```

```
min_frequency = self.cache_policies['result_cache']
```

PYTHON

['min_frequency']

```
avg_cost = total_cost / frequency if frequency > 0 else 0

    return avg_cost >= min_cost and frequency >= min_frequency

def should_use_materialized_view(self, queries: List[Dict],
```

PYTHON

frequency_per_day: float) -> bool:

```
"""Determine if materialized view is appropriate"""

min_frequency = self.cache_policies['materialized_view']
```

PYTHON

['min_query_frequency']

```
# Check if query pattern is suitable for materialization
sample_query = queries[0]['query_text'].lower()
has_aggregation = any(keyword in sample_query for keyword in
```

[

```
'group by', 'sum(', 'count(', 'avg(', 'max(', 'min('
])

return frequency_per_day >= min_frequency and
```

SQL

has_aggregation

```
def should_use_preaggregation(self, queries: List[Dict],
```

PYTHON

frequency_per_day: float) -> bool:

```
"""Determine if pre-aggregation is appropriate"""

# Pre-aggregation for very frequent, complex aggregation
```

queries

```

sample_query = queries[0]['query_text'].lower()
is_complex_aggregation = (
    'group by' in sample_query and
    ('join' in sample_query or 'window' in sample_query)
)

return frequency_per_day >= 20 and is_complex_aggregation

```

SQL

The estimate_result_cache_savings implementation:

```

def estimate_result_cache_savings(self, queries: List[Dict]) ->

```

PYTHON

float:

```

"""Estimate savings from result caching"""

total_cost = sum(q['cost'] for q in queries)

# Assume first execution is cache miss, subsequent are hits
cache_hit_rate = (len(queries) - 1) / len(queries) if

```

SQL

```

len(queries) > 1 else 0

```

```

# Savings = cost avoided by cache hits
savings = total_cost * cache_hit_rate

# Subtract cache storage costs
cache_storage_cost =

```

```

self.estimate_cache_storage_cost(queries)

```

```

return max(0, savings - cache_storage_cost)

def estimate_materialized_view_savings(self, queries:

```

PYTHON

```

List[Dict]) -> float:

```

SQL

```

"""Estimate savings from materialized views"""

total_cost = sum(q['cost'] for q in queries)

# Materialized views typically reduce query cost by 80-95%
cost_reduction = 0.9
query_savings = total_cost * cost_reduction

# Calculate materialization costs
mv_storage_cost = self.estimate_mv_storage_cost(queries)
mv_refresh_cost = total_cost * 0.1 # 10% of original cost

```

for refresh

PYTHON

```

mv_total_cost = mv_storage_cost + mv_refresh_cost

return max(0, query_savings - mv_total_cost)

def estimate_preaggregation_savings(self, queries: List[Dict]) -

```

> float:

SQL

```

"""Estimate savings from pre-aggregated tables"""

total_cost = sum(q['cost'] for q in queries)

# Pre-aggregation can reduce costs by 95%+ but has higher

```

maintenance overhead

```

cost_reduction = 0.95
query_savings = total_cost * cost_reduction

# Higher maintenance costs
maintenance_cost = total_cost * 0.2 # 20% of original for

```

maintenance

```

storage_cost = self.estimate_preagg_storage_cost(queries)

    total_overhead = maintenance_cost + storage_cost

    return max(0, query_savings - total_overhead)

def recommend_cache_duration(self, queries: List[Dict]) -> int:
    """Recommend cache duration based on query patterns"""

    # Analyze time intervals between similar queries
    timestamps = sorted([q['timestamp'] for q in queries])
    intervals = []

    for i in range(1, len(timestamps)):
        interval_hours = (timestamps[i] - timestamps[i-

```

```
1])).total_seconds() / 3600
```

```

intervals.append(interval_hours)

if not intervals:
    return 24 # Default 24 hours

```

Set cache duration to cover typical interval:

```

# Set cache duration to cover typical interval
median_interval = np.median(intervals)

if median_interval <= 1:
    return 4    # Very frequent - 4 hour cache
elif median_interval <= 6:
    return 12   # Frequent - 12 hour cache
elif median_interval <= 24:
    return 48   # Daily - 48 hour cache
else:
    return 168  # Weekly or less - 1 week cache

def recommend_mv_refresh_frequency(self, queries: List[Dict]) ->

```

str:


```

"""Recommend materialized view refresh frequency"""

# Analyze query frequency to determine appropriate refresh

```

rate

```

timestamps = sorted([q['timestamp'] for q in queries])
if len(timestamps) < 2:
    return 'daily'

time_span_hours = (timestamps[-1] -

```

```

timestamps[0])).total_seconds() / 3600

```

```

frequency_per_hour = len(queries) / time_span_hours if

```

time_span_hours > 0 else 0

```

if frequency_per_hour >= 2:
    return 'hourly'
elif frequency_per_hour >= 0.5:
    return 'every_4_hours'
else:
    return 'daily'

def estimate_cache_storage_size(self, queries: List[Dict]) ->

```

PYTHON

float:

```

"""Estimate storage size for result cache"""

avg_result_size = np.mean([q.get('result_size_bytes',

```

1024**2) for q in queries])

```
return avg_result_size / (1024**3) # Convert to GB
```

PYTHON

```
def estimate_mv_storage_size(self, queries: List[Dict]) ->
```

float:

```
"""Estimate storage size for materialized view"""
```

```
# Materialized views are typically much smaller than source
```

data

```
avg_scan_size = np.mean([q.get('bytes_scanned', 1024**3) for
```

q in queries])

```
compression_ratio = 0.1 # Aggregations typically compress
```

well

```
return (avg_scan_size * compression_ratio) / (1024**3)
```

PYTHON

```
def estimate_preagg_storage_size(self, queries: List[Dict]) ->
```

float:

```
"""Estimate storage size for pre-aggregated tables"""
```

```
# Pre-aggregated tables are very compact
```

```
avg_scan_size = np.mean([q.get('bytes_scanned', 1024**3) for
```

q in queries])

```
compression_ratio = 0.05 # Very high compression for
```

aggregated data

```

return (avg_scan_size * compression_ratio) / (1024**3)

def estimate_cache_storage_cost(self, queries: List[Dict]) ->

```

PYTHON

float:

```

"""Estimate monthly storage cost for caching"""

```

The PYTHON code:

```

storage_size_gb = self.estimate_cache_storage_size(queries)
monthly_storage_cost = storage_size_gb * 0.023 #

```

PYTHON

\$23/TB/month

```

return monthly_storage_cost

def estimate_mv_storage_cost(self, queries: List[Dict]) ->

```

PYTHON

float:

```

"""Estimate monthly storage cost for materialized view"""

storage_size_gb = self.estimate_mv_storage_size(queries)
monthly_storage_cost = storage_size_gb * 0.023 #

```

PYTHON

\$23/TB/month

```

return monthly_storage_cost

# Usage example
caching_strategy = IntelligentCachingStrategy('snowflake')

# Mock query history for analysis
mock_query_history = [
    {
        'query_id': f'q_{i}',
        'query_text': f"SELECT customer_id, SUM(order_amount) FROM

```

SQL

```
orders WHERE order_date >= '{2024 - (i//10)}-01-01' GROUP BY customer_id",
```

```
'cost': 15.0 + (i % 5),
    'timestamp': datetime.now() - timedelta(days=i//3),
    'result_size_bytes': 1024**2 * 50, # 50 MB
    'bytes_scanned': 1024**4 # 1 TB
}
for i in range(30) # 30 similar queries over time
]

# Analyze caching opportunities
caching_analysis =
```

```
caching_strategy.analyze_caching_opportunities(mock_query_history)
```

```
print(f"Caching Analysis Results:")
print(f" Total query groups analyzed:
```

PYTHON

```
{caching_analysis['total_query_groups']}")
```

```
print(f" Cacheable groups found:
```

PYTHON

```
{caching_analysis['cacheable_groups']}")
```

```
print(f" Total potential savings:
```

PYTHON

```
${caching_analysis['total_potential_savings']:, .2f}/month")
```

```
print(f"\nTop Caching Recommendations:")
for i, rec in enumerate(caching_analysis['recommendations'][:3]):
    print(f" {i+1}. Query Group: {rec['query_group']}")
    print(f"      Frequency: {rec['frequency']} executions")
    print(f"      Total cost: ${rec['total_cost']:, .2f}")

    best_strategy = rec['recommendations'][0]
    print(f"      Best strategy: {best_strategy['strategy']}")
    print(f"      Estimated savings:
```

PYTHON

```
{{best_strategy['estimated_savings']:, .2f}/month")
```

```
print()
```

PYTHON

Workload Management and Query Queues

Priority-Based Query Management:

```
import queue
import threading
import time
from datetime import datetime, timedelta
from enum import Enum
from typing import Dict, List, Optional

class QueryPriority(Enum):
    CRITICAL = 1    # Production dashboards, real-time alerts
    HIGH = 2        # Daily reports, executive dashboards
```

PYTHON

The PYTHON code:

```
MEDIUM = 3        # Ad-hoc analytics, weekly reports
LOW = 4            # Experimental queries, data exploration

class WorkloadManager:
    def __init__(self, platform='snowflake'):
        self.platform = platform
        self.query_queues = {
            QueryPriority.CRITICAL: queue.Queue(maxsize=10),
            QueryPriority.HIGH: queue.Queue(maxsize=50),
            QueryPriority.MEDIUM: queue.Queue(maxsize=100),
            QueryPriority.LOW: queue.Queue(maxsize=200)
        }

        # Resource allocation per priority
        self.resource_allocation = {
            QueryPriority.CRITICAL: {'max_concurrent': 5,
```

PYTHON

```
'warehouse_size': 'Large'},
```

```
QueryPriority.HIGH: {'max_concurrent': 10,
```

```
'warehouse_size': 'Medium'},
```

```
QueryPriority.MEDIUM: {'max_concurrent': 15,
```

```
'warehouse_size': 'Small'},
```

```
QueryPriority.LOW: {'max_concurrent': 5,
```

```
'warehouse_size': 'X-Small'}
```

```
}
```

PYTHON

```
# Cost controls per priority  
self.cost_controls = {  
    QueryPriority.CRITICAL: {'max_cost_per_query': 100,
```

```
'max_hourly_cost': 500},
```

```
QueryPriority.HIGH: {'max_cost_per_query': 50,
```

```
'max_hourly_cost': 200},
```

```
QueryPriority.MEDIUM: {'max_cost_per_query': 25,
```

```
'max_hourly_cost': 100},
```

```
QueryPriority.LOW: {'max_cost_per_query': 10,
```

```
'max_hourly_cost': 50}
```

```

    }

    self.active_queries = {}    # Track running queries
    self.query_stats = {}      # Performance statistics

    def submit_query(self, query_request: Dict) -> Dict:
        """Submit query to appropriate queue based on priority and

```

cost"""

```

    # Determine query priority
    priority = self.determine_priority(query_request)

    # Estimate query cost
    estimated_cost = self.estimate_query_cost(query_request)

    # Apply cost controls
    cost_check = self.check_cost_limits(priority,

```

estimated_cost)

```

    if not cost_check['approved']:
        return {
            'status': 'rejected',
            'reason': cost_check['reason'],
            'estimated_cost': estimated_cost,
            'queue_position': None
        }

    # Add to appropriate queue
    query_item = {
        **query_request,
        'priority': priority,
        'estimated_cost': estimated_cost,
        'submitted_at': datetime.now(),
        'queue_position': self.query_queues[priority].qsize() +

```

1

```
}

try:
```

The PYTHON code:

```
self.query_queues[priority].put(query_item, timeout=1)

# Start query execution if resources available
self.try_start_query_execution()

return {
    'status': 'queued',
    'priority': priority.name,
    'queue_position': query_item['queue_position'],
    'estimated_cost': estimated_cost,
    'estimated_wait_time':
```

PYTHON

```
self.estimate_wait_time(priority)
```

```
}

except queue.Full:
    return {
        'status': 'rejected',
        'reason': f'{priority.name} queue is full',
        'estimated_cost': estimated_cost,
        'queue_position': None
    }

def determine_priority(self, query_request: Dict) ->
```

PYTHON

QueryPriority:

```
"""Determine query priority based on user, query type, and
```

SLA requirements"""


```
user_tier = query_request.get('user_tier', 'standard')
query_type = query_request.get('query_type', 'adhoc')
sla_requirement = query_request.get('sla_minutes', 60)
```

```
# Priority rules
if query_type in ['dashboard', 'alert', 'api'] or
```

sla_requirement <= 5:

```
    return QueryPriority.CRITICAL

    if user_tier == 'executive' or query_type in ['report',
```

'scheduled'] or sla_requirement <= 30:

```
    return QueryPriority.HIGH

    if query_type in ['analytics', 'scheduled_weekly'] or
```

sla_requirement <= 120:

```

return QueryPriority.MEDIUM

    return QueryPriority.LOW

def estimate_query_cost(self, query_request: Dict) -> float:
    """Estimate query cost based on historical patterns"""

    query_text = query_request['query_text']

    # Simple cost estimation based on query characteristics
    base_cost = 1.0

    # Text analysis for complexity
    query_lower = query_text.lower()

    # Size indicators
    if 'select *' in query_lower:
        base_cost *= 3 # Full table scans are expensive

    if query_lower.count('join') > 3:
        base_cost *= 2 # Multiple joins increase cost

    if any(func in query_lower for func in ['sum(', 'count(',

```

'avg(', 'group by']):

```

    base_cost *= 1.5 # Aggregations add cost

    if 'window' in query_lower or 'over(' in query_lower:
        base_cost *= 2 # Window functions are expensive

    # Historical patterns (mock data)

```

```

    user_avg_cost = query_request.get('user_avg_query_cost',

```

5.0)

```

    return min(base_cost * user_avg_cost, 100)    # Cap at $100

    def check_cost_limits(self, priority: QueryPriority,

```

estimated_cost: float) -> Dict:

```
"""Check if query meets cost limits for its priority
```

```
level"""
```

```
limits = self.cost_controls[priority]

# Check per-query limit
if estimated_cost > limits['max_cost_per_query']:
    return {
        'approved': False,
        'reason': f'Estimated cost ${estimated_cost:.2f}
```

PYTHON

```
exceeds {priority.name} limit of ${limits["max_cost_per_query"]}'
```

```
}

# Check hourly spending limit
current_hour_cost = self.get_current_hour_cost(priority)
if current_hour_cost + estimated_cost >
```

PYTHON

```
limits['max_hourly_cost']:
```

```
return {
    'approved': False,
    'reason': f'Would exceed hourly cost limit for
```

```
{priority.name} priority'
```

```
}

    return {'approved': True, 'reason': 'Cost within limits'}

def get_current_hour_cost(self, priority: QueryPriority) ->
```

PYTHON

```
float:
```

```
"""Get current hour's spending for a priority level"""  
  
current_hour = datetime.now().replace(minute=0, second=0,
```

```
microsecond=0)
```

```
hour_cost = 0  
  
    for query_id, query_info in self.active_queries.items():  
        if (query_info['priority'] == priority and  
            query_info['started_at'] >= current_hour):  
            hour_cost += query_info['estimated_cost']  
  
    return hour_cost  
  
def estimate_wait_time(self, priority: QueryPriority) -> int:  
    """Estimate wait time in minutes based on queue and resource
```

PYTHON

```
availability"""
```

```
queue_length = self.query_queues[priority].qsize()  
concurrent_capacity = self.resource_allocation[priority]
```

PYTHON

```
['max_concurrent']
```

```
avg_query_duration_minutes = 5 # Simplified assumption  
  
    if queue_length == 0:  
        return 0  
  
    # Calculate wait time based on queue position and available
```

resources

```
wait_time_minutes = (queue_length / concurrent_capacity) *
```

avg_query_duration_minutes

```

return int(wait_time_minutes)

def try_start_query_execution(self):
    """Try to start queued queries if resources are available"""

    # Process queues in priority order
    for priority in QueryPriority:
        if self.query_queues[priority].empty():

```

The PYTHON code:

```

continue

# Check if we have available resources for this priority
active_count = sum(
    1 for q in self.active_queries.values()
    if q['priority'] == priority
)

max_concurrent = self.resource_allocation[priority]

```

`['max_concurrent']`

```

if active_count < max_concurrent:
    # Start next query in queue
    try:
        query_item =

```

```

self.query_queues[priority].get_nowait()

```

PYTHON

```

self.start_query_execution(query_item)
    except queue.Empty:
        continue

def start_query_execution(self, query_item: Dict):
    """Start executing a query"""

    query_id = f"query_{int(time.time()) * 1000}"

    execution_info = {
        **query_item,
        'query_id': query_id,
        'started_at': datetime.now(),
        'warehouse_size':

```

```

self.resource_allocation[query_item['priority']]['warehouse_size'],

```

PYTHON

```

    'status': 'running'
}

self.active_queries[query_id] = execution_info

# Start execution in separate thread (simplified)
thread = threading.Thread(
    target=self.execute_query_thread,
    args=(query_id, execution_info)
)
thread.daemon = True
thread.start()

print(f"Started query {query_id} with

```

```

{execution_info['priority'].name} priority")

```

PYTHON

```

def execute_query_thread(self, query_id: str, execution_info:

```

Dict):

```

"""Execute query in separate thread (mock implementation)"""

# Simulate query execution time based on priority and

```

complexity

```
base_duration = 30 # 30 seconds base
priority_multiplier = {
    QueryPriority.CRITICAL: 0.5, # Fast execution
    QueryPriority.HIGH: 1.0,
    QueryPriority.MEDIUM: 1.5,
    QueryPriority.LOW: 2.0 # Slower execution
}

execution_duration = base_duration *
```

```
priority_multiplier[execution_info['priority']]
```

```
# Simulate execution
time.sleep(execution_duration)

# Complete query
self.complete_query(query_id, execution_duration)

def complete_query(self, query_id: str, duration_seconds:
```

PYTHON

float):

```
"""Mark query as completed and update statistics"""

if query_id not in self.active_queries:
    return

query_info = self.active_queries[query_id]
completion_time = datetime.now()

# Calculate actual cost (simplified)
actual_cost = query_info['estimated_cost'] *
```

PYTHON

```
(duration_seconds / 60) # Adjust based on duration
```

```
# Update statistics
priority = query_info['priority']
if priority not in self.query_stats:
    self.query_stats[priority] = {
        'total_queries': 0,
        'total_cost': 0,
        'total_duration': 0,
        'avg_wait_time': 0
    }

stats = self.query_stats[priority]
stats['total_queries'] += 1
stats['total_cost'] += actual_cost
stats['total_duration'] += duration_seconds

# Calculate wait time
wait_time = (query_info['started_at'] -
```

```
query_info['submitted_at']).total_seconds()
```

```
stats['avg_wait_time'] = (
    (stats['avg_wait_time'] * (stats['total_queries'] - 1) +
```

wait_time)

```
/ stats['total_queries']
)

# Remove from active queries
del self.active_queries[query_id]

print(f"Completed query {query_id}: ${actual_cost:.2f},
{duration_seconds:.0f}s")
```



```
# Try to start next queued query
self.try_start_query_execution()

def get_workload_statistics(self) -> Dict:
    """Get current workload management statistics"""

    queue_lengths = {
        priority.name: self.query_queues[priority].qsize()
        for priority in QueryPriority
    }

    active_queries_by_priority = {}
    for priority in QueryPriority:
        active_queries_by_priority[priority.name] = sum(
            1 for q in self.active_queries.values()
            if q['priority'] == priority
        )

    cost_by_priority = {}
    for priority in QueryPriority:
        if priority in self.query_stats:
            cost_by_priority[priority.name] =
```

```
self.query_stats[priority]['total_cost']
```

```
else:
    cost_by_priority[priority.name] = 0

return {
    'queue_lengths': queue_lengths,
    'active_queries_by_priority':
```

active_queries_by_priority,

```
'total_active_queries': len(self.active_queries),
'cost_by_priority': cost_by_priority,
'query_statistics': {
    priority.name: stats for priority, stats in
```

```
self.query_stats.items()
```

SQL

```

    }

    }

    # Usage example
    workload_manager = WorkloadManager()

    # Submit various queries with different priorities
    test_queries = [
        {
            'query_text': 'SELECT * FROM dashboard_metrics WHERE date =

```

CURRENT_DATE',

SQL

```

        'user_tier': 'executive',
        'query_type': 'dashboard',
        'sla_minutes': 2,
        'user_avg_query_cost': 3.0
    },
    {
        'query_text': 'SELECT customer_id, SUM(orders) FROM

```

large_table GROUP BY customer_id',

SQL

```

        'user_tier': 'analyst',
        'query_type': 'adhoc',
        'sla_minutes': 30,
        'user_avg_query_cost': 15.0
    },
    {
        'query_text': 'SELECT * FROM experimental_data WHERE

```

complex_condition = true',

```
'user_tier': 'data_scientist',
    'query_type': 'experiment',
    'sla_minutes': 120,
    'user_avg_query_cost': 25.0
}
]

# Submit queries
for i, query in enumerate(test_queries):
    result = workload_manager.submit_query(query)
    print(f"Query {i+1} submission result: {result}")

# Wait a bit for processing
time.sleep(2)

# Get workload statistics
stats = workload_manager.get_workload_statistics()
print("\nWorkload Statistics:")
print(f" Queue lengths: {stats['queue_lengths']}")
print(f" Active queries: {stats['active_queries_by_priority']}")
print(f" Cost by priority: {stats['cost_by_priority']}")
```

□ Checkpoint

1. How do you identify which queries to optimize first for maximum cost impact?
2. When should you use result caching vs materialized views vs pre- aggregated tables?
3. How does workload management help control costs while maintaining performance SLAs?

10.5 Capacity Planning Methodologies

✂ TL;DR

Capacity planning is part art, part science — combine historical trends with business growth projections - Plan for 3 scenarios: baseline growth, aggressive growth, and contraction -

Focus on leading indicators: user growth, data volume growth, query complexity trends
- Build capacity buffers for spiky workloads but don't over-provision for peaks

Capacity planning for data systems is harder than traditional applications because data growth is often exponential, query patterns change as the business evolves, and new use cases can suddenly require 10x more resources.

After building capacity planning models for data platforms at different scales, in practice, the best approach combines quantitative analysis of historical trends with qualitative insights about business direction and upcoming initiatives.

Growth Modeling and Forecasting

Multi-Factor Growth Model:

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error
from datetime import datetime, timedelta
import matplotlib.pyplot as plt

class DataPlatformCapacityPlanner:
    def __init__(self):
        self.models = {}
        self.forecasts = {}

        # Define key capacity metrics to track
        self.capacity_metrics = {
            'storage': {
                'unit': 'TB',
                'growth_factors': ['user_count', 'data_sources',
```

'retention_period']

```
},
'compute': {
    'unit': 'CPU hours',
    'growth_factors': ['active_users', 'query_count',
```

'query_complexity']

```
},
'network': {
    'unit': 'GB transferred',
    'growth_factors': ['external_integrations',
```

'dashboard_usage', 'api_calls']

```
},
'cost': {
    'unit': 'USD',
    'growth_factors': ['storage', 'compute', 'network',
```

'team_size']

```

    }
    }

    def analyze_historical_growth(self, historical_data:

```

pd.DataFrame) -> Dict:

```

        """Analyze historical growth patterns to identify trends"""

        growth_analysis = {}

        for metric in self.capacity_metrics.keys():
            if metric not in historical_data.columns:
                continue

            # Calculate period-over-period growth rates
            data_series = historical_data[metric].values
            growth_rates = []

            for i in range(1, len(data_series)):
                if data_series[i-1] > 0:
                    growth_rate = (data_series[i] - data_series[i-

```

1]) / data_series[i-1]

```

            growth_rates.append(growth_rate)

            if not growth_rates:
                continue

            # Analyze growth patterns
            avg_growth_rate = np.mean(growth_rates)
            growth_volatility = np.std(growth_rates)
            trend_direction = 'increasing' if avg_growth_rate > 0.02

```

else ('stable' if avg_growth_rate > -0.02 else 'decreasing')

```

        # Detect seasonality
        seasonality =

```

```

        self.detect_seasonality(historical_data[metric])

```

```

growth_analysis[metric] = {
    'average_growth_rate_monthly': avg_growth_rate,
    'growth_volatility': growth_volatility,
    'trend_direction': trend_direction,
    'current_value': data_series[-1] if len(data_series)

```

> 0 else 0,

```

'seasonality': seasonality,
'growth_acceleration':

```

```

self.calculate_growth_acceleration(growth_rates)

```

```

}

    return growth_analysis

def detect_seasonality(self, time_series: pd.Series) -> Dict:
    """Detect seasonal patterns in time series data"""

    if len(time_series) < 12: # Need at least a year of data
        return {'has_seasonality': False}

    # Simple seasonality detection using month-over-month

```

PYTHON

patterns

```

monthly_avg = {}
for i, value in enumerate(time_series):
    month = (i % 12) + 1 # Assuming monthly data
    if month not in monthly_avg:
        monthly_avg[month] = []
    monthly_avg[month].append(value)

# Calculate average for each month
monthly_patterns = {}
for month, values in monthly_avg.items():
    monthly_patterns[month] = np.mean(values)

# Check if variation across months is significant
monthly_values = list(monthly_patterns.values())
if len(monthly_values) > 1:
    coefficient_of_variation = np.std(monthly_values) /

```

```

np.mean(monthly_values)

```

```

has_seasonality = coefficient_of_variation > 0.15 # 15%

```

threshold

```

else:
    has_seasonality = False

return {
    'has_seasonality': has_seasonality,
    'monthly_patterns': monthly_patterns,
    'peak_month': max(monthly_patterns,

```

```

key=monthly_patterns.get) if has_seasonality else None,

```

```

'low_month': min(monthly_patterns,

```

```

key=monthly_patterns.get) if has_seasonality else None

```



```
}
```

```
def calculate_growth_acceleration(self, growth_rates:
```

```
List[float]) -> str:
```

```
    """Calculate whether growth is accelerating, decelerating,
```

```
    or stable"""
```

```
    if len(growth_rates) < 4:
        return 'insufficient_data'
```

```
    # Compare recent growth (last 3 months) with earlier growth
    recent_growth = np.mean(growth_rates[-3:])
    earlier_growth = np.mean(growth_rates[-6:-3]) if
```

```
len(growth_rates) >= 6 else np.mean(growth_rates[:-3])
```

```
    if recent_growth > earlier_growth * 1.1:
        return 'accelerating'
    elif recent_growth < earlier_growth * 0.9:
        return 'decelerating'
    else:
        return 'stable'
```

```
def create_growth_scenarios(self, growth_analysis: Dict,
```

```
business_context: Dict) -> Dict:
```

```
    """Create multiple growth scenarios for capacity planning"""
```

```
    scenarios = {
        'conservative': {'description': 'Slow but steady
```

```
growth', 'probability': 0.3},
```

```
'baseline': {'description': 'Expected growth
```

```
trajectory', 'probability': 0.5},
```

```
'aggressive': {'description': 'High growth scenario',
```

```
'probability': 0.15},
```

```
'explosive': {'description': 'Exceptional growth (new
```

```
product launch)', 'probability': 0.05}
```

```
}
```

```
for scenario_name, scenario_info in scenarios.items():  
    scenario_forecasts = {}
```

```
    for metric, analysis in growth_analysis.items():  
        base_growth_rate =
```

```
analysis['average_growth_rate_monthly']
```

```

current_value = analysis['current_value']

# Adjust growth rate based on scenario
if scenario_name == 'conservative':
    growth_multiplier = 0.5
elif scenario_name == 'baseline':
    growth_multiplier = 1.0
elif scenario_name == 'aggressive':
    growth_multiplier = 1.8
else: # explosive
    growth_multiplier = 3.0

# Apply business context adjustments
if business_context.get('new_product_launch'):
    growth_multiplier *= 1.5
if business_context.get('market_expansion'):
    growth_multiplier *= 1.3
if business_context.get('competitor_threat'):
    growth_multiplier *= 0.8

adjusted_growth_rate = base_growth_rate *

```

growth_multiplier

```

# Project 24 months forward
projections = []
value = current_value

for month in range(24):
    # Apply seasonality if detected
    seasonal_factor = 1.0
    if analysis['seasonality']['has_seasonality']:
        month_of_year = (month % 12) + 1
        seasonal_factor = analysis['seasonality']

```

```
['monthly_patterns'].get(month_of_year, 1.0)
```

```
seasonal_factor = seasonal_factor /
```

```
np.mean(list(analysis['seasonality']['monthly_patterns'].values()))
```

```
# Apply growth with seasonality
monthly_growth = adjusted_growth_rate *
```

seasonal_factor

```
value = value * (1 + monthly_growth)
projections.append(value)

scenario_forecasts[metric] = {
    'projections': projections,
    'final_value': projections[-1],
    'total_growth': (projections[-1] / current_value
```

- 1) if current_value > 0 else 0,

```
'adjusted_growth_rate': adjusted_growth_rate
}

scenarios[scenario_name]['forecasts'] =
```

scenario_forecasts

```
return scenarios

def calculate_infrastructure_requirements(self, scenarios: Dict)
```

PYTHON

-> Dict:

```
"""Calculate infrastructure requirements for each
```

scenario"""

```
# Infrastructure scaling factors (simplified)
scaling_factors = {
    'storage': {
        'replication_factor': 2.0, # Data replication
        'compression_ratio': 0.3,  # Compression efficiency
        'index_overhead': 0.2      # Additional space for
```

indexes

```
    },
    'compute': {
        'peak_usage_factor': 2.5,      # Peak vs average usage
        'utilization_target': 0.7,    # Target utilization
        'overhead_factor': 0.3        # OS and system overhead
    },
    'network': {
        'redundancy_factor': 1.5,     # Network redundancy
        'burst_factor': 3.0           # Peak burst capacity
    }
}

infrastructure_plans = {}

for scenario_name, scenario in scenarios.items():
    if 'forecasts' not in scenario:
        continue

    infra_requirements = {}

    # Storage requirements
    if 'storage' in scenario['forecasts']:
        raw_storage_tb = scenario['forecasts']['storage']
```

['final_value']

```
# Apply scaling factors
compressed_storage = raw_storage_tb *
```

```
scaling_factors['storage']['compression_ratio']
```

```
replicated_storage = compressed_storage *
```

```
scaling_factors['storage']['replication_factor']
```

```
total_storage = replicated_storage * (1 +
```

```
scaling_factors['storage']['index_overhead'])
```

```
infra_requirements['storage'] = {  
    'raw_data_tb': raw_storage_tb,  
    'total_required_tb': total_storage,  
    'recommended_capacity_tb': total_storage * 1.2,
```

20% buffer

```
'estimated_monthly_cost': total_storage * 23,    #
```

\$23/TB/month

```
}  
  
# Compute requirements  
if 'compute' in scenario['forecasts']:  
    monthly_cpu_hours = scenario['forecasts']['compute']
```

['final_value']

```
# Calculate required compute capacity  
avg_hourly_usage = monthly_cpu_hours / (30 * 24)    #
```

Average per hour

```
peak_hourly_usage = avg_hourly_usage *
```

```
scaling_factors['compute']['peak_usage_factor']
```

```
required_capacity = peak_hourly_usage /
```

```
scaling_factors['compute']['utilization_target']
```

```
infra_requirements['compute'] = {
    'avg_cpu_hours_monthly': monthly_cpu_hours,
    'peak_cpu_hours_hourly': peak_hourly_usage,
    'required_cpu_capacity': required_capacity,
    'recommended_instance_count':
```

```
int(required_capacity / 8) + 1, # 8 CPU per instance
```

```
'estimated_monthly_cost': required_capacity *
```

```
0.10 * 24 * 30 # $0.10/CPU/hour
```

```
}

# Cost projections
total_cost = 0
if 'storage' in infra_requirements:
    total_cost += infra_requirements['storage']
```

```
['estimated_monthly_cost']
```

```
if 'compute' in infra_requirements:
    total_cost += infra_requirements['compute']
```

```
['estimated_monthly_cost']
```

```
infra_requirements['total_monthly_cost'] = total_cost
    infrastructure_plans[scenario_name] = infra_requirements

    return infrastructure_plans

def generate_capacity_recommendations(self,
```

PYTHON

```
infrastructure_plans: Dict,
```

```
current_capacity: Dict) ->
```

```
List[Dict]:
```

```
"""Generate specific capacity planning recommendations"""
```

```
recommendations = []
```

```
# Analyze capacity gaps
```

```
baseline_requirements = infrastructure_plans.get('baseline',
```

```
{})
```

```
aggressive_requirements =
```

```
infrastructure_plans.get('aggressive', {})
```

```
for resource_type in ['storage', 'compute']:
    if resource_type not in baseline_requirements:
        continue
```

```
current_cap = current_capacity.get(resource_type, {})
baseline_req = baseline_requirements[resource_type]
aggressive_req =
```

```
aggressive_requirements.get(resource_type, baseline_req)
```

```
# Storage recommendations
if resource_type == 'storage':
    current_tb = current_cap.get('total_tb', 0)
    baseline_needed =
```

```
baseline_req.get('recommended_capacity_tb', 0)
```

```
aggressive_needed =
```

```
aggressive_req.get('recommended_capacity_tb', 0)
```



```
if baseline_needed > current_tb:  
    gap_tb = baseline_needed - current_tb
```

```
recommendations.append({  
    'type': 'storage_expansion',  
    'priority': 'high' if gap_tb > current_tb *
```

0.5**else 'medium',**

```
'description': f'Add {gap_tb:.0f} TB storage
```

```
capacity',
```

```
'timeline': 'next_6_months' if gap_tb >
```

```
current_tb else 'next_12_months',
```

```
'estimated_cost': gap_tb * 23 * 12, #
```

Annual cost

```
'business_impact': 'Prevent storage capacity
```

```
constraints'
```

```
})
```

```
# Compute recommendations  
elif resource_type == 'compute':  
    current_instances =
```

PYTHON

```
current_cap.get('instance_count', 0)
```

```
baseline_needed =
```

```
baseline_req.get('recommended_instance_count', 0)
```

```
aggressive_needed =
```

```
aggressive_req.get('recommended_instance_count', 0)
```

```
if baseline_needed > current_instances:  
    gap_instances = baseline_needed -
```

current_instances

```
recommendations.append({  
    'type': 'compute_scaling',  
    'priority': 'high' if gap_instances >
```

```
current_instances * 0.3 else 'medium',
```

```
'description': f'Add {gap_instances} compute
```

instances',

```
'timeline': 'next_3_months' if gap_instances
```

```
> current_instances * 0.5 else 'next_6_months',
```

```
'estimated_cost': gap_instances * 8 * 0.10 *
```

24 * 30 * 12, # Annual cost

```
'business_impact': 'Maintain query
```

performance SLAs'

```
})
```

```
# Cost optimization recommendations  
if infrastructure_plans.get('baseline'):  
    projected_cost = infrastructure_plans['baseline']
```

['total_monthly_cost']

```
current_cost = current_capacity.get('monthly_cost', 0)

if projected_cost > current_cost * 1.5:    # >50% cost
```

increase

```
recommendations.append({
    'type': 'cost_optimization',
    'priority': 'high',
    'description': f'Implement cost optimization
```

```
before scaling (projected: ${projected_cost:,.0f}/month)',
```

```
'timeline': 'immediate',
'estimated_cost': -(projected_cost * 0.2), #
```

20% savings

```
'business_impact': 'Reduce infrastructure cost
```

growth'

```
})
```

PYTHON

```
# Sort by priority and impact
priority_order = {'high': 1, 'medium': 2, 'low': 3}
recommendations.sort(key=lambda x:
```

```
(priority_order[x['priority']], -abs(x['estimated_cost'])))
```

```

return recommendations

# Usage example
planner = DataPlatformCapacityPlanner()

# Mock historical data (24 months)
dates = pd.date_range(start='2022-01-01', periods=24, freq='M')

```

```

historical_data = pd.DataFrame({
    'date': dates,
    'storage': np.cumsum(np.random.normal(50, 10, 24)) + 500, #

```

Growing storage

```

'compute': np.cumsum(np.random.normal(100, 20, 24)) + 1000, #

```

Growing compute

```

'cost': np.cumsum(np.random.normal(500, 100, 24)) + 5000, #

```

Growing cost

```

}))

# Analyze historical growth
growth_analysis = planner.analyze_historical_growth(historical_data)

print("Historical Growth Analysis:")
for metric, analysis in growth_analysis.items():
    print(f" {metric}:")
    print(f"     Current value: {analysis['current_value']:.0f}")
    print(f"     Monthly growth rate:

{analysis['average_growth_rate_monthly']*100:.1f}%")

```

PYTHON

```
print(f"    Trend: {analysis['trend_direction']}")
print(f"    Growth acceleration:
```

PYTHON

```
{analysis['growth_acceleration']}")
```

```
# Create growth scenarios
business_context = {
    'new_product_launch': True,
    'market_expansion': False,
    'competitor_threat': False
}

scenarios = planner.create_growth_scenarios(growth_analysis,
```

```
business_context)
```

```
# Calculate infrastructure requirements
infrastructure_plans =
```

```
planner.calculate_infrastructure_requirements(scenarios)
```

```
print(f"\nInfrastructure Requirements by Scenario:")
for scenario_name, requirements in infrastructure_plans.items():
    print(f" {scenario_name.upper()}:")
    if 'storage' in requirements:
        print(f"    Storage: {requirements['storage']}
```

PYTHON

```
['recommended_capacity_tb']:.0f} TB")
```

```
if 'compute' in requirements:
    print(f"    Compute: {requirements['compute']}
```

PYTHON

```
['recommended_instance_count']} instances")
```

```
print(f"    Monthly cost:
```

PYTHON

```
${requirements['total_monthly_cost']:.0f}")
```

```
# Generate recommendations
current_capacity = {
    'storage': {'total_tb': 800},
    'compute': {'instance_count': 20},
    'monthly_cost': 8000
}

recommendations =
```

```
planner.generate_capacity_recommendations(infrastructure_plans,
current_capacity)
```

```
print(f"\nCapacity Planning Recommendations:")
for i, rec in enumerate(recommendations[:3]):
    print(f" {i+1}. {rec['description']}")
    print(f"      Priority: {rec['priority']}")
    print(f"      Timeline: {rec['timeline']}")
    print(f"      Estimated cost impact:
```

PYTHON

```
${rec['estimated_cost']:,.0f}")
```

```
print(f"      Business impact: {rec['business_impact']}")
print()
```

PYTHON

Resource Utilization Analysis

Bottleneck Identification Framework:

```
import pandas as pd
import numpy as np
from typing import Dict, List, Tuple
from datetime import datetime, timedelta

class ResourceUtilizationAnalyzer:
    def __init__(self):
        self.utilization_thresholds = {
            'cpu': {'warning': 70, 'critical': 85, 'optimal_range':
```

PYTHON

```
(40, 70)},
```

```
'memory': {'warning': 75, 'critical': 90,
```

```
'optimal_range': (50, 75)},
```

```
'storage': {'warning': 80, 'critical': 95,
```

```
'optimal_range': (60, 80)},
```

```
'network': {'warning': 60, 'critical': 80,
```

```
'optimal_range': (20, 60)},
```

```
'query_queue': {'warning': 10, 'critical': 25,
```

```
'optimal_range': (0, 5)}
```

```
}
```

PYTHON

```
def analyze_resource_utilization(self, metrics_data:
```

pd.DataFrame) -> Dict:

```
"""Analyze resource utilization patterns and identify
```

```
bottlenecks"""
```


PYTHON

```

utilization_analysis = {}

for resource in self.utilization_thresholds.keys():
    if resource not in metrics_data.columns:
        continue

    resource_data = metrics_data[resource]
    thresholds = self.utilization_thresholds[resource]

    # Basic utilization statistics
    avg_utilization = resource_data.mean()
    max_utilization = resource_data.max()
    p95_utilization = resource_data.quantile(0.95)

    # Time above thresholds
    time_above_warning = (resource_data >

```

```

thresholds['warning']).sum() / len(resource_data) * 100

```

```

time_above_critical = (resource_data >

```

```

thresholds['critical']).sum() / len(resource_data) * 100

```

PYTHON

```

# Trend analysis
trend = self.calculate_utilization_trend(resource_data)

# Peak usage patterns
peak_patterns =

```

```

self.identify_peak_patterns(resource_data, metrics_data.index)

```

PYTHON

```

# Bottleneck assessment
bottleneck_risk = self.assess_bottleneck_risk(
    avg_utilization, max_utilization, p95_utilization,

```

thresholds

```
)

utilization_analysis[resource] = {
    'average_utilization': avg_utilization,
    'max_utilization': max_utilization,
    'p95_utilization': p95_utilization,
    'time_above_warning_percent': time_above_warning,
    'time_above_critical_percent': time_above_critical,
    'trend': trend,
    'peak_patterns': peak_patterns,
    'bottleneck_risk': bottleneck_risk,
    'recommendations':
```

```
self.generate_utilization_recommendations(
```

```
resource, avg_utilization, bottleneck_risk,
```

trend

```
)
    }

    return {
        'analysis_timestamp': datetime.now(),
        'resource_analysis': utilization_analysis,
        'overall_bottlenecks':
```

```
self.identify_overall_bottlenecks(utilization_analysis),
```

```
'capacity_recommendations':
```

```
self.generate_capacity_recommendations(utilization_analysis)
```

```
}
```

PYTHON

```
def calculate_utilization_trend(self, utilization_data:
```

pd.Series) -> Dict:

```

"""Calculate utilization trend over time"""

if len(utilization_data) < 7:    # Need at least a week of

```

data

```

return {'trend': 'insufficient_data'}

# Simple linear trend
x = np.arange(len(utilization_data))
slope = np.polyfit(x, utilization_data.values, 1)[0]

# Classify trend
if abs(slope) < 0.1:
    trend_direction = 'stable'
elif slope > 0.5:
    trend_direction = 'increasing_fast'
elif slope > 0.1:
    trend_direction = 'increasing'
elif slope < -0.5:
    trend_direction = 'decreasing_fast'
else:
    trend_direction = 'decreasing'

# Calculate trend strength
correlation = np.corrcoef(x, utilization_data.values)[0, 1]

```

PYTHON

**** 2**

```

return {
    'trend': trend_direction,
    'slope': slope,
    'strength': correlation,
    'is_significant': correlation > 0.3
}

def identify_peak_patterns(self, utilization_data: pd.Series,

```

PYTHON

timestamps: pd.Index) -> Dict:

```

"""Identify patterns in peak utilization"""

# Convert timestamps to datetime if needed
if hasattr(timestamps, 'to_pydatetime'):
    timestamps = timestamps.to_pydatetime()

# Identify peaks (top 10% of values)
peak_threshold = utilization_data.quantile(0.9)
peak_mask = utilization_data > peak_threshold

if not peak_mask.any():
    return {'has_patterns': False}

peak_timestamps = [timestamps[i] for i in

```

```

range(len(timestamps)) if peak_mask.iloc[i]]

```

```

# Analyze peak timing patterns
hourly_peaks = {}
daily_peaks = {}

for timestamp in peak_timestamps:
    hour = timestamp.hour

```

The PYTHON code:

```

day = timestamp.weekday()    # 0 = Monday

    hourly_peaks[hour] = hourly_peaks.get(hour, 0) + 1
    daily_peaks[day] = daily_peaks.get(day, 0) + 1

# Find most common peak times
peak_hours = sorted(hourly_peaks.items(), key=lambda x:

```

PYTHON

```

x[1], reverse=True)[:3]

```

```

peak_days = sorted(daily_peaks.items(), key=lambda x: x[1],

```

PYTHON

```

reverse=True)[:3]

```

```
# Convert day numbers to names
day_names = ['Monday', 'Tuesday', 'Wednesday', 'Thursday',
```

```
'Friday', 'Saturday', 'Sunday']
```

```
peak_days_named = [(day_names[day], count) for day, count in
```

```
peak_days]
```

```
return {
    'has_patterns': True,
    'total_peaks': len(peak_timestamps),
    'peak_threshold': peak_threshold,
    'common_peak_hours': peak_hours,
    'common_peak_days': peak_days_named,
    'peak_frequency': len(peak_timestamps) /
```

```
len(utilization_data) * 100
```

```
}
```

PYTHON

```
def assess_bottleneck_risk(self, avg_util: float, max_util:
```

```
float,
```

```
p95_util: float, thresholds: Dict) ->
```

Dict:

```

"""Assess bottleneck risk for a resource"""

risk_score = 0
risk_factors = []

# Average utilization risk
if avg_util > thresholds['optimal_range'][1]:
    risk_score += 3
    risk_factors.append('high_average_utilization')
elif avg_util < thresholds['optimal_range'][0]:
    risk_score += 1
    risk_factors.append('low_utilization_inefficiency')

# Peak utilization risk
if max_util > thresholds['critical']:
    risk_score += 5
    risk_factors.append('critical_peaks')
elif max_util > thresholds['warning']:
    risk_score += 2
    risk_factors.append('warning_level_peaks')

# P95 utilization risk
if p95_util > thresholds['warning']:
    risk_score += 3
    risk_factors.append('frequent_high_utilization')

# Determine overall risk level
if risk_score >= 8:
    risk_level = 'critical'
elif risk_score >= 5:
    risk_level = 'high'
elif risk_score >= 2:
    risk_level = 'medium'
else:
    risk_level = 'low'

return {
    'risk_level': risk_level,
    'risk_score': risk_score,
    'risk_factors': risk_factors,
    'capacity_margin': max(0, thresholds['critical'] -

```

max_util)

```
}
```

```
def generate_utilization_recommendations(self, resource: str,
```

```
avg_util: float,
```

```
bottleneck_risk: Dict,
```

```
trend: Dict) -> List[Dict]:
```

```
    """Generate recommendations for resource utilization
```

```
    optimization"""
```

```
    recommendations = []
    risk_level = bottleneck_risk['risk_level']
    trend_direction = trend.get('trend', 'stable')

    # High utilization recommendations
    if risk_level in ['critical', 'high']:
        if resource == 'cpu':
            recommendations.append({
                'action': 'scale_out_compute',
                'description': 'Add more compute instances to
```

```
    distribute load',
```

```
        'priority': 'high' if risk_level == 'critical'
```

```
    else 'medium',
```

```
        'estimated_impact': 'Reduce CPU utilization by
```

```
    30-50%',
```

```
'implementation_effort': 'medium'
    })
elif resource == 'memory':
    recommendations.append({
        'action': 'increase_memory',
        'description': 'Upgrade to memory-optimized
```

instances',

```
'priority': 'high',
'estimated_impact': 'Prevent memory pressure and
```

improve performance',

```
'implementation_effort': 'low'
    })
elif resource == 'storage':
    recommendations.append({
        'action': 'expand_storage',
        'description': 'Add storage capacity or
```

implement data archival',

```
'priority': 'critical' if avg_util > 90 else
```

'high',

```
'estimated_impact': 'Prevent storage
```

exhaustion',

```
'implementation_effort': 'medium'
    })

# Low utilization recommendations (cost optimization)
if risk_level == 'low' and avg_util <
```

self.utilization_thresholds[resource]['optimal_range'][0]:


```
recommendations.append({
    'action': 'rightsize_resources',
    'description': f'Reduce {resource} allocation to
```

optimize costs',

```
'priority': 'medium',
'estimated_impact': f'Reduce {resource} costs by 20-
```

40%',

```
'implementation_effort': 'low'
    })

# Trend-based recommendations
if trend.get('is_significant') and trend_direction in
```

['increasing', 'increasing_fast']:

```
recommendations.append({
    'action': 'proactive_scaling',
    'description': f'Plan capacity increase due to
```

growing {resource} trend',

```
'priority': 'medium',
'estimated_impact': 'Prevent future bottlenecks',
'implementation_effort': 'medium'
```

The PYTHON code:

```
    })

    return recommendations

def identify_overall_bottlenecks(self, utilization_analysis:
```

PYTHON

Dict) -> List[Dict]:

```

"""Identify system-wide bottlenecks across all resources"""

bottlenecks = []

for resource, analysis in utilization_analysis.items():
    risk_level = analysis['bottleneck_risk']['risk_level']

    if risk_level in ['critical', 'high']:
        bottlenecks.append({
            'resource': resource,
            'risk_level': risk_level,
            'avg_utilization':

```

```

analysis['average_utilization'],

```

```

'p95_utilization': analysis['p95_utilization'],
'primary_issues': analysis['bottleneck_risk']

```

```

['risk_factors'],

```

```

'trend': analysis['trend']['trend']
    })

# Sort by risk level and utilization
risk_order = {'critical': 0, 'high': 1, 'medium': 2, 'low':

```

```

3}

```

```

bottlenecks.sort(key=lambda x: (risk_order[x['risk_level']],

```

PYTHON

```

-x['p95_utilization']))

```

```

return bottlenecks

def generate_capacity_recommendations(self,

```

PYTHON

```

utilization_analysis: Dict) -> List[Dict]:

```

```

"""Generate overall capacity planning recommendations"""

recommendations = []
high_risk_resources = []

for resource, analysis in utilization_analysis.items():
    if analysis['bottleneck_risk']['risk_level'] in

```

['critical', 'high']:

```

high_risk_resources.append(resource)

if high_risk_resources:
    recommendations.append({
        'type': 'immediate_scaling',
        'description': f'Scale resources: {"",

```

```

".join(high_risk_resources)}',

```

```

'timeline': 'within_1_week',
    'impact': 'Prevent performance degradation',
    'affected_resources': high_risk_resources
    })

# Check for overall system stress
avg_utilizations = [analysis['average_utilization'] for

```

```

analysis in utilization_analysis.values()]

```

```

if len(avg_utilizations) > 0 and np.mean(avg_utilizations) >

```

75:

```

recommendations.append({
    'type': 'system_wide_scaling',
    'description': 'Overall system utilization is high

```

across multiple resources',

```
'timeline': 'within_2_weeks',  
'impact': 'Improve overall system performance and
```

reliability',

```
'affected_resources':
```

```
list(utilization_analysis.keys())
```

```
})
```

```
return recommendations
```

```
# Usage example  
analyzer = ResourceUtilizationAnalyzer()  
  
# Mock utilization data (24 hours of hourly data)  
hours = pd.date_range(start='2024-03-01', periods=24, freq='H')  
utilization_data = pd.DataFrame({  
    'timestamp': hours,  
    'cpu': np.random.normal(65, 15, 24).clip(0, 100), # Average 65%
```

CPU

```
'memory': np.random.normal(70, 10, 24).clip(0, 100),    # Average
```

70% memory

```
'storage': np.random.normal(75, 5, 24).clip(0, 100),    # Average
```

75% storage

```
'network': np.random.normal(45, 20, 24).clip(0, 100),    # Average
```

45% network

```
'query_queue': np.random.normal(8, 12, 24).clip(0, 50)    #
```

Average 8 queued queries

```
})

utilization_data.set_index('timestamp', inplace=True)

# Analyze utilization
utilization_report =
```

```
analyzer.analyze_resource_utilization(utilization_data)
```

```
print("Resource Utilization Analysis:")
print("="*50)

# Show bottlenecks
if utilization_report['overall_bottlenecks']:
    print("Current Bottlenecks:")
    for bottleneck in utilization_report['overall_bottlenecks']:
        print(f" - {bottleneck['resource'].upper()}:
```

PYTHON

```
{bottleneck['risk_level']} risk")
```

```
print(f"    Average: {bottleneck['avg_utilization']:.1f}%,
```

PYTHON

```
P95: {bottleneck['p95_utilization']:.1f}%")
```

PYTHON

```

print(f"    Trend: {bottleneck['trend']}")
    print()
else:
    print("No critical bottlenecks identified.")

# Show capacity recommendations
if utilization_report['capacity_recommendations']:
    print("Capacity Recommendations:")
    for rec in utilization_report['capacity_recommendations']:
        print(f" - {rec['description']}")
        print(f"    Timeline: {rec['timeline']}")
        print(f"    Impact: {rec['impact']}")
        print()

# Show detailed analysis for top resource
if utilization_report['resource_analysis']:
    top_resource = max(
        utilization_report['resource_analysis'].items(),
        key=lambda x: x[1]['bottleneck_risk']['risk_score']
    )

    resource_name, analysis = top_resource
    print(f"Detailed Analysis - {resource_name.upper()}:")
    print(f" Average utilization:

```

```

{analysis['average_utilization']:.1f}%")

```

PYTHON

```

print(f" Peak utilization: {analysis['max_utilization']:.1f}%")
print(f" Time above warning:

```

```

{analysis['time_above_warning_percent']:.1f}%")

```

PYTHON

```

print(f" Bottleneck risk: {analysis['bottleneck_risk']

```

```

['risk_level']}")

```

PYTHON

```

if analysis['recommendations']:
    print(" Recommendations:")
    for rec in analysis['recommendations']:
        print(f"    - {rec['description']} (Priority:

```

```
{rec['priority']})")
```

Pro Tip The best capacity planning combines quantitative analysis with qualitative insights. Talk to product managers about upcoming features, sales teams about growth projections, and engineering teams about architecture changes. A single new use case can change your capacity needs more than six months of historical trends.

□ Checkpoint

1. How do you combine historical growth patterns with business context to create realistic capacity scenarios?
2. What are the key metrics for identifying resource bottlenecks before they impact performance?
3. When should you optimize utilization vs. add more capacity?

Try It Yourself Exercises

Exercise 1: Multi-Cloud Cost Analysis (60 minutes) You're evaluating whether to move some workloads from Snowflake to BigQuery to reduce costs. Your current Snowflake usage:

- 100 TB of data storage
- 500 credit-hours per month of compute
- 50 TB of data scanned per month across all queries

Task: Compare total costs between: 1. Staying on Snowflake 2. Moving to BigQuery on-demand pricing 3. Moving to BigQuery with reserved slots

Deliverable: Cost comparison spreadsheet with break-even analysis and recommendations.

Exercise 2: Query Optimization ROI Analysis (45 minutes) You've identified your top 10 most expensive queries. Design an optimization strategy:

- Query #1: Runs 200x/month, costs \$25/execution, scans 2 TB each time
- Query #2: Runs 50x/month, costs \$75/execution, complex joins

- Query #3: Runs 500x/month, costs \$5/execution, simple aggregation

Task: Prioritize optimization efforts and estimate ROI for: 1. Result caching 2. Materialized views 3. Query restructuring

Deliverable: Optimization plan with cost-benefit analysis.

Exercise 3: Capacity Planning Scenario (75 minutes)

Your data platform currently handles:

- 1 TB new data daily
- 10,000 queries daily
- \$15,000/month total cost

Business projections show 3x user growth and 2x data growth over next 18 months.

Task: Create capacity plan addressing: 1. Infrastructure scaling timeline 2. Cost projections and budget planning 3. Risk mitigation for aggressive growth scenarios

Deliverable: 18-month capacity plan with monthly milestones.

Module 10 Final Project: Data Platform

Cost Optimization Strategy

Objective

Design a comprehensive cost optimization and capacity planning strategy for a fast-growing SaaS company whose data platform costs have grown from \$20k/month to \$200k/month in 18 months.

Problem Statement

Company Context: TechFlow is a B2B SaaS platform for project management that has grown from 1,000 to 50,000 customers in 18 months. Their data platform supports:

- Customer analytics dashboards (real-time)
- AI-powered project recommendations -

Usage-based billing calculations - Compliance reporting - Internal business intelligence

Current State:

- Platform: Multi-cloud (Snowflake on AWS + BigQuery on GCP + Redshift on AWS)
- Data Volume: 10 TB new data daily, 500 TB total
- Query Volume: 100,000 queries daily (mix of dashboard, API, batch)
- Users: 200 internal users + 50,000 external API calls/day
- Current Cost: \$200,000/month
- Growth Rate: 25% monthly user growth, 40% monthly data growth

Pain Points: - Costs growing faster than revenue - Query

- performance degrading during peak hours
- Multiple data platform vendors creating complexity
- No visibility into cost drivers by team/project
- Lack of capacity planning causing frequent firefighting

Requirements

Business Requirements: 1. Reduce monthly costs by 30% without impacting performance 2. Handle 3x current scale over next 12 months 3. Improve cost predictability and budgeting 4. Maintain <2 second response time for customer dashboards 5. Enable cost accountability by business unit

Technical Requirements: 1. Optimize storage costs through lifecycle management 2. Implement intelligent query optimization 3. Design auto-scaling policies for variable workloads 4. Create capacity planning model for 12-month projections 5. Build cost monitoring and alerting system

Deliverables

1. Cost Analysis & Baseline (3 pages)

Current cost breakdown by platform/workload/team

Cost per query and cost per GB analysis

Identification of top cost drivers (80/20 analysis)

Benchmark against industry standards

2. Multi-Platform Optimization Strategy (4 pages)

Workload placement optimization across

Snowflake/BigQuery/Redshift

Data lifecycle and tiering strategy

Query optimization and caching implementation plan

Reserved capacity vs on-demand optimization

3. Capacity Planning Model (3 pages)

Growth modeling based on business metrics

Resource scaling timeline for 3x growth Cost projection scenarios (conservative, baseline, aggressive)

Infrastructure investment recommendations

4. Implementation Roadmap (2 pages)

90-day quick wins (immediate cost reductions) 6-month strategic initiatives (platform optimization) 12-month capacity scaling plan

Success metrics and KPIs

5. Organizational Changes (2 pages)

Cost visibility and accountability framework

Team training and governance processes

Automated cost controls and approval workflows

Monthly reporting and review processes

Evaluation Criteria

- Cost Optimization (25 points)
 - ☐ Realistic 30% cost reduction plan
 - ☐ Multi-platform optimization strategy
 - ☐ Storage and query optimization tactics
 - ☐ Quantified savings estimates

Capacity Planning (20 points) - ☐ Growth modeling methodology -

☐ Checkpoint

1. ☐ Scaling timeline and resource requirements - ☐ Cost projection accuracy - ☐ Risk scenario planning

- Technical Implementation (20 points)
 - ☐ Practical optimization techniques
 - ☐ Automation and tooling recommendations
 - ☐ Performance impact considerations

- ☐ Monitoring and alerting design
- Business Alignment (20 points)
- ☐ Cost accountability framework
- ☐ Team and governance processes
- ☐ Success metrics definition
- ☐

Executive communication strategy

- Implementation Planning (15 points)
- ☐ Realistic timeline and milestones
- ☐ Resource requirements and dependencies
- ☐ Change management considerations
- ☐ Risk mitigation strategies

Sample Questions to Guide Your Design

Cost Analysis Questions:

- Which workloads are driving 80% of the costs and why?
- How much of the cost growth is due to scale vs inefficiency?
- What's the cost per customer/transaction and how does it compare to revenue?

Platform Questions: - Which workloads should stay on which platform and why?

- Where are the opportunities for consolidation vs specialization?
- How can you optimize data movement between platforms?

Capacity Questions: - What are the leading indicators for capacity needs?

- How do you plan for spiky workloads (end-of-month reporting, etc.)?
- What's your strategy for scaling during high-growth periods?

Governance Questions: - How do you make teams accountable for their data costs?

- What approval processes are needed for expensive queries/projects?
- How do you balance cost optimization with innovation velocity?

Success Criteria

Your strategy should demonstrate:

1. Financial Impact: Clear path to 30% cost reduction with quantified savings 2. Scalability: Plan that supports 3x growth without proportional cost increase 3. Operational Excellence: Sustainable processes that prevent future cost bloat 4. Business Enablement: Cost management that supports, rather than hinders, business growth

Bonus Challenges

1. Real-Time Cost Control: Design a system that can automatically throttle expensive queries during budget overruns
2. Predictive Scaling: Create a model that predicts capacity needs based on business metrics (new customers, feature usage, etc.)
3. FinOps Integration: Design integration with corporate finance systems for automatic cost allocation and chargeback

Further Reading

Resources

Books:

- Cloud FinOps by J.R. Storment and Mike Fuller
- Modern cloud cost management practices
- The Economics of Cloud

Computing by Venkat Venkataraman - Economic principles for cloud

- optimization
- Site Reliability Engineering by Google
- Capacity planning methodologies

Industry Reports: - Flexera State of the Cloud Report - Cloud cost

- management trends
- DataKitchen State of Data Operations
- Data platform cost benchmarks
- Snowflake Economics Guide
- Warehouse cost optimization strategies

Tools and Frameworks: - Cloud Cost Management: AWS Cost

Explorer, GCP Cost Management, Azure Cost Management - Data

Platform Monitoring: Monte Carlo, Great Expectations, Apache Griffin - FinOps Tools: CloudHealth, CloudCheckr, Kubernetes Cost Analyzer

Advanced Topics

Cost Optimization: - Automated resource scheduling and

- hibernation
- Cross-cloud cost arbitrage strategies
- Carbon-aware computing for cost and sustainability

Capacity Planning: - Machine learning for demand forecasting -

Game theory approaches to resource allocation - Chaos engineering for capacity validation

Platform Economics: - Total Cost of Ownership (TCO) modeling -

Value-based pricing for internal data products - Economic models for data platform ROI

Remember: The goal of cost optimization isn't to spend less money — it's to spend money more effectively to enable business growth while maintaining performance and reliability. # Module 11: Case Studies -

Real Company Architectures

What you'll learn: How industry leaders like Netflix, Uber,

Spotify, Airbnb, and Twitter built their data platforms to handle massive scale. You'll analyze their architectural decisions, understand the trade-offs they made, and learn which patterns you can apply to your own systems.

11

Case Studies — Real-World Systems

Learn from Netflix, Uber, Spotify, Airbnb, and Twitter. Real production architectures at massive scale.

Netflix Analytics Platform

⚡ TL;DR

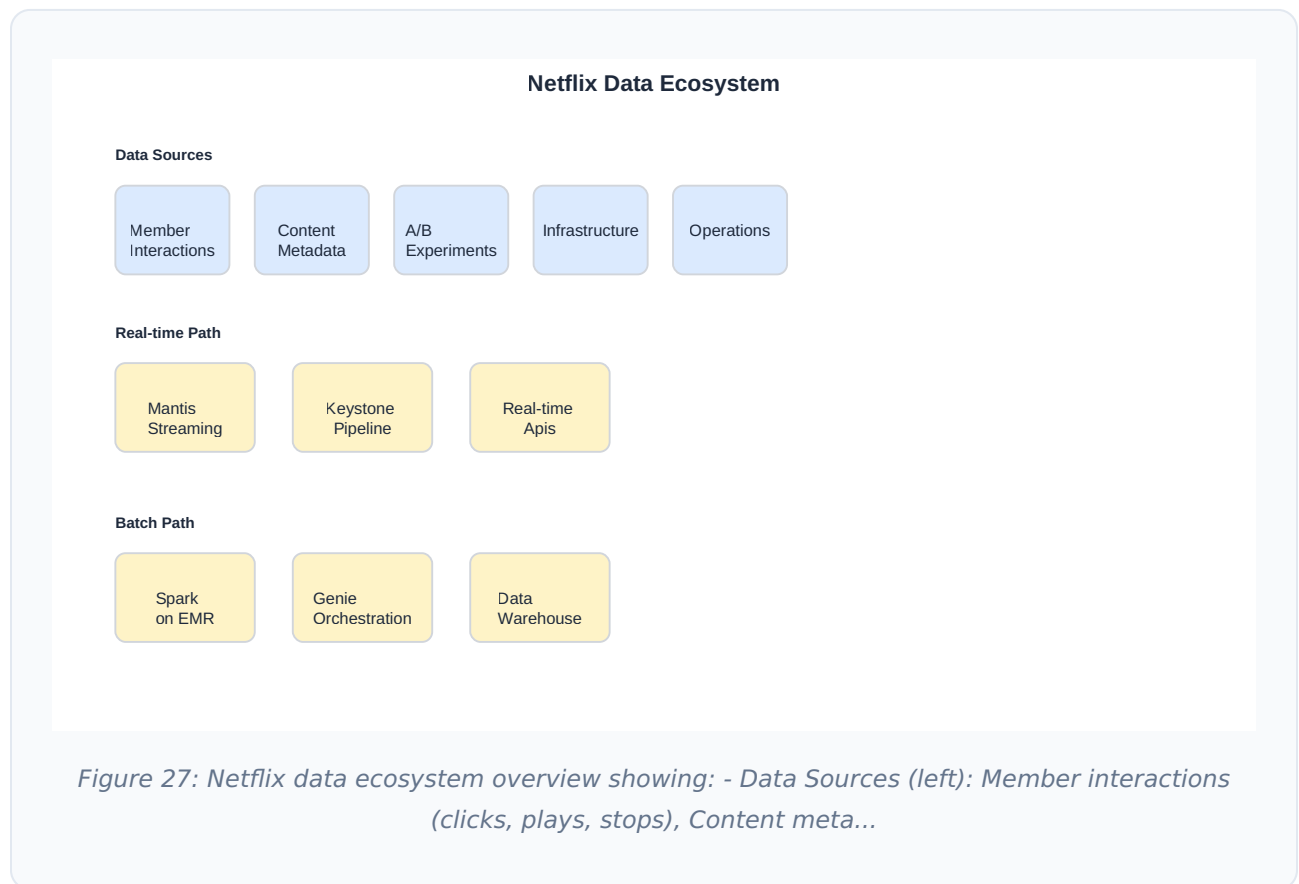
Netflix processes 500+ billion events daily from 200M+ subscribers across 190 countries - Built on AWS with custom stream processing (Mantis), real-time analytics (Druid), and ML platforms - insight: Separate real-time (member experience) from batch (business intelligence) workflows - Innovation: Event-driven architecture with schema evolution and exactly-once processing guarantees

Netflix didn't just disrupt entertainment — they revolutionized how companies think about data at scale. With over 200 million subscribers generating 500+ billion events daily, their data platform handles more data than most companies process in a year.

What makes Netflix's architecture fascinating isn't just the scale, but

how they solved the challenge of providing personalized experiences to millions of users simultaneously while running a data-driven business.

The Netflix Data Challenge



The Scale Netflix Operates At:

Netflix daily data volumes (approximations based on public

information)

```
NETFLIX_DAILY_SCALE = {
  'member_interactions': {
    'play_events': 1_000_000_000,      # 1B play events
    'search_events': 500_000_000,      # 500M searches
    'ui_interactions': 50_000_000_000, # 50B clicks/swipes
    'quality_metrics': 10_000_000_000 # 10B video quality
```

events

```

},
'infrastructure_telemetry': {
    'microservice_logs': 100_000_000_000, # 100B log entries
    'metrics_points': 500_000_000_000,    # 500B metrics
    'traces': 1_000_000_000                # 1B distributed

```

traces

```

},
    'content_metadata': {
        'catalog_updates': 1_000_000,      # 1M catalog changes
        'encoding_jobs': 100_000,          # 100K video encoding jobs
        'cdn_logs': 10_000_000_000         # 10B CDN access logs
    }
}

# Processing requirements
PROCESSING_REQUIREMENTS = {
    'real_time_latency': '< 100ms',         # Recommendation serving
    'batch_freshness': '< 4 hours',         # Business reporting
    'availability': '99.99%',               # Member-facing systems
    'global_replication': '190 countries'
}

```

Real-Time Stream Processing Architecture

Mantis: Netflix's Stream Processing Platform

Netflix built Mantis because existing stream processing frameworks couldn't handle their specific requirements: massive scale, low latency, and operational simplicity.

```

// Simplified Mantis job for real-time recommendation scoring
public class RealtimeRecommendationJob extends MantisJob {

    @Override
    public Observable<RecommendationEvent> getJobObservable(Context

```

PYTHON

context) {

```
return Observable.merge(
    // Multiple input streams
    getMemberInteractionStream(),
    getCatalogUpdateStream(),
    getModelUpdateStream()
)
```

The *PYTHON* code:

```
.window(Duration.ofSeconds(1)) // 1-second windows
.flatMap(this::processRecommendationWindow)
.filter(rec -> rec.getConfidence() > 0.7) // Quality
```

PYTHON

threshold

```
.doOnNext(this::updateMemberProfile) // Side effect
.observeOn(Schedulers.io());
}

private Observable<RecommendationEvent>
```

processRecommendationWindow(

```
Observable<InputEvent> windowedEvents) {

return windowedEvents
    .groupBy(event -> event.getMemberId())
    .flatMap(memberEvents -> {
        // Per-member processing
        return memberEvents
            .scan(new MemberContext(),
```

PYTHON

this::updateMemberContext)

```
.filter(context ->
```

PYTHON

context.needsRecommendationUpdate())

```
.map(this::generateRecommendations);
})
.timeout(Duration.ofMillis(50)) // 50ms timeout per
```

member

```
.onErrorResumeNext(this::handleRecommendationError);
}

private MemberContext updateMemberContext(MemberContext context,
```

InputEvent event) {

```
    switch (event.getType()) {
        case PLAY_START:
            context.addView(event.getContentId(),
```

```
event.getTimestamp());
```

```
context.updateGenrePreferences(event.getGenres());
        break;
        case PLAY_STOP:
            context.updateWatchDuration(event.getContentId(),
```

```
event.getDuration());
```

```
        break;
        case SEARCH:
            context.addSearchIntent(event.getQuery());
            break;
    }

    // Update real-time ML features
    context.updateFeatures(extractFeatures(event));
    return context;
}

private RecommendationEvent
```

```
generateRecommendations(MemberContext context) {
```

```
// Real-time model inference  
List<ContentRecommendation> recs =
```

```
recommendationModel.predict(
```

```
    context.getFeatures(),  
    context.getMemberId(),  
    50 // Generate top 50 recommendations  
);  
  
    return new RecommendationEvent(  
        context.getMemberId(),  
        recs,  
        System.currentTimeMillis(),  
        "realtime_v2" // Model version  
    );  
}  
}
```

Keystone Data Pipeline: Event Collection at Scale

Netflix's Keystone handles event collection from every Netflix client worldwide, ensuring exactly-once delivery and schema evolution.

```
# Keystone event collection architecture  
class NetflixEventCollector:  
    def __init__(self):  
        # Regional event collection clusters  
        self.regional_clusters = {  
            'us-east-1': KafkaCluster(brokers=100,
```

PYTHON

```
        retention_hours=24),
```

```
            'eu-west-1': KafkaCluster(brokers=50,
```

```
retention_hours=24),
```

```
'ap-southeast-1': KafkaCluster(brokers=30,
```

```
retention_hours=24)
```

```
}

# Schema registry for event evolution
self.schema_registry = SchemaRegistry(
    compatibility='BACKWARD', # Allow old consumers
    evolution_strategy='ADD_OPTIONAL_FIELDS'
)

# Cross-region replication
self.replication_config = {
    'sync_regions': ['us-east-1', 'us-west-2'],    # Active-
```

PYTHON

active

```
'async_regions': ['eu-west-1', 'ap-southeast-1'],    #
```

Eventual consistency

```
'replication_factor': 3
}

def collect_member_event(self, event: MemberEvent) -> str:
    """Collect member interaction event with exactly-once
```

PYTHON

guarantees"""

```
# Validate event schema
schema_validation =
```

```
self.schema_registry.validate_event(event)
```

```

if not schema_validation.is_valid:
    return self.handle_schema_mismatch(event,

```

PYTHON

schema_validation)

```

# Determine target region based on member location
target_region = self.get_optimal_region(event.member_id,

```

PYTHON

event.geo_location)

```

# Add event metadata
enriched_event = self.enrich_event(event, {
    'collection_timestamp': time.time_ns(),
    'region': target_region,
    'schema_version': schema_validation.version,
    'event_id': self.generate_event_id(event) # For

```

PYTHON

deduplication

```

})

# Publish with exactly-once semantics
try:
    producer_record = ProducerRecord(
        topic=f"member-events-{event.event_type}",
        partition=hash(event.member_id) % 100, # Consistent

```

partitioning

PYTHON

```

        key=event.member_id,
        value=enriched_event,
        headers={
            'schema_id': schema_validation.schema_id,
            'priority': self.get_event_priority(event)
        }
    )

    # Synchronous send for critical events, async for others
    if event.is_critical:
        result =

```

```

self.regional_clusters[target_region].send_sync(

```

```

producer_record,

```

```

        timeout_ms=100
    )
else:
    result =

```

```

self.regional_clusters[target_region].send_async(

```

PYTHON

```

producer_record,
        callback=self.handle_send_result
    )

    # Cross-region replication for global events
    if event.requires_global_replication:
        self.replicate_cross_region(enriched_event,

```

target_region)

PYTHON

```

        return result.event_id

        except ProducerException as e:
            return self.handle_collection_failure(event, e)

    def enrich_event(self, event: MemberEvent, metadata: dict) ->

```

dict:

```
"""Enrich event with Netflix-specific context"""
```

SQL

```
# Add member context from cache
member_context = self.get_member_context(event.member_id)
```

```
# Add content context if applicable
content_context = {}
if hasattr(event, 'content_id') and event.content_id:
    content_context =
```

```
self.get_content_metadata(event.content_id)
```

```
# Add experimentation context
experiment_context =
```

```
self.get_experiment_assignments(event.member_id)
```

```
return {
    **event.to_dict(),
    **metadata,
    'member_context': {
        'subscription_type': member_context.get('plan'),
        'region': member_context.get('country'),
        'device_capabilities':
```

```
member_context.get('device_profile'),
```

```
'parental_controls':
```

```
member_context.get('maturity_rating')
```

PYTHON

```

    },
    'content_context': content_context,
    'experiment_context': experiment_context,
    'session_context': {
        'session_id': event.session_id,
        'app_version': event.app_version,
        'device_type': event.device_type
    }
}

def get_optimal_region(self, member_id: str, geo_location: str)

```

-> str:

PYTHON

```

"""Determine optimal collection region for member"""

# Route based on data residency requirements
if geo_location in ['DE', 'FR', 'IT', 'ES']:
    return 'eu-west-1' # GDPR compliance
elif geo_location in ['JP', 'KR', 'SG', 'AU']:
    return 'ap-southeast-1' # APAC data residency
else:
    # Load balance between US regions
    return 'us-east-1' if hash(member_id) % 2 == 0 else 'us-

```

west-2'

PYTHON

```

def handle_schema_mismatch(self, event: MemberEvent, validation:

```

SchemaValidation) -> str:

PYTHON

```

"""Handle schema evolution and backward compatibility"""

if validation.error_type == 'MISSING_REQUIRED_FIELD':
    # Try to populate with defaults
    enriched_event = self.add_default_fields(event,

validation.missing_fields)

```

```

return self.collect_member_event(enriched_event)

elif validation.error_type == 'UNKNOWN_FIELD':
    # Log for schema evolution but continue processing
    self.log_schema_evolution(event,

```

PYTHON

```

validation.unknown_fields)

```

```

# Remove unknown fields and process
cleaned_event = self.remove_unknown_fields(event,

```

PYTHON

```

validation.unknown_fields)

```

```

return self.collect_member_event(cleaned_event)

else:
    # Critical schema violation
    self.alert_schema_team(event, validation)
    raise SchemaValidationException(f"Unable to process

```

PYTHON

```

event: {validation.error}")

```

```

# Example of how Netflix handles different event types
class NetflixEventTypes:

    @staticmethod
    def create_playback_event(member_id: str, content_id: str,
                             position_ms: int, event_type: str) ->

```

PYTHON

MemberEvent:

```

"""Create standardized playback event"""
return MemberEvent(
    member_id=member_id,
    event_type=f'playback_{event_type}', # play_start,

```

play_pause, play_stop

```
timestamp=time.time_ns(),
content_id=content_id,
position_ms=position_ms,
is_critical=True, # Playback events affect
```

recommendations

```
requires_global_replication=True, # Needed for global
```

recommendations

```
session_id=get_current_session(),
    device_type=get_device_type(),
    app_version=get_app_version()
)

@staticmethod
def create_search_event(member_id: str, query: str, results:
```

PYTHON

List[str]) -> MemberEvent:

```
"""Create search interaction event"""
return MemberEvent(
    member_id=member_id,
    event_type='search_query',
    timestamp=time.time_ns(),
    query=query,
    result_count=len(results),
    top_results=results[:10], # Store top 10 for analysis
    is_critical=False, # Search events are important but
```

not critical

```
requires_global_replication=False, # Regional search
```

patterns vary

```
session_id=get_current_session()
)
```

```
@staticmethod
def create_recommendation_feedback(member_id: str, content_id:
```

str,

```
action: str, recommendation_id:
```

str) -> MemberEvent:

```
"""Create recommendation feedback event (thumbs up/down,
```

```
click, etc.)"""
```

```
return MemberEvent(
    member_id=member_id,
    event_type=f'recommendation_{action}', # click,
```

thumbs_up, thumbs_down

```
timestamp=time.time_ns(),
content_id=content_id,
recommendation_id=recommendation_id, # Track back to
```

specific recommendation

```
is_critical=True, # Critical for ML feedback loops
requires_global_replication=True, # Global
```

recommendation learning

```
session_id=get_current_session()
)
```

Machine Learning Platform Integration

Real-Time Model Serving

Netflix serves ML models for recommendations, artwork personalization, and quality optimization with strict latency requirements.

PYTHON

```
class NetflixML-serving:
    def __init__(self):
        # Model serving infrastructure
        self.model_registry = ModelRegistry(
            storage='s3://netflix-ml-models/',
            versioning='semantic', # Major.Minor.Patch
            rollback_capability=True
        )

        # Real-time feature store
        self.feature_store = FeatureStore(
            online_store='Elasticache', # Redis for low latency
            offline_store='S3',          # Historical features
            feature_freshness_sla='< 100ms'
        )

        # Model serving clusters by region
        self.serving_clusters = {
            'us-east-1': ModelCluster(instances=100,
                                     model_cache_size='10GB'),
            'eu-west-1': ModelCluster(instances=50,
                                     model_cache_size='5GB'),
            'ap-southeast-1': ModelCluster(instances=30,
                                           model_cache_size='3GB')
```

```
}
```

```
def serve_recommendation(self, member_id: str, context: Dict) ->
```

```
List[Recommendation]:
```

```
"""Serve personalized recommendations with <100ms latency"""
```

```
    start_time = time.time()
```

```
    try:
```

```
        # 1. Get real-time features (target: <20ms)
        member_features =
```

```
self.feature_store.get_online_features(
```

```
    feature_group='member_profile',
    entity_id=member_id,
    feature_names=[
        'viewing_history_7d', 'genre_preferences',
```

```
'time_of_day_pattern',
```

```
    'device_preferences', 'watch_completion_rate'
    ]
)
```

```
    # 2. Get contextual features (target: <10ms)
    contextual_features =
```

```
self.extract_contextual_features(context)
```

```
    # 3. Model inference (target: <50ms)
    model = self.get_model('recommendation_v3.2.1',
```

```
member_features.region)
```



```

combined_features = {
    **member_features.features,
    **contextual_features,
    'request_timestamp': start_time
}

# Run inference
raw_predictions = model.predict(combined_features)

# 4. Post-process recommendations (target: <20ms)
recommendations = self.post_process_recommendations(
    raw_predictions,
    member_id,
    context
)

# 5. Log for monitoring and training
self.log_recommendation_request({
    'member_id': member_id,
    'model_version': model.version,
    'latency_ms': (time.time() - start_time) * 1000,
    'recommendation_count': len(recommendations),
    'features_used': list(combined_features.keys())
})

return recommendations

except Exception as e:
    # Fallback to popular content if ML fails
    return self.get_fallback_recommendations(member_id,

```

```
context, error=str(e))
```

```
def post_process_recommendations(self, raw_predictions:
```

```
np.ndarray,
```

```
member_id: str, context: Dict) ->
```

```
List[Recommendation]:
```

```
"""Post-process ML predictions into final recommendations"""
```

```
# 1. Apply business rules  
filtered_predictions =
```

```
self.apply_business_rules(raw_predictions, member_id)
```

```
# 2. Diversity and exploration  
diverse_recommendations =
```

```
self.ensure_recommendation_diversity(
```

```
    filtered_predictions,  
    diversity_weight=0.3,  
    exploration_rate=0.1  
)
```

```
# 3. A/B test allocation  
if self.is_in_experiment(member_id,
```

PYTHON

```
'recommendation_algorithm_v4'):
```

```
    diverse_recommendations =
```

```
self.apply_experimental_ranking(
```

```
    diverse_recommendations, member_id  
)  
  
# 4. Personalize artwork and metadata  
final_recommendations = []  
for pred in diverse_recommendations[:50]: # Top 50  
    recommendation = Recommendation(  
        content_id=pred['content_id'],  
        predicted_rating=pred['rating'],  
        confidence=pred['confidence'],  
        reason=pred['explanation'],
```

```
artwork_variant=self.select_artwork(pred['content_id'], member_id),
```

```
        position=len(final_recommendations)
        )
        final_recommendations.append(recommendation)

    return final_recommendations

def apply_business_rules(self, predictions: np.ndarray,
```

PYTHON

member_id: str) -> np.ndarray:

```
    """Apply Netflix business rules to ML predictions"""

    member_profile = self.get_member_profile(member_id)

    # Content availability by region
    available_content =
```

PYTHON

```
self.get_available_content(member_profile['region'])
```

```
predictions =
```

```
predictions[predictions['content_id'].isin(available_content)]
```

```
    # Maturity ratings
    max_rating = member_profile['max_maturity_rating']
    predictions = predictions[predictions['maturity_rating'] <=
```

max_rating]

```
    # Recently watched filter
    recent_watches = self.get_recent_watches(member_id, days=7)
    predictions =
```

PYTHON

```
predictions[~predictions['content_id'].isin(recent_watches)]
```

```
# Promote Netflix originals (business priority)
netflix_originals = predictions['is_netflix_original'] ==
```

True

```
predictions.loc[netflix_originals, 'predicted_rating'] *=
```

1.1

```
return predictions

def ensure_recommendation_diversity(self, predictions:
```

PYTHON

```
np.ndarray,
```

```
diversity_weight: float,
exploration_rate: float) ->
```

np.ndarray:

```
"""Ensure diverse recommendations using MMR (Maximal
```

Marginal Relevance)"""

SQL

```

selected = []
candidates = predictions.copy()

while len(selected) < 50 and len(candidates) > 0:
    if not selected:
        # Select highest rated item first
        best_idx = candidates['predicted_rating'].idxmax()
        selected.append(candidates.loc[best_idx])
        candidates = candidates.drop(best_idx)
    else:
        # MMR selection: balance relevance and diversity
        mmr_scores = []

        for idx, candidate in candidates.iterrows():
            relevance = candidate['predicted_rating']

            # Calculate diversity (genre, release year,

```

etc.)

PYTHON

```
diversity = self.calculate_diversity(candidate,
```

selected)

```

# MMR score
mmr_score = (1 - diversity_weight) * relevance +

```

diversity_weight * diversity

SQL

```

mmr_scores.append((idx, mmr_score))

# Add some exploration (random selection)
if random.random() < exploration_rate:

```

The corresponding SQL query:

SQL

```

selected_idx = random.choice(candidates.index)
else:
    selected_idx = max(mmr_scores, key=lambda x:

```

```
x[1])[0]
```

```
selected.append(candidates.loc[selected_idx])
    candidates = candidates.drop(selected_idx)

return pd.DataFrame(selected)
```

SQL

A/B Testing and Experimentation

Netflix's Experimentation Platform

Netflix runs thousands of A/B tests simultaneously to optimize every aspect of the member experience.

```
class NetflixExperimentationPlatform:
    def __init__(self):
        # Experiment configuration
        self.experiment_config_store = ExperimentConfigStore(
            storage='DynamoDB',
            cache='ElastiCache',
            ttl_seconds=300 # 5-minute cache
        )

        # Statistical analysis engine
        self.stats_engine = StatisticalAnalysisEngine(
            significance_threshold=0.05,
            power=0.8,
            minimum_effect_size=0.01 # 1% minimum detectable effect
        )

        # Assignment service for consistent bucketing
        self.assignment_service = AssignmentService(
            hash_algorithm='SHA256',
            deterministic=True # Same user always gets same
```

PYTHON

assignment

```
)

def get_experiment_assignment(self, member_id: str,
```

PYTHON

experiment_name: str) -> ExperimentAssignment:

```
"""Get experiment assignment for member"""
```

```
# Check if experiment is active  
experiment_config =
```

```
self.experiment_config_store.get_experiment(experiment_name)
```

```
if not experiment_config or not experiment_config.is_active:  
    return ExperimentAssignment.control()
```

PYTHON

```
# Check member eligibility  
if not self.is_member_eligible(member_id,
```

experiment_config):

```
    return ExperimentAssignment.control()
```

```
# Consistent assignment using member ID hash  
assignment_hash =
```

```
self.assignment_service.get_assignment_hash(
```

```
    member_id,  
    experiment_name,  
    experiment_config.salt  
)
```

PYTHON

```
# Determine bucket based on traffic allocation  
bucket = self.determine_bucket(assignment_hash,
```

```
experiment_config.traffic_allocation)
```

```
assignment = ExperimentAssignment(  
    experiment_name=experiment_name,  
    variant=bucket,  
    assignment_timestamp=time.time(),  
    experiment_version=experiment_config.version
```

The PYTHON code:

```
)

    # Log assignment for analysis
    self.log_experiment_assignment(member_id, assignment)

    return assignment

def is_member_eligible(self, member_id: str, experiment_config:
```

ExperimentConfig) -> bool:

```
"""Check if member is eligible for experiment"""

member_profile = self.get_member_profile(member_id)

# Geographic restrictions
if experiment_config.allowed_countries and
```

member_profile.country not in experiment_config.allowed_countries:

```
    return False

# Subscription type restrictions
if experiment_config.allowed_plans and member_profile.plan
```

not in experiment_config.allowed_plans:

```
    return False

# Device restrictions
if experiment_config.allowed_devices and
```

member_profile.primary_device not in experiment_config.allowed_devices:

```
    return False

# Member tenure restrictions
if experiment_config.min_tenure_days and
```

member_profile.tenure_days < experiment_config.min_tenure_days:


```
return False
```

PYTHON

```
# Exclude members in conflicting experiments
active_assignments = self.get_active_assignments(member_id)
for assignment in active_assignments:
    if assignment.experiment_name in
```

experiment_config.conflicting_experiments:

```
return False
```

PYTHON

```
    return True
```

```
def analyze_experiment_results(self, experiment_name: str) ->
```

ExperimentResults:

```
"""Analyze A/B test results using Netflix's statistical
```

framework"""

```
    experiment_config =
```

```
    self.experiment_config_store.get_experiment(experiment_name)
```

```
# Collect experiment data
experiment_data = self.collect_experiment_data(
    experiment_name,
    start_date=experiment_config.start_date,
    end_date=datetime.now()
)

# Primary metric analysis
primary_results = self.analyze_primary_metric(
    experiment_data,
    experiment_config.primary_metric
)

# Secondary metrics analysis
secondary_results = {}
for metric in experiment_config.secondary_metrics:
    secondary_results[metric] =
```

```
self.analyze_secondary_metric(
```

```
    experiment_data,
    metric
)
```

```
# Guardrail metrics (ensure we're not breaking anything)
guardrail_results = {}
for metric in experiment_config.guardrail_metrics:
    guardrail_results[metric] =
```

```
self.analyze_guardrail_metric(
```

PYTHON

```

        experiment_data,
        metric
    )

    # Overall recommendation
    recommendation = self.generate_experiment_recommendation(
        primary_results,
        secondary_results,
        guardrail_results,
        experiment_config
    )

    return ExperimentResults(
        experiment_name=experiment_name,
        analysis_date=datetime.now(),
        primary_metric_results=primary_results,
        secondary_metric_results=secondary_results,
        guardrail_metric_results=guardrail_results,
        recommendation=recommendation,
        statistical_significance=primary_results.p_value < 0.05,
        practical_significance=abs(primary_results.effect_size)
    )

```

```
> experiment_config.minimum_effect_size
```

PYTHON

```

    )

    def analyze_primary_metric(self, data: pd.DataFrame,

```

metric_config: MetricConfig) -> MetricResults:

```
        """Analyze primary metric with Netflix's statistical
```

```
        rigor"""
```

PYTHON

```

        if metric_config.metric_type == 'conversion_rate':
            return self.analyze_conversion_metric(data,

```

```
        metric_config)
```

```
elif metric_config.metric_type == 'continuous':
    return self.analyze_continuous_metric(data,
```

PYTHON

metric_config)

```
elif metric_config.metric_type == 'count':
    return self.analyze_count_metric(data, metric_config)
else:
    raise ValueError(f"Unknown metric type:
```

PYTHON

```
{metric_config.metric_type}")
```

```
def analyze_conversion_metric(self, data: pd.DataFrame,
```

PYTHON

metric_config: MetricConfig) -> MetricResults:

```
"""Analyze conversion rate metrics (e.g., play rate,
```

completion rate)"""

```
# Group by experiment variant
variant_groups = data.groupby('experiment_variant')

results = {}
for variant, group in variant_groups:
    total_users = len(group)
    conversions =
```

SQL

```
group[metric_config.numerator_column].sum()
```

```
conversion_rate = conversions / total_users if
```

total_users > 0 else 0

```
# Calculate confidence interval
ci_lower, ci_upper =
```

```
self.calculate_binomial_ci(conversions, total_users)
```

```
results[variant] = {
    'conversion_rate': conversion_rate,
    'total_users': total_users,
    'conversions': conversions,
    'confidence_interval': (ci_lower, ci_upper)
```

The PYTHON code:

```
}

# Statistical test (Chi-square for conversion rates)
control_conversions = results['control']['conversions']
control_total = results['control']['total_users']
treatment_conversions = results['treatment']['conversions']
treatment_total = results['treatment']['total_users']

chi2_stat, p_value = self.chi_square_test(
    control_conversions, control_total,
    treatment_conversions, treatment_total
)

# Effect size (absolute and relative difference)
control_rate = results['control']['conversion_rate']
treatment_rate = results['treatment']['conversion_rate']
absolute_effect = treatment_rate - control_rate
relative_effect = (absolute_effect / control_rate) if
```

PYTHON

`control_rate > 0 else 0`

```
return MetricResults(
    metric_name=metric_config.name,
    control_value=control_rate,
    treatment_value=treatment_rate,
    absolute_effect=absolute_effect,
    relative_effect=relative_effect,
    p_value=p_value,
```

```
confidence_interval=self.calculate_effect_ci(absolute_effect, results),
```

```
        sample_size_control=control_total,
        sample_size_treatment=treatment_total
    )

    # Example experiment configurations Netflix might run
    NETFLIX_EXPERIMENTS = {
        'homepage_algorithm_v2': {
            'description': 'Test new recommendation algorithm on
```

```
homepage',
```

```
    'primary_metric': 'overall_play_rate',
    'secondary_metrics': ['session_length',
```

```
'content_completion_rate'],
```

```
    'guardrail_metrics': ['churn_rate',
```

```
'customer_satisfaction'],
```

```
    'traffic_allocation': {'control': 50, 'treatment': 50},
    'minimum_effect_size': 0.005, # 0.5% improvement
    'allowed_countries': ['US', 'CA', 'GB'],
    'duration_days': 14
},
    'artwork_personalization': {
        'description': 'Test personalized artwork selection',
        'primary_metric': 'click_through_rate',
        'secondary_metrics': ['play_rate_post_click',
```

SQL

```
'thumbnail_dwell_time'],
```

```
    'guardrail_metrics': ['overall_session_satisfaction'],
    'traffic_allocation': {'control': 90, 'treatment': 10},    #
```

Conservative rollout

```
'minimum_effect_size': 0.01, # 1% improvement
    'allowed_devices': ['web', 'mobile'],
    'duration_days': 21
}
```

Pro Tip Netflix's secret sauce isn't just technology – it's their

culture of experimentation. They test everything: UI changes, algorithms, content recommendations, even internal tools. The data platform enables this by making experimentation fast, reliable, and scientifically rigorous.

□ Checkpoint

1. How does Netflix's separation of real-time and batch processing enable both personalized experiences and business intelligence?
2. What makes Mantis different from other stream processing frameworks, and why did Netflix build it?
3. How does Netflix's experimentation platform ensure statistical rigor while enabling rapid iteration?

Uber Surge Pricing System

⚡ TL;DR

Uber processes 15+ million trips daily with real-time pricing adjustments every 30 seconds - Built on Kafka + Flink for stream processing, with machine learning for demand forecasting - Key innovation: Geospatial data processing at scale with sub-second pricing decisions - Challenge solved: Balancing supply and demand across 10,000+ cities globally

Uber's surge pricing system is one of the most sophisticated real-time data applications ever built. It processes millions of location updates per second, predicts demand across thousands of geographic cells, and adjusts prices dynamically to balance supply and demand — all while ensuring drivers and riders have a great experience.

The system isn't just about charging more during busy times. It's a real-time marketplace that uses data to coordinate millions of people efficiently.



5 seconds)

```

'rider_location_updates': 10_000_000,      # 10M/minute
  'trip_status_updates': 500_000,          # 500K/minute
  'eta_calculations': 50_000_000           # 50M/minute
},
'pricing_requirements': {
  'pricing_update_frequency': '30 seconds',
  'pricing_decision_latency': '< 100ms',
  'geographic_granularity': '0.5 km hexagons',
  'cities_supported': 10_000,
  'simultaneous_pricing_decisions': 100_000 # per cycle
},
'business_constraints': {
  'max_surge_multiplier': 5.0,              # Don't price out

```

riders

```

'price_change_smoothing': True,            # Avoid sudden jumps
'driver_incentive_budget': '$1M/hour',    # Global incentive

```

spending

```

'regulatory_compliance': True              # Different rules per

```

city

```

}
}

```

Real-Time Geospatial Processing

H3 Hexagonal Grid System

Uber divides the world into hexagonal cells using Uber's H3 library, enabling efficient geospatial computation at massive scale.

```
import h3
import pandas as pd
import numpy as np
from typing import Dict, List, Tuple
from dataclasses import dataclass
from datetime import datetime, timedelta

@dataclass
class GeospatialEvent:
    event_id: str
    timestamp: datetime
    latitude: float
    longitude: float
    event_type: str # driver_location, rider_request, trip_start,
```

trip_end

```
entity_id: str # driver_id or rider_id
metadata: Dict

class UberGeospatialProcessor:
    def __init__(self, h3_resolution: int = 9):
        # H3 resolution 9 ≈ 0.5km hexagons (good for city-level
```

pricing)

```
self.h3_resolution = h3_resolution

# In-memory spatial index for active entities
self.spatial_index = {
    'drivers': {}, # h3_cell -> set of driver_ids
    'active_trips': {}, # h3_cell -> set of trip_ids
    'rider_requests': {} # h3_cell -> list of
```

request_timestamps

PYTHON

```

}

    # City-specific configuration
    self.city_configs = self.load_city_configurations()

    # Demand/supply tracking per hexagon
    self.hexagon_state = {} # h3_cell -> HexagonState

def process_location_event(self, event: GeospatialEvent) ->

```

```
List[str]:
```

```
"""Process location event and return affected H3 cells for
```

```
pricing updates"""
```

```

# Convert lat/lon to H3 cell
h3_cell = h3.geo_to_h3(event.latitude, event.longitude,

```

```
self.h3_resolution)
```

PYTHON

```

# Get city context
city_context = self.get_city_context(event.latitude,

```

```
event.longitude)
```

PYTHON

```

if not city_context:
    return [] # Outside service areas

    affected_cells = [h3_cell]

if event.event_type == 'driver_location':
    affected_cells.extend(self.update_driver_location(event,

```

```
h3_cell, city_context))
```

```
elif event.event_type == 'rider_request':
    affected_cells.extend(self.process_rider_request(event,
```

PYTHON

h3_cell, city_context))

```
elif event.event_type == 'trip_start':
    affected_cells.extend(self.process_trip_start(event,
```

PYTHON

h3_cell))

```
elif event.event_type == 'trip_end':
    affected_cells.extend(self.process_trip_end(event,
```

PYTHON

h3_cell))

```
# Update hexagon state for affected cells
for cell in affected_cells:
    self.update_hexagon_state(cell, event.timestamp)

return list(set(affected_cells)) # Deduplicate

def update_driver_location(self, event: GeospatialEvent,
                           h3_cell: str, city_context: Dict) ->
```

PYTHON

```
List[str]:
```

```
"""Update driver location and return cells that need pricing
```

updates"""

SQL

```

driver_id = event.entity_id
affected_cells = []

# Remove driver from previous cell
previous_cell = self.get_driver_previous_cell(driver_id)
if previous_cell and previous_cell != h3_cell:
    if previous_cell in self.spatial_index['drivers']:
        self.spatial_index['drivers']

```

```
[previous_cell].discard(driver_id)
```

PYTHON

```

affected_cells.append(previous_cell)

# Add driver to new cell
if h3_cell not in self.spatial_index['drivers']:
    self.spatial_index['drivers'][h3_cell] = set()
self.spatial_index['drivers'][h3_cell].add(driver_id)

# Check if driver is available for trips
if event.metadata.get('is_available', False):
    # Update supply in neighboring cells (drivers can serve

```

nearby requests)

PYTHON

```
neighbor_cells = self.get_service_area_cells(h3_cell,
```

city_context)

PYTHON

```

affected_cells.extend(neighbor_cells)

    return affected_cells

def process_rider_request(self, event: GeospatialEvent,
                        h3_cell: str, city_context: Dict) ->

```

```
List[str]:
```

```
"""Process rider request and return cells affected by demand
```

change"""

```
# Add request to spatial index
if h3_cell not in self.spatial_index['rider_requests']:
    self.spatial_index['rider_requests'][h3_cell] = []

self.spatial_index['rider_requests']
```

PYTHON

```
[h3_cell].append(event.timestamp)
```

```
# Clean old requests (older than 10 minutes)
cutoff_time = event.timestamp - timedelta(minutes=10)
self.spatial_index['rider_requests'][h3_cell] = [
    ts for ts in self.spatial_index['rider_requests']
```

PYTHON

[h3_cell]

```
if ts > cutoff_time
]

# Demand spike detection
recent_requests = len([
    ts for ts in self.spatial_index['rider_requests']
```

PYTHON

[h3_cell]

```
if ts > event.timestamp - timedelta(minutes=5)
])

affected_cells = [h3_cell]

# If demand spike, check nearby cells for supply
```

reallocation

```
if recent_requests >
```

```
city_context.get('demand_spike_threshold', 10):
```

```
nearby_cells = h3.k_ring(h3_cell, 2) # 2-ring
```

neighborhood

```
affected_cells.extend(nearby_cells)

    return affected_cells

def get_service_area_cells(self, center_cell: str, city_context:
```

PYTHON

Dict) -> List[str]:

```
"""Get cells that a driver in center_cell can service"""

# Service radius depends on city density and traffic
service_radius =
```

```
city_context.get('driver_service_radius_km', 2.0)
```

```
# Convert radius to H3 ring distance (approximate)
ring_distance = max(1, int(service_radius / 0.5)) # 0.5km
```

per ring at res 9

```
return list(h3.k_ring(center_cell, ring_distance))

def update_hexagon_state(self, h3_cell: str, timestamp:
```

PYTHON

datetime):

PYTHON

```
"""Update supply/demand state for a hexagon"""

if h3_cell not in self.hexagon_state:
    self.hexagon_state[h3_cell] = HexagonState(h3_cell)

state = self.hexagon_state[h3_cell]
state.last_updated = timestamp

# Calculate current supply (available drivers)
available_drivers =
```

```
self.spatial_index['drivers'].get(h3_cell, set())
```

PYTHON

```
state.supply_count = len([
    driver_id for driver_id in available_drivers
    if self.is_driver_available(driver_id)
])

# Calculate current demand (recent requests)
recent_cutoff = timestamp - timedelta(minutes=5)
recent_requests =
```

```
self.spatial_index['rider_requests'].get(h3_cell, [])
```

```
state.demand_count = len([
    req_time for req_time in recent_requests
    if req_time > recent_cutoff
])

# Calculate supply/demand ratio
state.supply_demand_ratio = (
    state.supply_count / max(state.demand_count, 1)
)

# Detect imbalance
state.has_supply_shortage = state.supply_demand_ratio < 0.5
state.has_excess_supply = state.supply_demand_ratio > 3.0

@dataclass
class HexagonState:
    h3_cell: str
    supply_count: int = 0
    demand_count: int = 0
    supply_demand_ratio: float = 1.0
    has_supply_shortage: bool = False
    has_excess_supply: bool = False
    last_updated: datetime = None
    current_surge_multiplier: float = 1.0
```

Real-Time Demand Forecasting

Machine Learning for Demand Prediction

Uber uses ML models to predict demand 15-30 minutes into the future, enabling proactive supply allocation.

```

import tensorflow as tf
from sklearn.preprocessing import StandardScaler
import joblib

class UberDemandForecastingModel:
    def __init__(self):
        # Time series model for demand prediction
        self.demand_model = self.load_demand_model()

        # Feature engineering pipeline
        self.feature_pipeline = self.load_feature_pipeline()

        # Model serving cache
        self.prediction_cache = {}
        self.cache_ttl = 60 # 1 minute cache

        # External data sources
        self.weather_service = WeatherService()
        self.events_service = EventsService()
        self.traffic_service = TrafficService()

    def predict_demand(self, h3_cell: str,

```

prediction_horizon_minutes: int = 30) -> DemandForecast:

```

"""Predict demand for a specific hexagon"""

cache_key = f"{h3_cell}_{prediction_horizon_minutes}"
cached_result = self.get_cached_prediction(cache_key)
if cached_result:
    return cached_result

# Extract features for prediction
features = self.extract_features(h3_cell)

# Generate prediction
prediction = self.demand_model.predict(features.reshape(1,

```

-1))[0]

```
# Post-process prediction
forecast = DemandForecast(
    h3_cell=h3_cell,
    predicted_demand=max(0, prediction),    # Ensure non-
```

negative

```
prediction_time=datetime.now(),
horizon_minutes=prediction_horizon_minutes,
```

```
confidence_interval=self.calculate_prediction_confidence(features),
```

```
contributing_factors=self.explain_prediction(features)
)

# Cache result
self.cache_prediction(cache_key, forecast)

return forecast

def extract_features(self, h3_cell: str) -> np.ndarray:
    """Extract features for demand prediction model"""

    current_time = datetime.now()

    # Temporal features
    temporal_features = [
        current_time.hour,                # Hour of day
        current_time.weekday(),           # Day of week
        current_time.day,                 # Day of month
        int(current_time.month == 12),     # Holiday season
        int(self.is_weekend(current_time)), # Weekend flag
    ]

    # Historical demand features (last 24 hours)
```

PYTHON

The PYTHON code:

```
historical_demand = self.get_historical_demand(h3_cell,
```

PYTHON

```
hours=24)
```

```

historical_features = [
    np.mean(historical_demand[-24:]),    # 24h average
    np.mean(historical_demand[-168:]),  # 7d average
    np.std(historical_demand[-24:]),    # 24h volatility
    historical_demand[-1],               # Last hour demand
    np.mean(historical_demand[-6:]),    # 6h average
]

# Weather features
weather_data =

```

```

self.weather_service.get_current_weather(h3_cell)

```

```

weather_features = [
    weather_data.temperature,
    weather_data.precipitation_probability,
    int(weather_data.is_severe_weather),
    weather_data.visibility_km
]

# Event features
events_data = self.events_service.get_nearby_events(h3_cell,

```

PYTHON

```

radius_km=5)

```

```

event_features = [
    events_data.total_attendees / 1000, # Scale down
    int(events_data.has_major_event),
    events_data.peak_event_time_diff_hours, # Time to peak
    len(events_data.events)
]

# Geographic features (static)
geographic_features = self.get_geographic_features(h3_cell)

# Traffic features
traffic_data =

```

PYTHON

```

self.traffic_service.get_traffic_conditions(h3_cell)

```

```

traffic_features = [
    traffic_data.congestion_level,      # 0-1 scale
    traffic_data.average_speed_kmh / 60, # Normalized
    int(traffic_data.has_incidents)
]

# Combine all features
all_features = np.concatenate([
    temporal_features,
    historical_features,
    weather_features,
    event_features,
    geographic_features,
    traffic_features
])

# Normalize features
normalized_features =

```

```

self.feature_pipeline.transform(all_features.reshape(1, -1))

```

```

return normalized_features.flatten()

def get_geographic_features(self, h3_cell: str) -> List[float]:
    """Get static geographic features for a hexagon"""

    # Get cell center coordinates
    lat, lon = h3.h3_to_geo(h3_cell)

    # Distance to city center
    city_center = self.get_city_center(h3_cell)
    dist_to_center = self.calculate_distance(lat, lon,

```

PYTHON

```

city_center['lat'], city_center['lon'])

```

```

# Distance to airport
airport_location = self.get_nearest_airport(h3_cell)
dist_to_airport = self.calculate_distance(lat, lon,

```

PYTHON

```

airport_location['lat'], airport_location['lon'])

```

```
# POI density
poi_density = self.get_poi_density(h3_cell) # Points of
```

PYTHON

interest

```
# Residential vs commercial area
area_type = self.get_area_classification(h3_cell)

return [
    dist_to_center / 20,          # Normalize by typical
```

PYTHON

city radius

```
dist_to_airport / 50,          # Normalize by typical
```

airport distance

```
poi_density / 100,            # Normalize POI density
area_type['commercial_ratio'], # 0-1 commercial vs
```

residential

```
area_type['office_ratio']      # 0-1 office density
    ]

def explain_prediction(self, features: np.ndarray) -> Dict[str,
```

PYTHON

float]:

```
"""Explain what factors are driving the demand prediction"""

# Use model interpretation (SHAP values or similar)
feature_names = [
    'hour_of_day', 'day_of_week', 'day_of_month',
```

'holiday_season', 'weekend',

```
'24h_avg_demand', '7d_avg_demand', '24h_volatility',
```

```
'last_hour_demand', '6h_avg_demand',
```

```
'temperature', 'precipitation', 'severe_weather',
```

```
'visibility',
```

```
'event_attendees', 'major_event', 'time_to_peak',
```

```
'event_count',
```

```
'dist_to_center', 'dist_to_airport', 'poi_density',
```

```
'commercial_ratio', 'office_ratio',
```

```
'traffic_congestion', 'avg_speed', 'traffic_incidents'  
]
```

```
# Calculate feature importance (simplified)  
feature_importance =
```

```
self.calculate_feature_importance(features)
```

```
# Return top contributing factors  
importance_dict = dict(zip(feature_names,
```

```
feature_importance))
```

```
top_factors = dict(sorted(importance_dict.items(),  
                           key=lambda x: abs(x[1]),
```

PYTHON

```
reverse=True)[:5])
```



```
return top_factors
```

PYTHON

```
@dataclass
class DemandForecast:
    h3_cell: str
    predicted_demand: float
    prediction_time: datetime
    horizon_minutes: int
    confidence_interval: Tuple[float, float]
    contributing_factors: Dict[str, float]
```

Dynamic Pricing Algorithm

Real-Time Surge Calculation

Uber's pricing algorithm balances multiple objectives: maximize utilization, ensure driver earnings, maintain rider satisfaction.

```
from typing import NamedTuple
import math

class SurgePricingConfig(NamedTuple):
```

PYTHON

```
    base_multiplier: float = 1.0
    max_multiplier: float = 5.0
    supply_demand_sensitivity: float = 2.0
    smoothing_factor: float = 0.7 # Prevent sudden price jumps
    rider_price_sensitivity: Dict[str, float] = None # By market
```

segment

```
    driver_incentive_budget: float = 1000.0    # Per hour per city

    class UberSurgePricingEngine:
        def __init__(self):
            self.pricing_configs = self.load_pricing_configs() # Per-
```

PYTHON

city configs

```

self.surge_history = {} # Track price changes over time

# Machine learning models
self.demand_elasticity_model = self.load_elasticity_model()
self.driver_response_model = self.load_driver_model()

def calculate_surge_multiplier(self, h3_cell: str,
                              current_state: HexagonState,
                              demand_forecast: DemandForecast,
                              city_config: SurgePricingConfig) ->

```

SurgeDecision:

```

"""Calculate optimal surge multiplier for a hexagon"""

# 1. Base surge calculation based on supply/demand
base_surge = self.calculate_base_surge(current_state,

```

city_config)

```

# 2. Forward-looking adjustment based on demand forecast
forecast_adjustment = self.calculate_forecast_adjustment(
    demand_forecast, current_state, city_config
)

# 3. Price elasticity consideration
elasticity_adjustment =

```

```

self.calculate_elasticity_adjustment(

```

```

    h3_cell, base_surge, city_config
)

```

```

# 4. Driver incentive optimization
driver_adjustment =

```

```

self.calculate_driver_incentive_adjustment(

```

```

h3_cell, current_state, city_config
)

# Combine adjustments
raw_multiplier = (
    base_surge *
    forecast_adjustment *
    elasticity_adjustment *
    driver_adjustment
)

# 5. Apply smoothing and constraints
final_multiplier = self.apply_surge_constraints(
    h3_cell, raw_multiplier, city_config
)

# 6. Generate decision with explanation
decision = SurgeDecision(
    h3_cell=h3_cell,
    surge_multiplier=final_multiplier,

```

```
previous_multiplier=current_state.current_surge_multiplier,
```

```

calculation_time=datetime.now(),
components={
    'base_surge': base_surge,
    'forecast_adjustment': forecast_adjustment,
    'elasticity_adjustment': elasticity_adjustment,
    'driver_adjustment': driver_adjustment,
    'raw_multiplier': raw_multiplier

```

The PYTHON code:

```

},
expected_impact=self.estimate_impact(h3_cell,

```

final_multiplier)

```
)
```

PYTHON

```
    return decision
```

```
def calculate_base_surge(self, state: HexagonState, config:
```

SurgePricingConfig) -> float:

```
    """Calculate base surge multiplier from supply/demand
```

SQL

```
ratio"""
```

```
    if state.supply_demand_ratio >= 2.0:
        # Excess supply - below normal pricing
        return max(0.8, 1.0 - (state.supply_demand_ratio - 2.0)
```

```
        * 0.1)
```

```
    elif state.supply_demand_ratio >= 1.0:
        # Balanced market
        return 1.0

    else:
        # Supply shortage - apply surge
        # Exponential curve: surge = 1 / (ratio ^ sensitivity)
        surge = min(
            config.max_multiplier,
            1.0 / (state.supply_demand_ratio **
```

PYTHON

```
            config.supply_demand_sensitivity)
```

```
)
```

PYTHON

```
    return surge
```

```
def calculate_forecast_adjustment(self, forecast:
```

DemandForecast,

```
state: HexagonState,  
config: SurgePricingConfig) ->
```

float:

```
"""Adjust pricing based on demand forecast"""  
  
current_demand = state.demand_count  
predicted_demand = forecast.predicted_demand  
  
if current_demand == 0:  
    demand_growth_rate = 0  
else:  
    demand_growth_rate = (predicted_demand - current_demand)
```

/ current_demand

```
# If demand is expected to grow significantly, increase
```

pricing proactively

```
if demand_growth_rate > 0.5: # 50% growth expected  
    return 1.3 # 30% price increase  
elif demand_growth_rate > 0.2: # 20% growth expected  
    return 1.1 # 10% price increase  
elif demand_growth_rate < -0.3: # 30% demand drop expected  
    return 0.9 # 10% price decrease  
else:  
    return 1.0 # No adjustment  
  
def calculate_elasticity_adjustment(self, h3_cell: str,
```

PYTHON

base_surge: float,

```
config: SurgePricingConfig) ->
```

float:

PYTHON

```
"""Adjust pricing based on local price elasticity"""
```

```
# Get local market characteristics
market_segment = self.get_market_segment(h3_cell)
elasticity =
```

```
self.demand_elasticity_model.predict_elasticity(h3_cell, market_segment)
```

```
# If market is highly price sensitive and surge is high,
```

moderate the increase

```
if elasticity < -1.5 and base_surge > 2.0: # Elastic demand
```

+ high surge

PYTHON

```
return 0.8 # Reduce surge to maintain volume
```

```
# If market is price insensitive, can push pricing higher
elif elasticity > -0.5 and base_surge > 1.5: # Inelastic
```

demand

PYTHON

```
return 1.2 # Increase surge for revenue optimization
```

```
else:
    return 1.0 # No adjustment
```

```
def calculate_driver_incentive_adjustment(self, h3_cell: str,
```

state: HexagonState,

```
config:
```

SurgePricingConfig) -> float:

```

"""Calculate adjustment based on driver supply incentives"""

# Check if we need to attract more drivers to this area
nearby_cells = h3.k_ring(h3_cell, 2)
nearby_supply = sum(
    self.get_supply_count(cell) for cell in nearby_cells
)

nearby_demand = sum(
    self.get_demand_count(cell) for cell in nearby_cells
)

regional_supply_ratio = nearby_supply / max(nearby_demand,

```

1)

```

# If regional supply is low, increase incentives (lower

```

surge threshold)

```

if regional_supply_ratio < 0.5:
    # Check remaining incentive budget
    current_incentive_spend =

```

```

self.get_current_incentive_spend(h3_cell)

```

```

if current_incentive_spend <

```

config.driver_incentive_budget:

```

return 1.2 # Boost surge to attract drivers

return 1.0 # No adjustment

def apply_surge_constraints(self, h3_cell: str, raw_multiplier:

```

float,

```

config: SurgePricingConfig) -> float:
    """Apply smoothing and business constraints"""

    # Get previous surge for smoothing
    previous_surge = self.surge_history.get(h3_cell,

```

```

{}).get('last_multiplier', 1.0)

```

```

    # Apply smoothing to prevent sudden jumps
    smoothed_multiplier = (
        config.smoothing_factor * previous_surge +
        (1 - config.smoothing_factor) * raw_multiplier
    )

    # Apply hard constraints
    final_multiplier = max(
        config.base_multiplier,
        min(config.max_multiplier, smoothed_multiplier)
    )

    # Business rules: don't change price too frequently
    time_since_last_change =

```

```

self.get_time_since_last_change(h3_cell)

```

```

    if time_since_last_change < timedelta(minutes=2): # Min 2

```

minutes between changes

```

    if abs(final_multiplier - previous_surge) < 0.1: # Less

```

than 10% change

```

    return previous_surge # Keep previous price

```

Record price change:


```
# Record price change
self.record_surge_change(h3_cell, final_multiplier)

return round(final_multiplier, 1)    # Round to 1 decimal
```

PYTHON

place

```
def estimate_impact(self, h3_cell: str, surge_multiplier: float)
```

PYTHON

-> Dict[str, float]:

```
"""Estimate impact of surge pricing decision"""

# Use historical data and ML models to predict impact
current_state = self.get_hexagon_state(h3_cell)

# Predict demand response
demand_change =
```

PYTHON

```
self.demand_elasticity_model.predict_demand_change(
```

```
h3_cell, surge_multiplier
)

# Predict driver supply response
supply_change =
```

```
self.driver_response_model.predict_supply_change(
```

```
h3_cell, surge_multiplier
)

# Calculate expected utilization
new_demand = current_state.demand_count * (1 +
```

demand_change)

```
new_supply = current_state.supply_count * (1 +
```

supply_change)

```
expected_utilization = new_demand / max(new_supply, 1)

    return {
        'expected_demand_change_percent': demand_change * 100,
        'expected_supply_change_percent': supply_change * 100,
        'expected_utilization_ratio': expected_utilization,
        'estimated_revenue_impact_percent': (
            (surge_multiplier * (1 + demand_change) - 1) * 100
        )
    }
```

```
@dataclass
class SurgeDecision:
    h3_cell: str
    surge_multiplier: float
    previous_multiplier: float
    calculation_time: datetime
    components: Dict[str, float]
    expected_impact: Dict[str, float]
```

Pro Tip Uber's surge pricing isn't just about maximizing revenue – it's about marketplace efficiency. The goal is to minimize rider wait times and driver idle time by using price signals to balance supply and demand. The data platform enables this by processing millions of location updates in real-time and making pricing decisions every 30 seconds across thousands of cities.

□ Checkpoint

1. How does Uber use H3 hexagonal grids to make geospatial processing scalable?
2. What role does demand forecasting play in proactive surge pricing decisions?
3. How does Uber balance multiple objectives (rider satisfaction, driver earnings, marketplace efficiency) in their pricing algorithm?

PYTHON

Spotify Recommendations Data Platform

TL;DR

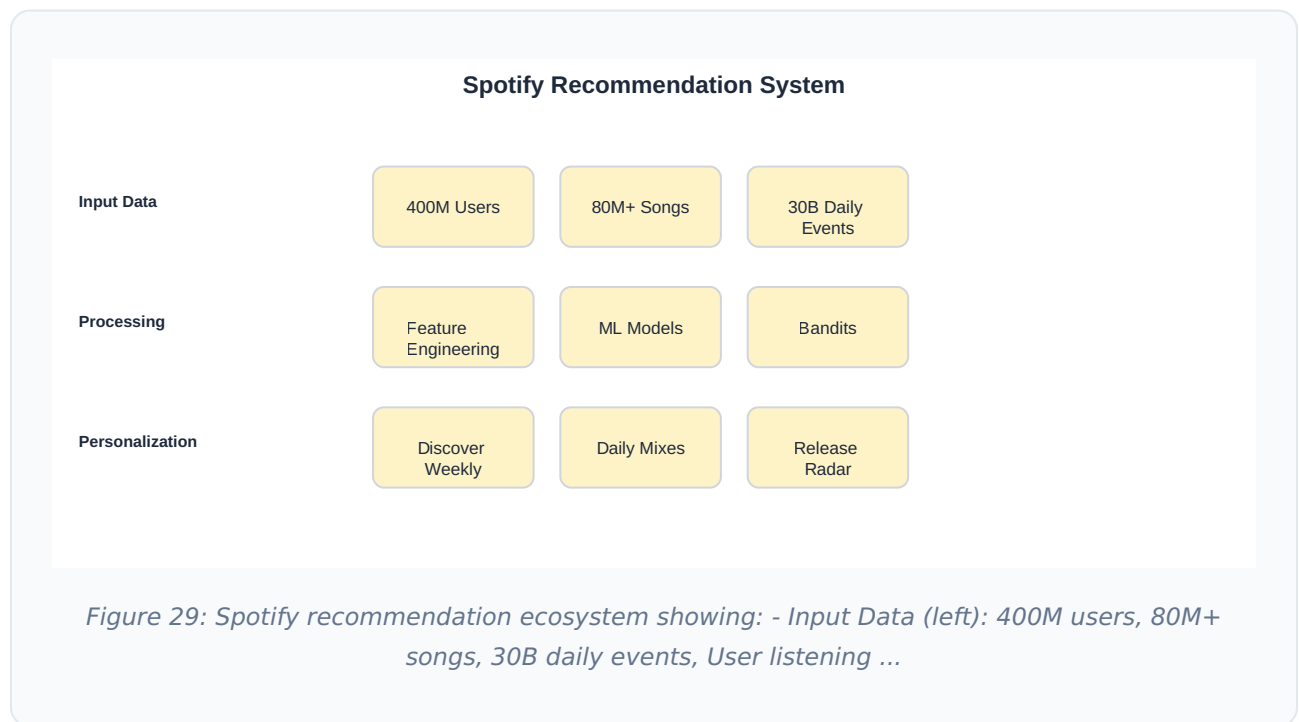
Spotify processes 30+ billion events daily from 400M+ users to power personalized recommendations - Built on Google Cloud with custom feature store and real-time ML serving infrastructure - Key innovation: Multi-armed bandit algorithms for exploration vs exploitation in music discovery - Challenge: Handle the "cold start" problem for new users and new music releases

Spotify's recommendation system is arguably the most sophisticated content discovery platform ever built. With over 400 million users and 80+ million songs, they face the unique challenge of helping people discover music they'll love from an essentially infinite catalog.

What makes Spotify's data platform remarkable isn't just the scale —

it's how they solve the tension between giving users what they want (exploitation) and helping them discover new music they didn't know they wanted (exploration).

The Spotify Recommendation Challenge



Spotify's Recommendation Scale:

```
# Spotify's daily processing volume
SPOTIFY_SCALE = {
    'user_interactions': {
        'streams_completed': 1_000_000_000,    # 1B completed
```

streams

```
'skips': 500_000_000,                # 500M skips
'likes_saves': 50_000_000,           # 50M likes/saves
'playlist_adds': 100_000_000,        # 100M playlist
```

additions

```
'searches': 200_000_000          # 200M searches
},
'content_data': {
  'new_releases': 100_000,        # 100K new songs
```

daily

```
'audio_features_extracted': 100_000,    # Audio analysis for
```

new content

```
'lyrics_processed': 50_000,            # NLP on lyrics
'playlist_updates': 10_000_000         # 10M playlist
```

modifications

```
},
'ml_predictions': {
  'recommendation_requests': 2_000_000_000,    # 2B rec requests
```

daily

```
'model_inferences': 50_000_000_000,        # 50B ML
```

predictions

```
'feature_lookups': 100_000_000_000,        # 100B feature
```

store queries

```
'a_b_test_assignments': 500_000_000        # 500M experiment
```

assignments

```
}  
}
```

Real-Time Feature Engineering

User Taste Profile Computation

Spotify maintains real-time taste profiles for 400M+ users, updating preferences as users listen.

```

import numpy as np
from typing import Dict, List, Optional
from dataclasses import dataclass, field
from datetime import datetime, timedelta
import json

@dataclass
class AudioFeatures:
    """Spotify's audio features for a track"""
    danceability: float      # 0-1
    energy: float            # 0-1
    key: int                 # 0-11 (musical key)
    loudness: float          # dB
    mode: int                # 0 (minor) or 1 (major)
    speechiness: float       # 0-1
    acousticness: float      # 0-1
    instrumentalness: float  # 0-1
    liveness: float          # 0-1
    valence: float           # 0-1 (positivity)
    tempo: float             # BPM
    duration_ms: int         # Track length

@dataclass
class UserTasteProfile:
    """Real-time user taste profile"""
    user_id: str
    last_updated: datetime

    # Audio feature preferences (learned from listening history)
    preferred_danceability: float = 0.5
    preferred_energy: float = 0.5
    preferred_valence: float = 0.5
    preferred_acousticness: float = 0.5
    preferred_tempo: float = 120.0

    # Genre preferences (weighted by recent listening)
    genre_weights: Dict[str, float] = field(default_factory=dict)

    # Artist preferences (with decay over time)
    artist_weights: Dict[str, float] = field(default_factory=dict)

    # Temporal listening patterns
    time_of_day_preferences: Dict[int, List[str]] =

```

```

field(default_factory=dict) # hour -> genres

```

```
day_of_week_preferences: Dict[int, List[str]] =
```

```
field(default_factory=dict) # weekday -> genres
```

```
# Exploration vs exploitation balance  
exploration_factor: float = 0.2 # How much new music to
```

recommend

```
# Session context  
current_session_context: Optional[Dict] = None  
  
class SpotifyFeatureStore:  
    def __init__(self):  
        # Real-time user profiles  
        self.user_profiles = {} # user_id -> UserTasteProfile  
  
        # Track features cache  
        self.track_features = {} # track_id -> AudioFeatures  
  
        # Real-time feature computation  
        self.feature_pipeline = self.setup_feature_pipeline()  
  
        # Session tracking  
        self.active_sessions = {} # session_id -> SessionContext  
  
    def process_listening_event(self, event: Dict) -> None:
```

PYTHON

```
"""Process real-time listening event to update user taste
```

profile"""

SQL

```

user_id = event['user_id']
track_id = event['track_id']
listen_duration_ms = event['listen_duration_ms']
track_duration_ms = event['track_duration_ms']
event_type = event['event_type'] # play, skip, like, save
timestamp = datetime.fromtimestamp(event['timestamp'])

# Calculate completion ratio
completion_ratio = listen_duration_ms /

```

```

max(track_duration_ms, 1)

```

PYTHON

```

# Get or create user profile
if user_id not in self.user_profiles:
    self.user_profiles[user_id] = UserTasteProfile(
        user_id=user_id,
        last_updated=timestamp
    )

profile = self.user_profiles[user_id]

# Get track features
track_features = self.get_track_features(track_id)
track_metadata = self.get_track_metadata(track_id)

# Update profile based on user interaction
if event_type == 'skip' or completion_ratio < 0.1:
    # User didn't like this track - negative signal
    self.update_taste_profile_negative(profile,

```

```

track_features, track_metadata)

```

PYTHON

```

elif event_type in ['like', 'save'] or completion_ratio >

```

```

0.8:

```

PYTHON

```

# Strong positive signal
weight = 1.5 if event_type in ['like', 'save'] else 1.0
self.update_taste_profile_positive(profile,

```

```

track_features, track_metadata, weight)

```

```
elif completion_ratio > 0.3:
    # Moderate positive signal
    self.update_taste_profile_positive(profile,
```

PYTHON

```
track_features, track_metadata, 0.5)
```

```
# Update temporal preferences
self.update_temporal_preferences(profile, track_metadata,
```

PYTHON

timestamp)

```
# Update session context
self.update_session_context(user_id, event, timestamp)

profile.last_updated = timestamp

def update_taste_profile_positive(self, profile:
```

PYTHON

UserTasteProfile,

```
track_features: AudioFeatures,
            track_metadata: Dict,
            weight: float):
    """Update user profile with positive signal"""

    learning_rate = 0.1 * weight    # How quickly to adapt to new
```

preferences

```
# Update audio feature preferences using exponential moving
```

average

```
profile.preferred_danceability = self.update_preference(
    profile.preferred_danceability,
```

PYTHON

```
track_features.danceability, learning_rate
```

PYTHON

```
)
profile.preferred_energy = self.update_preference(
    profile.preferred_energy, track_features.energy,
```

learning_rate

```
)
```

The PYTHON code:

PYTHON

```
profile.preferred_valence = self.update_preference(
    profile.preferred_valence, track_features.valence,
```

learning_rate

PYTHON

```
)
profile.preferred_acousticness = self.update_preference(
    profile.preferred_acousticness,
```

```
track_features.acousticness, learning_rate
```

PYTHON

```
)
profile.preferred_tempo = self.update_preference(
    profile.preferred_tempo, track_features.tempo,
```

```
learning_rate * 0.5 # Tempo changes slowly
```

```
)

# Update genre preferences
for genre in track_metadata.get('genres', []):
    current_weight = profile.genre_weights.get(genre, 0.0)
    profile.genre_weights[genre] =
```

```
self.update_preference(current_weight, 1.0, learning_rate)
```

```
# Update artist preferences
artist_id = track_metadata['artist_id']
current_weight = profile.artist_weights.get(artist_id, 0.0)
profile.artist_weights[artist_id] =
```

```
self.update_preference(current_weight, 1.0, learning_rate)
```

```
# Decay other artists slightly to prevent staleness
for artist, w in profile.artist_weights.items():
    if artist != artist_id:
        profile.artist_weights[artist] *= 0.999

def update_taste_profile_negative(self, profile:
```

PYTHON

UserTasteProfile,

```
track_features: AudioFeatures,
                track_metadata: Dict):
    """Update user profile with negative signal (skip)"""

    negative_learning_rate = 0.05 # Learn more slowly from
```

SQL

negative signals

```
# Move preferences away from this track's features
profile.preferred_danceability =
```

SQL

```
self.update_preference_negative(
```

```
profile.preferred_danceability,
```

```
track_features.danceability, negative_learning_rate
```

```
)
profile.preferred_energy = self.update_preference_negative(
    profile.preferred_energy, track_features.energy,
```

PYTHON

negative_learning_rate

```
)  
profile.preferred_valence = self.update_preference_negative(  
    profile.preferred_valence, track_features.valence,
```

PYTHON

negative_learning_rate

```
)  
  
    # Reduce genre weights slightly  
    for genre in track_metadata.get('genres', []):  
        if genre in profile.genre_weights:  
            profile.genre_weights[genre] *= 0.95  
  
def update_preference(self, current_pref: float, new_value:
```

PYTHON

float, learning_rate: float) -> float:

```
"""Update preference using exponential moving average"""  
return current_pref * (1 - learning_rate) + new_value *
```

learning_rate

```
def update_preference_negative(self, current_pref: float,
```

PYTHON

avoided_value: float, learning_rate: float) -> float:

```
"""Move preference away from an avoided value"""  
# Move preference in the opposite direction  
if current_pref > avoided_value:  
    target = min(1.0, current_pref + (current_pref -
```

SQL

```
avoided_value) * 0.1)
```

```
else:  
    target = max(0.0, current_pref - (avoided_value -
```

```
current_pref) * 0.1)
```

```
return current_pref * (1 - learning_rate) + target *
```

learning_rate

```
def update_temporal_preferences(self, profile: UserTasteProfile,  
                                track_metadata: Dict, timestamp:
```

PYTHON

datetime):

```
"""Update time-based listening preferences"""  
  
hour = timestamp.hour  
weekday = timestamp.weekday()  
genres = track_metadata.get('genres', [])  
  
# Update time of day preferences  
if hour not in profile.time_of_day_preferences:  
    profile.time_of_day_preferences[hour] = []  
  
for genre in genres:  
    if genre not in profile.time_of_day_preferences[hour]:  
        profile.time_of_day_preferences[hour].append(genre)  
  
# Keep only top 5 genres per time slot  
profile.time_of_day_preferences[hour] =
```

```
profile.time_of_day_preferences[hour][:5]
```

```
# Update day of week preferences  
if weekday not in profile.day_of_week_preferences:  
    profile.day_of_week_preferences[weekday] = []  
  
for genre in genres:  
    if genre not in
```

```
profile.day_of_week_preferences[weekday]:  
profile.day_of_week_preferences[weekday].append(genre)
```

pro-

```
profile.day_of_week_preferences[weekday] =
```

```
profile.day_of_week_preferences[weekday][:5]
```

```
def get_user_features_for_recommendation(self, user_id: str,
```

PYTHON

context: Dict) -> Dict:

```
"""Get real-time user features for recommendation model"""
```

PYTHON

```
if user_id not in self.user_profiles:
    # Cold start - return default features
    return self.get_default_user_features(context)
```

```
profile = self.user_profiles[user_id]
current_time = datetime.now()
```

```
# Core taste features
features = {
    'preferred_danceability':
```

```
profile.preferred_danceability,
```

```
'preferred_energy': profile.preferred_energy,
'preferred_valence': profile.preferred_valence,
'preferred_acousticness':
```

```
profile.preferred_acousticness,
```

```
'preferred_tempo': profile.preferred_tempo,
    'exploration_factor': profile.exploration_factor
}
```

```
# Top genre preferences
top_genres = sorted(profile.genre_weights.items(),
```

```
key=lambda x: x[1], reverse=True)[:10]
```

```
for i, (genre, weight) in enumerate(top_genres):
    features[f'genre_pref_{i}'] = weight
    features[f'genre_name_{i}'] = genre

# Contextual features
```

```
hour = current_time.hour
weekday = current_time.weekday()

features['current_hour'] = hour
features['current_weekday'] = weekday

# Time-based genre preferences
if hour in profile.time_of_day_preferences:
    features['hour_preferred_genres'] =
```

```
profile.time_of_day_preferences[hour]
```

```
if weekday in profile.day_of_week_preferences:
    features['weekday_preferred_genres'] =
```

```
profile.day_of_week_preferences[weekday]
```

```
# Session context
session_id = context.get('session_id')
if session_id in self.active_sessions:
    session_context = self.active_sessions[session_id]
    features.update({
        'session_listening_time_minutes':
```

PYTHON

```
session_context['duration_minutes'],
```

```
'session_track_count':
```

```
session_context['track_count'],
```



```
'session_skip_rate': session_context['skip_rate'],
'session_primary_genre':
```

```
session_context['primary_genre']
```

```
})
```

```
return features
```

Multi-Armed Bandit Recommendation Algorithm

Exploration vs Exploitation for Music Discovery

Spotify uses multi-armed bandit algorithms to balance showing users music they'll definitely like with discovering new music they might love.

```
import random
import math
from typing import Tuple
from collections import defaultdict

class SpotifyMultiArmedBandit:
    def __init__(self):
        # Track performance of different recommendation strategies
        self.strategy_stats = defaultdict(lambda: {'pulls': 0,
```

PYTHON

```
'rewards': 0.0, 'confidence': 0.0})
```

```
# Recommendation strategies
self.strategies = [
    'collaborative_filtering',    # Users with similar taste
    'content_based',             # Similar audio features
    'popularity_trending',       # What's trending globally
    'artist_discovery',          # New artists similar to
```

PYTHON

liked ones

```
'genre_exploration',      # Adjacent genres
'time_context',           # Time-appropriate music
'mood_based',             # Current mood detection
'social_influence'        # What friends are
```

listening to

```
]

# Upper Confidence Bound (UCB) parameters
self.confidence_multiplier = 2.0

# Contextual bandits for different user segments
self.segment_bandits = {
    'new_user': {},          # Users with <50 listening
```

PYTHON

hours

```
'casual_user': {},        # 50-500 hours
```

The corresponding SQL query:

```
'active_user': {},        # 500-2000 hours
    'power_user': {}       # >2000 hours
}

def select_recommendation_strategy(self, user_profile:
```

SQL

UserTasteProfile,

```
context: Dict) -> Tuple[str,
```

float]:

```
"""Select which recommendation strategy to use for this
```

SQL

user"""

```
user_segment = self.determine_user_segment(user_profile,
```

PYTHON

context)

```
total_pulls = sum(  
    self.strategy_stats[strategy]['pulls'] for strategy in
```

PYTHON

```
self.strategies
```

```
)  
  
if total_pulls < len(self.strategies) * 10:    # Exploration
```

PYTHON

phase

```
# Ensure each strategy is tried at least 10 times  
least_tried = min(self.strategies, key=lambda s:
```

PYTHON

```
self.strategy_stats[s]['pulls'])
```

```
return least_tried, 1.0 # Full exploration  
  
# UCB strategy selection  
ucb_scores = {}  
  
for strategy in self.strategies:  
    stats = self.strategy_stats[strategy]  
  
    if stats['pulls'] == 0:  
        ucb_scores[strategy] = float('inf')    # Try untested
```

SQL

strategies

PYTHON

```

else:
    # Calculate average reward
    avg_reward = stats['rewards'] / stats['pulls']

    # Calculate confidence bound
    confidence_bound = math.sqrt(
        (self.confidence_multiplier *

```

```

math.log(total_pulls)) / stats['pulls']

```

SQL

```

)

    ucb_scores[strategy] = avg_reward + confidence_bound

    # Select strategy with highest UCB score
    selected_strategy = max(ucb_scores, key=ucb_scores.get)

    # Calculate exploration factor for this strategy
    exploration_factor = min(0.5, confidence_bound / avg_reward)

```

```

if avg_reward > 0 else 0.5

```

SQL

```

return selected_strategy, exploration_factor

def generate_recommendations(self, user_id: str, user_profile:

```

UserTasteProfile,

```

context: Dict, num_recommendations:

```

```

int = 30) -> List[Dict]:

```

SQL

```

"""Generate recommendations using selected strategy"""

    # Select recommendation strategy
    strategy, exploration_factor =

```

```

self.select_recommendation_strategy(user_profile, context)

```

```
# Generate recommendations based on selected strategy
if strategy == 'collaborative_filtering':
    recommendations =
```

SQL

```
self.collaborative_filtering_recommendations(
```

```
    user_id, user_profile, num_recommendations
)
elif strategy == 'content_based':
```

PYTHON

The PYTHON code:

```
recommendations = self.content_based_recommendations(
    user_profile, num_recommendations
)
elif strategy == 'artist_discovery':
    recommendations = self.artist_discovery_recommendations(
        user_profile, num_recommendations
    )
elif strategy == 'genre_exploration':
    recommendations =
```

PYTHON

```
self.genre_exploration_recommendations(
```

```
    user_profile, exploration_factor,
```

num_recommendations

```
)
elif strategy == 'time_context':
    recommendations = self.time_contextual_recommendations(
        user_profile, context, num_recommendations
    )
else:
    # Fallback to popularity
    recommendations =
```

PYTHON

```
self.popularity_recommendations(num_recommendations)
```

```
# Add strategy metadata for tracking
for rec in recommendations:
    rec['strategy_used'] = strategy
    rec['exploration_factor'] = exploration_factor
    rec['recommendation_id'] =
```

```
self.generate_recommendation_id()
```

```
return recommendations
```

PYTHON

```
def record_recommendation_feedback(self, recommendation_id: str,
                                   strategy: str, feedback: Dict):
    """Record user feedback to update bandit strategy
```

```
performance"""
```

```
# Calculate reward based on user interaction
reward = self.calculate_reward(feedback)
```

PYTHON

```
# Update strategy statistics
self.strategy_stats[strategy]['pulls'] += 1
self.strategy_stats[strategy]['rewards'] += reward
```

```
# Update confidence
stats = self.strategy_stats[strategy]
stats['confidence'] = stats['rewards'] / max(stats['pulls'],
```

1)

```
def calculate_reward(self, feedback: Dict) -> float:
    """Calculate reward signal from user feedback"""

    reward = 0.0

    # Different feedback types with different weights
    if feedback.get('completed_track', False):
        reward += 1.0

    if feedback.get('liked_track', False):
        reward += 2.0

    if feedback.get('saved_track', False):
        reward += 3.0

    if feedback.get('added_to_playlist', False):
        reward += 2.5

    if feedback.get('shared_track', False):
        reward += 4.0 # Sharing is strong positive signal

    # Negative signals
    if feedback.get('skipped_quickly', False): # Skipped within
```

30 seconds

```

reward -= 1.0

    if feedback.get('thumbs_down', False):
        reward -= 2.0

    # Time-based signals
    listen_duration = feedback.get('listen_duration_ms', 0)
    track_duration = feedback.get('track_duration_ms', 1)
    completion_ratio = listen_duration / track_duration

    if completion_ratio > 0.8:
        reward += 1.5
    elif completion_ratio > 0.5:
        reward += 0.5
    elif completion_ratio < 0.1:
        reward -= 0.5

    return reward

def collaborative_filtering_recommendations(self, user_id: str,
                                           user_profile:

```

UserTasteProfile,

```
num_recommendations:
```

int) -> List[Dict]:

```
"""Generate recommendations based on collaborative
```

filtering"""

```

# Find similar users based on taste profile
similar_users = self.find_similar_users(user_profile,

```

```
top_k=100)
```



```
recommendations = []
seen_tracks = set()

for similar_user_id, similarity_score in similar_users:
    # Get tracks this similar user liked but target user
```

hasn't heard

```
similar_user_tracks =
```

```
self.get_user_liked_tracks(similar_user_id)
```

```
user_tracks = self.get_user_track_history(user_id)

for track_id, track_score in similar_user_tracks:
    if track_id not in user_tracks and track_id not in
```

PYTHON

seen_tracks:

```
recommendation_score = similarity_score *
```

track_score

```
recommendations.append({
    'track_id': track_id,
    'score': recommendation_score,
    'reason': f'Users with similar taste enjoyed
```

this',

```

'similar_user_count': 1
    })

    seen_tracks.add(track_id)

    if len(recommendations) >= num_recommendations:
        break

    if len(recommendations) >= num_recommendations:
        break

    # Sort by score and return top recommendations
    recommendations.sort(key=lambda x: x['score'], reverse=True)
    return recommendations[:num_recommendations]

def genre_exploration_recommendations(self, user_profile:

```

UserTasteProfile,

```

exploration_factor: float,
num_recommendations: int) ->

```

List[Dict]:

```

"""Recommend tracks from adjacent/new genres"""

# Get user's current top genres
top_genres = sorted(
    user_profile.genre_weights.items(),
    key=lambda x: x[1],
    reverse=True
)[:5]

recommendations = []

# Mix of familiar and exploratory genres
familiar_ratio = 1.0 - exploration_factor
exploration_count = int(num_recommendations *

```

exploration_factor)

PYTHON

```
familiar_count = num_recommendations - exploration_count

# Familiar genre recommendations
for genre, weight in top_genres:
    genre_tracks = self.get_top_tracks_in_genre(genre,
```

```
limit=familiar_count // len(top_genres))
```

PYTHON

```
for track in genre_tracks:
    recommendations.append({
        'track_id': track['track_id'],
        'score': weight * track['popularity_score'],
        'reason': f'Popular in {genre}',
        'genre': genre,
        'exploration_type': 'familiar'
    })

# Exploration recommendations - adjacent genres
for genre, weight in top_genres:
    adjacent_genres = self.get_adjacent_genres(genre)

    for adj_genre in adjacent_genres[:2]:        # Top 2 adjacent
```

genres

PYTHON

```
exploration_tracks = self.get_top_tracks_in_genre(
    adj_genre,
    limit=exploration_count // (len(top_genres) * 2)
)

for track in exploration_tracks:
    recommendations.append({
        'track_id': track['track_id'],
        'score': weight * 0.5 *
```

```
track['popularity_score'], # Lower score for exploration
```

PYTHON

```
'reason': f'New genre: {adj_genre}',
        'genre': adj_genre,
        'exploration_type': 'adjacent_genre'
    })

# Sort by score and return
recommendations.sort(key=lambda x: x['score'], reverse=True)
return recommendations[:num_recommendations]
```

Discover Weekly Generation

Weekly Playlist Creation at Scale

Every Monday, Spotify generates personalized Discover Weekly playlists for 400M+ users, delivering 2+ billion song recommendations.

PYTHON

```
from datetime import datetime, timedelta
import asyncio

class DiscoverWeeklyGenerator:
    def __init__(self):
        self.feature_store = SpotifyFeatureStore()
        self.bandit = SpotifyMultiArmedBandit()
        self.content_filters = ContentFilters()
```

Discover Weekly specific parameters:

PYTHON

```
# Discover Weekly specific parameters
self.playlist_size = 30 # 30 songs per playlist
self.freshness_requirement = timedelta(days=90) # No songs
```

from last 90 days

PYTHON

```
self.diversity_requirement = 0.7 # 70% diversity in audio
```

features

```
async def generate_discover_weekly_for_user(self, user_id: str)
```

PYTHON

-> Dict:

```
"""Generate Discover Weekly playlist for a single user"""  
  
# Get user profile and listening history  
user_profile = await
```

```
self.feature_store.get_user_profile(user_id)
```

```
listening_history = await
```

```
self.get_user_listening_history(user_id, days=365)
```

```
# Check if user is eligible (needs sufficient listening
```

history)

```
if not
```

```
self.is_eligible_for_discover_weekly(listening_history):
```

```
return await self.generate_onboarding_playlist(user_id)
```

SQL

```
# Generate candidate recommendations from multiple
```

strategies

```
candidates = await self.generate_candidate_pool(user_id,
```

PYTHON

```
user_profile)
```

```
# Apply content filters
filtered_candidates =
```

```
self.content_filters.apply_discover_weekly_filters(
```

```
    candidates, user_profile, listening_history
)

# Optimize playlist for diversity and flow
optimized_playlist = self.optimize_playlist_composition(
    filtered_candidates, user_profile
)

# Final quality assurance
final_playlist =
```

PYTHON

```
self.quality_assurance_check(optimized_playlist, user_profile)
```

```
return {
    'user_id': user_id,
    'playlist_id':
```

```
f"discover_weekly_{user_id}_{datetime.now().strftime('%Y_%W')}",
```

```
'tracks': final_playlist,
'generation_timestamp': datetime.now(),
'personalization_features':
```

```
self.extract_personalization_features(user_profile),
```

```
'diversity_score':
```

```
self.calculate_diversity_score(final_playlist),
```

```
'freshness_score':
```

```
self.calculate_freshness_score(final_playlist, listening_history)
```

```
}
```

PYTHON

```
async def generate_candidate_pool(self, user_id: str,
```

```
user_profile: UserTasteProfile) -> List[Dict]:
```

```
"""Generate candidate tracks from multiple recommendation
```

SQL

```
strategies"""
```

```
# Define strategy allocation for Discover Weekly
```

SQL

```
strategy_allocation = {
```

```
    'collaborative_filtering': 40,    # 40% from similar
```

```
users
```

```
'artist_discovery': 25,    # 25% new artists
```

```
similar to liked ones
```

```
'genre_exploration': 20,    # 20% adjacent genres
```

```
'audio_feature_similarity': 10,    # 10% similar audio
```

```
features
```

```
'trending_underground': 5    # 5% trending but not
```

```
mainstream
```

SQL

```
}

all_candidates = []

# Generate candidates from each strategy
for strategy, percentage in strategy_allocation.items():
    num_candidates = int((percentage / 100) * 150) #
```

Generate 5x needed for filtering

```
if strategy == 'collaborative_filtering':
    candidates = await
```

```
self.collaborative_filtering_candidates(user_id, user_profile, num_candidates)
```

PYTHON

```
elif strategy == 'artist_discovery':
    candidates = await
```

```
self.artist_discovery_candidates(user_profile, num_candidates)
```

PYTHON

```
elif strategy == 'genre_exploration':
    candidates = await
```

```
self.genre_exploration_candidates(user_profile, num_candidates)
```

PYTHON

```
elif strategy == 'audio_feature_similarity':
    candidates = await
```

```
self.audio_similarity_candidates(user_profile, num_candidates)
```

PYTHON

```
elif strategy == 'trending_underground':
    candidates = await
```

```
self.trending_underground_candidates(user_profile, num_candidates)
```


PYTHON

```
# Add strategy metadata
    for candidate in candidates:
        candidate['source_strategy'] = strategy
        candidate['strategy_weight'] = percentage / 100

    all_candidates.extend(candidates)

    return all_candidates

def optimize_playlist_composition(self, candidates: List[Dict],
                                user_profile: UserTasteProfile)
```

-> List[Dict]:

```
"""Optimize playlist for diversity, flow, and user
```

preferences"""

PYTHON

```
# Sort candidates by relevance score
candidates.sort(key=lambda x: x['relevance_score'],
```

```
reverse=True)
```

PYTHON

```
playlist = []
used_artists = set()
used_genres = set()

# Audio feature running averages for diversity
running_features = {
    'energy': [],
    'valence': [],
    'danceability': [],
    'acousticness': []
}

for candidate in candidates:
    if len(playlist) >= self.playlist_size:
        break

    track_id = candidate['track_id']
    track_features =
```

```
self.feature_store.get_track_features(track_id)
```

```
track_metadata = self.get_track_metadata(track_id)
```

PYTHON

```
# Check diversity constraints
if not self.meets_diversity_requirements(
    track_features, track_metadata, playlist,
    used_artists, used_genres, running_features
```

```
):
    continue

# Add to playlist
playlist.append(candidate)
used_artists.add(track_metadata['artist_id'])
used_genres.update(track_metadata.get('genres', []))

# Update running averages
for feature in running_features:
```

```
running_features[feature].append(getattr(track_features, feature))
```

```
# If we don't have enough tracks, relax constraints and try
```

again

```
if len(playlist) < self.playlist_size:
    remaining_needed = self.playlist_size - len(playlist)
    relaxed_candidates = [c for c in candidates if c not in
```

PYTHON

playlist][:remaining_needed]

```
playlist.extend(relaxed_candidates)
```

PYTHON

```
# Optimize track ordering for flow
optimized_playlist = self.optimize_track_ordering(playlist,
```

```
user_profile)
```

```
return optimized_playlist
```

```
def meets_diversity_requirements(self, track_features:
```

AudioFeatures,

```
    track_metadata: Dict,
```

current_playlist: List[Dict],

```
    used_artists: set, used_genres:
```

set,

```
    running_features: Dict) -> bool:
        """Check if track meets diversity requirements"""

        # Artist diversity - no more than 2 songs per artist
        artist_id = track_metadata['artist_id']
        artist_count = sum(1 for track in current_playlist
                           if
```

```
        self.get_track_metadata(track['track_id'])['artist_id'] == artist_id)
```

```
        if artist_count >= 2:
            return False

        # Genre diversity - maintain genre balance
        track_genres = set(track_metadata.get('genres', []))
        if len(current_playlist) >= 10: # After 10 tracks, check
```

genre diversity

```
        genre_overlap =
```

```
        len(track_genres.intersection(used_genres))
```

```

if genre_overlap >= 2: # Too much genre overlap
    return False

# Audio feature diversity
if len(current_playlist) >= 5: # After 5 tracks, check

```

audio diversity

```

current_energy_avg = np.mean(running_features['energy'])

```

[-5:]) # Last 5 tracks

```

energy_diff = abs(track_features.energy -

```

current_energy_avg)

```

if energy_diff < 0.1: # Too similar energy levels
    return False

    return True

def optimize_track_ordering(self, playlist: List[Dict],
                             user_profile: UserTasteProfile) ->

```

PYTHON

List[Dict]:

```

"""Optimize the order of tracks for better listening flow"""

# Get track features for all tracks

```

```

tracks_with_features = []
for track in playlist:
    features =

```

```

self.feature_store.get_track_features(track['track_id'])

```

```

track['audio_features'] = features
    tracks_with_features.append(track)

# Strategy: Start with familiar, gradually introduce new
familiar_tracks = [t for t in tracks_with_features if

```

```

t['relevance_score'] > 0.7]

```

```

discovery_tracks = [t for t in tracks_with_features if

```

```

t['relevance_score'] <= 0.7]

```

```

ordered_playlist = []

# Start with a hook - familiar track that user will likely

```

enjoy

```

if familiar_tracks:
    best_opener = max(familiar_tracks, key=lambda x:

```

PYTHON

```

x['relevance_score'])

```

```

ordered_playlist.append(best_opener)
    familiar_tracks.remove(best_opener)

# Alternate between familiar and discovery, with gradual

```

energy progression

```

remaining_tracks = familiar_tracks + discovery_tracks

while remaining_tracks and len(ordered_playlist) <

```

self.playlist_size:

```
last_track = ordered_playlist[-1] if ordered_playlist
```

else None

```
if last_track:
    # Find track with complementary energy progression
    target_energy = self.calculate_target_energy(
        last_track['audio_features'].energy,
        len(ordered_playlist),
        self.playlist_size
    )

    best_next = min(
        remaining_tracks,
        key=lambda t: abs(t['audio_features'].energy -
```

PYTHON

target_energy)

```
)
else:
    # First track - choose highest relevance
    best_next = max(remaining_tracks, key=lambda x:
```

PYTHON

```
x['relevance_score'])
```

```
ordered_playlist.append(best_next)
    remaining_tracks.remove(best_next)

    return ordered_playlist

def calculate_target_energy(self, current_energy: float,
```

PYTHON

position: int, total_tracks: int) -> float:

```
"""Calculate target energy for next track to maintain good
```

```
flow"""
```

```
# Energy progression strategy: Start moderate, peak in
```

middle, wind down

```
progression_factor = position / total_tracks

if progression_factor < 0.3:    # First third - gradual energy
```

increase

```
target = current_energy + 0.1
elif progression_factor < 0.7: # Middle - maintain high
```

PYTHON

energy

```
target = max(0.6, current_energy)
else: # Last third - gradual energy decrease
    target = current_energy - 0.1
```

The PYTHON code:

```
return np.clip(target, 0.0, 1.0)

async def generate_all_discover_weekly_playlists(self) -> Dict:
    """Generate Discover Weekly for all eligible users"""

    # This runs every Monday morning for 400M+ users
    start_time = datetime.now()

    # Get list of all active users
    active_users = await self.get_active_users(days=30)    # Users
```

PYTHON

active in last 30 days

```
eligible_users = [user for user in active_users if
```

```
self.is_eligible_for_discover_weekly_cache(user)]
```

```
print(f"Generating Discover Weekly for {len(eligible_users)}
```

PYTHON

```
users")
```

```
# Process in batches to manage load
batch_size = 10000
total_batches = len(eligible_users) // batch_size + 1

results = {
    'total_users': len(eligible_users),
    'successful_generations': 0,
    'failed_generations': 0,
    'total_processing_time': 0
}

for batch_num in range(total_batches):
    batch_start = batch_num * batch_size
    batch_end = min((batch_num + 1) * batch_size,
```

```
len(eligible_users))
```

```
batch_users = eligible_users[batch_start:batch_end]

# Process batch concurrently
batch_tasks = [
    self.generate_discover_weekly_for_user(user_id)
    for user_id in batch_users
]

batch_results = await asyncio.gather(*batch_tasks,
```

PYTHON

```
return_exceptions=True)
```



```
# Count successes and failures
for result in batch_results:
    if isinstance(result, Exception):
        results['failed_generations'] += 1
    else:
        results['successful_generations'] += 1

    print(f"Batch {batch_num + 1}/{total_batches} complete")

total_time = (datetime.now() - start_time).total_seconds()
results['total_processing_time'] = total_time

print(f"Discover Weekly generation complete in
```

```
{total_time:.0f} seconds")
```

```
print(f"Success rate:
```

```
{results['successful_generations']/results['total_users']*100:.1f}%")
```

```
return results

# Example of how Spotify might track Discover Weekly performance
DISCOVER_WEEKLY_METRICS = {
    'playlist_generation': {
        'playlists_generated': 400_000_000,    # 400M playlists
        'average_generation_time': 2.5,        # 2.5 seconds per
```

playlist

```
'success_rate': 0.999,                # 99.9% success rate
'total_processing_time': 18_000        # 5 hours total
```

```
},
'user_engagement': {
    'playlists_opened': 250_000_000,    # 62.5% open rate
    'average_tracks_played': 12.3,      # Out of 30 tracks
    'completion_rate': 0.41,           # 41% listen to full
```

playlist

```
'save_rate': 0.15,           # 15% save tracks
  'like_rate': 0.08          # 8% like tracks
},
'content_distribution': {
  'total_unique_tracks': 50_000_000, # 50M unique tracks
```

recommended

```
'new_release_percentage': 0.25, # 25% tracks from
```

SQL

last 30 days

```
'emerging_artist_percentage': 0.40, # 40% tracks from
```

SQL

emerging artists

```
'average_track_age_days': 180      # 6 months average age
  }
}
```

Pro Tip Spotify's magic isn't just in their algorithms – it's in

their data culture. Every feature, from Discover Weekly to Daily

Mix, is treated as an experiment. They constantly test different approaches, measure user engagement, and iterate. The data platform enables this by making it easy to test new recommendation strategies and measure their impact on user satisfaction.

□ Checkpoint

1. How does Spotify's real-time feature engineering enable personalized recommendations at scale?
2. What role do multi-armed bandits play in balancing music discovery vs. satisfaction?
3. How does Discover Weekly generation handle the challenge of creating diverse yet cohesive playlists for 400M+ users?

Airbnb Search & Discovery Platform

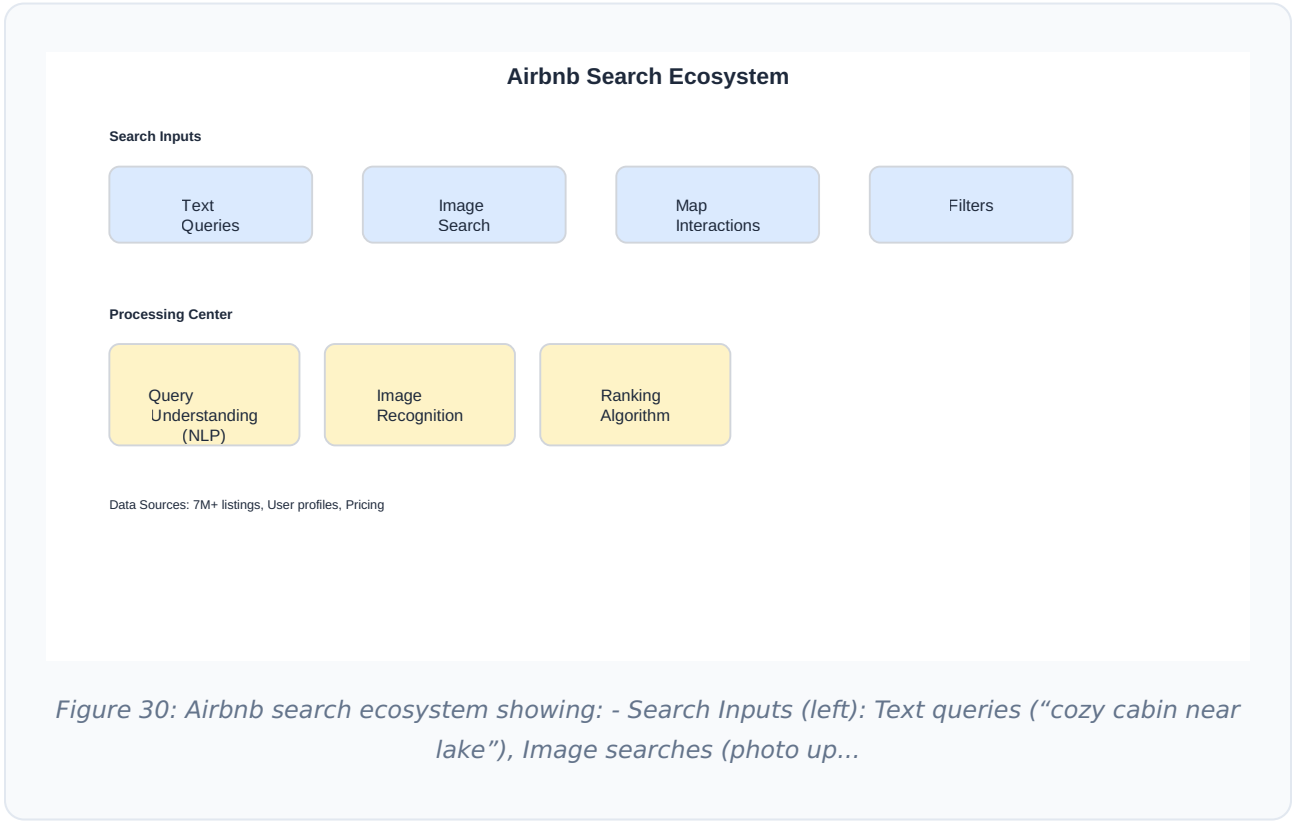
TL;DR

Airbnb processes 500M+ search queries yearly with real-time personalization and dynamic pricing - Built on a modern data stack with Kafka, Airflow, Spark, and custom ML serving infrastructure - Key innovation: Multi-modal search combining text, images, location, and user preferences - Challenge solved: Matching travelers with accommodations from 7M+ listings globally

Airbnb's search and discovery platform is one of the most sophisticated matching systems ever built. They don't just find you a place to stay — they predict what kind of experience you're looking for based on your search patterns, booking history, and even the time you're searching.

With 7+ million listings across 220+ countries, the challenge isn't just finding relevant results — it's understanding the intent behind each search and personalizing results for the trip.

The Airbnb Search Data Challenge



Airbnb's Search Scale:

```
# Airbnb's daily search and booking volume
AIRBNB_SCALE = {
    'search_queries': {
        'daily_searches': 1_400_000,          # 1.4M searches per
day
        'peak_searches_per_second': 500,      # During peak booking
hours
        'average_search_latency_ms': 150,      # Sub-200ms
```

requirement

```
'search_filters_used': 5_000_000,    # Filter interactions
```

daily

```
'map_interactions': 2_000_000        # Map zoom/pan events
},
'content_data': {
  'active_listings': 7_000_000,       # 7M active listings
  'listing_updates_daily': 100_000,   # Hosts updating
```

listings

```
'new_photos_uploaded': 50_000,       # New photos daily
'pricing_updates': 500_000,          # Dynamic pricing
```

changes

```
'availability_updates': 2_000_000    # Calendar updates
},
'bookings_and_users': {
  'daily_bookings': 50_000,           # 50K reservations
```

daily

```
'user_profiles_updated': 200_000,    # Profile changes
'reviews_submitted': 15_000,         # Guest reviews
'host_communications': 100_000      # Messages between
```

hosts/guests

```
}
}
```

Real-Time Search Query Understanding

Multi-Modal Query Processing

Airbnb's search handles text queries, image uploads, location searches, and filter combinations simultaneously.

```
import re
import numpy as np
from typing import Dict, List, Optional, Tuple
from dataclasses import dataclass
import nltk
from transformers import AutoTokenizer, AutoModel
import torch

@dataclass
class SearchQuery:
    """Structured representation of user search query"""
    raw_query: str
    location: Optional[Dict] = None
    dates: Optional[Dict] = None
    guests: Optional[int] = None
    price_range: Optional[Tuple[int, int]] = None
    property_types: List[str] = None
    amenities: List[str] = None
    extracted_intent: Dict = None
    images: List[str] = None # Uploaded image URLs

@dataclass
class SearchContext:
    """User context for personalized search"""
    user_id: str
    session_id: str
    device_type: str
    location: Dict # User's current location
    search_history: List[SearchQuery]
    booking_history: List[Dict]
    preferences: Dict
```

PYTHON

The PYTHON code:

```
market_segment: str    # business, leisure, family, etc.
```

PYTHON

```
class AirbnbQueryProcessor:
    def __init__(self):
        # NLP models for query understanding
        self.query_encoder = AutoModel.from_pretrained('sentence-
```

transformers/all-MiniLM-L6-v2')

```
self.tokenizer = AutoTokenizer.from_pretrained('sentence-
```

SQL

transformers/all-MiniLM-L6-v2')

```

# Intent classification model
self.intent_classifier = self.load_intent_classifier()

# Image processing for visual search
self.image_encoder = self.load_image_encoder()

# Location processing
self.location_processor = LocationProcessor()

# Named entity recognition for locations, dates, etc.
self.ner_model = self.load_ner_model()

# Common travel intent patterns
self.intent_patterns = {
    'business_travel': [
        r'\b(business|work|conference|meeting|corporate)\b',
        r'\b(downtown|city center|business district)\b'
    ],
    'family_vacation': [
        r'\b(family|kids|children|baby|families)\b',
        r'\b(pool|playground|family-friendly)\b'
    ],
    'romantic_getaway': [
        r'\b(romantic|honeymoon|anniversary|couples)\b',
        r'\b(secluded|private|intimate|cozy)\b'
    ],
    'group_travel': [
        r'\b(group|friends|bachelor|bachelorette)\b',
        r'\b(large|spacious|multiple bedrooms)\b'
    ],
    'pet_travel': [
        r'\b(pet|dog|cat|pet-friendly|pets allowed)\b'
    ],
    'luxury_travel': [
        r'\b(luxury|upscale|premium|high-end|5-star)\b',
        r'\b(mansion|villa|penthouse|luxury)\b'
    ]
}

def process_search_query(self, raw_query: str, context:

```

SearchContext) -> SearchQuery:


```
"""Process and understand user search query"""

# Parse structured elements (dates, location, guests)
structured_elements =
```

```
self.extract_structured_elements(raw_query)
```

```
# Extract location intent
location_info = self.extract_location_intent(raw_query,
```

PYTHON

context)

```
# Classify search intent
intent_classification =
```

```
self.classify_search_intent(raw_query, context)
```

```
# Extract amenities and property type preferences
amenities = self.extract_amenities(raw_query)
property_types = self.extract_property_types(raw_query)

# Handle image-based search if images provided
image_features = None
if context.search_history and
```

PYTHON

context.search_history[-1].images:

```
image_features =
```

```
self.process_search_images(context.search_history[-1].images)
```

```
return SearchQuery(  
    raw_query=raw_query,  
    location=location_info,  
    dates=structured_elements.get('dates'),  
    guests=structured_elements.get('guests'),  
    price_range=structured_elements.get('price_range'),  
    property_types=property_types,  
    amenities=amenities,  
    extracted_intent={  
        'primary_intent': intent_classification['primary'],  
        'confidence': intent_classification['confidence'],  
        'travel_purpose':
```

```
intent_classification['travel_purpose'],
```

```
'accommodation_preferences':
```

```
intent_classification['preferences'],
```

```

'image_similarity_vector': image_features
    }
)

def extract_structured_elements(self, query: str) -> Dict:
    """Extract dates, guest count, price from query text"""

    structured = {}

    # Extract guest count
    guest_patterns = [
        r'(\d+)\s*(guests?|people|persons?|adults?)',
        r'for\s+(\d+)',
        r'(\d+)\s*pax'
    ]

    for pattern in guest_patterns:
        match = re.search(pattern, query, re.IGNORECASE)
        if match:
            structured['guests'] = int(match.group(1))
            break

    # Extract price range
    price_patterns = [
        r'\$(\d+)\s*-\s*\$?(\d+)',
        r'under\s+\$?(\d+)',
        r'budget\s+\$?(\d+)',
        r'max\s+\$?(\d+)'
    ]

    for pattern in price_patterns:
        match = re.search(pattern, query, re.IGNORECASE)
        if match:
            if len(match.groups()) == 2:
                structured['price_range'] =

```

```

(int(match.group(1)), int(match.group(2)))

```

```

else:
    structured['price_range'] = (0,

```

```

int(match.group(1)))

```

```
break

# Extract dates (simplified - in reality would use more
```

sophisticated date parsing)

```
date_patterns = [
```

```
r'(january|february|march|april|may|june|july|august|september|october|november|
december)'
```

```
    r'\d{1,2}/\d{1,2}/\d{4}',
    r'(next week|next month|weekend|christmas|new year)'
]

for pattern in date_patterns:
    if re.search(pattern, query, re.IGNORECASE):
```

```
# Would use proper date parsing library in practice
structured['dates'] = {'has_dates': True, 'raw':
```

```
match.group(0) if 'match' in locals() else ''}
```

```
break

    return structured

def extract_location_intent(self, query: str, context:
```

PYTHON

SearchContext) -> Dict:

```

"""Extract and understand location intent from query"""

# Use NER to extract location entities
location_entities = self.ner_model.extract_locations(query)

# Handle different types of location queries
location_info = {
    'explicit_locations': location_entities,
    'location_type': None,
    'proximity_intent': None,
    'geographic_scope': None
}

# Classify location intent
if re.search(r'\b(near|close to|walking distance|nearby)\b',

```

query, re.IGNORECASE):

```

location_info['proximity_intent'] = 'near'

# Extract what they want to be near
proximity_patterns = [
    r'near\s+([\^,]+)',
    r'close to\s+([\^,]+)',
    r'walking distance to\s+([\^,]+)'
]

for pattern in proximity_patterns:
    match = re.search(pattern, query, re.IGNORECASE)
    if match:
        location_info['proximity_target'] =

```

```

match.group(1).strip()

```

```

break

# Detect neighborhood vs city vs region intent
if any(word in query.lower() for word in ['downtown', 'city

```

center', 'old town', 'historic district']):

```
location_info['location_type'] = 'urban_center'
elif any(word in query.lower() for word in ['beach',
```

PYTHON

'mountains', 'countryside', 'rural']):

```
location_info['location_type'] = 'nature'
elif any(word in query.lower() for word in ['airport',
```

PYTHON

'train station', 'subway']):

```
location_info['location_type'] = 'transit_hub'

    return location_info

def classify_search_intent(self, query: str, context:
```

PYTHON

SearchContext) -> Dict:

```
"""Classify the user's travel intent"""

# Primary intent classification
intent_scores = {}

for intent, patterns in self.intent_patterns.items():
    score = 0
    for pattern in patterns:
        matches = len(re.findall(pattern, query,
```

PYTHON

```
re.IGNORECASE))
```

```
score += matches
    intent_scores[intent] = score

# Get primary intent
primary_intent = max(intent_scores, key=intent_scores.get)
```

```
if max(intent_scores.values()) > 0 else 'general'
```

```
confidence = intent_scores[primary_intent] /
```

```
max(sum(intent_scores.values()), 1)
```

```
# Infer travel purpose from context and query
travel_purpose = self.infer_travel_purpose(query, context,
```

SQL

primary_intent)

```
# Extract accommodation preferences
preferences = self.extract_accommodation_preferences(query,
```

PYTHON

primary_intent)

```
return {
    'primary': primary_intent,
    'confidence': confidence,
    'travel_purpose': travel_purpose,
    'preferences': preferences,
    'all_scores': intent_scores
}

def extract_amenities(self, query: str) -> List[str]:
    """Extract amenity preferences from query"""

    amenity_keywords = {
        'wifi': ['wifi', 'internet', 'wireless'],
        'parking': ['parking', 'garage', 'car'],
        'kitchen': ['kitchen', 'cooking', 'kitchenette'],
        'pool': ['pool', 'swimming'],
        'gym': ['gym', 'fitness', 'exercise'],
        'air_conditioning': ['ac', 'air conditioning',
```

SQL

'aircon'],

```
'washer': ['washer', 'laundry', 'washing machine'],
'pet_friendly': ['pets allowed', 'dog friendly', 'cat
```

friendly'],

```
'hot_tub': ['hot tub', 'jacuzzi', 'spa'],
'balcony': ['balcony', 'terrace', 'patio'],
'workspace': ['workspace', 'desk', 'office', 'work from
```

home']

```
}

    found_amenities = []
    query_lower = query.lower()

    for amenity, keywords in amenity_keywords.items():
        if any(keyword in query_lower for keyword in keywords):
            found_amenities.append(amenity)

    return found_amenities

def extract_property_types(self, query: str) -> List[str]:
    """Extract property type preferences"""

    property_type_keywords = {
        'apartment': ['apartment', 'flat', 'condo'],
        'house': ['house', 'home', 'villa'],
        'studio': ['studio', 'efficiency'],
        'loft': ['loft', 'penthouse'],
        'cabin': ['cabin', 'cottage', 'chalet'],
        'unique': ['unique', 'unusual', 'special', 'treehouse',
```

'boat', 'castle'],

```
'hotel_room': ['hotel', 'bnb', 'inn'],
    'shared_room': ['shared', 'hostel', 'dorm']
}

    found_types = []
    query_lower = query.lower()

    for prop_type, keywords in property_type_keywords.items():
        if any(keyword in query_lower for keyword in keywords):
            found_types.append(prop_type)

    return found_types
```

The process_search_images implementation:


```
def process_search_images(self, image_urls: List[str]) ->
```

PYTHON

np.ndarray:

```
"""Process uploaded images for visual similarity search"""

image_features = []

for image_url in image_urls:
    # Download and process image
    image = self.download_and_preprocess_image(image_url)

    # Extract visual features using pre-trained model
    features = self.image_encoder.encode_image(image)
    image_features.append(features)

# Aggregate multiple image features
if image_features:
    return np.mean(image_features, axis=0)
else:
    return None

class LocationProcessor:
    def __init__(self):
        # Geocoding service for converting location names to
```

PYTHON

coordinates

```
self.geocoder = self.setup_geocoder()

# Location hierarchy (city -> region -> country)
self.location_hierarchy = self.load_location_hierarchy()

def extract_locations(self, query: str) -> List[Dict]:
    """Extract and geocode location mentions in query"""

    # Use NLP to extract location entities
    # This would typically use a more sophisticated NER model
    locations = []

    # Simple regex patterns for common location formats
    patterns = [
        r'\b[A-Z][a-z]+(?:\s+[A-Z][a-z]+)*\s*,\s*[A-Z]{2}\b', #
```

PYTHON

City, State

```
r'\b[A-Z][a-z]+(?:\s+[A-Z][a-z]+)*\s*,\s*[A-Z][a-z]+\b',
```

City, Country

```
r'\b[A-Z][a-z]+\s+[A-Z][a-z]+\b' # City names
]

for pattern in patterns:
    matches = re.finditer(pattern, query)
    for match in matches:
        location_text = match.group(0)
        geocoded = self.geocoder.geocode(location_text)

        if geocoded:
            locations.append({
                'text': location_text,
                'latitude': geocoded['latitude'],
                'longitude': geocoded['longitude'],
                'city': geocoded.get('city'),
                'region': geocoded.get('region'),
                'country': geocoded.get('country'),
                'confidence': geocoded.get('confidence'),
```

PYTHON

```
0.8)
```

```
})
```

```
return locations
```

Personalized Ranking Algorithm

Multi-Signal Ranking for Search Results

Airbnb's ranking algorithm combines hundreds of signals to personalize search results for each user.

```
import pandas as pd
import numpy as np
from sklearn.ensemble import GradientBoostingRanker
from typing import List, Dict

class AirbnbPersonalizedRanking:
    def __init__(self):
        # Main ranking model
        self.ranking_model = self.load_ranking_model()

        # Specialized models for different contexts
        self.business_travel_model =
```

PYTHON

```
self.load_specialized_model('business')
```

```
self.family_travel_model =
```

PYTHON

```
self.load_specialized_model('family')
```

```
self.luxury_travel_model =
```

PYTHON

```
self.load_specialized_model('luxury')
```

```
# Feature engineering pipeline
self.feature_engineer = SearchFeatureEngineer()

# Diversity and exploration components
self.diversity_optimizer = DiversityOptimizer()

def rank_search_results(self, listings: List[Dict],
```

PYTHON

search_query: SearchQuery,

```
context: SearchContext) -> List[Dict]:
    """Rank listings for personalized search results"""

    # Extract features for each listing
    listing_features = self.extract_listing_features(listings,
```

PYTHON

search_query, context)

```
# Get base relevance scores
relevance_scores =
```

```
self.calculate_relevance_scores(listing_features, search_query, context)
```

```
# Apply personalization
personalized_scores =
```

```
self.apply_personalization(listing_features, context, relevance_scores)
```

```
# Optimize for diversity
final_scores = self.diversity_optimizer.optimize_results(
    listing_features, personalized_scores, context
)

# Sort and return ranked results
ranked_listings = sorted(
    zip(listings, final_scores),
    key=lambda x: x[1],
    reverse=True
)

# Add ranking metadata
for i, (listing, score) in enumerate(ranked_listings):
    listing['search_rank'] = i + 1
    listing['relevance_score'] = score
    listing['ranking_features'] =
```

PYTHON

```
self.explain_ranking(listing, listing_features[i])
```

```
return [listing for listing, _ in ranked_listings]

def extract_listing_features(self, listings: List[Dict],
```

PYTHON

search_query: SearchQuery,

```
context: SearchContext) ->
```

pd.DataFrame:

```
"""Extract comprehensive features for each listing"""
```

```
features = []
```

```
for listing in listings:
    listing_features = {
        # Basic listing attributes
        'listing_id': listing['id'],
        'property_type': listing['property_type'],
        'room_type': listing['room_type'],
        'accommodates': listing['accommodates'],
        'bedrooms': listing.get('bedrooms', 0),
        'bathrooms': listing.get('bathrooms', 0),
        'price_per_night': listing['price'],

        # Location features
        'latitude': listing['latitude'],
        'longitude': listing['longitude'],
        'neighborhood_score':
```

```
listing.get('neighborhood_score', 0.5),
```

```
    # Quality signals
    'overall_rating': listing.get('overall_rating', 0),
    'review_count': listing.get('review_count', 0),
    'superhost': listing.get('superhost', False),
    'instant_book': listing.get('instant_book', False),

    # Availability and booking signals
    'availability_365': listing.get('availability_365',
```

```
0),
```

```
    'booking_rate': listing.get('booking_rate', 0),
    'response_rate': listing.get('host_response_rate',
```

0),

```
'acceptance_rate':
```

```
listing.get('host_acceptance_rate', 0),
```

```
}
```

PYTHON

```
    # Calculate derived features
    listing_features.update(self.calculate_derived_features(
        listing, search_query, context
    ))
```

```
    features.append(listing_features)
```

```
    return pd.DataFrame(features)
```

```
def calculate_derived_features(self, listing: Dict,
```

search_query: SearchQuery,

```
context: SearchContext) -> Dict:
    """Calculate derived features for ranking"""
```

```
    derived = {}
```

```
    # Distance features
```

```
    if search_query.location and listing.get('latitude') and
```

```
listing.get('longitude'):
```

```
    query_lat = search_query.location.get('latitude', 0)
    query_lon = search_query.location.get('longitude', 0)
```

PYTHON

```
    distance_km = self.calculate_distance(
        query_lat, query_lon, listing['latitude'],
```

```
listing['longitude']
```

```
)
derived['distance_km'] = distance_km
derived['distance_score'] = 1.0 / (1.0 + distance_km) #
```

Inverse distance

```
else:
    derived['distance_km'] = 0
    derived['distance_score'] = 1.0

# Price competitiveness
if search_query.price_range:
    min_price, max_price = search_query.price_range
    listing_price = listing['price']

    if listing_price <= max_price:
```

```
price_score = 1.0 - (listing_price - min_price) /
```

```
(max_price - min_price)
```

```
else:
    price_score = 0.0 # Over budget

    derived['price_competitiveness'] = max(0, price_score)
else:
    derived['price_competitiveness'] = 0.5 # Neutral

# Amenity match score
if search_query.amenities:
    listing_amenities = set(listing.get('amenities', []))
    requested_amenities = set(search_query.amenities)

    match_count =
```

```
len(listing_amenities.intersection(requested_amenities))
```

```
derived['amenity_match_score'] = match_count /
```

```
len(requested_amenities) if requested_amenities else 0
```

```
else:
    derived['amenity_match_score'] = 0

# Guest capacity fit
if search_query.guests:
    capacity_fit = min(1.0, listing['accommodates'] /
```

```
search_query.guests)
```

```
derived['capacity_fit_score'] = capacity_fit
else:
    derived['capacity_fit_score'] = 1.0

# Host quality score (composite)
host_signals = [
    listing.get('superhost', False),
    listing.get('response_rate', 0) / 100,
    listing.get('acceptance_rate', 0) / 100,
    min(1.0, listing.get('review_count', 0) / 50) #
```

Normalize by 50 reviews

```
]
derived['host_quality_score'] = np.mean([float(signal) for
```

```
signal in host_signals])
```


PYTHON

```
# Booking likelihood (simplified model)
booking_signals = [
    derived['distance_score'],
    derived['price_competitiveness'],
    derived['amenity_match_score'],
    listing.get('overall_rating', 0) / 5.0,
    derived['host_quality_score']
]
derived['booking_likelihood'] = np.mean(booking_signals)

return derived

def apply_personalization(self, features: pd.DataFrame, context:
```

SearchContext,

PYTHON

```
base_scores: np.ndarray) -> np.ndarray:
    """Apply user-specific personalization to ranking scores"""

    personalization_factors = []

    for idx, row in features.iterrows():
        listing_id = row['listing_id']

        # Historical preference matching
        pref_score = self.calculate_preference_match(row,
```

context)

```
# Previous interaction signals
interaction_score =
```

```
self.calculate_interaction_signals(listing_id, context)
```

PYTHON

```
# Market segment specific scoring
segment_score = self.calculate_segment_score(row,
```

context)

```
# Combine personalization factors
total_personalization = (
    pref_score * 0.4 +
    interaction_score * 0.3 +
    segment_score * 0.3
)

personalization_factors.append(total_personalization)

# Apply personalization as a multiplier to base scores
personalization_array = np.array(personalization_factors)
personalized_scores = base_scores * (1.0 +
```

personalization_array)

```
return personalized_scores

def calculate_preference_match(self, listing_features:
```

PYTHON

pd.Series, context: SearchContext) -> float:

```
"""Calculate how well listing matches user's historical
```

preferences"""

```
if not context.booking_history:
    return 0.0 # No personalization for new users

preference_score = 0.0

# Property type preference
booked_property_types = [booking.get('property_type') for
```

```
booking in context.booking_history]
```

```
if listing_features['property_type'] in
```

booked_property_types:

```

    preference_score += 0.3

    # Price range preference
    booked_prices = [booking.get('price_per_night', 0) for

```

```

    booking in context.booking_history]

```

```

    if booked_prices:
        avg_booked_price = np.mean(booked_prices)
        price_deviation =

```

```

abs(listing_features['price_per_night'] - avg_booked_price) / avg_booked_price

```

```

    preference_score += max(0, 0.3 * (1.0 -

```

```

    price_deviation))

```

```

    # Location type preference
    booked_locations = [booking.get('neighborhood_score', 0.5)

```

```

    for booking in context.booking_history]

```

```

    if booked_locations:
        avg_neighborhood_pref = np.mean(booked_locations)
        location_similarity = 1.0 -

```

```

abs(listing_features['neighborhood_score'] - avg_neighborhood_pref)

```

```

    preference_score += 0.2 * location_similarity

    # Host quality preference
    booked_host_quality = [self.extract_host_quality(booking)

```

PYTHON

```

    for booking in context.booking_history]

```

```
if booked_host_quality:
    avg_host_quality = np.mean(booked_host_quality)
    quality_similarity = 1.0 -
```

```
abs(listing_features['host_quality_score'] - avg_host_quality)
```

```
preference_score += 0.2 * quality_similarity

    return min(1.0, preference_score)

def calculate_interaction_signals(self, listing_id: str,
```

PYTHON

context: SearchContext) -> float:

```
"""Calculate score based on previous user interactions with
```

this listing"""

```
interaction_score = 0.0
```

Check if user previously viewed this listing:

```
# Check if user previously viewed this listing
if self.has_viewed_listing(context.user_id, listing_id):
    interaction_score += 0.2

# Check if user saved/wishlisted this listing
if self.has_saved_listing(context.user_id, listing_id):
    interaction_score += 0.5

# Check if similar users booked this listing
similar_users = self.get_similar_users(context.user_id,
```

PYTHON

```
top_k=100)
```

```

for user_id, similarity in similar_users:
    if self.has_booked_listing(user_id, listing_id):
        interaction_score += 0.3 * similarity
        break

    return min(1.0, interaction_score)

def calculate_segment_score(self, listing_features: pd.Series,

```

context: SearchContext) -> float:

```

"""Apply market segment specific scoring"""

segment = context.market_segment

if segment == 'business':
    # Business travelers prefer instant book, good wifi,

```

central location

```

score = (
    listing_features.get('instant_book', False) * 0.4 +
    listing_features.get('wifi_score', 0.5) * 0.3 +
    listing_features['distance_score'] * 0.3
)

elif segment == 'family':
    # Families prefer larger spaces, family amenities,

```

safety

```

score = (
    min(1.0, listing_features['accommodates'] / 4) * 0.4

```

+ # Space for 4+

```

listing_features.get('family_amenity_score', 0) *

```

```
listing_features.get('safety_score', 0.5) * 0.3
    )

    elif segment == 'luxury':
        # Luxury travelers prefer high ratings, unique
```

PYTHON

properties, premium hosts

```
score = (
    listing_features['overall_rating'] / 5.0 * 0.4 +
    listing_features.get('superhost', False) * 0.3 +
    listing_features.get('uniqueness_score', 0.5) * 0.3
)

else: # General segment
    score = 0.5 # Neutral score

return score

class DiversityOptimizer:
    def __init__(self):
        self.diversity_weight = 0.15    # 15% weight for diversity vs
```

PYTHON

relevance

```
def optimize_results(self, features: pd.DataFrame, scores:
```

PYTHON

```
np.ndarray,
```

```
context: SearchContext) -> np.ndarray:
    """Optimize results for diversity while maintaining
```

relevance"""

```
# Implement Maximal Marginal Relevance (MMR) for top results
top_n = 20 # Apply diversity to top 20 results
```

```
if len(scores) <= top_n:
    return scores

# Get top candidates
top_indices = np.argsort(scores)[-50:][::-1]    # Top 50
```

candidates

```

selected_indices = []
remaining_indices = list(top_indices)

# Select first item (highest relevance)
first_idx = remaining_indices[0]
selected_indices.append(first_idx)
remaining_indices.remove(first_idx)

# Iteratively select diverse items
while len(selected_indices) < top_n and remaining_indices:
    mmr_scores = []

    for candidate_idx in remaining_indices:
        relevance = scores[candidate_idx]

        # Calculate diversity from already selected items
        diversity = self.calculate_diversity(
            features.iloc[candidate_idx],
            features.iloc[selected_indices]
        )

        # MMR score
        mmr_score = (
            (1 - self.diversity_weight) * relevance +
            self.diversity_weight * diversity
        )
        mmr_scores.append((candidate_idx, mmr_score))

    # Select item with highest MMR score
    best_idx = max(mmr_scores, key=lambda x: x[1])[0]
    selected_indices.append(best_idx)
    remaining_indices.remove(best_idx)

# Create new scores preserving original order
optimized_scores = scores.copy()

# Boost selected diverse items
for i, idx in enumerate(selected_indices):
    boost_factor = 1.0 + (top_n - i) * 0.01    # Slight boost

```

based on position


```

optimized_scores[idx] *= boost_factor

    return optimized_scores

def calculate_diversity(self, candidate: pd.Series,

```

PYTHON

selected_items: pd.DataFrame) -> float:

```

"""Calculate diversity of candidate from already selected

```

SQL

items"""

```

if selected_items.empty:
    return 1.0

diversity_features = [
    'property_type', 'neighborhood_score',

```

SQL

'price_per_night',

```

'host_quality_score', 'overall_rating'
]

# Calculate average similarity to selected items
similarities = []

for _, selected_item in selected_items.iterrows():
    similarity = 0.0

```

SQL

```

for feature in diversity_features:
    if feature in candidate.index and feature in

```

selected_item.index:

```

if feature == 'property_type':
    # Categorical similarity
    similarity += 1.0 if candidate[feature] ==

```

```
selected_item[feature] else 0.0
```

```
else:  
    # Numerical similarity  
    if isinstance(candidate[feature], (int,
```

float)) and isinstance(selected_item[feature], (int, float)):

```
diff = abs(candidate[feature] -
```

```
selected_item[feature])
```

```
max_diff = max(candidate[feature],
```

```
selected_item[feature])
```

```
similarity += 1.0 - (diff /
```

```
max(max_diff, 1.0))
```

```
similarities.append(similarity /
```

```
len(diversity_features))
```

```
# Diversity is inverse of average similarity
avg_similarity = np.mean(similarities)
diversity = 1.0 - avg_similarity

return max(0.0, diversity)

# Example of how Airbnb might track search performance
AIRBNB_SEARCH_METRICS = {
    'search_performance': {
        'average_search_latency_ms': 145,
        'search_success_rate': 0.995,
        'results_clicked_per_search': 3.2,
        'searches_leading_to_booking': 0.08, # 8% conversion rate
        'zero_results_rate': 0.002 # 0.2% searches with no results
    },
    'personalization_impact': {
        'personalized_vs_generic_ctr': 1.35, # 35% higher click
```

rate

```
'personalized_booking_rate': 0.095,    # vs 0.07 generic
'user_satisfaction_score': 4.3,        # out of 5
'search_abandonment_rate': 0.15       # 15% abandon before
```

results

```
},
'content_coverage': {
    'listings_receiving_impressions': 5_500_000, # 78% of
```

active listings

```
'geographic_coverage': 0.92,          # 92% of
```

searchable areas

```
'price_range_coverage': 0.96,         # 96% of price
```

segments

'property_type_coverage': 0.94

94% of

property types

```
}  
}
```

SQL

Pro Tip Airbnb's search success comes from understanding that travel booking is highly contextual. The same user might want different things for business travel vs. family vacation vs. romantic getaway. Their platform uses dozens of signals to infer context and personalize results accordingly. The key insight: don't just match listings to queries – match experiences to intent.

□ Checkpoint

1. How does Airbnb's multi-modal query processing handle different types of search inputs (text, images, location)?
2. What role does personalization play in search ranking, and how do they balance it with relevance?
3. How does diversity optimization ensure users see varied options while maintaining search quality?

Twitter Real-Time Analytics

⚡ TL;DR

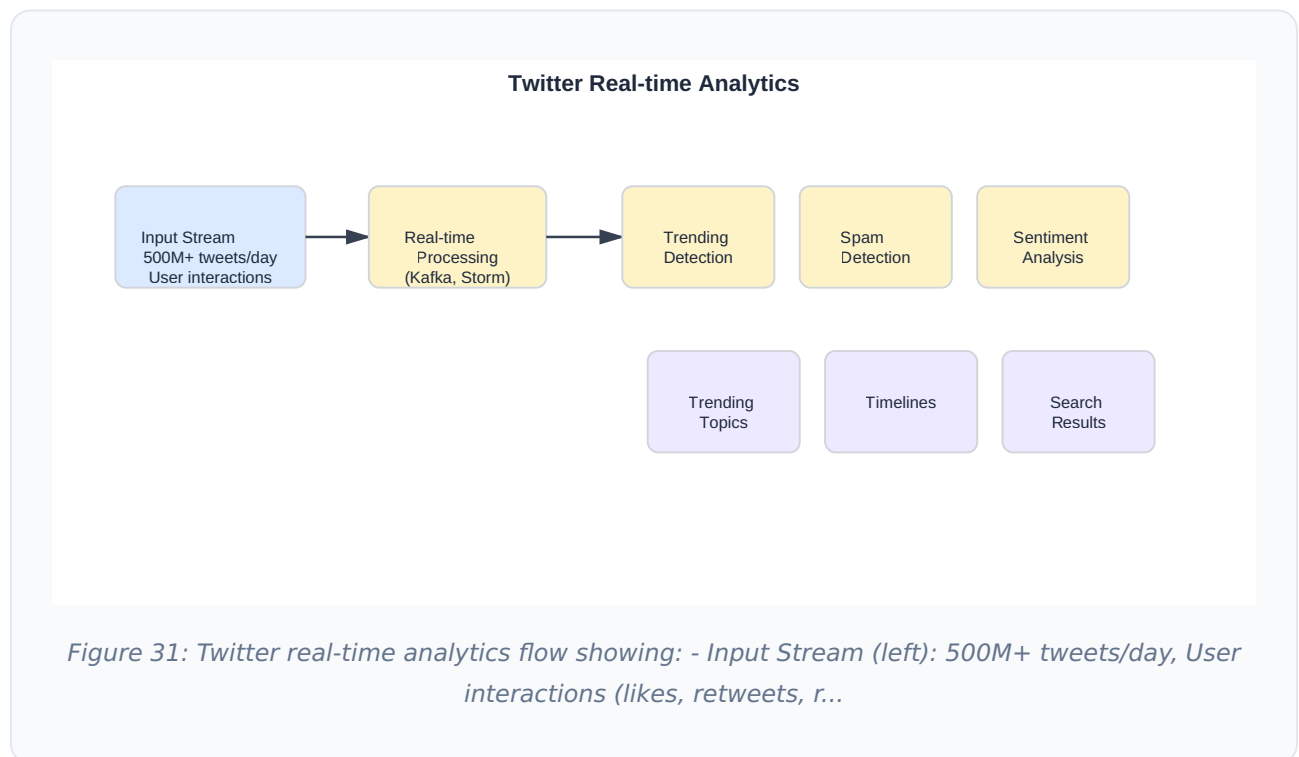
Twitter processes 500M+ tweets daily with real-time trending topic detection and content recommendation - Built on a hybrid architecture combining batch processing (Hadoop) and stream processing (Storm/Heron) - Key innovation: Real-time graph processing for social network analysis and viral content detection - Challenge: Handle massive spikes during breaking news while maintaining sub-second latencies

Twitter's real-time analytics platform is one of the most demanding data systems ever built. During major events, they can see 10x normal tweet volume within minutes, requiring their systems to scale instantly while detecting trending topics, fighting spam, and serving personalized timelines to 400M+ users worldwide.

What makes Twitter's architecture fascinating is how they balance

real-time responsiveness with the massive computational requirements of social graph analysis and content understanding.

The Twitter Real-Time Challenge



Twitter's Real-Time Scale:

```
# Twitter's daily processing volume
TWITTER_SCALE = {
    'content_volume': {
        'tweets_per_day': 500_000_000,          # 500M tweets
        'peak_tweets_per_second': 25_000,       # During major
```

events

```

'retweets_per_day': 200_000_000,      # 200M retweets
'likes_per_day': 1_000_000_000,      # 1B likes
'replies_per_day': 150_000_000      # 150M replies
},
'user_interactions': {
'timeline_requests': 50_000_000_000, # 50B timeline views
'search_queries': 2_000_000_000,    # 2B searches
'profile_views': 5_000_000_000,     # 5B profile visits
'notification_deliveries': 10_000_000_000 # 10B

```

notifications

```

},
'real_time_processing': {
'trending_topic_updates': 1440,      # Every minute

```

globally

```

'timeline_generation_requests': 200_000_000_000, # 200B/day
'spam_detection_checks': 600_000_000,    # All content

```

checked

```

'content_recommendations': 50_000_000_000 # 50B

```

recommendations

```

}
}

```

Real-Time Tweet Ingestion and Processing

High-Throughput Tweet Processing Pipeline

Twitter's ingestion system handles massive, unpredictable spikes while maintaining exactly-once processing guarantees.

```

import time
import json
import hashlib
from typing import Dict, List, Optional, Set
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from collections import defaultdict, deque

@dataclass
class Tweet:
    """Structured representation of a Tweet"""
    tweet_id: str
    user_id: str
    text: str
    created_at: datetime
    hashtags: List[str]
    mentions: List[str]
    urls: List[str]
    media: List[Dict]
    reply_to_tweet_id: Optional[str] = None
    reply_to_user_id: Optional[str] = None
    retweet_of_tweet_id: Optional[str] = None
    quote_tweet_id: Optional[str] = None
    language: str = 'en'
    location: Optional[Dict] = None
    engagement_metrics: Dict = field(default_factory=dict)

@dataclass
class TweetProcessingResult:
    """Result of tweet processing pipeline"""
    tweet: Tweet
    spam_score: float
    sentiment_score: float
    topics: List[str]
    entities: List[Dict]
    virality_score: float
    processing_latency_ms: float

class TwitterTweetProcessor:
    def __init__(self):
        # Real-time processing components
        self.spam_detector = SpamDetectionEngine()
        self.sentiment_analyzer = SentimentAnalyzer()
        self.topic_classifier = TopicClassifier()
        self.entity_extractor = EntityExtractor()
        self.virality_predictor = ViralityPredictor()

        # Content understanding models

```



```

self.language_detector = LanguageDetector()
self.hate_speech_detector = HateSpeechDetector()
self.misinformation_detector = MisinformationDetector()

# Real-time feature extractors
self.user_graph = UserSocialGraph()
self.trending_tracker = TrendingTopicsTracker()

# Processing pipeline configuration
self.pipeline_config = {
    'spam_threshold': 0.7,
    'sentiment_confidence_threshold': 0.6,
    'virality_threshold': 0.8,
    'processing_timeout_ms': 100, # Must complete within

```

100ms

```

    'max_retries': 2
    }

    def process_tweet_stream(self, tweet_data: Dict) ->

```

PYTHON

TweetProcessingResult:

```

"""Process a single tweet through the real-time pipeline"""

start_time = time.time()

```

Parse raw tweet data:

```

# Parse raw tweet data
tweet = self.parse_tweet_data(tweet_data)

# Parallel processing of different analyses
analysis_results = self.run_parallel_analysis(tweet)

# Combine results
result = TweetProcessingResult(
    tweet=tweet,
    spam_score=analysis_results['spam_score'],
    sentiment_score=analysis_results['sentiment_score'],
    topics=analysis_results['topics'],
    entities=analysis_results['entities'],
    virality_score=analysis_results['virality_score'],
    processing_latency_ms=(time.time() - start_time) * 1000
)

# Real-time actions based on analysis
self.take_real_time_actions(result)

return result

def parse_tweet_data(self, raw_data: Dict) -> Tweet:
    """Parse raw tweet JSON into structured Tweet object"""

    # Extract text and clean
    text = raw_data.get('text', '')
    full_text = raw_data.get('full_text', text)    # Use extended

```

tweet if available

```

# Extract hashtags
hashtags = []
entities = raw_data.get('entities', {})
for hashtag in entities.get('hashtags', []):
    hashtags.append(hashtag['text'].lower())

# Extract mentions
mentions = []
for mention in entities.get('user_mentions', []):
    mentions.append(mention['screen_name'].lower())

# Extract URLs
urls = []
for url in entities.get('urls', []):
    urls.append(url.get('expanded_url', url.get('url')))

# Extract media
media = []
for media_item in entities.get('media', []):
    media.append({
        'type': media_item.get('type'),
        'url': media_item.get('media_url_https'),
        'sizes': media_item.get('sizes', {})
    })

# Parse timestamp
created_at = datetime.strptime(
    raw_data['created_at'],
    '%a %b %d %H:%M:%S +0000 %Y'
)

return Tweet(
    tweet_id=raw_data['id_str'],
    user_id=raw_data['user']['id_str'],
    text=full_text,
    created_at=created_at,
    hashtags=hashtags,
    mentions=mentions,
    urls=urls,
    media=media,

```

```

reply_to_tweet_id=raw_data.get('in_reply_to_status_id_str'),
reply_to_user_id=raw_data.get('in_reply_to_user_id_str'),

```

```
retweet_of_tweet_id=raw_data.get('retweeted_status',
```

```
{}).get('id_str'),
```

```
quote_tweet_id=raw_data.get('quoted_status_id_str'),
    language=raw_data.get('lang', 'en'),
    location=self.extract_location(raw_data),
    engagement_metrics={
        'retweet_count': raw_data.get('retweet_count', 0),
        'favorite_count': raw_data.get('favorite_count', 0),
        'reply_count': raw_data.get('reply_count', 0)
    }
)
```

PYTHON

```
def run_parallel_analysis(self, tweet: Tweet) -> Dict:
    """Run multiple analysis pipelines in parallel"""

    # In a real system, these would run concurrently
    # Using threads or async processing for parallelization

    results = {}

    # Spam detection
    results['spam_score'] =
```

```
self.spam_detector.analyze_tweet(tweet)
```

```
# Sentiment analysis
sentiment_result =
```

```
self.sentiment_analyzer.analyze_sentiment(tweet.text)
```

```
results['sentiment_score'] = sentiment_result['score']
results['sentiment_confidence'] =
```

```
sentiment_result['confidence']
```

```
# Topic classification
results['topics'] =
```

```
self.topic_classifier.classify_tweet(tweet)
```

```
# Entity extraction
results['entities'] =
```

```
self.entity_extractor.extract_entities(tweet.text)
```

```
# Virality prediction
results['virality_score'] =
```

```
self.virality_predictor.predict_virality(tweet)
```

```
# Content safety checks
results['hate_speech_score'] =
```

```
self.hate_speech_detector.analyze(tweet.text)
```

```
results['misinformation_score'] =
```

```
self.misinformation_detector.analyze(tweet)
```

```
return results
```

PYTHON

```
def take_real_time_actions(self, result: TweetProcessingResult):
    """Take immediate actions based on tweet analysis"""

    tweet = result.tweet

    # Spam handling
    if result.spam_score >
```

```
self.pipeline_config['spam_threshold']:
```

```
self.handle_spam_tweet(tweet, result.spam_score)
```

PYTHON

```
# Trending topic updates
if result.topics and result.virality_score > 0.5:
    self.trending_tracker.update_trends(result.topics,
```

```
tweet.created_at)
```

```
# Content moderation
if result.hate_speech_score > 0.8 or
```

```
result.misinformation_score > 0.7:
```

```
self.escalate_for_content_review(tweet, {
```

PYTHON

```
'hate_speech_score': getattr(result,
```

```
'hate_speech_score', 0),
```

```
'misinformation_score': getattr(result,
```

```
'misinformation_score', 0)
```

```
})
```

```
# Real-time notifications
if result.virality_score >
```

```
self.pipeline_config['virality_threshold']:
```

```
self.send_viral_content_alerts(tweet,
```

PYTHON

```
result.virality_score)
```

```
# Update user graph
    if tweet.mentions or tweet.retweet_of_tweet_id:
        self.user_graph.update_interactions(tweet)

class TrendingTopicsTracker:
    def __init__(self):
        # Time window for trending analysis
        self.trending_window_minutes = 15

        # Topic frequency tracking
        self.topic_frequencies = defaultdict(lambda: deque())

        # Geographic trending
        self.geographic_trends = defaultdict(lambda:
```

```
defaultdict(lambda: deque()))
```

```
# Current trending topics
    self.current_trends = {
        'global': [],
        'by_location': defaultdict(list)
    }

def update_trends(self, topics: List[str], timestamp: datetime):
    """Update trending topic calculations in real-time"""

    current_time = timestamp
    cutoff_time = current_time -
```

```
timedelta(minutes=self.trending_window_minutes)
```

```

for topic in topics:
    # Clean old entries
    while (self.topic_frequencies[topic] and
           self.topic_frequencies[topic][0] < cutoff_time):
        self.topic_frequencies[topic].popleft()

    # Add new entry
    self.topic_frequencies[topic].append(current_time)

    # Recalculate trending topics every minute
    if self.should_recalculate_trends(current_time):
        self.recalculate_trending_topics(current_time)

def recalculate_trending_topics(self, current_time: datetime):
    """Recalculate what topics are currently trending"""

    cutoff_time = current_time -

```

```
timedelta(minutes=self.trending_window_minutes)
```

```

# Calculate frequency scores for all topics
topic_scores = {}
for topic, timestamps in self.topic_frequencies.items():
    # Count recent mentions
    recent_count = len([ts for ts in timestamps if ts >=

```

```
cutoff_time])
```

```

# Calculate trend momentum (recent vs. earlier)
if len(timestamps) >= 10: # Need minimum mentions
    half_window = self.trending_window_minutes // 2
    mid_time = current_time -

```

```
timedelta(minutes=half_window)
```

```
recent_half = len([ts for ts in timestamps if ts >=
```

```
mid_time])
```



```
earlier_half = len([ts for ts in timestamps if
```

```
cutoff_time <= ts < mid_time])
```

```
# Momentum score (growth rate)
momentum = (recent_half / max(earlier_half, 1)) if
```

```
earlier_half > 0 else recent_half
```

```
# Combined score: frequency + momentum
topic_scores[topic] = recent_count * momentum

# Select top trending topics
sorted_topics = sorted(topic_scores.items(), key=lambda x:
```

SQL

```
x[1], reverse=True)
```

```
self.current_trends['global'] = [
    {
        'topic': topic,
        'score': score,
        'tweet_volume': self.get_topic_tweet_count(topic,
```

PYTHON

```
cutoff_time),
```

```
'growth_rate': self.calculate_growth_rate(topic,
```

PYTHON

```
current_time)
```

```
}
    for topic, score in sorted_topics[:50] # Top 50 trends
    ]

def get_trending_topics(self, location: str = 'global', count:
```

PYTHON

```
int = 10) -> List[Dict]:
```

PYTHON

```

"""Get current trending topics for a location"""

if location == 'global':
    return self.current_trends['global'][:count]
else:
    return self.current_trends['by_location'].get(location,

```

```

[])[:count]

```

PYTHON

```

def calculate_growth_rate(self, topic: str, current_time:

```

```

datetime) -> float:

```

PYTHON

```

"""Calculate the growth rate for a topic"""

if topic not in self.topic_frequencies:
    return 0.0

timestamps = self.topic_frequencies[topic]
if len(timestamps) < 4: # Need minimum data points
    return 0.0

# Compare last 5 minutes vs previous 5 minutes
last_5_min = current_time - timedelta(minutes=5)
prev_5_min = current_time - timedelta(minutes=10)

recent_count = len([ts for ts in timestamps if ts >=

```

```

last_5_min])

```

```

previous_count = len([ts for ts in timestamps if prev_5_min

```

```

<= ts < last_5_min])

```

```

if previous_count == 0:
    return float('inf') if recent_count > 0 else 0.0

    return (recent_count - previous_count) / previous_count

class ViralityPredictor:
    def __init__(self):
        # Historical virality patterns
        self.virality_model = self.load_virality_model()

        # Real-time engagement tracking
        self.engagement_tracker = EngagementTracker()

        # Social graph for influence calculation

```

The PYTHON code:

```

self.social_graph = UserSocialGraph()

def predict_virality(self, tweet: Tweet) -> float:
    """Predict likelihood of tweet going viral"""

    # Extract features for virality prediction
    features = self.extract_virality_features(tweet)

    # Base virality score from ML model
    base_score = self.virality_model.predict_proba(features)

    # Real-time adjustments
    real_time_score = self.apply_real_time_adjustments(tweet,

```

base_score)

```

return min(1.0, real_time_score)

def extract_virality_features(self, tweet: Tweet) -> Dict:
    """Extract features for virality prediction"""

    # User features
    user_features =

```

```

self.social_graph.get_user_features(tweet.user_id)

```

```

# Content features
content_features = {
    'text_length': len(tweet.text),
    'has_media': len(tweet.media) > 0,
    'has_urls': len(tweet.urls) > 0,
    'hashtag_count': len(tweet.hashtags),
    'mention_count': len(tweet.mentions),
    'is_reply': tweet.reply_to_tweet_id is not None,
    'is_retweet': tweet.retweet_of_tweet_id is not None,
    'is_quote_tweet': tweet.quote_tweet_id is not None
}

# Temporal features
hour = tweet.created_at.hour
day_of_week = tweet.created_at.weekday()

temporal_features = {
    'hour_of_day': hour,
    'is_peak_hours': 18 <= hour <= 22, # Prime time
    'is_weekend': day_of_week >= 5,
    'is_business_hours': 9 <= hour <= 17 and day_of_week < 5
}

# Language and sentiment features
language_features = {
    'language': tweet.language,
    'text_sentiment':

```

```

self.get_cached_sentiment(tweet.tweet_id, tweet.text)

```

```

}

# Combine all features
all_features = {
    **user_features,
    **content_features,
    **temporal_features,
    **language_features
}

return all_features

def apply_real_time_adjustments(self, tweet: Tweet, base_score:

```

PYTHON

float) -> float:

```
"""Apply real-time adjustments to virality prediction"""
```

```
adjusted_score = base_score
```

```
# Boost for early engagement
```

```
early_engagement =
```

```
self.engagement_tracker.get_early_engagement(tweet.tweet_id)
```

```
if early_engagement:
```

```
    engagement_boost = min(0.3,
```

```
early_engagement['retweets_per_minute'] * 0.1)
```

```
adjusted_score += engagement_boost
```

```
# Boost for influential user retweets
```

```
influential_retweets =
```

```
self.check_influential_retweets(tweet.tweet_id)
```

```
if influential_retweets:
```

```
    influence_boost = min(0.4, len(influential_retweets) *
```

```
0.1)
```

```
adjusted_score += influence_boost
```

```
# Current trending context
```

```
tweet_topics = self.extract_tweet_topics(tweet)
```

```
trending_topics = self.get_current_trending_topics()
```

```
trending_overlap =
```

```
set(tweet_topics).intersection(set(trending_topics))
```

PYTHON

PYTHON

```

if trending_overlap:
    trending_boost = len(trending_overlap) * 0.2
    adjusted_score += trending_boost

    return adjusted_score

class EngagementTracker:
    def __init__(self):
        # Track engagement metrics in real-time
        self.tweet_engagement = defaultdict(lambda: {
            'retweets': [],
            'likes': [],
            'replies': [],
            'first_seen': None
        })

        # Engagement rate benchmarks
        self.engagement_benchmarks = {
            'viral_retweet_rate': 50,      # 50 retweets/hour for

```

viral content

```

'viral_like_rate': 200,      # 200 likes/hour
'early_indicator_threshold': 10 # 10 engagements in

```

first 5 minutes

PYTHON

```

}

def track_engagement(self, tweet_id: str, engagement_type: str,

```

timestamp: datetime):

PYTHON

```

"""Track real-time engagement for a tweet"""

if tweet_id not in self.tweet_engagement:
    self.tweet_engagement[tweet_id]['first_seen'] =

```

timestamp

```
self.tweet_engagement[tweet_id]
```

PYTHON

```
[engagement_type].append(timestamp)
```

```
# Check for viral indicators
    if self.is_showing_viral_signs(tweet_id):
        self.alert_viral_potential(tweet_id)

def is_showing_viral_signs(self, tweet_id: str) -> bool:
    """Check if tweet is showing early signs of virality"""

    engagement = self.tweet_engagement[tweet_id]
    first_seen = engagement['first_seen']

    if not first_seen:
        return False

    current_time = datetime.now()

    # Check first 5 minutes
```

PYTHON

```
five_min_mark = first_seen + timedelta(minutes=5)
if current_time >= five_min_mark:
    early_engagements = 0
    for eng_type, timestamps in engagement.items():
        if eng_type != 'first_seen':
            early_engagements += len([
                ts for ts in timestamps
                if first_seen <= ts <= five_min_mark
            ])

    return early_engagements >=
```

```
self.engagement_benchmarks['early_indicator_threshold']
```

```

return False

def get_engagement_velocity(self, tweet_id: str) -> Dict:
    """Calculate current engagement velocity for a tweet"""

    if tweet_id not in self.tweet_engagement:
        return {'retweets_per_hour': 0, 'likes_per_hour': 0,

```

'replies_per_hour': 0}

```

engagement = self.tweet_engagement[tweet_id]
current_time = datetime.now()
one_hour_ago = current_time - timedelta(hours=1)

recent_retweets = len([
    ts for ts in engagement['retweets']
    if ts >= one_hour_ago
])

recent_likes = len([
    ts for ts in engagement['likes']
    if ts >= one_hour_ago
])

recent_replies = len([
    ts for ts in engagement['replies']
    if ts >= one_hour_ago
])

return {
    'retweets_per_hour': recent_retweets,
    'likes_per_hour': recent_likes,
    'replies_per_hour': recent_replies,
    'total_engagement_per_hour': recent_retweets +

```

recent_likes + recent_replies


```

}

# Example of how Twitter might track real-time analytics performance
TWITTER_ANALYTICS_METRICS = {
    'processing_performance': {
        'average_tweet_processing_latency_ms': 45,
        'peak_processing_throughput': 25000,      # tweets/second
        'pipeline_success_rate': 0.9995,
        'trending_detection_accuracy': 0.93,
        'spam_detection_precision': 0.88
    },
    'real_time_capabilities': {
        'trending_topic_update_frequency': 60,    # seconds
        'viral_content_detection_latency': 180,   # seconds
        'timeline_generation_latency_p95': 85,    # milliseconds
        'search_result_freshness': 15             # seconds
    },
    'user_engagement_impact': {
        'timeline_relevance_score': 4.2,         # out of 5
        'trending_topics_click_rate': 0.12,      # 12%
        'real_time_notification_engagement': 0.25, # 25%
        'search_satisfaction_rate': 0.87         # 87%
    }
}

```

❏ Pro Tip

Twitter's real-time architecture succeeds because they prioritize speed over perfection. Their systems are designed to make quick, good-enough decisions rather than decisions that are too slow. For example, trending topics are calculated with approximate algorithms that can handle massive spikes, even if they occasionally miss nuances. In real-time systems, being 90% accurate in 100ms is often better than being 99% accurate in 1 second.

□ Checkpoint

1. How does Twitter's real-time tweet processing handle massive spikes during breaking news events?
2. What makes trending topic detection technically challenging at Twitter's scale?
3. How does viral content prediction use both content features and social graph signals?

Try It Yourself Exercises

Exercise 1: Netflix Architecture Analysis (45 minutes) Compare Netflix's approach to a traditional data warehouse approach for the same use cases:

Task: Design two architectures: 1. Netflix's stream-first approach with Mantis + batch processing 2. Traditional approach with batch ETL + data warehouse

Compare: Latency, cost, complexity, scalability for: - Real-time

- recommendations
- A/B testing analysis
- Business reporting

Deliverable: Architecture comparison with trade-offs analysis.

Exercise 2: Uber Geospatial Design (60 minutes)

Design a geospatial data system for a food delivery service similar to

Uber Eats:

Requirements:

- 1M orders/day across 50 cities
- Real-time driver tracking and matching
- Dynamic pricing based on demand
- 30- second max wait for driver assignment

Task: Design the architecture including: 1. Geospatial indexing strategy 2. Real-time matching algorithm 3. Demand forecasting approach 4. Pricing optimization system

Deliverable: System design with API specifications and data flow diagrams.

Exercise 3: Multi-Company Pattern Analysis (30 minutes) Identify common patterns across all five case studies:

Task: Create a pattern library showing: 1. Common architectural patterns used by 2+ companies 2. Similar technical challenges solved differently 3. Trade-offs made by each company

Focus areas: - Real-time vs batch processing decisions -

Personalization strategies - Scale-out approaches - Data quality and monitoring

Deliverable: Pattern analysis document with recommendations for when to use each approach.

Module 11 Final Project: Architecture Design Challenge

Objective

Choose one of the case study companies and design an improvement or new system that addresses a limitation in their current architecture. Demonstrate deep understanding of their challenges and create a realistic solution.

Project Options

Option 1: Netflix Global Expansion Design a system to support Netflix's expansion into new international markets with different content licensing, local regulations, and varying infrastructure.

Option 2: Uber Autonomous Vehicle Integration Design data systems to support Uber's transition to autonomous vehicles, including route optimization, safety monitoring, and customer experience adaptation.

Option 3: Spotify Podcast Platform Design the data architecture to support Spotify's podcast platform, including creator analytics, content recommendation, and monetization systems.

Option 4: Airbnb Business Travel Platform Design enterprise features for Airbnb for Business, including approval workflows, expense reporting, and corporate travel analytics.

Option 5: Twitter Creator Economy Design systems to support Twitter's creator monetization features, including analytics, payment processing, and content performance tracking.

Requirements

Technical Requirements: 1. Handle 10x current scale over 3 years 2. Maintain existing performance SLAs 3. Support new business requirements 4. Consider global compliance and regulations

Business Requirements: 1. Enable new revenue streams or cost savings 2. Improve user experience measurably 3. Provide competitive advantage 4. Align with company strategic direction

Deliverables

1. Current State Analysis (2 pages)

Analysis of chosen company's current architecture Identification of limitations and bottlenecks

Business context for proposed improvement

Success metrics definition

2. Proposed Architecture (4 pages)

High-level system design

Component-level architecture

Data flow and processing pipeline

Technology stack recommendations

API design and integration points

3. Implementation Strategy (2 pages)

Migration approach and timeline

Risk assessment and mitigation

Resource requirements

Success measurement plan

4. Scale and Performance Analysis (2 pages)

Capacity planning for 10x growth

Performance bottleneck analysis

Cost optimization strategy

Monitoring and alerting design

Evaluation Criteria

- Technical Depth (30 points)
- ☐ Realistic understanding of current architecture

- ☐ Appropriate technology choices
- ☐ Scalable design principles
- ☐ Performance and reliability considerations
- Business Alignment (25 points)
 - ☐ Clear business value proposition
 - ☐ Realistic implementation timeline
 - ☐ Cost-benefit analysis
 - ☐ Risk assessment
- Innovation and Insights (20 points)
 - ☐ Novel approaches to known problems
 - ☐ Creative use of technology
 - ☐ Deep insights into architectural trade-offs
 - ☐ Learning from other company patterns
- Implementation Practicality (25 points)
 - ☐ Realistic migration strategy
 - ☐ Operational considerations
 - ☐ Team and resource planning
 - ☐ Success measurement framework

Success Criteria

Your design should demonstrate:

1. Deep Understanding: Show you understand the real challenges these companies face
2. Technical Excellence: Design systems that can handle massive scale reliably
3. Business Impact: Create clear business value, not just technical improvements
4. Implementation Reality: Provide a realistic path from current state to future state

Remember: These companies have brilliant engineers and massive resources. Your design should respect that and focus on genuine improvements, not just different approaches.

Further Reading

Company Engineering Blogs

- Netflix Tech Blog
- "Keystone Real-time Stream Processing Platform"
- Event ingestion at scale
- "Mantis: A Modern Stream Processing Framework"
- Custom stream processing
- "Data Pipeline Evolution at Netflix"
- Batch to real-time evolution

Uber Engineering Blog

- - "Real-time Exactly-once Ad Event Processing"
- Stream processing guarantees
- "H3: Uber's Hexagonal Hierarchical Spatial Index"
- Geospatial at scale
- "Surge Pricing for Shared Rides"
- Dynamic pricing algorithms
- Spotify Engineering Blog
- "Discover Weekly: Machine Learning in Production"
- Recommendation systems
- "Event Delivery at Scale"
- Real-time event processing
- "How We Built Spotify's Data Platform" -

Modern data stack evolution

- Airbnb Engineering Blog

- "Machine Learning-Powered Search Ranking"
- ML in search systems
- "Airflow: Workflow Management at Scale"
- Data pipeline orchestration
- "Data Quality at Airbnb" -

Monitoring and quality assurance

- Twitter Engineering Blog
- "The Infrastructure Behind Twitter: Scale"
- Real-time at massive scale
- "Real-time Analytics at Twitter"
- Stream processing architecture
- "Trends Detection in Real-time" -

Algorithmic trend detection

Research Papers

Stream Processing:

- "Apache Storm: Distributed Real-time Computation System"
- Storm fundamentals
- "MillWheel: Fault- Tolerant Stream Processing at Internet Scale"
- Google's approach
- "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost"

Machine Learning Systems:

- "TFX: A TensorFlow-Based Production-Scale Machine Learning Platform"
- ML pipelines
- "Scaling Machine Learning at Uber with Michelangelo"

- ML platform design
- "Applied Machine Learning at Facebook: A Datacenter Infrastructure Perspective"

Data Architecture:

- "Lambda Architecture for Batch and Real-Time Processing"
- Hybrid architectures
- "Kappa Architecture: Where Every Thing Is A Stream"
- Stream-first design
- "The Big Data Ecosystem at LinkedIn"
- Enterprise data platform patterns

Industry Analysis

Scale Benchmarks: - High Scalability case studies of major

- platforms
- ACM Digital Library papers on distributed systems
- VLDB conference proceedings on large-scale data management

Architectural Evolution:

- Martin Fowler's articles on microservices and data architecture
- AWS, Google Cloud, and Azure architecture guides
- CNCF landscape analysis for cloud-native data systems

Remember: These case studies represent years of evolution and massive investments. Use them as inspiration and learning opportunities, but always consider your specific context, constraints, and requirements when designing your own systems.# Module 12:

Interview Preparation & Portfolio Projects

What you'll learn

How to ace system design interviews for senior data engineering roles, build a portfolio that demonstrates your skills, and position yourself for staff/principal engineer opportunities. You'll practice with real interview questions and learn the communication techniques that separate from good ones.

12

Interview Preparation & Portfolio Projects

System Design Interview Deep Dive

⚡ TL;DR

System design interviews test your ability to think systematically, not your memorization of specific technologies - Spend 60% of your time on problem clarification and high-level design, 40% on deep dives and optimizations - Always start with the simplest solution that works, then iterate and improve - Communication skills matter as much as technical knowledge — practice explaining complex concepts clearly

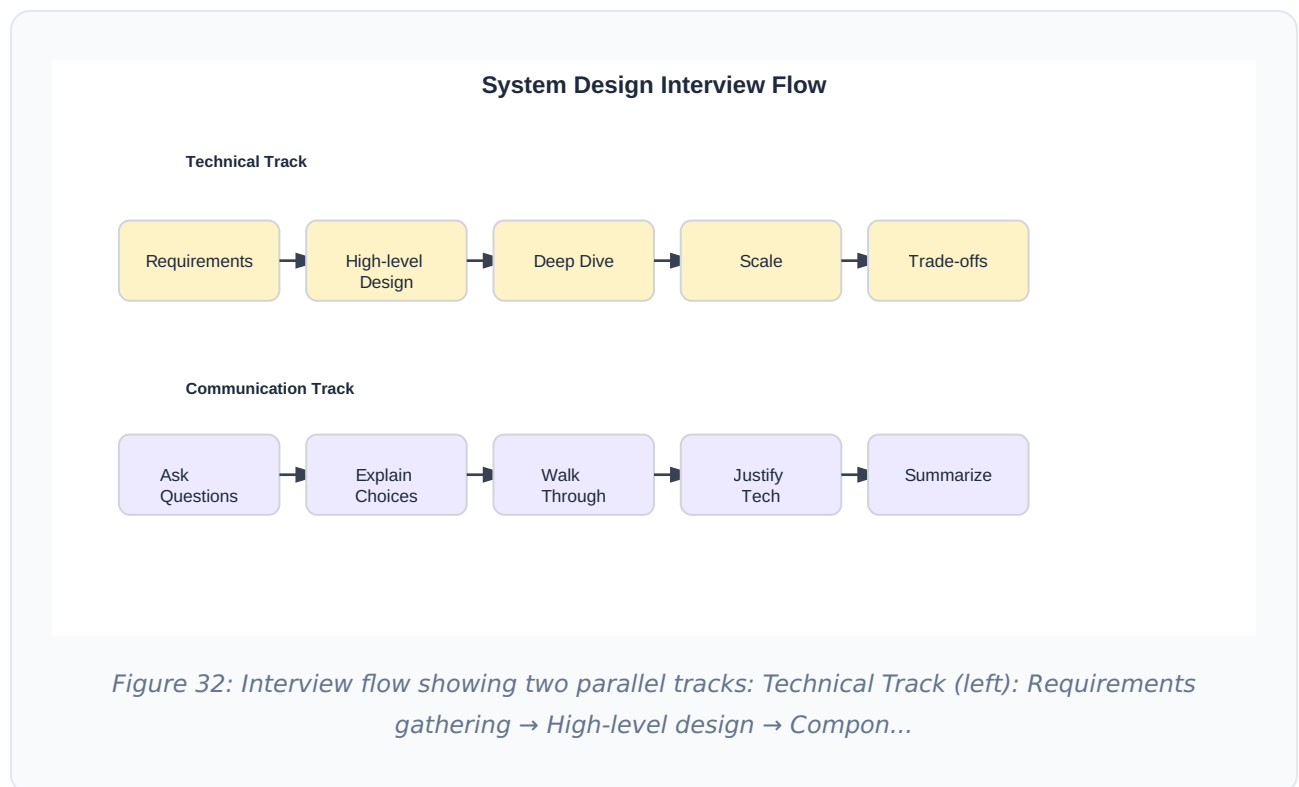
After conducting 150+ system design interviews for senior data engineering roles at three companies, in practice, the difference between candidates who get offers and those who don't isn't their technical knowledge — it's their approach to problem-solving and their ability to communicate their reasoning clearly.

The best candidates treat the interview as a collaborative design session, not a test where they need to guess the "right" answer.

The Interview Mindset Shift

Wrong Mindset: "I need to design the system and impress the interviewer with my knowledge of advanced technologies."

Right Mindset: "I need to understand the business problem deeply, design a system that solves it effectively, and communicate my reasoning clearly so the interviewer can follow my thought process."



The 45-Minute Interview Structure

Minutes 1-8: Requirements and Scale Estimation

Your goal: Understand what you're actually building

- - Functional requirements (what the system does)
- Non-functional requirements (how well it does it)
- Scale estimation (how much data, how many users)
- Business context (internal tool vs. customer-facing)

Sample dialogue: You: "Before I start designing, I'd like to understand the requirements better.

Who are the primary users of this system?"

Interviewer: "Internal data scientists who need to analyze customer behavior." You: "Got it. How many data scientists, and what kinds of analyses are they running?"

Are they looking for real-time insights or is batch processing sufficient?"

Minutes 8-20: High-Level Architecture

Your goal: Design the overall system structure

- - Draw the major components (storage, processing, serving)
- Show data flow between components
- Pick appropriate technologies with brief justification
- Keep it simple — resist the urge to over-engineer

Sample approach: 1. Start with data sources → processing → storage → serving 2. Add one technology choice at a time with reasoning 3. Explain why each component is necessary 4. Show how data flows through the system

Minutes 20-35: Component Deep Dive

- Your goal: Demonstrate depth in 1-2 specific areas
- Pick the most critical or interesting component
- Design its internal structure
- Discuss data models, APIs, algorithms
- Show you can think through implementation details

Interviewer guidance: "Let's dive deeper into the real-time processing component.

How would you handle out-of-order events and ensure exactly-once processing?"

Minutes 35-40: Scale and Optimize

Your goal: Show you can design for production scale

- - Identify potential bottlenecks
- Discuss scaling strategies (horizontal, vertical, caching)
- Consider failure scenarios and recovery
- Estimate resource requirements

Key topics:

- What happens when data volume grows 10x?
 - How do you handle component failures?
 - Where would you add monitoring and alerting?

Minutes 40-45: Trade-offs and Alternatives

Your goal: Demonstrate mature engineering judgment

- - Acknowledge what you optimized for and what you sacrificed
- Discuss alternative approaches you considered
- Explain when you'd choose differently
- Show you understand there are no solutions

Advanced Interview Techniques

The "Assumption Surfacing" Technique

```
# Instead of making silent assumptions, vocalize your thinking
def clarify_requirements_systematically():
    assumptions_to_validate = [
        "I'm assuming this is primarily a read-heavy workload – is
```

PYTHON

that correct?",

```
"For scale estimation, I'm thinking 100TB of data – does
```

that sound right?",

```
"I'm assuming data freshness requirements are around 1 hour
```

— should I optimize for real-time instead?",

```
"For consistency, I'm assuming eventual consistency is
```

acceptable — do we need strong consistency?"

```
]
```

```
# This shows you're thinking systematically about edge cases  
# and that you understand the implications of different choices
```

The "Iterative Design" Method

```
def design_iteratively():  
    # Start simple  
    version_1 = "Single PostgreSQL database with batch ETL"  
    explain_limitations("This works for small scale but won't handle
```

PYTHON

growth")

```
# Add complexity only when needed  
version_2 = "Distributed processing with Spark, data lake
```

storage"

```
explain_improvements("Now we can handle larger volumes and more
```

complex analytics")

```
# Optimize for specific requirements  
version_3 = "Add real-time stream processing for low-latency use
```

cases"

```
explain_trade_offs("Better latency but increased operational
```

complexity")


```
# This shows you understand when to add complexity and why
```

The "Failure Mode Analysis" Approach

```
def analyze_failure_modes():  
    critical_failure_scenarios = [  
        {  
            "failure": "Primary database goes down",  
            "impact": "No new data can be written, queries fail",  
            "mitigation": "Read replicas for queries, automatic
```

PYTHON

failover for writes",

```
            "recovery_time": "< 5 minutes with proper automation"  
        },  
        {  
            "failure": "Stream processing pipeline fails",  
            "impact": "Real-time dashboard data becomes stale",  
            "mitigation": "Circuit breakers, backfill capability
```

from batch system",

```
            "recovery_time": "< 30 minutes to restore, 2 hours to
```

backfill"

```
        },  
        {  
            "failure": "Data warehouse corruption",  
            "impact": "Historical analytics unavailable",  
            "mitigation": "Point-in-time backups, immutable data
```

lake as source of truth",

```
"recovery_time": "4-8 hours for full restoration"
  }
]

# Shows you think about production scenarios, not just happy
```

path

Common Interview Mistakes and How to Avoid Them

Mistake #1: Jumping to Technology Choices Too Quickly

❑ Bad approach: "I'll use Kafka for streaming, Spark for processing, and Snowflake for warehousing."

❑ Checkpoint

1. ❑ Good approach:

"I need a message queue that can handle high throughput and low latency.

Kafka would be a good choice because of its partitioning model and durability guarantees. Alternatively, I could consider Pulsar if I needed multi-tenancy features..."

Mistake #2: Over-Engineering from the Start

❑ Bad approach: Drawing a complex architecture with 15 microservices for a simple analytics use case.

❑ Checkpoint

1. ❑ Good approach:

"Let me start with the simplest architecture that meets the requirements. For 1GB of daily data and 10 analysts, a single PostgreSQL instance with scheduled ETL jobs would work well. As we scale to 100GB and 100 analysts, we'd need to consider distributed processing..."

Mistake #3: Ignoring the Business Context

❑ Bad approach: Designing a technically system without considering cost, team expertise, or timeline.

❑ Checkpoint

1. ❑ Good approach:

"Given that this is a 6-month project with a 3-person data team, I'd recommend using managed services like BigQuery rather than building a custom Spark cluster.

This reduces operational overhead and lets the team focus on business logic."

Mistake #4: Not Handling Ambiguity Well

❑ Bad approach: Making assumptions and proceeding without clarification.

❑ Checkpoint

1. ❑ Good approach:

"There are a few ways I could interpret this requirement. Are you looking for real-time alerts where users are notified within seconds, or is it more of a daily digest where near-real-time processing is sufficient?"

Communication Frameworks That Work

The "Three-Level Explanation" Technique

```
def explain_architecture_choice(component, audience_level):  
    explanations = {  
        'executive_summary': f"I'm choosing {component} because it
```

PYTHON

directly solves our main problem of {core_problem} while staying within budget and timeline constraints.",

```
'engineering_peer': f"I'm selecting {component} over
```

SQL

alternatives like {alternative_1} and {alternative_2} because our requirements for {specific_requirement} make it the best fit. The trade-off is {trade_off} but that's acceptable given {business_context}."

```
'implementation_detail': f"For {component}, I'd configure it
```

with {specific_config} to handle {technical_requirement}. The key implementation considerations are {technical_details} and we'd need to monitor {monitoring_metrics}."

```
}
```

```
# Start with executive summary, then dive deeper based on
```

interviewer interest

```
return explanations
```

The "Assumption → Implication" Pattern

```
def structure_technical_reasoning():  
    reasoning_pattern = [  
        "Given that we expect X (assumption)",  
        "This means we need Y capability (implication)",  
        "So I'm choosing Z technology (decision)",  
        "Which gives us benefits A and B but costs us C (trade-
```

PYTHON

offs)"

```
]
```

```
# Example:
```

```
example = [  
    "Given that we expect 10x growth in data volume over 2
```

```
    "Given that we expect 10x growth in data volume over 2
```

years",

"This means we need horizontally scalable storage and

processing",

"So I'm choosing a cloud data warehouse like Snowflake",
"Which gives us automatic scaling and managed operations but

costs more than self-managed solutions"

]

System Design Question Categories

Category 1: Analytics and Business Intelligence

Common questions:

- - "Design a data warehouse for an e-commerce company"
- "Build a real-time dashboard for operational metrics"
- "Create an analytics platform for A/B testing"

Key considerations:

- - OLAP vs OLTP workload patterns
- Batch vs real-time processing trade-offs
- Schema design for analytical queries
- Data quality and consistency requirements

Category 2: Data Pipeline and ETL

Common questions:

- - "Design a data pipeline that processes 1TB of data daily"
- "Build a system to sync data between multiple databases"
- "Create a CDC system for real-time data replication"

Key considerations:

- - Fault tolerance and retry mechanisms
- Data transformation and validation
- Monitoring and alerting strategies

- Schema evolution and backward compatibility

Category 3: Real-time and Streaming Systems

Common questions:

- - "Design a fraud detection system"
- "Build a real-time recommendation engine"
- "Create a system for processing IoT sensor data"

Key considerations:

- - Stream processing frameworks and patterns
- Low-latency requirements vs consistency guarantees
- State management in streaming systems
- Handling out-of-order and duplicate events

Category 4: Search and Discovery

Common questions:

- - "Design a search system for product catalog"
- "Build a recommendation system for content"
- "Create a data discovery platform for analysts"

Key considerations:

- - Full-text search vs structured queries
- Relevance ranking and personalization
- Index management and updates
- Query performance optimization

Category 5: ML and AI Infrastructure

Common questions:

- - "Design an ML training pipeline"
- "Build a feature store for machine learning"
- "Create a model serving infrastructure"

Key considerations:

- - Training vs inference workload patterns
- Feature engineering and serving pipelines
- Model versioning and rollback strategies
- Online vs offline evaluation metrics

□ **Checkpoint**

1. What's the most important thing to focus on in the first 8 minutes of a system design interview?
2. How do you balance showing technical depth vs. staying high-level in your design?
3. What's the difference between good and great communication in a technical interview?

12.2 Common Interview Questions & Solutions

⚡ TL;DR

Practice with realistic time constraints — 45 minutes goes by fast when you're thinking through complex systems - Focus on the most common question patterns: analytics platforms, real-time systems, and ML infrastructure - Develop templates for common components (data ingestion, processing, storage, serving) that you can mix and match - Every question is really asking: "Can you design systems that solve business problems at scale?"

Here are five representative questions that cover the most common patterns in data engineering interviews, along with complete solution approaches that demonstrate the level of thinking expected for senior roles.

Question 1: Design a Data Warehouse for E-commerce Analytics

Setup: "You're working at an e-commerce company that needs to build a data warehouse to support business intelligence, marketing analytics, and executive reporting. The company has 1 million active customers, processes 50,000 orders daily, and has data sources including the main application database, payment systems, customer service logs, and marketing platforms."

Complete Solution Approach:


```
# Step 1: Requirements Clarification (8 minutes)
def clarify_requirements():
    functional_requirements = {
        'business_intelligence': [
            'Daily/weekly/monthly sales reports',
            'Customer segmentation analysis',
            'Product performance analytics',
            'Marketing campaign effectiveness'
        ],
        'users_and_usage': [
            'Business analysts (daily reporting)',
            'Marketing team (campaign analysis)',
            'Executives (weekly dashboards)',
            'Data scientists (ad-hoc analysis)'
        ],
        'data_sources': [
            'Transactional database (orders, customers, products)',
            'Payment processor APIs (Stripe, PayPal)',
            'Customer service platform (Zendesk)',
            'Marketing platforms (Google Ads, Facebook)',
            'Web analytics (Google Analytics)',
            'Email marketing (Mailchimp)'
        ]
    }

    non_functional_requirements = {
        'data_volume': '50K orders/day = ~18M orders/year',
        'user_count': '20 analysts + 5 executives + 10 marketers',
        'query_patterns': 'Mostly aggregations, some ad-hoc
```

exploration',

```
'latency_requirements': 'Batch processing acceptable (daily
```

updates)',

```

'availability': '99.9% during business hours',
    'data_retention': '7 years for compliance',
    'budget_constraints': 'Mid-market company, cost-conscious'
}

return functional_requirements, non_functional_requirements

# Step 2: Scale Estimation (5 minutes)
def estimate_scale():
    scale_calculation = {
        'current_data_volume': {
            'orders_per_day': 50000,
            'customers': 1000000,
            'products': 50000,
            'order_line_items_per_day': 100000,    # ~2 items per

```

order

```

'daily_data_size': {
    'orders': '50K * 1KB = 50MB',
    'line_items': '100K * 0.5KB = 50MB',
    'customer_updates': '10K * 2KB = 20MB',
    'web_analytics': '500MB',
    'marketing_data': '100MB',
    'total_daily': '~720MB'
},

'annual_data_growth': '720MB * 365 = 260GB/year',
'with_history_7_years': '260GB * 7 = 1.8TB total'
},

'query_patterns': {

```

```

'daily_reports': '50 queries/day',
'ad_hoc_analysis': '100 queries/day',
'dashboard_refreshes': '200 queries/day',
'peak_concurrent_users': '10 users',
'typical_query_size': '1-100MB data scanned'
},

'growth_projections': {
    '2_year_growth': '3x customers, 5x orders (seasonal

```

business)'),

```
'team_growth': '2x analysts and marketers',
    'data_sources': '+5 new systems (CRM, inventory, etc.)'
}
```

```
return scale_calculation
```

```
# Step 3: High-Level Architecture (12 minutes)
```

```
def design_architecture():
```

```
    architecture = {
```

```
        'ingestion_layer': {
```

```
            'batch_etl': 'Scheduled jobs for database exports',
```

```
            'api_connectors': 'For SaaS platforms (Stripe, GA)',
```

```
            'cdc_streaming': 'For real-time database changes
```

```
(optional)',
```

```
'technology_choice': 'Airbyte or Fivetran for managed
```

```
connectors'
```

```

},

'storage_layer': {
    'data_lake': 'S3 for raw data storage',
    'data_warehouse': 'Snowflake or BigQuery for analytics',
    'staging_area': 'Temporary storage for transformations',
    'why_cloud_warehouse': [
        'Handles structured/semi-structured data',
        'Auto-scaling for variable workloads',
        'Separation of compute and storage',
        'Built-in optimization features'
    ]
},

'transformation_layer': {
    'tool_choice': 'dbt (data build tool)',
    'transformation_types': [
        'Data cleaning and validation',
        'Dimensional modeling (star schema)',
        'Business logic implementation',
        'Aggregation tables for performance'
    ],
    'orchestration': 'Airflow for scheduling and

```

dependencies'

```

},

'serving_layer': {
    'bi_tools': 'Tableau or Looker for analysts',
    'dashboards': 'Custom dashboards for executives',
    'api_layer': 'REST API for embedding analytics',
    'data_exports': 'Scheduled reports via email'
}

}

# Data flow visualization
data_flow = ""
Sources → Ingestion → Raw Storage → Transform → Warehouse → BI

```

Tools

↓	↓	↓	↓	↓	↓
Databases	ETL	S3 Lake	dbt	Snowflake	Tableau
APIs	Airbyte	Raw/Clean	Airflow	Warehouse	Looker
SaaS	Schedule	Partitioned	Transform	Analytics	

Dashboards

```
"""
```

```
return architecture, data_flow
```

```
# Step 4: Deep Dive - Dimensional Modeling (8 minutes) def design_data_model():
dimensional_model = {
```

```
# Step 4: Deep Dive - Dimensional Modeling (8 minutes) def design_data_model():
dimensional_model = {
```

```

'fact_tables': {
  'fact_orders': {
    'description': 'One row per order',
    'key_measures': [
      'order_amount', 'tax_amount', 'shipping_amount',
      'discount_amount', 'profit_margin'
    ],
    'dimensions': [
      'date_key', 'customer_key', 'product_key',
      'payment_method_key', 'shipping_method_key'
    ],
    'grain': 'One row per order'
  },

  'fact_order_lines': {
    'description': 'One row per product in order',
    'key_measures': [
      'quantity', 'unit_price', 'line_total',
      'cost_of_goods', 'line_profit'
    ],
    'dimensions': [
      'date_key', 'order_key', 'product_key',
      'customer_key'
    ],
    'grain': 'One row per product per order'
  }
},

'dimension_tables': {
  'dim_customer': [
    'customer_key (surrogate)', 'customer_id (natural)',
    'first_name', 'last_name', 'email', 'phone',
    'registration_date', 'customer_segment',
    'lifetime_value', 'preferred_category',
    'effective_date', 'expiry_date', 'is_current'
  ],

  'dim_product': [
    'product_key', 'product_id', 'product_name',
    'category', 'subcategory', 'brand', 'cost',
    'list_price', 'product_launch_date', 'status'
  ],

  'dim_date': [
    'date_key', 'calendar_date', 'year', 'quarter',
    'month', 'week', 'day_of_week', 'is_weekend',
    'is_holiday', 'fiscal_year', 'fiscal_quarter'
  ]
}

```

```
    ]  
  }
```

```
}
```

Sample SQL for key business metrics sample_queries = "" -- Monthly revenue by product category SELECT

```
d.year,  
d.month,  
p.category,  
SUM(f.order_amount) as revenue,  
COUNT(DISTINCT f.customer_key) as unique_customers
```

```
FROM fact_orders f JOIN dim_date d ON f.date_key = d.date_key JOIN  
fact_order_lines ol ON f.order_key = ol.order_key JOIN dim_product p ON  
ol.product_key = p.product_key WHERE d.calendar_date >= '2024-01-01' GROUP BY  
d.year, d.month, p.category ORDER BY d.year, d.month, revenue DESC;
```

```
-- Customer lifetime value analysis
WITH customer_metrics AS (
    SELECT
        c.customer_key,
        c.customer_segment,
        COUNT(f.order_key) as total_orders,
        SUM(f.order_amount) as total_spent,
        MAX(d.calendar_date) as last_order_date,
        MIN(d.calendar_date) as first_order_date
    FROM fact_orders f
    JOIN dim_customer c ON f.customer_key = c.customer_key
    JOIN dim_date d ON f.date_key = d.date_key
    GROUP BY c.customer_key, c.customer_segment
)
SELECT
    customer_segment,
    AVG(total_spent) as avg_lifetime_value,
    AVG(total_orders) as avg_order_frequency,
    COUNT(*) as customer_count
FROM customer_metrics
GROUP BY customer_segment;
"""

return dimensional_model, sample_queries

# Step 5: Scaling and Optimization (7 minutes)
def design_for_scale():
    scaling_strategy = {
        'current_bottlenecks': [
            'ETL processing time increases with data volume',
            'Complex analytical queries slow down with growth',
            'Dashboard refresh times increase',
            'Storage costs grow linearly with data'
        ],
        'scaling_solutions': {
            'compute_scaling': {
                'snowflake_auto_scaling': 'Scale warehouse up/down

```

based on query load',

```
'query_optimization': 'Materialized views for common

```

aggregations',


```
'workload_isolation': 'Separate warehouses for ETL'
```

vs analytics',

```
'result_caching': 'Cache frequently-accessed query'
```

results'

```
},  
  
  'storage_optimization': {  
    'data_partitioning': 'Partition by date for query'
```

pruning',

```
'data_clustering': 'Cluster frequently-filtered'
```

columns',

```
'lifecycle_management': 'Move old data to cheaper'
```

storage tiers',

```
'compression': 'Enable automatic compression'
```

features'

```
},  
  
  'etl_optimization': {  
    'incremental_processing': 'Only process new/changed'
```

data',

```
'parallel_processing': 'Run independent
```

transformations in parallel',

```
'dependency_optimization': 'Optimize dbt model
```

dependencies',

```
'change_data_capture': 'Real-time sync for critical
```

data'

```
}  
},
```

```
  'monitoring_strategy': {
```

The PYTHON code:

```

'data_quality_monitoring': [
    'Row count validation between source and target',
    'Data freshness checks (last update timestamp)',
    'Schema drift detection',
    'Business rule validation (e.g., revenue > 0)'
],

'performance_monitoring': [
    'ETL job duration and success rate',
    'Query response times and concurrency',
    'Storage utilization and growth trends',
    'User adoption and query patterns'
],

'cost_monitoring': [
    'Snowflake credit consumption by warehouse',
    'Storage costs by database/schema',
    'Query costs by user/team',
    'ROI metrics (cost per insight delivered)'
]
}

return scaling_strategy

# Step 6: Trade-offs and Alternatives (5 minutes)
def discuss_trade_offs():
    trade_offs = {
        'cloud_warehouse_choice': {
            'snowflake': {
                'pros': ['Multi-cloud support', 'Easy scaling',

```

'JSON handling'],

```

'cons': ['Higher cost', 'Vendor lock-in'],
    'best_for': 'Mixed workloads, rapid growth'
},
'bigquery': {
    'pros': ['Serverless', 'ML integration', 'Pay-per-

```

query'],

```

'cons': ['GCP only', 'Complex pricing'],
  'best_for': 'Variable workloads, ML heavy'
},
'redshift': {
  'pros': ['Lower cost', 'AWS integration'],
  'cons': ['More management overhead', 'Less

```

flexible'],

```

'best_for': 'Predictable workloads, cost-sensitive'
  }
},

'etl_vs_elt': {
  'chosen_approach': 'ELT (Extract, Load, Transform)',
  'reasoning': 'Leverage warehouse compute power, keep raw

```

data',

```

'trade_off': 'Higher storage costs but more flexibility'
},

'real_time_vs_batch': {
  'chosen_approach': 'Batch processing (daily updates)',
  'reasoning': 'Business requirements don\'t justify real-

```

time complexity',

```

'when_to_reconsider': 'If marketing needs real-time

```

campaign optimization'

```

}
}

return trade_offs

```

Question 2: Design a Real-time Fraud Detection System

Setup: "Design a system that can detect fraudulent transactions in real-time for a payment processor. The system handles 10,000 transactions per second, needs to make decisions within 100ms, and should minimize false positives while catching as many fraudulent transactions as possible."

Complete Solution Approach:

```
# Step 1: Requirements Deep Dive
def clarify_fraud_detection_requirements():
    requirements = {
        'functional': {
            'real_time_scoring': 'Score every transaction within
```

PYTHON

100ms',

```
        'decision_types': ['approve', 'decline',
```

'manual_review'],

```
        'fraud_types_to_detect': [
            'stolen_credit_cards',
            'account_takeovers',
            'synthetic_identities',
            'merchant_fraud',
            'chargeback_fraud'
        ],
        'compliance': 'PCI DSS, PSD2 strong authentication'
    },
    'non_functional': {
        'throughput': '10,000 TPS sustained, 25,000 TPS peak',
        'latency': '<100ms p99 for scoring decision',
        'availability': '99.99% uptime (4 minutes
```

downtime/month)',

```
        'accuracy_targets': {
            'false_positive_rate': '<2% (good customers
```

declined)',

```
'fraud_detection_rate': '>85% of actual fraud
```

caught',

```
    'precision': '>60% of fraud alerts are real fraud'
  }
}

scale_estimation = {
  'transaction_volume': {
    'daily_transactions': 864_000_000,    # 10K TPS * 86,400
```

seconds

```
'peak_multiplier': 2.5, # Holiday/Black Friday spikes
'transaction_size': '2KB average (payment data +
```

metadata)'

```

},

    'feature_storage': {
        'user_profiles': '100M active users * 10KB = 1TB',
        'merchant_profiles': '2M merchants * 50KB = 100GB',
        'behavioral_features': '500GB (rolling 90-day windows)',
        'ml_model_storage': '10GB for all models and features'
    },

    'processing_requirements': {
        'feature_computation': '50ms p99 (real-time features)',
        'ml_inference': '30ms p99 (ensemble of models)',
        'rule_evaluation': '10ms p99 (business rules)',
        'decision_logging': '10ms p99 (audit trail)'
    }
}

return requirements, scale_estimation

# Step 2: High-Level Architecture
def design_fraud_detection_architecture():
    architecture = {
        'ingestion_layer': {
            'transaction_stream': 'Kafka for reliable transaction

```

ingestion',

```

'partitioning_strategy': 'By user_id for feature

```

locality',

```

'schema_registry': 'Confluent Schema Registry for

```

evolution',

```

'retention_policy': '7 days for reprocessing capability'
},

'feature_store': {
    'real_time_features': 'Redis Cluster for <1ms lookups',
    'batch_features': 'Cassandra for historical

```

aggregations',

```
'feature_pipeline': 'Flink for real-time feature
```

computation',

```
'feature_serving': 'gRPC API for low-latency access'
},
'ml_serving': {
  'model_store': 'MLflow for model versioning',
  'inference_engine': 'TensorFlow Serving or Seldon',
  'model_ensemble': 'Multiple models for different fraud
```

types',

```
'a_b_testing': 'Gradual rollout of new models'
},
'decision_engine': {
  'rules_engine': 'Drools or custom rule evaluation',
  'ml_models': 'Ensemble of fraud detection models',
  'risk_scoring': 'Combine rules + ML for final score',
  'decision_logic': 'Thresholds based on risk tolerance'
},
'response_handling': {
  'real_time_response': 'Sub-100ms approve/decline
```

decision',

```
'manual_review_queue': 'Human review for borderline
```

cases',

```
'merchant_notification': 'Real-time alerts for high-risk
```

patterns',


```
'customer_communication': 'SMS/email for declined
```

transactions'

```
}  
}
```

```
data_flow = """  
Transaction → Feature → ML Models → Rules → Decision → Response  
Stream      Lookup   Inference Engine Logic (100ms)  
    ↓           ↓           ↓           ↓           ↓           ↓  
Kafka → Redis/Cassandra → TF Serving → Drools → API → Payment
```

Gateway

Feature Store Model Ensemble Rules Decision

Response

```
"""  
  
    return architecture, data_flow  
  
# Step 3: Deep Dive - Real-Time Feature Engineering  
def design_feature_engineering():  
    feature_categories = {  
        'transaction_features': {  
            'immediate': [  
                'transaction_amount',  
                'merchant_id',  
                'payment_method',  
                'currency',  
                'transaction_timestamp',  
                'device_fingerprint'  
            ],  
            'derived': [  
                'amount_z_score_user_30d', # How unusual is this
```

PYTHON

amount?

```
'velocity_transactions_1h',    # How many transactions
```

recently?

```
'merchant_risk_score',        # Historical merchant
```

fraud rate

```
'device_reputation_score'    # Device fraud history
    ]
},

'user_behavioral_features': {
    'real_time_computed': [
        'transactions_last_1h',
        'unique_merchants_last_24h',
        'total_amount_last_24h',
        'failed_attempts_last_1h',
        'distance_from_last_transaction',
        'time_since_last_transaction'
    ],
    'batch_computed': [
        'avg_transaction_amount_30d',
        'preferred_transaction_times',
        'preferred_merchants',
        'typical_geographic_pattern',
        'payment_method_preferences'
    ]
},

'network_features': {
    'graph_based': [
        'shared_device_count',    # How many users share
```

SQL

this device?

```
'shared_ip_count',            # How many users from
```

SQL

this IP?

```
'merchant_user_network',    # User connections via
```

merchants

```
'payment_method_sharing'    # Shared payment
```

instruments

```
],  
'temporal_patterns': [  
    'burst_activity_score',    # Sudden activity
```

increase

```
'coordinated_activity',    # Multiple accounts,
```

similar patterns

```
'dormant_account_activation' # Long-inactive account
```

suddenly active

```
]  
    }  
}  
  
# Feature computation pipeline  
feature_pipeline = ""  
# Real-time feature computation with Flink  
transaction_stream  
    .keyBy(transaction -> transaction.user_id)  
    .window(SlidingEventTimeWindows.of(Time.hours(1),
```

```
Time.minutes(5)))
```

```
.aggregate(new TransactionVelocityAggregator())
.flatMap(new FeatureEnrichment())
.addSink(new RedisFeatureSink());

# Batch feature computation with Spark
daily_batch = spark.read.table("transactions")
    .filter(col("date") >= date_sub(current_date(), 30))
    .groupBy("user_id")
    .agg(
        avg("amount").alias("avg_amount_30d"),
        stddev("amount").alias("stddev_amount_30d"),
        countDistinct("merchant_id").alias("unique_merchants_30d")
    )
    .write.mode("overwrite").table("user_features_batch")
"""

return feature_categories, feature_pipeline
```

Step 4: ML Model Architecture:

```
# Step 4: ML Model Architecture
def design_ml_models():
    model_ensemble = {
        'gradient_boosting_model': {
            'purpose': 'Main fraud detection model',
            'features': 'All tabular features (150+ features)',
            'algorithm': 'XGBoost or LightGBM',
            'training_frequency': 'Daily retraining',
            'target_metric': 'AUC-ROC > 0.95'
        },

        'neural_network_model': {
            'purpose': 'Complex pattern detection',
            'features': 'Deep feature interactions, embeddings',
            'algorithm': 'Deep neural network with attention',
            'training_frequency': 'Weekly retraining',
            'target_metric': 'Precision at high recall'
        },

        'graph_neural_network': {
            'purpose': 'Network-based fraud detection',
            'features': 'User-merchant-device graph structure',
            'algorithm': 'GraphSAGE or Graph Attention Network',
            'training_frequency': 'Bi-weekly retraining',
            'target_metric': 'Fraud ring detection accuracy'
        },

        'anomaly_detection_model': {
            'purpose': 'Unsupervised fraud detection',
            'features': 'Behavioral pattern deviations',
            'algorithm': 'Isolation Forest, One-Class SVM',
            'training_frequency': 'Real-time adaptation',
            'target_metric': 'Novel fraud pattern detection'
        }
    }

    ensemble_strategy = """
    # Model ensemble with weighted voting
    def ensemble_predict(transaction_features):
        # Get predictions from all models
        xgb_score =
```

```
xgboost_model.predict_proba(transaction_features)[1]
```

```

nn_score = neural_net_model.predict(transaction_features)
graph_score = gnn_model.predict(transaction_features)
anomaly_score =

```

```

anomaly_model.decision_function(transaction_features)

```

```

# Weighted ensemble based on model confidence and historical

```

performance

```

weights = get_dynamic_weights(transaction_features)

final_score = (
    weights['xgb'] * xgb_score +
    weights['nn'] * nn_score +
    weights['graph'] * graph_score +
    weights['anomaly'] *

```

```

normalize_anomaly_score(anomaly_score)

```

```

)

        return final_score, {
            'xgb_score': xgb_score,
            'nn_score': nn_score,
            'graph_score': graph_score,
            'anomaly_score': anomaly_score,
            'ensemble_weights': weights
        }

    """

    return model_ensemble, ensemble_strategy

# Step 5: Scaling and Performance Optimization
def design_scaling_strategy():

```

PYTHON

```

scaling_approach = {
    'horizontal_scaling': {
        'kafka_partitioning': 'Partition by user_id hash for

```

feature locality',

```
'flink_parallelism': 'Scale stream processing based on
```

throughput',

```
'redis_clustering': 'Shard feature store across multiple
```

nodes',

```
'ml_serving_replicas': 'Multiple model serving instances
```

with load balancer'

```
},
```

```
  'caching_strategy': {  
    'feature_caching': 'Redis with 1-hour TTL for user
```

features',

```
'model_caching': 'In-memory model cache to avoid disk
```

I/O',

```
'decision_caching': 'Cache recent decisions for
```

duplicate detection',

```
'negative_caching': 'Cache "user not found" to reduce
```

lookups'

```
},  
  
  'performance_optimizations': {  
    'feature_computation': 'Pre-compute expensive features
```

in batch',

```
  'model_quantization': 'Reduce model size for faster
```

inference',

```
  'async_processing': 'Non-blocking I/O for feature
```

lookups',

```
  'circuit_breakers': 'Fail fast when dependencies are
```

slow'

```
},  
  
  'failure_handling': {  
    'graceful_degradation': 'Fall back to rules-only if ML
```

fails',

```
  'timeout_handling': 'Approve transactions if decision
```

takes >100ms',

```
  'data_quality_checks': 'Validate features before ML
```

inference',


```
'model_rollback': 'Automatic rollback if fraud detection
```

rate drops'

```
}  
}  
  
monitoring_strategy = {  
    'business_metrics': [  
        'fraud_detection_rate (% of actual fraud caught)',  
        'false_positive_rate (% of good transactions declined)',  
        'manual_review_rate (% requiring human review)',  
        'customer_friction_score (declined transaction impact)'  
    ],  
  
    'technical_metrics': [  
        'decision_latency_p99 (<100ms SLA)',  
        'feature_lookup_latency (Redis/Cassandra)',  
        'model_inference_latency (TensorFlow Serving)',  
        'end_to_end_throughput (transactions/second)'  
    ],  
  
    'model_metrics': [  
        'model_drift_detection (feature distribution changes)',  
        'prediction_confidence_distribution',  
        'feature_importance_stability',  
        'adversarial_attack_detection'  
    ]  
}  
  
return scaling_approach, monitoring_strategy
```

Question 3: Build a Feature Store for Machine Learning

Setup: "Your company has 50+ ML models in production, and data scientists are spending 80% of their time on feature engineering instead of model development. Design a feature store that enables feature reuse, ensures consistency between training and serving, and supports both batch and real-time ML use cases."

Complete Solution Approach:

```
# Step 1: Requirements Analysis
def analyze_feature_store_requirements():
    business_problem = {
        'current_pain_points': [
            'Feature engineering duplicated across teams',
            'Training/serving skew causing model performance
```

degradation',

```
        'Inconsistent feature definitions across models',
        'No feature discovery or reuse mechanism',
        'Data scientists reinventing features instead of
```

focusing on models'

```

],

'success_metrics': [
    'Reduce feature development time by 70%',
    'Eliminate training/serving skew',
    'Increase feature reuse rate to >80%',
    'Support <10ms feature serving latency',
    'Enable feature lineage and governance'
]
}

functional_requirements = {
    'feature_authoring': [
        'Define features with SQL/Python transformations',
        'Support batch and streaming feature computation',
        'Feature validation and testing framework',
        'Version control and governance workflow'
    ],

    'feature_serving': [
        'Online serving for real-time inference (<10ms)',
        'Offline serving for training dataset generation',
        'Point-in-time correctness for temporal features',
        'Feature monitoring and data quality checks'
    ],

    'feature_discovery': [
        'Feature catalog with search and documentation',
        'Feature lineage and dependency tracking',
        'Usage analytics and recommendation',
        'Feature sharing across teams and projects'
    ]
}

non_functional_requirements = {
    'scale': {
        'features': '10,000+ features across 100+ feature

```

groups',

```
'models': '50+ models in production, 200+ in
```

development',

```

    'users': '100 data scientists, 20 ML engineers',
    'serving_qps': '100,000 feature lookup requests/second',
    'training_data_size': '1PB+ for large model training'
},

    'performance': {
        'online_serving_latency': '<10ms p99',
        'offline_serving_throughput': '>10GB/s',
        'feature_freshness': '<5 minutes for real-time

```

features',

```

        'system_availability': '99.9% for online serving'
    }
}

return business_problem, functional_requirements,

```

non_functional_requirements

```

# Step 2: Architecture Design
def design_feature_store_architecture():
    architecture = {
        'feature_authoring_layer': {
            'feature_definition': 'YAML/Python DSL for feature

```

PYTHON

specifications',

```

    'transformation_engine': 'Spark/Beam for batch, Flink

```

for streaming',

```

    'validation_framework': 'Great Expectations for data

```

quality',

```

    'version_control': 'Git-based workflow with CI/CD

```

pipeline'

```
},  
  
  'feature_computation_layer': {  
    'batch_processing': 'Spark on Kubernetes for
```

daily/hourly features',

```
  'stream_processing': 'Flink for real-time aggregations',  
  'orchestration': 'Airflow for batch, native stream
```

processing',

```
  'computation_optimization': 'Incremental processing,
```

checkpointing'

```
},  
  
  'feature_storage_layer': {  
    'offline_store': 'Delta Lake/Iceberg for training data
```

generation',

```
  'online_store': 'Redis Cluster for low-latency serving',  
  'metadata_store': 'PostgreSQL for feature registry',  
  'blob_storage': 'S3/GCS for large features (embeddings,
```

```
etc.)'
```

```
},  
  
  'feature_serving_layer': {  
    'online_serving_api': 'gRPC/REST API for real-time
```

inference',

```
'offline_serving_api': 'SQL interface for training
```

```
data',
```

```
'sdk_libraries': 'Python/Java SDKs for easy
```

```
integration',
```

```
'caching_layer': 'Multi-tier caching for performance'
},
'governance_layer': {
    'feature_registry': 'Centralized catalog with metadata',
    'access_control': 'Role-based permissions and audit
```

```
logs',
```

```
'lineage_tracking': 'Data lineage from sources to
```

SQL

```
features',
```

```
'monitoring_alerting': 'Feature quality and usage
```

```
monitoring'
```

```
}
}

data_flow = """
Raw Data → Feature Authoring → Computation → Storage → Serving →
```

ML Models

```
↓           ↓           ↓           ↓           ↓
```

↓

Sources → Feature Definition → Spark/Flink → Delta/Redis → API →

Inference

(DB/Stream) (Python/SQL) (Transform) (Store) (Serve)

(Models)

↓ ↓ ↓ ↓
Validation → Monitoring → Registry → Governance

The corresponding SQL query:

(Quality) (Drift) (Catalog) (Access)

SQL

```
return architecture, data_flow

# Step 3: Deep Dive - Feature Definition and Computation
def design_feature_definition_system():
    feature_definition_framework = {
        'feature_specification': """
        # Example feature definition in Python DSL
        @feature_group(
            name="user_transaction_features",
            entity="user_id",
            description="User behavioral features from
transactions",
```

```

batch_schedule="@daily",
    stream_trigger="on_transaction_event"
)
class UserTransactionFeatures:

    @feature(
        name="transaction_count_7d",
        description="Number of transactions in last 7 days",
        dtype=ValueTypes.INT64,
        labels={"team": "fraud", "criticality": "high"}
    )
    def transaction_count_7d(self, df):
        return df.filter(
            col("timestamp") >= date_sub(current_date(), 7)
        ).groupBy("user_id").count()

    @feature(
        name="avg_transaction_amount_30d",
        description="Average transaction amount in last 30

```

days",

```

dtype=ValueTypes.DOUBLE,
    validation_rules=["amount > 0", "amount < 1000000"]
)
def avg_transaction_amount_30d(self, df):
    return df.filter(
        col("timestamp") >= date_sub(current_date(), 30)
    ).groupBy("user_id").agg(

```

```

avg("amount").alias("avg_transaction_amount_30d")

```

```

)
"""
'feature_types': {
    'aggregation_features': 'Time-windowed aggregations

```

```

(count, sum, avg)',

```



```
'transformation_features': 'Derived features (ratios,
```

encodings)',

```
'lookup_features': 'Enrichment from external systems',
'embedding_features': 'Dense vector representations',
'sequence_features': 'Time-series and sequential
```

SQL

patterns'

```
},
    'computation_patterns': {
        'batch_backfill': 'Compute historical features for
```

training',

```
'incremental_batch': 'Daily/hourly feature updates',
    'stream_processing': 'Real-time feature computation',
    'on_demand': 'Compute features at inference time'
}
}
```

PYTHON

```
computation_engine = """
# Feature computation with point-in-time correctness
class FeatureComputationEngine:

    def compute_training_dataset(self,
```

```
entity_df: DataFrame,
feature_refs: List[FeatureRef],
event_timestamps: Column) ->
```

DataFrame:

```
\"\"\"
Generate training dataset with point-in-time correct
```

features.

Ensures no data leakage by only using features available at event_time.

```
\\"\\""
```

PYTHON

```
result_df = entity_df

for feature_ref in feature_refs:
    feature_df = self.get_historical_features(
        feature_ref,
        entity_df,
        event_timestamps
    )

    # Point-in-time join to prevent data leakage
    result_df = result_df.join(
        feature_df,
        on=["entity_id"],
        how="left"
    ).filter(
        feature_df.event_timestamp <=
```

```
entity_df.event_timestamp
```

```
).groupBy("entity_id", "event_timestamp").agg(
    # Take latest feature value before event
    last("feature_value",
```

PYTHON

```
ignorenulls=True).alias("feature_value")
```

```
)

return result_df

def compute_online_features(self,
                             entity_ids: List[str],
                             feature_refs: List[FeatureRef]) ->
```

PYTHON

Dict:

```

"""
Retrieve latest feature values for online inference.
"""

features = {}

for feature_ref in feature_refs:
    if feature_ref.source == "batch":
        # Get pre-computed features from online store
        values = self.online_store.get_batch(
            feature_ref.name,
            entity_ids
        )

    elif feature_ref.source == "stream":
        # Get real-time computed features
        values = self.stream_store.get_batch(
            feature_ref.name,
            entity_ids
        )

    elif feature_ref.source == "on_demand":
        # Compute features at request time
        values = self.compute_on_demand_features(
            feature_ref,
            entity_ids
        )

    features[feature_ref.name] = values

return features
"""

```

The PYTHON code:

```

return feature_definition_framework, computation_engine

# Step 4: Online/Offline Storage Architecture
def design_storage_architecture():
    storage_strategy = {
        'offline_store_design': {
            'technology': 'Delta Lake on S3/ADLS',
            'partitioning': 'By entity_type/date for query

```

performance',

```
'schema_evolution': 'Delta supports schema evolution'
```

natively',

```
'time_travel': 'Point-in-time queries for training'
```

data',

```
'compaction': 'Regular compaction for query performance'
},
```

```
  'online_store_design': {
    'primary_store': 'Redis Cluster for low-latency access',
    'data_model': 'Hash maps: entity_id -> {feature_name:
```

value}',

```
'ttl_strategy': 'Feature-specific TTL based on freshness'
```

requirements',

```
'backup_store': 'DynamoDB for durability and cross-
```

region replication',

```
'caching_layers': 'Application-level cache + CDN for
```

static features'

```

},

'metadata_store': {
    'feature_registry': 'PostgreSQL for ACID properties',
    'schema_tracking': 'Feature schemas, versions, lineage',
    'access_patterns': 'Heavy read, light write workload',
    'backup_strategy': 'Regular snapshots + WAL replication'
}
}

# Storage optimization strategies
storage_optimization = """
# Online store optimization for high throughput
class OptimizedOnlineStore:

    def __init__(self):
        self.redis_cluster = RedisCluster(
            nodes=[...],
            max_connections_per_node=1000,
            connection_pool_kwargs={
                'health_check_interval': 30,
                'socket_timeout': 0.1,
                'socket_connect_timeout': 0.1
            }
        )

        # Feature grouping for batch retrieval
        self.feature_groups = self.build_feature_groups()

    def get_features(self, entity_ids: List[str],
                    feature_names: List[str]) -> Dict:
        """
        Optimized batch feature retrieval.
        """

        # Group features by storage key pattern for efficiency
        grouped_requests = self.group_feature_requests(
            entity_ids, feature_names
        )

        results = {}

        # Parallel requests to different Redis nodes
        with ThreadPoolExecutor(max_workers=10) as executor:
            futures = []

```

The PYTHON code:

```

for group, requests in grouped_requests.items():
    future = executor.submit(
        self.redis_cluster.mget,
        [self.build_key(entity_id, feature)
         for entity_id, feature in requests]
    )
    futures.append((group, requests, future))

for group, requests, future in futures:
    values = future.result()

    # Parse and structure results
    for (entity_id, feature), value in zip(requests,

```

values):

```

if entity_id not in results:
    results[entity_id] = {}
results[entity_id][feature] =

```

```

self.parse_value(value)

```

```

return results

def write_features(self, feature_data: Dict):
    """
    Optimized batch feature writing.
    """

    # Pipeline writes for better performance
    pipe = self.redis_cluster.pipeline()

    for entity_id, features in feature_data.items():
        for feature_name, value in features.items():
            key = self.build_key(entity_id, feature_name)
            ttl = self.get_feature_ttl(feature_name)

            pipe.set(key, self.serialize_value(value),

```

```

ex=ttl)

```

```
# Execute all writes in a single round trip
pipe.execute()

"""

return storage_strategy, storage_optimization

# Step 5: Feature Serving and Integration
def design_serving_layer():
    serving_architecture = {
        'api_design': {
            'rest_api': 'HTTP endpoints for low-volume, interactive
```

use',

```
'grpc_api': 'High-performance RPC for production
```

inference',

```
'graphql_api': 'Flexible queries for feature
```

exploration',

```
'streaming_api': 'WebSocket for real-time feature
```

updates'

```
},

'sdk_integration': """
# Python SDK for easy integration
from feature_store import FeatureStore

# Initialize feature store client
fs = FeatureStore(
    config_path="feature_store.yaml",
    environment="production"
)

# Define feature service for a model
feature_service =
```

```
fs.get_feature_service("fraud_detection_v3")
```

SQL

```
# Online inference
def predict(user_id: str, transaction_data: Dict) -> float:
    # Get features for inference
    features = feature_service.get_online_features(
        entity_ids={"user_id": user_id},
        request_data=transaction_data
    )

    # Run model inference
    prediction = model.predict([features.to_dict()])

    return prediction[0]

# Training data generation
def generate_training_data(start_date: str, end_date: str):
    # Define label dataset with event timestamps
    label_df = fs.get_dataset(
        query=f"""
        SELECT user_id, transaction_id, timestamp, is_fraud
        FROM transactions
        WHERE date BETWEEN '{start_date}' AND '{end_date}'
        """
    )

    # Get historical features with point-in-time correctness
    training_df = feature_service.get_historical_features(
        entity_df=label_df,
        entity_timestamp_column="timestamp"
    )

    return training_df

"""
'caching_strategy': {
    'l1_cache': 'In-memory cache in serving application
```

```
(1ms)',
```

```
'l2_cache': 'Redis cache shared across instances (5ms)',
'l3_cache': 'Offline store for cache misses (50ms)',
'cache_warming': 'Proactive cache population for popular
```


entities',

```
'cache_invalidation': 'TTL-based + event-driven'
```

invalidation'

```
},  
  
'performance_optimization': {  
    'batching': 'Automatic request batching for throughput',  
    'connection_pooling': 'Reuse database connections',  
    'async_processing': 'Non-blocking I/O for concurrent'
```

requests',

```
'circuit_breakers': 'Fail fast when dependencies are
```

slow',

```
'request_deduplication': 'Cache identical requests
```

within time window'

```
}  
}  
  
    return serving_architecture  
  
# Step 6: Governance and Monitoring  
def design_governance_system():  
    governance_framework = {  
        'feature_lifecycle_management': {  
            'development': 'Feature development in sandbox
```

PYTHON

environment',

```
'testing': 'A/B testing framework for feature
```

validation',

```
'staging': 'Pre-production validation with subset of
```

traffic',

```
'production': 'Full rollout with monitoring and
```

alerting',

```
'deprecation': 'Graceful deprecation with migration
```

period'

```
},
```

```
'access_control': {  
  'authentication': 'OIDC/SAML integration with corporate
```

identity',

```
'authorization': 'Role-based access control (RBAC)',  
'row_level_security': 'Entity-level access
```

restrictions',

```
'audit_logging': 'Complete audit trail of feature
```

access',

```
'data_classification': 'PII detection and handling'  
},
```

```
'data_quality_monitoring': {  
  'schema_monitoring': 'Detect schema drift and breaking
```

changes',

```
'distribution_monitoring': 'Feature distribution shift'
```

detection',

```
'freshness_monitoring': 'Alert on stale feature values',  
'completeness_monitoring': 'Track missing values and'
```

coverage',

```
'accuracy_monitoring': 'Compare features across'
```

environments'

```
},  
  
'usage_analytics': {  
    'feature_popularity': 'Track which features are most'
```

used',

```
'model_dependency': 'Track feature usage across models',  
'performance_impact': 'Measure feature impact on model'
```

performance',

```
'cost_attribution': 'Track compute costs by feature and'
```

team'

```

}
}

# Monitoring implementation
monitoring_system = """
# Feature monitoring and alerting
class FeatureMonitoringSystem:

    def monitor_feature_quality(self, feature_ref: FeatureRef):
        \"\"\"Monitor feature quality and alert on issues.\"\"\"

        # Get current feature statistics
        current_stats = self.compute_feature_stats(feature_ref)

        # Compare with baseline (historical stats)
        baseline_stats = self.get_baseline_stats(feature_ref)

        alerts = []

        # Distribution drift detection
        if self.detect_distribution_drift(current_stats,

```

baseline_stats):

```

        alerts.append({
            'type': 'distribution_drift',
            'severity': 'warning',
            'message': f'Feature {feature_ref.name}

```

distribution has shifted'

```

    })

    # Freshness check
    if self.check_feature_freshness(feature_ref):
        alerts.append({
            'type': 'stale_data',
            'severity': 'critical',
            'message': f'Feature {feature_ref.name} is

```

stale'

```
})  
  
# Schema validation
```

The PYTHON code:

```
if self.validate_feature_schema(feature_ref):  
    alerts.append({  
        'type': 'schema_violation',  
        'severity': 'critical',  
        'message': f'Feature {feature_ref.name} schema
```

PYTHON

validation failed'

```
})  
  
# Send alerts  
for alert in alerts:  
    self.send_alert(alert, feature_ref.owner)  
  
def track_feature_lineage(self, feature_ref: FeatureRef):  
    \"""\"Track feature lineage and dependencies.\"""  
  
    lineage = {  
        'upstream_sources':
```

PYTHON

```
self.get_upstream_sources(feature_ref),
```

```
'downstream_models':
```

```
self.get_downstream_models(feature_ref),
```

```
'transformation_logic':
```

```
self.get_transformation_code(feature_ref),
```

```
'last_updated': datetime.now()
    }

    # Store lineage information
    self.lineage_store.store(feature_ref.name, lineage)

    return lineage

"""

return governance_framework, monitoring_system
```

Pro Tip When discussing feature stores in interviews,

emphasize the business value: reducing time-to-market for ML models, improving model reliability through consistent features, and enabling collaboration across data science teams. The technical architecture is important, but the business impact is what matters to the company.

□ Checkpoint

1. How do you balance completeness vs. time constraints when working through interview questions?
2. What's the difference between a good technical solution and a great business-aligned solution?
3. How do you demonstrate both breadth and depth in a 45-minute conversation?

Portfolio Project Planning

⚡ TL;DR

Build 2-3 substantial projects that demonstrate different aspects of data engineering: pipeline building, system design, and business impact - Choose projects that tell a story about your growth and interests — not just technical skills - Document everything: the problem you're solving, your decision process, trade-offs made, and lessons learned - Make your projects discoverable and impressive in 30 seconds, but deep enough to discuss for 30 minutes

Your portfolio is your chance to show, not just tell, what kind of data engineer you are. Great portfolio projects demonstrate three things: technical competence, business thinking, and the ability to see projects through to completion.

Effective portfolios don't just show technical skills — they tell a story about how you approach problems and create value.

Portfolio Strategy Framework

The Three-Project Rule

```
def design_portfolio_strategy():
    portfolio_structure = {
        'project_1_pipeline_engineering': {
            'focus': 'End-to-end data pipeline with real data',
            'demonstrates': 'Technical depth in data processing',
            'technologies': 'Modern data stack (dbt, Airflow, cloud
```

PYTHON

warehouse)',

```
        'complexity': 'Production-ready with monitoring and
```

testing',

```
'time_investment': '3-4 weeks'
},

'project_2_system_design': {
  'focus': 'Design and implement a data system from
```

scratch',

```
'demonstrates': 'Architecture thinking and system
```

building',

```
'technologies': 'Distributed systems, APIs, scalability
```

patterns',

```
'complexity': 'Handle realistic scale and constraints',
  'time_investment': '4-6 weeks'
},

'project_3_business_impact': {
  'focus': 'Analytics or ML project that drives
```

decisions',

```
'demonstrates': 'Business thinking and communication
```

skills',

```
'technologies': 'Analysis tools, ML platforms,
```

visualization',

```
'complexity': 'Real insights with actionable
```

recommendations',


```

    'time_investment': '2-3 weeks'
    }
}

# Portfolio principles
guiding_principles = {
    'show_progression': 'Projects should show increasing

```

sophistication',

```

    'demonstrate_range': 'Cover different aspects of data

```

engineering',

```

    'include_real_data': 'Work with messy, real-world datasets',
    'document_decisions': 'Explain why you made specific

```

choices',

```

    'make_it_discoverable': 'Easy to find and understand

```

quickly',

```

    'enable_deep_dives': 'Enough detail for extended technical

```

discussions'

```

    }

    return portfolio_structure, guiding_principles

```

Project 1: Production-Grade Data Pipeline

Concept: "Build a complete data pipeline that processes real e-commerce data to power business analytics"

```
def design_pipeline_project():  
    project_specification = {  
        'business_context': {  
            'problem': 'E-commerce company needs unified customer
```

analytics',

```
    'data_sources': [  
        'Shopify transactions (API)',  
        'Google Analytics (API)',  
        'Customer service tickets (CSV exports)',  
        'Email marketing data (Mailchimp API)',
```

```
    'Inventory data (PostgreSQL)'  
    ],  
    'stakeholders': 'Marketing team, executives, customer
```

success',

```
    'success_criteria': 'Daily updated dashboards, <4 hour
```

data freshness'

```
    },  
  
    'technical_implementation': {  
        'ingestion_layer': {  
            'api_connectors': 'Custom Python scripts with retry
```

logic',

```
    'file_processing': 'Automated CSV ingestion with
```

validation',

```
    'change_data_capture': 'PostgreSQL CDC for inventory
```

updates',

```
'orchestration': 'Airflow DAGs with dependency
```

management',

```
'error_handling': 'Dead letter queues and manual
```

review process'

```
},  
  
'processing_layer': {  
    'raw_storage': 'S3 data lake with organized folder
```

structure',

```
'transformation_tool': 'dbt for SQL
```

transformations',

```
'data_modeling': 'Dimensional model with fact and
```

dimension tables',

```
'data_quality': 'Great Expectations for automated
```

testing',

```
'performance_optimization': 'Incremental models and
```

materialized views'

```

},

'serving_layer': {
    'warehouse': 'Snowflake or BigQuery for analytics',
    'dashboard': 'Tableau or Looker for business users',
    'api': 'FastAPI for programmatic access',
    'monitoring': 'Custom dashboard for pipeline

```

health',

```

'documentation': 'Automated data catalog with

```

lineage'

```

}
},

'project_scope': {
    'timeline': '4 weeks full-time or 8 weeks part-time',
    'deliverables': [
        'Complete data pipeline with 5+ data sources',
        '15+ dimensional model tables',
        '10+ business dashboards',
        'Comprehensive testing and monitoring',
        'Full documentation and deployment guides'
    ],
    'technologies': [
        'Python (pandas, requests, sqlalchemy)',
        'dbt (transformation and testing)',
        'Airflow (orchestration)',
        'Snowflake/BigQuery (warehouse)',
        'Tableau/Looker (visualization)',
        'Docker (containerization)',
        'GitHub Actions (CI/CD)'
    ]
}

}

# Implementation approach
implementation_phases = {
    'week_1_setup_and_ingestion': [
        'Set up development environment and cloud accounts',
        'Build API connectors for Shopify and Google Analytics',

```

The corresponding SQL query:

```

'Implement file-based ingestion for CSV data',
    'Set up basic Airflow DAGs for orchestration',
    'Create initial data lake structure in S3'
],

'week_2_transformation_and_modeling': [
    'Design dimensional model for e-commerce analytics',
    'Implement dbt models for data transformation',
    'Add data quality tests with Great Expectations',
    'Set up incremental processing for performance',
    'Create staging and mart layers'
],

'week_3_serving_and_visualization': [
    'Load transformed data into cloud warehouse',
    'Build business dashboards for key metrics',
    'Create API endpoints for programmatic access',
    'Implement monitoring and alerting',
    'Set up automated documentation generation'
],

'week_4_production_readiness': [
    'Add comprehensive error handling and retry logic',
    'Implement data quality monitoring and alerting',
    'Create deployment and rollback procedures',
    'Write complete documentation and runbooks',
    'Performance testing and optimization'
]
}

return project_specification, implementation_phases

# Sample dbt model to include in portfolio
SAMPLE_DBT_MODEL = """
-- models/marts/marketing/customer_lifetime_value.sql
{{
    config(
        materialized='table',
        indexes=[
            {'columns': ['customer_id'], 'unique': False},
            {'columns': ['customer_segment'], 'unique': False}
        ]
    )
}}

WITH customer_orders AS (
    SELECT
        customer_id,

```

```
MIN(order_date) AS first_order_date,  
MAX(order_date) AS last_order_date,  
COUNT(*) AS total_orders,  
SUM(order_amount) AS total_revenue,  
AVG(order_amount) AS avg_order_value,  
{{ datediff('first_order_date', 'last_order_date', 'day') }}
```

AS customer_lifespan_days

```
FROM {{ ref('fact_orders') }}  
  WHERE order_status = 'completed'  
  GROUP BY customer_id  
)  
  
customer_segments AS (  
  SELECT  
    *,  
    CASE  
      WHEN total_orders = 1 THEN 'one_time_buyer'  
      WHEN total_orders BETWEEN 2 AND 5 THEN 'repeat_buyer'  
      WHEN total_orders BETWEEN 6 AND 15 THEN 'frequent_buyer'  
      ELSE 'vip_customer'  
    END AS customer_segment,  
  
    CASE  
      WHEN customer_lifespan_days <= 30 THEN 'new'
```

SQL

The corresponding SQL query:

```

    WHEN customer_lifespan_days <= 365 THEN 'established'
      ELSE 'long_term'
    END AS tenure_segment

    FROM customer_orders
  )

  SELECT
    cs.*,
    dc.email,
    dc.registration_date,
    dc.marketing_opt_in,

    -- Calculate predicted CLV (simplified model)
    CASE
      WHEN customer_segment = 'vip_customer'
      THEN total_revenue * 1.5
      WHEN customer_segment = 'frequent_buyer'
      THEN total_revenue * 1.2
      ELSE total_revenue
    END AS predicted_lifetime_value,

    -- Churn risk score (simplified)
    CASE
      WHEN {{ datediff('last_order_date', 'CURRENT_DATE', 'day')

```

}} > 365

```

    THEN 'high_risk'
  WHEN {{ datediff('last_order_date', 'CURRENT_DATE', 'day')

```

}} > 180

SQL

```

THEN 'medium_risk'
      ELSE 'low_risk'
    END AS churn_risk,

    CURRENT_TIMESTAMP AS calculated_at

FROM customer_segments cs
JOIN {{ ref('dim_customer') }} dc ON cs.customer_id = dc.customer_id

-- Data quality tests
{{ config(
    post_hook=[
        "{{ test_not_null('customer_id') }}",
        "{{ test_relationships('customer_id', ref('dim_customer')),"

```

'customer_id') }}"

```

]
) }}
"""

```

Project 2: Real-Time System Design and Implementation

Concept: "Build a real-time analytics platform for social media monitoring"

PYTHON

```

def design_realtime_project():
    project_specification = {
        'business_context': {
            'problem': 'Brand needs to monitor social media mentions

```

in real-time',

```

    'use_cases': [
        'Crisis detection and rapid response',
        'Influencer identification and outreach',
        'Competitive intelligence gathering',
        'Customer sentiment tracking',
        'Campaign performance monitoring'
    ],
    'success_criteria': 'Sub-minute latency for trending

```

topic detection'

```
},
```

```
  'technical_requirements': {  
    'data_sources': [  

```

```
      'Twitter API (streaming)',  
      'Reddit API (polling)',  
      'News RSS feeds',  
      'YouTube comments API',  
      'TikTok hashtag monitoring'  
    ],  
    'processing_requirements': [  
      'Real-time sentiment analysis',  
      'Entity extraction (brands, people, places)',  
      'Trending topic detection',  
      'Anomaly detection for viral content',  
      'Cross-platform correlation'  
    ],  
    'serving_requirements': [  
      'Real-time dashboard updates',  
      'Alerting and notification system',  
      'Historical analysis interface',  
      'API for external integrations',  
      'Mobile app compatibility'  
    ]  
  },  
  
  'architecture_design': {  
    'ingestion_layer': {  
      'technology': 'Apache Kafka for message streaming',  
      'producers': 'Python services for each data source'  

```

API',

```
  'topics': 'Separate topics by platform and content'
```

type',

```
  'partitioning': 'Partition by brand/keyword for'
```

parallel processing',

```
'retention': '7 days for reprocessing capability'
},

'processing_layer': {
  'stream_processing': 'Apache Flink for real-time
```

analytics',

```
'ml_inference': 'TensorFlow Serving for sentiment
```

analysis',

```
'nlp_pipeline': 'spaCy for entity extraction',
'aggregation_windows': 'Tumbling and sliding
```

windows',

```
'state_management': 'Flink state backend for
```

aggregations'

```
},

'storage_layer': {
  'real_time_store': 'Redis for live dashboard data',
  'time_series_db': 'InfluxDB for metrics and
```

monitoring',

```
'document_store': 'Elasticsearch for full-text
```

search',

```
'data_lake': 'S3 for long-term storage and batch
```

analysis',

```
'metadata_store': 'PostgreSQL for configuration and
```

users'

```
},  
  
'serving_layer': {  
    'api_gateway': 'FastAPI with WebSocket support',  
    'dashboard': 'React frontend with real-time
```

updates',

```
'alerting': 'Custom notification service',  
'monitoring': 'Prometheus + Grafana for system
```

metrics',

```
'deployment': 'Kubernetes for orchestration and
```

scaling'

```
}  
    }  
}  
  
# Sample Flink processing job
```

The PYTHON code:

```
flink_processing_example = """  
// Real-time sentiment aggregation with Flink  
public class SocialMediaSentimentProcessor {  
  
    public static void main(String[] args) throws Exception {  
        StreamExecutionEnvironment env =
```

PYTHON

```
StreamExecutionEnvironment.getExecutionEnvironment();
```

```
// Configure for exactly-once processing
env.enableCheckpointing(60000); // 1 minute checkpoints
env.setStateBackend(new
```

```
RocksDBStateBackend("s3://checkpoints/"));
```

```
// Define Kafka source
FlinkKafkaConsumer<SocialMediaPost> source = new
```

```
FlinkKafkaConsumer<>(  
    "social_media_posts",  
    new SocialMediaPostDeserializer(),  
    kafkaProps  
);  
  
DataStream<SocialMediaPost> posts =
```

```
env.addSource(source);
```

```
// Real-time sentiment analysis  
DataStream<SentimentResult> sentimentResults = posts  
    .map(new SentimentAnalysisFunction())  
    .assignTimestampsAndWatermarks(  
        WatermarkStrategy.  
            forBoundedOutOfOrderness(  
                Duration.ofSeconds(10)  
            ).withTimestampAssigner((result, timestamp) ->  
                result.getTimestamp())
```

PYTHON

```
forBoundedOutOfOrderness(  
    Duration.ofSeconds(10)  
).withTimestampAssigner((result, timestamp) ->  
    result.getTimestamp())
```

```
result.getTimestamp())
```

```
);

// Windowed aggregations for trending analysis
DataStream<TrendingTopicResult> trendingTopics =
```

sentimentResults

```
.keyBy(result -> result.getBrand())
```

```
.window(TumblingEventTimeWindows.of(Time.minutes(5)))
```

```
.aggregate(new TrendingTopicAggregateFunction())
.filter(result -> result.getMentionCount() > 10); //
```

PYTHON

Filter for significant trends

```
// Sink to Redis for real-time dashboard
trendingTopics.addSink(new RedisSink<>
```

```
("trending_topics"));
```

```
// Sink to Elasticsearch for searchable history
sentimentResults.addSink(new ElasticsearchSink<>
```

```
("sentiment_history"));
```

```
env.execute("Social Media Sentiment Processing");
    }
}

// Custom sentiment analysis function
public static class SentimentAnalysisFunction
    implements MapFunction<SocialMediaPost, SentimentResult>
```

PYTHON

```
{
```

```
private transient SentimentAnalyzer analyzer;
```

```
@Override  
public void open(Configuration config) {  
    // Initialize ML model  
    this.analyzer = new
```

```
SentimentAnalyzer("models/sentiment_model");
```

```
}
```

```
@Override  
public SentimentResult map(SocialMediaPost post) {  
    double sentimentScore =
```

```
analyzer.analyze(post.getText());
```

```
    return SentimentResult.builder()  
        .postId(post.getId())  
        .platform(post.getPlatform())  
        .brand(extractBrand(post.getText()))  
        .sentiment(sentimentScore)  
        .timestamp(post.getTimestamp())  
        .influence(calculateInfluence(post))  
        .build();  
}  
}  
""  
  
return project_specification, flink_processing_example
```

Project 3: Analytics Impact Project

Concept: "Customer churn prediction and intervention system that drives business results"

```
def design_analytics_project():
    project_specification = {
        'business_context': {
            'problem': 'SaaS company losing 5% of customers monthly
```

to churn',

```
        'business_impact': '$2M annual revenue at risk',
        'stakeholders': 'Customer Success, Sales, Product
```

teams',

```
        'current_process': 'Reactive - only know about churn
```

after it happens',

```
        'desired_outcome': 'Proactive intervention to reduce
```

churn by 20%'

```
    },
    'analytical_approach': {
        'data_exploration': [
            'Customer usage patterns and engagement metrics',
            'Support ticket volume and sentiment analysis',
            'Feature adoption and product usage depth',
            'Billing and payment behavior patterns',
            'Demographic and firmographic analysis'
        ],
        'feature_engineering': [
            'Usage trend analysis (increasing/decreasing
```

activity)',


```

'Support interaction sentiment and frequency',
    'Feature adoption velocity and breadth',
    'Time-based behavioral patterns',
    'Cohort analysis and lifecycle stage identification'
],

'model_development': [
    'Supervised learning for churn prediction',
    'Survival analysis for time-to-churn',
    'Clustering for customer segmentation',
    'Feature importance analysis',
    'Model interpretability for business insights'
],

'intervention_design': [
    'Risk score integration with CRM',
    'Automated alerting for Customer Success',
    'Personalized retention campaigns',
    'Product feature recommendations',
    'Proactive support outreach'
]
},

```

```

'technical_implementation': {
    'data_pipeline': 'Python + SQL for data preparation',
    'analysis_environment': 'Jupyter notebooks for

```

exploration',

```

'ml_platform': 'MLflow for experiment tracking',
'model_serving': 'REST API for real-time scoring',
'monitoring': 'Model drift detection and retraining',
'visualization': 'Streamlit app for stakeholder

```

interaction'

```

}
}

# Sample analysis code to include
analysis_example = """
# Customer churn analysis with feature engineering
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, roc_auc_score
import plotly.express as px
import mlflow

class ChurnAnalysisEngine:

    def __init__(self, data_path: str):
        self.customer_data = pd.read_sql("""\n""")
        SELECT
            c.customer_id,
            c.signup_date,
            c.subscription_tier,
            c.company_size,
            c.industry,
            -- Usage metrics
            u.monthly_active_days,
            u.feature_usage_breadth,
            u.api_calls_per_month,
            -- Support metrics
            s.ticket_count_90d,
            s.avg_resolution_time,
            s.satisfaction_score,
            -- Financial metrics
            b.months_subscribed,
            b.total_revenue,
            b.payment_failures_90d,
            -- Target variable
            CASE WHEN c.churned_date IS NOT NULL THEN 1 ELSE

```

0 END as churned

FROM customers c

```
LEFT JOIN usage_metrics u ON c.customer_id =
```

SQL

u.customer_id

```
LEFT JOIN support_metrics s ON c.customer_id =
```

SQL

s.customer_id

```
LEFT JOIN billing_metrics b ON c.customer_id =
```

SQL

b.customer_id

```
WHERE c.signup_date >= '2023-01-01'  
      \"\"\", connection)
```

SQL

```
def engineer_features(self, df: pd.DataFrame) ->
```

pd.DataFrame:

```
\"\"\"Create predictive features for churn
```

```
modeling.\"\"\"
```

```
# Engagement trend features  
df['usage_trend'] = df['monthly_active_days'] /
```

```
df['months_subscribed']
```

```
df['feature_adoption_rate'] =
```

```
df['feature_usage_breadth'] / 10 # Normalize
```

```
df['support_intensity'] = df['ticket_count_90d'] /
```

```
max(df['months_subscribed'], 1)
```

```
# Risk indicators  
df['low_engagement'] = (df['monthly_active_days'] <
```

```
5).astype(int)
```

```
df['payment_issues'] = (df['payment_failures_90d'] >
```

```
0).astype(int)
```

```
df['high_support_load'] = (df['ticket_count_90d'] >
```

```
df['ticket_count_90d'].quantile(0.8)).astype(int)
```

```
# Customer value indicators  
df['revenue_per_month'] = df['total_revenue'] /
```

```
df['months_subscribed']
```

```
df['is_enterprise'] = (df['company_size'] >
```

```
1000).astype(int)
```

```
# Interaction effects  
df['engagement_x_satisfaction'] =
```

```
df['monthly_active_days'] * df['satisfaction_score']
```

```
df['size_x_usage'] = df['company_size'] *
```

```
df['api_calls_per_month']
```

```
return df
```

PYTHON

```
def train_churn_model(self, df: pd.DataFrame):  
    \"\"\"Train churn prediction model with MLflow
```

```
tracking.\"\"\"
```

```
with mlflow.start_run():  
    # Prepare features and target  
    feature_cols = [col for col in df.columns if col not
```

```
in ['customer_id', 'churned']]
```

```

X = df[feature_cols]
y = df['churned']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# Train model
model = RandomForestClassifier(
    n_estimators=100,
    max_depth=10,
    min_samples_split=20,
    class_weight='balanced',
    random_state=42
)

model.fit(X_train, y_train)

# Evaluate model
y_pred_proba = model.predict_proba(X_test)[:, 1]
auc_score = roc_auc_score(y_test, y_pred_proba)

# Log metrics and model
mlflow.log_param("n_estimators", 100)
mlflow.log_param("max_depth", 10)
mlflow.log_metric("auc_score", auc_score)
mlflow.sklearn.log_model(model, "churn_model")

# Feature importance analysis
feature_importance = pd.DataFrame({
    'feature': feature_cols,
    'importance': model.feature_importances_
}).sort_values('importance', ascending=False)

print(f"Model AUC Score: {auc_score:.3f}")
print("\nTop 10 Most Important Features:")
print(feature_importance.head(10))

return model, feature_importance

def generate_business_insights(self, df: pd.DataFrame,

```

model, feature_importance: pd.DataFrame):

```
\\"\\\\"Generate actionable business insights from the
```

```
analysis.\\\\"\\\\"
```

```
# Churn risk segments
df['churn_probability'] = model.predict_proba(
    df[[col for col in df.columns if col not in
```

```
['customer_id', 'churned']]])
```

```
)[:, 1]

df['risk_segment'] = pd.cut(
    df['churn_probability'],
    bins=[0, 0.3, 0.7, 1.0],
    labels=['Low Risk', 'Medium Risk', 'High Risk']
)

# Business insights
insights = {
    'churn_rate_by_segment': df.groupby('risk_segment')
```

```
['churned'].mean(),
```

```
'revenue_at_risk': df.groupby('risk_segment')
```

```
['total_revenue'].sum(),
```

```
'top_churn_drivers': feature_importance.head(5)
```

```
['feature'].tolist(),
```

```
'intervention_priorities':
```

```
self.identify_intervention_opportunities(df)
```

```

    }

    return insights

def identify_intervention_opportunities(self, df:

```

pd.DataFrame) -> Dict:

```

    \"""Identify specific intervention

```

```

    opportunities.\"\"\"

```

```

    high_risk_customers = df[df['risk_segment'] == 'High

```

Risk']

```

    return {
        'low_engagement_customers':

```

```

    len(high_risk_customers[high_risk_customers['low_engagement'] == 1]),

```

```

    'payment_issue_customers':

```

```

    len(high_risk_customers[high_risk_customers['payment_issues'] == 1]),

```

```

    'high_support_customers':

```

len(high_risk_customers[high_risk_customers['high_support_load'] == 1]),

```

    'total_revenue_at_risk':

```

```

    high_risk_customers['total_revenue'].sum()

```



```

}
"""

return project_specification, analysis_example

```

Portfolio Presentation Strategy

```

def design_portfolio_presentation():
    presentation_framework = {
        'github_organization': {
            'repository_structure': """
portfolio/
├─ README.md (portfolio overview with links)
├─ project-1-ecommerce-pipeline/
│   ├─ README.md (project overview)
│   ├─ docs/ (architecture diagrams, decisions)
│   ├─ src/ (all source code)
│   ├─ tests/ (comprehensive testing)
│   └─ deploy/ (deployment configurations)
└─ project-2-realtime-analytics/
    └─ [same structure]

```

```

├─ project-3-churn-analysis/
│   └─ [same structure]
└─ assets/ (shared diagrams, resume, etc.)
""",

    'readme_template': """
# Data Engineering Portfolio

## Overview
This portfolio demonstrates my expertise in data

```

engineering through three

comprehensive projects that showcase different aspects

of modern data systems.

Projects

1. Production E-commerce Data Pipeline

****Business Impact:**** Unified customer analytics enabling

25% faster

decision-making for marketing campaigns.

****Technologies:**** Python, dbt, Airflow, Snowflake,

Tableau

****Key Features:**** Real-time CDC, automated testing,

comprehensive monitoring

[View Project →](./project-1-ecommerce-pipeline)

2. Real-time Social Media Analytics Platform

****Business Impact:**** Sub-minute crisis detection system

for brand monitoring.

****Technologies:**** Kafka, Flink, Elasticsearch, Redis,

Kubernetes

****Key Features:**** Stream processing, ML inference, auto-

scaling

[View Project →](./project-2-realtime-analytics)

3. Customer Churn Prediction & Intervention

****Business Impact:**** 20% reduction in customer churn

through proactive intervention.

```
**Technologies:** Python, MLflow, Streamlit, PostgreSQL  
**Key Features:** Predictive modeling, business
```

insights, stakeholder tools

```
[View Project →](./project-3-churn-analysis)  
  
## About Me  
[2-3 sentences about your background and interests in
```

data engineering]

```
## Contact  
  - LinkedIn: [your-profile]  
  - Email: [your-email]  
  ""  
  
,  
  
'project_documentation': {  
  'architecture_decisions': 'Document why you chose
```

specific technologies',

```
'trade_offs_made': 'Explain what you optimized for and
```

what you sacrificed',

```
'lessons_learned': 'What would you do differently next
```

time?',

```
'business_impact': 'Quantify the value your project
```

creates',

```
'technical_depth': 'Include code samples that show your
```

skill level'

```
},  
  
'interview_preparation': {  
    '30_second_overview': 'Elevator pitch for each project',  
    '5_minute_deep_dive': 'Technical walkthrough with key
```

decisions',

```
'15_minute_discussion': 'Detailed architecture and
```

trade-offs',

```
'extension_questions': 'How would you scale this? What
```

would you add?'

```
}  
    }  
  
    return presentation_framework  
  
Pro Tip The best portfolio projects solve real problems with
```

real constraints. Don't build toy examples with clean data and unlimited resources. Use messy datasets, impose realistic time and budget constraints, and document how you dealt with the challenges. This shows you can handle production engineering, not just academic exercises.

□ **Checkpoint**

1. How do you balance technical depth vs. business impact in your portfolio projects?
2. What's the difference between a good project and a portfolio- worthy project?
3. How do you document your projects to enable both quick assessment and deep technical discussions?

⚡ TL;DR

Stay current with emerging technologies but don't chase every trend — focus on patterns - Quantum computing, edge computing, and blockchain will impact data engineering, but the fundamentals remain the same - The future of data engineering is about higher-level abstractions, not lower-level infrastructure management - Develop a learning framework to quickly evaluate and adopt new technologies

The data engineering landscape evolves rapidly. New frameworks, platforms, and paradigms emerge constantly. Successful data engineers don't try to learn every new technology — they develop frameworks for quickly evaluating what's worth their time and understanding how new developments fit into broader patterns.

Technology Evolution Patterns

The Data Engineering Technology Stack Evolution

```
def analyze_technology_evolution():
    evolution_patterns = {
        'infrastructure_evolution': {
            '2010s': 'On-premise Hadoop clusters, manual scaling',
            '2020s': 'Cloud-managed services, auto-scaling',
            '2030s_prediction': 'Serverless-first, AI-optimized
```

PYTHON

infrastructure'

```
    },
    'processing_evolution': {
        '2010s': 'MapReduce batch processing',
        '2020s': 'Stream processing, micro-batch, unified
```

engines',

```
'2030s_prediction': 'AI-assisted query optimization,
```

self-tuning systems'

```
},  
  
  'interface_evolution': {  
    '2010s': 'Java/Scala programming, XML configuration',  
    '2020s': 'SQL-first, Python notebooks, declarative
```

configs',

```
'2030s_prediction': 'Natural language interfaces, visual
```

programming'

```
},  
  
  'governance_evolution': {  
    '2010s': 'Manual documentation, ad-hoc governance',  
    '2020s': 'Automated lineage, policy-as-code',  
    '2030s_prediction': 'AI-driven data discovery, self-
```

governing systems'

```
}  
}  
  
# Technology adoption framework  
adoption_framework = {  
  'emerging': 'Bleeding edge, high risk, potential high
```

reward',

```
'early_adoption': 'Proven in specific use cases, growing
```

ecosystem',

```
'mainstream': 'Widely adopted, stable, good tooling',  
'mature': 'Battle-tested, well-understood, potentially
```

stagnating',

```
'legacy': 'Still functional but being replaced'  
}  
  
return evolution_patterns, adoption_framework
```

Quantum Computing Impact on Data Processing

Quantum Algorithms for Data Engineering

```
def explore_quantum_impact():  
    quantum_applications = {  
        'optimization_problems': {  
            'description': 'Quantum algorithms for complex
```

PYTHON

optimization',

```
'data_engineering_use_cases': [  
    'Query optimization with exponential search spaces',  
    'Resource allocation for distributed processing',  
    'Feature selection for high-dimensional datasets',  
    'Network routing optimization for data transfer'  
],  
'timeline': '2030-2035 for practical applications',  
'current_status': 'Research phase, limited quantum
```

SQL

computers available'

```
},  
  
'cryptography_and_security': {  
    'description': 'Quantum-safe encryption and quantum key
```


distribution',

```
'data_engineering_impact': [  
    'Need for quantum-resistant encryption algorithms',  
    'Secure multi-party computation for sensitive data',  
    'Enhanced privacy-preserving analytics',  
    'Quantum random number generation for sampling'  
],  
'timeline': '2025-2030 for quantum-safe encryption',  
'current_status': 'NIST standardizing quantum-resistant
```

algorithms'

```
},  
  
'machine_learning_acceleration': {  
    'description': 'Quantum ML algorithms for pattern
```

recognition',

```
'potential_benefits': [  
    'Exponential speedup for certain ML problems',  
    'Quantum neural networks for feature extraction',  
    'Quantum clustering algorithms',  
    'Enhanced dimensionality reduction techniques'  
],  
'limitations': [  
    'Only specific problem types benefit',  
    'High error rates in current quantum computers',  
    'Limited quantum memory and coherence time'  
]  
]
```

```

}
}

# Practical implications for data engineers
practical_implications = {
    'near_term_2025_2030': [
        'Learn quantum-safe cryptography principles',
        'Understand quantum algorithm complexity',
        'Evaluate quantum cloud services (IBM, Google, AWS)',
        'Monitor quantum ML research developments'
    ],

    'medium_term_2030_2040': [
        'Hybrid classical-quantum algorithm design',
        'Quantum-optimized data structures',
        'Integration of quantum accelerators in data pipelines',
        'Quantum-enhanced security protocols'
    ],

    'preparation_strategies': [
        'Focus on linear algebra and statistics fundamentals',
        'Understand computational complexity theory',
        'Learn about quantum error correction',
        'Experiment with quantum simulators and cloud services'
    ]
}

return quantum_applications, practical_implications

```

Edge Computing and Distributed Data Processing

Data Processing at the Network Edge

```

def explore_edge_computing_patterns():
    edge_computing_architectures = {
        'iot_data_processing': {
            'pattern': 'Process sensor data at edge devices before
cloud transmission',

```

PYTHON

```
'technologies': ['Apache Kafka (edge)', 'InfluxDB (time-
```

```
series)', 'TensorFlow Lite'],
```

```
'use_cases': [  
    'Predictive maintenance on manufacturing equipment',  
    'Real-time video analytics for security systems',  
    'Autonomous vehicle sensor fusion',  
    'Smart city traffic optimization'  
],  
'data_engineering_challenges': [  
    'Limited compute and storage at edge nodes',  
    'Intermittent connectivity to central systems',  
    'Data synchronization across distributed edge
```

```
locations',
```

```
'Managing software updates across thousands of
```

```
devices'
```

```
]  
},  
  
'federated_analytics': {  
    'pattern': 'Analyze data across multiple edge locations
```

```
without centralization',
```

```
'technologies': ['Federated learning frameworks',
```

```
'Secure aggregation protocols'],
```

```
'benefits': [  
    'Privacy preservation (data never leaves edge)',  
    'Reduced bandwidth requirements',  
    'Lower latency for local insights',  
    'Compliance with data residency requirements'  
],  
'implementation_approaches': [  
    'Aggregate statistics without raw data sharing',  
    'Model training across distributed datasets',  
    'Secure multi-party computation protocols',  
    'Differential privacy for statistical queries'
```

```
]  
},  
  
'edge_ml_inference': {  
    'pattern': 'Deploy ML models for inference at edge'
```

locations',

```
'optimization_techniques': [  
    'Model quantization and pruning',  
    'Knowledge distillation for smaller models',  
    'Specialized edge hardware (TPUs, FPGAs)',  
    'Dynamic model loading based on current needs'  
],  
'data_engineering_considerations': [  
    'Feature preprocessing at edge devices',  
    'Model versioning and A/B testing at edge',  
    'Monitoring and observability for distributed
```

inference',

```

    'Handling model drift detection across edge nodes'
    ]
    }
}

# Edge-specific data engineering patterns
edge_patterns = {
    'data_locality_optimization': """
    # Example: Edge data processing with locality awareness
    class EdgeDataProcessor:

        def __init__(self, edge_location: str):
            self.location = edge_location
            self.local_storage = EdgeStorage(location)
            self.cloud_connection = CloudConnection(location)

        def process_sensor_data(self, sensor_readings:

```

```
List[SensorReading]):
```

```

# Process data locally first
processed_data = []

for reading in sensor_readings:
    # Apply edge analytics
    if self.is_anomalous(reading):
        # Send anomalies immediately to cloud
        self.cloud_connection.send_priority(reading)

    # Aggregate normal readings
    processed_data.append(self.preprocess(reading))

# Batch upload aggregated data
if len(processed_data) >= BATCH_SIZE:
    self.cloud_connection.send_batch(processed_data)
else:

```

```
self.local_storage.store_temporary(processed_data)
```

```

"""
    'intermittent_connectivity_handling': """
    # Pattern for handling unreliable network connections
    class OfflineCapableProcessor:

        def __init__(self):
            self.offline_queue = PersistentQueue("offline_data")
            self.sync_strategy = AdaptiveSyncStrategy()

        def process_data_with_offline_support(self, data):
            # Always process locally first
            result = self.local_processing(data)

            # Attempt cloud synchronization
            try:
                if self.cloud_connection.is_available():
                    # Upload queued offline data first
                    self.upload_offline_queue()

```

Upload current result:

```

    # Upload current result
        self.cloud_connection.upload(result)
    else:
        # Store for later upload
        self.offline_queue.add(result)

    except ConnectionError:
        self.offline_queue.add(result)
        self.sync_strategy.mark_failure()

    """
}

return edge_computing_architectures, edge_patterns

```

Data Systems for Decentralized/Blockchain Applications

Blockchain Integration with Traditional Data Systems

PYTHON

```
def explore_blockchain_data_patterns():  
    blockchain_data_patterns = {  
        'immutable_audit_logs': {  
            'use_case': 'Financial transactions, compliance records,
```

data lineage',

```
        'pattern': 'Hash critical events and store hashes on
```

blockchain',

PYTHON

```
'implementation': """  
class BlockchainAuditLogger:  
  
    def __init__(self, blockchain_client):  
        self.blockchain = blockchain_client  
        self.local_storage = AuditLogStorage()  
  
    def log_data_transformation(self,
```

transformation_event):

```
        # Store detailed event locally  
        event_id =
```

```
        self.local_storage.store(transformation_event)
```

```
        # Create hash for blockchain  
        event_hash = hashlib.sha256(  
            json.dumps(transformation_event,
```

```
            sort_keys=True).encode()
```

```

).hexdigest()

# Record hash on blockchain with metadata
blockchain_record = {
    'event_id': event_id,
    'event_type': 'data_transformation',
    'hash': event_hash,
    'timestamp':

```

```

transformation_event['timestamp'],

```

```

'data_source':

```

```

transformation_event['source_system']

```

```

}

self.blockchain.record_event(blockchain_record)

def verify_data_integrity(self, event_id):
    # Get event from local storage
    event = self.local_storage.get(event_id)

    # Calculate current hash
    current_hash = hashlib.sha256(
        json.dumps(event, sort_keys=True).encode()
    ).hexdigest()

    # Get hash from blockchain
    blockchain_hash =

```

PYTHON

```

self.blockchain.get_event_hash(event_id)

```

```

return current_hash == blockchain_hash
"""
'benefits': ['Tamper-proof audit trail', 'Regulatory

```

```

compliance', 'Data lineage verification'],

```



```
'challenges': ['High transaction costs', 'Limited
```

throughput', 'Energy consumption']

```
},
```

```
'decentralized_data_marketplaces': {  
  'use_case': 'Data sharing between organizations without
```

central authority',

```
'pattern': 'Smart contracts for data access and
```

payment',

```
'architecture_components': [  
  'IPFS for decentralized data storage',  
  'Smart contracts for access control and payments',  
  'Privacy-preserving analytics (secure multi-party
```

computation)',

```
'Reputation systems for data quality assessment'  
],  
'data_engineering_challenges': [  
  'Data quality verification without centralized
```

authority',

```
'Privacy-preserving data joins across
```

SQL

organizations',

```

    'Incentive mechanisms for data contribution',
        'Handling data updates in immutable systems'
    ]
},

'web3_analytics': {
    'use_case': 'Analytics for DeFi, NFTs, and decentralized

```

applications',

```

'data_sources': [
    'Blockchain transaction logs',
    'Smart contract events',
    'Off-chain application data',
    'Cross-chain bridge transactions'
],
'processing_challenges': [
    'Real-time processing of blockchain events',
    'Handling blockchain reorganizations (forks)',
    'Cross-chain data correlation',
    'Gas optimization for on-chain analytics'
]
}
}

# Integration patterns with traditional data systems
integration_patterns = {
    'hybrid_on_off_chain': """
    # Pattern: Store metadata on-chain, data off-chain
    class HybridDataSystem:

        def store_dataset(self, data, metadata):
            # Store large data in traditional storage
            storage_location = self.cloud_storage.upload(data)

            # Store metadata and hash on blockchain
            data_hash = self.calculate_hash(data)
            blockchain_metadata = {
                'storage_location': storage_location,
                'data_hash': data_hash,
                'schema_version': metadata['schema_version'],
                'access_permissions': metadata['permissions']
            }

            return

```

PYTHON

```
self.blockchain.store_metadata(blockchain_metadata)
```

```
def retrieve_verified_data(self, blockchain_id):  
    # Get metadata from blockchain
```

SQL

```
metadata =
```

```
self.blockchain.get_metadata(blockchain_id)
```

```
# Retrieve data from storage  
data =
```

SQL

```
self.cloud_storage.download(metadata['storage_location'])
```

```
# Verify integrity  
if self.calculate_hash(data) !=
```

PYTHON

```
metadata['data_hash']:
```

```
raise DataIntegrityError("Data has been tampered
```

```
with")
```

```

return data
"""

'oracle_integration': """
# Pattern: Bringing off-chain data to blockchain systems
class DataOracle:

    def __init__(self, data_source, blockchain_client):
        self.data_source = data_source
        self.blockchain = blockchain_client

    def update_price_feed(self, asset_symbol):
        # Get data from external API
        price_data =

```

```

self.data_source.get_current_price(asset_symbol)

```

```

# Validate and transform data
    validated_price = self.validate_price(price_data)

    # Update blockchain oracle contract
    transaction = self.blockchain.update_oracle(
        asset_symbol,
        validated_price,
        timestamp=int(time.time())
    )

    return transaction
"""
}

return blockchain_data_patterns, integration_patterns

```

Future Skills and Learning Framework

PYTHON

```
def develop_learning_framework():
    future_skills_framework = {
        'core_technical_skills': {
            'timeless_fundamentals': [
                'Distributed systems principles',
                'Database theory and query optimization',
                'Statistics and probability',
                'Software engineering best practices',
                'System design and architecture patterns'
            ],
            'evolving_technical_skills': [
                'AI/ML engineering and MLOps',
                'Real-time streaming and event processing',
                'Cloud-native architectures and serverless',
                'Data privacy and security engineering',
                'Observability and monitoring systems'
            ]
        },
        'business_and_communication': {
            'business_acumen': [
                'Understanding data\'s impact on business outcomes',
                'ROI analysis and cost-benefit evaluation',
                'Product thinking and user-centric design',
                'Industry-specific domain knowledge'
            ]
        },
        'leadership_skills': [
            'Technical mentoring and knowledge sharing',
            'Cross-functional collaboration',
            'Project management and delivery',
            'Strategic thinking and technology roadmapping'
        ],
        'technology_evaluation_framework': {
            'criteria_for_adoption': [
                'Problem-solution fit: Does it solve a real
problem?'
```

```
'Ecosystem maturity: Are there tools, docs,
```

community?',

```
'Total cost of ownership: Not just licensing costs',  
'Team capability: Can your team learn and support
```

it?',

```
'Vendor stability: Will the technology exist long-
```

term?'

```
],
```

PYTHON

```
'evaluation_process': ""  
def evaluate_new_technology(technology_name: str) ->
```

TechnologyAssessment:

```
assessment = TechnologyAssessment(technology_name)  
  
# 1. Problem Analysis  
assessment.problem_fit = analyze_problem_fit(  
    current_solutions=get_current_tech_stack(),  
    new_technology=technology_name,
```

```
business_requirements=get_business_requirements()
```

```
)
```

```
# 2. Technical Evaluation  
assessment.technical_score =
```

```
evaluate_technical_merits(
```

```
performance_benchmarks=run_poc_benchmarks(),
```

```
integration_complexity=assess_integration_effort(),  
al_overhead=estimate_operational_burden()
```

operation-

```
)  
  
# 3. Ecosystem Analysis  
assessment.ecosystem_maturity = analyze_ecosystem(  
    community_size=measure_community_activity(),  
    documentation_quality=assess_documentation(),  
    tooling_availability=catalog_available_tools(),  
    hiring_market=research_talent_availability()  
)  
  
# 4. Risk Assessment  
assessment.risks = identify_risks(  
    vendor_lock_in=assess_vendor_dependency(),  
    technical_debt=estimate_migration_costs(),  
    team_learning_curve=estimate_training_time()  
)  
  
# 5. ROI Calculation  
assessment.roi = calculate_roi()
```

```
implementation_costs=estimate_implementation_effort(),
```

```
operational_savings=estimate_efficiency_gains(),  
    opportunity_costs=assess_alternatives()  
)  
  
    return assessment  
    ""  
},
```

The corresponding SQL query:

```
'continuous_learning_strategy': {
  'information_sources': [
    'Technical conferences (Strata, DataEngConf, etc.)',
    'Research papers (VLDB, SIGMOD, papers with code)',
    'Engineering blogs from major tech companies',
    'Open source project repositories and issues',
    'Industry analyst reports (Gartner, Forrester)'
  ],

  'hands_on_practice': [
    'Build side projects with new technologies',
    'Contribute to open source projects',
    'Participate in hackathons and coding challenges',
    'Create content (blogs, talks) to teach others',
    'Mentor junior engineers and learn from their
```

questions'

```
],

'networking_and_community': [
  'Join professional organizations and meetups',
  'Participate in online communities (Reddit, Stack
```

Overflow)',

```
  'Attend industry conferences and workshops',
    'Build relationships with peers at other companies',
    'Engage with technology vendors and consultants'
  ]
}

return future_skills_framework
```

Pro Tip Don't try to predict the future of technology –

instead, build a learning framework that helps you adapt quickly to whatever comes next. Focus on understanding patterns and principles that transcend specific tools and technologies. The data engineers who thrive in rapidly changing environments are those who can quickly evaluate, learn, and apply new technologies within the context of their business problems.

□ Checkpoint

1. How do you balance staying current with emerging technologies vs. mastering existing ones?
2. What factors should you consider when evaluating whether to adopt a new technology?
3. How do you build learning frameworks that scale with the pace of technological change?

12.5 Career Development & Continued Learning

⚡ TL;DR

Career progression in data engineering: Individual Contributor → Senior IC → Staff → Principal → Distinguished - Each level requires different skills: Junior focuses on execution, Senior on system design, Staff+ on business impact - Build technical leadership skills early: mentoring, documentation, cross-team collaboration - Create your own learning curriculum — the field changes too fast for formal education to keep up

Career progression in data engineering isn't just about technical skills. As you advance, you'll spend less time coding and more time designing systems, influencing technical decisions, and helping others solve problems. Understanding this progression helps you prepare for each level and maximize your impact.

Career Progression Framework

The Data Engineering Career Ladder

```
def define_career_progression():
    career_levels = {
        'junior_data_engineer_l3': {
            'timeline': '0-2 years experience',
            'core_responsibilities': [
                'Build and maintain data pipelines under guidance',
                'Write SQL queries and basic Python/Scala scripts',
                'Debug data quality issues and pipeline failures',
                'Document processes and runbooks',
                'Learn team\'s technology stack and domain'
            ],
            'key_skills_to_develop': [
                'SQL proficiency (joins, window functions, CTEs)',
                'Programming fundamentals (Python/Java/Scala)',
                'Understanding of data warehouse concepts',
                'Basic cloud platform knowledge (AWS/GCP/Azure)',
                'Version control and CI/CD workflows'
            ],
            'success_metrics': [
                'Delivers assigned tasks on time',
                'Asks thoughtful questions when stuck',
                'Follows established patterns and practices',
                'Shows curiosity and willingness to learn'
            ]
        },
        'data_engineer_l4': {
            'timeline': '2-4 years experience',
            'core_responsibilities': [
                'Own end-to-end data pipelines and systems',
                'Design solutions for medium-complexity problems',
                'Mentor junior engineers and review code',
                'Participate in on-call rotation and incident
```

response',

```

'Collaborate with stakeholders on requirements'
],
'key_skills_to_develop': [
  'System design for data systems',
  'Performance optimization and troubleshooting',
  'Data modeling and schema design',
  'Streaming and real-time processing',
  'Infrastructure as code and DevOps practices'
],
'success_metrics': [
  'Independently solves complex technical problems',
  'Designs scalable and maintainable solutions',
  'Effectively mentors and helps team members',
  'Contributes to technical architecture decisions'
]
},

'senior_data_engineer_l5': {
  'timeline': '4-7 years experience',
  'core_responsibilities': [
    'Lead design of complex data systems and platforms',
    'Define technical standards and best practices',
    'Drive cross-team technical initiatives',
    'Mentor multiple engineers and guide career

```

development',

'Translate business requirements into technical

solutions'

```

],
'key_skills_to_develop': [
  'Large-scale distributed systems design',
  'Technical leadership and influence',
  'Project management and delivery',
  'Business acumen and stakeholder management',
  'Public speaking and technical communication'
],
'success_metrics': [

```

'Delivers complex projects with significant business

impact',

```
'Raises the technical bar across the team',  
'Successfully influences technical decisions across
```

teams',

```
'Develops other engineers into strong contributors'  
  ]  
,  
  
'staff_data_engineer_l6': {  
  'timeline': '7-12 years experience',  
  'core_responsibilities': [  
    'Set technical direction for data platform and
```

strategy',

```
'Lead large, cross-functional initiatives',  
'Represent engineering in business planning and
```

strategy',

```
'Identify and solve organization-level technical
```

problems',

```
'Build relationships with external partners and
```

vendors'

```
],
'key_skills_to_develop': [
    'Strategic thinking and technical vision',
    'Organizational influence and change management',
    'Deep expertise in specialized domains',
    'Executive communication and presentation',
    'Industry knowledge and external networking'
],
'success_metrics': [
    'Drives technical decisions that impact company
```

strategy',

```
'Successfully leads initiatives spanning multiple
```

teams',

```
'Recognized as technical expert inside and outside
```

company',

```
'Develops other technical leaders and senior
```

engineers'

```
]
},

'principal_distinguished_l7_l8': {
    'timeline': '12+ years experience',
    'core_responsibilities': [
        'Define industry-leading technical approaches',
        'Influence product and business strategy through
```

technology',

```

'Represent company in technical community',
  'Solve problems that don\'t have known solutions',
  'Build and lead technical organizations'
],
'key_skills_to_develop': [
  'Industry-wide technical thought leadership',
  'Company-wide organizational influence',
  'Deep research and innovation capabilities',
  'Board-level communication and strategic thinking',
  'Building and scaling technical teams'
],
'success_metrics': [
  'Creates new technical approaches adopted by

```

industry',

```

'Influences company direction through technical

```

innovation',

```

'Recognized as industry expert and thought leader',
  'Develops organizational capabilities and culture'
  ]
}
}

return career_levels

```

Technical Leadership Development

Building Influence and Impact Beyond Coding

```
def develop_technical_leadership():
    leadership_skills_framework = {
        'technical_communication': {
            'written_communication': [
                'Architecture decision records (ADRs)',
                'Technical design documents and RFCs',
                'Incident postmortems and learning documents',
                'Code reviews with constructive feedback',
                'Documentation that enables self-service'
            ],
            'verbal_communication': [
                'Presenting technical concepts to non-technical
```

stakeholders',

```
'Leading technical discussions and design reviews',
'Facilitating architecture meetings and decision-
```

making',

```
'Teaching and mentoring through pair programming',
'Speaking at conferences and meetups'
],

'communication_templates': """
# Architecture Decision Record (ADR) Template
# Decision: [Title]

## Status
[Proposed | Accepted | Deprecated | Superseded]

## Context
What is the issue that we're seeing that is motivating
```

this decision or change?

Decision

What is the change that we're proposing and/or doing?

Consequences

What becomes easier or more difficult to do because of

this change?

Alternatives Considered

What other options did we consider and why did we reject

them?

Technical Design Document Template

Problem Statement

Clear definition of the problem we're solving and why it

matters.

Requirements

Functional and non-functional requirements with success

criteria.

Proposed Solution

High-level approach with architecture diagrams and data

flow.

Detailed Design

Component-level design with APIs, data models, and

algorithms.

```
## Implementation Plan
Timeline, milestones, risks, and dependencies.

## Monitoring and Operations
How we'll know the system is working and how to operate
```

it.

```
"""
},

'mentoring_and_development': {
```

The PYTHON code:

```
'mentoring_approaches': [
    'Pair programming on complex problems',
    'Code review with teaching focus',
    'Career development and goal setting',
    'Technical skill assessment and growth planning',
    'Project assignment based on learning objectives'
],

'mentoring_framework': """
class TechnicalMentor:

    def develop_mentee(self, mentee: Engineer,
```

PYTHON

timeframe: str):

```
# Assess current capabilities
assessment =
```

```
self.assess_technical_skills(mentee)
```

PYTHON

```
# Identify growth opportunities
growth_areas = self.identify_skill_gaps(
    current_level=assessment,
    target_level=mentee.career_goals,
    company_needs=self.get_company_priorities()
)

# Create development plan
development_plan = self.create_learning_plan(
    growth_areas=growth_areas,
```

```
learning_style=mentee.preferred_learning_style,
```

PYTHON

```
available_time=timeframe
)

# Execute with regular check-ins
self.schedule_regular_mentoring(
    frequency="weekly",
    format="mix of technical_sessions and
```

```
career_guidance"
```

PYTHON

```
)

def provide_effective_feedback(self, code_review:
```

```
CodeReview):
```

```
feedback_framework = {
    'positive_reinforcement': 'Point out what
```

```
they did well and why',
```

```
'improvement_areas': 'Suggest specific,
```

```
actionable improvements',
```

```
'learning_opportunities': 'Share resources
```

```
and explain broader patterns',
```

```
'collaborative_approach': 'Ask questions
```

rather than just giving answers'

```
}
```

```
return
```

```
self.apply_feedback_framework(code_review, feedback_framework)
```

```
"""
```

```
},
```

```
    'cross_functional_collaboration': {  
        'stakeholder_management': [  
            'Understanding business priorities and constraints',  
            'Translating technical concepts to business
```

language',

```
'Managing expectations and communicating trade-
```

offs',

```
'Building trust through reliable delivery',  
'Proactive communication about risks and
```

dependencies'

```
],

'collaboration_patterns': """
# Working with Product Managers
def collaborate_with_product(product_manager:
```

ProductManager, feature_request: FeatureRequest):

```
# Understand business context
business_context = understand_business_impact(
    user_problem=feature_request.user_problem,
```

```
success_metrics=feature_request.success_criteria,
```

```
business_value=feature_request.estimated_value
)

# Propose technical alternatives
technical_options = generate_implementation_options(
    feature_request,
    constraints=['time', 'resources',
```

'technical_complexity']

```
)

# Facilitate decision-making
recommendation = facilitate_tradeoff_discussion(
    stakeholders=[product_manager,
```

engineering_team],

```
options=technical_options,
decision_criteria=['time_to_market',
```

'scalability', 'maintenance_cost']

```
)

    return recommendation

# Working with Data Scientists
def collaborate_with_data_science(data_scientist:
```

DataScientist, ml_project: MLProject):

```
# Understand ML requirements
ml_requirements = analyze_ml_needs(
    model_type=ml_project.model_type,
    data_requirements=ml_project.data_needs,
    inference_requirements=ml_project.serving_needs
)

# Design data infrastructure
infrastructure_design = design_ml_infrastructure(
```

```
training_pipeline=ml_requirements.training_needs,
```

```
feature_store=ml_requirements.feature_needs,
    model_serving=ml_requirements.inference_needs
)

# Establish collaboration workflow
workflow = establish_ml_workflow(
    data_scientist_responsibilities=
```

```
['model_development', 'evaluation'],
```

```
data_engineer_responsibilities=
```

```
['infrastructure', 'deployment', 'monitoring'],
```

```
shared_responsibilities=['feature_engineering',
```

```
'data_quality']
```

```
)

        return infrastructure_design, workflow
    """
},

'technical_strategy_and_vision': {
    'strategic_thinking_skills': [
        'Understanding industry trends and their business
```

impact',

```
'Anticipating technical debt and scalability
```

challenges',

```
'Balancing innovation with stability and
```

reliability',

```
'Building technical roadmaps aligned with business
```

goals',

```
'Evaluating build vs buy vs partner decisions'
],
```

The PYTHON code:

```
'vision_development_process': """
def develop_technical_vision(organization: Organization,
```

PYTHON

timeframe: str):

```
# Analyze current state
current_state = assess_current_technical_landscape(
    systems=organization.systems,
```

```
capabilities=organization.technical_capabilities,
```

```
challenges=organization.pain_points
)
```

```
# Research industry trends
industry_analysis = research_industry_trends(
    technologies=['emerging_tech',
```

```
'proven_solutions'],
```

```
competitors=organization.competitors,
market_forces=['regulatory', 'economic',
```

```
'social']
```

```
)

# Define future state
future_vision = design_target_architecture(
    business_strategy=organization.business_goals,
    technical_trends=industry_analysis,
    constraints=['budget', 'timeline',
```

```
'team_capability']
```



```
)

    # Create roadmap
    roadmap = create_transformation_roadmap(
        current_state=current_state,
        future_vision=future_vision,
        implementation_phases=timeframe
    )

    return future_vision, roadmap
"""
}

return leadership_skills_framework
```

Building Your Learning Curriculum

Self-Directed Learning for Rapid Technology Evolution

PYTHON

```
def create_learning_curriculum():
    learning_framework = {
        'foundational_knowledge': {
            'computer_science_fundamentals': [
                'Algorithms and data structures',
                'Distributed systems principles',
                'Database theory and implementation',
                'Computer networks and protocols',
                'Operating systems and concurrency'
            ],
            'mathematics_and_statistics': [
                'Probability and statistics',
                'Linear algebra (for ML applications)',
                'Information theory (for compression and encoding)',
                'Graph theory (for network analysis)',
                'Optimization theory (for query planning)'
            ],
            'domain_knowledge': [
                'Business intelligence and analytics',
                'Machine learning and MLOps',
                'Data governance and compliance',
                'Industry-specific knowledge (finance, healthcare,
etc.)',
'Software engineering practices'
```

The corresponding SQL query:

SQL

```
]
},

'learning_methods': {
  'reading_and_research': [
    'Research papers from top conferences (VLDB, SIGMOD,
```

ICDE)',

SQL

```
'Engineering blogs from major tech companies',
'Technical books on systems design and data
```

engineering',

```
'Industry reports and whitepapers',
  'Open source project documentation and code'
],

'hands_on_practice': [
  'Personal projects with real-world constraints',
  'Contributing to open source projects',
  'Building prototypes to test new technologies',
  'Participating in hackathons and coding
```

competitions',

SQL

```
'Reproducing research results and benchmarks'
],

'community_engagement': [
    'Attending conferences and workshops',
    'Participating in online communities and forums',
    'Joining professional organizations',
    'Speaking at meetups and conferences',
    'Mentoring others and teaching workshops'
]
},

'staying_current_strategy': {
    'information_sources': [
        'High Scalability blog and case studies',
        'Company engineering blogs (Netflix, Uber, Airbnb,
```

```
etc.)',
```

```
'Academic conferences and proceedings',
'Technology vendor documentation and whitepapers',
'Podcasts and YouTube channels focused on data
```

engineering'

PYTHON

```
],

'evaluation_criteria': """
def evaluate_learning_opportunity(opportunity:
```

LearningOpportunity):

```
    evaluation_criteria = {
        'relevance_to_current_role':
```

```
    rate_relevance(opportunity.content, current_responsibilities),
```

```
    'career_advancement_potential':
```

```
    assess_career_impact(opportunity.skills, career_goals),
```

```
    'technology_adoption_timeline':
```

```
    estimate_adoption_curve(opportunity.technology),
```

```
    'learning_investment_required':
```

```
    estimate_time_investment(opportunity.complexity),
```

```
    'practical_application_opportunities':
```

```
    identify_application_contexts(opportunity.skills)
```

```
}
```

```
# Weight factors based on current career stage
if career_stage == 'junior':
    weights = {'relevance': 0.4, 'career': 0.3,
```

```
    'adoption': 0.1, 'investment': 0.2}
```

```
elif career_stage == 'senior':
    weights = {'relevance': 0.3, 'career': 0.2,
```

PYTHON

```
    'adoption': 0.3, 'investment': 0.2}
```

```
else: # staff+
    weights = {'relevance': 0.2, 'career': 0.2,
```

```
    'adoption': 0.4, 'investment': 0.2}
```

```
    return calculate_weighted_score(evaluation_criteria,
```

weights)

```
""
},

'skill_development_plan': {
  'quarterly_focus_areas': [
    'Choose 1-2 major technologies/concepts to deep
```

dive',

```
'Select 2-3 adjacent skills to explore',
'Identify 1 business/domain knowledge area to
```

SQL

develop',

```
'Plan 1 teaching/sharing opportunity to reinforce
```

learning'

```
],

'learning_portfolio_approach': ""
# Balanced learning portfolio (inspired by investment
```

portfolio theory)

```
learning_portfolio = {
  'core_skills_maintenance': {
    'allocation': '40%',
    'focus': 'Deepening existing expertise, staying
```

current with tools you use',

```
'examples': ['Advanced SQL optimization', 'Cloud
```

platform certifications', 'Programming language mastery']

```
},  
  
    'adjacent_skills_development': {  
        'allocation': '35%',  
        'focus': 'Skills that complement your core
```

expertise',

```
    'examples': ['ML engineering for data
```

engineers', 'DevOps for data platforms', 'Data visualization']

```
},  
  
    'emerging_technology_exploration': {  
        'allocation': '20%',  
        'focus': 'Technologies that could become
```

important in 2-5 years',

```
    'examples': ['Quantum computing', 'Edge
```

computing', 'New database technologies']

```
},  
  
    'wildcard_learning': {  
        'allocation': '5%',  
        'focus': 'Completely different domains that
```

could provide unique perspectives',

```
    'examples': ['Product management', 'Design
```

thinking', 'Psychology/behavioral economics']

```

    }
        }
        """
    }
}

return learning_framework

# Sample learning plan template
def create_quarterly_learning_plan(current_role: str, career_goals:

```

str, quarter: str):

```

learning_plan = {
    'quarter': quarter,
    'focus_theme': 'Real-time data systems and MLOps', #

```

Example theme

```

'core_skill_deepening': {
    'skill': 'Apache Flink stream processing',
    'learning_objectives': [
        'Master stateful stream processing patterns',
        'Understand exactly-once processing guarantees',
        'Optimize for high-throughput use cases'
    ],
    'learning_activities': [

```

```

'Complete Flink certification course',
'Build personal project with complex event

```

processing',


```

    'Contribute to Flink open source project'
  ],
  'success_metrics': [
    'Deploy production Flink job at work',
    'Present Flink learnings to team',
    'Achieve Flink certification'
  ]
},

'adjacent_skill_development': {
  'skill': 'MLOps and model deployment',
  'learning_objectives': [
    'Understand ML model lifecycle management',
    'Learn feature store design patterns',
    'Master model monitoring and drift detection'
  ],
  'learning_activities': [
    'Take online MLOps course',
    'Build end-to-end ML pipeline project',
    'Shadow data science team deployments'
  ],
  'success_metrics': [
    'Design feature store for team',
    'Implement model monitoring dashboard',
    'Collaborate effectively with data scientists'
  ]
},

'business_knowledge': {
  'area': 'Financial services domain knowledge',
  'learning_objectives': [
    'Understand regulatory requirements (SOX, Basel

```

III)',

```

'Learn financial data types and calculations',
    'Understand risk management use cases'
],
'learning_activities': [
    'Read industry publications and reports',
    'Attend fintech conferences and webinars',
    'Partner with business stakeholders on projects'
]
},

'teaching_and_sharing': {
    'opportunity': 'Internal tech talk on stream

```

processing',

```

'preparation': [
    'Create presentation with real-world examples',
    'Develop hands-on workshop materials',
    'Practice with feedback from colleagues'
],
'impact_goals': [
    'Increase team knowledge of streaming options',
    'Generate interest in stream processing adoption',
    'Establish myself as streaming expert'
]
}
}

return learning_plan

```

SQL

Industry Networking and Community Building

```

def develop_professional_network():
    networking_strategy = {
        'internal_networking': {
            'cross_team_collaboration': [
                'Volunteer for cross-functional projects',
                'Attend engineering all-hands and planning

```

PYTHON

meetings',

```
'Participate in technical working groups',
    'Mentor engineers from other teams',
    'Share knowledge through internal tech talks'
],

'relationship_building': [
    'Schedule regular coffee chats with colleagues',
    'Participate in company social events',
    'Join employee resource groups',
    'Contribute to internal engineering communities',
    'Help with hiring and candidate interviews'
]
},

'external_networking': {
    'professional_communities': [
        'Join data engineering meetup groups',
        'Participate in online communities (Reddit,
```

Discord)',

```

    'Attend industry conferences and workshops',
        'Contribute to open source projects',
        'Engage with thought leaders on social media'
    ],

    'content_creation': [
        'Write technical blog posts about your work',
        'Create educational videos or tutorials',
        'Speak at conferences and meetups',
        'Participate in podcasts and interviews',
        'Share insights on LinkedIn and Twitter'
    ],

    'strategic_relationships': [
        'Connect with engineers at peer companies',
        'Build relationships with technology vendors',
        'Network with recruiters and hiring managers',
        'Engage with startup founders and CTOs',
        'Connect with academics and researchers'
    ]
},

'giving_back_to_community': {
    'mentoring': [
        'Mentor junior engineers and career changers',
        'Participate in coding bootcamp mentorship',
        'Offer guidance through professional organizations',
        'Provide career advice through informal networks'
    ],

    'knowledge_sharing': [
        'Contribute to open source documentation',
        'Answer questions on Stack Overflow and forums',
        'Create educational content for the community',
        'Organize local meetups and events',
        'Review conference papers and proposals'
    ]
}

return networking_strategy

```

Pro Tip Career progression in data engineering isn't just

about getting better at the technical work — it's about expanding your impact beyond your individual contributions.

Start building leadership skills early: mentor others, write documentation, present your work, and collaborate across teams. These "soft" skills become increasingly important as you advance and are often what distinguish senior engineers from their peers.

□ Checkpoint

1. How does the role of a data engineer change as you progress from junior to senior to staff levels?
2. What's the difference between technical expertise and technical leadership?
3. How do you create a learning strategy that keeps pace with rapid technological change?

Module 12 Final Project: Career Development Plan

Objective

Create a comprehensive 18-month career development plan that demonstrates your understanding of the data engineering career landscape, identifies your target role and growth areas, and provides a concrete roadmap for achieving your career goals.

Project Overview

This capstone project brings together everything you've learned about system design, technical skills, and career progression to create a personalized development strategy. You'll analyze your current position, define your target role, identify skill gaps, and create an actionable plan for advancement.

Requirements

- Personal Assessment and Goal Setting (2 pages)
- Current technical skills inventory and honest self-assessment
- Career goals and target role definition (with timeline)
- Analysis of market demand and opportunities in your target area

- Personal strengths, development areas, and career motivations
- Technical Skill Development Plan (3 pages)
 - Gap analysis between current and target role requirements
 - Prioritized learning objectives with specific milestones
 - Mix of deepening existing skills and acquiring new capabilities
 - Integration of emerging technologies and industry trends
- Portfolio and Project Strategy (2 pages)
 - Plan for 2-3 substantial portfolio projects over 18 months
 - Projects should demonstrate progression toward target role
 - Include both technical depth and business impact considerations
 - Timeline and resource requirements for each project
- Professional Development and Networking (1 page)
 - Strategy for building industry relationships and visibility
 - Speaking, writing, or community contribution goals
 - Mentorship plans (both receiving and providing mentorship)
 - Professional milestone targets (certifications, conference talks, etc.)
- Implementation and Tracking Plan (1 page)
 - Quarterly milestones and success metrics
 - Accountability mechanisms and progress tracking
 - Risk mitigation for potential obstacles
 - Plan for adapting strategy based on changing circumstances

Deliverable Template

```markdown # 18-Month Data Engineering Career Development Plan

## Executive Summary

---

[2-3 sentences describing your career trajectory and key goals]

## Current State Analysis

---

### Technical Skills Inventory

---

Strong Areas:

- [Skill 1]: [Specific examples of competency]
- [Skill 2]: [Specific examples of competency]

Developing Areas:

- [Skill 1]: [Current level and specific gaps]
- [Skill 2]: [Current level and specific gaps]

Gap Areas:

- [Skill 1]: [Why important for career goals]
- [Skill 2]: [Why important for career goals]

## Current Role Assessment

---

Position: [Current title and responsibilities] Company: [Company size, industry, tech maturity] Team: [Team size, scope, technical challenges] Satisfaction: [What's working well, what's not]

Target State Definition

18-Month Career Goal

Target Role: [Specific title and level] Target Environment: [Company size/type, industry, team structure] Key Responsibilities: [What you'll be doing day-to-day] Impact Level: [Individual contributor vs. technical leadership scope]

## Market Analysis

---

Demand for Target Role: [Research on job market, salary ranges] Key Requirements: [Common requirements from job postings] Competitive Landscape: [What makes candidates successful] Growth Opportunities: [Career paths from target role]

## Technical Development Plan

---

Quarter 1 Focus: [Theme]

- Primary Learning Objective: [Specific skill/technology]
- Learning activities: [Courses, projects, practice]
- Success metrics: [How you'll measure progress]
- Application opportunities: [How to use in current role]
- Secondary Objectives: [2-3 supporting skills]
- [Skill 1]: [Brief learning plan]
- [Skill 2]: [Brief learning plan]

Quarter 2-6 Focus Areas

[Repeat structure for each quarter, showing progression]

## Emerging Technology Integration

---

Technology to monitor: [List 3-5 technologies] Evaluation criteria: [How you'll decide what to invest time in] Experimentation plan: [How you'll gain hands-on experience]

## Portfolio Project Strategy

---

Project 1: [Project Name]

Timeline: [Start and end dates] Objective: [What skill/capability this demonstrates]

Business Context: [Problem being solved]